INDUSTRY STARTER PACKS · THE ARCHITECTURE WORKBOOK

Retail

Running the ADM in a retailer — a step-by-step workbook for the repository you have been handed

The companion volume. Not something to read — something to work through. Every phase of the ADM as numbered steps: what you produce, which sheet of the repository it goes in, which columns, who signs it, how you know it is done, and the way it usually goes wrong. Written against a real populated metamodel and a real comparison tool, not against the standard in the abstract.

Use it with the learner guide. South African lens. October 2026.

First edition · October 2026

Thotnr Consulting

Contents

BEFORE YOU START

Chapter 1 · How to use this workbook 3

PRELIMINARY PHASE

Chapter 2 · Standing the practice up 23

PHASE A

Chapter 3 · Architecture Vision 48

PHASE B

Chapter 4 · Business Architecture 67

PHASE C

Chapter 5 · Data Architecture 96

Chapter 6 · Application Architecture 121

PHASE D

Chapter 7 · Technology Architecture 147

PHASE E

Chapter 8 · Opportunities and Solutions 171

PHASE F

Chapter 9 · Migration Planning 201

PHASE G

Chapter 10 · Implementation Governance 225

PHASE H

Chapter 11 · Architecture Change Management 249

CONTINUOUS

Chapter 12 · Requirements Management 271

APPENDICES

Appendix A · The sheet-by-sheet reference 292

Appendix B · The report catalogue 317

Appendix C · Workshop packs, checklists and the ninety-day calendar 328

Appendix D · The repository in ArchiMate 4 348

Appendix E · Where to look in volume 1 368

Appendix F · The forms 384

Appendix G · The principles and standards starter set 440

Appendix H · Running the maturity assessment 470

CHAPTER 1

How to use this workbook

One repository, one invented retailer, and the change control it does not have — which you add before you load a single record.

One repository, one invented retailer, and a hole in its governance you have to close before you type anything into it. This chapter is the orientation: what you were handed, how the pieces fit, and the handful of conventions that make the other nineteen chapters usable. It contains no steps. Everything from chapter 2 onward does.

> **HALVERSON STORES GROUP IS INVENTED**
>
> Halverson Stores Group does not exist. The repository says so on its own front page: *"Halverson Stores Group is fictional. The figures are internally consistent and describe no real organisation"* [1]. Milkwood Retail Group, which appears throughout volume 1, is a separate invention. Do not mix their data, and do not import volume 1's system names — Harvest, Loom, Scrip, Tally, Lantern, Tide, Gantry, Trellis — into this repository. Halverson's applications are called `A01 Merchandising System` through `A12 Retail Analytics`.

What you have, and what each thing is for

Thing	What it is	What it is not
`Retail_TOGAF_Metamodel.xlsx`	The repository. 105 sheets: one per object type, one per relationship type, thirteen reports, an industry extension and an integrity check	Not a document. Nothing in it is prose to read front to back
`Retail_Architecture_Comparison_Tool_TEMPLATE.xlsx`	A diff instrument: five paste targets, a `Classification` column separating a design delta from governed drift from ungoverned change, a human review sheet, and a gap candidates sheet shaped to paste into `029 Gap`	Not populated, and not a gap catalogue. It is a template. It produces candidates; a person classifies them
The two volumes	*Retail — a starter pack for enterprise architects* is the content. This volume is the method	Neither is a reference manual. Volume 1 is read once; this one is opened at a step

Say the second one's name in full when you ask for it. It is a **template**: the insurance edition of this set shipped a comparison workbook already loaded with a worked example, and the retail edition does not. Chapter 8 works Phase E against the template's design, and says plainly that both extracts have to be produced by hand, because **this repository has no export sheets at all**.

The repository is the centre of gravity, so learn its shape first.

- `00 Index` — the catalogue, one row per object type with its record count and its domain, under a banner reading `65 object types · 25 relationship types · 842 records · 429 relationship instances`.
- **64 object sheets**, `001 Driver` through `066 Architecture Increment`, with `034` and `051` absent. The file counts *sixty-five* object types because `EXT Industry Extension` is the sixty-fifth.
- **25 relationship sheets**, `R01` to `R25`.
- **13 report sheets**, `RPT01 Dashboard` to `RPT13 Orphan Analysis`.
- `EXT Industry Extension` — six object types retail needs beyond the TOGAF core. Chapter 2 gives it three steps.
- `ZZ Checks` — fifteen lines of the repository grading itself against a threshold column. Read it before and after every phase.

The repository's own numbers, and where they do not reconcile

`RPT01 Dashboard` is the one-page summary: 65 object types, 842 records, 25 relationship types, 429 relationship instances, 13 reports, 12 applications, 29 interfaces, 16 of them unmanaged, 20 with no logging, 9 direct database reads, 3 platform components past vendor support, 4 security controls absent, 3 control objectives with no evidence, 2 data entities with no lineage, 10 level-1 capabilities [2]. `ZZ Checks` repeats most of those with a `Threshold` column beside each, and nearly every threshold is zero, because what a healthy estate shows is nothing [3].

Most of those numbers survive a count. Three do not, and one of them is interesting rather than wrong.

THREE NUMBERS TO CHECK BEFORE YOU QUOTE ANY OF THEM

842 against 813. The per-sheet counts in `00 Index` total 813 — 807 across the sixty-four `O` sheets plus the six rows of `EXT`. The banner, `RPT01 Dashboard` and `ZZ Checks` all say 842. The missing twenty-nine are the `INT-001` to `INT-029` rows of `R06 Application interfaces with`, which carry their own `Ref` and are records in all but filing. The total is defensible on a definition the file never states, and which concedes that an interface is an object with no object sheet. Decide which number you will quote and write the decision down. The relationship total, 429, reconciles exactly.

"Every type carries records" against an empty sheet and a one-row sheet. `ZZ Checks` line one reads `Object types populated` = 65 with the threshold `Every type carries records`. `00 Index` puts `O59 Compliance Assessment` at 0 and `O29 Gap` at 1, and `R24 Compliance Assessment tests` is empty too.

Platform components past support = 3, and only two dates agree. `O27 Physical Technology Compone` has a typed `Position` column and a generated `Support end` column. `PTC-06 Warehouse control systems` at 2026-02 and `PTC-12 Data platform cluster` at 2026-08 are genuinely behind us; `PTC-01 Store server estate` reads `Past support` with a support end of **2027-03**, which is in the future. `RPT10 Technology Currency` repeats the figure and adds "3 components are running past vendor support today" [7]. One of those columns is wrong and the report believes the wrong one.

Two further numbers are green for a bad reason: `Vacant architecture roles` = 0 because every row of `R21 Role performed by Actor` reads `Filled`, and `Records validated against a real organisation` = 0 is the repository declaring its own data illustrative. The habit to take from this is that where a typed total and a counted total disagree, the count wins. Chapter 2, step 2.4 makes it a step.

The relationship sheets, and how they are named

The insurance repository named its relationship sheets after their endpoints — `R08 ACT-ROL`. This one names them after what they mean: `R01 Capability decomposition`, `R12 Gap closed by Work Package`, `R21 Role performed by Actor`. Easier to read, harder to script against. Six relationship tab names run past Excel's thirty-one-character limit and are cut off mid-word, as do two object sheets, `O26 Logical Technology Componen` and `O27 Physical Technology Compone`. **Spell them the way the tab spells them**, truncation included. This volume does, everywhere, and will not say so again.

Most relationship sheets have no `Ref` of their own: source reference, source name, target reference, target name, then a property or two. Only `R06 Application interfaces with` and `R19 Data Entity relates to Data` carry a `Ref`. `00 Index` claims that because every row names both ends by reference *and* by name, a broken link is visible without a lookup [1]. The name columns hold typed text, not formulas, and the claim is already false on one sheet. In `R05 Application manages Data En` the fourteen `Master` rows put an application **name** in the `Application` column, `POS`, `Retail Analytics`, `Workforce Management`, where the fourteen `Consumer` rows correctly put `A02`, `A12`, `A10`. The sheet that decides which system masters which entity is the one whose left-hand column is half reference and half prose. Treat the name column as a courtesy, not a control.

The ADM as this workbook runs it

phase	the sheets it fills	what proves it
Preliminary	O05 O09-O12 O57 O58 O01-O04 O31 EXT	ZZ RPT13
A Vision	O06-O08 O09-O11 O14 O31	RPT01 06
B Business	O13 O15-O19 O35 R01-R03 R22 R25	RPT03
C Data	O20-O22 O36-O44 R05 R19	RPT05
C Appl'n	O23 O24 O45 R04 R06 R20	RPT02 04
D Technology	O25-O28 O32 O46-O50 R07 R18	RPT10 11 12
E Opportunities	O29 O52 O54 O55 R09 R12 R13	RPT07
F Migration	O30 O53 O56 O66 R14 R15 R23	RPT08
G Governance	O33 O59-O65 R16 R17 R24	RPT09
H Change	nothing in the file — build it in ch 2	ZZ + yours
Requirements	O06 R08 R09	RPT06

Figure 1.1 The ADM as this volume runs it: each phase, the sheets it fills and the report that shows it landed. Phase H has no sheets in this repository — you build them in chapter 2.

Twenty chapters: one per ADM phase, Requirements Management as a running twelfth, then eight appendices. The phase order is the standard one and this volume does not argue with it [4]. What it adds is the mapping — which sheets a phase fills and which report tells you it landed.

Ch	Phase	Primary sheets	What shows it landed
2	Preliminary	`O05 Principle`, `O09` – `O12`, `O57 Governance Body`, `O58 Governance Gate`, `O01` – `O04`, `O31 Deliverable`, `EXT Industry Extension`	`ZZ Checks`, `RPT13 Orphan Analysis`
3	A — Vision	`O06 Requirement`, `O07 Constraint`, `O08 Assumption`, `O09` – `O11`, `O14 Value Stream`, `O31 Deliverable`	`RPT01 Dashboard`, `RPT06 Requirements Traceability`
4	B — Business	`O13 Business Capability`, `O15` – `O19`, `O35`, `R01` – `R03`, `R22`, `R25`	`RPT03 Capability Heatmap`
5	C — Data	`O20 Data Entity`, `O21`, `O22`, `O36` – `O44`, `R05`, `R19`	`RPT05 Data Ownership`
6	C — Application	`O23 Application Component`, `O24`, `O45`, `R04`, `R06`, `R20`	`RPT02 Application Portfolio`, `RPT04 Interface Catalogue`
7	D — Technology	`O25` – `O28`, `O32 Standard`, `O46` – `O50`, `R07`, `R18`	`RPT10 Technology Currency`, `RPT11 Security Position`, `RPT12 Standards Conformance`
8	E — Opportunities	`O29 Gap`, `O52`, `O54 Architecture Alternative`, `O55 Building Block`, `R09`, `R12`, `R13`	`RPT07 Gap Analysis`
9	F — Migration	`O30 Work Package`, `O53 Transition Architecture`, `O56 Project`, `O66`, `R14`, `R15`, `R23`	`RPT08 Roadmap`
10	G — Governance	`O33 Architecture Contract`, `O59` – `O65`, `R16`, `R17`, `R24`	`RPT09 Regulatory Coverage`
11	H — Change	Nothing. The sheets this phase needs are not in the file	`ZZ Checks`, and whatever you added in chapter 2
12	Requirements	`O06 Requirement`, `R08`, `R09`	`RPT06 Requirements Traceability`

Two things in that table deserve a second look.

Row eleven is empty on purpose. The four sheets a change-management phase runs on — version control, a change log, change requests against records, and baselines — are not in this repository. Chapter 2 tells you to build them in week one and chapter 11 runs the phase against what you built. The next major section is about nothing else.

The reports are smaller than the sheets they read. `RPT03 Capability Heatmap` holds ten rows against `O13`'s hundred, because it reports level 1 only; `RPT07 Gap Analysis` holds one row because `O29 Gap` holds one. A report here is a view, not a calculation, and it inherits every silence beneath it. On numbering, unlearn the insurance set: `RPT07` **is** `Gap Analysis` and `RPT09` is `Regulatory Coverage`, no two sheets share a name, and there are thirteen reports rather than thirty-two. Appendix B catalogues what each answers.

How a step is laid out

From chapter 2 onward this volume is numbered steps — around a hundred and fifty of them. Every one has the same eight parts in the same order. Learn the shape once and you can read any step in forty seconds.

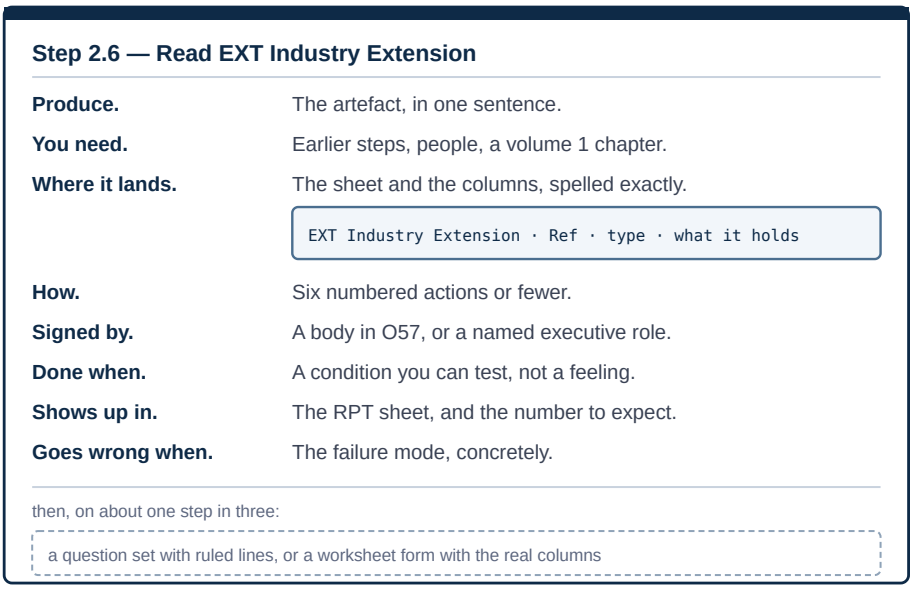


Figure 1.2 The step template. Eight parts, always in this order. You will meet it about a hundred and fifty times.

- **Produce.** The artefact, in one sentence. If you cannot say what the step produces, the step is a meeting.
- **You need.** Earlier steps, the people to book, the volume 1 chapter behind it.
- **Where it lands.** The sheet, named exactly, and the columns to fill. This is the part the volume exists for.
- **How.** Six numbered actions or fewer. If something needs more than six, it is two steps.
- **Signed by.** A body from `O57 Governance Body` or a named executive role. Architecture nobody signed is a draft forever.
- **Done when.** A condition you can test. "The stakeholders are aligned" is not one. "Every `Critical` capability in `O13` has an owning unit in `R22`" is.
- **Shows up in.** The report sheet where the work becomes visible, and the number to expect if the step went well.
- **Goes wrong when.** The real failure mode, concretely enough to recognise in your own repository.

About one step in three carries a question set with ruled lines under it, for the judgements only you can make, and some carry a worksheet form whose columns are the real columns of the real sheet. The catalogue forms live in appendix F. Write in the book: a judgement you did not write down is one you will make differently next month.

The change-control gap, and what to add before you load anything

This is the finding. Every other defect in this volume is a record you can correct in an afternoon. This one is structural.

There is no record-keeping anywhere in the file — no `Version`, `Record Status`, `In Repository Since`, `Last Changed`, `Changed By`, `Change Ref` or `Approved By` column on any of the sixty-four object sheets or any of the twenty-five relationship sheets, and no `01 Version Control`, `02 Change Log`, `03 Change Requests` or `04 Baselines` sheet to carry what those columns would point at [1] [5].

It is easy to mistake one sheet for the mechanism. `O61 Governance Change Request` is not it: five rows with `Ref`, `Change request`, `Against`, `Type`, `Status`, `Expires` — no raised date, no decided date, no previous value, no new value, no approver. Two of them, `CR-G03 Raise unfunded building blocks as gaps` and `CR-G04 Record benefit realisation dates per package`, are typed `Correction` and aimed at `Against` = `Repository` [6]. Somebody noticed the repository needed correcting, wrote it down, and had nowhere to apply it. That is the gap, recorded inside the gap.

What to add, before the first record

Four sheets and seven columns. Add them in this order; it takes a morning.

Add this sheet	Columns	What it is for
`01 Version Control`	`Rule`, `Statement`, `Owner` on top; `Workbook Version`, `State As At`, `Requests Raised`, `Approved`, `Open`, `Changes Logged`, `Baselines Taken` below	The rules, stated once, and the repository's current state in eight cells
`02 Change Log`	`Ref`, `Date`, `Sheet`, `Record`, `Column`, `Previous Value`, `New Value`, `Changed By`, `Change Ref`	The evidence base. Never truncated, never sorted destructively
`03 Change Requests`	`Ref`, `Raised`, `Raised By`, `Type`, `Scope`, `Description`, `Status`, `Decided`, `Approved By`, `Decision`	Nothing changes without a row here first
`04 Baselines`	`Ref`, `Name`, `Date`, `Workbook Version`, `Approved By`, `Status`, `Note`, `Changes Since`	A named, dated, approved fixed point to compare against

Then seven columns at the right-hand end of every object and relationship sheet you intend to maintain. Not all sixty-four — only the ones your year-one tailoring decision in step 2.5 puts in scope.

Column	What it tells you	What a bad value looks like
`Version`	`1.0` is the load; the minor number increments once per approved change	Above `1.0` with an empty `Change Ref`
`Record Status`	`Baseline`, `Approved`, or `Draft — change pending`	Still pending against a request decided months ago
`In Repository Since`	When the record first appeared	A date before the baseline that does not contain it
`Last Changed`	The most recent approved change	Equal to `In Repository Since` on a record that has moved
`Changed By`	`Repository load`, otherwise the **role** that changed it	A person's name. Roles outlive people
`Change Ref`	The join back to `03 Change Requests`	Empty on anything that is not `1.0`
`Approved By`	`Architecture Board` where a principle is engaged, `Chief Enterprise Architect` otherwise	A body not in `057 Governance Body`

Two naming cautions. **Do not prefix your change requests `CR-`:** `061 Governance Change Request` already uses `CR-G01` to `CR-G05`, and a bare `CR-012` in a `Change Ref` column will be read as one of those by the next architect and by any script. Use `RCR-`. And **do not derive `Version` by hand** — it follows from the count of approved rows in `02 Change Log`, which is a formula, not a judgement.

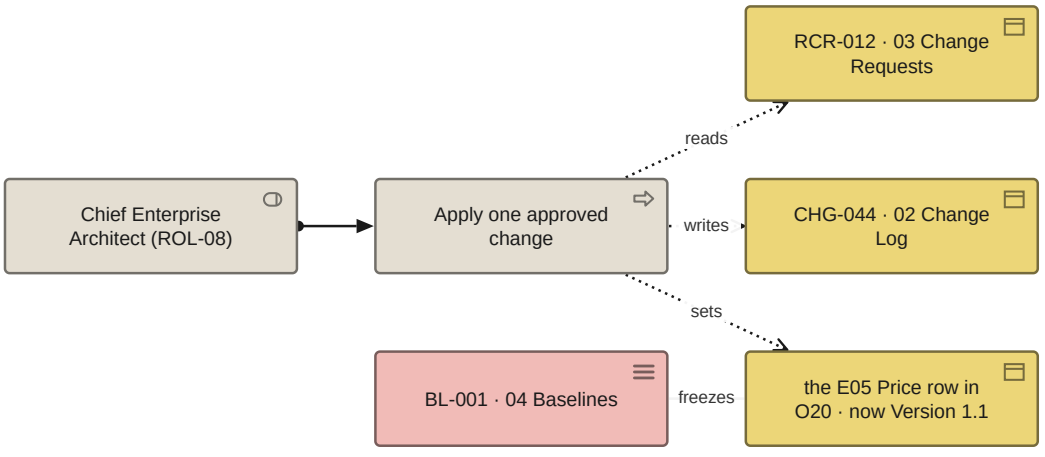


Figure 1.3 The change-control spine you must add, in ArchiMate 4. One approved change: the architect's process reads the request, writes the log entry and sets the record's version — which is why the version is derived and never typed. The baseline freezes the record at a named version to compare against. None of these four sheets exists in the file as shipped.

What goes wrong if you skip this

Not a style point. Five specific failures, in the order you will meet them.

1. **You cannot answer the only question an executive asks about a repository.** "What has changed since the board saw this?" With no log, the honest answer is that you do not know, and the answer you will give instead is a guess.

2. **Gap analysis loses its fixed point.** The comparison tool wants a baseline extract in one paste target and a later extract in another, each labelled and dated. With no `04 Baselines` there is nothing to name, so every difference reads as a design delta and nothing can be classified as governed drift. The tool's whole value is in that distinction.
3. **The `ZZ Checks` numbers stop being a trend.** Sixteen unmanaged interfaces is not a finding. Sixteen against twenty-four last quarter is a finding; sixteen against nine is an escalation. Without a dated snapshot you have a photograph where you needed a film.
4. **The repository stops being evidence, and in retail that bites at shelf level.** Section 23(6)(b) of the Consumer Protection Act is unforgiving: where two prices are displayed for the same goods, the lower binds the retailer [8]. A shelf label that silently failed to update is a legal exposure, and the question that follows is which system is the source of the price, when it changed and who approved the change. If the repository is the thing that names the master system for `E05 Price`, a repository with no change history cannot support the answer. Section 56 does the same for a recall — six months from delivery, with the *consumer* electing repair, replacement or refund [9] — and `CO-05 Product safety and recall` already reads `In breach`.
5. **The `Owner` columns rot silently.** `O20 Data Entity` names seven distinct owners and only `Data Steward` appears in `O11 Role` [10]. Nobody can tell whether those titles were wrong when typed or stopped being right afterwards, because nothing dates them.

> **THE WEEK-ONE JOB**
>
> Add the four sheets and the seven columns, load `04 Baselines` with one row describing the repository exactly as you received it, and only then open a sheet to edit it. Chapter 2 makes this steps 2.2 and 2.3 and puts them before everything except fixing the enterprise boundary. If you take one instruction from this chapter, take that one.

The `Ref` conventions already in use

Every object sheet puts the reference in column A and the name in column B, under a three-row preamble: a title, a line reading `Halverson Stores Group · <domain> domain · <n> records`, a blank. **The header is row 4.** Any script you write reads from row 5. The relationship sheets use the same preamble with `· <n> instances` instead of records.

The prefixes are set. Do not invent new ones for sheets that have them.

Prefix	Sheet	n	Prefix	Sheet	n
DRV-	001 Driver	12	GAP-	029 Gap	1
GOL-	002 Goal	6	WP-	030 Work Package	12
OBJ-	003 Objective	14	DLV-	031 Deliverable	20
MSR-	004 Measure	16	STD-	032 Standard	16
PRN-	005 Principle	24	ACO-	033 Architecture Contract	8
REQ-	006 Requirement	31	BBB-	035 Business Building Block	10
CON-	007 Constraint	8	REL- ATR- DST- MIG- MAP-	036 – 040	95
ASM-	008 Assumption	6	SEC- MTD- MDR- DBB-	041 – 044	42
ORG-	009 Organization Unit	12	ABB-	045 Application Building Block	10
ACT-	010 Actor	18	TBB-	046 Technology Building Block	7
ROL-	011 Role	18	RME-	047 Reference Model Entry	11
LOC-	012 Location	8	ENV- RES- NWL-	048 – 050	26
C01.01.01	013 Business Capability	100	IF-	052 Implementation Factor	8
VS01	014 Value Stream	5	TA-1 ALT-A	053 , 054	8
BSV- BPR- BFN-	015 – 017	40	BBB- DBB- ABB- TBB-	055 Building Block	37
PRD- CTR-	018 , 019	15	PRJ-	056 Project	12
E01	020 Data Entity	14	GB- GT-	057 , 058	9
LDC- PDC-	021 , 022	24	DEC- CR-G RSK-	060 – 062	20
A01	023 Application Component	12	CO- POL- EP- INC-	063 – 066	21
ISS-	024 Information System Service	12	X01	EXT Industry Extension	6
TSV- LTC- PTC- PSV-	025 – 028	39	INT-	R06 , on a relationship sheet	29

—	`O59 Compliance Assessment`	0

Eight irregularities in that list. None is a mistake to tidy; all are load-bearing.

Six reference shapes in one file. Most sheets use `XXX-nn`. Five use a bare letter and digits — `C01`, `E01`, `A01`, `VS01`, `X01`. Three use a single-digit suffix, `GT-0`, `TA-1`, `INC-1`. One uses a letter suffix, `ALT-A` to `ALT-E`. Three use three digits, `REQ-001`, `ATR-001`, `INT-001`. And `O13 Business Capability` uses no prefix at all: dotted references, `C01`, `C01.01`, `C01.01.01`, the dots carrying the level, which is why `R01 Capability decomposition` holds ninety rows with a `Level` column and no separate hierarchy table. Sort a mixed column as text and `A1` will not land where you expect.

`O55 Building Block` has no reference space of its own. Its thirty-seven rows reuse `BBB-` from `O35`, `DBB-` from `O44`, `ABB-` from `O45` and `TBB-` from `O46`. A bare `ABB-03` is therefore ambiguous between `O45` and `O55`, and `R09 Requirement satisfied by Bu` and `R13 Work Package delivers Build` both point into the ambiguity. Always name the sheet.

`O46 Technology Building Block` has two columns called `Ref`. Column A holds `TBB-01` to `TBB-07`; column C, also headed `Ref`, holds `T01` to `T07`, and **nothing else in the repository uses `T01`**. The seven things it means are `LTC-01` to `LTC-07` in `O26 Logical Technology Componen`, same names in the same order — a dangling reference to the sheet it should have pointed at. The duplicate header also breaks any reader that keys on column names.

`GT-` starts at zero, because the gate set starts at `G0 Charter`. A script assuming references begin at `01` loses the gate that tests whether the work should exist.

The only interface reference sits on a relationship sheet. `INT-001` to `INT-029` live in `R06 Application interfaces with` and there is no `O` sheet for an interface — which is why the record total needs explaining, and why an interface here has no owner, no disposition and no lifecycle.

`O59 Compliance Assessment` has no prefix because it has no rows. You choose; `CA-` is conventional, and since every join here is a text match `R24` will accept whatever you type. Choose once and write it into `01 Version Control`. And **`CR-G` is taken** — that is the collision you will walk into.

And never renumber. References are matched as text, from the relationship sheets and from the inline reference columns on about half the object sheets. Renumber `E06 Promotion` and you break `R05`, `R19` and the `Entity` columns of `O21`, `O37`, `O39`, `O40`, `O43` and `O44` — and because the adjacent name column is typed rather than computed, nothing turns red to tell you.

The two volumes, and how to use them together

The division of labour is strict. **Volume 1 is the content**: the formats and their economics, the vocabulary, the business model, who does what, the money, the operating model, fourteen BPMN level-2 processes, the data and information models, twenty-five conceptual applications, the real estate of systems, the reference architecture, the vendors, the regulation, and how to draw all of it in ArchiMate 4. Read it front to back, once. It answers *what is true about retail*. **Volume 2 is the method**: which sheet, which columns, in what order, signed by whom, and the report that proves it. It answers *what do I do on Tuesday*, and it never re-teaches retail.

Each chapter from 2 onward opens with a *Before you start this phase* box, and steps resting on a judgement volume 1 has already worked carry a **Look it up.** line naming the chapter and the section inside it.

One warning, repeated wherever it bites. Volume 1's catalogues are numbered for volume 1 — `P-01` … `P-12`, `C-01` … `C-20`, `R-01` … `R-33`, `D-01` … `D-20`, `A-01` … `A-25`, all in its appendix A. This repository is numbered for itself: ten `C01`-style level-1 capabilities, twenty `BPR-` processes, eighteen `ROL-` roles, fourteen `E01`-style entities, twelve `A01`-style applications. Same industry, different grain, built independently. **Map by name, never by number.** `A-11` is not `A11`. `D-05` is not `E05`. Twenty-five conceptual applications against twelve `A` rows is not a coincidence waiting to be aligned; it is two models at different resolutions.

Workbook chapter	Phase	Volume 1 chapters to have read
1 How to use this workbook	Before you start	1, 16
2 Standing the practice up	Preliminary	2, 5, 7, 18
3 Architecture Vision	A	4, 5, 7, 17
4 Business Architecture	B	2, 5, 7, 8, 19
5 Data Architecture	C	9, 10, 15, 19
6 Application Architecture	C	11, 12, 13, 19
7 Technology Architecture	D	12, 13, 14, 15
8 Opportunities and Solutions	E	6, 11, 13, 14
9 Migration Planning	F	12, 14, 17, 18
10 Implementation Governance	G	13, 15, 16
11 Architecture Change Management	H	7, 14, 15
12 Requirements Management	Continuous	10, 15, 16
13 The sheet-by-sheet reference	Reference	16, 20
14 The report catalogue	Reference	6, 18
15 Workshop packs and the calendar	Reference	5, 18
16 The repository in ArchiMate 4	Reference	16, 17
17 Where to look in volume 1	Reference	19, 20
18 The forms	Reference	—
19 Principles and standards starter set	Reference	13, 21
20 Running the maturity assessment	Reference	7, 18

That column is the minimum, not a reading list.

The notation: ArchiMate 4, and the one exception

Every architecture diagram in this volume is ArchiMate 4. Volume 1, chapter 16 (Modelling retail in ArchiMate) teaches the notation under *Five decisions that settle most of a retail model*. This section says only how it lands on *these sheets*, which volume 1 cannot tell you because it does not know which sheets you have. The repository is a TOGAF content metamodel, not an ArchiMate model: its sheets are object and relationship types, and ArchiMate is how you draw them. The mapping below is fixed, so that two architects drawing the same corner produce the same picture [11].

Sheet	ArchiMate 4 element	Sheet	ArchiMate 4 element
`001 Driver`	Driver	`014 Value Stream`	Value Stream
`002 Goal`	Goal	`015 Business Service`	Service
`003 Objective`	Outcome	`016 Business Process`	Process
`004 Measure`	Assessment	`017 Business Function`	Function
`005 Principle`	Principle	`018 Product`	Product
`006 Requirement`	Requirement	`019 Contract`	Business Object
`007 Constraint`	Requirement, dashed	`020 Data Entity`	Business Object
`008 Assumption`	Meaning	`021 Logical Data Component`	Data Object
`009 Organization Unit`	Business Actor	`022 Physical Data Component`	Artifact
`010 Actor`	Business Actor	`023 Application Component`	Application Component
`011 Role`	Role, or Stakeholder	`024 Information System Service`	Service
`012 Location`	Location, or Facility	`025 Technology Service`	Service
`013 Business Capability`	Capability	`028 Platform Service`	Service

Sheet	ArchiMate 4 element	Sheet	ArchiMate 4 element
`026 Logical Technology Componen`	Node	`053 Transition Architecture`	Plateau
`027 Physical Technology Compone`	Node, Device or System Software	`054 Architecture Alternative`	Course of Action
`029 Gap`	Assessment	`055 Building Block`	Grouping
`030 Work Package`	Work Package	`056 Project`	Work Package
`031 Deliverable`	Deliverable	`057 Governance Body`	Business Actor
`032 Standard`	Requirement	`058 Governance Gate`	Event
`041 Security Control`	Node, or Function	`062 Delivery Risk`	Assessment
`048 Environment`	Grouping	`063 Control Objective`	Requirement
`049 Resilience Tier`	Assessment	`064 Security Policy`	Principle
`050 Network Link`	Path	`065 Enforcement Point`	Application Interface
`052 Implementation Factor`	Assessment	`066 Architecture Increment`	Plateau

The industry extension needs its own row set, because five of its six types are the ones retail argues about and none maps to the obvious element.

`EXT` type	ArchiMate 4 element	The trap
`X01 SKU / Article`	Business Object	Not `Product`. The ArchiMate `Product` element is a commercial bundle; a SKU is the identified thing on the shelf. `018 Product`'s five rows are the bundles
`X02 Store / Location`	Location, with a `Facility` where the building matters	Draw the `Location` for the network, the `Facility` when the fixture, the yard or the dispensary is the subject
`X03 Assortment`	Business Object, or Grouping when drawn as the set	A `Grouping` of SKUs looks right and loses the date. An assortment is a set *for a location and a period*
`X04 Promotion`	Process, with an Event that starts it and a Business Object for the terms	Volume 1 is blunt under *A promotion is behaviour, not a thing*. Drawn as an object it cannot show uplift, funding or an end date
`X05 Planogram`	Business Object, realised by an Artifact	The plan is the object; the file the space tool exports is the artifact. Separate them or compliance has nothing to compare against
`X06 Fulfilment Node`	Location	**Not an ArchiMate `Node`.** The word collides, and it is the commonest notation error in a retail model. An ArchiMate `Node` is technology

Four notes on the sheet mapping, each of which catches somebody. **`O15`, `O24`, `O25` and `O28` all map to `Service`,** and so does a business service at the top of a layered view; the layer is carried by what *realises* the service, not by the element. **`O27 Physical Technology Compone` takes whichever element fits the thing** — a store server or a payment terminal is a `Device`, a database engine is `System Software`, everything else a `Node` — and the sheet's `Class` column is the only place that says which. **Gaps, measures, risks and resilience tiers are all `Assessment`,** statements about the state of affairs; a gap is not a `Requirement`, the requirement is what closes it. **`O53` and `O66` are both `Plateau` and not the same plateau:** `O53` holds three states, `O66` six things added at months 10 to 56, and they do not line up in the file either — `TA-3 Business visible` contains `WP-07` and `WP-11`, while `R23 Increment delivered by Proj` has `INC-4 Business visible` delivered by `PRJ-07 Core platform renewal`. Label which sheet each plateau came from.

A stakeholder is the `Stakeholder` element, not a `Business Actor`: the same `O11 Role` row is a `Role` in a business view and a `Stakeholder` in a motivation view. The element is `Role`; there is no "Business Role". Relationships follow ArchiMate's rules, not the relationship sheet's name — an application **realises** a service, a role is **assigned to** a process, a process **accesses** a business object, a capability is **composed of** its sub-capabilities, a driver **influences** a goal. Where a relationship sheet joins two rows in a way ArchiMate will not allow, the sheet has found you a modelling error, not a notation problem.

> **THE EXCEPTION: LEVEL-2 PROCESS MODELS ARE BPMN 2.0**
>
> ArchiMate has no swimlanes and no way to show a hand-off from one lane to another. A level-2 retail process is mostly hand-offs — buyer to planner, planner to space, space to store, store to the till, the till to the supplier portal — so drawing one in ArchiMate loses the thing worth drawing. Volume 1, chapter 8 (The process model) publishes fourteen level-2 processes in BPMN 2.0 and sets out the levels and the choice under *L0, L1, L2, L3 — and the warning*. This volume keeps to that: everything at level 1 and above stays in ArchiMate, with `O16 Business Process` as a `Process` joined to its value stream, its owning role and the objects it touches. Volume 1, chapter 16 says where the notation stops under *The honest limits*, and the limit that matters most here is that ArchiMate cannot date a price — which is why `E05 Price` is an entity and not an attribute.

The three ways people wreck a repository

Modelling everything. The symptom is a repository with more records than relationships, and this one has it: 813 records against 429 relationship instances. **Nineteen of the sixty-four object sheets have no row that any relationship sheet resolves** — `O05 Principle`, `O07 Constraint`, `O08 Assumption`, `O16 Business Process`, `O19 Contract`, `O21`, `O22`, `O25 Technology Service`, `O31 Deliverable`, `O33 Architecture Contract`, `O38 Data Set`, `O42 Metadata Element`, `O52 Implementation Factor`, `O54 Architecture Alternative`, `O57 Governance Body`, `O58 Governance Gate`, `O60`, `O61` and `O62 Delivery Risk` — and `O59 Compliance Assessment` makes twenty sheets outside the relationship web. A third of those are joined by an inline reference column instead — `O16` has a `Value stream`, `O21` an `Entity`, `O33` a `Work package` — a weaker join but a real one. The rest are joined to nothing by any reading: twenty-four principles that govern nothing, five bodies that

oversee nothing, four gates that test nothing. Before you populate a sheet, name the relationship sheet that will join it to something else.

Note what `RPT13 Orphan Analysis` does with that. It reports **two** orphans, `A11 Supplier Portal` and `A12 Retail Analytics`, both with the finding "No business service maps to it" [12]. Look at why: `R04 Business Service supported` has exactly ten rows and runs down the diagonal, `BSV-01` to `A01` through `BSV-10` to `A10`, so the two orphans are the two the diagonal ran out of rows for. The report's closing line, *an orphan is not always wrong; it is always worth a question*, is the best sentence in the file. The question here is not about the Supplier Portal. It is about `R04`.

Modelling nothing. The opposite failure, and the more common one. The architect reads the ADM, agrees with it, produces slides, and the repository stays exactly as loaded. The insurance repository caught this with a version-control report grading each sheet `Unchanged since baseline`. This one cannot, because there is no change log. Until you build one, the only evidence that you have done anything is the `ZZ Checks` numbers moving, and they move slowly.

Modelling the org chart. The most expensive of the three because it looks like work. `O09 Organization Unit` holds twelve rows of generic corporate structure — Executive, Finance, Risk & Compliance, Human Resources, Technology, Data & Analytics, Operations, Commercial, Customer Service, Supply & Logistics, Internal Audit, Legal. There is no Buying, no Merchandising, no Stores, no Pharmacy, no Property: the units that decide what to sell and the units that sell it are both absent. Then `R22 Org Unit owns Capability` joins the two sheets on the diagonal, `ORG-01` to `C01` down to `ORG-10` to `C10`, so that Executive owns Merchandising & Assortment, Human Resources owns Supply Chain & Replenishment, Technology owns Store Operations and Supply & Logistics owns Finance & Margin Management [13]. Ten rows, ten wrong answers, two units owning nothing, and the sheet looks complete.

Capabilities are what the enterprise can do; organisation units are who currently does it. Split the buying office by category or move space planning under supply chain and the capability map should not move. Chapter 2, step 2.1 rebuilds `O09`; chapter 4 rebuilds `R22`. Not the second before the first.

Adapting this to a retailer that is not Halverson

Most of what you were handed is reusable. Some of it is Halverson's and must go.

Keep as shipped	Re-derive for your retailer	Do not delete
The sheet structure, the three-row preamble, the header-on-row-4 convention and the `Ref` prefixes	`009`, `010`, `011`, `012`, `018`, `019`, `023` — your organisation, actors, roles, sites, products, contracts and applications	Nothing. Mark a record superseded; never delete a row that a relationship sheet or an inline reference column points at
`EXT Industry Extension` — the six retail object types, which are right	`001`–`004`: your drivers, goals, objectives and measures, and every `Baseline` figure in `004`	
`005 Principle` as a starting set of twenty-four to argue with	`013` levels 2 and 3, and every `Current` and `Target` score	
`057`, `058` — five bodies and four gates, renamed to your forums	Every `Owner`, `Head`, `Master system` and `Golden source` value in the file	
`063 Control Objective`'s six obligations, which are the right six for a South African retailer	`029 Gap`, `030 Work Package`, `056 Project` and everything downstream of them	

The order matters. Do `009 Organization Unit` and `012 Location` first, because `Head`, `Location` and `Owner` values across a dozen other sheets reference them by name. Then `011 Role`, because `Owner` columns should point at roles. Then `018 Product`. Then the motivation sheets. Leave capabilities until you have value streams, for the reason chapter 3 gives.

Four retail-specific adjustments you will need almost immediately, all of them metamodel extensions rather than data entry.

Format and ownership model have nowhere to live. `012 Location` carries four columns — `Ref`, `Location`, `Type`, `Scale` — with `Type` holding Corporate, Trading, Logistics and Technology. No format, no ownership, no banner, no trading area, no opening date. The sheet also mixes three kinds of thing: `LOC-01 Support Centre` is a site, `LOC-04 Convenience Format` is a format carrying "240 people", `LOC-08 Cloud Region` is an abstraction carrying "293 people". Add `Format`, `Ownership`, `Banner`, `Trading area` and `Opened`, and decide whether a row is a site or a class of site. Step 2.1 makes the ownership decision first because everything else depends on it.

The money is in the wrong currency. `030 Work Package` costs read `$2.8m` to `$10.0m`; `054 Architecture Alternative` reads `$24m` to `$88m`. For a retailer reporting in rands, a dollar sign in a board paper is a conversation you do not want. Change the symbol, or convert and record the rate and date in `01 Version Control`.

Classification is wrong, not merely absent. `020 Data Entity`'s `Classification` column reads `Public` against `E04 Assortment`, `E07 Planogram` and `E12 Customer` [10]. Customer as public is a POPIA finding; assortment and planogram as public is a commercial one, because a forward range plan and a space plan are among the most sensitive documents a retailer holds. Chapter 5 rebuilds the column. Until then do not publish `RPT05 Data Ownership` to anybody.

Two sheets claim the golden source and disagree on every row. `020 Data Entity` has `Master system`; `043 Master Data Rule` has `Golden source`. For all ten entities they share they name

different systems: `E01 SKU / Article` is `POS` against `Demand Forecasting & Replenishment`, `E05 Price` is `Workforce Management` against `POS` [14]. Both are nonsense, which is useful, because it shows what a mastering column looks like when nobody owns it. Pick one as authoritative in your tailoring decision and blank the other, before Phase C.

> **READ-ONLY FOR A WEEK**
>
> Before you change a cell, work through `ZZ Checks`, `RPT01 Dashboard` and `RPT13 Orphan Analysis`, then count the records in `00 Index` yourself and count how many object sheets no relationship sheet reaches. Four readings and one count. They tell you how big the repository is, where it is unlinked, what it believes about itself and where that belief is wrong. If you are inheriting a repository rather than starting one, those five numbers are your handover, and chapter 2 opens with them as step 2.4.

How to use this volume day to day

Open it at the step you are on. The part heading names the phase, the step numbers are stable, and chapters 13 to 20 are reference. When a step says volume 1's appendix A already drafts a catalogue, go and start from there, then argue with it. Your retailer is not Milkwood and it is certainly not Halverson.

THE CHAPTER IN TEN LINES

- The repository is 105 sheets: 64 object sheets, 25 relationship sheets, 13 reports, an index, an industry extension and an integrity check.
- `EXT Industry Extension` is the sixty-fifth object type, which is why the file says 65 and has 64 `O` sheets.
- 842 records only reconciles against a counted 813 if you also count `R06`'s 29 `INT-` interface rows as records — which concedes that an interface is an object with no object sheet.
- `ZZ Checks` claims every object type carries records while `O59 Compliance Assessment` is empty and `O29 Gap` holds one row.
- `PTC-01 Store server estate` reads `Past support` with a support end of 2027-03. The typed column and the dated column disagree, and `RPT10` believes the typed one.
- There is no version column, no change log, no change-request register and no baseline sheet anywhere in the file. Add four sheets and seven columns in week one.
- Do not prefix your own change requests `CR-`. `O61 Governance Change Request` already uses `CR-G01` to `CR-G05`.
- `O13` is dotted and unprefixed, `O55` reuses four other sheets' prefixes, `O46` has two columns called `Ref` and the second holds `T01`–`T07`, which nothing else in the file uses.
- Everything architectural is ArchiMate 4. The single exception is level-2 process models, which are BPMN 2.0, per volume 1 chapter 8. `X06 Fulfilment Node` is a `Location`, not an ArchiMate `Node`.
- Nineteen object sheets are reached by no relationship sheet. `RPT13 Orphan Analysis` reports two orphans, and both are artefacts of `R04` running down the diagonal and stopping at ten rows.

SOURCES FOR THIS CHAPTER

- **1.** Retail_TOGAF_Metamodel.xlsx, sheet 00 Index
- **2.** Retail_TOGAF_Metamodel.xlsx, sheet RPT01 Dashboard
- **3.** Retail_TOGAF_Metamodel.xlsx, sheet ZZ Checks
- **4.** The Open Group, TOGAF Standard, 10th Edition — Architecture Development Method https://pubs.opengroup.org/architecture/togaf10-doc/adm/
- **5.** Retail_TOGAF_Metamodel.xlsx — full sheet and column inventory, all 105 sheets
- **6.** Retail_TOGAF_Metamodel.xlsx, sheet O61 Governance Change Request
- **7.** Retail_TOGAF_Metamodel.xlsx, sheets O27 Physical Technology Compone and RPT10 Technology Currency
- **8.** Consumer Protection Act 68 of 2008, s 23 — disclosure of price of goods or services https://www.acts.co.za/consumer-protection/23_disclosure_of_price_of_goods_or_services
- **9.** Consumer Protection Act 68 of 2008, s 56 — implied warranty of quality; six months from delivery, with the consumer electing repair, replacement or refund
- **10.** Retail_TOGAF_Metamodel.xlsx, sheets O20 Data Entity and O11 Role
- **11.** The Open Group, ArchiMate Specification — elements, relationships and viewpoints https://pubs.opengroup.org/architecture/archimate3-doc/
- **12.** Retail_TOGAF_Metamodel.xlsx, sheets RPT13 Orphan Analysis and R04 Business Service supported
- **13.** Retail_TOGAF_Metamodel.xlsx, sheets O09 Organization Unit and R22 Org Unit owns Capability
- **14.** Retail_TOGAF_Metamodel.xlsx, sheets O20 Data Entity and O43 Master Data Rule

CHAPTER 2

Standing the practice up

The phase that decides who may say no, to what, on what evidence — and, in a retailer, the phase that first has to settle which formats and which ownership models are even in scope.

The Preliminary Phase has a bad reputation because it is usually done as a slide deck. Done properly it decides whether anything else in this volume will work: who may say no, to what, on what evidence, and whether anyone can prove next year what the repository said this year. Fifteen steps, four to six weeks if the sponsorship is real [6].

Halverson Stores Group, the retailer in the repository, is invented and its figures illustrative. Its Preliminary Phase went half right: sixty-four object sheets loaded, twenty-four principles written, five bodies and four gates described — and `005 Principle`, `031 Deliverable`, `057 Governance Body`, `058 Governance Gate` and `062 Delivery Risk` joined to nothing at all, while `DEC-01` to `DEC-04` in `060 Governance Decision` all read `Outstanding`, including `DEC-04 Designate data owners` [1] [2]. The structure exists; the practice does not.

> **BEFORE STEP 2.1**
>
> Do step 2.4 first if you are inheriting this repository rather than starting it. `ZZ Checks` takes twenty minutes to read and it will change what you do in steps 2.5 and 2.8. Chapter 1 explains why its numbers need counting before they are quoted.

> **BEFORE YOU START THIS PHASE**
>
> Read these first and keep volume 1 open at them.
>
> - **Volume 1, chapter 2, *The ownership models, and what each does to the architecture*.** Corporate, franchise and voluntary trading, and what each does to who masters a price. Step 2.1 is unanswerable without it.
> - **Volume 1, chapter 7, *Who owns the shop*.** The same question from the operating-model side.
> - **Volume 1, chapter 5, *Your real counterparties*.** The people whose agreement you need and who will never come to an architecture forum.
> - **Volume 1, chapter 18, *Six numbers that tell you whether the estate is healthy*.**

The shape of the phase

Steps	What they settle
2.1 – 2.4	Formats, ownership models and boundary; change control; the first baseline; the health check
2.5 – 2.8	Tailoring, the industry extension, and the one convention that fixes `Owner`
2.9 – 2.12	Principles, bodies, gates, and the architecture function itself
2.13 – 2.14	Maturity, and whether the enterprise's stated intent holds together
2.15	The deliverable register, and closing against a baseline

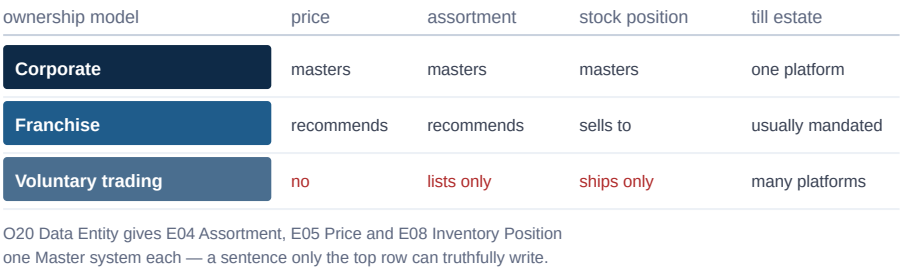


Figure 2.1 Step 2.1 in one table. What the group masters depends on the ownership model, not on the reference architecture — and the repository's `Master system` column assumes the top row. Read volume 1, chapter 2 before you fill it in.

Step 2.1 — Settle which formats and which ownership models are in scope

Produce. A one-page scope statement naming every format, banner and ownership model inside the architecture boundary — and, explicitly, the ones outside it.

You need. The group structure, the store list with ownership against each site, and the franchise or membership agreement if one exists.

Look it up. Volume 1, chapter 2, *The ownership models, and what each does to the architecture* — corporate, franchise and voluntary trading, and what each relocates.

Where it lands. `009 Organization Unit` and `012 Location`, after you have added the columns neither has.

> **`O12 Location — four columns, three kinds of thing, and nothing about format`**
>
> `Ref` — the next `LOC-` number. `Location` — the site or the class of site, and you must decide which. `Type` — Corporate, Trading, Logistics, Technology. `Scale` — a headcount, the only size the sheet records.
>
> The eight shipped rows mix three things. `LOC-01 Support Centre` and `LOC-05 National Distribution Centre` are sites; `LOC-03 Metro Store Format` and `LOC-04 Convenience Format` are formats, and `LOC-04` carries "240 people" as though a format had a headcount; `LOC-08 Cloud Region` is neither. There is no `Format`, `Ownership`, `Banner`, `Trading area` or `Opened` column, so the sheet cannot say that a store is franchised — the one fact everything downstream turns on.

How.

1. List the formats — grocery, convenience, pharmacy and health, apparel, cash-and-carry, online, whichever the group runs. One row each.
2. Against each, write the ownership model: corporate, franchise or voluntary trading. If a format runs two, split it.
3. For each ownership model answer three questions in writing: does the group master the price, the assortment, or the stock position? In a franchise or voluntary estate the answer to all three is usually no for most of the turnover.
4. Add `Format`, `Ownership`, `Banner`, `Trading area` and `Opened` to `O12`, then decide whether a row is a site or a class of site and never mix them again.
5. Replace all twelve rows of `O09 Organization Unit` rather than editing them, and write the exclusions down as sentences with reasons.

Signed by. The Chief Executive, or the Executive Committee (`GB-04` in `O57 Governance Body`). Nobody below that can rule a format out of scope.

Done when. Every format appears in the scope statement with an ownership model against it; the three mastering questions have written answers; and the statement carries a date and a signature.

Shows up in. Nothing yet, and that is the first thing to notice. No report counts locations or organisation units, and `R22 Org Unit owns Capability` is the only relationship sheet reaching `O09`. The effect appears in chapter 4.

Goes wrong when. You assume the corporate model because the reference architecture does. The repository already does it for you: `E04 Assortment`, `E05 Price` and `E08 Inventory Position` each carry a single `Master system`, a sentence a voluntary-trading group cannot truthfully write. Phase C then rests on a claim the business does not hold.

`009 ORGANIZATION UNIT` IS NOT A RETAILER'S ORGANISATION CHART

The twelve shipped rows are a generic corporate structure: Executive, Finance, Risk & Compliance, Human Resources, Technology, Data & Analytics, Operations, Commercial, Customer Service, Supply & Logistics, Internal Audit, Legal. No Buying, no Merchandising, no Stores, no Pharmacy, no Property. And `Head` holds role titles of which six of the twelve resolve to no row in `011 Role`, among them `Chief Commercial Officer`, `General Counsel` and `Head of Supply` [7]. Replace the sheet, do not patch it.

BEFORE YOU DRAW THE BOUNDARY

For each format, does the group master the price, the assortment and the stock position? Write the three answers, not one.

Which format are you most tempted to leave out, and what turns up in Phase C if you do?

Which role — not which person — can rule a format out of scope, and has that role agreed to be asked?

Step 2.2 — Add change control before you touch a single record

Produce. Four new sheets and seven new columns, so every later change leaves a trace.

You need. The file and a morning. Do not wait for a governance decision; `GB-05 Architecture Function` already holds the mandate to maintain the repository [1].

Where it lands. Four sheets that do not exist yet, inserted before `001 Driver`.

> **The four sheets to add, and the minimum columns in each**
>
> Chapter 1 gives the full column list and the value set for each. In short: `01 Version Control` for the rules and the workbook's current state; `02 Change Log` with `Ref`, `Date`, `Sheet`, `Record`, `Column`, `Previous Value`, `New Value`, `Changed By`, `Change Ref`, one row per cell changed; `03 Change Requests` with `Ref`, `Raised`, `Raised By`, `Type`, `Scope`, `Description`, `Status`, `Decided`, `Approved By`, `Decision`, prefixed `RCR-` because `061 Governance Change Request` already holds `CR-G01` to `CR-G05`; and `04 Baselines` with `Ref`, `Name`, `Date`, `Workbook Version`, `Approved By`, `Status`, `Note`, `Changes Since`.
>
> Then seven columns at the right-hand end of every sheet in your year-one set: `Version`, `Record Status`, `In Repository Since`, `Last Changed`, `Changed By`, `Change Ref`, `Approved By`.

How.

1. Insert the four sheets with the headers above.
2. Write the rules into `01 Version Control`. The one that matters: `Version` is derived from `02 Change Log` and never typed.
3. Add the seven columns to the sheets step 2.5 will keep. Leave the rest alone.
4. Set every existing record to `Version` = 1.0, `Record Status` = Baseline, `Changed By` = Repository load, `In Repository Since` = the date you received the file.
5. Lock the seven columns against typing and leave `Version` as a formula.
6. Tell the domain architects, in writing, that an edit without a request is a defect and will be reverted.

Signed by. The Chief Enterprise Architect (`ROL-08`). A working practice, not an architecture decision — waiting for a board slot is how it does not happen.

Done when. The four sheets exist with headers, every record in your year-one set reads `Version` = 1.0 and `Record Status` = Baseline, and `01 Version Control` names a version and a state date.

Shows up in. `01 Version Control`, and from here on every `Change Ref` in this volume. There is no `RPT` sheet for it; add a fourteenth once the log has thirty rows.

Goes wrong when. You defer it until after the first phase, because the phase is urgent and the housekeeping is not. The first phase is exactly the period in which the repository changes most, and nothing from it will be recoverable.

Step 2.3 — Cut the first baseline, which means building the sheet first

Produce. A named, dated, approved snapshot of the repository as you received it, before you change anything.

You need. Step 2.2. If you have already loaded your own data you are too late for a clean baseline — say so in the `Note` column rather than pretending otherwise.

Where it lands. `04 Baselines`, one row: `BL-001`, `Name` = Preliminary entry baseline, `Date`, `Workbook Version` = 1.0, `Approved By` = Chief Enterprise Architect, `Status` = Current, `Note` = what it is for and what is known to be wrong with it.

How.

1. Save a read-only copy with the baseline reference in the filename.
2. Add the `04 Baselines` row with `Status` = Current. Exactly one row may read Current.
3. Produce the extract by hand. This repository has **no export sheets**, so copy the sheets you care about into a flat `Sheet`, `Record`, `Column`, `Value` layout and keep it beside the baseline copy.
4. Record what the baseline already knows is wrong: 842 against a counted 813, the empty `O59`, the one-row `O29`, the `Owner` columns, `PTC-01`'s support date.

Signed by. The Chief Enterprise Architect for the first baseline, the Architecture Board (`GB-01`) for every one after.

Done when. `04 Baselines` has exactly one `Current` row, a read-only file exists with that reference in its name, and a flat extract sits beside it.

Shows up in. `01 Version Control`, `Baselines Taken` = 1.

Goes wrong when. You cut the baseline after loading your own data, so you can never show what you inherited. Or two rows read Current, and every later comparison quietly measures against the wrong one.

Step 2.4 — Run `ZZ Checks` and `RPT13 Orphan Analysis` on day one, and check their arithmetic

Produce. Three numbers you can state without opening the file, and a list of defects with a decision against each.

You need. Nothing. This is the first thing you do, before step 2.1 if you are inheriting the repository.

Look it up. Volume 1, chapter 18, *Six numbers that tell you whether the estate is healthy* — the six an architect should be able to quote in a corridor.

Where it lands. Nowhere yet. The readings go into your own notes and then into `03 Change Requests`, one request per genuine defect.

> **`ZZ Checks` — fifteen lines, and the four that do not hold**
>
> Three columns: `Check`, `Result`, `Threshold`. Nearly every threshold is zero, because what a healthy estate shows is nothing. The results: 16 unmanaged interfaces, 20 with no logging, 9 direct database reads, 3 platform components past support, 4 security controls absent, 3 control objectives without evidence, 2 data entities without lineage, 7 work packages past their end date, 0 vacant architecture roles [3]. Nine stand up to a count against `R06`, `O27`, `O41`, `O63`, `O20` and `O30`. Four do not.
>
> `Object types populated` = 65 against `Every type carries records` is false: `O59` holds none, `O29 Gap` holds one. `Records` = 842 against a counted 813. `Platform components past support` = 3, but only `PTC-06` and `PTC-12` have a `Support end` behind us. And `Vacant architecture roles` = 0 because every row of `R21 Role performed by Actor` reads `Filled` — a statement about the sheet, not the retailer.
>
> The closing line is what makes the sheet worth reading: *the thresholds are what a healthy estate would show; the results are what this one does; that distance is the work.*

How.

1. Read `ZZ Checks` and write down the three worst numbers. For most retailers those are unmanaged interfaces, interfaces with no logging, and direct database reads.
2. Count the records yourself. The per-sheet counts in `00 Index` total 813 against a headline of 842 [1][2][3]. Decide which you will quote and write the decision down.
3. Read `RPT13 Orphan Analysis`. Two orphans, `A11 Supplier Portal` and `A12 Retail Analytics`, both reading "No business service maps to it" [4]. Then look at `R04 Business Service supported`: ten rows down the diagonal, which is why exactly the last two applications have nothing mapped to them.
4. Run the count the report does not. For each of the sixty-four object sheets, ask whether any relationship sheet names it. Nineteen do not.
5. Raise one change request per genuine defect. A request that says "fix the repository" is never closed.
6. Put the three numbers on the Architecture Board's standing agenda, with last month's beside them.

Signed by. Nobody. This is the architect's own instrument, and `GB-05 Architecture Function` already carries the mandate to maintain the repository.

Done when. You can state the three numbers from memory, every defect has a change request or a written reason for being left alone, and the four `ZZ Checks` lines above are annotated in your notes with what they should read.

Shows up in. Itself, next month. A healthy trend is unmanaged interfaces and unreached object sheets both falling. Nothing in the file computes the trend, which is why step 2.2 came first.

Goes wrong when. The sheet is read once, at the end of a phase, as a formality. By then the defects are six months old. Read it monthly and each one costs ten minutes.

READING YOUR OWN REPOSITORY BEFORE YOU ADD TO IT

Which three numbers on `ZZ Checks` would most embarrass this retailer in front of an inspector, and who owns each?

Which green number on `ZZ Checks` or `RPT01 Dashboard` is green because the sheet behind it is empty or uniform?

Of the nineteen object sheets no relationship sheet reaches, which three will you link this quarter, and to what?

Step 2.5 — Tailor the metamodel, and write down what you are not doing

Produce. A one-page tailoring decision naming the object sheets, relationship sheets and reports in scope for year one, and the ones deliberately left empty.

You need. Step 2.4, and an honest estimate of how much modelling time you actually have.

Where it lands. `01 Version Control` as a rule row, and `O31 Deliverable` as a deliverable.

How.

1. Start from the reports you must produce. Thirteen reports read about twenty-two object sheets between them; everything else is optional in year one.
2. Put these in scope: `O01` – `O14`, `O20 Data Entity`, `O23 Application Component`, `O29 Gap`, `O30 Work Package`, `O31 Deliverable`, `O55 Building Block`, `O57`, `O58`, `EXT`, and the relationship sheets `R01` – `R06`, `R08`, `R21`, `R22`. Roughly a third of the file, feeding eleven of the thirteen reports.
3. Put these out of scope and say so: `O37 Data Attribute` with its sixty rows, `O40 Source To Target Mapping`, `O42 Metadata Element`, `O47 Reference Model Entry`, `O50 Network Link`. They belong to Phases C and D.
4. For each sheet in scope, name the relationship sheet that will join it. If there is not one, move it out of scope.
5. Add the seven record-keeping columns to the in-scope sheets only, then put the decision through the Architecture Board with a date.

Signed by. The Architecture Board (`GB-01`).

Done when. Every sheet in the file is on one of two lists, and every in-scope sheet has a named relationship sheet against it.

Shows up in. `RPT01 Dashboard`, where `Records` should grow in the sheets you chose and stay flat everywhere else. Growth in an out-of-scope sheet is somebody modelling for pleasure.

Goes wrong when. You decide to populate everything because the structure is there. Sixty-four sheets is a two-year job with a team of four, and the repository will be stale in the sheets that matter before it is complete in the sheets that do not.

Step 2.6 — Read `EXT Industry Extension`, and work out why the TOGAF core cannot hold six retail types

Produce. A written position on each of the six extension types: what it is, which core sheet shadows it, and whether you will instantiate it.

You need. Step 2.5. Volume 1 chapters 9 and 16, open.

Look it up. Volume 1, chapter 9, *Article, style, option, SKU: the argument the model lives or dies on* — the argument the data model lives or dies on, and the reason `X01` exists.

Where it lands. `EXT Industry Extension`, which has three columns and holds no instances: `Ref`, `Industry extension type`, `What it holds and why the core metamodel is not enough` [5].

> **`EXT Industry Extension` — six rows, and five of them already have a shadow**
>
> `X01 SKU / Article`, shadowed by `E01` in `O20 Data Entity`. The core offers `Product`, which is the commercial bundle, and `Data Entity`, which is the record *about* the thing. Neither is the thing.
>
> `X02 Store / Location`, shadowed by `E03` and partly by `O12 Location`. The core `Location` carries no format, no capacity and no ownership model.
>
> `X03 Assortment`, shadowed by `E04`. No core type is bounded by a location *and* a period at once.
>
> `X04 Promotion`, shadowed by `E06`. The core has no type for funded behaviour with an end date.
>
> `X05 Planogram`, shadowed by `E07`. A spatial plan is neither a data entity nor a physical component.
>
> `X06 Fulfilment Node`, **shadowed by nothing.** `LOC-06 Fulfilment Centre` is a site, not a node, and the point of a fulfilment node is that the same store is one on Tuesday and not on Wednesday.

Ref	extension type	what it holds	where an instance already lives
X01	**SKU / Article**	smallest sellable unit	O20, E01 SKU / Article
X02	**Store / Location**	trading or fulfilment site	O20 E03; O12 Location
X03	**Assortment**	ranged set, location and period	O20, E04 Assortment
X04	**Promotion**	funded offer with mechanics	O20, E06 Promotion
X05	**Planogram**	shelf layout by fixture	O20, E07 Planogram
X06	**Fulfilment Node**	can source, pick or hand over	nowhere at all

EXT holds no instances. It is a register of six types, three columns wide.

Figure 2.2 `EXT Industry Extension`, all six rows, with the sheet that already holds each type's instances. Five of the six are recorded elsewhere under a different name. `X06 Fulfilment Node` is recorded nowhere — and it is the one type whose membership changes from day to day.

How.

1. Read all six rows and the sentence against each.
2. Find the shadow for each. Five are in `O20 Data Entity`; one is nowhere.
3. Decide, per type, whether the extension row is the register and `O20` holds the instances, or the reverse. Do not leave both claiming it.
4. Write the position for `X06 Fulfilment Node` first: it has no shadow, and it is the type that breaks when the retailer turns a store into a dark store.
5. Record the six decisions in `03 Change Requests` with `Type` = Extension.

Signed by. The Chief Enterprise Architect, with the Chief Data Officer countersigning anything that moves an instance between sheets.

Done when. Each of the six rows has a written position naming the sheet that holds its instances, and no type is claimed by two sheets.

Shows up in. `RPT05 Data Ownership`, where every entity you decided `O20` keeps must have an `Owner` and a `Master system` that survives step 2.8.

Goes wrong when. The extension is read as decoration. Six declared types with no instances and no relationship sheet is the most expensive kind of nothing: it looks like the industry has been modelled.

Step 2.7 — Extend the extension without breaking the model

Produce. A rule for adding an extension type, and nought to three new types with that rule applied.

You need. Step 2.6, and the format and ownership decision from step 2.1.

Look it up. Volume 1, chapter 16, *The mapping* — the element each retail concept maps to, which is the test of whether a new type is really new.

Where it lands. `EXT Industry Extension` for the type, and a new object sheet for its instances. Never columns bolted onto `EXT`: it is a register of types, three columns wide, and it should stay that way.

How.

1. Apply three tests before adding anything. Does the thing have an identity that outlives a transaction? Will a relationship sheet join it to something else? Does every core sheet hold it only by lying about what it is? Three yeses or do not add it.
2. Number the type `X07` onward. Never reuse `X01` – `X06`, even one you decided not to instantiate.
3. Create the instance sheet at the end of the object range with the next free number, the three-row preamble and the header on row 4. Do not insert it in the middle; the gap at `O34` and `O51` shows what happens when somebody does.
4. Create the relationship sheet at the same time. A type with no relationship sheet fails test two.
5. Record the extension in `03 Change Requests` with `Type` = Extension and the three test answers in `Description`, and update `00 Index` and the `Object types populated` line in `ZZ Checks`, or the file's arithmetic drifts further.

Signed by. The Architecture Board (`GB-01`). A metamodel change is the one kind of change a working practice cannot absorb.

Done when. Every new type has an instance sheet, a relationship sheet, three recorded test answers and an `00 Index` row.

Shows up in. `ZZ Checks`, `Object types populated` and `Records`, both of which you now maintain by hand because nothing in the file computes them.

Goes wrong when. Somebody adds a type for a thing the core already holds — a supplier, a contract, a purchase order — and the repository grows two places to look for one fact. The three tests exist to stop exactly that.

THE EXTENSION DECISION

Of `X01` to `X06`, which will you instantiate this year, and in which sheet do the instances live?

`X06 Fulfilment Node` has no shadow anywhere in the file. What does your estate call it, and which system is authoritative?

Name one candidate for `X07`, and answer the three tests against it in writing.

Step 2.8 — Settle one convention for every `Owner` column, and apply it

Produce. One rule for what an `Owner`, `Head`, `Lead` or `Owner`-like value means, applied across every sheet in your year-one set.

You need. Step 2.1's rebuilt `O09` and a rebuilt `O11 Role`.

Where it lands. Every `Owner` column in scope, and one rule row in `01 Version Control`.

> **`One column name, four conventions, no integrity check`**
>
> `O05 Principle` holds `Architecture Board` on all twenty-four rows — a governance body from `O57`, not a role. `O23 Application Component` holds organisation unit names on all twelve, with `A01 Merchandising System` owned by `Internal Audit`, which is an independence problem as well as a convention one. `O20 Data Entity` holds role titles of which one in seven resolves: `Data Steward` is `ROL-16`, while `Head of Buying`, `Head of Merchandising`, `Head of Pricing`, `Head of Finance`, `Head of Store Operations` and `Head of Order Fulfilment` appear nowhere in `O11 Role` [7]. `O62 Delivery Risk` and `O56 Project` hold role titles that mostly do resolve, except `Head of Procurement`. Four conventions, one column name, no validation anywhere in the file.

How.

1. Settle the rule: an owner is a `ROL-` reference from `O11 Role`, and the name beside it is a courtesy column.
2. Rebuild `O11 Role` first. The eighteen shipped rows are executives, four domain architects, a data steward and an auditor — no buyer, no planner, no store manager, no pharmacist. Chapter 3, step 3.3 names the ones to add.

3. Walk the `Owner` columns in your year-one set. Replace what resolves and **blank what does not**, rather than guessing a title.
4. For `O23`, decide whether an application owner is a role or a unit; if a unit, rename the column in your extension.
5. Log every change in `02 Change Log`. This step produces a hundred-odd log rows, which is what the log is for.
6. Put the rule in `01 Version Control` and in the onboarding note for new architects.

Signed by. The Chief Enterprise Architect, with the Data Governance Council (`GB-03`) for the `O20` owners, since `DEC-04 Designate data owners` is its outstanding decision.

Done when. Every `Owner` value in scope either resolves to a row in `O11 Role` or is blank, and the rule is written down once.

Shows up in. `RPT05 Data Ownership`, whose `Owner` column becomes meaningful for the first time. Expect it to look worse before it looks better: blanks are honest.

Goes wrong when. You fill the blanks with plausible titles. An invented owner is worse than an empty cell, because an empty cell gets chased and a plausible one gets quoted.

Step 2.9 — Adopt, edit or retire the twenty-four principles, and add the columns `005` lacks

Produce. A principle set your Architecture Board will actually enforce, with the structure a principle needs.

You need. Steps 2.1 and 2.8. Two hours of the board's time, not twenty minutes at the end of an agenda.

Look it up. Volume 1, chapter 15, *What an inspector actually asks to see* — what a principle has to be able to produce when somebody comes looking.

Where it lands. `O05 Principle`, after you have extended it.

> **`O05 Principle — 24 rows, six columns, and four columns missing`**
>
> `Ref` `PRN-01` to `PRN-24`. `Principle` — the name. `Category` — Business, Application, Data, Technology, Security. `Statement` — one sentence. `Owner` — `Architecture Board` on every row. `Status` — Approved or Draft.
>
> **There is no `Tier` column**, so nothing distinguishes a principle binding the whole enterprise from one binding a domain or a single solution — and every principle is then enforced at the same weight, which means none is. Add `Tier` with the values Enterprise, Domain and Solution. Add `Rationale` and `Implications` too: a principle with a statement and no implications cannot be tested at a gate. Add `Approved` and `Review date`, because `Status` with no date cannot age.
>
> Two patterns before you edit a row. `Status` reads `Draft` on `PRN-05`, `PRN-10`, `PRN-15` and `PRN-20` — every fifth row, a generation pattern rather than an assessment. And `Category` is unreliable: `PRN-15 Requirements-Based Change` is filed under Technology, `PRN-23 Models are Governed` under Security.

How.

1. Add the five columns before you touch a statement.
2. Read all twenty-four aloud to the board. Anything nobody will defend gets retired, not softened.
3. Set `Tier` on every surviving row. Expect four to six at Enterprise; more than ten and the tier means nothing.
4. Write `Implications` for every Enterprise-tier principle. If you cannot name a consequence, the principle is a slogan.
5. Add the retail-specific ones the set is missing: price integrity at the shelf edge, one article identity across the estate, traceability to the batch for recall, the trading calendar mastered once.
6. Reset `Status` and `Approved` from the board's decision and clear the generated `Draft` values.

Signed by. The Architecture Board (`GB-01`), which `O57` already says holds `G1` and `G2`.

Done when. Every row has a `Tier`, every Enterprise-tier row has `Implications`, and no `Status` value is inherited from the load.

Shows up in. Nowhere, and that is the finding. No report reads `O05` and no relationship sheet reaches it, so twenty-four principles govern nothing. Chapter 10 joins them to gate criteria; until then the only evidence they exist is the board minute.

Goes wrong when. The set is adopted wholesale because it looks like TOGAF's. Five of the twenty-four are about security and none is about price, availability or shelf-edge accuracy — the three things a retail board will ask about.

Form P-1 — the principle register

One row per principle. These are the six real columns of `005 Principle` in sheet order; `Tier`, `Rationale`, `Implications`, `Approved` and `Review date` are the columns you added in step 2.9 and they are typed straight into Excel. Twenty to thirty rows for a first pass. Signed by the Architecture Board.

Ref	Principle	Category	Statement	Owner	Status
PRN-25	Price Integrity at the Shelf	Data	Example, delete me		Draft

Step 2.10 — Constitute the governance bodies, and record what each has refused

Produce. Five constituted bodies with a membership, a cadence, a mandate and a first decision each.

You need. Step 2.1's signature, the existing committee calendar, and the trading calendar beside it.

Look it up. Volume 1, chapter 5, *Your real counterparties* — who actually decides, and who merely attends.

Where it lands. `O57 Governance Body` and `O60 Governance Decision`.

> **`O57 Governance Body` — five rows, and three things the sheet cannot tell you**
>
> `Ref` `GB-01` to `GB-05`. `Governance body` — Architecture Board, Investment Committee, Data Governance Council, Executive Committee, Architecture Function. `Cadence` — Monthly, Quarterly, or `Continuous` for the function. `Membership` — a prose list. `Mandate` — a prose sentence, and the only place the file says which body holds which gate.
>
> No `Chair`, no `Standing`, no `Last met`, so the sheet cannot distinguish a body that exists from one that meets. No reference to `O58 Governance Gate` either: `GB-01`'s mandate reads "Holds G1 and G2" as text, which no report can read. And `Membership` names roles `O11 Role` does not hold, such as a CISO delegate.

How.

1. Map each of the five rows onto a forum that already meets. A new forum is a year of diary negotiation.
2. Add `Chair`, `Standing` and `Last met`. Set `Standing` to Constituted only when the body has met once with the mandate in front of it.
3. Put the gate reference in a real column rather than in the `Mandate` sentence.
4. Set the cadence against the trading calendar, not the finance calendar. A monthly board cannot gate a range review that runs twice a year.
5. Record each body's first decision in `O60 Governance Decision`, including the refusals, and clear the four `Outstanding` rows or restate them with a date.

Signed by. The Executive Committee (`GB-04`), which is the sheet's own escalation point.

Done when. Each of the five rows reads Constituted with a chair and a date, and `O60` holds at least one decided row per body.

Shows up in. `O60 Governance Decision`. Five bodies and seven decisions, four of them `Outstanding`, is a governance framework that has been designed and not started.

Goes wrong when. All five are constituted and none refuses anything for two quarters. A board that has never said no has not been tested, and the first time it tries, it will lose.

Form P-2 — the governance body register

One row per body. These are the five real columns of `057 Governance Body` in sheet order; `Chair`, `Standing` and `Last met` are the columns you added and they are typed straight into Excel. Five to eight rows. Signed by the Executive Committee.

Ref	Governance body	Cadence	Membership	Mandate
GB-06	Trading & Range Forum	Twice yearly	Example, delete me	

Step 2.11 — Adopt the four gates, and write what each one tests

Produce. Four gates with a test, a body, an evidence requirement and a place in the trading calendar.

You need. Steps 2.9 and 2.10. The delivery lifecycle as it is actually run.

Where it lands. `058 Governance Gate`, and `059 Compliance Assessment` from the first gate you hold.

> **`058 Governance Gate` — four rows, and no column for who holds them**
>
> `Ref` `GT-0` to `GT-3`, numbering from zero because the set starts at `G0 Charter`. `Gate` — `G0 Charter`, `G1 Design`, `G2 Build complete`, `G3 Benefit`. `When` — Before work starts, Before build, Before release, After operation. `What it tests` — one good sentence each.
>
> No `Held by`, no `Evidence required`, no `Outcome`. The body is only recoverable from `057`'s `Mandate` prose. And `059 Compliance Assessment`, the sheet a held gate writes into, has zero rows, as does `R24 Compliance Assessment tests`. Four gates described, none held.

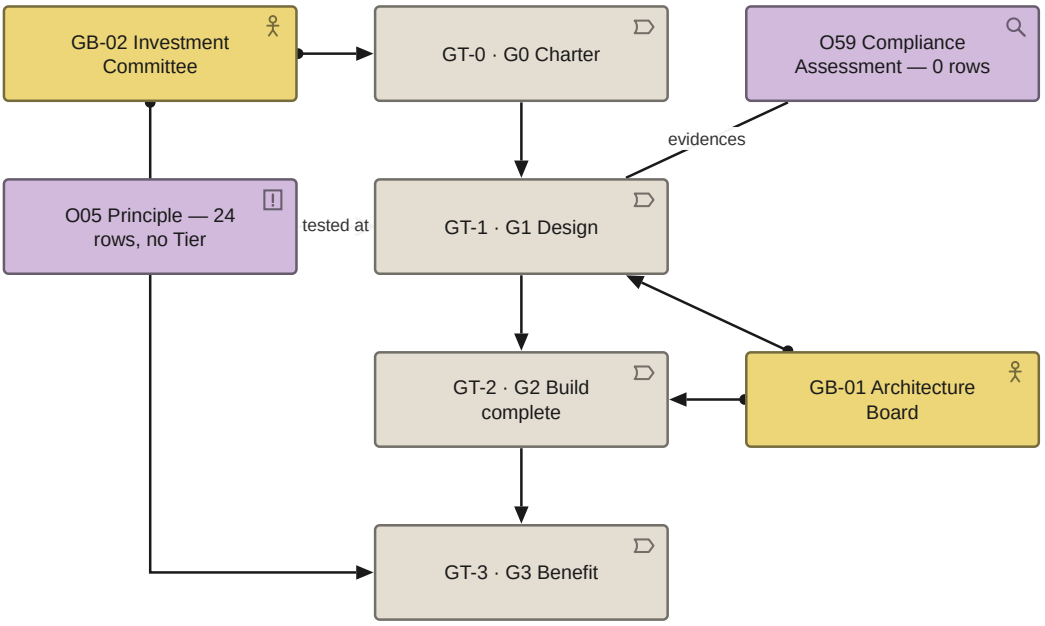


Figure 2.3 The four gates of `058 Governance Gate` in ArchiMate 4. A gate is an `Event` — a point the work either passes or does not — triggering the next; the body that holds it is a `Business Actor` assigned to it, which `057 Governance Body` states only in prose. The principles that `G1` tests are in `005`, which no relationship sheet reaches, and the assessment the gates should write into holds zero rows. Halverson Stores Group is invented.

How.

1. Add `Held by`, `Evidence required` and `Outcome` to `058`.
2. Against each gate, write the evidence a submission must bring. `G1 Design` tests design against principles, standards and the reference architecture, so it needs the principle references from step 2.9.
3. Put the gate dates in the trading calendar. Nothing is gated in the peak window, and `CON-06 Change freeze windows` already says so.
4. Hold `G0` on something small, this month, and write the result into `059 Compliance Assessment` with your chosen prefix and the join into `R24`.
5. Publish the criteria before the first submission, not after.

Signed by. The Architecture Board for `G1` and `G2`, the Investment Committee for `G0` and `G3`, as `057` states in prose and you are now stating in a column.

Done when. Each gate has a body, an evidence list and a published criterion, and `059` holds at least one row.

Shows up in. `059 Compliance Assessment` and `R24 Compliance Assessment tests`. Both empty means the gates are a diagram.

Goes wrong when. `G1` becomes a design review with no power to refuse. `061` shows what that looks like: `CR-G01` grants a dispensation for point-to-point file transfer expiring 2029-06,

and `CR-G05` asks the board to refuse or re-baseline the integration package. A gate with a dispensation route and no refusal is a queue.

Step 2.12 — Stand up the architecture function as an organisation unit and a capability

Produce. The architecture function recorded as something the enterprise has, not something an individual does.

You need. Step 2.1's rebuilt `O09`, and step 2.8's rebuilt `O11`.

Where it lands. `O09 Organization Unit`, `O13 Business Capability`, `R22 Org Unit owns Capability`.

How.

1. Add an `ORG-` row for the architecture function with a `Head` that resolves to `ROL-08 Chief Enterprise Architect`.
2. Add a level-1 capability for architecture and portfolio governance. `O13`'s ten level-1 rows are all trading capabilities — there is no architecture capability in the file, which is why nothing in `RPT03 Capability Heatmap` grades the practice itself.
3. Decompose it to level 2 only: repository, standards, assurance, portfolio.
4. Join unit to capability in `R22`, and do not touch the other ten rows; chapter 4 rebuilds them.
5. Confirm the five architecture rows in `O11`, `ROL-08` to `ROL-12`, against who actually does the work, and record a vacancy as a vacancy.

Signed by. The Chief Information Officer, with the Executive Committee (`GB-04`) noting the capability.

Done when. `O09` has an architecture unit, `O13` has an architecture capability with children, and `R22` joins them.

Shows up in. `RPT03 Capability Heatmap`, which reports level 1 only, so the new capability appears as an eleventh row. If its `Current` is higher than the trading capabilities you are measuring, re-read your own assessment.

Goes wrong when. The function is recorded as five named people. People leave; `O10 Actor` holds eighteen rows all reading `Filled`, which is exactly the fiction this step is meant to break.

Step 2.13 — Run the maturity assessment, and decide where the result lands

Produce. A current and target maturity for every level-1 capability, with the scale published and the evidence recorded.

You need. Steps 2.1 and 2.12. Appendix H holds the instrument and the question set.

Look it up. Volume 1, chapter 7, *The value chain is six clocks, not six steps* — why a retailer's capabilities mature at different speeds and cannot be graded on one axis.

Where it lands. `O13 Business Capability`'s `Current` and `Target` columns, or a sheet you add. Decide which before you run it.

> **`There is no maturity sheet, and O13 has no scale`**
>
> `O13 Business Capability` carries `Criticality`, `Current`, `Target` and `Disposition` on all hundred rows. `Current` and `Target` are integers between 1 and 5, and **nothing in the file says what the five mean**, who assessed them, or when.
>
> The values will not survive a reading [8]. `C01 Merchandising & Assortment` is `Critical` with `Current` 1, `Target` 2 and `Disposition` Sustain — you do not sustain a critical capability at maturity 1, and the single row in `O29 Gap` is this capability's uplift. `C04 Supply Chain & Replenishment` is `Medium`, which in a retailer means availability is a medium concern. Five of the ten level-1 rows read Low or Medium. Treat all two hundred numbers as unset.

How.

1. Publish the scale in `01 Version Control` before you assess anything. Five levels, one sentence each, with an observable test.
2. Add `Assessed by`, `Assessed on` and `Evidence` to `O13`, or put the assessment in its own sheet and leave `O13` holding the latest value only.
3. Assess level 1 only. Ten capabilities, two half-days, two assessors scoring independently and then arguing.
4. Set `Target` from the business's stated intent, not from five minus the current score.
5. Reset `Criticality` at the same sitting. It decides where attention goes and it is currently wrong on at least half the rows.
6. Record the ten disagreements, not the ten averages.

Signed by. The Executive Committee (`GB-04`). A maturity score the business has not agreed is an architect's opinion with a number on it.

Done when. The scale is published, every level-1 row has an assessor and a date, and `Criticality` has been set by someone who trades.

Shows up in. `RPT03 Capability Heatmap`, whose `Gap` column is `Target` minus `Current`. Today it reads 1 or 2 on every row, which is the signature of generated data.

Goes wrong when. The assessment is run at level 3 because the sheet has a hundred rows. A hundred scores take a month and nobody reads them. Ten scores take two days and go to the board.

BEFORE YOU PUBLISH A MATURITY SCORE

What does level 3 mean, in one sentence with an observable test?

Which two level-1 capabilities did your assessors most disagree on, and why?

Who in the business will be asked to defend `Criticality`, and to whom?

Step 2.14 — Load drivers, goals, objectives and measures, and find the missing join

Produce. A motivation layer that joins end to end, from an external pressure to a number somebody is measured on.

You need. Steps 2.1 and 2.13. The last two sets of published results and the executive scorecards.

Look it up. Volume 1, chapter 6, *The four numbers a retail executive watches* — the four, and why a measure the executive does not already watch will not be maintained.

Where it lands. `O01 Driver`, `O02 Goal`, `O03 Objective`, `O04 Measure`, joined by `R08 Requirement derives from Dr`, `R10 Objective measured by Measu` and `R11 Goal realised by Objective`.

> **`Why no baseline in O04 Measure can be read`**
>
> Sixteen rows with `Measure`, `Unit`, `Domain`, `Baseline`, `Cadence`. Every `Baseline` is a two-digit number regardless of unit: `MSR-03 Unit cost to serve` 32 index, `MSR-09 Change lead time` 22 days, `MSR-13 Recovery objectives tested` 51 count. No column records when the baseline was taken or who took it. Treat all sixteen as unset.
>
> `O01 Driver` has the same problem in a different column: six regulatory drivers all owned by `Chief Compliance Officer` and six business drivers all owned by `Group Executive`, which resolves to no row in `O10 Actor` or `O11 Role` [9].

How.

1. Replace `O01 Driver` with your own pressures. The six regulatory rows are the right six for a South African retailer; the six business rows are generic.

2. Rebuild `O02` and `O03` from the business plan, then check `R11`: fourteen rows joining six goals to fourteen objectives, right in shape.
3. Re-baseline `O04`. Add `Baseline date` and `Baselined by`, and leave a measure out rather than invent a figure.
4. Check `R10`: every objective must reach a measure, and a measure nobody reports is one to retire.
5. **Build the join nothing in the file has.** There is no driver-to-goal relationship sheet: `R08` joins drivers to requirements, `R11` joins goals to objectives, so the motivation layer is two disconnected chains.
6. Draw the result end to end once, and show it to the executive whose number is at the end of it.

Signed by. The Chief Executive for goals, the Chief Financial Officer for measures.

Done when. Every goal reaches a driver through the sheet you added, every objective reaches a measure in `R10`, and every measure has a dated baseline or no baseline at all.

Shows up in. `RPT06 Requirements Traceability`, whose `Reaches a block` column is the test of whether `R08` and `R09` actually join up. All thirty-one requirements reach a block today, which is a diagonal, not an achievement.

Goes wrong when. You accept the two-digit baselines because they look like data. Every later benefit claim is then measured against a number nobody took.

Step 2.15 — Register the deliverables and close the phase against a baseline

Produce. A deliverable register that matches what you actually produced, and a second baseline.

You need. Everything above. An hour.

Where it lands. `O31 Deliverable`, then `04 Baselines`.

> **`O31 Deliverable` — twenty rows, four columns, and three things it lacks**
>
> `Ref` `DLV-01` to `DLV-20`. `Deliverable` — the TOGAF names. `Phase` — A, B, C, D, E, F, G, or RM. `Status` — Approved on thirteen rows, Draft on seven.
>
> No `Owner`, no `Due`, no `Approved by`, no `Version`, no link to a baseline. And the statuses do not survive a reading: `DLV-13 Consolidated Gaps` reads Approved against an `O29 Gap` holding one row, and `DLV-19 Compliance Assessment` reads Draft against a sheet holding none. There is also no row for the Request for Architecture Work, which chapter 3 adds as `DLV-21`.

How.

1. Add `Owner`, `Due`, `Approved by` and `Version` to `O31`.

2. Add rows for the Preliminary deliverables the register omits: the scope statement from 2.1, the tailoring decision from 2.5, the extension position from 2.6, the principle set from 2.9, the governance framework from 2.10 and 2.11, the maturity assessment from 2.13.
3. Reset `Status` from evidence. A deliverable is Approved when a body approved it on a date.
4. Cut `BL-002` in `04 Baselines` with `Status` = Current, set `BL-001` to Superseded, and produce the flat extract again by hand.
5. Read `ZZ Checks` and write the numbers beside the ones from step 2.4.

Signed by. The Architecture Board (`GB-01`), with the Executive Committee noting the phase closed.

Done when. `O31` holds a row per deliverable you produced, every Approved row names a body and a date, and `04 Baselines` has exactly one Current row.

Shows up in. `01 Version Control` , `Baselines Taken` = 2, and `RPT01 Dashboard` , where `Records` has grown only in the sheets step 2.5 put in scope.

Goes wrong when. The register is filled in from the TOGAF deliverable list rather than from what exists. Thirteen rows reading Approved is what that looks like, and it is already in the file.

Before you move to Phase A

Two of the fifteen steps above were not architecture at all: adding change control and cutting a baseline. They are here because without them the other thirteen cannot be shown to have happened. Everything else in this phase is reversible; those two are what make reversal visible.

THE PRELIMINARY PHASE — A REVIEWER'S QUESTIONS

For each format in scope, does the group master the price, the assortment and the stock position? Read your step 2.1 answers back and say which has changed.

How many rows are in `02 Change Log` today, and what was the last one?

Which of the twenty-four principles has the board refused to apply, and what happened next?

Does `059 Compliance Assessment` have rows in it? If not, what did your `G0` gate actually test?

Name the three `ZZ Checks` numbers you quoted in step 2.4 and what they read today.

THE CHAPTER IN TEN LINES

- Do step 2.4 first if you inherited the repository: `ZZ Checks` and `RPT13 Orphan Analysis` cost half a day and change the next three steps.
- Settle the formats and the ownership models before anything else. In a franchise or voluntary-trading estate the group masters neither the price, nor the assortment, nor the stock position for most of its turnover.
- `O12 Location` has no `Format` and no `Ownership` column, and its eight rows mix sites, formats and a cloud region with headcounts against all three.
- Add `01 Version Control`, `02 Change Log`, `03 Change Requests` and `04 Baselines` plus seven record-keeping columns before you edit a cell. Prefix your requests `RCR-`, not `CR-`.
- Four of the fifteen `ZZ Checks` lines do not survive a count: 842 records against 813, every type carrying records against an empty `O59`, three components past support against two dates, and nought vacant roles over a uniform `R21`.
- `EXT Industry Extension` declares six retail object types and holds no instances. Five are already shadowed in `O20 Data Entity`; `X06 Fulfilment Node` is shadowed nowhere.
- One `Owner` column name carries four conventions — a governance body, an organisation unit, a resolving role and a non-resolving one. Settle a `ROL-` reference and blank what does not resolve.
- `O05 Principle` has no `Tier`, `Rationale`, `Implications`, `Approved` or `Review date` column, and its `Draft` values fall on every fifth row. No relationship sheet reaches it, so twenty-four principles govern nothing.
- `O57` has no chair and no last-met date; `O58` has no column for who holds a gate; `O59 Compliance Assessment` and `R24` are both empty. Four gates described, none held.
- There is no maturity sheet and no published scale behind `O13`'s two hundred `Current` and `Target` values. There is also no driver-to-goal relationship sheet, so the motivation layer is two disconnected chains until you build one.

SOURCES FOR THIS CHAPTER

- **1.** Retail_TOGAF_Metamodel.xlsx, sheet O57 Governance Body
- **2.** Retail_TOGAF_Metamodel.xlsx, sheets O60 Governance Decision and O61 Governance Change Request
- **3.** Retail_TOGAF_Metamodel.xlsx, sheet ZZ Checks
- **4.** Retail_TOGAF_Metamodel.xlsx, sheets RPT13 Orphan Analysis and R04 Business Service supported
- **5.** Retail_TOGAF_Metamodel.xlsx, sheet EXT Industry Extension
- **6.** The Open Group, TOGAF Standard, 10th Edition — Preliminary Phase https://pubs.opengroup.org/architecture/togaf10-doc/adm/
- **7.** Retail_TOGAF_Metamodel.xlsx, sheets O09 Organization Unit, O11 Role, O20 Data Entity and O23 Application Component
- **8.** Retail_TOGAF_Metamodel.xlsx, sheets O13 Business Capability and RPT03 Capability Heatmap
- **9.** Retail_TOGAF_Metamodel.xlsx, sheets O01 Driver, O04 Measure, R08, R10 and R11

CHAPTER 3

Architecture Vision

Two documents and one signature. This repository holds all three Phase A deliverables marked approved, and no mandate behind any of them.

Phase A produces two documents and one signature. The repository holds all three Phase A deliverables marked `Approved` — `DLV-01 Architecture Vision`, `DLV-02 Statement of Architecture Work` and `DLV-03 Stakeholder Map` — and no row anywhere for the document that is supposed to have started them [1]. Twelve steps, three to five weeks, and most of the work is in the room rather than in the file [5].

Halverson Stores Group, the retailer in the repository, is invented. Where this chapter quotes its rows it is showing you a shape, not a benchmark.

> **BEFORE YOU START THIS PHASE**
>
> - **Volume 1, chapter 4, *The question you will be asked in week three*.** The question, and why a capability answer beats a system answer.
> - **Volume 1, chapter 5, *Who decides what, on one page*.** The decision rights, which are not the reporting lines.
> - **Volume 1, chapter 7, *The trading calendar*.** Read this before you book a single workshop.
> - **Volume 1, chapter 17, *The group in one page*.** A worked multi-format group, end to end, for the shape of a vision.

The shape of the phase

Steps	What they settle
3.1	The mandate, or the honest record that there is none
3.2 – 3.4	Who the stakeholders are, which ones the repository omits, and what each is measured on
3.5 – 3.7	The business model, the value streams, and the scope across formats
3.8 – 3.9	Constraints and assumptions, each with an owner and a date
3.10 – 3.12	The two documents, and the gate

Step 3.1 — Get a Request for Architecture Work, or write one and have it signed

Produce. A one-page request naming the sponsor, the business problem, the scope, the budget and the date by which an answer is needed.

You need. A sponsor. If you cannot name one, that is the finding and step 3.1 is where you record it.

Look it up. Volume 1, chapter 18, *What to refuse in month one* — what to decline, and how, when the mandate is thin.

Where it lands. `031 Deliverable`, which has no row for it.

> **`031 Deliverable` has no row for the document that starts the ADM**
>
> Twenty rows, `DLV-01` to `DLV-20`, with `Phase` values A, B, C, D, E, F, G and RM. The three Phase A rows all read `Approved`. There is no Request for Architecture Work and no Architecture Definition Document.
>
> Add `DLV-21 Request for Architecture Work` with `Phase` = A. Add the `Owner`, `Due` and `Approved by` columns from step 2.15 at the same time, because a request with no named approver is the thing this step exists to avoid.

How.

1. Write the request yourself. Waiting for the business to write one is how Phase A takes a quarter.
2. State the problem in the sponsor's language and in one sentence. "Availability is falling in convenience while stock cover is rising" is a problem. "The estate is fragmented" is not.
3. Name the formats and the ownership models in scope, from step 2.1.
4. Name the date the answer is needed, and check it against the trading calendar before you agree to it.
5. Take it to one person for signature, not a committee.
6. If nobody will sign, do one of three things rather than nothing: record a minute in `060 Governance Decision`, attach the work to a funded package in `030 Work Package`, or write the request, send it, and record the silence with a date.

Signed by. The sponsor — an executive who owns a number, not the Chief Information Officer by default.

Done when. `DLV-21` exists with an approver and a date, or a dated record exists of who was asked and declined.

Shows up in. `031 Deliverable`, twenty-one rows rather than twenty. And in `RPT01 Dashboard` not at all, which tells you how little the repository cares whether you have a mandate.

Goes wrong when. The architecture work proceeds on a verbal agreement from a meeting. Six months later the sponsor has moved and nothing in writing says the work was ever wanted.

Step 3.2 — Build the stakeholder map, and break the actor-role identity

Produce. A stakeholder map that distinguishes a party from a responsibility.

You need. Step 2.8's rebuilt `O11 Role`, and the rebuilt `O09` from step 2.1.

Look it up. Volume 1, chapter 5, *The decision map* — who holds which decision, which is not what the reporting lines say.

Where it lands. `O09 Organization Unit`, `O10 Actor`, `O11 Role`, joined by `R21 Role performed by Actor`.

> **`O10` and `O11` hold the same eighteen names**
>
> `O10 Actor` has `Ref`, `Actor`, `Type`, `Status`. `O11 Role` has `Ref`, `Role`, `Type`, `Performed by`. The eighteen names are identical in both, in the same order, and `R21 Role performed by Actor` joins `ROL-01` to `ACT-01` through `ROL-18` to `ACT-18`, one to one [2].
>
> Every `Status` in `O10` reads `Filled` and every `Performed by` in `O11` reads `Person`. That is why `ZZ Checks` reports nought vacant architecture roles.
>
> An actor is a party — a person, a team, an organisation. A role is a responsibility that a party is assigned to. One party performs several roles, and in a retailer that is the normal case: the same person is often the category buyer and the range signatory, and a store manager is the loss-prevention owner for that site. Keep the sheets separate and let `R21` carry many-to-many.

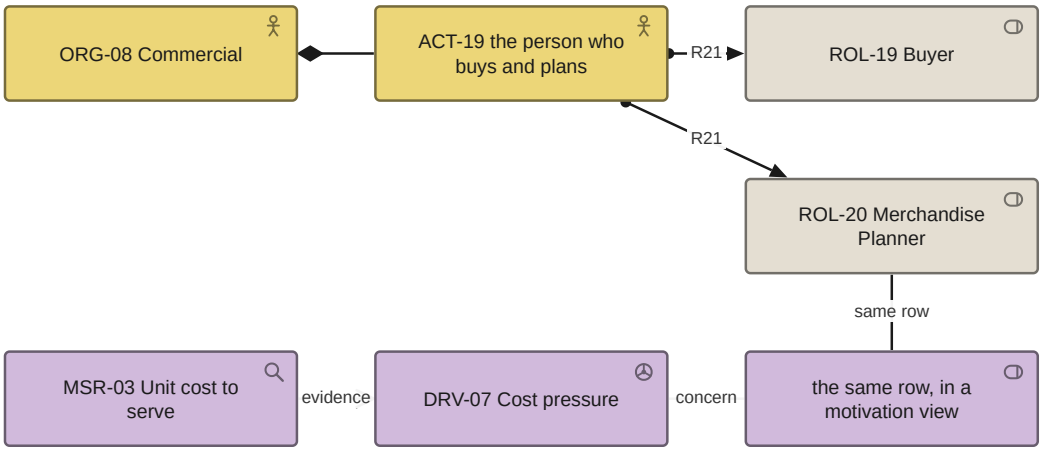


Figure 3.1 Three sheets, four ArchiMate 4 elements. An organisation unit is composed of actors, an actor is assigned to its roles (`R21`) — and in a retailer one actor performs several, which is the pattern the repository cannot record because `R21` joins eighteen roles to eighteen actors one to one. The same `O11 Role` row is drawn as a `Stakeholder` in a motivation view, where the concern it holds is a `Driver` and the evidence is a `Measure`. Halverson Stores Group is invented.

How.

1. Load `O09` from step 2.1 and check every `Head` resolves to a row in `O11`.

2. Rebuild `O10 Actor` as parties: named individuals where you have them, teams where you do not, and external parties — a franchisee association, a member guild, a major supplier — as actors too.
3. Rebuild `O11 Role` as responsibilities, using step 3.3's list.
4. Rebuild `R21` to carry the real pattern. Expect more roles than actors, and at least three actors performing two roles each.
5. Set `Status` in `O10` honestly. A vacancy is a finding, and `ZZ Checks` has a line waiting for it.
6. Draw the result once, as a motivation view, with each stakeholder's concern beside it.

Signed by. The Chief Enterprise Architect, with the sponsor confirming the first-group list.

Done when. `O10` and `O11` no longer hold the same list, `R21` is not one-to-one, and at least one `Status` reads something other than `Filled`.

Shows up in. `ZZ Checks`, `Vacant architecture roles`, which should stop reading zero. A nonzero number here is an improvement in the repository even though it is bad news about the organisation.

Goes wrong when. You keep the one-to-one join because it is tidy. Everything in Phase B that asks who performs a process then has one answer per role, and the retailer's real pattern — one buyer across four categories, one planner across two formats — cannot be recorded.

Step 3.3 — Add the six stakeholders a retailer's map always misses

Produce. Six role rows, each with a named concern, that no generic stakeholder list produces.

You need. Step 3.2. An hour with somebody who has worked in a shop.

Look it up. Volume 1, chapter 5, *Buying and merchandising* — why the buyer and the planner are two jobs, and what happens when the model assumes one.

Where it lands. `O11 Role`, `O10 Actor`, `R21 Role performed by Actor`.

How.

1. **The buyer and the merchandise planner, as two rows.** The buyer decides what is in the range; the planner decides whether the money allows it. If one person describes both jobs, the organisation is mis-designed, and your capability map will inherit the error. The repository's eighteen roles include neither.
2. **The space planner.** Owns the planogram, and therefore owns shelf capacity — which is an input to every replenishment calculation in the estate. Ask how often the planogram matches the fixture, then ask who measures it. Usually nobody.
3. **The responsible or superintendent pharmacist.** Personally accountable, in law, for what happens in the dispensary. Not a stakeholder you consult; a stakeholder who can stop you. Add the row even if the pharmacy format is one per cent of turnover.

4. **The franchise or member liaison.** The only person who can tell you what the agreement compels a franchisee or a member to share. Without that row, step 2.1's ownership decision has no owner.
5. **The supplier-income controller.** Owns the rebate and promotional-funding entitlement, which is reliably the largest unautomated process in a retailer and the one with the biggest audit exposure.
6. Add each as a `ROL-` row with a `Type`, join it in `R21`, and write its concern into the Architecture Vision — because no sheet in the repository has a column for a concern.

Signed by. The sponsor, who must agree these people get meetings.

Done when. All six rows exist in `O11 Role` with an actor against each in `R21`, and each has one sentence of concern recorded in the vision.

Shows up in. `O11 Role`, twenty-four rows rather than eighteen, and `R21` with a many-to-many shape. Nothing computes a stakeholder report; there is no `RPT` sheet for it.

Goes wrong when. The map is built from the organisation chart, so it contains eight executives and no buyer. The architecture then describes a retailer nobody who trades would recognise.

BEFORE YOU FIX THE STAKEHOLDER LIST

Who is the buyer and who is the planner, by name? If it is one person, what does that tell you?

Who can stop you in the dispensary, and have you met them?

Which stakeholder has a concern that no column in this repository can hold?

Form A-1 — the stakeholder and concern register

One row per stakeholder. The first four columns are the real columns of `011 Role` in sheet order and transcribe straight into it. `Concern` and `Decides or advises` have no column anywhere in the repository, so they go into the Architecture Vision. Twenty to thirty rows for a first pass. Signed by the Chief Enterprise Architect.

Ref	Role	Type	Performed by	Concern, one sentence	Decides or advises
ROL-19	Merchandise Planner	Commercial	Person	Example, delete me	Decides

Step 3.4 — Rank the stakeholders, and record what each one is measured on

Produce. A short list of the stakeholders whose agreement the vision needs, each with the number they are judged on.

You need. Step 3.3. The last two sets of published results and the executive scorecards if you can get them.

Look it up. Volume 1, chapter 6, *The four numbers a retail executive watches* — the four, and why anything else you offer them is noise.

Where it lands. Nowhere directly. There is no stakeholder-concern column anywhere in the repository; `001 Driver` has `Origin` and `Owner`, and that is the closest the model gets.

How.

1. Sort the list into three groups: decides, is consulted, is informed. Only the first group gets a meeting.
2. Against each name in the first group, write the measure they are judged on, and match it to an `MSR-` row where you can. Remember from step 2.14 that every `004 Measure` baseline is unset.
3. Where no `MSR-` row fits, add one rather than bending an existing measure to fit.
4. Check the match against `002 Goal`'s owners. Five of the six goals are owned by a C-level role that resolves in `011`; `GOL-03 Grow revenue from existing customers` is owned by `Chief Commercial Officer`, which does not [7].
5. Name, for each first-group stakeholder, the one thing the architecture will give them inside twelve months.
6. Put the list in the vision, not in a sheet.

Signed by. The sponsor.

Done when. Every first-group stakeholder has a measure and a twelve-month deliverable against their name.

Shows up in. `004 Measure`, where new rows appear for measures the executives actually watch, and `R10 Objective measured by Measu` once they are joined.

Goes wrong when. You rank by seniority. The person who can stop the work is often two levels down and owns a system nobody else understands.

Step 3.5 — Draw the business model canvas, and find the blocks with no home

Produce. One canvas per ownership model, and a written list of the blocks the repository cannot hold.

You need. Step 2.1, step 3.4, and two hours with the commercial director.

Look it up. Volume 1, chapter 4, *The canvas for a multi-format retail group* — the canvas for a group that runs several formats, and, under *What the canvas leaves out*, the parts it will not tell you.

Where it lands. Partly in `O18 Product` and `O19 Contract`; mostly nowhere, and that is the point of the step.

How.

1. Draw one canvas per ownership model in scope. A corporate format and a franchise format do not share a revenue model, a cost structure or a customer relationship.
2. Fill the blocks the repository can hold. `O18 Product` takes products — five rows, with `Ref`, `Product`, `Type`, `Description` and nothing about money. `O19 Contract` takes key partners, with `Counterparty`, `Type` and `Expiry`.
3. List the blocks with no home: customer segments, revenue streams, cost structure, channels, and anything about margin on a product row. Five of the nine.
4. Decide whether to extend `O18` with a line-of-business and margin column or to keep the canvas outside the repository. Either is defensible; leaving it undecided is not.
5. Reconcile the canvas against `O14 Value Stream`. A revenue stream with no value stream behind it is either a gap or a canvas that flatters the business.
6. Put the canvas in the vision as a figure.

Signed by. The Chief Commercial Officer, or whoever owns trading.

Done when. One canvas exists per ownership model, and the blocks with no home are written down with a decision against each.

Shows up in. `O18 Product` and `O19 Contract` if you extended them. Otherwise nothing, which is the honest answer.

Goes wrong when. One canvas is drawn for the group. A group running corporate and franchise formats has two business models, and a single canvas will describe the larger one and hide the riskier one.

Step 3.6 — Name the value streams, and reconcile them against the relationship sheet

Produce. Five to eight value streams, each with a triggering stakeholder, stages that match the process sheet, and capabilities that match the relationship sheet.

You need. Steps 3.3 and 3.5.

Look it up. Volume 1, chapter 7, *The value chain is six clocks, not six steps* — why a retailer's streams run at different speeds and cannot be drawn as one sequence.

Where it lands. `O14 Value Stream`, `O16 Business Process`, `R02 Value stream uses Capabilit`.

`O14` and `R02` disagree, and the relationship sheet wins

`O14 Value Stream` has `Ref`, `Value stream`, `Scope`, `Stages` and an inline `Capabilities` column. Five rows: `VS01 Plan to Range`, `VS02 Source to Shelf`, `VS03 Browse to Buy`, `VS04 Order to Doorstep`, `VS05 Return to Resolution`. The names are good and the five are the right five to start from.

Two things do not hold. **`Stages` declares 4, 5, 6, 4 and 5 — twenty-four in all — while `O16 Business Process` holds exactly four rows per stream, twenty in total.** `VS03 Browse to Buy` claims six stages and has four processes behind it.

And the inline `Capabilities` column contradicts `R02`. For `VS01` to `VS03` they agree. For `VS04 Order to Doorstep` the inline column reads `C01, C02, C03` while `R02` names `C04`, `C05`, `C06`; for `VS05` the inline column reads `C02, C03, C04` against `R02`'s `C05`, `C06`, `C07` [3]. Look at `R02` as a whole and the reason is plain: it is a three-wide sliding window, `VS01` to `C01`–`C03`, `VS02` to `C02`–`C04`, down to `VS05` to `C05`–`C07`. Neither column was derived from a conversation. The reports read `R02`, so `R02` is the one to rebuild — and then make the inline column a formula or delete it.

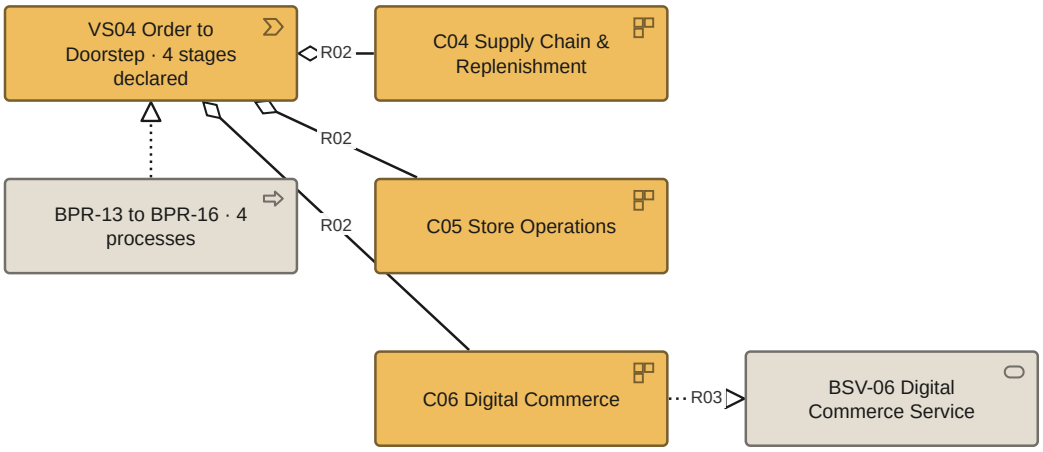


Figure 3.2 `VS04 Order to Doorstep` as `R02 Value stream uses Capabilit` joins it. The stream declares four stages in `O14` and has four processes behind it in `O16`, one of only two streams that reconcile — and its inline `Capabilities` column names `C01`, `C02` and `C03` instead of the three above. `R02` is a three-wide sliding window across all five streams. The reports read `R02`.

How.

1. Name the streams with the business, in the business's words. Keep the five if they survive the room.
2. For each, name the triggering stakeholder from step 3.3. A stream with no trigger is a function.
3. Count the stages with the business and set `Stages` from that count, not from the sheet.
4. Rebuild `R02` from the same conversation. Expect four to seven capabilities per stream and expect overlap.

5. Delete the inline `Capabilities` column or make it compute from `R02`. Two sources of one fact is the defect, not the mismatch.
6. Add `O16` rows so that the process count per stream equals the declared stage count.

Signed by. The Chief Operating Officer, with the commercial director for `VS01` and `VS02`.

Done when. Every stream's `Stages` equals its `O16` row count, and `R02` and the inline column agree or the inline column is gone.

Shows up in. `RPT03 Capability Heatmap` indirectly — a capability no value stream uses is a capability to question — and in `R02`, which should stop being a diagonal.

Goes wrong when. You accept the five streams and the fifteen `R02` rows because they look complete. The sliding window produces a plausible map in which returns do not touch store operations and fulfilment does not touch merchandising.

SETTING THE VALUE STREAMS AND THE SCOPE

Which value stream does your sponsor's problem live in, and who triggers it?

Which capability appears in no value stream once you have rebuilt `R02`?

Which format is in scope for the vision but out of scope for the first increment, and why?

Step 3.7 — Set the scope in four dimensions, and write the exclusions down

Produce. A scope statement in four dimensions, with the exclusions as sentences carrying reasons.

You need. Steps 2.1, 3.5 and 3.6.

Look it up. Volume 1, chapter 2, *What the format mix does to the application estate* — why the format mix, not the group size, decides how many systems you are looking at.

Where it lands. The Statement of Architecture Work in step 3.11. Partly recoverable from `O09`, `O12`, `O18`, `O13` and `00 Index`'s domain column, and nowhere as a scope record.

How.

1. **Breadth.** Which formats, banners, ownership models, legal entities and products. From step 2.1, and it is the dimension this industry gets wrong.
2. **Depth.** How far down `013 Business Capability` you model. Three levels exist; level 2 and level 3 are templated — every level-2 row reads "X within Y" and every level-3 row is a Define or an Operate pair. Decide to rebuild levels 2 and 3 or to work at level 1 only, and say which.
3. **Horizon.** One date. `003 Objective`'s `Horizon` column says 12 months, 18 months, 2 years and 3 years; `066 Architecture Increment` lands increments at months 10, 16, 22, 30, 42 and 56 [8]. Those cannot both be the target, and month 56 is not a horizon anybody will hold you to.
4. **Domains.** Which of Business, Data, Application and Technology are in this cycle. All four is the default and it is usually wrong for a first cycle.
5. Write each exclusion as a sentence with a reason and a date to revisit.
6. Read the four dimensions back to the sponsor and get the exclusions acknowledged, not just the inclusions.

Signed by. The sponsor, with the Investment Committee (`GB-02`) where funding follows scope.

Done when. All four dimensions have one answer each, every exclusion has a reason, and the horizon is a single date.

Shows up in. `RPT03 Capability Heatmap` for depth, and `066 Architecture Increment` for horizon once chapter 9 rebuilds it.

Goes wrong when. Scope is set by breadth alone. "The whole group" with no depth, horizon or domain answer is not a scope; it is an undertaking.

Step 3.8 — Record the constraints, with an owner and an expiry

Produce. A constraint list where every row has somebody who can lift it and a date on which it stops being true.

You need. Step 3.7.

Look it up. Volume 1, chapter 7, *The trading calendar* — the freeze windows, and why the calendar is an architectural object rather than a reporting convention.

Where it lands. `007 Constraint`.

> **`007 Constraint` — eight rows, four columns, and no way to tell a live constraint from a stale one**
>
> `Ref`, `Constraint`, `Description`, `Type`. The `Type` values are Financial, Resource, External, Regulatory, Operational and Organisational, and the eight rows are plausible: `CON-01 Flat technology budget`, `CON-04 Regulatory change in flight`, `CON-05 Data residency`, `CON-06 Change freeze windows` with the description "Peak periods are closed to change", `CON-08 Union and works council consultation`.
>
> There is no `Owner`, no `Raised`, no `Expires` and no `Source` [4]. No relationship sheet reaches the sheet at all, so no requirement, work package or design is ever shown to be constrained by any of it. A constraint with no owner cannot be lifted and a constraint with no date cannot expire, so the list only ever grows.

How.

1. Add `Owner`, `Source`, `Raised` and `Expires` before you add a row.
2. Walk the eight shipped rows. Keep the ones that are true of your retailer, and `CON-06` almost certainly is.
3. Turn `CON-06 Change freeze windows` into dates. A freeze window is a constraint only when somebody has written down which weeks it covers.
4. Against each kept row, name the role that can lift it. If nobody can, it is not a constraint, it is a fact — move it to `008 Assumption` or to the vision.
5. Distinguish a constraint from a requirement. `CON-05 Data residency` is a requirement with a regulator behind it; the constraint is the platform choice it removes.
6. Review the list at every gate and close what has expired.

Signed by. The Chief Enterprise Architect, with each constraint's named owner acknowledging it.

Done when. Every row has an owner, a source and either an expiry date or a written reason for having none.

Shows up in. Nothing. No report reads `007` and no relationship sheet reaches it, so the constraint list is only as useful as the gate that reads it aloud.

Goes wrong when. The freeze window is recorded as a constraint and not as dates. Phase A then schedules its own workshops inside it, which is the failure the next step and step 3.12 are both about.

Step 3.9 — Test the assumptions, and give each one a falsification date

Produce. An assumption list where every row says what would prove it false and when you will check.

You need. Step 3.8.

Look it up. Volume 1, chapter 12, *First question: who owns the store?* — the assumption that breaks the most application landscapes.

Where it lands. `O08 Assumption`.

> **`O08 Assumption` — three columns, six rows, and no date anywhere**
>
> `Ref`, `Assumption`, `Status`. That is all. `Status` reads `Open` on four rows and `At risk` on two — `ASM-03 Vendor selections complete before the dependent phase` and `ASM-04 Delivery capacity can be staffed to plan`.
>
> No owner, no test, no date, no consequence. An assumption without a falsification test is an opinion, and an assumption at risk with no date is a risk that has not been raised. Add `Owner`, `Test`, `Check by` and `If false`, and move anything that is really a risk into `O62 Delivery Risk`, which has `Likelihood`, `Impact` and `Owner` columns waiting.

How.

1. Add the four columns.
2. For each assumption, write the observation that would prove it false. "Executive sponsorship holds" is tested by whether the sponsor attended the last two gates.
3. Set `Check by` to a date inside this phase or the next. An assumption nobody rechecks becomes a fact by neglect.
4. Add the retail-specific one the list omits: that the group can obtain point-of-sale and inventory data from its franchised or member stores at all, in a usable format, within an agreed lag.
5. Move the two `At risk` rows into `O62 Delivery Risk` with a likelihood and an impact, and leave a pointer behind.
6. Read the list at the Phase A gate and record which assumptions moved.

Signed by. The Chief Enterprise Architect.

Done when. Every row has a test, a check-by date and a named owner, and nothing reads `At risk` without a corresponding row in `O62`.

Shows up in. `O62 Delivery Risk`, which should gain rows from this step. Eight rows today, none of them traceable to an assumption.

Goes wrong when. The assumption that the group masters the shelf price survives into Phase C. Everything about pricing then gets designed for a format that does not exist in half the estate.

CONSTRAINTS AND ASSUMPTIONS

Which constraint has no owner who can lift it, and what does that make it?

What observation would prove your biggest assumption false, and when will you look?

Which weeks of next year are closed to change, by date?

Step 3.10 — Write the Architecture Vision

Produce. A short document the sponsor can read in ten minutes and repeat in one.

You need. Steps 3.1 to 3.9.

Look it up. Volume 1, chapter 17, *The motivation view: why this group is changing* — a worked motivation view for a multi-format group, which is the spine of a vision.

Where it lands. `DLV-01 Architecture Vision` in `O31 Deliverable`, which already reads `Approved` and should not until you have written it.

How.

1. Open with the sponsor's problem in the sponsor's sentence, from step 3.1.
2. Put the motivation chain next: the driver, the goal, the objective and the measure, as one figure. Use the join you built in step 2.14, because the file has none.
3. Put the business model canvas and the value streams in as figures, not as prose.
4. State the target in capability terms, at level 1, with `Current` and `Target` from step 2.13 and the scale printed beside them.
5. State what is out of scope, in the sponsor's words, with the reasons from step 3.7.
6. Reset `DLV-01`'s `Status` to Draft until a body approves it on a date.

Signed by. The sponsor, then noted by the Executive Committee (`GB-04`).

Done when. The sponsor can restate the vision in one sentence without the document in front of them.

Shows up in. `O31 Deliverable`, `DLV-01` with an approver and a date in the columns you added in step 2.15.

Goes wrong when. The vision is written in architecture language. A document that opens with the metamodel has already lost the only reader who matters.

Step 3.11 — Write the Statement of Architecture Work

Produce. The contract between the architecture function and the sponsor: what will be produced, by when, by whom, and what it will cost.

You need. Steps 3.7 and 3.10.

Look it up. Volume 1, chapter 18, *The three artefacts that buy you credibility* — what to produce first, and why those three.

Where it lands. `DLV-02 Statement of Architecture Work`, and the deliverable rows it promises.

How.

1. List the deliverables by `DLV-` reference. If a deliverable you are promising has no row, add it.
2. Put the four scope dimensions from step 3.7 in verbatim, exclusions included.
3. Name the roles doing the work, from `011 Role`, and the effort against each. Five architecture roles exist in the file; say which are filled.
4. Name the governance route: which body approves which deliverable, at which gate, from steps 2.10 and 2.11.
5. Put the dates against the trading calendar. A deliverable due in the peak window is a deliverable due in January.
6. Have it signed, and cut a baseline in `04 Baselines` the same day.

Signed by. The sponsor and the Investment Committee (`GB-02`), which `057` says holds `G0` funding.

Done when. Every promised deliverable has a `DLV-` row, a date outside a freeze window, and a named approving body.

Shows up in. `031 Deliverable`, where the rows you promised exist with owners and dates, and `01 Version Control`, `Baselines Taken` = 3.

Goes wrong when. The statement promises an Architecture Definition Document and the register has no row for one. Nobody notices until Phase B closes and the deliverable cannot be marked complete.

BEFORE THE GATE

What is the one sentence the sponsor will use to describe this work to their peers?

Which promised deliverable has no `DLV-` row today?

Which of your Phase A dates falls inside a freeze window or a range-review week?

Step 3.12 — Hold the Phase A gate inside the trading calendar

Produce. A held `G0 Charter` gate with a recorded verdict and evidence.

You need. Steps 2.11, 3.10 and 3.11.

Look it up. Volume 1, chapter 5, *The trading calendar governs every diary in the building* — the clock every role in a retailer works to, and the one an architect from another sector underweights.

Where it lands. `059 Compliance Assessment`, which holds no rows, and `R24 Compliance Assessment tests`, which holds none either.

> **THE TRADING CALENDAR DECIDES WHEN YOU CAN GET ANYONE IN A ROOM**
>
> This is the scheduling constraint that has no equivalent in banking or insurance, and it governs Phase A more than any governance decision does. For roughly eight weeks around peak trading, nobody in buying, planning, supply chain or stores will attend a workshop, and `CON-06 Change freeze windows` already records that peak periods are closed to change. For the weeks around each range review, the buyers and planners are unavailable for a different reason: they are doing the thing your architecture is about.
>
> The consequence is practical. Phase A has two or three usable windows a year, not twelve. Book the stakeholder workshops from steps 3.3 and 3.6 into one of them, hold the gate at the end of it, and set the Statement of Architecture Work's dates from the calendar rather than from a project plan. An architect who schedules a capability workshop in the week before peak will be told, politely, to come back in February — and will have lost a quarter.

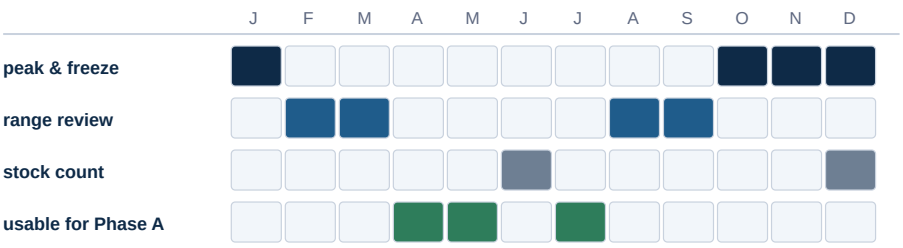


Figure 3.3 The scheduling constraint that governs Phase A in a retailer. Peak and the weeks around each range review take the people you need out of the room, and `CON-06 Change freeze windows` already records that peak is closed to change. Set the Statement of Architecture Work's dates from this, not from a project plan.

How.

1. Lay the trading calendar out first: peak windows, range-review weeks, the stock-count weeks, and the financial year end, which in retail is rarely December.
2. Mark the two or three windows in which the first group of stakeholders is actually available.
3. Schedule the gate at the end of a window, not at a month end.
4. Publish the gate criteria before the submission, per step 2.11.
5. Hold the gate and write the verdict into `O59 Compliance Assessment` with the prefix you chose in chapter 1, plus the join in `R24`.
6. Record the decision in `O60 Governance Decision` with a date, including a refusal.

Signed by. The Investment Committee (`GB-02`) for `G0`, as `O57 Governance Body` states in prose [6].

Done when. `O59` holds at least one row, `O60` holds a dated decision, and every Phase B date sits outside a freeze window.

Shows up in. `O59 Compliance Assessment` and `R24`. If both are still empty after your gate, the gate was a meeting.

Goes wrong when. The gate is held on the date the programme plan wanted. Half the people who could have refused it were in a range review, so it passed by default — which is the worst outcome a gate can produce.

Before you move to Phase B

Phase A is the only phase whose output is entirely prose. Two documents, a stakeholder list, four scope answers and a signature, and almost none of it lands in a sheet — which is exactly why the repository can hold three `Approved` Phase A deliverables and no mandate behind any of them.

PHASE A — A REVIEWER'S QUESTIONS

Who signed the Request for Architecture Work, and what would they say this work is for, in their own words?

Of the six stakeholders from step 3.3, which have you met and which declined?

Does `059 Compliance Assessment` now have rows in it? If not, what did your gate actually test?

Your horizon is one date. Name three things in `023 Application Component` that will be out of vendor support before it.

Which two of your Phase B workshop dates will you have to move once somebody shows you the trading calendar?

THE CHAPTER IN TEN LINES

- The repository marks all three Phase A deliverables `Approved` and holds no Request for Architecture Work row at all. Add `DLV-21`.
- When nobody will sign, take a minute in `O60 Governance Decision`, attach the work to a funded package in `O30`, or record the silence with a date. Not nothing.
- `O10 Actor` and `O11 Role` hold the same eighteen names and `R21` joins them one to one, with every `Status` reading `Filled`. An actor is a party; a role is a responsibility, and one party performs several.
- Six stakeholders a retailer's map always misses: the buyer and the merchandise planner as two people, the space planner, the responsible or superintendent pharmacist, the franchise or member liaison, and the supplier-income controller. None is in `O11`.
- No sheet in the repository has a column for a stakeholder's concern. It goes in the vision, and into `O01 Driver` where it is a real pressure.
- Five of the nine business model canvas blocks have no home in the file: customer segments, revenue streams, cost structure, channels, and anything about margin on a product row.
- `O14 Value Stream` declares twenty-four stages against `O16`'s twenty processes, and its inline `Capabilities` column contradicts `R02` on `VS04` and `VS05`. `R02` is a sliding window. Rebuild it from a conversation.
- Set scope in four dimensions and write the exclusions as sentences with reasons. `O03`'s three-year horizons and `O66`'s month-56 increment cannot both be the target.
- `O07 Constraint` has no owner, no source and no expiry; `O08 Assumption` has three columns and no date. Add the columns before you add rows, and turn `CON-06` into actual weeks.
- The trading calendar, not the governance calendar, decides when Phase A can happen. Two or three usable windows a year, and a gate held on the programme plan's date is a gate half the people who could refuse it did not attend.

SOURCES FOR THIS CHAPTER

- **1.** Retail_TOGAF_Metamodel.xlsx, sheet O31 Deliverable
- **2.** Retail_TOGAF_Metamodel.xlsx, sheets O10 Actor, O11 Role and R21 Role performed by Actor
- **3.** Retail_TOGAF_Metamodel.xlsx, sheets O14 Value Stream, O16 Business Process and R02 Value stream uses Capabilit
- **4.** Retail_TOGAF_Metamodel.xlsx, sheets O07 Constraint, O08 Assumption and O62 Delivery Risk
- **5.** The Open Group, TOGAF Standard, 10th Edition — Phase A, Architecture Vision https://pubs.opengroup.org/architecture/togaf10-doc/adm/
- **6.** Retail_TOGAF_Metamodel.xlsx, sheets O57 Governance Body, O58 Governance Gate, O59 Compliance Assessment and R24
- **7.** Retail_TOGAF_Metamodel.xlsx, sheets O02 Goal, O04 Measure and O11 Role
- **8.** Retail_TOGAF_Metamodel.xlsx, sheets O03 Objective and O66 Architecture Increment

CHAPTER 4

Business Architecture

The phase that decides whether anybody believes the rest of the work.

Two sheets decide everything downstream — `O13 Business Capability` and `O16 Business Process` — and both arrive already filled, which is harder to deal with than empty [1]. A third set of objects decides whether the model is about retail at all, and the TOGAF core has nowhere to put them: `X01 SKU / Article`, `X02 Store / Location` and `X03 Assortment` on `EXT Industry Extension` [3].

Halverson Stores Group, the retailer in the repository, is an invention. So is Milkwood Retail Group, the retailer in volume 1. Their numbers do not mix.

> **BEFORE YOU OPEN A SHEET**
>
> Most of volume 1 is business-layer teaching, so this is where it saves the most time.
>
> - **Volume 1, chapter 2, *The ownership models, and what each does to the architecture*.** Why a corporate store, a franchise and a voluntary-trading member are three architectures.
> - **Volume 1, chapter 5, *The decision map* and *The distinction outsiders always collapse*.** What each role decides, and the buyer-versus-planner split.
> - **Volume 1, chapter 7, *The value chain is six clocks, not six steps*.** Why a retail value stream is a cadence, not a sequence.
> - **Volume 1, chapter 8, *The L1 process map*.** Twelve L1 processes, each with a trigger, an outcome and an owner.
> - **Volume 1, chapter 19, *Business capabilities*, *Roles* and *Business processes*.** The pre-drafted catalogues, with their own local numbering.

What you are filling, and in what order

Phase B here is eleven object sheets, three industry-extension types and seven relationship sheets. The objects are `O09 Organization Unit`, `O10 Actor`, `O11 Role`, `O12 Location`, `O13 Business Capability`, `O14 Value Stream`, `O15 Business Service`, `O16 Business Process`, `O17 Business Function`, `O18 Product` and `O19 Contract`. The extension types are `X01 SKU / Article`, `X02 Store / Location` and `X03 Assortment`. The relationships are `R01 Capability decomposition`, `R02 Value stream uses Capabilit`, `R03 Capability realised by Busi`, `R04 Business Service supported `, `R21 Role performed by Actor`, `R22 Org Unit owns Capability` and `R25 Application supports Produc` [1].

Several of those tab names are cut off. Excel truncates a sheet name at thirty-one characters, so `R02 Value stream uses Capabilit` really is missing its final `y` and `R04 Business Service supported`

really does end in a space. Spell them the way the tab spells them or your `VLOOKUP` will fail and you will blame the data.

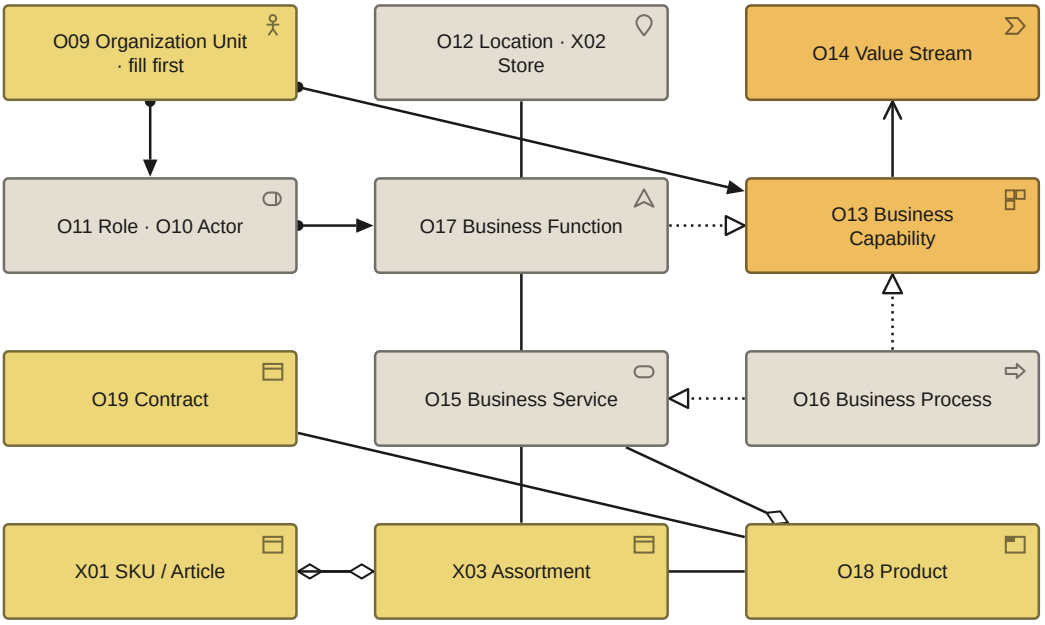


Figure 4.1 The Phase B sheets as one ArchiMate 4 business-layer view, carrying the relationships the seven `R` sheets actually hold, with the three industry-extension objects the TOGAF core has no element for. Nothing on it stands alone, which is why the fill order is not negotiable.

The order is not decoration. Skip `O09` and nothing downstream has an owner. Skip `O14` and the capability map has nothing to test it. Skip `X02` and every location-dependent number — price, range, stock, labour — is averaged across formats that do not resemble each other.

> **THE ONE RULE THAT HOLDS FOR EVERY SHEET IN THIS CHAPTER**
>
> Fill a relationship sheet in the same sitting as the second of its two endpoint sheets. A relationship backlog is not a backlog — it is a decision nobody made. Here `O14`'s `Capabilities` column and `R02` were filled at different times by different logic and now disagree about two of five value streams. Nobody noticed, because nothing reads both.

The steps

Step 4.1 — Fix the Phase B boundary before you open a sheet

Produce. One page: which value streams, which formats and which capability levels are in scope, and what "out of scope" means operationally.

You need. The approved Statement of Architecture Work (`O31 Deliverable`, `DLV-02`) and twenty minutes with the sponsor.

Where it lands. Not in a sheet. `031 Deliverable` carries `DLV-04 Business Architecture`, `DLV-05 Capability Map` and `DLV-06 Value Stream Definition`, all three already marked `Approved`. The scope page goes into `DLV-04`.

How.

1. Name the value streams in scope from the five rows of `014 Value Stream`. Three is a defensible iteration; five is a quarter.
2. Name the formats in scope. A group with a supermarket estate, a pharmacy chain and an apparel business has three Phase B results below level 2, not one.
3. Decide the level you will score. `013` holds 100 rows across three levels; scoring level 3 honestly is sixty conversations.
4. Write the exclusions as consequences — no workshop, no score, no gap raised — and make the sponsor say them aloud first.

Signed by. The sponsor named on the Statement of Architecture Work, countersigned by the Chief Enterprise Architect (`ROL-08`, `GB-05 Architecture Function` on `057 Governance Body`).

Done when. Every row of `014` is marked in or out, the format list is explicit, and the capability level is a number on a page somebody signed.

Shows up in. Nowhere in the thirteen reports. Scope is the one Phase B artefact this repository cannot hold.

Goes wrong when. Scope is set by capability count rather than by value stream and format. "The whole map" is 100 rows at three levels across three formats, which is a year.

FIXING THE PHASE B BOUNDARY

Which value streams are in this iteration, and what makes them the ones?

Which formats does this iteration cover, and which are deferred to a separate pass?

For each exclusion, what is the operational consequence, and who said it aloud?

Step 4.2 — Load the organisation and the actors, and make the org carry format and ownership model

Produce. `O09 Organization Unit` with one head and one headcount per unit; `O12 Location` recording the format and the stock owner of every trading location; `O10 Actor` including the parties the group does not employ.

You need. The group structure, the store list from Property, the franchise or membership schedule from Legal, and the supplier master from Buying.

Look it up. Volume 1, chapter 2, *The ownership models, and what each does to the architecture*; chapter 7, *Who owns the shop*; chapter 5, *Your real counterparties*.

Where it lands.

> **`O09 Organization Unit` · `O12 Location` · `O10 Actor`**
>
> `O09` holds `Ref`, `Organisation unit`, `Type`, `Head`, `Headcount`. `Type` takes `Group`, `Corporate`, `Line` and `Assurance`, and the gap between `Corporate` and `Line` tells you whether a unit can be told what to do. There is **no `Parent` column**. `O12` holds `Ref`, `Location`, `Type`, `Scale` — and **no `Format`, no `Ownership model`, no `Trading area`, no count and no province.** `O10` holds `Ref`, `Actor`, `Type`, `Status`, with **no `Internal/External` and no `Population`.** Add all six columns before you load a row.

How.

1. Add `Parent` to `O09` and load top down, so it never points at a blank.
2. Add `Format` to `O12` from the economics, not the square metreage: supermarket, convenience, hypermarket, pharmacy, apparel, cash-and-carry, forecourt, online.
3. Add `Ownership model` with at least `Corporate`, `Franchise` and `Voluntary member`, per location rather than per format. One format usually carries two of the three.
4. Replace the four shipped trading rows. `LOC-03 Metro Store Format` and `LOC-04 Convenience Format` are formats masquerading as locations, and `LOC-08 Cloud Region` carries 293 people.
5. Load the external actors. All eighteen `ACT-` rows are employees: no Customer, no Supplier, no Franchisee, no Landlord, no National Consumer Commission, no Information Regulator, no medical scheme, no acquiring bank. Set `Population` to the store or member count where it applies.

Signed by. Chief People Officer (`ORG-04`) for the structure; Chief Operating Officer (`ORG-07`) for the locations; General Counsel (`ORG-12`) for the ownership model, which is a contractual fact.

Done when. Every `Parent` resolves, every trading location carries a format and an ownership model, and `O10` holds at least six parties the group does not employ.

Shows up in. Nothing reads `O09` except `R22 Org Unit owns Capability`, and nothing reads `O12` except `R20 Application located at Loca`.

Goes wrong when. The ownership model is left out, and every price, range and stock statement is then quietly false for the part of the estate the group does not own. In a voluntary-trading group the members own their stores and may buy elsewhere; in a franchise the franchisee owns the stock. For much of the turnover the group has a recommendation rather than a price and an advisory range rather than an assortment [6]. It belongs on the first page of `DLV-04`.

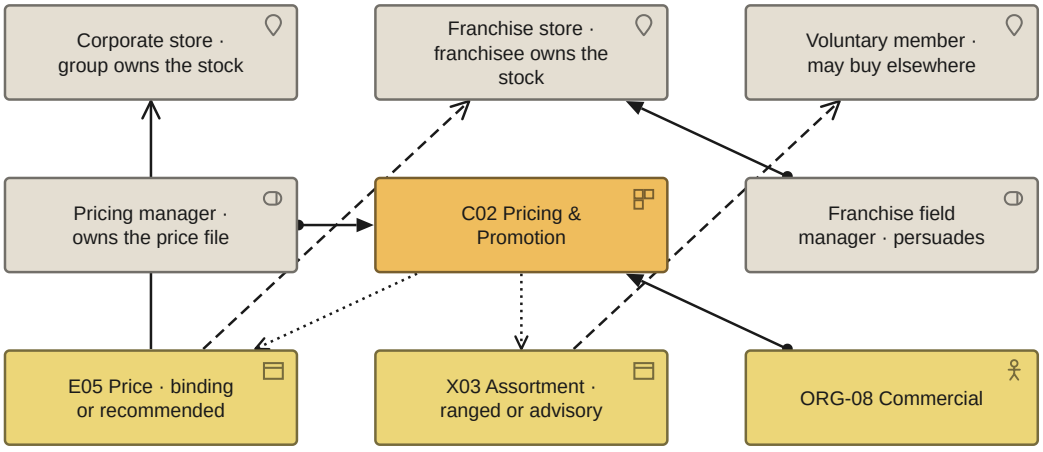


Figure 4.2 The ownership model as architecture. The group sets a price for a corporate store and recommends one everywhere else, so the same `X01` and `X03` records mean three different things depending on who owns the stock. The repository has no column for any of it.

Step 4.3 — Build the role catalogue the retailer actually staffs

Produce. A role catalogue where every row is a position whose authority differs from its neighbour's, and `R21 Role performed by Actor` showing who holds what.

You need. `O10` complete. Volume 1 appendix A drafts a retail role set as `R-01` to `R-33` — load it and argue with it.

Look it up. Volume 1, chapter 19, *Roles*, for the thirty-three drafted roles, and chapter 5, *Who decides what, on one page*.

Where it lands.

> **`O11 Role`**
>
> `Ref`, `Role`, `Type`, `Performed by`. `Type` groups for reporting — `Executive`, `Architecture`, `Delivery`, `Operations`, `Data`, `Security`, `Assurance` — and you will add retail types: `Buying`, `Planning`, `Space`, `Store`, `Supply chain`, `Pricing`, `Pharmacy`, `Loss prevention`. `Performed by` holds a kind, not a reference: all eighteen rows read `Person`.

How.

1. Load `R-01` … `R-33` as candidates and **map by name, never by number.** Thirty-three drafted roles against eighteen `ROL-` rows is not a renumbering, and `R-21 Responsible pharmacist` in volume 1 has nothing to do with `R21 Role performed by Actor` in the repository. That collision of two `R-` schemes has caught people already.
2. Add every position the repository missed. Halverson's eighteen roles are all executive, architecture, delivery or assurance: no Buyer, no Merchandise Planner, no Allocator, no Space Planner, no Store Manager, no Cashier, no Front-end Supervisor, no Pricing Manager, no Responsible Pharmacist. A retailer's role catalogue with no retail roles is the clearest sign of a template fill.
3. Apply the one test: does this row's authority differ from the row above it? `R-17 Front-end supervisor` is a role because it authorises overrides, voids and refunds.
4. Set `Performed by` to `Organisation` where a franchisee, a member, a concession or an outsourced desk performs the role, and fill `R21` in the same sitting.

Signed by. Each role's accountable executive — buying and planning by the Chief Commercial Officer (`ORG-08`), store roles by the Chief Operating Officer (`ORG-07`), pharmacy roles by the responsible pharmacist. Never by the architect.

Done when. Every row is a position with distinct authority, and the count of roles no longer equals the count of actors.

Shows up in. Nowhere. `O11` feeds no report, and an unreported sheet rots quietly.

Goes wrong when. Actor and role collapse into one list. `ACT-01 Chief Executive` and `ROL-01 Chief Executive`, eighteen rows each, identical names, identical order, joined one-for-one by `R21`. The model loses the only thing the distinction is for — that one person holds several roles, and one role can be held by a franchisee's staff.

Form B-1 — Role catalogue, the retail roles the repository is missing

One row per position whose authority differs. Columns are the real `O11 Role` columns in sheet order. A first pass for a multi-format group needs thirty to forty rows. Signed by each role's accountable executive. The full catalogue is in appendix F.

Ref	Role	Type	Performed by
ROL-19	Buyer / category manager — example, delete me	Buying	Person

Continue on a second copy of this form.

Step 4.4 — Separate the buyer from the planner, and record the difference as authority

Produce. Four role rows an outsider would have written as one: buyer, merchandise planner, range planner and allocator, each with the decision it owns and the number it answers to.

You need. Step 4.3's catalogue, the open-to-buy policy from Finance, the range review calendar from Merchandising.

Look it up. Volume 1, chapter 5, *The distinction outsiders always collapse*, and chapter 6, *Open-to-buy, the control that actually governs the buying organisation*.

Where it lands. Four rows on `O11 Role`. Put the buyer under `Buying` and the three planners under `Planning`; the point of the split is lost if they share a grouping.

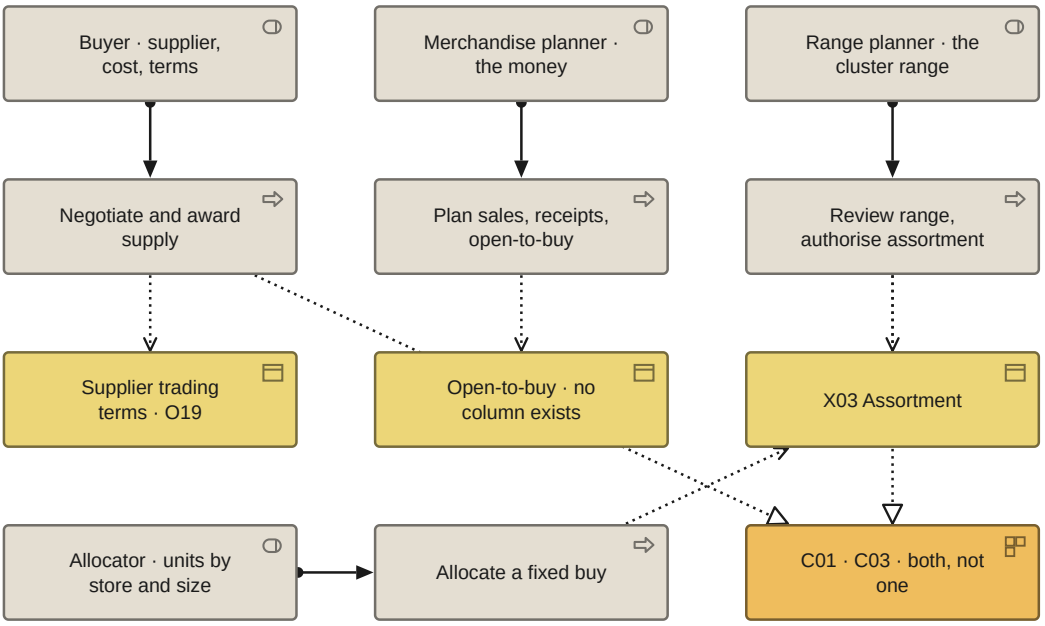


Figure 4.3 Four roles an architect arriving from outside retail usually writes as one. Each owns a different decision over a different object, and each answers to a different number. Collapsing them is the commonest Phase B error in this industry.

How.

1. The **buyer** picks the supplier, the range, the cost price and the trading terms, and is held to margin and supplier income.
2. The **merchandise planner** plans sales and receipts *in money* and owns the open-to-buy, and is held to stock turn and markdown.
3. The **range planner** decides the assortment per cluster, including the tail, and is held to availability and space productivity.
4. The **allocator** decides which store gets which units in which sizes, and is held to size-curve accuracy and in-store availability.
5. Record the collision in `DLV-04`: the buyer and the merchandise planner can both block a purchase order, for different reasons, and the system must let both.

Signed by. Chief Commercial Officer (`ORG-08`).

Done when. Four rows exist, each with one sentence naming the decision it owns and one naming the measure it answers to, and no two sentences are interchangeable.

Shows up in. Nothing automated. Read the four sentences to a trading director and see whether the right name comes back for each.

Goes wrong when. They collapse into one "Category Manager" row. The open-to-buy then has no owner, the assortment has no owner, and every capability in `C01 Merchandising & Assortment` and `C03 Buying & Supplier Management` lands on the same person. It is the commonest error an architect brings to a retailer.

Step 4.5 — Name the value streams and their stages, and reconcile `O14` against `R02`

Produce. Five to eight value streams with a triggering party, an outcome, a stage count that matches reality, and a capability set that matches `R02`.

You need. Steps 4.3 and 4.4, and the trading calendar.

Look it up. Volume 1, chapter 7, *The value chain is six clocks, not six steps*, and chapter 16, *Capabilities, value streams and the scarcest resource*.

Where it lands.

> **`O14 Value Stream`**
>
> `Ref`, `Value stream`, `Scope`, `Stages`, `Capabilities`. `Stages` is free text reading "4 stages", so it carries a count and no names. `Capabilities` duplicates `R02` and disagrees with it. Treat `R02` as authoritative; either delete `Capabilities` or regenerate it from `R02`.

How.

1. Keep the five shipped names. `VS01 Plan to Range`, `VS02 Source to Shelf`, `VS03 Browse to Buy`, `VS04 Order to Doorstep` and `VS05 Return to Resolution` are a defensible retail set and the best-named rows in the repository.
2. Replace the count with named stages, in a column you add. Three of the five counts are wrong against `O16`: `VS02` claims five stages and has four processes, `VS03` claims six and has four, `VS05` claims five and has four.
3. Reconcile `Capabilities` against `R02`. They agree for `VS01`, `VS02` and `VS03` and disagree for `VS04` and `VS05` — `O14` gives `VS04` the set `C01, C02, C03`, `R02` gives it `C04, C05, C06`. One was generated by a window that reset, the other by a window that kept sliding.
4. Notice which capabilities no stream uses. `R02` reaches `C01` to `C07` only; `C08 Customer & Loyalty`, `C09 Space & Range Planning` and `C10 Finance & Margin Management` appear in none, and `VS03 Browse to Buy` uses neither `C06 Digital Commerce` nor `C08`. A retailer whose purchase stream touches neither e-commerce nor loyalty has described somebody else's business.
5. Add the streams the set is missing: markdown-to-clearance and range review, both quarterly events the executive plans around.

Signed by. Chief Operating Officer (`ORG-07`) for the operational streams; Chief Commercial Officer (`ORG-08`) for the merchandising streams.

Done when. Every stream has a named triggering party, named stages, and a capability set `R02` and `O14` report identically.

Shows up in. No report reads `O14` or `R02`. The reconciliation is yours to run and record.

Goes wrong when. Value streams are written as the organisation's departments in sequence. A retailer runs six clocks at different speeds, and a stream crossing three of them needs its cadence written beside it.

Form B-2 — Value stream catalogue

One row per value stream. Columns are the real `014 Value Stream` columns in sheet order. A first pass needs six to eight rows. Signed by the Chief Operating Officer, countersigned by the Chief Commercial Officer. Name the stages in a column you add; `Stages` as shipped holds a count.

Ref	Value stream	Scope	Stages	Capabilities
VS06	Markdown to Clearance — example, delete me	Markdown decision through clearance and residual disposal.	4 stages	

Step 4.6 — Rebuild the capability map, and apply the four tests

Produce. A capability map whose level 1 rows the executive recognises and whose level 2 rows somebody will defend, with `R01 Capability decomposition` matching `Parent` exactly.

You need. Steps 4.1 and 4.5. Volume 1 appendix A drafts `C-01` … `C-20`.

Look it up. Volume 1, chapter 19, *Business capabilities*, and chapter 7, *Which boundaries are real*.

Where it lands.

> **`013 Business Capability`**
>
> `Ref`, `Capability`, `Level`, `Parent`, `Description`, `Criticality`, `Current`, `Target`, `Disposition`. `Parent` carries the hierarchy and `R01` repeats it; keep them identical. `Description` must say what is in and what is out. There is **no `Owner` column** — ownership lives only in `R22`, one level deep.

How.

1. Load `C-01` … `C-20` as candidates and **map by name, never by number.** Twenty drafted capabilities against ten level 1 rows is not a renumbering.
2. Keep the ten level 1 names. `C01 Merchandising & Assortment` through `C10 Finance & Margin Management` are a sound retail level 1, and their five-word descriptions are the only honest descriptions on the sheet.
3. Delete the ninety rows below level 1 and rebuild them. They are a word list crossed with the level 1 names: three children each drawn from `Administration`, `Optimisation`, `Onboarding`, `Reporting`, `Assurance`, `Support`, `Planning`, `Execution` and `Design`, and two children each suffixed `– Define` and `– Operate`. Ten, thirty, sixty. No retailer decomposes that evenly, and the descriptions restate the names.
4. Apply the four tests to every candidate. A capability is a **noun, not a verb**; it **survives a reorganisation**; it has **one nameable owner**; and it is **scorable** — somebody can say what level 3 looks like and what evidence would prove it.
5. Rebuild `R01` from `Parent` by formula, not by hand, and check the counts match.

Signed by. `GB-01 Architecture Board`, with each level 1 capability's owning executive signing their own branch.

Done when. No row restates its parent's name, every row passes the four tests, and `R01` and `Parent` agree on every row you kept.

Shows up in. `RPT03 Capability Heatmap`, which reports the ten level 1 rows. Ninety rows appear in no report at all, which is why nobody noticed how they were made.

Goes wrong when. The generated decomposition is kept because deleting ninety rows looks like losing work. Every score, gap and roadmap item built on a word list inherits the fiction.

THE FOUR TESTS, BEFORE YOU KEEP A ROW

Write the one sentence that decides whether a row belongs on this map here.

Name the capability whose boundary you expect to be argued about.

Which of the ninety generated rows, if any, survives the four tests? Name it and say why.

Step 4.7 — Assign capability ownership, and take `R22` apart

Produce. One accountable organisation unit per capability, at every level you scored, with `R22 Org Unit owns Capability` carrying it.

You need. `O09` and `O13` complete.

Where it lands. `R22 Org Unit owns Capability` — `Org unit`, `Unit name`, `Capability`, `Capability name`. One row per owned capability. There is no `Accountable` or `Consulted` distinction, so a shared capability needs a note in `DLV-04`, not a second row.

How.

1. Read the ten shipped rows and recognise the pattern. `ORG-01` owns `C01`, `ORG-02` owns `C02`, straight down the diagonal — so `ORG-03 Risk & Compliance` owns `C03 Buying & Supplier Management`, `ORG-04 Human Resources` owns `C04 Supply Chain & Replenishment`, `ORG-05 Technology` owns `C05 Store Operations` and `ORG-09 Customer Service` owns `C09 Space & Range Planning`. All ten rows are an index join, not a decision.
2. Reassign by asking one question per row: which unit's executive loses their bonus if this capability fails? `C01`, `C02` and `C03` go to `ORG-08 Commercial`; `C04` and `C07` to `ORG-10 Supply & Logistics`; `C05` to `ORG-07 Operations`; `C10` to `ORG-02 Finance`.
3. Extend `R22` below level 1. Ninety capabilities have no owner, and `RPT03` will never tell you, because it reports level 1 only.
4. Where two units claim a capability, pick one and write the other into `DLV-04` as consulted. A capability with two owners has none.

Signed by. `GB-04 Executive Committee`, because reassigning ownership is a conversation between executives and the architect is not a party to it.

Done when. Every capability you scored has exactly one owning unit, and the owning executive has said the words aloud in a meeting with the other nine.

Shows up in. Nothing reads `R22`. `RPT03` shows maturity without ownership, which is how a capability map is discussed for a year with nobody accountable for a row on it.

Goes wrong when. Ownership goes to the unit that operates the system rather than the unit that owns the outcome. `ORG-05 Technology` owning `C05 Store Operations` is that error pure.

Step 4.8 — Score `Current` and `Target` with a method you can defend

Produce. A maturity score per capability in scope, the evidence behind each score, and a `Target` somebody has agreed to fund.

You need. The map from step 4.6, the owning executive from step 4.7, three hours per level 1 branch.

Look it up. Volume 1, chapter 6, *The four numbers a retail executive watches*, for what evidence of a working capability looks like in this industry.

Where it lands. `Current`, `Target` and `Disposition` on `013 Business Capability`. Both scores are integers; the shipped data uses 1 to 4 for `Current` and up to 5 for `Target`, so five levels is the scale you inherit.

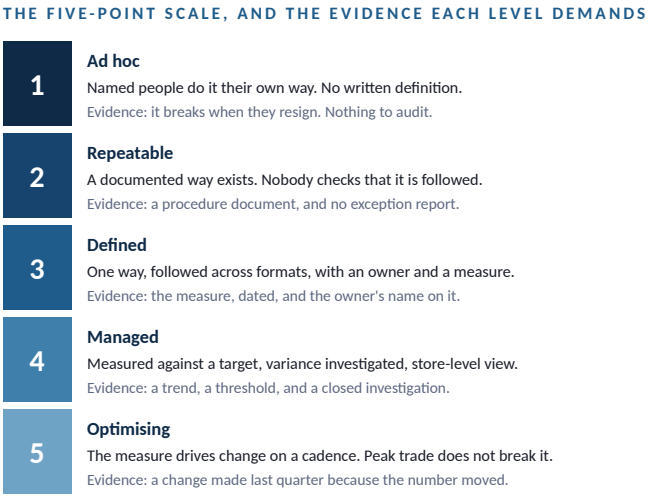


Figure 4.4 The scale to publish three days before the first workshop. `013` already holds `Current` values 1 to 4 and `Target` values up to 5, so five levels is the scale you inherit. The right-hand line is the part that makes a score defensible.

How.

1. Publish the scale and the evidence each level demands three days before the first workshop. A score without a stated evidence test is an opinion with a number on it.
2. Score blind: the owning executive, the operating manager and you, then compare. The argument is the output.
3. Set `Target` against the funded plan, not the ambition. Thirty-two of the hundred shipped rows carry a `Target` of 5 — best-in-class asserted in a third of everything the group does.

4. Set `Disposition` as a judgement. In the shipped data every row is `Build` where `Target` minus `Current` is 2 or more and `Sustain` otherwise, with no exceptions in a hundred rows and only two values on the sheet. That is a formula, and a formula is not a decision.
5. Re-read `Criticality` against the scores. Thirty-three rows are graded more critical than their own parent, and four of the ten level 1 capabilities are `Low`: `C07 Order Fulfilment & Returns`, `C08 Customer & Loyalty`, `C09 Space & Range Planning` and `C10 Finance & Margin Management`. A retailer where margin management, loyalty and shelf space are all low-criticality is not a retailer.

Signed by. The owning executive from `R22` signs their own branch. `GB-01 Architecture Board` approves the scale.

Done when. Every score has a named evidence statement, no child is graded more critical than its parent without a written reason, and `Disposition` disagrees with the arithmetic on at least one row.

Shows up in. `RPT03 Capability Heatmap`, which recomputes `Gap` as `Target` minus `Current` across ten rows. Read it after this step.

Goes wrong when. The workshop scores everything 2 or 3. Publish the evidence test first and the scores spread, because a 4 then costs somebody a document.

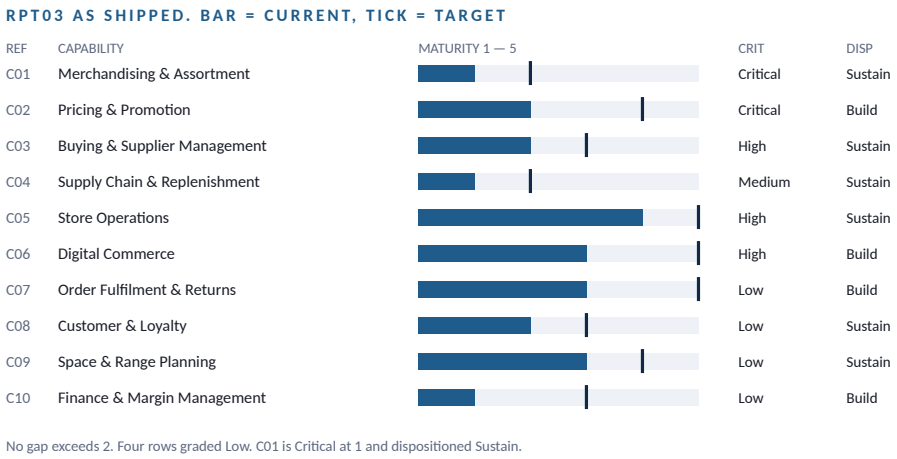


Figure 4.5 `RPT03 Capability Heatmap` exactly as shipped: ten level 1 rows, no gap larger than 2, four capabilities graded `Low`, and a `Disposition` column that is arithmetic rather than judgement. The ninety rows below level 1 appear in no report at all.

BEFORE THE FIRST SCORING WORKSHOP

What evidence will you accept as proof of level 4, in one sentence a sceptic would sign?

Which three capabilities do you expect the executive and the operating manager to score differently, and by how much?

Step 4.9 — Keep business functions and business processes apart

Produce. `O17 Business Function` holding what the organisation continuously does, and a written rule keeping it out of `O16 Business Process`.

You need. `O13` and `O09` complete.

Look it up. Volume 1, chapter 16, *Is it a service or a process?*

Where it lands.

> **`O17 Business Function`**
>
> `Ref`, `Business function`, `Capability`, `Description`. `Capability` is a single reference, so a function serving two capabilities cannot be recorded. There is **no `Organisation unit` column**, which is the one thing a function needs.

How.

1. Apply the test: a function is ongoing and has no end state; a process has a trigger and an outcome. "Pricing & Promotion" is a function. "Execute a markdown" is a process.
2. Delete the duplication. `O17`'s ten rows carry the same names as `O13`'s level 1 rows and `O15`'s services. Three sheets recording one list is not a model.
3. Add the functions no capability names: supplier income accounting, loss prevention, store development, pharmacy compliance, then add `Organisation unit` and fill it from `O09`.

Signed by. Chief Operating Officer (`ORG-07`).

Done when. No `O17` row shares a name with an `O13` or `O15` row, and every function names its unit.

Shows up in. Nowhere. `O17` feeds no report.

Goes wrong when. The function sheet becomes a second capability map with different refs, and `R03` and `R04` then have two candidate parents for every service.

Step 4.10 — Build the process inventory at L1, and point at the L2 models

Produce. `O16 Business Process` holding the group's L1 processes, each with a trigger, an outcome, an owner and a volume, and a pointer to the L2 models beneath them.

You need. Step 4.5. Volume 1 chapter 8 publishes twelve L1 processes and fourteen BPMN L2 models — start from those.

Look it up. Volume 1, chapter 8, *The L1 process map*, and *L0, L1, L2, L3 — and the warning*.

Where it lands.

> **`O16 Business Process`**
>
> `Ref`, `Business process`, `Value stream`, `Description`, `Automation`. `Automation` takes `Manual`, `Partly automated` and `Automated`. There is **no `Level`, no `Trigger`, no `Outcome`, no `Owner` and no `Volume p.a.`** Without `Level` the sheet cannot hold an L1 and an L2 process at once.

How.

1. Add `Level`, `Trigger`, `Outcome`, `Owner` and `Volume p.a.`
2. Replace the twenty shipped rows. They are not processes: every one reads `<Value stream> – stage N` with the description "Stage N of <value stream>", four per stream, and `Automation` varies without anything to vary against. Stage placeholders tell you the value streams were typed and the processes never were.
3. Load volume 1's twelve L1 processes by name and map them onto `Value stream`: a price change and a promotion going live, a prescription dispensed, an online order to collection, supplier onboarding and listing, range review, order-receive-pay, forecast and replenish, allocate a fixed buy, serve a customer at the point of sale, take a return, count the stock, execute a markdown.
4. Fill `Volume p.a.` from the data warehouse, not from an estimate. It decides which processes get an L2 model and which get a paragraph.
5. Reference the fourteen L2 models by name in `Description`. There is no sheet for a BPMN model and no attachment column; the model lives in the modelling tool and the repository holds the pointer.

Signed by. The process owner you just added to the `Owner` column. Not the architect, and not the value stream owner.

Done when. Every row has a trigger, an outcome, a named owner and an annual volume, and `Level` distinguishes L1 from L2.

Shows up in. No report reads `O16`, which is why twenty placeholder rows survived delivery.

Goes wrong when. Processes are modelled from what the business said in a workshop rather than from what the logs show. Ask for the exception volumes: a process with no recorded exceptions is a slide.

Step 4.11 — Define the business services, and refuse the one-for-one

Produce. `O15 Business Service` holding only what a party outside the group can ask for, with `R03 Capability realised by Busi` and `R04 Business Service supported` filled against reality.

You need. Steps 4.6 and 4.9, and `O23 Application Component`.

Look it up. Volume 1, chapter 16, *Is it a service or a process?*

Where it lands.

> **`O15 Business Service`**
>
> `Ref`, `Business service`, `Capability`, `Description`, `Exposure`, `Criticality`. `Exposure` takes `Internal` or `External` and decides whether a row belongs on the sheet at all. `Criticality` duplicates `O13`'s column and disagrees with it.

How.

1. Apply the test: can a party outside the group ask for this, by name, without knowing how the group is organised? If not, it is a function and belongs in `O17`.
2. Delete the one-for-one. Ten capabilities, ten services, ten identical names with `Service` appended, and `R03` joining them straight across. The symmetry is the tell.
3. Fix `Exposure` against channel reality. `BSV-09 Space & Range Planning Service` is `External`, which would mean a customer asks the group to planogram a shelf; `BSV-06 Digital Commerce Service` is `Internal`, in a retailer whose customers transact online.
4. Reconcile `Criticality` with `O13`. Six of ten rows disagree, and `O15` reads `Medium` on seven. `C01 Merchandising & Assortment` is `Critical`; `BSV-01` is `Medium`. One of those is wrong and it is not the capability.
5. Fill `R04` against the real estate and read every row aloud. As shipped it is an offset diagonal: nine of ten rows are wrong and each looks plausible. `BSV-02 Pricing & Promotion Service` is supported by `A02 POS` rather than `A08 Price & Promotion Engine`; `BSV-06` by `A06 Demand Forecasting & Replenishment`; and `BSV-09 Space & Range Planning Service` by `A09 Loyalty & CRM` while an application called `A07 Space & Range Planning` sits unused two rows away.

Signed by. The capability owner for the definition; the application owner named in `O23` for each `R04` row.

Done when. Every service passes the outside-party test, `Criticality` agrees with its capability or the disagreement is recorded, and every `R04` row has been read aloud by

somebody who knows the system.

Shows up in. `RPT13 Orphan Analysis` tests `R04` and nothing else, which is how `A11 Supplier Portal` and `A12 Retail Analytics` come to be the only recorded orphans. Re-read `RPT13` after this step, not before.

Goes wrong when. `R04` is filled from a vendor diagram. The test for a wrong row: which team would you call when the service fails? If it is not the team owning the application in the row, the row is wrong.

Step 4.12 — Register the products, and say out loud that this is not the article master

Produce. `018 Product` holding the group's merchandise and service propositions, and a one-line statement in `DLV-04` that the article master is not here.

You need. The category structure from Merchandising, and the services the group sells beyond merchandise.

Look it up. Volume 1, chapter 16, *Is the article a Product?*, and chapter 3, *Every system in the estate has its own answer to "what is a SKU"*.

Where it lands.

> **`018 Product`**
>
> `Ref`, `Product`, `Type`, `Description`. `Type` mixes two axes in the shipped data — `Own brand` and `Branded` are about who owns the label, `Non-food` is about the category, `Service` about neither. Pick one axis per column. There is **no `Owner`, no `Status` and no `Format`.**

> **`018 PRODUCT` IS NOT THE ARTICLE MASTER, AND TREATING IT AS ONE WRECKS PHASE C**
>
> Halverson's five rows — `PRD-01 Own Brand Grocery`, `PRD-02 Branded Grocery`, `PRD-03 General Merchandise`, `PRD-04 Fresh and Chilled`, `PRD-05 Financial Services` — are a proposition list, which is the right content for this sheet. The thing a retailer sells, the article with its GTIN, tax code, unit of measure and supplier, is `X01 SKU / Article` on `EXT Industry Extension`, and in the live estate it is several hundred thousand rows. **It does not belong on `018` and it never will.** Write that sentence into `DLV-04` in the first week, or somebody will try to load the item master into the repository and spend a month producing nothing [3].

How.

1. Load the propositions, not the articles. Five to fifteen rows is normal; a format-by-category grid is not a product list.
2. Add `Owner` and fill it with a product or category director from `011`.
3. Record the service propositions a retailer sells alongside merchandise: retail media, insurance and credit in store, delivery subscriptions, marketplace listing. `PRD-05 Financial`

`Services` is the only one on the sheet.

4. Fill `R25 Application supports Produc` from the administering system. As shipped it steps three applications at a time — `PRD-03 General Merchandise` supported by `A07 Space & Range Planning`, `PRD-04 Fresh and Chilled` by `A10 Workforce Management` — and four of the five rows are wrong.

Signed by. Chief Commercial Officer (`ORG-08`) for the propositions; `GB-01 Architecture Board` for the statement that the article master lives in `X01`.

Done when. `018` holds propositions, each with a named owner, and `DLV-04` carries the sentence about the article master.

Shows up in. `R25` only, and through it `RPT02 Application Portfolio`.

Goes wrong when. Products are loaded at category-line granularity to make the sheet look complete, and Phase C then cannot tell whether `E01 SKU / Article` and `018` are the same thing.

Step 4.13 — Register the contracts, starting with the ones the repository forgot

Produce. `019 Contract` holding both the technology agreements and the supplier trading terms, with a legal entity in `Counterparty`.

You need. The vendor contract register from Procurement, and the trading terms register from Buying and supplier income.

Look it up. Volume 1, chapter 6, *Supplier income*, and chapter 5, *Finance, and the supplier-income team*.

Where it lands.

> **`019 Contract`**
>
> `Ref`, `Contract`, `Counterparty`, `Type`, `Expiry`. `Counterparty` must be the legal entity you would sue. `Type` takes `Perpetual`, `Subscription` and `Managed service`, and you will need `Trading terms` and `Concession`. There is **no `Value`, no `Notice period`, no `Renewal`** and **no `Owner`**.

How.

1. Fix `Counterparty` on all ten shipped rows. Every one names the **application** — `CTR-01` reads `Merchandising System` — not the company that signed.
2. Resolve `Type` against `Expiry`. Four contracts are `Perpetual` and all four carry an expiry date. A perpetual licence with a support renewal is two facts; record them in two columns.
3. Add the trading terms. The ten shipped rows are all IT vendor agreements, and in a retailer the contracts that bind the architecture hardest are the supplier ones: listing fees, rebates, settlement discount, promotional funding, drop-shipment and concession terms. The

Competition Commission's grocery inquiry found back-margin terms opaque, and a group whose income terms live in spreadsheets cannot answer the Commission or its own auditor [6][7].

4. Add `Owner` and `Notice period`. A contract without a notice period cannot be exited on a roadmap, which makes Phase F guesswork.
5. Add the concession and franchise agreements, because they decide who owns the stock and the price in part of the estate.

Signed by. General Counsel (`ORG-12`) for every row, because a contract register with the wrong counterparty is a legal exposure rather than a modelling defect; Chief Financial Officer (`ORG-02`) for the trading terms.

Done when. Every row names a legal entity, `Type` and `Expiry` no longer contradict each other, and the register holds at least one supplier trading agreement.

Shows up in. `O19` feeds no report, so expiry surfaces nowhere until Phase D reads `Support end` on `O27 Physical Technology Compone`. Put the dates on the Phase F roadmap by hand.

Goes wrong when. Only the IT contracts are loaded, and the roadmap has no idea that a supplier income agreement renegotiates every February and resets the margin assumption under every business case.

Step 4.14 — Stand up `X01 SKU / Article`, the object the core metamodel cannot hold

Produce. A definition of `X01 SKU / Article` stating the level it is mastered at, the identifier schemes it carries and the one system that owns it — plus a sheet to hold it.

You need. Step 4.12, volume 1 chapter 9's three levels, and the item master counts from Merchandising.

Look it up. Volume 1, chapter 9, *Article, style, option, SKU: the argument the model lives or dies on*, and chapter 3, *The identifiers, and the two defects that cause most of the pain*.

Where it lands.

> **`EXT` Industry Extension**
>
> `Ref`, `Industry extension type`, `What it holds and why the core metamodel is not enough`. `X01 SKU / Article` reads "The smallest sellable unit with its trading attributes" — twelve words and no columns. `EXT` is a **catalogue of six types, not a sheet of records.** It names the extension; it cannot hold one.

How.

1. Decide the level `X01` is mastered at before you add a column. Style, option and SKU are three grains; a group with an apparel business masters at all three while a grocer masters at one. Write the decision down — Phase C's `E01` inherits it.
2. Create the sheet `EXT` only names. Give it `Ref`, `Article`, `Level`, `Parent article`, `GTIN`, `Supplier`, `Unit of measure`, `Tax code`, `Status` and `Master system`, and accept that it holds a sample. Several hundred thousand rows do not belong in a workbook.
3. Record the identifier rule. A GTIN identifies a trade item at a packaging level; it is not a primary key and it is not stable across a product change [8]. Say which identifier is the key and which is merely recorded.
4. Join `X01` to `O18 Product` as a many-to-one and write the rule into `DLV-04`. There is no relationship sheet for it, so the join is documentation.
5. Say what is out of scope: pricing, stock and ranging are `E05`, `E08` and `X03`, and letting them creep into the article definition is how an item master becomes a second database.

Signed by. Chief Commercial Officer (`ORG-08`) for the level decision, Chief Data Officer (`ORG-06`) for the identifier rule.

Done when. The grain is a written sentence, the identifier rule distinguishes key from attribute, and a sample sheet holds ten real articles.

Shows up in. Nothing. No report reads `EXT`, and the sheet you add will not be read until Phase C's `R05 Application manages Data En` points at it.

Goes wrong when. `X01` is treated as documentation rather than an object, and the grain is then decided implicitly by whichever system is loaded first. Every downstream argument about

availability, price and margin traces back to that sentence.

THE THREE OBJECTS THE CORE CANNOT HOLD

At what level is an article mastered in this group, and who signed that sentence?

Does `X02` extend `O12 Location` or replace it, and which of the two is authoritative?

Is the assortment binding or advisory, and for which part of the estate is the answer different?

Step 4.15 — Stand up `X02 Store / Location`, the dimension every retail number hangs on

Produce. A location object carrying format, ownership model, trading hours, licences and fulfilment role — and a statement of how it relates to `O12 Location`.

You need. Step 4.2, and the licence register from Compliance.

Look it up. Volume 1, chapter 2, *A format is an economic machine, not a shop size*, and chapter 7, *Channels, formats, and what each costs to serve*.

Where it lands. `X02 Store / Location` on `EXT Industry Extension`, which reads "A trading or fulfilment location with its format and capacity" and has no columns. `O12 Location` holds eight rows and four columns and is the nearest thing the core gives you.

How.

1. Decide whether `X02` extends `O12` or replaces it. Extending is right under about fifty locations; replacing is right otherwise, and `O12` then keeps only the corporate, logistics and technology sites.
2. Give `X02` the columns the core has nowhere for: `Format`, `Ownership model`, `Trading area`, `Trading hours`, `Licences held`, `Fulfilment role`, `Cluster`, `Open date`, `Close date`.
3. Record `Licences held` as a list, not a flag. A liquor licence, a pharmacy licence and a tobacco permit each bind what may lawfully be sold at that location, and the till has to enforce it.
4. Record `Fulfilment role` and reconcile it with `X06 Fulfilment Node`. A store that picks online orders is both; the two types overlap by design, so say which is authoritative.
5. Record `Cluster`, because the assortment in step 4.16 is set per cluster and not per store.

Signed by. Chief Operating Officer (`ORG-07`) for format and trading, Chief Risk Officer (`ORG-03`) for the licences.

Done when. Every trading location carries a format, an ownership model and a licence list, and the relationship between `X02` and `O12` is written down.

Shows up in. `R20 Application located at Loca` reads `O12`, so a location recorded only in `X02` is invisible to the application landscape. Settle the authoritative sheet before Phase C.

Goes wrong when. Locations are recorded at format level, which is what the repository does. `LOC-04 Convenience Format` with 240 people is a format row pretending to be a location, and every per-store number from it averages a population nobody has counted.

Form B-3 — Store register, first pass

One row per trading location, or per cluster where the estate is large. The first four columns are the real `O12 Location` columns in sheet order; `Format` and `Ownership model` are the two columns you add in step 4.2, and they are the reason the form exists. A first pass needs one row per cluster, usually eight to twenty. Signed by the Chief Operating Officer. The licences, trading hours and fulfilment role go straight into Excel.

Ref	Location	Type	Scale	Format	Ownership model
LOC-09	Riverside — example, delete me	Trading	2,400 m²	Supermarket	Franchise

Step 4.16 — Stand up `X03 Assortment` as the decision, not the list

Produce. An assortment object recording, per cluster and per period, which articles are ranged — and who decided.

You need. Steps 4.14 and 4.15, and the range review calendar.

Look it up. Volume 1, chapter 9, *The six hierarchies, all of them correct*, and chapter 2, *What the format mix does to the application estate*.

Where it lands. `X03 Assortment` on `EXT Industry Extension`, which reads "The ranged set of SKUs for a location and period". Note what that already commits you to: an assortment is dated, and it is per location or cluster. The core metamodel has no object with those two properties.

How.

1. Decide the grain: article by cluster by period, or article by store by period. Cluster is normal; store-level ranging in a large estate is a data volume problem rather than a modelling one.
2. Record the three states separately. Ranged, planogrammed and orderable are not the same thing, and an article can be any one without the others. A single `Ranged` flag loses the distinction behind most availability disputes.
3. Date it properly. `Effective from` and `Effective to` are not optional: a range review taking effect in week 14 and an article delisted in week 9 are both facts about the same cluster.
4. Record `Decided by` and `Decision date`, which make an assortment a governed decision rather than a system extract.
5. For the franchised and voluntary-trading estate, record whether the assortment is binding or advisory. In a voluntary-trading group it is advisory, and a report that treats it as binding overstates compliance for every member store [6].

Signed by. The range planner from step 4.4, countersigned by the Chief Commercial Officer (`ORG-08`).

Done when. Every assortment row carries a cluster, a period, three state flags, a decider and a binding-or-advisory marker.

Shows up in. Nothing, and that is the gap to raise in `O29 Gap`. No report here can answer "what is ranged where", which in a retailer is close to the first question anybody asks.

Goes wrong when. Assortment is modelled as an attribute of the article. It is a relationship between an article, a location set and a period, and flattening it into the item master is the commonest data-model defect in retail.

Step 4.17 — Close Phase B through `RPT03` and `RPT13`

Produce. A signed Phase B result: the capability map, the value streams, the process inventory, the three extension objects, the gaps raised, and a written list of every repository defect you found and did not fix.

You need. Steps 4.1 to 4.16 complete for the capabilities and formats in scope.

Where it lands. `O31 Deliverable` rows `DLV-04 Business Architecture`, `DLV-05 Capability Map` and `DLV-06 Value Stream Definition`. All three are already marked

`Approved`, against the data this chapter has been taking apart. Set them back to draft, or your sign-off means nothing.

How.

1. Re-run `ZZ Checks` and check its own numbers. It reports `Object types populated: 65` against the threshold "Every type carries records" while `O59 Compliance Assessment` holds zero records and `O29 Gap` holds one. The check passes and is false; record the discrepancy [2].
2. Read `RPT03 Capability Heatmap` top to bottom and ask whether you would defend each of the ten rows to the Audit Committee. `C01 Merchandising & Assortment` is `Critical`, scores 1, targets 2 and is dispositioned `Sustain`. That row alone is a finding.
3. Run the four checks `RPT13 Orphan Analysis` does not: capabilities with no `R22` owner below level 1, capabilities used by no value stream in `R02`, locations in `X02` and not `O12`, and applications referenced by `R25` but not `R04`.
4. Count your `O29 Gap` rows with `Domain` = `Business`. The repository shipped with one, `GAP-01`, due 2028-06. Fewer than five and you have under-reported.
5. Write the defect list. It is the most valuable artefact Phase B produces here, and it belongs in front of `GB-01 Architecture Board`, not in an appendix.

Signed by. `GB-01 Architecture Board` approves the result at gate `GT-1 G1 Design` on `O58 Governance Gate`. The sponsor signs the scope.

Done when. `RPT07 Gap Analysis` has more business-domain rows than it started with, the three deliverables carry a status you set yourself, and the defect list has a named owner per line.

Shows up in. `RPT01 Dashboard`, `RPT03`, `RPT07 Gap Analysis` and `RPT13`. If `RPT07` still shows one gap, Phase B produced a description rather than an architecture.

Goes wrong when. Phase B closes with a tidy capability map and a near-empty gap sheet. That combination is not a clean estate. It is a phase that did not look.

What a finished Phase B looks like in the numbers

Reading	As delivered	After Phase B
`013` rows at levels 2 and 3	90, generated from a word list	As many as you can score, each named by its owner
`013` `Disposition` values	2, derived from `Target` minus `Current`	4, at least one disagreeing with the arithmetic
`013` level 1 rows graded `Low`	4, including margin and loyalty	None you cannot defend
`010` external parties	0 of 18	Customers, suppliers, franchisees, regulators, schemes
`011` retail roles	0 of 18	The buyer, the planners, the store, the till, the pharmacy
`014` versus `R02` agreement	3 of 5 streams	5 of 5
`016` rows that are processes	0 of 20	Twelve L1 processes with a trigger and an outcome
`R04` rows naming the right application	1 of 10	10 of 10
`R22` rows that are ownership decisions	0 of 10	One per capability, at every level scored
`EXT` types with a sheet behind them	0 of 6	`X01`, `X02` and `X03` at least
`029 Gap` rows, `Domain` = `Business`	1	5 or more

The distance between those two columns is Phase B. Nothing in the first column is an Excel defect: every sheet is structurally sound, almost every reference resolves, `ZZ Checks` passes most of its thresholds, and `00 Index` states 842 records and 429 relationship instances, which is true. The repository is complete and almost entirely untrue, which is the normal condition of a populated metamodel you did not fill yourself [1][2][3].

PHASE B — A REVIEWER'S QUESTIONS

Which capability will be argued about, and what is the sentence that settles it?

Which repository defect did you find and decide not to fix, and who agreed to carry it?

If the Chief Commercial Officer reads `RPT03` and disagrees with one row, which row is it?

Where in your model is the part of the estate the group does not own, and what is true of it that is not true of the rest?

How many business-domain gaps did Phase B raise, and who owns the largest one?

THE CHAPTER IN TEN LINES

- Phase B here is eleven object sheets, three industry-extension types and seven relationship sheets. The relationships are the part that lies.
- Several `R` tab names are truncated at thirty-one characters. Spell them as the tab spells them.
- `O09` has no `Parent` column and `O12` has no `Format` and no `Ownership model`. Add all three before loading a row.
- A corporate store, a franchise and a voluntary-trading member are governed differently, and for much of the turnover the group has a recommendation rather than a price.
- `O10 Actor` and `O11 Role` hold the same eighteen names in the same order, and not one of them is a retail role.
- Buyer, merchandise planner, range planner and allocator are four roles. Collapsing them is the commonest error an architect brings to a retailer.
- `O14`'s `Capabilities` column and `R02` disagree for `VS04` and `VS05`. Make `R02` authoritative.
- `R02` reaches `C01` to `C07` only. Loyalty, space planning and margin management are used by no value stream.
- Four tests for a capability: noun not verb, survives a reorganisation, one nameable owner, scorable.
- `O13`'s ninety rows below level 1 are a word list crossed with the level 1 names, and they appear in no report. Delete and rebuild.
- `Disposition` is `Build` where `Target` minus `Current` is 2 or more, on all hundred rows. That is a formula, not a decision.
- Thirty-three capabilities are graded more critical than their own parent, and four level 1 rows are `Low`, including margin and loyalty.
- `R22` is a diagonal: Human Resources owns Supply Chain and Technology owns Store Operations. Ninety capabilities have no owner at all.
- `O13` level 1, `O15` and `O17` carry the same ten names. Three sheets recording one list is not a model.
- `R04` is offset by one. Nine of ten rows are wrong, and `BSV-09 Space & Range Planning Service` is supported by Loyalty & CRM.
- `O16` has no `Level`, `Trigger`, `Outcome`, `Owner` or `Volume p.a.`, and its twenty rows are stage placeholders.
- `O18 Product` is a proposition list, not the article master. The article is `X01` and it never belongs on `O18`.
- `O19 Contract` names applications as counterparties, holds four perpetual contracts with expiry dates, and contains no supplier trading terms.
- `X01`, `X02` and `X03` are the three objects Phase B turns on, and `EXT` names them without holding one record.
- `O29 Gap` holds one row. A Phase B that raises one business gap did not look.

SOURCES FOR THIS CHAPTER

- **1.** Retail_TOGAF_Metamodel.xlsx, Phase B sheets O09–O19 and R01, R02, R03, R04, R21, R22, R25 (Halverson Stores Group, illustrative data)
- **2.** Retail_TOGAF_Metamodel.xlsx, sheets RPT03 Capability Heatmap, RPT07 Gap Analysis, RPT13 Orphan Analysis, RPT01 Dashboard and ZZ Checks
- **3.** Retail_TOGAF_Metamodel.xlsx, sheets EXT Industry Extension, O12 Location, O23 Application Component, O29 Gap, O31 Deliverable, O57 Governance Body and O58 Governance Gate
- **4.** The Open Group — TOGAF Standard, 10th Edition: ADM Methods, Phase B Business Architecture https://pubs.opengroup.org/architecture/togaf10-doc/adm/
- **5.** APQC — Process Classification Framework (PCF) for Retail, version 6.1.1 https://www.apqc.org/resource-library/resource-listing/apqc-process-classification-framework-pcf-retail-excel-version
- **6.** Competition Commission of South Africa — Grocery Retail Market Inquiry Final Report: Summary of the Findings https://www.compcom.co.za/wp-content/uploads/2019/12/Grocery-Retail-Market-Inquiry-SUMMARY_.pdf
- **7.** Competition Commission of South Africa — Buyer Power Enforcement Guidelines, 12 May 2020 https://www.compcom.co.za/wp-content/uploads/2020/05/Buyer-Power-Guidelines.pdf
- **8.** GS1 — General Specifications, release 26.0, and the GTIN Management Rules https://ref.gs1.org/standards/genspecs/
- **9.** Object Management Group — Retail Domain Task Force work programme (UnifiedPOS 1.15.1, Digital Receipt API) https://www.omg.org/retail/

CHAPTER 5

Data Architecture

One column says what a word means. Another says which system is allowed to say it. The rest describe.

Two columns decide this phase. `Definition` on `O20 Data Entity` settles what a word means, and `Master system` says which system is allowed to say it. The other six columns describe. If you leave this chapter with fourteen definitions a lawyer and a developer would both sign, and fourteen master systems the owning executives will defend, Phase C has succeeded [1].

Halverson Stores Group, the retailer in the repository, is an invention, as is Milkwood Retail Group in volume 1. Their data does not mix.

> **BEFORE YOU OPEN A SHEET**
>
> Volume 1 does the teaching. This chapter does the filling.
>
> - **Volume 1, chapter 9, *Article, style, option, SKU: the argument the model lives or dies on*.** The one argument that has to be settled before anything else happens.
> - **Volume 1, chapter 9, *The fifteen on one page*.** The spine, and what hangs off it.
> - **Volume 1, chapter 10, *Ownership, lineage and the gate that actually works*.** Why `Master system` is the column that matters.
> - **Volume 1, chapter 10, *Classification, consent and POPIA*.** The columns POPIA actually needs.
> - **Volume 1, chapter 15, *Section 23, and why a stale shelf label is a legal exposure*.** The control this chapter turns into data.

What you are filling

The data half of Phase C is eleven object sheets and two relationship sheets: `O20 Data Entity`, `O21 Logical Data Component`, `O22 Physical Data Component`, `O36 Data Entity Relationship`, `O37 Data Attribute`, `O38 Data Set`, `O39 Data Migration`, `O40 Source To Target Mapping`, `O42 Metadata Element`, `O43 Master Data Rule`, `O44 Data Building Block`, plus `R05 Application manages Data En` and `R19 Data Entity relates to Data` — the last two truncated at Excel's thirty-one characters, so spell them as the tabs spell them.

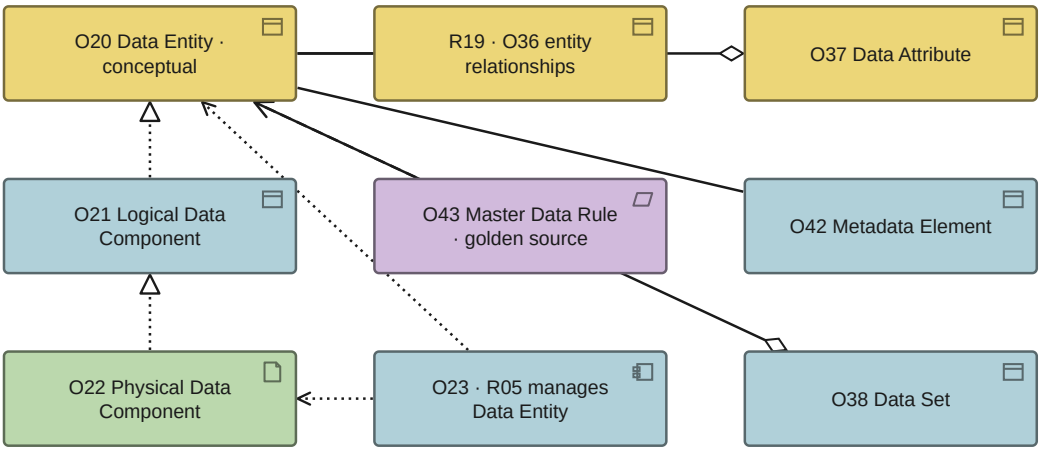


Figure 5.1 The three data layers as the repository holds them, and the join that is missing. `021` points at an entity, `022` points at an application, and no column on either names the other — so the logical-to-physical trace an architect needs has to be added before Phase D can use it.

One report reads any of it. `RPT05 Data Ownership` is `O20` with `Definition` dropped, and it carries a computed footer: "2 of 14 entities have no lineage from report back to source." The footer is arithmetically right and conceptually thin — it counts only the two rows marked `Absent` and ignores the six marked `Partial`. It is still the only automated statement this repository makes about its own data layer [2].

> **THE RULE FOR THIS PHASE**
>
> `O36 Data Entity Relationship` and `R19 Data Entity relates to Data` hold the same eleven rows, identically. An object sheet and a relationship sheet recording one fact is two places to maintain and one place to be wrong. Pick `R19`, keep `O36` as a view, and say so in writing before anybody edits either.

The steps

Step 5.1 — Inherit Phase B and set the entity scope

Produce. One page: which entities this iteration covers, at which layer, and which are explicitly deferred.

You need. The Phase B result — `O13`, `O14` and `O16` as you left them, plus the grain decision from step 4.14. The fourteen rows of `O20 Data Entity` as delivered.

Look it up. Volume 1, chapter 9, *The afternoon that produces one*, for how long a conceptual model takes, and *Three tests for whether something is an entity*.

Where it lands. `DLV-07 Data Architecture` and `DLV-08 Data Model` on `O31 Deliverable`, both marked `Approved` in the repository you were given. Set them to draft.

How.

1. List the entities each in-scope value stream touches. If `VS02 Source to Shelf` is in scope, so are Article, Assortment, Supplier, Purchase order, Inventory position and Location.
2. Decide the layer. Conceptual (`O20`) for everything in scope; logical (`O21`) for what a project will build against; physical (`O22`) only for systems Phase E will touch.
3. Set the attribute boundary now. `O37 Data Attribute` holds sixty rows and will hold six hundred if you let it.
4. Write the deferral list with a date, so a deferred entity is a decision rather than an oversight.

Signed by. Chief Data Officer (`ORG-06` , `ACT-05`), with `GB-03 Data Governance Council` noting the scope.

Done when. Every row of `O20` is marked in, out or deferred, and the attribute boundary is a written list of entity names.

Shows up in. `RPT05 Data Ownership` shows all fourteen entities regardless of scope, so the scope page is the only place the boundary exists.

Goes wrong when. Scope is set by sheet rather than value stream, and three weeks go on `O37` while `Master system` on `O20` still says a till masters the article file.

Step 5.2 — Settle the article argument in `Class` and `Definition` before anything else

Produce. One paragraph in `Definition` on `E01 SKU / Article` that states the level the group masters at, what hangs above it, what hangs below it, and what is explicitly not an article.

You need. Step 4.14's grain decision. The item master row counts. A buyer, a planner and a merchandising systems analyst in one room for two hours.

Look it up. Volume 1, chapter 9, *Article, style, option, SKU: the argument the model lives or dies on*, and *The three levels that actually exist*, then *The rule that settles it*.

Where it lands.

> **`O20 Data Entity`**
>
> `Ref` , `Data entity` , `Class` , `Definition` , `Classification` , `Owner` , `Master system` , `Lineage` . `Class` takes `Master` and `Transactional` in the shipped data and **has no `Reference` value**, which leaves unit of measure, country of origin, tax code and barcode type with nowhere to live. `Definition` is the column that settles arguments: it must say what is in and what is out, not restate the name.

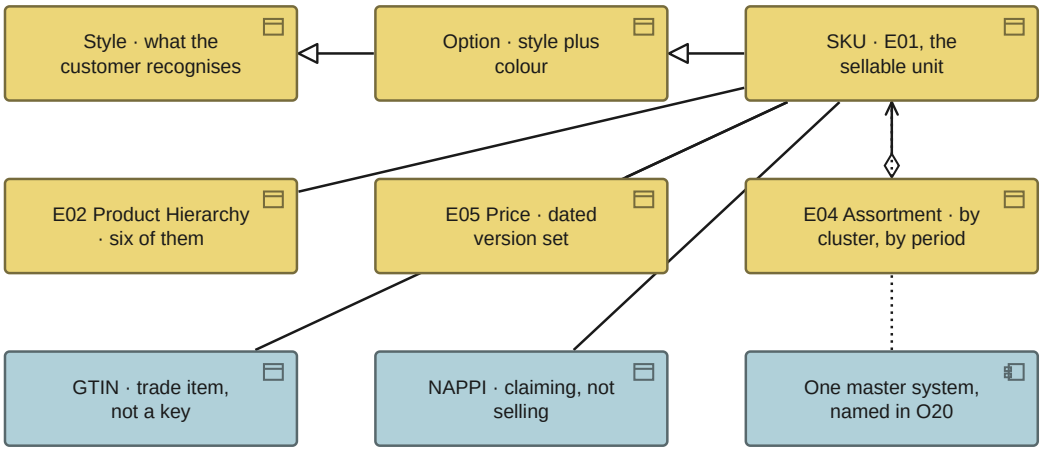


Figure 5.2 The argument step 5.2 has to settle. Three candidate grains, the objects that hang off whichever one is chosen, and the two identifiers that are not keys. `E01 SKU / Article` is one row on `O20`, which records the answer without stating the question.

How.

1. Write the three levels down by the names this group uses, not the names a vendor uses. Style, option and SKU is the apparel vocabulary; a grocer usually has one level and calls it the article. The six hierarchies in volume 1 all hang off whichever level you pick.
2. State the mastering level in one sentence and name the consequence. Master at style and the price file cannot vary by size. Master at SKU and the range review has ten times the rows to approve.
3. Say what an article is not. A barcode is not an article, a pack configuration is not an article, and a supplier's product code is not an article. Each of those is an attribute or a separate entity.
4. Add `Reference` to `Class` and move the code lists onto it. Reference data changes on a different cadence from master data and is governed by a different body.
5. Record the decision in `Definition` on `E01` and in `DLV-08`, and date it. Every argument for the next two years will be settled by reading it back.

Signed by. Chief Commercial Officer (`ORG-08`) and Chief Data Officer (`ORG-06`) jointly. A grain decision signed only by the architect will be reopened by the first project that finds it inconvenient.

Done when. `E01`'s `Definition` names the level, the two adjacent levels and three things an article is not, in under 120 words.

Shows up in. `RPT05 Data Ownership` drops `Definition`, so the most important column in the phase appears in no report. Put the paragraph in `DLV-08` as well.

Goes wrong when. The argument is deferred as "a detail for the logical model". Availability, price, margin and every forecast number are computed at this grain, and a group that has not written it down has three systems each assuming a different answer.

BEFORE YOU WRITE `E01`'S DEFINITION

At what level does this group master an article, and what breaks at the level above and below?

Name three things people in this business call an article that are not one.

Who signs this paragraph besides you?

Step 5.3 — Reconcile the entity catalogue against the spine, and add what is missing

Produce. A conceptual entity list the business recognises, with a definition per entity that says what is in and what is out.

You need. `O20` as delivered. Volume 1 chapter 9's fifteen-entity spine and appendix A's `D-01` … `D-20`.

Look it up. Volume 1, chapter 19, *Conceptual data entities*, for the drafted twenty, and chapter 9, *The entities that should not be there*.

Where it lands. `O20 Data Entity`, fourteen rows as delivered and twenty or so when you have finished.

How.

1. Load `D-01` … `D-20` as candidates and **map by name, never by number.** Twenty drafted entities against fourteen `E` rows is not a renumbering; the repository's `Ref` scheme is its own.
2. Keep what holds up. `E01 SKU / Article` through `E14 Supplier` are a defensible retail set, and the one-line definitions on them are better than anything else in this repository.
3. Add what the spine has and `O20` does not: **Inventory movement**, **Forecast**, **Employee**, **Dispensing record**, **Allocation**, **Shrink event** and **Supplier income agreement**. The first is the most serious omission — `E08 Inventory Position` records a quantity and nothing records why it changed, so no stock figure in the model can be explained.
4. Resolve the overlap with `EXT Industry Extension`. `X01`, `X02`, `X03`, `X04` and `X05` are already `E01`, `E03`, `E04`, `E06` and `E07` on `O20`. Decide which sheet is authoritative; do not maintain both.
5. Note what `EXT` has and `O20` does not, and the reverse. `X06 Fulfilment Node` has no entity. `E05 Price` has no extension type, in a sector where price is the most contested object in the estate.

Signed by. Chief Data Officer (`ORG-06`) for the list; each entity's business owner for its definition.

Done when. Every row has a definition that names an exclusion, and every entity on volume 1's spine is either present or on a dated deferral list.

Shows up in. `RPT05 Data Ownership`, which lists whatever `O20` holds, with blank ownership on anything new.

Goes wrong when. The list is extended to twenty-five entities to look thorough. Volume 1's test is whether a room argues about it; if nobody argues, it is an attribute.

Step 5.4 — Fix the entity relationships, which are a chain and should be a graph

Produce. An entity relationship set with correct verbs and correct cardinality, held in one place.

You need. Step 5.3 complete.

Look it up. Volume 1, chapter 9, *The buying chain, which is not a sequence*, and *Location, down to the bin*.

Where it lands.

> **`O36 Data Entity Relationship · R19 Data Entity relates to Data`**
>
> `O36` holds `Ref`, `From entity`, `Relationship`, `To entity`, `Cardinality`. `R19` holds `Ref`, `From`, `Relationship`, `To`, `Cardinality` and the **same eleven rows**. `Relationship` takes `references`, `has` and `is composed of`. Endpoints are entity **names**, not refs, so a renamed entity silently breaks both sheets.

How.

1. Read the eleven rows in order and see what they are. `REL-01` joins entity 1 to entity 2, `REL-02` joins 2 to 3, and so on to `REL-11`, which joins 11 to 12. It is the sheet order turned into a chain, and it produces "Product Hierarchy is composed of Store / Location", "Planogram is composed of Inventory Position" and "Purchase Order is composed of Customer Order".
2. Delete all eleven and rebuild from the spine. A retail entity model is a graph around Article, Price, Location, Supplier, Party, Inventory movement and Transaction, not a line.
3. Fix the cardinalities. `REL-01` gives SKU to Product Hierarchy as `1:1`; it is `N:1`, and if the group runs six hierarchies it is `N:M` through a hierarchy node.
4. Add the relationships that carry the retail meaning: Article to Assortment through Location and period, Price to Article and Location and channel and period, Inventory movement to Location and Article, Transaction to Party and Promotion.
5. Give `E13 Loyalty Account` and `E14 Supplier` at least one relationship each. Neither appears anywhere in the eleven rows, which makes two of fourteen entities unreachable.

Signed by. Chief Data Officer (`ORG-06`). The business owners sign the verbs, because "is composed of" and "references" mean different things to a developer.

Done when. Every entity appears in at least one relationship, no relationship joins adjacent sheet rows by accident, and each cardinality has been argued about by two people.

Shows up in. Nothing reads either sheet. The relationship set is invisible to all thirteen reports, which is why a chain survived delivery.

Goes wrong when. The sheet is filled from a database diagram. A conceptual relationship set is about meaning and a foreign key is about storage, and a retailer's price table has keys that correspond to no business relationship at all.

Step 5.5 — Build the logical and physical layers, and connect them

Produce. `O21 Logical Data Component` grouping entities the way a project will build them, `O22 Physical Data Component` naming where they actually sit, and a recorded join between the two.

You need. Step 5.3, and `O23 Application Component`.

Look it up. Volume 1, chapter 10, *The analytics layer, and the object everybody draws too small*.

Where it lands.

> **`O21 Logical Data Component` · `O22 Physical Data Component`**
>
> `O21` holds `Ref`, `Logical data component`, `Entity`, `Description`. `O22` holds `Ref`, `Physical data component`, `Application`, `Type`. **Nothing joins them.** `O21` points at an entity, `O22` points at an application, and there is no column on either that names the other. The logical-to-physical trace an architect needs does not exist in this repository.

How.

1. Add `Logical component` to `O22`, or a small relationship sheet, and fill it. Without it Phase D cannot tell you what breaks when a database moves.
2. Rebuild `O21` as groupings, not as one component per entity. Twelve rows named "`<entity>` store" is the entity list with a suffix, and it leaves `E13 Loyalty Account` and `E14 Supplier` with no logical component at all.
3. Group by how data is built and consumed: an item and hierarchy store, a price and promotion store, a stock ledger, a customer and consent store, a transaction store, an analytics layer.
4. Fix `Type` on `O22`. Four of the twelve physical components are typed `File`, including `PDC-02 POS database` and `PDC-12 Retail Analytics database`. A till journal is a file; a POS database is not, and an analytics platform on a file store is a finding or a typing error.
5. Record the analytics layer as several components. It is where the retail volume problem lives, and one row called "`Retail Analytics database`" hides every lineage question in

the phase.

Signed by. Chief Data Officer (`ORG-06`) for `O21` ; the application owner from `O23` for each `O22` row.

Done when. Every logical component names the entities it holds, every physical component names its logical parent, and no entity in scope lacks a logical component.

Shows up in. `RPT02 Application Portfolio` shows the applications, not their data. No report reads `O21` or `O22` .

Goes wrong when. The logical layer is generated from the conceptual layer one-for-one, which is what has happened here. A component that mirrors an entity teaches nobody anything and costs a maintenance cycle to keep.

Step 5.6 — Record attributes only where they earn it

Produce. Attribute rows for the entities where an attribute is contested, a key, or a legal obligation — and no rows at all for the rest.

You need. Step 5.1's attribute boundary.

Look it up. Volume 1, chapter 10, *Attributes, where they earn their place*, and *Keys, and why the barcode is not one*.

Where it lands.

> **`O37 Data Attribute`**
>
> `Ref` , `Entity` , `Attribute` , `Type` , `Key` , `Classification` . Sixty rows, six per entity, for `E01` to `E10` only — **`E11 Transaction / Basket`** , **`E12 Customer`** , **`E13 Loyalty Account`** and **`E14 Supplier`** **have no attributes recorded at all.** There is **no `Mandatory`** , **no `Length`** , **no `Source`** , **no `Retention`** and **no `Special personal information`** **flag.**

How.

1. Delete the generated set. Every entity carries the same six attributes — `Identifier` , `Name` , `Status` , `Effective date` , `Owner` , `Classification` — and `Key` reads `Yes` on both `Identifier` and `Name` , which asserts a two-part key of identity plus description on every entity in the model.
2. Record the attributes that are argued about: GTIN and NAPPI, unit of measure, VAT rate and zero-rating, shelf life and date code, allergen set, scheduled-substance status, size and colour, cost and retail price, promotional funding source.
3. Record the keys properly. A barcode is not a primary key: a GTIN identifies a trade item at a packaging level, and is reused and reassigned under the management rules [10].
4. Add `Special personal information` and set it on the customer attributes that attract it — health, biometric, children. POPIA treats those differently and the repository has no way to

say so.

5. Give `E12 Customer` its attributes first. It has none, and it is the entity with the most legal weight in the model.

Signed by. Each entity's business owner. The pharmacy attributes by the responsible pharmacist, because scheduling status is a statutory field.

Done when. No entity carries a generated attribute set, and `Key` names one key per entity.

Shows up in. Nothing reads `037`. Sixty rows of it, and no report.

Goes wrong when. The sheet becomes a data dictionary. That is the physical model's job, it runs to tens of thousands of rows, and a repository that tries to hold it stops being read.

Step 5.7 — Name one owner and one master system per entity

Produce. Fourteen rows where `Owner` is a position that exists in `011 Role` and `Master system` is one application from `023` that the owner will defend.

You need. Step 5.3. The owning executives in one room. `023 Application Component`.

Look it up. Volume 1, chapter 10, *Ownership, lineage and the gate that actually works*, and *Reference and master data, and the five distribution channels*.

Where it lands. `Owner` and `Master system` on `020 Data Entity`, read through `RPT05 Data Ownership`. `Owner` is free text; `Master system` is an application name, not a ref, so it does not join to `023` and nothing checks it.

TWO COLUMNS, ONE MEANING, TEN DISAGREEMENTS

ENTITY		020 MASTER SYSTEM	043 GOLDEN SOURCE
E01	SKU / Article	POS	≠ Demand Forecasting & Repl.
E02	Product Hierarchy	Retail Analytics	≠ Supplier Portal
E03	Store / Location	Loyalty & CRM	≠ Merchandising System
E04	Assortment	WMS	≠ Supplier Portal
E05	Price	Workforce Management	≠ POS
E06	Promotion	Retail Analytics	≠ Space & Range Planning
E07	Planogram	Loyalty & CRM	≠ Price & Promotion Engine
E08	Inventory Position	E-commerce Platform	≠ WMS
E09	Purchase Order	WMS	≠ Order Management System
E10	Customer Order	Price & Promotion Engine	≠ Loyalty & CRM

Ten rows. Ten disagreements. No report reads either column against the other.

Figure 5.3 `Master system` on `020 Data Entity` against `Golden source` on `043 Master Data Rule`, as shipped. The two columns have to mean the same thing, and they agree on none of the ten entities `043` covers. Whichever one you keep, derive or delete the other.

How.

1. Fix `Owner` first. Twelve of the fourteen values are job titles that exist nowhere in the repository — `Head of Finance`, `Head of Buying`, `Head of Merchandising`, `Head of Pricing`, `Head of Store Operations`, `Head of Order Fulfilment`. Only `Data Steward`

resolves, to `ROL-16`. Either add the roles to `O11` in Phase B or replace the values with roles that exist.

2. Reassign the obvious errors. `Head of Finance` owns four entities including the article master and the loyalty account; `Head of Pricing` owns `E04 Assortment` and `E11 Transaction / Basket` but not `E05 Price`; `Head of Merchandising` owns `E03 Store / Location` and no merchandise entity.
3. Fix `Master system` row by row, with the owner present. `E01 SKU / Article` is mastered in `POS`; `E03 Store / Location` in `Loyalty & CRM`; `E04 Assortment` in `WMS`; `E05 Price` and `E12 Customer` in `Workforce Management`; `E09 Purchase Order` in `WMS`; `E13 Loyalty Account` in `Supplier Portal`. Not one of those is where the data lives.
4. Resolve the ten-for-ten contradiction with `O43 Master Data Rule`. The `Golden source` column there disagrees with `Master system` on `O20` for **every one of the ten entities it covers**. Two columns that must mean the same thing and agree nowhere is the single worst defect in this repository, and whichever you pick, delete or derive the other.
5. Make `Lineage` an assertion somebody can demonstrate. It takes `Full`, `Partial` and `Absent`, and `ZZ Checks` reports two entities without lineage — `E04 Assortment` and `E11 Transaction / Basket`. Six more are `Partial` and the check ignores them, so the honest number is eight of fourteen.

Signed by. `GB-03 Data Governance Council`, whose stated mandate is to designate data owners and resolve disputes. This is the step that mandate exists for.

Done when. Every `Owner` resolves to a role in `O11`, every `Master system` resolves to an application in `O23`, `O43`'s `Golden source` agrees with `O20` on every row, and every `Full` lineage claim has been demonstrated once.

Shows up in. `RPT05 Data Ownership`, which reprints `O20` and recomputes the footer. When the footer reads "0 of 14" and you believe it, this step is done.

Goes wrong when. `Master system` is set to the system the business talks about rather than the one that writes the record. The test is a single question: if two systems disagree about this entity tomorrow morning, which is right by policy? If nobody can answer in a sentence, the entity has no master.

OWNERSHIP, BEFORE THE ROOM LEAVES

For each entity in scope: if two systems disagree tomorrow morning, which one is right by policy?

Which `Owner` value in `RPT05` resolves to nobody, and who will you put there instead?

Form C-1 — Entity catalogue, part A: identify

One row per conceptual entity. Columns are the first four of `O20 Data Entity` in sheet order. A first pass needs eighteen to twenty-two rows. Signed by the Chief Data Officer, with each entity's business owner signing its `Definition`. Part B below covers the assessment columns.

Ref	Data entity	Class	Definition
E15	Inventory Movement — example, delete me	Transactional	Why a position changed, on whose authority. Append-only and reason-coded.

Form C-2 — Entity catalogue, part B: assess

The same rows as form C-1. Columns are the last four of `O20 Data Entity` in sheet order, with `Ref` repeated so the two forms join. Fill it with the owning executives in the room, not afterwards. Signed by `GB-03 Data Governance Council`.

Ref	Data entity	Classification	Owner	Master system	Lineage
E15	Inventory Movement — example, delete me	Internal			Absent

Step 5.8 — Rewrite the master data rules, and make `Golden source` agree with `O20`

Produce. One mastering rule per master entity, with a golden source that matches `Master system` and a rule type that says what the rule does.

You need. Step 5.7 complete and signed.

Look it up. Volume 1, chapter 10, *Reference and master data, and the five distribution channels*.

Where it lands.

> **`O43 Master Data Rule`**
>
> `Ref`, `Entity`, `Rule type`, `Description`, `Golden source`. `Rule type` takes `Validation`, `Match rule`, `Survivorship` and `Golden source` — and that last value duplicates the column beside it. Ten rows covering `E01` to `E10`, so **`E11 Transaction / Basket`, `E12 Customer`, `E13 Loyalty Account` and `E14 Supplier` have no mastering rule.**

How.

1. Make `Golden source` a formula pointing at `Master system` on `O20`, or delete it. Ten rows, ten disagreements, is not a data-quality issue; it is two architectures in one workbook.
2. Drop `Golden source` from the `Rule type` list. A rule type says what kind of rule this is; naming the source is the other column's job.
3. Replace the ten descriptions. Every one reads "Rule governing `<entity>` mastering", which is the entity name in a sentence. A real rule says what happens: which field wins on conflict, what match confidence creates a duplicate, what validation blocks a save.
4. Add rules for the four entities with none. `E12 Customer` needs a match rule and a survivorship rule before any loyalty consolidation can be designed, and `E14 Supplier` needs one before supplier income can be reconciled.
5. Write the distribution rule next to each mastering rule. Mastering says where it is written; distribution says how the other eleven systems get it, and in a retailer the answer differs by entity — the article file goes out nightly, the price file to a deadline, the promotion file on approval.

Signed by. `GB-03 Data Governance Council`, with the master system's application owner from `O23` accepting the operational load.

Done when. Every master entity has a rule, every rule describes a behaviour rather than restating the entity, and `Golden source` and `Master system` cannot disagree.

Shows up in. Nothing reads `O43`. Its contradiction with `O20` is invisible to every report, which is how it shipped.

Goes wrong when. Rules are written for the entities with an obvious system and skipped for the contested ones, which are the reason the sheet exists.

Step 5.9 — Make the metadata sheet say what is true

Produce. `042 Metadata Element` with a `Coverage` value per element that an auditor could test, and a plan for the four that read `Absent`.

You need. Step 5.7. The data catalogue tool, if there is one.

Look it up. Volume 1, chapter 10, *What a single version of the truth has to mean*.

Where it lands.

> **`042 Metadata Element`**
>
> `Ref`, `Metadata element`, `Category`, `What it records`, `Coverage`. `Category` takes `Business`, `Governance`, `Technical` and `Operational`. `Coverage` takes `Full`, `Partial`, `Absent` and the honest value `Recorded, not held`.

How.

1. Read the four `Absent` rows and decide which matter: `MTD-03 Steward`, `MTD-09 Schema`, `MTD-10 Primary key`, `MTD-14 Quality result`. Steward and quality result matter most, because without them no ownership claim is testable.
2. Resolve `MTD-02 Business owner` reading `Recorded, not held` against `O20`'s fourteen filled `Owner` values. One of the two sheets is wrong about whether ownership is held.
3. Resolve `MTD-08 Lineage` reading `Partial` against `O20`, where six entities claim `Full`. A metadata sheet and an entity sheet disagreeing about lineage is the kind of contradiction `ZZ Checks` was built to catch and does not.
4. Raise `MTD-07 Consent basis` from `Partial` to `Full` before any marketing work starts. Step 5.11 depends on it entirely.
5. Add the two elements a retailer needs and the sheet lacks: effective-dating method, and retail-calendar alignment. Volume 1 is clear that a retail week is not a calendar week, and a metadata layer that cannot say which calendar a figure is on will restate numbers it did not mean to.

Signed by. Chief Data Officer (`ORG-06`).

Done when. Every `Coverage` value has a test behind it, and no two sheets disagree about whether owner, lineage or consent is recorded.

Shows up in. No report reads `042`, so its fourteen rows are a self-assessment nobody audits. Put the `Absent` four into `O29 Gap`.

Goes wrong when. `Coverage` is read off a catalogue tool's completeness percentage. A tool holding a field is not a business maintaining it.

LINEAGE AND METADATA, HONESTLY

Which of the six `Full` lineage claims on `020` could you demonstrate this week?

`042` says business owner is recorded but not held, `020` holds fourteen owners. Which sheet is lying?

Step 5.10 — Rebuild the data sets as retail reporting sets

Produce. `038 Data Set` holding the curated sets this business actually publishes, each with the entities it draws on and the audience it serves.

You need. Steps 5.3 and 5.5.

Look it up. Volume 1, chapter 6, *Like-for-like, and why the definition is a data-model requirement*.

Where it lands.

> **`038 Data Set`**
>
> `Ref`, `Data set`, `Description`. Three columns and six rows. There is **no `Entities`, no `Owner`, no `Refresh`, no `Audience` and no `Retention`.** Add `Entities`, `Owner` and `Refresh` at least, or a data set is a name.

How.

1. Read the six shipped rows and notice where they came from. `DST-05 Risk measurement set — Exposure and position data` is banking language, and nothing on the sheet is recognisably retail.
2. Replace them with the sets a retailer publishes: daily trade, availability and gaps, margin and markdown, shrink and loss, promotional performance, supplier income, and the statutory submissions.
3. Define like-for-like as its own set with its own exclusion policy. Volume 1 treats the definition as a data-model requirement, and a group whose like-for-like rule lives in a spreadsheet cannot restate a prior year.
4. Add `Refresh` and be honest about it. A daily trade set landing at 06:30 and a shrink set landing six weeks after a count are different kinds of object.
5. Name the regulatory set's regulator. `DST-01 Regulatory reporting set` says "statutory submissions" and names nobody.

Signed by. Chief Financial Officer (`ORG-02`) for the trading and margin sets; Chief Data Officer (`ORG-06`) for the rest.

Done when. Every set names its entities, its owner and its refresh, and the like-for-like exclusion policy is written down and signed.

Shows up in. Nothing. `O38` feeds no report.

Goes wrong when. Data sets are modelled as the reports that already exist, which makes every new report a new data set.

Step 5.11 — Turn POPIA into columns, not a policy document

Produce. The four facts POPIA needs recorded per personal-data entity — lawful basis, consent state, retention clock and special-information flag — plus a decision on each of the four retail processing activities a regulator will ask about.

You need. Step 5.7. The Information Officer. The marketing consent extract.

Look it up. Volume 1, chapter 15, *POPIA, with the retail bite*, and chapter 10, *Classification, consent and POPIA*. This workbook does not re-teach either.

Where it lands. `Classification` on `O20 Data Entity`, `MTD-07 Consent basis` on `O42 Metadata Element`, and the columns you add to `O20`: `Lawful basis`, `Retention`, `Special personal information`. `O41 Security Control` and `O64 Security Policy` carry the controls; `RPT09 Regulatory Coverage` carries the obligation and names no statute.

How.

1. Fix `Classification` on the rows that are inverted. `E12 Customer` is classified **`Public`**, as are `E04 Assortment` and `E07 Planogram`, while `E01 SKU / Article` is `Restricted`. A customer record classified public is a notifiable finding on its own, and the range plan is the most commercially sensitive object on the sheet.
2. Record the lawful basis per entity and per purpose. Loyalty enrolment, transaction processing and direct marketing are three purposes over one data set, and consent for one is not consent for another [7].
3. Decide the in-store processing questions in writing: CCTV retention and signage, any biometric or facial matching at the door or the till, loss-prevention watch lists, and whether a shelf camera counts as personal information when it captures a shopper.
4. Record the automated-decisioning positions. Personalised pricing, loyalty tier assignment, credit screening in store and fraud scoring on returns are all decisions made about a person by a system, and each needs a stated basis and a human-review route.
5. Set the retention clock per entity and take the longest. Tax wants an invoice kept, the Consumer Protection Act creates per-sale clocks, pharmacy records have their own, and marketing consent has none of those. The longest clock wins, and it is not one clock.

Signed by. The Information Officer, countersigned by General Counsel (`ORG-12`) and `GB-03 Data Governance Council`.

Done when. No personal-data entity is classified `Public`, every purpose has a lawful basis written against it, and the four in-store questions have a dated answer.

Shows up in. `RPT09 Regulatory Coverage` shows `CO-04 Data protection and marketing consent` with `Evidence` = `Partial`. That stays `Partial` until this step is done and the evidence is named.

Goes wrong when. POPIA is handled as a policy document and never reaches a column. A policy cannot answer "which entities hold special personal information"; a column can, and that is the question the regulator asks first.

Step 5.12 — Model price and promotion as data, and build the shelf-label control

Produce. `E05 Price` modelled as a dated, channel-aware, location-aware version set; the promotion modelled as a funded offer with mechanics; and one named control that detects a shelf label which failed to update.

You need. Steps 5.3 and 5.7. The price change calendar. The electronic shelf label estate, if there is one.

Look it up. Volume 1, chapter 9, *Price is an entity, not an attribute*; chapter 15, *Section 23, and why a stale shelf label is a legal exposure*; chapter 16, *Price, which the notation cannot date*.

Where it lands. `E05 Price` and `E06 Promotion` on `O20 Data Entity`; `X04 Promotion` on `EXT Industry Extension`, which exists; and **no `X` type for price at all**, which is the omission worth naming out loud.

Figure 5.4 A price change as six timed events. The fourth can fail without anyone being told, and the fifth is the control — a reconciliation of price file against label estate against till, per location, with a named escalation inside the trading day.

How.

1. Model price as a version, not a value. Price by article, by location or zone, by channel, from an effective date and time, with the preceding version retained. A price column on an article

row cannot answer "what was the price on Saturday", and the Consumer Protection Act question is always about a moment.

2. Add the price extension type `EXT` left out. Promotion, planogram and assortment got types; price, which more systems read than any other object in a retailer, did not.
3. Model the promotion as the offer and the outcome separately. What was offered, who funded it and what actually fired in the basket are three facts, and supplier-funded promotions reconcile against the third.
4. Build the control. Where two prices are displayed for the same goods, the lower binds the retailer [5][6]. So the control is not "update labels": it is a reconciliation that compares the price file to the label estate and the till, per location, after every price change, and reports any article where the three disagree. Give it an owner, a frequency tied to the price change calendar, and an escalation that reaches a store manager the same trading day.
5. Record the control in `041 Security Control` or `063 Control Objective`, and link it to `CO-01 Consumer protection and pricing accuracy` on `RPT09 Regulatory Coverage`, which currently shows `Evidence` = `Partial`.

Signed by. The pricing manager from step 4.4 owns the price model; General Counsel (`ORG-12`) owns the control, because the exposure is legal rather than operational.

Done when. Price is a dated version set, promotion separates offer from outcome, and the label reconciliation exists with an owner, a frequency and an escalation route.

Shows up in. `RPT09 Regulatory Coverage` against `CO-01`. It reads `Partial` as shipped, and a named reconciliation with evidence is what moves it.

Goes wrong when. The shelf label is treated as a data-quality problem with a tolerance. It is not: a label that silently fails to update is a price the retailer is bound by. Note also that `A08 Price & Promotion Engine` is graded `Critical` on `O23`, in `Poor` health, in service since 2022 and dispositioned `Retire` — four facts that cannot all be sensible together, and the control has to survive whatever replaces it.

PRICE, PROMOTION AND THE LABEL

What was the price of a given article in a given store last Saturday at 14:00, and which system can answer?

Who finds out, and how soon, when a label and the till disagree?

Which entity holds what actually fired in the basket, as opposed to what was offered?

Step 5.13 — Record the dual identifier problem in a pharmacy estate

Produce. An identifier model for the pharmacy article that carries both schemes, names the owner of the mapping between them, and states what happens when they disagree.

You need. Step 5.2's grain decision. The dispensing system's item file. The claims reconciliation team.

Look it up. Volume 1, chapter 2, *Two identifier schemes for the same product*, and chapter 3, *The identifiers, and the two defects that cause most of the pain*.

Where it lands. Attributes on `E01 SKU / Article` in `037 Data Attribute`, and a mastering rule on `043 Master Data Rule`. There is no identifier entity and no cross-reference sheet, so the mapping is an attribute pair plus a rule.

TWO IDENTIFIER SCHEMES FOR ONE PHARMACY ITEM

	GTIN	NAPPI
Who maintains it	GS1 and the brand owner	MediKredit
What it is used for	Receiving, scanning, selling	Claiming and reimbursement
Changes when	Pack or trade item changes	Scheme listing changes
Cardinality to the item	One per packaging level	Several across pack sizes
Is it a primary key	No — reused and reassigned	No — not unique to a GTIN
Who owns the mapping	Nobody, unless you name them	Nobody, unless you name them

The bottom row is the step. The retailer owns a bridge nobody else maintains.

Figure 5.5 The dual-identifier problem in a pharmacy estate, as a table an architect can take into a workshop. Neither scheme is a primary key, the two do not map one to one, and the last row is the one that has to be answered by a named role.

How.

1. Record both identifiers as attributes, with different owners. The GTIN is used for receiving, scanning and selling; the NAPPI code is used for claiming and reimbursement [9].
2. State that they do not map one to one, and record the cardinality you actually have. One dispensed item can carry several NAPPI codes across pack sizes, and a GTIN change does not trigger a NAPPI change.
3. Name the owner of the mapping. The two schemes are maintained by different bodies on different cadences, and the retailer owns a bridge nobody else maintains. If that owner is not a named role in `011`, the bridge has no owner.
4. Write the disagreement rule: what the till does, what the claim does, and which one blocks. A mismatch that blocks a sale and a mismatch that blocks a claim are different incidents with different costs.
5. Record the content-currency obligation alongside it. Drug-interaction software has to be the current version, which turns a content database into a compliance item on the critical path — and an offline store is the case to design for.

Signed by. The responsible pharmacist for the scheme definitions; Chief Data Officer (`ORG-06`) for the mapping ownership.

Done when. Both identifiers are recorded with owners, the mapping has a named owner in `O11`, and the disagreement rule is written down.

Shows up in. `RPT09 Regulatory Coverage` under `CO-05 Product safety and recall`, which reads `In breach` as shipped. A recall needs an identifier that resolves to stock on a shelf, which is what this step provides.

Goes wrong when. One scheme is chosen as the key and the other recorded as a note. The claim side then reconciles by hand, every month, for years.

Step 5.14 — Fill `R05`, and fix the half of it that uses the wrong key

Produce. `R05 Application manages Data En` recording, for every entity, the one application that masters it and every application that consumes it.

You need. Step 5.7, and `O23 Application Component`.

Look it up. Volume 1, chapter 19, *Application to data, with the system of record*.

Where it lands.

> **`R05 Application manages Data En`**
>
> `Application`, `Application name`, `Data entity`, `Entity name`, `Role`. `Role` takes `Master` and `Consumer` only — there is **no `Creates`, no `Updates` and no `Reads`**, so the sheet cannot hold a CRUD matrix.

How.

1. Fix the key. The fourteen `Master` rows put an application **name** in the `Application` column — `POS`, `Retail Analytics`, `WMS` — while the fourteen `Consumer` rows put a ref, `A01`, `A04`, `A07`, `A10`. Half the sheet's foreign keys do not resolve.
2. Rebuild the `Master` rows from `O20`'s corrected `Master system`, once step 5.7 is signed. They currently mirror the uncorrected values, so every error in `O20` is duplicated here.
3. Rebuild the `Consumer` rows. They cycle `A01`, `A04`, `A07`, `A10` and nothing else, which leaves eight of twelve applications consuming no data at all — including `A02 POS`, `A03 E-commerce Platform` and `A08 Price & Promotion Engine`.
4. Add the `Role` values the sheet needs, then fill the matrix from interface evidence rather than from memory. Volume 1's appendix A publishes a process-to-data CRUD map; use it as the candidate set and map by name.
5. Check the single-master rule. Every entity has exactly one `Master` row or the mastering decision in step 5.7 was not real.

Signed by. Each application's owner from `O23`, with the data owner from `O20` accepting the master row.

Done when. Every `Application` is a ref that resolves, every entity has one master, and no application in `O23` is absent from the sheet.

Shows up in. No report reads `R05` directly, but `RPT13 Orphan Analysis` reads `R04` only, so an application in `R05` and not `R04` is reported as an orphan while holding data. `A12 Retail Analytics` is exactly that case.

Goes wrong when. `R05` is filled before `O20`'s `Master system` is corrected, which turns one sheet of errors into two.

Step 5.15 — Make the migration sheets usable, or mark them as placeholders

Produce. `O39 Data Migration` and `O40 Source To Target Mapping` holding either real migrations with volumes and cutover dates, or a written statement that they are empty by design until Phase F.

You need. Steps 5.7 and 5.14. The Phase E work package list, when it exists.

Look it up. Volume 1, chapter 10, *The restatement problem*, and *Late-arriving store data*.

Where it lands.

> **`O39 Data Migration` · `O40 Source To Target Mapping`**
>
> `O39` holds `Ref`, `Entity`, `From system`, `To system`, `Status`. `O40` holds `Ref`, `Entity`, `Source field`, `Target field`, `Rule`. There is **no `Volume`, no `Cutover date`, no `Reconciliation` and no `Owner`** on either, and `Status` has no `Complete` value.

How.

1. Read `O40` and see that it holds no fields. Every `Source field` reads "`<Application>` field" and every `Target field` the same — twelve rows of placeholder text where the field names should be. A mapping sheet with no fields cannot be tested.
2. Read `O39`'s six rows against `O20`. `MIG-01` moves `SKU / Article` from `POS` to `Order Management System`, which is a migration between two systems that should master neither. All six rows step through the application list in order.
3. Delete both sets or mark the sheets `Placeholder` in a column you add. A placeholder labelled as one is honest; a placeholder that looks like a plan is dangerous in Phase F.
4. Add `Volume`, `Cutover date`, `Reconciliation rule` and `Owner` to `O39` before the first real migration row. A migration without a reconciliation rule is a data loss waiting for an audit.
5. Add `Complete` to `Status`, so a finished migration can be recorded as finished.

Signed by. Chief Data Officer (`ORG-06`), with the receiving application's owner accepting the reconciliation rule.

Done when. Every row either names real fields and a reconciliation rule, or the sheet is marked a placeholder with a date by which it will be filled.

Shows up in. Nothing. Neither sheet feeds a report, which is why twelve rows of placeholder text passed delivery.

Goes wrong when. The placeholder rows are inherited into a Phase F plan because they look populated. Twelve rows of "`<Application>` field" is this repository's clearest example of structure without content.

Step 5.16 — Run the Phase C data checkpoint

Produce. A signed data architecture: the entity catalogue, the ownership table, the master data rules, the POPIA columns, the price control, and the list of defects you found and did not fix.

You need. Steps 5.1 to 5.15 complete for the entities in scope.

Where it lands. `DLV-07 Data Architecture` and `DLV-08 Data Model` on `O31 Deliverable`. Both arrived `Approved`; set the status yourself.

How.

1. Re-run `ZZ Checks` and read `Data entities without lineage` against `O20`. It reports 2. Six more are `Partial`. Record the threshold definition as a finding, because a check that counts only `Absent` will always look better than the estate is.
2. Run the five checks no report performs: entities with no logical component, entities in no relationship, `Owner` values that resolve to no role, `Master system` values that resolve to no application, and `Golden source` values that disagree with `Master system`.
3. Read `RPT05 Data Ownership` as a stranger would and ask whether you would show it to the Information Regulator with `E12 Customer` classified as it is.
4. Count your `O29 Gap` rows with `Domain` = `Data`. There are none as shipped — the single gap row is `Business`. Fewer than four after this chapter and you have under-reported.
5. Write the defect list, and put the `O20`-versus-`O43` contradiction at the top of it.

Signed by. `GB-03 Data Governance Council` for the ownership and the rules; `GB-01 Architecture Board` at gate `GT-1 G1 Design` on `O58 Governance Gate` for the phase.

Done when. `RPT07 Gap Analysis` carries data-domain rows, the two deliverables carry a status you set, and every `Full` lineage claim has been demonstrated once in front of a witness.

Shows up in. `RPT01 Dashboard`, `RPT05`, `RPT07 Gap Analysis` and `RPT09 Regulatory Coverage`. If `RPT05`'s footer still reads "2 of 14" and you have not argued with the number, the checkpoint did not happen.

Goes wrong when. Phase C closes on a complete-looking `O20` with fourteen owners who do not exist and fourteen master systems that do not hold the data. Completeness is not the test. Somebody being willing to defend a row is the test.

What a finished Phase C data architecture looks like in the numbers

Reading	As delivered	After Phase C
`O20` `Owner` values that resolve to a role	1 of 14	14 of 14
`O20` `Master system` values that hold the data	0 of 14	14 of 14
`O20` versus `O43` agreement on the source	0 of 10	10 of 10
`O20` `Class` values available	2, no `Reference`	3, with the code lists on it
Personal-data entities classified `Public`	1, `E12 Customer`	0
Entities with no relationship in `R19`	2, `E13` and `E14`	0
`O40` rows naming a real field	0 of 12	All of them, or the sheet marked a placeholder
`R05` rows whose `Application` resolves	14 of 28	28 of 28
Applications consuming no data in `R05`	8 of 12	0
`O29 Gap` rows, `Domain` = `Data`	0	4 or more

PHASE C — A REVIEWER'S QUESTIONS

Which system masters the article file, and what is the sentence that proves it rather than asserts it?

Which entity's definition do you expect a project to reopen within a year, and why?

Name the control that catches a shelf label which failed to update, and who it escalates to.

Which of your `Full` lineage claims have you personally watched somebody demonstrate?

How many data-domain gaps did Phase C raise, and which one would you fund first?

THE CHAPTER IN TEN LINES

- Two columns decide Phase C: `Definition` on `O20`, which settles what a word means, and `Master system`, which says who may say it.
- Settle the article, style, option and SKU argument in writing before anything else. Every availability and margin number is computed at that grain.
- `O20`'s `Class` has `Master` and `Transactional` and no `Reference`, so the code lists have nowhere to live.
- Twelve of fourteen `Owner` values are job titles that exist in no sheet. Only `Data Steward` resolves.
- `Master system` is wrong on every row that matters: the till masters the article file, Workforce Management masters price and customer.
- `O43`'s `Golden source` disagrees with `O20`'s `Master system` on all ten rows it covers. That is the worst defect in the repository.
- `E12 Customer`, `E04 Assortment` and `E07 Planogram` are classified `Public`. One of those is a notifiable finding, the other two are commercial.
- `ZZ Checks` reports two entities without lineage. Six more are `Partial` and the check ignores them, so the honest number is eight of fourteen.
- `O36` and `R19` hold the same eleven rows, and those rows are the sheet order turned into a chain. Rebuild from the spine.
- `E13 Loyalty Account` and `E14 Supplier` appear in no relationship and have no logical data component.
- Nothing joins `O21` to `O22`, so the logical-to-physical trace does not exist. Add a column before Phase D needs it.
- `O37` holds the same six generated attributes per entity, with `Key` = `Yes` on both identifier and name, and nothing for Customer.
- POPIA belongs in columns: lawful basis, consent state, retention clock, special personal information. Point at volume 1 chapter 15 for the law.
- `EXT` gives Promotion a type and gives price none, in a sector where price is the most contested object in the estate.
- Model price as a dated version set, and build a reconciliation of price file against label estate against till. A stale label is a price the retailer is bound by.
- A pharmacy item carries GTIN and NAPPI, maintained by different bodies, mapping imperfectly, with the bridge owned by the retailer and nobody else.
- `R05`'s fourteen `Master` rows key on application names and its fourteen `Consumer` rows on refs. Half the sheet does not resolve.
- Eight of twelve applications consume no data in `R05`, including the POS, the e-commerce platform and the price engine.
- `O40`'s twelve rows name no fields — every one reads "`<Application>` field". Mark the sheet a placeholder or fill it.

- `O29 Gap` holds no data-domain row. Fewer than four after this chapter and Phase C under-reported.

SOURCES FOR THIS CHAPTER

- **1.** Retail_TOGAF_Metamodel.xlsx, Phase C data sheets O20, O21, O22, O36, O37, O38, O39, O40, O42, O43, O44 and R05, R19 (Halverson Stores Group, illustrative data)
- **2.** Retail_TOGAF_Metamodel.xlsx, sheets RPT05 Data Ownership, RPT09 Regulatory Coverage, RPT13 Orphan Analysis and ZZ Checks
- **3.** Retail_TOGAF_Metamodel.xlsx, sheets O06 Requirement, O23 Application Component, O29 Gap, O31 Deliverable, O32 Standard, O41 Security Control, O57 Governance Body and EXT Industry Extension
- **4.** The Open Group — TOGAF Standard, 10th Edition: ADM Methods, Phase C Information Systems Architectures https://pubs.opengroup.org/architecture/togaf10-doc/adm/
- **5.** Consumer Protection Act 68 of 2008, section 23 — disclosure of price of goods or services https://www.acts.co.za/consumer-protection/23_disclosure_of_price_of_goods_or_services
- **6.** Consumer Goods and Services Ombud — Advisory Note 2: Disclosure of Price of Goods or Services: Pricing Errors, 1 March 2021 https://www.saflii.org/za/cases/ZACGSO/2021/2.html
- **7.** Information Regulator (South Africa) — Guidance Note on Direct Marketing in terms of the Protection of Personal Information Act 4 of 2013 https://inforegulator.org.za/wp-content/uploads/2020/07/GUIDANCE-NOTE-ON-DIRECT-MARKETING-IN-TERMS-OF-THE-PROTECTION-OF-PERSONAL-INFORMATION-ACT-4-OF-2013-POPIA.pdf
- **8.** Information Regulator (South Africa) — Guidance Note on Information Officers https://inforegulator.org.za/wp-content/uploads/2025/07/Guidance_Note_on_Information_Officers_.pdf
- **9.** MediKredit — NAPPI (scheme description, trademark ownership and file terms) https://www.medikredit.co.za/products-and-services/nappi/
- **10.** GS1 — GTIN Management Rules, decision support, and the General Specifications release 26.0 https://gs1.org/1/gtinrules/en/decision-support/change/4

CHAPTER 6

Application Architecture

The half of Phase C where the arguments get settled, because an application has a contract, a run team and somebody's name against it.

Phase C has two halves and they are not symmetrical [7]. The data half settles what the business means; the application half settles who is accountable for a thing with a contract, a run team and a renewal date. `O20 Data Entity` can be wrong for a year and nobody is embarrassed; a `Retire` against a system somebody's bonus depends on is contested within a day.

Halverson Stores Group, the retailer inside your repository, is an invention, as is Milkwood Retail Group in volume 1. Their data does not mix. Numbers here are Halverson's [1].

Start at the model, not the estate

The temptation is to open `O23 Application Component`, type in what the service desk knows about, and call the result an application architecture. Do not. An inventory has survivor bias built in: the systems easiest to see are the ones somebody already monitors, and in a retailer the capabilities that matter most are held in things nobody monitors. Volume 1's conceptual model gives you twenty-five capabilities, `A-01` to `A-25`, each with a mastering statement. Walk that list, ask who does this job here, write down the answer including `None`. Then open `O23`.

Look it up. Volume 1, chapter 11, *The table*, lists all twenty-five with purpose, user roles, data mastered and the typical system. Chapter 11, *The five boundaries that are actually contested*, gives the five arguments you will have in the first fortnight.

Map onto `O23` **by name, never by number**. Halverson's refs run `A01` … `A12` and volume 1's `A-01` … `A-25`; they look alike and mean different things. Halverson's `A08` answers volume 1's `A-15` and `A-16` together. Nothing in the repository warns you about the collision.

The walk against the shipped file: `A01` carries `A-01` and most of `A-03`; `A07` carries `A-04`; `A06` carries `A-07` and `A-08` in one box, the conflation volume 1 warns against; `A05` carries `A-10`; `A02 POS` carries `A-13` and claims `A-12`; `A08` carries `A-15` and `A-16`; `A09` carries `A-17`; `A03` and `A04` split `A-18` and `A-19`; `A12` carries `A-24`.

Then the holes. Nothing answers `A-02` product content, `A-05` merchandise financial planning and open-to-buy, `A-20` store task and execution confirmation, `A-22` pharmacy dispensing, `A-23` finance and supplier income, or `A-25` document and common services. Four of the five volume 1 calls usually missing are missing here, and `A-18`'s sourcing half is claimed by an `A04` reading `Poor` and `Replace`.

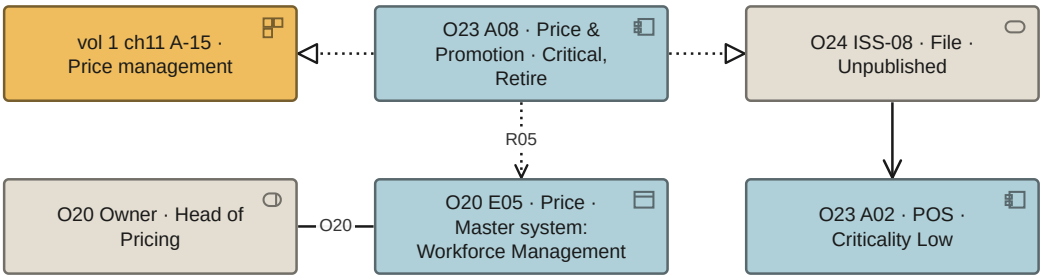


Figure 6.1 The application spine in ArchiMate 4, read against the shipped file. A conceptual capability from volume 1 chapter 11, the component that answers it, the service it exposes and the entity it is recorded as mastering — which `O20` says is somewhere else [2].

> **THE SPINE OF THE APPLICATION HALF OF PHASE C**
>
> Walk the conceptual model. Map the real estate onto it. Record what survives in `O23`. The count of `None` answers is the most useful number you produce this phase, and it is in no `RPT` sheet.

Step 6.1 — Lay the conceptual model across the estate

Produce. A one-page mapping from `A-01` … `A-25` to the systems you believe deliver each, with the blanks left visibly blank.

You need. Volume 1 chapters 11 and 12. Ninety minutes each with the heads of buying, merchandising, supply chain, store operations, pricing, e-commerce and finance — and the responsible pharmacist separately, if there is a dispensary.

Look it up. Volume 1, chapter 11, *How to use it*, describes the room that produces this: the twenty-five on a wall with their mastering statements, one question per box.

Where it lands.

> **`047 Reference Model Entry`**
>
> Four columns: `Ref`, `Reference model entry`, `Description`, `Status`. Eleven rows, five `Adopted` and six `Candidate`. The nearest thing to a home for this mapping, and not actually that: its rows are target patterns, not jobs to be done. Add nothing yet — keep the mapping as a working page and bring it back at step 6.15.

How.

1. Copy the twenty-five into a working list, the five usually-missing at the top, and against each write the system you expect **before** you research.
2. Hold the interviews. One question per entry: what does this job here?
3. Two systems, record two. A spreadsheet, record its filename. No answer, write `None`.
4. Count the `None` rows and the duplicates. Both numbers open the next conversation.

Signed by. Chief Enterprise Architect, with the Chief Information Officer (`ORG-05`) acknowledging the system column.

Done when. All twenty-five carry a system, a filename or `None` with a reason.

Shows up in. Nothing, which is the point: the page exists before any report can be trusted.

Goes wrong when. You map the model onto the estate instead of the reverse. The tell: every capability gets one system and nothing reads `None`.

The inventory itself

`O23 Application Component` is ten columns and that is all: `Ref`, `Application`, `Type`, `Description`, `Disposition`, `Criticality`, `Health`, `In service`, `Users`, `Owner` [2]. Twelve rows. No cost, vendor, hosting or lifecycle column, and no record-keeping columns anywhere in this repository.

Group	Columns	Who fills it	When
Identity	`Ref`, `Application`, `Type`, `Description`	Architect	Step 6.2
Accountability	`Owner`, `Criticality`	Architect with the owning exec	Step 6.3
State and size	`Disposition`, `Health`, `In service`, `Users`	Owning exec, finance, run team	Steps 6.3, 6.4, 6.7

`Type` takes three values: ten `Package`, one `Custom` (`A11`), one `Platform` (`A12`). It records build provenance, not whether a row is running or designed. To hold a target application — and in retail you always must — add a column. Do not borrow `Type`.

Step 6.2 — Open a row for every application you can name

Produce. `O23` populated with `Ref`, `Application`, `Type` and `Description` for every application, including the ones that do not exist yet.

You need. Step 6.1. The service-management asset list if there is one. `O09 Organization Unit` and `O11 Role` from Phase B, because step 6.3 cannot fill `Owner` against empty sheets.

Where it lands.

> **`O23 Application Component`**
>
> `Ref` is `Ann`, sequential, never reused. `Application` is what people in the building call it, not the vendor's product name. `Description` is one sentence on what the thing administers: `A01` reads "Article master, hierarchy, cost and retail", the merchandising boundary in seven words. `Type` from `Package`, `Custom`, `Platform`.

How.

1. One row per system named at step 6.1, in the words the business used.
2. Write `Description` about what it holds and posts, never about its technology.
3. Keep the grain honest. Twelve rows counts applications; four hundred is an asset register.
4. Keep vendor product names out of `Application`: products get renamed and resold, the job does not.
5. Add rows for the target applications that do not exist yet, flagged at step 6.4.

Signed by. Chief Enterprise Architect.

Done when. Every step 6.1 entry that resolved to a system has a row here, and each `None` has either a target row or a written decision not to build one.

Shows up in. `RPT02 Application Portfolio`, which reprints `O23` without `Description`. The row count should equal step 6.1's distinct systems plus your target rows, and nothing else.

Goes wrong when. Somebody loads the service-management export. Four hundred rows, twenty of them applications and the rest servers and scheduled jobs, and the sheet is never trusted again.

Step 6.3 — Fix the accountability columns before anybody quotes the sheet

Produce. `Owner` and `Criticality` filled by named people who agreed in front of each other.

You need. Step 6.2. `O09 Organization Unit`, twelve rows with a `Head` column. `O13 Business Capability`, whose level 1 rows carry `Criticality`. Two hours with the executive committee's delegates.

Where it lands.

> **`O23 Application Component`**
>
> `Owner` — the shipped file holds organisation unit names here (`Technology`, `Internal Audit`, `Commercial`), not people. `O05 Principle PRN-13` says an owner is a person, not a committee, so the column breaches a principle the same workbook publishes. Write the role title from `O09`'s `Head` column. `Criticality` from `Critical`, `High`, `Medium`, `Low`, derived from the highest `Criticality` of the capability the application realises, not from how loudly its owner argues.

How.

1. Pull the level 1 `Criticality` from `O13` for each capability the application realises, and propose the maximum as the application's own.
2. Read the proposal out in the room, recording every disagreement with the name attached.
3. Fix `Owner` to a person. An organisation unit cannot be telephoned at nine on a Saturday.
4. Challenge every assurance function that appears as an owner. The shipped file gives `A01` to `Internal Audit` and `A04` to `Risk & Compliance` — functions that cannot audit what they

own.

Signed by. Chief Operating Officer (`ORG-07`) for the trading applications, Chief Information Officer (`ORG-05`) for the rest, in a single `GB-01 Architecture Board` minute.

Done when. No `Owner` cell names a function rather than a person, and every `Criticality` value traces to a capability row in `O13` .

Shows up in. `RPT02 Application Portfolio` , columns `Criticality` and `Owner` . If it went well, the till is not `Low` .

Goes wrong when. Nobody tests the till. The shipped file grades `A02 POS` `Low` — the one application in a retailer that cannot be unavailable for ten minutes without closing the shop. `O49 Resilience Tier` compounds it in chapter 7.

BEFORE YOU LET ANYONE QUOTE RPT02

Which application's `Criticality` would the Chief Executive dispute, and what would they say?

Name the three `Owner` cells that are a function rather than a person. Who replaces each?

`A08 Price & Promotion Engine` reads `Critical` , `Poor` and `Retire` at once. Which of the three is wrong?

Step 6.4 — Record the five columns O23 does not have

Produce. Five added columns, each with a stated source and a refresh owner, so the portfolio can be assessed at all.

You need. Step 6.2. `O19 Contract` , ten rows with `Counterparty` , `Type` and `Expiry` . `R07 Application deployed on Tec` . The annual run-cost extract from finance.

Where it lands.

> **O23 Application Component**
>
> Add, right of `Owner`: `Vendor` (the legal counterparty from `O19`, not the brand); `Annual run cost` (licence, support, hosting and run team, one currency, one stated financial year); `Hosting` (`On premise`, `Private cloud`, `Public cloud`, `SaaS`, `In store`); `Lifecycle` (`Proposed`, `Live`, `Sunset`, `Retired`) so the sheet can hold a target row without lying; and `Contract expiry` from `O19`.

How.

1. Take `Vendor` and `Contract expiry` from `O19` by name, recording where the sheets disagree rather than picking one.
2. Get `Annual run cost` from finance in writing. An architect's estimate is worth nothing in the room where disposition gets decided.
3. Set `Hosting` from `R07 Application deployed on Tec` where that join is credible and from the run team where it is not. Chapter 7 shows why it usually is not.
4. Mark target rows `Proposed` in `Lifecycle`, and write each column's refresh owner into the sheet's notes.

Signed by. Chief Financial Officer (`ORG-02`) for `Annual run cost`, Chief Information Officer for the rest.

Done when. Every live row carries a cost, a vendor and a contract expiry, and `RPT02`'s reprint has been re-run so the new columns do not silently fall off it.

Shows up in. `RPT02` does not pick up added columns; it reprints the nine it knows. Say so, or publish the extended sheet beside it.

Goes wrong when. `Annual run cost` mixes licence-only and fully-loaded figures across rows. Every consolidation case after that is arithmetic nobody can defend.

Finding the applications nobody declares

In retail the undeclared systems are not random. They cluster in the capabilities volume 1 names as usually missing, because none of those sits between a transaction and a ledger. Nothing bounces when they are absent, so a spreadsheet becomes the system of record and stays there for years. Three hunting grounds, nearly always occupied in a multi-format retailer.

The spreadsheet that holds the range plan. `A-04` and `A-05`. Halverson has `A07` for planograms and clustering and nothing for the money side. Somewhere in the commercial team sits a workbook per category holding the plan by week, the open-to-buy, the intake and the sign-off, with a version in its filename. It is what the buying organisation is actually governed by, and if you replace the merchandising system without finding it you meet it in user acceptance testing, when a buying controller explains that the new system cannot produce the number the trading director reads every Monday.

The Access database behind supplier income. `A-23` — rebates, trading terms, promotional funding, long-term agreements and claims. It matters more than its obscurity suggests, because supplier income lands directly in reported gross margin. Usually a desktop database or linked workbooks kept by two people in finance, with its own supplier master that does not reconcile to `E14 Supplier` and claim logic that exists nowhere else. Halverson's `O20 Data Entity` has no entity for a trading term or a claim, which tells you the repository has never looked.

The store-level workaround nobody at head office knows about. `A-20`. Where that capability is absent, stores invent it: a shared drive of task sheets, a regional messaging group, a printed pack the manager signs, a locally-written count sheet because the central screen is slow on the store line. Invisible from the support centre by construction, and the reason a process measured at ninety-five per cent compliance runs at sixty in the aisle. Find them by visiting four stores of different formats and asking the manager what they do that the system does not.

THE THREE RETAIL HUNTING GROUNDS · STEP 6.5

Hunting ground	vol 1	What you will find	Where you look	What it becomes
The range plan	A-04, A-05	A workbook per category: plan by week, open-to-buy, intake, sign-off	Commercial team shared drive	Row in O23, Type Custom
Supplier income	A-23	Desktop database or linked workbooks: rebates, terms, funding, claims	Two people in finance	Row in O23 + entity in O20
The store workaround	A-20	Task sheets, regional messaging group, locally written count sheet	Four stores, different formats	Requirement, with a tolerance date

None of the three sits between a transaction and a ledger. That is why nothing bounces.

Figure 6.2 The three places an undeclared retail system hides, what to read, where to look and what each find becomes. Halverson Stores Group is an invention; the capability references are volume 1's.

Step 6.5 — Run the three hunting grounds and close the inventory

Produce. A discovery log: every undeclared system found, who maintains it, what it masters, and the verdict — becomes a row in `023`, becomes a requirement, or is tolerated with an expiry.

You need. Step 6.1's `None` list. Four store visits across formats. An hour each with the supplier-income team and the merchandise planners. Read access to the commercial and finance shared drives.

Look it up. Volume 1, chapter 6, *Supplier income*, explains why the rebate file is a margin system and not an administrative one. Volume 1, chapter 12, *ERP, finance and supplier income*, gives the systems that normally sit there and what they do not cover.

Where it lands.

> **`023 Application Component`**
>
> A found system that masters data somebody else relies on gets a row, `Type` `Custom`, `Description` naming the file or database, `Owner` the person who maintains it — that name is the finding. A true workaround with no data of record gets no row: write a requirement at step 6.13 and record the tolerance date.

How.

1. For each `None`, ask the capability's business owner one question: what do you open to do this today?
2. Ask for the file, not a description of it. Open it. Count the sheets with formulas against the sheets that are pasted values.
3. Test mastering: does anything else consume a number this file produces? If so it is a system of record whatever its file extension.
4. In the stores, ask the manager what they keep that head office did not give them. Photograph it.
5. Decide per find: row, requirement, or tolerated with a date. Write the date.

Signed by. Chief Enterprise Architect, with the owning line executive signing each verdict.

Done when. Every `None` has a verdict and none reads "under investigation".

Shows up in. `RPT02`'s row count rises and `RPT13 Orphan Analysis` gains entries: a found spreadsheet has no business service mapped to it yet.

Goes wrong when. You promise to replace the spreadsheet. The spreadsheet is evidence of a capability gap, and the person maintaining it has been doing the enterprise a favour for six years. Say so out loud or the next discovery conversation goes nowhere.

Form C-3 — the undeclared systems log

One row per find. Columns one to four transcribe into `O23` once the verdict is `Row`; the rest stay here. A first pass usually fills eight to fourteen rows. Signed by the Chief Enterprise Architect and the owning line executive.

Application	Type	Description	Owner	Masters what	Verdict and date
Range plan workbook	Custom	Example, delete me		Open-to-buy by category	Row, 2027-03

Assessing the portfolio

`RPT02 Application Portfolio` is where this phase becomes visible to people who never open a relationship sheet. Read it before you quote it: it computes nothing, reprinting `O23`'s nine assessable columns in sheet order and stopping. The judgement sits entirely in `Disposition`, `Criticality` and `Health`, and in the shipped file those three were typed independently.

The evidence is on one page. `A08 Price & Promotion Engine` is the only `Critical` row; its `Health` is `Poor`, it went in during 2022, and its `Disposition` is `Retire` — the most critical thing in the estate, four years old, retired with no successor named anywhere. `A02 POS` is `Low` and `Adequate`, which is the till. `A12` went live in 2023 and already reads `Replace`. Five of twelve rows read `Replace`, which is not an assessment.

Figure 6.3 `O23 Application Component`'s own `Criticality` and `Health` columns plotted against each other, with `Disposition` written into the cell. The two flagged cells are the findings `RPT02 Application Portfolio` reprints without noticing [2].

So produce the assessment the report does not. Two axes, one to five, and the second has evidence behind it in this repository.

Axis	What it is	Evidence
Business fitness	What the business loses if this stops for one trading day	`O13 Criticality`, `O15 Exposure`, `R25`, the trading calendar
Technical fitness	Whether it can be changed safely at the pace the business needs	`R06` managed and logged share, `R18` conformance, `O27 Support end` via `R07`, `049 Tested`, `O23 In service`

Technical fitness scores five sub-tests, one point each: no unmanaged interface; every interface logged; one standard assessed and not in breach; the platform inside vendor support; a resilience test inside twelve months. No row scores five and `A08` scores one.

Step 6.6 — Score both axes, in one workshop, with two scorers

Produce. A scored portfolio: two numbers per application, the sub-test detail behind the technical score, and a note wherever the scorers disagreed by more than one point.

You need. Steps 6.2 to 6.4. `RPT04 Interface Catalogue`, `RPT12 Standards Conformance`, `RPT10 Technology Currency`, `049 Resilience Tier`. Three hours, one room, owning executives and run leads together.

Where it lands.

> **`O23 Application Component`**
>
> Two more added columns, `Business fitness` and `Technical fitness`, each 1 to 5. Keep the five sub-test results in a working sheet beside the repository: a column per sub-test makes `O23` unreadable.

How.

1. Score business fitness first and alone, with the trading calendar on the table. The question is one trading day in the week before Christmas, not an average day.
2. Score technical fitness mechanically from the five sub-tests. The only thing negotiable is whether a sub-test was read correctly.
3. Two scorers, scoring blind, then compare. A gap of two or more points is a finding about the evidence, not the application.
4. Record disagreements with names, and show nobody the quadrant grid until both axes are locked.

Signed by. `GB-01 Architecture Board`, in minute, with the Chief Financial Officer present because step 6.7 has spending consequences.

Done when. Every row has two scores, every technical score reconciles to its sub-tests, and every gap of two or more carries a named note.

Shows up in. Nowhere in the thirteen reports, so the scored sheet travels as an appendix to the Application Portfolio Catalog.

Goes wrong when. The workshop scores business fitness by headcount. `O23`'s `Users` column invites it: the shipped file gives `A02 POS` 91 users against `A06`'s 279, which would make the forecasting tool three times more important than the till. `Users` counts named logins, so it under-counts every system used on a shared till sign-on and over-counts every reporting licence.

Step 6.7 — Derive the disposition from the grid, and test it against what is there

Produce. A disposition per application derived from the two scores, beside the `Disposition` already in the sheet, with every disagreement written up.

You need. Step 6.6.

Where it lands.

> **`O23 Application Component`**
>
> `Disposition` takes five values: `Retain`, `Consolidate`, `Replace`, `Retire`, `Invest`. Four quadrants, five values — `Replace` is a decision about *how*, not *where*, and the sheet conflates them. Derive the quadrant first (`Invest` high-high, `Retain` high-business low-technical, `Consolidate` high-technical low-business, `Retire` low-low), then choose `Replace` separately as the route out of `Retire` or `Consolidate`.

How.

1. Plot the twelve, name the quadrant for each, and compare to the `Disposition` already written. Every mismatch gets a row in the disagreement table.

2. For each `Retire`, name the successor and the date or change the disposition. A retirement with no successor is a risk entry, not a plan.
3. For each `Replace`, state the quadrant it came from: that decides whether it is funded as risk reduction or as growth.
4. Take the disagreements to the board as a decision list, not a finding list.

Signed by. `GB-02 Investment Committee`, which holds G0 funding.

Done when. Every `Disposition` matches the derived quadrant or carries a signed exception whose reason is not "the vendor contract runs to 2030".

Shows up in. `RPT02`, column `Disposition`. If it went well, no `Critical` row reads `Retire` without a named successor and the `Replace` count falls well below five of twelve.

Goes wrong when. The grid is drawn after the dispositions are politically settled. You will know, because every point lands in the quadrant somebody already named.

WHERE THE GRID AND THE SHEET DISAGREE

How many of the twelve dispositions change once the grid is drawn? Which one will be fought hardest?

`A08` is `Critical` and `Retire`. Name the successor and the date, or write the sentence that changes the disposition.

Services, coverage, and the duplication the sheet cannot express

Step 6.8 — Name the information system services and record which are published

Produce. `O24` populated, `Style` and `Published` answered honestly, the gap between published and consumed stated as a number.

You need. Step 6.2. `R06 Application interfaces with` for what is actually consumed.

Where it lands.

> **`O24 Information System Service`**
>
> Six columns: `Ref`, `Information system service`, `Application`, `Description`, `Style`, `Published`. Twelve rows, one per application, every `Description` reading "Service exposed by *X*" — placeholder text, not architecture. `Style` from `API`, `Event`, `File`, `Batch`. `Published` from `Published`, `Unpublished`. One row per application is the giveaway: a real estate has several services per application, or none.

How.

1. Delete the one-row-per-application pattern. List the services consumers actually call.
2. Write `Description` as the contract in one sentence: what it returns, at what latency.
3. Set `Style` from how it is carried, then cross it against `R06`'s `Type` for the same pair. `ISS-08` is `File` while `A08`'s `INT-016` is an `API`; one of the two is wrong.
4. Set `Published` only where a consumer could build against it without asking a person. Seven of twelve read `Unpublished`, which is the number that funds step 6.12.
5. Count services consumed but not listed: your integration debt in one figure.

Signed by. Chief Information Officer (`ORG-05`).

Done when. No `Description` contains the words "Service exposed by", and every `R06` row resolves to a listed service or an explicit exception.

Shows up in. No report reads `O24`. Publish the `Unpublished` count yourself.

Goes wrong when. `Published` is read as "documented somewhere". The test is whether a consuming team can build against it without a meeting.

Step 6.9 — Join business services to applications, and grade the join you were given

Produce. `R04` rebuilt from evidence, with the unverifiable rows marked rather than deleted.

You need. Step 6.8. `O15 Business Service`, ten rows. `R03 Capability realised by Busi`.

Where it lands.

> **`R04 Business Service supported`**
>
> Four columns: `Business service`, `Service name`, `Application`, `Application name`, ten rows. Note the truncated tab name: several relationship tabs here are cut at Excel's thirty-one characters, and you spell them as the tab spells them.

How.

1. Read the ten rows before trusting them. `BSV-01` maps to `A01`, `BSV-02` to `A02`, and so on to `BSV-10` and `A10`. That is positional, not architectural.
2. Test each. `BSV-02 Pricing & Promotion Service` is mapped to `A02 POS`, but the engine that sets price is `A08`. `BSV-08 Customer & Loyalty Service` is mapped to `A08`; loyalty is

`A09`. Only `BSV-01` survives, by coincidence.

3. Rebuild from the contract: which application answers the call this business service makes?
4. Allow many-to-many. A business service realised by three applications is a fact.
5. Mark `A11` and `A12` deliberately. They have no business service, which is why they are the only two rows in `RPT13`.

Signed by. Chief Operating Officer (`ORG-07`); the business service names are the business's words.

Done when. No `R04` row survives only because its ref number lines up with another sheet's.

Shows up in. `RPT13 Orphan Analysis`, which tests this join and nothing else. Expect the orphan list to grow once the positional rows go.

Goes wrong when. You delete the unverifiable rows rather than marking them, and coverage then reads better than before you started.

Step 6.10 — Read capability coverage, and name the duplication the model cannot hold

Produce. A coverage statement per level 1 capability, and a separate duplication finding with a verdict per case.

You need. Step 6.9. `O13`'s level 1 rows. `RPT03 Capability Heatmap`. `EXT X02 Store / Location` for the format dimension.

Look it up. Volume 1, chapter 19, *Capability to application — the coverage map*, is already drafted and worth arguing with. Volume 1, chapter 2, *What the format mix does to the application estate*, is the chapter that decides whether the duplication you are about to find is a defect.

Where it lands.

> **`R03 Capability realised by Busi`**
>
> Four columns: `Capability`, `Capability name`, `Business service`, `Service name`. Coverage is a two-hop read — capability to business service here, business service to application in `R04`. Nothing in the repository holds a direct capability-to-application link.

How.

1. Join `R03` to `R04` and count applications per level 1 capability. In the shipped file every one of the ten gets exactly one, so coverage reads ten of ten and means nothing.
2. Re-read it after step 6.9. Real coverage is uneven, and the uneven part is the finding.
3. Ask the format question per capability: is this realised more than once, once per trading format?
4. Apply the three-part test below to every duplicate. Record the verdict and the evidence.
5. Write up step 6.1's `None` capabilities as gaps: `O29 Gap` holds one row and will not find them for you.

Signed by. Chief Enterprise Architect, with the format managing directors countersigning the duplication verdicts.

Done when. Every capability has a count, every count above one has a verdict, and every verdict cites the part of the test it passed.

Shows up in. `RPT03 Capability Heatmap` carries `Criticality`, `Current`, `Target`, `Gap` and `Disposition` but no application count, so the coverage statement travels as your own table.

Goes wrong when. You call every duplicate a consolidation candidate. In a multi-format retailer it often is not, and the programme gets cancelled two years in by the format that could not be migrated.

In a group running supermarkets, pharmacy and apparel the same capability is frequently realised three times, once per format, and sometimes that is correct. A duplicate must pass at least one part of this test to survive.

1. **Regulatory separation.** Pharmacy dispensing is a separately regulated business inside the store, mastering patient, prescription, dispensing act and the scheduled-substance register alone. A second instance is not duplication; it is the wall, and the rules specify it.
2. **Genuinely different trading rules.** Apparel replenishes against a size curve and a season with no repeat; grocery against daily demand on a continuous range. Different algorithms, not different configurations — and one engine serving both usually means the smaller format got the larger one's rules and quietly stopped using it.
3. **Disjoint or clearly subordinate data.** Either the instances master nothing in common or one is authoritative and the others demonstrably consume. Two instances both claiming `E01 SKU / Article` fails whatever else is true.

A duplicate passing none of the three is a consolidation candidate, and the arithmetic is the three annual run costs from step 6.4 against one, over the remaining term in `O19 Contract`.

Form C-4 — the capability coverage and duplication worksheet

One row per level 1 capability from `013`. The first two columns are `013`'s own; the rest are this step's judgement and stay beside the repository. Ten rows is a first pass. Signed by the Chief Enterprise Architect with the format managing directors.

Ref	Capability	Apps	Formats served	Verdict	Test part passed
C05	Store Operations	3	Grocery, pharmacy, apparel	Example, delete me	1

COVERAGE, DUPLICATION, AND THE EMPTY GAP SHEET

Coverage reads ten of ten before you touch `R04`. What does it read after?

Which duplicated capability in your estate passes the test, and on which of the three parts?

`029 Gap` holds one row. How many gaps did step 6.1 and step 6.10 produce between them?

Data authority, interfaces, requirements, standards

Step 6.11 — Record which application manages which data entity, and who is authoritative

Produce. `R05` complete, with exactly one `Master` row per data entity and every other claimant recorded as `Consumer`.

You need. `O20 Data Entity` from Phase C's data half, fourteen rows. Step 6.2.

Look it up. Volume 1, chapter 10, *Ownership, lineage and the gate that actually works*, gives the test for a master system. Volume 1, chapter 19, *Application to data, with the system of record*, drafts the map — argue with it, do not copy it.

Where it lands.

> **`R05 Application manages Data En`**
>
> Five columns: `Application`, `Application name`, `Data entity`, `Entity name`, `Role`. `Role` takes `Master` or `Consumer`. Twenty-eight rows, fourteen of each. Two repairs before use: the `Master` rows put the application **name** in the `Application` column while the `Consumer` rows put the **ref**, so a join on `Application` returns half the sheet; and only four applications appear as consumers at all, which cannot be true of an estate with twenty-nine interfaces.

How.

1. Normalise the key: the `Ann` ref in `Application`, the name in `Application name`. Nothing else works until this is done.
2. For each of the fourteen entities name one `Master`, read it out to `O20`'s data owner, and minute the agreement.
3. Cross-check `O20`'s own `Master system` column. The two sheets agree and both are wrong: `E01 SKU / Article` mastered in `POS`, `E05 Price` in `Workforce Management`, `E13 Loyalty Account` in `Supplier Portal`.
4. Add a `Consumer` row for every application that reads the entity, from `R06`. That list is the replacement scope for the entity's master.
5. Flag every `Lineage` reading `Absent` or `Partial` — twelve of fourteen, including `E11 Transaction / Basket`, the most-read number in the business.

Signed by. `GB-03 Data Governance Council`, which designates data owners and resolves disputes.

Done when. Every entity has exactly one `Master`, the key column is consistent, and no entity's master is an application whose `Disposition` reads `Retire` without a named transfer.

Shows up in. `RPT05 Data Ownership`, which reprints `O20` without `Definition` and adds a footer counting entities with no lineage. It reads `Master system` from `O20`, not `R05`, so the two can disagree indefinitely and no report will say so.

Goes wrong when. Nobody checks `Classification` while they are in there. The shipped file classifies both `E12 Customer` and `E04 Assortment` as `Public` — a POPIA problem and a commercial one in one column.

O20 MASTER SYSTEM AS THE SHIPPED FILE RECORDS IT

Ref	Entity	Classification	Master system	Lineage	Credible?
E01	SKU / Article	Restricted	POS	Full	No
E03	Store / Location	Internal	Loyalty & CRM	Full	No
E04	Assortment	Public	WMS	Absent	No
E05	Price	Internal	Workforce Management	Full	No
E08	Inventory Position	Internal	E-commerce Platform	Partial	No
E11	Transaction / Basket	Restricted	Space & Range Planning	Absent	No
E12	Customer	Public	Workforce Management	Partial	No
E13	Loyalty Account	Restricted	Supplier Portal	Partial	No

Eight of fourteen entities. R05 agrees with O20 and both are wrong; no report compares them.

Figure 6.4 `O20 Data Entity`'s `Master system` column as shipped, which `RPT05 Data Ownership` reprints and `R05 Application manages Data En` repeats. The till masters the article; the workforce system masters price [1].

Step 6.12 — Build the interface register and count the unmanaged

Produce. `R06` complete and assessed, and a one-page finding with three numbers on it.

You need. Steps 6.8 and 6.11. The integration team's routing configuration, not their diagram. `041 Security Control`, `064 Security Policy`, `065 Enforcement Point`.

Look it up. Volume 1, chapter 12, *Integration, as it really is*, describes the estate this register will find. Volume 1, chapter 7, *The batch clock*, explains why `Frequency` is a trading constraint, not a technical preference.

Where it lands.

> **`R06 Application interfaces with`**
>
> Nine columns: `Ref`, `Source`, `Source name`, `Target`, `Target name`, `Type`, `Frequency`, `Managed`, `Logged`. It is the only relationship sheet with its own `Ref` — `INT-001` upward — which makes an interface citable in a design review, so cite them. `Type` from `API`, `Event`, `File`, `Batch`, `Direct DB read`.

How.

1. One row per directed pair per `Type`. A nightly file and an hourly API call between the same two systems are two rows.
2. Set `Managed` from the routing configuration. Managed means it passes a gateway or broker that can be inventoried, throttled and replayed; a point-to-point connection configured in 2014 is `Unmanaged`, however reliable.
3. Set `Logged` from whether a named person can answer "what crossed this interface at 14:05 last Tuesday?" without writing a query.

4. Cross `Type` against `Frequency` and repair the contradictions. Four rows read `Batch` and `Real time` together, one reads `File` and `Real time`.
5. Count the three findings: unmanaged, unlogged, direct database reads.

Signed by. Chief Information Officer (`ORG-05`), with the security delegate on `GB-01` countersigning the `Logged` column.

Done when. Every interface has a `Ref`, a `Managed` verdict traceable to configuration and a `Logged` verdict traceable to somebody who could answer the Tuesday question.

Shows up in. `RPT04 Interface Catalogue`, which reprints `R06` with names substituted for refs, drops `Ref`'s source and target refs, and adds one computed footer: *16 of 29 interfaces are unmanaged and 20 carry no logging.*

Goes wrong when. The register is built from the integration team's diagram. The diagram shows the managed estate, which is the part they built. The nine direct database reads are in nobody's diagram, which is exactly why they are nine.

Those three numbers fund a design gate. Present them as one page, no adjectives.

Finding	Shipped file	Threshold on `ZZ Checks`	What it means in a trading week
Interfaces unmanaged	16 of 29	0	55% of the estate's integration cannot be throttled, replayed or inventoried
Interfaces with no logging	20 of 29	0	For 69% of the estate there is no record of what crossed
Direct database reads	9 of 29	0	Nine applications read another application's store, so nine replacements are nine times harder

The supporting evidence is in the repository and it lines up. `O64 POL-03` — material activity is logged and observable — reads `Breached`. `O41 SEC-05 Interface logging` reads `Absent`. `O65 EP-03 API gateway` reads "Product not yet selected". The policy exists, the control does not, and the thing that would enforce it has not been bought: a complete argument for a gate in four cells.

The gate: no design passes `O58 GT-1 G1 Design` without a named `R06` entry for every interface it introduces, `Managed` reading `Managed` and `Logged` reading `Yes`. `GB-01` holds G1 and can grant a dispensation with an expiry, which is the right mechanism — a blanket refusal in an estate 55% unmanaged stops all delivery.

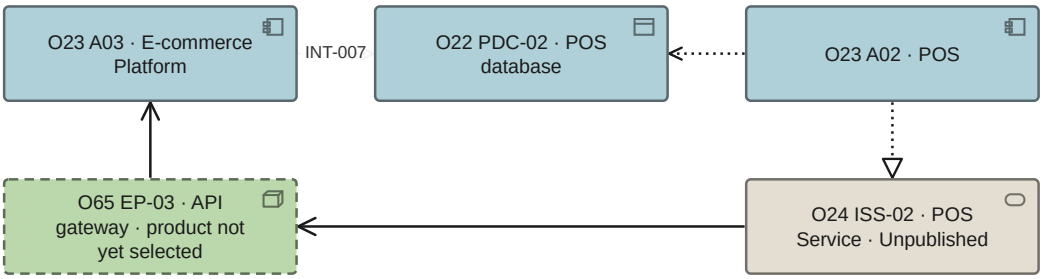


Figure 6.5 `INT-007` in ArchiMate 4: the e-commerce platform reading the till's own store hourly, beside the published route that does not exist because `065 EP-03` has no product behind it. Nine of twenty-nine interfaces in the shipped file are direct database reads [1].

THE UNMANAGED INTERFACES, BEFORE YOU PRESENT THEM

Which of the nine direct database reads crosses into the card estate, and what does that do to PCI scope?

The gate will block the next three designs. Which dispensations do you pre-agree, and with what expiry?

Step 6.13 — Trace requirements to applications, through the building blocks

Produce. `R09` rebuilt so the trace means something, plus a count of requirements that reach nothing.

You need. `006 Requirement`, thirty-one rows. `045 Application Building Block`, ten rows. `055 Building Block`, thirty-seven rows across four domains.

Where it lands.

> **`R09 Requirement satisfied by Bu`**
>
> Five columns: `Requirement`, `Requirement name`, `Building block`, `Block name`, `Traced`. Thirty-one rows, mapped positionally — `REQ-001` to `BBB-01`, `REQ-002` to `BBB-02`, onward — so `REQ-002 Evidence PCI DSS` is satisfied by `BBB-02 Pricing & Promotion capability uplift` and `REQ-031 Adopt PCI-scoped payment tokenisation` by `TBB-01 Edge at store`. Rebuild it all.

How.

1. Sort `006` by `Category`: six `Regulatory`, eight `Data`, six `Integration`, six `Business`, five `Technology`. That tells you which block domain each should reach.

2. Map each requirement to the block that will deliver it. A regulatory requirement reaching a capability-uplift block is almost always a mis-trace. One requirement may reach several blocks, and several may reach one.
3. Set `Traced` to `No` honestly. A requirement that reaches nothing is a requirement nothing will deliver, and that is the step's useful output.
4. Check the integration requirements against step 6.12. `REQ-015` to `REQ-020` demand that every interface to or from six named applications be managed, published and logged. All six fail today.

Signed by. Chief Enterprise Architect, with each requirement's business sponsor confirming the block.

Done when. No row in `R09` pairs a requirement with a block purely by row position, and the `Traced = No` count is a number you would defend to the sponsor.

Shows up in. `RPT06 Requirements Traceability`, which reprints `O06` and copies `R09`'s `Traced` column under the heading `Reaches a block`. It computes nothing. `Traced` reads `No` on every fourth row — 1, 5, 9, 13, 17, 21, 25, 29 — a pattern, not an assessment, and the surest proof in the file that the column was generated.

Goes wrong when. `REQ-001` to `REQ-006` get traced to capability blocks because the block names sound like the business. The six regulatory requirements belong against the control objectives in `O63` and the controls in `O41`, which chapter 7 handles.

Step 6.14 — Assess standards conformance, application by application

Produce. `R18` covering every application and current standard, each conformance verdict backed by evidence.

You need. `O32 Standard`, sixteen rows. Step 6.12 for the integration standards, step 6.11 for the data standards.

Where it lands.

> **`R18 Standard applies to Applica`**
>
> Five columns: `Standard`, `Standard name`, `Application`, `Application name`, `Conformance`. Sixteen rows, one per standard, each against exactly one application — and the twelve applications are covered by six, cycling `A01`, `A03`, `A05`, `A07`, `A09`, `A11`. Every even-numbered application carries no standard at all, including `A02 POS` and `A08`.

How.

1. Build the matrix in a working sheet — sixteen standards against twelve applications, each cell `Applies` or `Does not apply` — then write an `R18` row for every `Applies`, and a reason for every `Does not apply`.

2. Set `Conformance` from `Conformant`, `Partial`, `Breach`, `Not assessed`, treating `Not assessed` as a work item with an owner rather than a neutral state.
3. Cross-check each verdict against evidence you already hold. `STD-07 Logging and monitoring standard` reads `Conformant` against `A01`, whose four `R06` interfaces all read `Logged = No`. One of those cells is false.
4. Deal with the deprecated rows. `STD-09`, `STD-11` and `STD-13` read `Deprecated` and still carry a verdict, `Conformant` in two cases. A deprecated standard carries a successor and no verdict.

Signed by. `GB-01 Architecture Board`, which publishes the criteria it assesses against.

Done when. Every current standard has an `R18` row, no application is uncovered without a stated reason, and no deprecated standard carries a verdict.

Shows up in. `RPT12 Standards Conformance`, which reprints `O32` verbatim and does **not** roll up `R18`. The conformance you read there is a column somebody typed on `O32`, not a computation over the applications. Say that in the paper.

Goes wrong when. `Not assessed` stands because it reads better than `Breach`. Three of sixteen read `Not assessed`, including `STD-01 API design standard` and `STD-05 Identity and access standard` — the two you would most want assessed.

The reference model, and the cheapest finding in the set

`O47 Reference Model Entry` holds two kinds of thing in eleven rows. `RME-01` to `RME-07` are retail patterns — edge at store, real-time inventory, headless commerce, event streaming, PCI-scoped payment tokenisation, electronic shelf labels, peak-elastic infrastructure. `RME-08` to `RME-11` are generic layered models for application, integration, analytics and security. A target architecture and a framework catalogue sharing one sheet.

What is absent matters more: no row names a single external reference model. No ARTS, no UnifiedPOS, no GS1, no APQC, no PCI DSS. The sheet whose name promises reference models contains none, which is the cheapest finding in Phase C.

Look it up. Volume 1, chapter 13, *ARTS, the NRF and the OMG Retail Domain Task Force*, has the picture; *What is alive* and *What is dormant, and what to say about it* separate the halves.

Three cautions, because this is where architects get caught.

- **UnifiedPOS is alive and it is the Object Management Group's.** Release 1.15.1 was adopted in December 2024 under a RAND intellectual-property mode with machine-readable schema files, and the 2024 contributor list runs Aptos, Diebold Nixdorf, Digimarc, GS1, Microsoft and Toshiba [3]. Do not call it legacy, and do put it in `O47` if the estate has a till.
- **The ARTS data models may be unobtainable.** "ARTS" is a historical name; the Retail Domain Task Force runs the live programme. The Operational Data Model and the Data Warehouse Model are described on its pages but absent from the formal specification catalogue, and the only hard version marker recoverable is a 2006 certification [4][5]. Do not send anyone to get the ARTS

ODM. Record it `Candidate` with a note that you have asked whether it exists in usable form, and the date you asked.

- **GS1 is the one standards body that genuinely binds a retailer**, through the identification keys, the General Specifications and the synchronisation network. Put the keys in `047` and the identifier work in `037 Data Attribute`. Never write that GS1 requires two-dimensional barcodes by 2027: Sunrise 2027 is a GS1 US industry capability ambition with no legal date and a voluntary adoption model [6]. The durable consequence stands without the deadline — the till becomes a traceability capture point and the scan resolves to content the retailer does not own.

Retail has no equivalent of banking's BIAN, and saying so with the evidence beats pretending.

Step 6.15 — Reconcile the reference model entries and add the external models

Produce. `047` split into populations, with every external reference model the estate uses recorded, dated and status-marked.

You need. Step 6.1's mapping page. Volume 1 chapters 13 and 21. The till vendor's conformance statement, if there is one.

Where it lands.

> **`047 Reference Model Entry`**
>
> Four columns: `Ref`, `Reference model entry`, `Description`, `Status`, the last from `Adopted` or `Candidate`. Keep the eleven, add the external models as rows, and put the population — `Pattern`, `Layered model`, `External standard` — in `Description`'s first clause. There is no column for it.

How.

1. Mark each of the eleven as pattern or layered model, and say which population the board is adopting.
2. Add one row per external model in use or under consideration: UnifiedPOS, the GS1 keys and General Specifications, the Digital Receipt API, APQC's retail process framework if Phase B used it.
3. Give each external row a `Status` and, in `Description`, the version and the date you checked it. A version with no check date is a liability.
4. Reconcile against `026 Logical Technology Componen`, which names `RME-01` to `RME-07` again as `LTC-01` to `LTC-07` with a different status on every one. Record that the two sheets disagree seven times out of seven; chapter 7 repairs it.

Signed by. `GB-01 Architecture Board`.

Done when. Every row is marked as pattern, layered model or external standard; every external row carries a version and a check date; and the `047`-against-`026` disagreement is written up with its seven rows.

Shows up in. No report reads `047`. It reaches the board through the Phase C pack and through `055 Building Block`'s technology rows, which name the same seven patterns a third time.

Goes wrong when. A vendor reference architecture goes in as `Adopted` because it was in the proposal. Volume 1, chapter 13, *How to choose, and how to tell when a model is standing in for thinking*, is the test: vendor models are examples of a shape, never a commitment.

Step 6.16 — Run the orphan analysis, then publish the Application Portfolio Catalog

Produce. The Application Portfolio Catalog, signed, with the orphan list and the scored grid as appendices.

You need. Every step above. `RPT13`, `RPT02`, `RPT04`, `RPT05`, `RPT12`. `031 Deliverable`.

Where it lands.

> **`031 Deliverable`**
>
> Four columns: `Ref`, `Deliverable`, `Phase`, `Status`. `DLV-09 Application Architecture` and `DLV-10 Interface Catalogue` both read `Approved` over an estate that fails eight of the fourteen `ZZ Checks` thresholds. Set them to `Draft` until this chapter's steps are done, and minute why.

How.

1. Run `RPT13`. It tests one join — application against business service in `R04` — and returns two rows, `A11 Supplier Portal` and `A12 Retail Analytics`.
2. Run the four orphan tests it does not. Applications with no standard in `R18`: six of twelve. Applications supporting no product in `R25`: eight of twelve, including the e-commerce platform. Entities with no `Master` once step 6.11 is normalised. Reference model entries referenced by nothing.
3. Assemble the catalogue: `RPT02` extended with the added columns, the `R04` rebuild, the `R05` master table, `RPT04`'s three numbers, the `R18` matrix, and step 6.1's mapping page.
4. Put the `None` count on the first page. It is the sentence an executive can act on.
5. Take it through `GB-01` for architectural approval, then `GB-02` for the funding consequences.

Signed by. `GB-01 Architecture Board` for the catalogue, Chief Enterprise Architect as author, Chief Information Officer and Chief Operating Officer as accepting owners.

Done when. `DLV-09` reads `Approved` because it was approved, and every number can be re-derived from the repository by somebody who was not in the room.

Shows up in. `RPT02`, `RPT04`, `RPT05`, `RPT12` and `RPT13`. Run `ZZ Checks` before and after and put both columns in the pack: the distance between them is the phase's output.

Goes wrong when. The catalogue is published as a document rather than a view over the repository. Six weeks later the two disagree and the sheets lose, because the document is what people have in their inbox.

PHASE C, THE APPLICATION HALF — A REVIEWER'S QUESTIONS

How many of the twenty-five conceptual capabilities read `None`, and which three will be argued?

Name every application whose `Disposition` the grid changed, and who signed each change.

`RPT04` says 16 unmanaged and 20 unlogged. What are the two numbers after this phase, and by when?

Which single interface, if logged tomorrow, would answer the most questions? Why is it not logged?

What did you put in `047` for the ARTS data models, and on what date did you ask the task force?

THE CHAPTER IN TEN LINES

- Walk A-01…A-25 first; the count of `None` answers is the phase's most useful number.
- Map volume 1's IDs onto `O23` by name, never by number — `A08` and `A-08` are different things.
- `O23` is ten columns; add vendor, run cost, hosting, lifecycle and contract expiry before you assess anything.
- Fix `Owner` to a person and `Criticality` from `O13`. The till is not `Low`.
- The undeclared systems sit in the range plan, supplier income and the store workaround.
- `RPT02` computes nothing. Score business and technical fitness yourself, two scorers, blind.
- `R04` is positional in the shipped file; rebuild it before you quote coverage.
- Triple realisation per format is sometimes right — regulatory separation, different trading rules, disjoint data.
- 16 unmanaged, 20 unlogged, 9 direct reads: that is the paper that funds the G1 design gate.
- UnifiedPOS is alive and OMG's; the ARTS data models may be unobtainable. Record both honestly.

SOURCES FOR THIS CHAPTER

- **1.** Retail_TOGAF_Metamodel.xlsx, Halverson Stores Group — 105 sheets, 842 records, 429 relationship instances (sheets 00 Index, R05, R06, RPT04, RPT05, RPT13, ZZ Checks)
- **2.** Retail_TOGAF_Metamodel.xlsx, sheet O23 Application Component — 12 records, 10 columns
- **3.** Object Management Group, "About the Unified POS Specification version 1.15.1" (adopted December 2024) https://www.omg.org/spec/UPOS/1.15.1/About-UPOS
- **4.** Object Management Group, formal specifications in the Retail category https://www.omg.org/spec/category/retail/
- **5.** Object Management Group, retail industry page — ARTS chronology, Operational Data Model and Data Warehouse Model https://www.omg.org/industries/retail.htm
- **6.** GS1 US, "Sunrise 2027" topic page https://www.gs1us.org/industries-and-insights/by-topic/sunrise-2027
- **7.** The Open Group, TOGAF Standard — the Architecture Development Method https://pubs.opengroup.org/architecture/togaf9-doc/arch/
- **8.** Retail_TOGAF_Metamodel.xlsx, sheet ZZ Checks — fourteen computed checks against their thresholds

CHAPTER 7

Technology Architecture

Four questions, four sheets — and a centre of gravity that is not the data centre but the shop that has to keep selling.

Phase D answers four questions in order: what the estate offers, what kind of thing offers it, which product on which contract, and where it physically sits [7]. Four sheets, one per question, and three of the four are populated in the shipped file by a rule rather than by evidence.

Two tab names are cut at Excel's thirty-one-character limit — `026 Logical Technology Componen` and `027 Physical Technology Compone`. Spell them as the tabs spell them; this is the only time the chapter says so. Halverson Stores Group is an invention, as is volume 1's Milkwood Retail Group [1].

In a retailer Phase D's centre of gravity is not the data centre. It is the store: a site with a network link, a till estate, a server or deliberately not, a cold chain and a trading licence — and a store that cannot take money is closed.

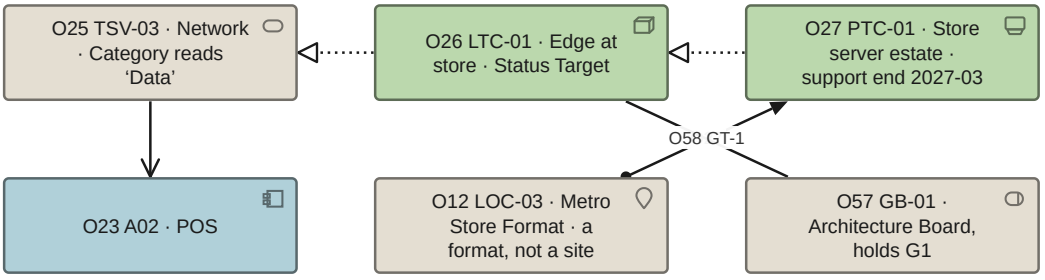


Figure 7.1 The four Phase D questions in ArchiMate 4, with the two relationships you have to rebuild. `R07 Application deployed on Tec` and `027`'s own `Location` column are both positional in the shipped file, and `O12`'s `Metro Store Format` is a format rather than a site [1].

Services before things

Step 7.1 — Name the technology services, and fix the Category column

Produce. `025` with a service list the business can recognise and a `Category` column that classifies rather than decorates.

You need. Phase C's application architecture. The infrastructure team's service catalogue if one exists. One hour with the head of infrastructure.

Where it lands.

> **`025 Technology Service`**
>
> Four columns: `Ref`, `Technology service`, `Description`, `Category`. Twelve rows: Compute, Storage, Network, Identity, Integration, Database, Backup and Recovery, Monitoring, Data Platform, End User Compute, Security Operations, Automation. A sound list. `Category` is not: Compute reads `Data`, Storage and Database read `Security`, Security Operations reads `Platform`. The column was filled by rotation.

How.

1. Keep the twelve names. They are the right grain — a service a consuming team can request.
2. Rewrite `Category` from a stated scheme and publish the scheme: `Infrastructure`, `Platform`, `Data`, `Security`, `Workplace` works and takes five minutes.
3. Add the two retail services the list omits: store connectivity, and in-store compute. Neither is a subset of `Network` or `Compute` in an estate with hundreds of sites.
4. Write `Description` as what a consumer gets, not what the team runs.

Signed by. Chief Information Officer (`ORG-05`).

Done when. Every `Category` value traces to a published scheme.

Shows up in. No report reads `025`. It feeds `046 Technology Building Block` and the standards catalogue, so an error here propagates silently.

Goes wrong when. The list becomes a product list. A technology service is what is offered, not what provides it; step 7.4 is where products go.

Step 7.2 — Record what each application runs on, and refuse the join you were given

Produce. `R07` rebuilt from deployment evidence, with non-production environments present.

You need. Step 7.1. `023`, `027`, `048`. The configuration-management export, or the run team's own list.

Where it lands.

> **`R07 Application deployed on Tec`**
>
> Five columns: `Application`, `Application name`, `Platform component`, `Component name`, `Environment`. Twelve rows, one per application, every one reading `Production`. The join is positional — `A01` to `PTC-01`, `A02` to `PTC-02`, onward — so `A03 E-commerce Platform` runs on `PTC-03 Merchandising application tier` and `A11 Supplier Portal` on `PTC-11 Payment terminals`. Delete the lot and rebuild.

How.

1. Take the real deployment per application from the run team, component by component.

2. Expect several rows per application. One application on one component is a sign nobody looked.
3. Fill `Environment` from `048`'s four values. An estate with four environments and twelve production-only rows has three environments nobody can see.
4. Flag every application whose component is `Past support` in `027`. That list is step 7.7's opening page.

Signed by. Chief Information Officer (`ORG-05`), with the infrastructure lead per row.

Done when. No row survives because its ref number matched another sheet's, and every application with a non-production environment has a row for it.

Shows up in. No report reads `R07` directly, but `023`'s added `Hosting` column from chapter 6 is derived from it, so the error travels into the application portfolio.

Goes wrong when. Payment terminals appear as the platform hosting a supplier portal and nobody notices, because the sheet is read as a list rather than a set of claims.

THE JOIN YOU WERE HANDED

How many of the twelve `R07` rows survive contact with the run team?

`048` has four environments and `R07` has twelve `Production` rows. Where are the other three environments recorded, and by whom?

Logical and physical, kept apart

Step 7.3 — Write the logical component catalogue and reconcile the three sheets that duplicate it

Produce. `026` as a catalogue of component kinds, and a written reconciliation against `047` and `046`.

You need. Step 7.1. Chapter 6 step 6.15's `047` work.

Where it lands.

> **O26 Logical Technology Componen**
>
> Four columns: `Ref`, `Logical technology component`, `Rationale`, `Status`. Seven rows, and they are not component kinds: they are the seven retail patterns `O47 Reference Model Entry` holds as `RME-01` to `RME-07` and `O46 Technology Building Block` holds a third time as `TBB-01` to `TBB-07`. `Status` from `Current`, `Target`, `Emerging`.

The disagreement is total and it is the finding.

Pattern	`O47` `Status`	`O26` `Status`
Edge at store	`Adopted`	`Target`
Real-time inventory	`Adopted`	`Target`
Headless commerce	`Candidate`	`Emerging`
Event streaming	`Adopted`	`Current`
PCI-scoped payment tokenisation	`Candidate`	`Current`
Electronic shelf labels	`Candidate`	`Current`
Peak-elastic infrastructure	`Adopted`	`Current`

Seven patterns, seven disagreements, and the two that matter most are inverted: tokenisation and shelf labels read `Candidate` in the reference model and `Current` in the component catalogue. One sheet says the estate has them, the other that it is considering them, and nothing computes either.

How.

1. Decide what `O26` is for. The useful answer: logical component kinds — message broker, container platform, store controller, till, shelf-label gateway, pin pad — independent of product.
2. Move the seven patterns out of `O26` and leave them in `O47`, one copy.
3. Rewrite `O26` as component kinds, each with a `Rationale` sentence saying why the estate needs that kind of thing at all.
4. Publish the reconciliation table above as a governance finding, with one owner and one date.

Signed by. `GB-01 Architecture Board`, which holds G1 and owns the catalogue designs are tested against.

Done when. Each of the seven patterns has one status in one sheet, and `O26` holds no row that is also in `O47`.

Shows up in. `O46 Technology Building Block`, whose second column is also headed `Ref` and carries `T01` to `T07`, references to nothing else in the repository. Fix or delete them.

Goes wrong when. Somebody reconciles by copying one status over the other. They differ because two people assessed the same thing and disagreed; find out which had evidence.

Step 7.4 — Record the physical estate, with support dates taken from the contract

Produce. `027` complete, every `Support end` date sourced from a contract or a vendor lifecycle page, and every `Position` derived from the date rather than typed beside it.

You need. Step 7.3. `019 Contract`, ten rows. `012 Location`. Vendor lifecycle statements in writing.

Look it up. Volume 1, chapter 14, *Store inventory, point of sale and store systems*, names the categories this sheet should distinguish, and chapter 12, *The legacy question*, explains why a store component's replacement cycle is not the application's.

Where it lands.

> **`027 Physical Technology Compone`**
>
> Seven columns: `Ref`, `Physical technology component`, `Vendor`, `Class`, `Support end`, `Position`, `Location`. Twelve rows. `Vendor` literally reads "Vendor" on eight of them, the rest reading `Microsoft`, `Oracle-class`, `VMware-class`, `Hyperscaler`, `Open source` — mostly category labels, not counterparties. `Position` from `Supported`, `Past support`.

How.

1. Replace every `Vendor` cell with the legal counterparty from `019`. A product class is not a vendor and cannot be telephoned.
2. Source every `Support end` from a document. Record the document and the date you read it beside the sheet; vendor lifecycle dates move.
3. Derive `Position` from `Support end` against today's date. Never type it independently — the shipped file does, and `PTC-01 Store server estate` reads `Past support` with a `Support end` of 2027-03, which is in the future.
4. Set `Location` to a real site from `012`, not a format name. Three rows here name store formats, which are categories of site, not sites.
5. Add the rows the sheet omits: the pin-pad estate as distinct from the till, the shelf-label gateway if there is one, the store network edge device.

Signed by. Chief Information Officer (`ORG-05`), with the Chief Financial Officer (`ORG-02`) acknowledging the contract column.

Done when. Every `Position` is computed from its own `Support end`, every `Vendor` is an `019` counterparty, and the `Past support` count is defensible to the month.

Shows up in. `RPT10 Technology Currency`, which reprints all seven `027` columns verbatim and adds one footer line: *3 components are running past vendor support today.* It does not compute `Position` from `Support end`, so that footer counts typed cells, not expired dates.

Goes wrong when. The dates come from the infrastructure team's own planning spreadsheet. Those are internal replacement targets, not vendor support ends, and the two differ by years.

Step 7.5 — Publish the platform services and name who consumes each

Produce. `028` with a consumer list per service and an owner, so a platform service without a consumer becomes visible.

You need. Steps 7.1 and 7.3. Chapter 6's `024` work.

Where it lands.

> **`028 Platform Service`**
>
> Three columns only: `Ref`, `Platform service`, `Description`. Eight rows — container platform, API gateway, event backbone, secrets management, CI/CD, observability, data catalogue, feature store. No owner, status or consumer column, so the sheet cannot say whether any of the eight exists. Keep status and consumers beside the repository, and read `O65 Enforcement Point` for the gateway's real position.

How.

1. For each of the eight, write `Exists`, `Procuring` or `Proposed` in your working sheet and get the infrastructure lead to sign it.
2. Cross-check the API gateway against `O65 EP-03`, which reads "Product not yet selected". A platform service listed as though it exists while its enforcement point has no product is the gap that chapter 6's interface finding depends on.
3. Name consumers from `R06` and `R07`. A platform service with no consumer is either new or abandoned; say which.
4. Put the feature store's governance position in writing, because `O05 PRN-23` requires models to be governed like any other asset.

Signed by. Chief Information Officer (`ORG-05`).

Done when. Each of the eight has a status, an owner, and either a consumer or a stated reason for having none.

Shows up in. No report reads `028`. The gateway's absence surfaces through `RPT11 Security Position`, where `SEC-05 Interface logging` reads `Absent`.

Goes wrong when. The sheet is read as an inventory. It is an ambition list until somebody adds the status column.

The standards catalogue, and the finding that gets budget

Step 7.6 — Write the standards catalogue and give conformance somewhere to live

Produce. `O32` with every current standard, a successor for every deprecated one, and conformance recorded against components rather than asserted in a column.

You need. Step 7.4. `R18 Standard applies to Applica` from chapter 6.

Where it lands.

> **`032 Standard`**
>
> Five columns: `Ref`, `Standard`, `Category`, `Status`, `Conformance`. Sixteen rows. `Category` from `Integration`, `Data`, `Security`, `Platform`, `Delivery`, `Governance`. `Status` from `Current`, `Emerging`, `Deprecated`. `Conformance` from `Conformant`, `Partial`, `Breach`, `Not assessed` — and this is the column to distrust, because it is typed, not computed.

How.

1. Mark the three deprecated rows — `STD-09 Container image standard`, `STD-11 Network segmentation standard`, `STD-13 Environment standard` — and give each a successor or withdraw it. Two of the three currently read `Deprecated` and `Conformant` at once, which is not a state a standard can be in.
2. Add the standards a retail technology estate needs and this catalogue lacks: store connectivity, till and pin-pad build, cold-chain telemetry, shelf-label interface.
3. Leave `Conformance` empty on `O32` and record it per component instead. One roll-up value across an estate of twelve components tells nobody anything.
4. Give every `Not assessed` row an assessment owner and a date. Three of sixteen read `Not assessed`, including identity and access.

Signed by. `GB-01 Architecture Board`.

Done when. No `Deprecated` row carries a verdict, and every `Current` row has an assessment owner.

Shows up in. `RPT12 Standards Conformance`, which reprints `O32`'s five columns verbatim and computes nothing. It does not roll up `R18`, so the two can disagree permanently.

Goes wrong when. `Emerging` avoids a decision. `STD-12` and `STD-16` read `Emerging` and `Partial` — standards nobody has ratified against which something partly conforms.

Step 7.7 — Cross support dates against standard status, and write the paper that gets budget

Produce. A one-page currency finding: components past support, components inside twelve months of it, the standards each breaches, and a funded replacement window.

You need. Steps 7.4 and 7.6. `RPT10`, `RPT12`, and `O19 Contract` for the renewal dates that give you room to negotiate.

Where it lands.

> **`O29 Gap`**
>
> Seven columns: `Ref`, `Gap`, `Domain`, `Building block`, `Severity`, `Status`, `Due`. One row in the shipped file, and it is a business-domain capability gap. Every technology-currency finding belongs here with `Domain` `Technology`, a `Severity` and a `Due` month. Expect to add three to five rows from this step alone.

How.

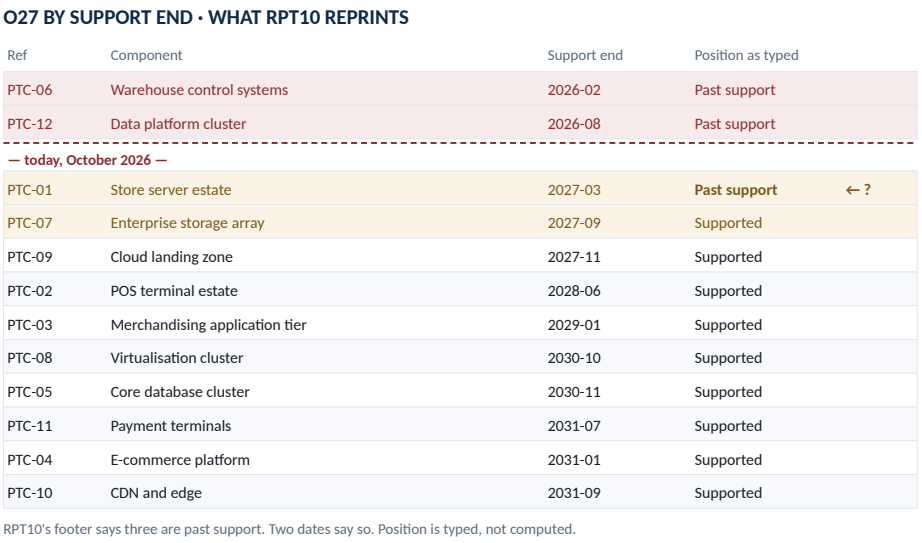
1. Sort `O27` by `Support end` and draw the line at today. Two of Halverson's twelve are genuinely expired — `PTC-06 Warehouse control systems` at 2026-02 and `PTC-12 Data platform cluster` at 2026-08.
2. Draw a second line at twelve months: `PTC-01` (2027-03) and `PTC-07 Enterprise storage array` (2027-09) sit inside it, `PTC-09 Cloud landing zone` at 2027-11 just outside.
3. Name the standards each breaches from `R18` and `O32`. `PTC-12` is past support and `STD-09 Container image standard` reads `Deprecated` and `Breach`: an unsupported platform assessed against a withdrawn standard.
4. Name the applications each carries from `R07` and the business services from `R04`. That is the sentence the paper needs — not "the cluster is unsupported" but "the cluster carrying analytics is unsupported and the standard it fails has been withdrawn".
5. Put the contract renewal date beside each row. A support end three months after a renewal is a negotiation; the other way round is a cheque.

Signed by. `GB-02 Investment Committee`, which holds G0 funding.

Done when. Every expired and near-expiry component has an `O29 Gap` row with a `Due` month naming the application and business service above it.

Shows up in. `RPT10` for the components, `RPT07 Gap Analysis` for the gaps you added, and `ZZ Checks`, where *Platform components past support* should fall towards zero.

Goes wrong when. The paper leads with the component. Nobody funds a storage array; they fund the warehouse that stops despatching — `PTC-06`, carrying `A06` in the shipped file's join and probably `A05 WMS` in reality.



O27 BY SUPPORT END · WHAT RPT10 REPRINTS

Ref	Component	Support end	Position as typed	
PTC-06	Warehouse control systems	2026-02	Past support	
PTC-12	Data platform cluster	2026-08	Past support	
— today, October 2026 —				
PTC-01	Store server estate	2027-03	**Past support**	← ?
PTC-07	Enterprise storage array	2027-09	Supported	
PTC-09	Cloud landing zone	2027-11	Supported	
PTC-02	POS terminal estate	2028-06	Supported	
PTC-03	Merchandising application tier	2029-01	Supported	
PTC-08	Virtualisation cluster	2030-10	Supported	
PTC-05	Core database cluster	2030-11	Supported	
PTC-11	Payment terminals	2031-07	Supported	
PTC-04	E-commerce platform	2031-01	Supported	
PTC-10	CDN and edge	2031-09	Supported	

RPT10's footer says three are past support. Two dates say so. Position is typed, not computed.

Figure 7.2 `O27 Physical Technology Compone` sorted by `Support end`, which is what `RPT10 Technology Currency` reprints before adding its one footer line. `PTC-01` is flagged `Past support` with a date eighteen months away [2].

Form D-1 — the technology currency paper

One row per component past support or inside twelve months of it. Columns one to four are `027`'s own; the last two are this step's work and paste into `029 Gap`. Three to six rows is normal. Signed by the Chief Information Officer and taken to `GB-02`.

Ref	Physical technology component	Support end	Position	Carries (app · service)	Due
PTC-06	Warehouse control systems	2026-02	Past support	Example, delete me	2027-06

THE CURRENCY PAPER, BEFORE YOU PRESENT IT

How many components are genuinely past support once `Position` is computed from `Support end` rather than read?

Which component's expiry lands before its contract renewal, and what does that buy you?

In retail the location master is the estate

This is the step most Phase D work skips and retail Phase D work cannot. A retailer's technology architecture is distributed across hundreds of trading sites, and each is an architectural object with properties: a network link and its diversity, a till count, a server or a deliberate decision not to have one, refrigeration with a telemetry obligation, a trading licence that may be dated and format-specific. Treat locations as a lookup list and everything downstream — resilience tiering, connectivity design, rollout sequencing, card scope — has nowhere to stand.

Look it up. Volume 1, chapter 7, *Channels, formats, and what each costs to serve*, gives the cost shape per format. Volume 1, chapter 16, *The store, the till and what the link does*, gives the modelling decisions.

`O12 Location` will not hold this on its own.

Step 7.8 — Build the estate master, and use the industry extension for what the core cannot hold

Produce. A location master with one row per site, and `EXT X02 Store / Location` recorded as the extension type that justifies it.

You need. Step 7.4. The property team's site list. The store operations estate list. The licence register.

Where it lands.

> **`O12 Location`**
>
> Four columns: `Ref`, `Location`, `Type`, `Scale`. Eight rows: `Support Centre`, `Flagship Store`, `Metro Store Format`, `Convenience Format`, `National Distribution Centre`, `Fulfilment Centre`, `Primary Data Centre`, `Cloud Region`. `Type` from `Corporate`, `Trading`, `Logistics`, `Technology`. `Scale` is a headcount string: `Cloud Region` reads "293 people". Three of the eight are formats, which are classes of site.

> **`EXT Industry Extension`**
>
> Three columns: `Ref`, `Industry extension type`, `What it holds and why the core metamodel is not enough`. `X02 Store / Location` reads "A trading or fulfilment location with its format and capacity" — the one place in the repository that admits the core metamodel's `Location` cannot carry a trading site. That is the sheet to cite when you ask for the columns below.

How.

1. Decide the grain: one row per trading or fulfilment site, with format as an attribute. Keep the formats as a separate list; they are a classification, not a location.
2. Add the columns a retail estate needs beside `O12`: `Format`, `Till count`, `Store server` (`Yes` / `No`), `Primary link` and `Secondary link` with bandwidth, `Generator`

(`Yes` / `No` / `Partial`), `Cold chain` (`Yes` / `No`), `Licence` and `Licence expiry` , `Trading hours` .

3. Replace `Scale` 's headcount string with something architectural: selling area, till count, or both in separate columns.
4. Rebuild `R20 Application located at Loca` , which is positional in the shipped file — `A01` to `LOC-01` , `A02` to `LOC-02` and onward, putting the e-commerce platform in a store format and the price engine in the cloud region.
5. Record the extension itself: if you add columns, say in `EXT` that `X02` now carries them, so the next architect knows where the model was stretched.

Signed by. Chief Operating Officer (`ORG-07`) for the site list, Chief Information Officer for the technical columns, Legal (`ORG-12`) for the licence columns.

Done when. Every trading site has a row with a primary link and a licence status, and no row in `O12` is a format rather than a site.

Shows up in. `RPT10` 's `Location` column stops reading like a category list, and `049 Resilience Tier` becomes answerable per site rather than per application.

Goes wrong when. The master is built from the property system alone. Property knows the lease and the square metres; it does not know the till count, the link, or whether there is a generator.

Step 7.9 — Record the environments and the personal data in them

Produce. `048` corrected, masking position stated per environment.

You need. Step 7.2. `O20` 's `Classification` column. `041` `SEC-07` .

Look it up. Volume 1, chapter 15, *POPIA, with the retail bite*, gives the obligation a non-production copy of the loyalty estate breaches.

Where it lands.

> **`048 Environment`**
>
> Four columns: `Ref` , `Environment` , `Description` , `Personal data present` . Four rows: `Production` , `Pre-production` , `Test` , `Development` . `Personal data present` reads `No` for Production and `Yes` for Pre-production and Development — inverted. Production holds the loyalty estate; Development should not.

How.

1. Answer `Personal data present` from evidence — a query against the environment, not an opinion.
2. For every `Yes` outside production, cross-check `041` `SEC-07 Non-production masking` , which reads `Absent` , and `O65` `EP-05` , which reads "Most refreshes are unmasked".

3. Add a column beside the sheet for the masking position per environment and the date of the last refresh.
4. Raise an `O29 Gap` row for every unmasked non-production environment holding `Restricted` or `Confidential` data from `O20`.

Signed by. Chief Risk Officer (`ORG-03`), with the Chief Data Officer (`ORG-06`) on the classification.

Done when. No `Personal data present` value contradicts a query result, and every `Yes` outside production has a gap row or a dated exception.

Shows up in. `RPT11 Security Position` through `SEC-07`, and `RPT09 Regulatory Coverage` through `CO-04 Data protection and marketing consent`, which reads `Partial`.

Goes wrong when. Nobody asks about the data-platform environments — often the largest holders of personal data outside production and rarely in the environment list.

Store connectivity, which is the centre of this chapter

Step 7.10 — Record the network links, de-duplicate them, and model the estate's real shape

Produce. `O50` rebuilt as the estate's real topology, diversity recorded per site, duplicate pairs resolved.

You need. Step 7.8's estate master. The provider's circuit list with contract references.

Where it lands.

> **`O50 Network Link`**
>
> Five columns: `Ref`, `From`, `To`, `Bandwidth`, `Diverse path`. Ten rows naming `O12` locations at both ends. Two problems. `NWL-01` and `NWL-09` are the same pair with different bandwidth and contradictory `Diverse path` answers, and so are `NWL-02` and `NWL-10`. And the topology is a ring — store to store to store to distribution centre to fulfilment centre to data centre to cloud to support centre and back. No store has a link to the data centre at all.

How.

1. Resolve the duplicate pairs. Two rows for one pair means two circuits, which is good — but then both rows carry the same `Diverse path` answer, or the answer is `No`.
2. Rebuild the topology as it is: stores to a regional or national aggregation point, aggregation to data centre and cloud. Delete the store-to-store links unless they genuinely exist.
3. Record `Diverse path` per site, not per link, and define it: two circuits on two physically separate routes. Two circuits from one provider down one trench is not diversity.
4. Record bandwidth as the committed rate during a trading peak, not peak capacity.

5. Carry the result into step 7.8's `Primary link` and `Secondary link` columns so the two registers cannot drift.

Signed by. Chief Information Officer (`ORG-05`), with the provider confirming diversity in writing.

Done when. No pair appears twice with contradictory answers, every trading site has a primary link, and `Diverse path` has a published definition.

Shows up in. No report reads `O50`, which is itself the finding: connectivity is invisible in all thirteen reports. Publish it in the Phase D pack.

Goes wrong when. `Diverse path` is answered from the order form. Diversity is a property of the physical route; the only reliable test is asking the provider to state both routes.

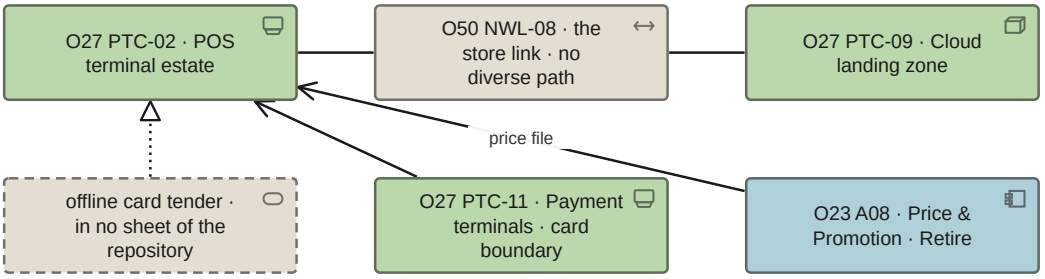


Figure 7.3 The store in ArchiMate 4: the till estate, the single link with no diverse path, the pin pad on the card-data boundary, and the offline tender capability that appears in no sheet of the repository [1].

Step 7.11 — Model offline capability explicitly, as four separate things

Produce. Four named capabilities per store format, each with a recorded position, the tender one tested.

You need. Step 7.8. The till vendor's offline specification. The acquirer's floor-limit and stand-in parameters, in writing.

Look it up. Volume 1, chapter 7, *The store has to keep selling*, separates the four and gives the design rule: the store's ability to complete a sale must not depend on any system outside the store [3]. Operator submissions to ICASA recorded network availability falling below 93% at Stage 2, so card authorisation failed for reasons outside the retailer's estate [10].

Where it lands.

> **046 Technology Building Block**
>
> Five columns, the second also headed `Ref`: `Ref`, `Technology building block`, `Ref`, `What it must do`, `Acceptance criteria`. `TBB-01 Edge at store` exists with the criterion "Edge at store in production for at least one service", which is far too weak. Replace it with four rows, one per offline capability, each with a testable criterion.

How.

1. Offline item lookup and pricing: the till holds a local price file and promotion rule set and prices any basket with no call. Usually present.
2. Offline transaction logging: the till journals locally and forwards on recovery, with duplicate suppression. Usually present.
3. Offline tender acceptance, the hard one. Cash is trivial; card needs a floor limit, terminal or controller stand-in, or store-and-forward authorisation. Each trades fraud risk for availability, and the parameters are a commercial decision wearing technical clothes.
4. Offline loyalty and gift card, separately: accrual can be deferred, redemption cannot be safely, because the balance is unknown.
5. Write the criterion as a test: *a store with its link down completes a card basket and the journal reconciles on recovery, demonstrated in a named store on a named date.*

Signed by. Chief Operating Officer (`ORG-07`), with the Chief Financial Officer (`ORG-02`) signing the floor limits, because they carry the fraud exposure.

Done when. All four are recorded per format and the tender one has a dated test in a real store rather than a vendor statement.

Shows up in. `049 Resilience Tier` and `055 Building Block`'s technology rows. `RME-01 Edge at store` reads `Adopted` in `047`; after this step you will know whether that is true.

Goes wrong when. "We have offline mode" is accepted as an answer. A store with lookup and logging but no tender stand-in cannot trade when the line is down, because in a card-heavy basket mix most baskets cannot be tendered at all.

Figure 7.4 The four offline capabilities from volume 1 chapter 7, each a separate thing to record and test. Number three is the one that decides whether a store can trade with the line down [3].

Step 7.12 — Set the resilience tiers and make them evidence something

Produce. `049` with tiers derived from business impact, recovery objectives consistent with the tier, and a test date against every row.

You need. Step 7.11. `023`'s `Criticality` column after chapter 6. The last disaster-recovery test report, if there is one.

Look it up. Volume 1, chapter 7, *What load-shedding did*, gives the four system-level consequences and is explicit that every figure in it is **historic peak from the 2022 to 2024 period, not a run rate**. Date every number you quote from it.

Where it lands.

> **`049 Resilience Tier`**
>
> Five columns: `Ref`, `Application`, `Tier`, `Recovery objective`, `Tested`. Twelve rows. Nothing forces `Tier` and `Recovery objective` to agree and they do not: `RES-05 WMS` is `Tier 1` with 4 hours, `RES-09 Loyalty & CRM` is `Tier 3` with 15 minutes, `RES-02 POS` is `Tier 3` with 4 hours. Eight of twelve read `Never tested`.

How.

1. Define the three tiers once, by recovery objective, and publish the definition. A tier that does not imply a number is a label.
2. Re-tier from business impact. The till is Tier 1 in a retailer; `RES-02` is `Tier 3`, the most consequential error in this sheet.
3. Make `Recovery objective` follow from `Tier`. Where an owner wants a tighter objective than the tier allows, change the tier or refuse the objective.
4. Replace `Tested` with a test date and the result. "Tested" with no date decays into fiction within a year.
5. Add the scenario the sheet has no room for — a site on generator with its link down — as a tested scenario per format, beside the sheet.

Signed by. Chief Operating Officer (`ORG-07`) per tier, with `GB-01` ratifying the definitions.

Done when. Every `Recovery objective` is consistent with its `Tier`, the till is Tier 1, and every row carries a test date inside twelve months or a dated plan to get one.

Shows up in. No report reads `049`, so publish it beside `RPT02`. `ZZ Checks` does not count untested tiers either — a check worth adding.

Goes wrong when. The load-shedding case is argued on an undated figure. Use the sourced ones and date them: Shoprite's diesel spend was reported at R1.3bn in September 2023 [9] and R754m in August 2024 [4]; Eskom reported R17.25bn in year-on-year national diesel savings after the 2024 suspension [5]. All historic peak, all dated, none a run rate.

RESILIENCE YOU CAN DEMONSTRATE

Which application's tier and recovery objective contradict each other most expensively?

Name the store, the date and the result of the last offline-tender test. If there is none, name the date you will run one.

Form D-2 — the offline capability worksheet

One row per trading format. The four columns are the four capabilities from step 7.11; each cell reads `Present`, `Partial` or `Absent`. Feeds four `046 Technology Building Block` rows. Signed by the Chief Operating Officer.

Format	Lookup and pricing	Local journalling	Card tender stand-in	Loyalty accrual
Convenience	Present	Present	Example, delete me	Partial

The till and payment estate, and the obligation that shapes the architecture

The card estate is the one part of a retail technology architecture governed by a contract rather than a statute, and architects routinely get the distinction wrong. The Payment Card Industry Data Security Standard is published by a council formed by the card schemes: a contractual obligation flowing from the merchant agreement and the scheme rules [6]. Nobody is prosecuted for failing it. An acquirer can impose assessments, raise fees, require a forensic investigation or decline to continue acquiring — a trading risk, and a retail board acts on trading risk faster than on legal risk.

Look it up. Volume 1, chapter 13, *PCI DSS and EMV — contractual, not statutory*, states the current version and explains why it prints no other date. Follow that discipline: cite the version, cite the date you checked it, settle card-present dates from the scheme directly.

Three architectural decisions move the obligation by an order of magnitude, and all three belong to Phase D.

- **Whether the till ever sees a card number.** Point-to-point encryption and a pin pad that never exposes card data to the application collapse the scope dramatically. In `O27` that means the pin-pad estate is a separate component from the till estate, with its own certification cycle — the shipped file has `PTC-02 POS terminal estate` and `PTC-11 Payment terminals` as two rows, which is right, and then gives both a plain `Vendor` and no certification column.
- **Whether the store network is segmented** from the corporate network and from guest wireless. `STD-11 Network segmentation standard` reads `Deprecated` and `Conformant` in `O32`, which is the worst possible pair of values on the standard that decides card-estate scope.
- **Where tokenisation happens and who holds the token vault.** The vault is the exit cost when the acquirer or gateway changes. `O47` `RME-05 PCI-scoped payment tokenisation` reads `Candidate`; `O26` `LTC-05` reads `Current`. Settle that before anybody designs against it.

Two further boundaries worth writing down. The requirements concerning payment-page scripts land on the e-commerce team, not the store team. And the pin pad and its kernel are certified assets with their own release cycle, independent of the point-of-sale application — so a till upgrade and a terminal certification are two programmes with two approval bodies.

Step 7.13 — Record the till and payment estate as its own scoped segment

Produce. The card-data environment as components, links and enforcement points, with scope stated and the obligation recorded as a control objective.

You need. Steps 7.4, 7.8 and 7.10. The merchant agreement. The acquirer's scope determination. The pin-pad certification schedule.

Where it lands.

> **`O63 Control Objective`**
>
> Five columns: `Ref`, `Obligation`, `Source`, `Control objective`, `Evidence`. `CO-02 PCI DSS` exists with `Evidence In breach`. The `Source` column holds a description — "Cardholder data protection across the estate" — rather than a source, and `Control objective` reads "Control that evidences PCI DSS", which is a tautology. Rewrite both: `Source` names the merchant agreement clause, `Control objective` names what must be true.

How.

1. Draw the segment: which components in `O27` are in scope, which links in `O50` cross the boundary, which enforcement points in `O65` sit on it.
2. State the scope decision and who made it. Scope is an architectural choice, not a security-team finding.
3. Record the pin-pad certification dates beside `O27` as a separate lifecycle from `Support end`.
4. Rewrite `CO-02`'s `Source` and `Control objective`, and raise an `O29 Gap` row for the `In breach` evidence with a `Due` month.
5. Check the direct database reads from chapter 6 step 6.12 against the boundary. `INT-007` has the e-commerce platform reading the POS store hourly, which is a scope question before it is an integration one.

Signed by. Chief Risk Officer (`ORG-03`) for the obligation, Chief Information Officer for the segment, `GB-01` ratifying the scope statement.

Done when. The boundary is drawn on one page, `CO-02` has a real source and objective, and the `In breach` evidence has a dated gap row.

Shows up in. `RPT09 Regulatory Coverage`, which lists six obligations and closes with *3 obligations have no mechanism or a mechanism in breach* — `CO-02 PCI DSS`, `CO-05 Product safety and recall` and `CO-06 Modern slavery and supply chain due diligence`.

Goes wrong when. Vendor products get named as the answer. The products that collapse card scope are examples of a pattern, never a recommendation, and the pattern is what goes in `O26`.

Security as architecture

Step 7.14 — Repair the security chain, link by link

Produce. `O41`, `O63`, `O64`, `O65`, `R16` and `R17` consistent with each other, the four absent controls carried as gaps.

You need. Steps 7.9 and 7.13. `RPT11 Security Position`, `RPT09 Regulatory Coverage`.

Look it up. Volume 1, chapter 10, *Classification, consent and POPIA*, settles what travels with the data and what therefore has to be enforced at the read.

Where it lands.

> **O41 Security Control**
>
> Five columns: `Ref`, `Security control`, `What it does`, `Position`, `Enforced at`. Eight rows. `Position` from `In place`, `Partial`, `Absent` — and four read `Absent`: `SEC-04 Attributable access`, `SEC-05 Interface logging`, `SEC-06 Periodic access review`, `SEC-07 Non-production masking`. `RPT11`'s footer puts it plainly: *4 controls are absent. A control that is absent is not a control.*

> **R16 Control Objective met by Co**
>
> Five columns: `Control objective`, `Obligation`, `Control`, `Control name`, `Position`. Six rows, mapped positionally — `CO-01 Consumer protection and pricing accuracy` met by `SEC-01 Identity lifecycle`, `CO-02 PCI DSS` met by `SEC-02 Design review`, `CO-03 Food safety and labelling` met by `SEC-03 Privileged access management`. None of the six pairings is semantic. Worse, the `Position` column carries the **obligation's** evidence status from `O63`, not the control's own `Position` from `O41` — so `SEC-02 Design review`, which is `In place`, appears as `In breach`.

How.

1. Rebuild `R16` semantically. An obligation is met by the controls that produce its evidence, and one obligation usually needs several.
2. Take `R16`'s `Position` from `O41` or delete the column. A relationship sheet reprinting the wrong parent's status is worse than one with no status.
3. Check `R17 Policy enforced at Enforcem` the same way. Four rows for five enforcement points, and the one left out is `EP-05 Non-production provisioning`, which reads "Most refreshes are unmasked": the point with the worst position is the one no policy references.
4. Test every `Met` in `O64 Security Policy` against the controls beneath it. `POL-01` — personal information protected wherever it is, explicitly across production and non-production — reads `Met` while `SEC-07 Non-production masking` reads `Absent`. Only `POL-03` reads `Breached`, and it is the only honest row.
5. Raise an `O29 Gap` row per absent control and per control objective with no mechanism.

Signed by. Chief Risk Officer (`ORG-03`), with the security delegate on `GB-01` countersigning each `Position`.

Done when. No policy reads `Met` above an absent control, every enforcement point is referenced by a policy, and `R16`'s `Position` either matches `O41` or is gone.

Shows up in. `RPT11 Security Position` for the controls, `RPT09 Regulatory Coverage` for the obligations. Both reprint their source sheet and add one computed footer; neither joins the two, which is why the chain can break in the middle unnoticed.

Goes wrong when. The four absent controls are treated as a security backlog. Three of them — attributable access, interface logging and non-production masking — are architectural properties, and no security team can add them to an estate not designed for them.

R16 PAIRS THEM POSITIONALLY AND COPIES THE WRONG STATUS

O63 obligation	O63 evidence	O41 control named by R16	O41 position	Clash
CO-01 Consumer protection and pricing	Partial	SEC-01 Identity lifecycle	Partial	—
CO-02 PCI DSS	In breach	SEC-02 Design review	In place	yes
CO-03 Food safety and labelling	Evidenced	SEC-03 Privileged access mgmt	Partial	yes
CO-04 Data protection and consent	Partial	SEC-04 Attributable access	Absent	yes
CO-05 Product safety and recall	In breach	SEC-05 Interface logging	Absent	—
CO-06 Modern slavery due diligence	No mechanism	SEC-06 Access review	Absent	—

RPT11: 4 controls are absent. RPT09: 3 obligations have no mechanism or one in breach.
Neither report joins the two sheets, so the middle of the chain is never tested.

Figure 7.5 `R16 Control Objective met by Co` as shipped: obligations paired with controls by row position, and a `Position` column copied from `063`'s `Evidence` rather than `041`'s own. Four of the six pairings clash [1].

THE SECURITY CHAIN, UNSOFTENED

Which obligation in `063` is met by a control that does not exist, and who signed that pairing?

`R16` reprints the obligation's status, not the control's. Name the three rows where that hides a working control behind a failing obligation.

Step 7.15 — Publish the Technology Standards Catalogue and pass the Phase D checkpoint

Produce. The Technology Standards Catalogue, signed, and a Phase D checkpoint record with `ZZ Checks` run before and after.

You need. Every step above. `031 Deliverable` rows `DLV-11` and `DLV-12`. `058 Governance Gate`.

Where it lands.

> **`031 Deliverable`**
>
> Four columns: `Ref`, `Deliverable`, `Phase`, `Status`. `DLV-11` and `DLV-12` both read `Approved` in the shipped file, over an estate with three components recorded past support, four controls absent and no connectivity register any report reads. Set both to `Draft`, do the work, then approve them and minute the date.

How.

1. Assemble the catalogue: `032` with successors for the deprecated rows, `R18` as a matrix rather than a cycling list, the conformance position per component, and the retail standards added at step 7.6.

2. Attach the currency paper from step 7.7 and the estate master from step 7.8. Those two are what a reader comes to this catalogue for.
3. Run `ZZ Checks` [8]. Record the before-and-after columns for *Platform components past support*, *Security controls absent* and *Control objectives without evidence*.
4. Take the checkpoint through `GB-01`, which holds `GT-1 G1 Design` and tests designs against principles, standards and the reference architecture — the three things this chapter has just made assessable.
5. Hand Phase E a list, not a narrative: the `O29 Gap` rows this chapter added, each with a domain, a severity and a due month.

Signed by. `GB-01 Architecture Board` for the catalogue, Chief Information Officer as accepting owner, `GB-02` noting the funding consequences.

Done when. `DLV-11` and `DLV-12` read `Approved` because they were, every standard has an owner and a status, and the `ZZ Checks` before-and-after is in the pack.

Shows up in. `RPT12`, `RPT10`, `RPT11` and `RPT09`. If the phase went well, `RPT10` 's footer counts expired dates rather than typed cells and `RPT11` 's counts fewer than four.

Goes wrong when. The catalogue is published without the estate master. A standards catalogue that cannot say which of four hundred sites conforms is a document, and Phase G will be unable to assess anything against it.

PHASE D — A REVIEWER'S QUESTIONS

How many `O27` components are past support by date rather than by typed `Position`, and what does `RPT10`'s footer say?

Which trading sites have no diverse path, and what is the revenue at risk for a day of outage at each?

The four offline capabilities, per format: which cells read `Absent`, and which of those is funded?

`POL-01` reads `Met` above an absent masking control. Who signed it, and what changes first — the policy or the control?

Name the three `O29 Gap` rows this chapter added that you expect Phase E to drop, and why.

THE CHAPTER IN TEN LINES

- Four questions in order: what is offered, what kind of thing offers it, which product, where it sits.
- `O25`'s `Category` column was filled by rotation. Republish it against a stated scheme.
- `R07` is positional and production-only. Rebuild it from deployment evidence.
- The same seven patterns sit in `O47`, `O26` and `O46` with seven status disagreements.
- Derive `Position` from `Support end`. `RPT10`'s footer counts typed cells, not expired dates.
- In retail the location master is the estate. `EXT X02` is the sheet that justifies the extra columns.
- `O50` has duplicate pairs, a ring topology and no store-to-data-centre link.
- Model offline capability as four separate things, and test the card tender in a named store.
- Load-shedding figures are historic peak from 2022–2024. Date every one.
- PCI DSS is contractual, not statutory — and card scope is decided by the architecture.

SOURCES FOR THIS CHAPTER

- **1.** Retail_TOGAF_Metamodel.xlsx, Halverson Stores Group — sheets O12, O25, O26, O41, O47, O48, O49, O50, O63, O64, O65, R07, R16, R17, R20, EXT Industry Extension
- **2.** Retail_TOGAF_Metamodel.xlsx, sheets O27 Physical Technology Compone and RPT10 Technology Currency — 12 records, 7 columns
- **3.** Retail — a starter pack for enterprise architects (volume 1), chapter 7, 'The store has to keep selling'
- **4.** IOL, "Shoprite spent R754 million on diesel due to load shedding" (1 August 2024) https://iol.co.za/news/energy/2024-08-01-shoprite-spent-r754-million-on-diesel-due-to-load-shedding
- **5.** Eskom, "Loadshedding continues to be suspended: R17.25 billion in year-on-year diesel savings" https://www.eskom.co.za/loadshedding-continues-to-be-suspended-r17-25-billion-in-year-on-year-diesel-savings/
- **6.** PCI Security Standards Council, PCI DSS standards page https://www.pcisecuritystandards.org/standards/pci-dss/
- **7.** The Open Group, TOGAF Standard — the Architecture Development Method https://pubs.opengroup.org/architecture/togaf9-doc/arch/
- **8.** Retail_TOGAF_Metamodel.xlsx, sheet ZZ Checks — fourteen computed checks against their thresholds
- **9.** The Citizen, "Shoprite's impressive results dampened by R1.3bn spent on diesel during load shedding" (September 2023) https://www.citizen.co.za/business/shoprite-results-growth-diesel-load-shedding-september-2023/
- **10.** ICASA, "Electricity Crisis — Final Findings Document" https://www.icasa.org.za/uploads/files/Electricity-Crisis-Final-Findings-Document-Long-Version.pdf

CHAPTER 8

Opportunities and Solutions

The repository hands you thirty-seven building blocks, twelve work packages and a roadmap. It hands you one gap. Everything in this chapter is about putting the missing half back underneath the half you were given.

Phase E is where the architecture stops describing the retailer and starts committing money [11]. You leave it with a gap register, building blocks carrying acceptance criteria, work packages, transition states you could stop at, costed alternatives and a funded roadmap.

Halverson Stores Group hands you the second half and almost none of the first. `O55 Building Block` holds thirty-seven blocks with acceptance criteria. `O30 Work Package` holds twelve packages worth $63.7m. `O53 Transition Architecture` holds three states, `O54 Architecture Alternative` five options, and `O31 Deliverable` records `DLV-13 Consolidated Gaps` as `Approved` [1][7].

And `O29 Gap` holds **one row**: `GAP-01 Merchandising & Assortment capability uplift not in place`, `Business`, building block `BBB-01`, severity `Medium`, due `2028-06`. `R12 Gap closed by Work Package` holds one matching instance. So `RPT07 Gap Analysis` is a one-line report and sixty-three and a half million dollars of work rests on a single `Medium` finding [1][2][4].

Read that row closely, because it shows how the register was built. The gap text is `BBB-01`'s own name with "not in place" appended, and `BBB-01` is the block the gap points at: the register was written backwards from the solution. And `O13 Business Capability` rates `C01 Merchandising & Assortment` `Critical`, so the only gap in the repository against the retailer's central capability is a `Medium`.

That is this chapter. Build the register that should have come first, then go back through the realization sheets and keep what survives contact with it.

Halverson Stores Group is invented and its data is real within this workbook. Milkwood Retail Group is volume 1's invented retailer and appears here only when pointing back at volume 1.

> **THE COSTS ARE IN DOLLARS**
>
> `Cost` on `O30`, `O56` and `O54` reads `$2.8m`, `$10.0m`, `$46m`. A South African investment paper is in rands. Convert before the first review, state the rate and the date once, and never run two currencies through one pack.

BEFORE YOU START THIS PHASE

The tool finds differences between two things somebody wrote down. What a retailer is *structurally* missing comes from volume 1.

- **Volume 1, chapter 11, *The patterns*.** The five application capabilities most retailers are missing entirely, and why the absence is silent.
- **Volume 1, chapter 14, *Build, buy or configure*.** Sourcing per capability, and the question that actually decides it.
- **Volume 1, chapter 17, *Gaps, and the roadmap in two plateaus*.** What an honest two-plateau roadmap looks like.
- **Volume 1, chapter 7, *The trading calendar*.** The dates Phase E cannot move, which is most of the ones it wants.

Run `ZZ Checks` and write its fifteen result values down before you touch anything. It is the cheapest before-and-after evidence you will produce, eight of its rows become gaps in step 8.9, and one of its own claims is already false [8].

Part one — the comparison, and what it can honestly tell you

A `Retail_Architecture_Comparison_Tool_TEMPLATE.xlsx` is built for this set. It takes two extracts of the repository and separates three things a naive diff reports identically: a **design delta**, where the target deliberately says something different and you have a genuine gap; **governed drift**, where the baseline moved under an approved change and the target is now stale; and **ungoverned change**, where a value moved and nothing explains it [3].

There are no export sheets. The insurance repository shipped three; this one ships none, so you build both extracts by hand [1].

There is no change log sheet either — no `02 Change Log`, and no `Change Ref` or `Approved By` column anywhere. The fifth paste target has nothing to paste from until change control exists, which is chapter 2's step. Say the consequence out loud: with an empty change log the tool cannot separate governed drift from ungoverned change, so every real difference classifies as `UNGOVERNED CHANGE` [1] [3].

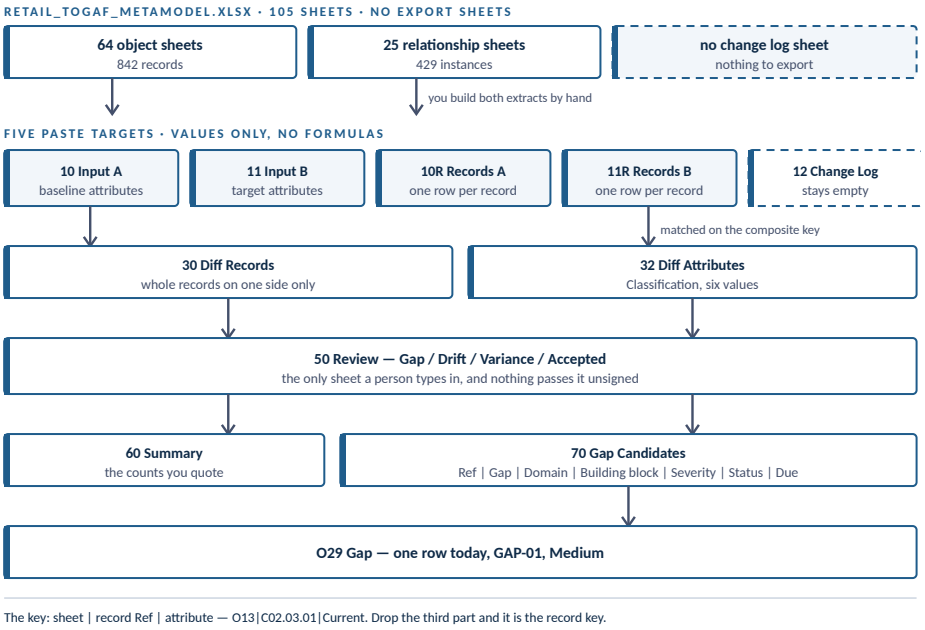


Figure 8.1 Producing the comparison by hand. This repository has no export sheets and no change log sheet, so two of the inputs are typed and the fifth paste target stays empty. The key at the foot is the only thing the whole tool runs on.

THE COMPOSITE KEY IS THE WHOLE TRICK

`O13|C02.03.01|Current` is three things joined by a pipe: sheet, record `Ref`, attribute. Every match in the tool is a lookup on that one string, and the record key is the same string with the third part dropped.

One piece of luck and one trap. The tool's key column is headed `Ref` and this repository's first column is `Ref`, so for once nothing has to be translated. The trap is renumbering: change a catalogue's `Ref` scheme between snapshots and every key changes, so the whole catalogue reports as removed and re-added. `O13`'s hundred rows on the four-part `C01.01.01` scheme are where that bites.

Step 8.1 — Produce the two extracts by hand

Produce. Four blocks of pasted values: baseline attributes, target attributes, and the two record lists.

You need. A named, dated copy of the repository as the baseline; a decision on what snapshot B is; the extract scope below; somebody who can write a Power Query unpivot.

How.

1. Copy the workbook, rename it with a date, treat the copy as read-only. There is no `04 Baselines` sheet, so the filename and a line in the deliverable are the whole baseline record.
2. For every sheet in scope, build one row per record per material attribute: `Key | Kind | Type | Type Name | Ref | Name | Attribute | Value`. `Kind` is `Object` or `Relationship`, `Type` is the sheet prefix, `Key` is the three-part composite.

3. Build the record list the same way, one row per record, down to `Name`. Relationship sheets go in it too — a link present on one side only is a real finding and `30 Diff Records` is the only place it shows.
4. Paste Special, Values Only, into `10`, `11`, `10R` and `11R`. Never link: external references break on rename.
5. Leave `12 Input Change Log` empty and note on `01 Config` that it is empty and why.
6. Check the counts. The repository states 842 records and 429 relationship instances, so a record list materially short of 1,271 rows a side has a sheet missing [8].

Signed by. Chief Enterprise Architect, `ROL-08`, recorded as `Prepared by` on `01 Config`.

Done when. `60 Summary` reads the same `Records in A` and `Records in B` as your own row counts, and no cell in the five input sheets holds a formula.

Shows up in. `60 Summary` — `Records in A`, `Records in B`, `Attribute rows compared`, `Change log entries supplied`. The last reads zero, correctly.

Goes wrong when. Somebody extracts `Description` as well. A diff across free text buries every real finding under rewording, and `O13` alone carries a hundred descriptions written to a template.

Where it lands.

> **`10 Input A Baseline · 11 Input B Target · 10R Records A · 11R Records B`**
>
> Values only. `10` and `11` take the attribute extract, `10R` and `11R` the record lists — without those two `30 Diff Records` is silently empty, which looks like a result. `12 Input Change Log` stays empty here.

Extract the attributes a gap could turn on, and nothing else.

Sheet	Attributes in the extract	Why it is material
`013 Business Capability`	`Criticality`, `Current`, `Target`, `Disposition`	The largest single source of candidate gaps
`020 Data Entity`	`Classification`, `Owner`, `Master system`, `Lineage`	A changed master system is a changed architecture
`023 Application Component`	`Disposition`, `Criticality`, `Health`, `Owner`	`Disposition` is the portfolio decision
`027 Physical Technology Compone`	`Support end`, `Position`	A support date that moves moves the roadmap
`030 Work Package`	`Cost`, `Start`, `End`, `Approval`, `Delivery`	The roadmap, and the rows that disagree with themselves
`043 Master Data Rule`	`Rule type`, `Golden source`	In a retailer the golden source *is* the argument
`049 Resilience Tier`	`Tier`, `Recovery objective`, `Tested`	Eight of twelve read `Never tested`
`R06 Application interfaces with`	`Type`, `Frequency`, `Managed`, `Logged`	Sixteen unmanaged, twenty unlogged, nine direct reads

Add `032 Standard` on `Status` and `Conformance`, and `EXT Industry Extension` whole — six rows, and a change there is a change to what the model can hold at all [7][9]. Several tab names are truncated at thirty-one characters; spell them as the tabs do.

Step 8.2 — Set the configuration before you read a single result

Produce. A dated, labelled, attributed comparison that states which question it answers.

You need. The baseline filename and date from 8.1; the date snapshot B was drawn; a decision on mode.

How.

1. Set `Snapshot A label` to the baseline's filename and date. There is no baseline register here, so the label is the register.
2. Set `Snapshot A date` to the day you froze the copy, not the day you ran the tool, and `Snapshot B label` and `Snapshot B date` to the state you extracted and the day you extracted it.
3. Set `Comparison mode`. `Baseline vs Target` makes an unexplained difference a gap candidate; `Snapshot vs Snapshot` makes it an ungoverned change. It changes no formula — it changes what you may conclude, and with no change log it is the only thing separating the two readings.
4. Set `Prepared by` to a role from `011 Role` and `Prepared on` to today.

Signed by. Chief Enterprise Architect, `ROL-08`.

Done when. `60 Summary` echoes all five fields and the dates bracket a period you can describe in one sentence.

Shows up in. `60 Summary`, top block. A blank there is a comparison nobody can date.

Goes wrong when. Snapshot B is dated the day you ran the tool. That is a fortnight after the extract, somebody edits a sheet in between, and you defend a difference that arrived after your own cut.

Where it lands.

> **`01 Config`**
> `Snapshot A label`, `Snapshot A date`, `Snapshot B label`, `Snapshot B date`, `Comparison mode`, `Prepared by`, `Prepared on`. Nothing is pre-filled and `60 Summary` reads back from here.

BEFORE YOU RUN THE COMPARISON, ANSWER THESE

What exactly is snapshot B? The target you drew in Phases B to D, or the repository as it stands today? One sentence.

If a difference turns out to be unexplained, who do you go and ask? Name the role, not the team.

What will you do with the count of unexplained differences, given that nothing in this repository can explain any of them?

Step 8.3 — Read the record-level differences

Produce. Whole records present on one side only, triaged into additions, removals and renumbering noise.

You need. Steps 8.1 and 8.2.

How.

1. Filter `30 Diff Records` on `Status` and read the two scopes separately: `From A` walks the baseline, `From B` reports only what the baseline lacked, so nothing is counted twice.
2. Group `REMOVED in B` by `Type`. Where nearly every record of a type is removed and a similar number added, compare the `Ref` values — a changed identifier shape means

renumbering, not removal.

3. Read the relationship rows alone. `R06 Application interfaces with` carries twenty-nine instances, and a row present only in B is an integration nobody decided to build — the most useful finding this sheet produces in a retailer, where integrations appear at a store, in a hurry, during a peak.
4. Account for every genuine removal as decommissioning, omission or error, and say which.

Signed by. Chief Enterprise Architect, `ROL-08`.

Done when. Every `REMOVED in B` row is accounted for and the renumbered ones are out of any count you say out loud.

Shows up in. `60 Summary` — `Records added` and `Records removed`, which size a catalogue build and not a gap register.

Goes wrong when. You quote the raw count. Renumber `013` and the sheet reports a hundred removals and a hundred additions out of 842 records. Nothing moved, and a register built off that raises a hundred capability gaps that do not exist.

Where it lands.

> **`30 Diff Records`**
>
> `Scope | Key | Kind | Type | Type Name | Ref | Name | In A | In B | Status`. `In A` and `In B` are 1 or 0 from a match on the record key; `Status` is `Present both`, `REMOVED in B` or `ADDED in B`. Relationships are compared here and only here.

Step 8.4 — Read the classification column, and know what it can mean

Produce. Attribute-level differences sorted into the six values the tool computes.

You need. Step 8.3 complete, so renumbering noise is already identified.

How.

1. Filter `32 Diff Attributes` on `Classification` and read in this order: `UNGOVERNED CHANGE`, `Added in B`, `Record removed in B`, `Governed drift + further change`, `Governed drift`, `Same`.
2. For every `UNGOVERNED CHANGE` row, read `Value in A` against `Value in B` and decide which story fits: a deliberate design decision nobody logged, or somebody editing a cell. Different owners, different remedies.
3. A design delta is deliberate and somebody can name the decision behind it. It earns a gap and eventually a work package, and `Added in B` is its usual shape.
4. An ungoverned change is a governance defect. It earns a change request against `061 Governance Change Request`. Writing it into `029 Gap` funds somebody's unlogged edit.

Signed by. Chief Enterprise Architect; the domain architect for the sheet co-signs their own domain, `ROL-09` to `ROL-12`.

Done when. Every row not classified `Same` has been read by a person and sorted into design delta, ungoverned change or noise.

Shows up in. `60 Summary`, the classification split — read it before the totals. A split that is almost entirely `UNGOVERNED CHANGE` is telling you about your change control, not your architecture.

Goes wrong when. Somebody takes the `UNGOVERNED CHANGE` count to an audit committee as a count of unauthorised changes. Here it counts *unexplained* differences, because nothing exists to explain any of them. **Look it up.** Volume 1, chapter 15, *What an inspector actually asks to see*.

Where it lands.

`32 Diff Attributes`

`Scope | Key | Type | Type Name | Ref | Name | Attribute | Value in A | Value in B | Differs | Logged change | Change Request | Logged new value | Classification`. `Logged change` matches the key against `12 Input Change Log`. The sheet makes no judgement about significance — that is step 8.5.

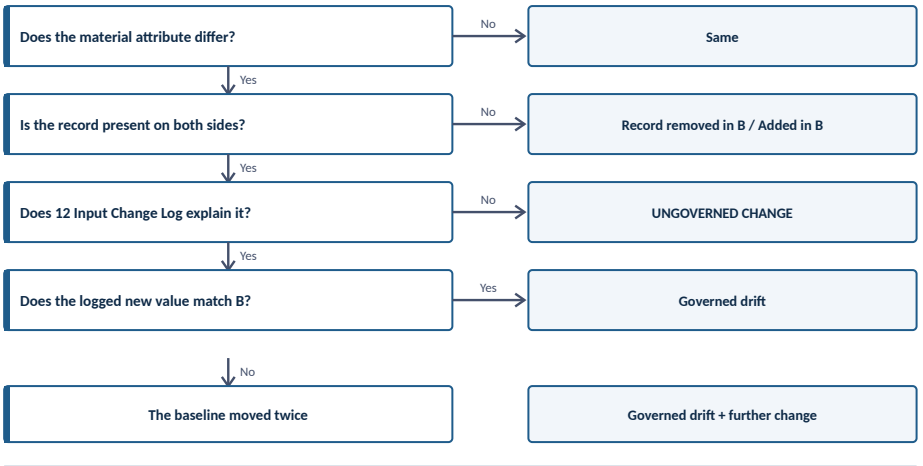


Figure 8.2 What the `Classification` column decides: four questions, six answers, and no judgement about whether a difference matters. With `12 Input Change Log` empty, every real difference leaves at question three.

Step 8.5 — Classify every difference on `50 Review`, with a name against it

Produce. A human verdict on each difference, with a rationale and an owner.

You need. Steps 8.3 and 8.4; `011 Role`; ninety minutes with each domain architect.

How.

1. Copy each difference needing a decision from `32 Diff Attributes` into `50 Review`: key, type name, `Ref`, attribute, tool classification.
2. Set `Your classification` to one of four: **Gap**, a real architectural difference; **Drift**, the baseline moved and the target is stale; **Variance**, a modelling difference with no architectural meaning; **Accepted**, deliberate and fine.
3. Write `Rationale` as a sentence a stranger can read in eighteen months. "Price & Promotion Engine disposition moved from Retire to Invest after the shelf-label assessment" is a rationale. "Agreed" is not.
4. Set `Owner` to a role from `011 Role`. All eighteen read `Performed by: Person`, so nobody may name a committee.
5. Set `Action`: a Gap goes to `029`, a Drift reissues a target artefact, a Variance is a modelling correction, an Accepted row gets a line in the deliverable.

Signed by. Each row by its named `Owner`; the sheet by the Chief Enterprise Architect.

Done when. Every row has a classification, a `Rationale` of more than four words and an `Owner` from `011 Role`.

Shows up in. `60 Summary`, the review split, and `70 Gap Candidates`, which carries only the Gap rows.

Goes wrong when. The architect classifies all of it alone in an afternoon. The rule is the point of the sheet: **a difference is not a finding until a named person says it is.** A tool that presents raw differences as findings loses its audience permanently, and quickly.

Where it lands.

> **`50 Review`**
>
> `Key | Type Name | Ref | Attribute | Tool classification | Your classification | Rationale | Owner | Action`. The only sheet anyone types in, and nothing reaches `70 Gap Candidates` until a row here reads `Gap`.

Three worked rows, so the shape is concrete [9].

Key	What the values say	Verdict	Owner		
`O23\	A02\	Criticality`	`POS` is `Criticality` `Low`	Gap — the only system in the estate that takes money is rated below the analytics platform	Chief Operating Officer
`O20\	E05\	Master system`	`Price` is mastered in `Workforce Management`, and `O43 MDR-05` names `POS` as its `Golden source`	Gap — two sheets, two golden sources, one price	Chief Data Officer
`O49\	RES-01\	Tested`	`Merchandising System`, `Tier 1`, `15 min`, `Never tested`	Gap — an untested fifteen-minute objective is an assertion	Head of Operations

Step 8.6 — Lift `70 Gap Candidates` and set the severity yourself

Produce. A block of gap candidates shaped to paste into `O29 Gap`, with a severity you can defend.

You need. `50 Review`; `O63 Control Objective`; `RPT09 Regulatory Coverage`; the trading calendar.

How.

1. Read `70 Gap Candidates`. It carries only rows a person classified `Gap`, in `O29 Gap`'s own column order — `Ref | Gap | Domain | Building block | Severity | Status | Due` — plus two columns `O29` does not have, `Source difference` and `Owner`. `Severity` arrives as a placeholder because the tool has no basis on which to set it.
2. Set `Severity` on **consequence, not size**. Critical: a displayed price, a shelf label, a recall or a dispensing obligation fails. High: a control objective in `O63` has no mechanism, or an application the group cannot trade without has no tested recovery. Medium: cost, margin or change lead time. Low: modelling hygiene.
3. Cross-check `RPT09 Regulatory Coverage`. `CO-02 PCI DSS` and `CO-05 Product safety and recall` read `In breach`, `CO-06 Modern slavery` reads `No mechanism`, and `CO-01 Consumer protection and pricing accuracy` and `CO-04 Data protection and marketing consent` read `Partial` [4]. Any gap touching those five is Critical before you argue.
4. Set `Due` off the retailer's calendar rather than the programme's convenience. Step 9.4 turns that calendar into a date rule.
5. Decide where `Source difference` and `Owner` go, because `O29 Gap` has neither column: add them, or keep the comparison workbook as the attachment of record and cite its `50 Review` key inside the gap text.

Signed by. Chief Enterprise Architect proposes; `GB-01 Architecture Board` sets severity on anything Critical.

Done when. No row carries a placeholder severity, and every Critical row names the obligation or the trading consequence behind it.

Shows up in. `RPT07 Gap Analysis`, once step 8.10 has loaded `O29`.

Goes wrong when. Severity is set on how annoying the gap is to the architect. It is the ranking an investment committee will use to cut your programme, so set it as though somebody will take the bottom third away.

Where it lands.

`70 Gap Candidates`

Shaped to paste into `O29 Gap`, with `Source difference` and `Owner` alongside. Give candidates a distinct `Ref` prefix while they are candidates, so a half-finished paste cannot be mistaken for the register. `Building block` stays empty until 8.11.

Part two — the three sources of gap the tool cannot find

A diff compares two things somebody wrote down. It cannot find what neither of them says, and in a retailer the gaps outside it are usually the larger half.

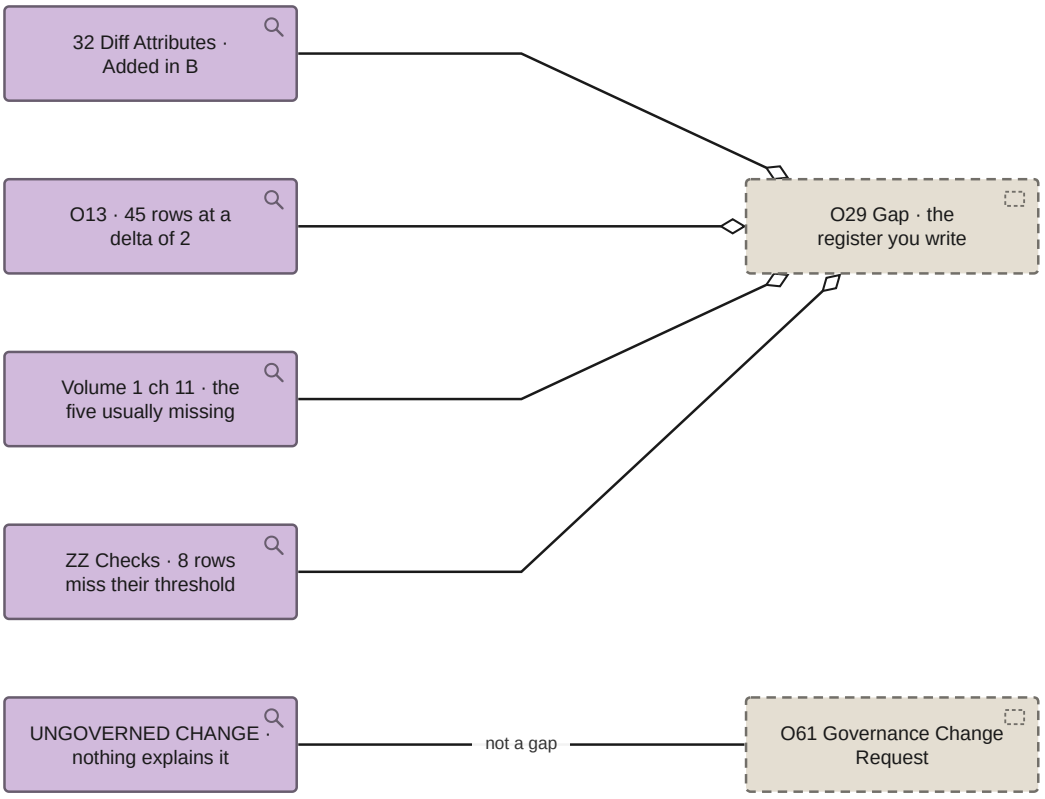


Figure 8.3 Four sources aggregate into the gap register. The fifth is the one that looks like a gap and is not: an ungoverned change earns a change request against `O61`, never a work package.

Step 8.7 — Derive capability gaps from the maturity delta, not from the heatmap

Produce. A ranked list of capability gaps from `O13 Business Capability`.

You need. `O13`; `RPT03 Capability Heatmap`; `R22 Org Unit owns Capability` for the owner.

How.

1. Compute `Target` minus `Current` across all hundred rows — ten at level 1, thirty at level 2, sixty at level 3 [1].
2. Read what the sheet is doing before you use it. **Every one of the hundred rows has `Target` above `Current`**: fifty-five at a delta of one, forty-five at two. `Disposition` reads `Build` on exactly those forty-five and `Sustain` on the other fifty-five, so the column is the arithmetic restated. Re-derive it yourself.
3. Filter to a delta of two **and** `Criticality` `Critical` or `High`. Eighteen rows, and a gap list you can defend in a room.
4. Read the two level-1 rows in it — `C02 Pricing & Promotion`, `Critical`, 2 against 4, and `C06 Digital Commerce`, `High`, 3 against 5 — then read the one that is not in it and should worry you more. `C01 Merchandising & Assortment` is `Critical`, sits at maturity 1, carries a target of 2 and a `Disposition` of `Sustain`. Somebody has set the retailer's central capability a target of barely competent, and that decision needs a name against it.
5. Go two levels down, because `RPT03 Capability Heatmap` shows level 1 only — ten rows of a hundred. Six `Critical` rows sit below level 1 at a delta of two, among them `C01.02 Merchandising Optimisation` at 3 against 5, `C05.02.02 Store Operations Reporting – Operate` at 1 against 3, and `C09.01 Space Assurance` at 3 against 5.
6. Aggregate to the level you will fund. One gap per row is a hundred gaps and a register nobody reads.

Signed by. The owning executive from `R22` per level-1 capability; `GB-01 Architecture Board` for the set.

Done when. Every gap names a capability `Ref`, a delta and a criticality, and the count is one a board will sit through.

Shows up in. `RPT03 Capability Heatmap` — its `Gap` column is the delta, and after this step every row at `Gap` 2 with `Criticality` `Critical` or `High` has an `O29` row behind it.

Goes wrong when. You take `RPT03` as the gap analysis. In its ten rows of level-1 summary `C05.02.02 Store Operations Reporting – Operate` is invisible, and that is the capability that tells head office whether anything it sent to a store actually happened.

Look it up. Volume 1, chapter 19, *Capability to application — the coverage map* — and map by name, never by number. Volume 1's `C-01…C-20` are local identifiers and this repository runs `C01…C10`.

Where it lands.

> **`029 Gap`**
>
> For a capability gap, `Domain` is `Business` and `Gap` is a shortfall with a number in it: "Pricing & Promotion at maturity 2 against a target of 4; Critical; the price file reaches the shelf by hand." Not "improve pricing".

Form 8-A — capability gap candidates

One row per gap you will fund, from the eighteen-row filter in 8.7. Columns are `029 Gap`'s own in sheet order, less `Status` — type that into Excel as `Open`. A first pass usually needs eight to fourteen rows. Signed by the owning executive from `R22 Org Unit owns Capability`.

Ref	Gap	Domain	Building block	Severity	Due
GAP-02	Example, delete me — C02 at maturity 2 against target 4	Business		Critical	2028-06

Step 8.8 — Test the five capabilities a retailer is usually missing entirely

Produce. Five explicit findings, each either "present and evidenced" or a gap.

You need. Volume 1 chapter 11; `O23`; `O20`; `O19 Contract`; `O37 Data Attribute`; `RPT13 Orphan Analysis`.

How.

1. Take the five by name, not by number. Volume 1's `A-nn` identifiers are local, and this repository runs twelve applications where volume 1 has twenty-five conceptual boxes.
2. **Merchandise financial planning and open-to-buy.** Nothing in `O23`'s twelve applications is it, no `O20` entity holds a plan, and `C10 Finance & Margin Management` is `Low` criticality at

maturity 1 against a target of 3. The open-to-buy that governs every buying decision in the group is unmodelled. Gap, Business.

3. **Product information and content.** `E01 SKU / Article` carries six attributes in `O37` — identifier, name, status, effective date, owner, classification — and not one is content. No image, no nutrient value, no allergen statement, no translation: a commercial gap and a labelling exposure at once. Gap, Data.
4. **Order management, the half behind the website.** `A04 Order Management System` exists, which beats most retailers. Then read the rest of the row: `Replace`, `Criticality Low`, `Health Poor`, in service since 2010, owner `Risk & Compliance` [9] — and `E10 Customer Order` mastered in `Price & Promotion Engine`. The box is there and the capability is not. Gap, Application.
5. **Store task and execution confirmation.** `A10 Workforce Management` is `Retire`, nothing records whether a store did what it was told, and `C05.02.02` is `Critical` at maturity 1. Every central system issues work to a store and none can confirm it was done. Gap, Business.
6. **Supplier income and trading terms.** `O19 Contract` holds ten contracts with a counterparty and an expiry and nothing about terms. `A11 Supplier Portal` is `Invest`, and `RPT13 Orphan Analysis` reports no business service maps to it [4]. Supplier income is a material part of retail gross margin and nothing here accrues it. Gap, Data and Business.

Signed by. Chief Enterprise Architect; the Chief Data Officer co-signs the product-information and supplier-income findings, the Chief Financial Officer the open-to-buy one.

Done when. All five carry a written verdict, including any you decide is present, with the evidence named.

Shows up in. `O29 Gap`, and indirectly `RPT13` — an application named for a capability it does not have is usually also an application nothing references.

Goes wrong when. You conclude a capability is present because an application carries its name. `A04 Order Management System` is named for it and rated `Low` and `Poor`.

Look it up. Volume 1, chapter 11, *The patterns*, and the sentence under it that matters most: none of the five sits on the path of a financial transaction, so nothing bounces when they are absent and the gap stays invisible until gross margin becomes unexplainable. All five are where margin is made and lost rather than where cash moves, which is why all five are funded out of somebody else's budget or not at all.

Where it lands.

> **`O29 Gap` · `O55 Building Block`**
>
> Each becomes one gap, not several, and one building block at 8.11. A capability split across four blocks is one nobody will fund.

THE FIVE, AGAINST YOUR OWN RETAILER

Which of the five does your retailer genuinely have, with evidence somebody outside the architecture team would accept?

For the ones it does not have — whose budget would have had to pay, and who decided not to?

Which single one of the five, if it existed, would close the most rows in your gap register?

Step 8.9 — Harvest the integrity findings and the orphans as gaps with numbers attached

Produce. A set of gaps whose acceptance criterion is already written, because `ZZ Checks` wrote it.

You need. `ZZ Checks`; `RPT01 Dashboard`; `RPT13 Orphan Analysis`; `O04 Measure`; `O05 Principle`.

How.

1. Read `ZZ Checks`: fifteen rows, each with a `Result` and a `Threshold`, and a closing line worth quoting — the thresholds are what a healthy estate would show, the results are what this one does, and that distance is the work [8].
2. Check its own numbers first, because one is wrong. It claims `Object types populated` 65 against "Every type carries records" while `O59 Compliance Assessment` is empty and `O29 Gap` holds one row. The integrity sheet fails its own first check, and it is the sheet you were about to use as evidence.
3. Write every row where `Result` differs from `Threshold` as a gap — eight of them. The threshold is the acceptance criterion and the result the baseline, so these gaps are testable the day you write them.
4. Read `RPT13 Orphan Analysis`: `A11 Supplier Portal`, `Invest`, and `A12 Retail Analytics`, `Replace`, neither with a business service mapping to it. An orphan is not always wrong. It is always worth a question.
5. Ask about `A12` first. An analytics platform marked `Replace` that nothing maps to, in an estate where `E04 Assortment` and `E11 Transaction / Basket` both read `Lineage Absent`, means nobody can trace a margin number back to a basket. **Look it up.** Volume 1, chapter 10, *Ownership, lineage and the gate that actually works*.

6. Tie each gap to the measure that will show it closing, and name the principle it offends — checking that principle's `Status` first.

Signed by. Chief Enterprise Architect; `GB-05 Architecture Function` owns the check sheet.

Done when. Every row that misses its threshold has a gap or a written reason it does not need one.

Shows up in. `RPT01 Dashboard`, which carries the same counts and is what an executive actually looks at.

Goes wrong when. You raise a gap against a principle still `Draft`. `PRN-05 Common Use Applications`, `PRN-10 Data is Shared`, `PRN-15 Requirements-Based Change` and `PRN-20 Least Privilege` are all `Draft` in `O05` [6], and a gap raised against a draft principle gets argued away in the meeting.

Look it up. Volume 1, chapter 18, *Six numbers that tell you whether the estate is healthy*.

Where it lands.

> **`O29 Gap`**
>
> One gap per failing check, with the number inside the `Gap` text: "Sixteen of twenty-nine application interfaces are unmanaged, against a threshold of zero."

`ZZ Checks` row	Result against threshold	The measure it closes against
Interfaces unmanaged	16 against 0	`MSR-08 Managed interfaces`, baseline 28%
Interfaces with no logging	20 against 0	`MSR-08`, and `OBJ-08` at 18 months
Direct database reads	9 against 0	`PRN-10 Data is Shared` — still `Draft`
Platform components past support	3 against 0	`MSR-07 Components within vendor support`, baseline 50%
Security controls absent	4 against 0	No measure exists. Say so, or add one
Control objectives without evidence	3 against 0	`MSR-01 Obligations with evidence`, baseline 38%
Data entities without lineage	2 against 0	`MSR-11 Lineage coverage of regulatory reports`, baseline 38%
Work packages past end date	7 against 0	Not a gap — a plan defect, step 9.13

Part three — from gaps to blocks to packages

Step 8.10 — Rewrite the gap register, and set the numbering scheme

Produce. `029 Gap` populated properly, for the first time in this repository's life.

You need. The candidates from 8.6, the capability gaps from 8.7, the five from 8.8, the integrity gaps from 8.9.

How.

1. Keep `GAP-01`'s `Ref` and rewrite everything else in the row. Its text is `BBB-01`'s name with "not in place" appended, its severity is `Medium` against a `Critical` capability, and its `Building block` was filled before a gap existed to need one.
2. Set the `Ref` scheme and write it down: `GAP-01` upward, zero-padded, never reused. Nothing in the sheet records your choice, so the deliverable has to.
3. Write `Gap` as a shortfall with a quantity and a consequence. "Price is mastered in Workforce Management while the master data rule names POS as the golden source, so nothing can answer what this article will ring at tomorrow" is a gap. "Improve data management" is a budget request.
4. Set `Domain` to `Business`, `Data`, `Application` or `Technology`, spelled as `055`'s `Domain` column spells it — the only thing joining the two sheets.
5. Leave `Building block` empty on this pass. You cannot fill it honestly until 8.11, so the register gets written twice.
6. Sanity-check the count. Over about thirty rows and you are recording differences rather than gaps; under ten, against sixteen unmanaged interfaces and eight untested recovery objectives, and you are being polite.

Signed by. `GB-01 Architecture Board` approves the register; each row's owning executive approves its own row.

Done when. `RPT07 Gap Analysis` has rows in it, every row has a `Severity` and a `Due`, and `031 Deliverable` can honestly carry `DLV-13 Consolidated Gaps` as `Approved` — which today it does on the strength of one row.

Shows up in. `RPT07`, all seven columns, and `RPT01 Dashboard`'s `Gaps overdue`, which reads 0 because the register is nearly empty rather than because nothing is late.

Goes wrong when. One gap per difference. The tool will hand you eight rows on `049 Resilience Tier` alone. That is one gap with eight instances.

Where it lands.

029 Gap

`Ref | Gap | Domain | Building block | Severity | Status | Due`. `Severity` is set on consequence, `Status` runs `Open`, `In progress`, `Closed`, and `Due` is a date somebody outside the programme chose. There is no `Owner` column and no `Principle` column; 8.6 decided where those live.

BEFORE THE REGISTER IS SIGNED

Which gap would you be most embarrassed to have missed if an inspector read this sheet next month?

Which gap are you raising because it is real, and which because it funds something you already wanted?

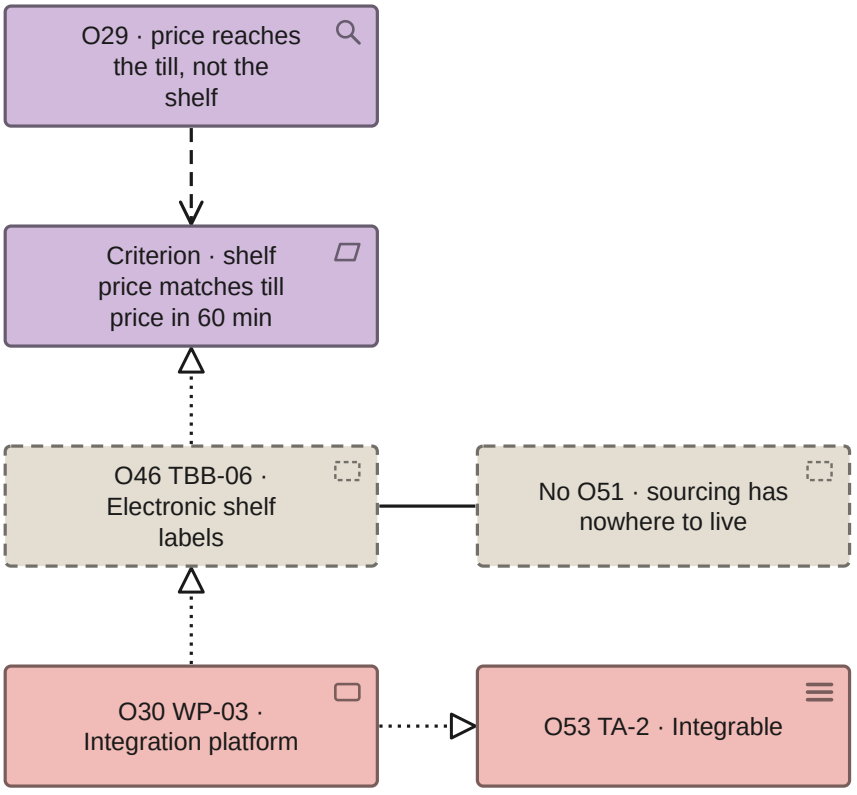


Figure 8.4 The Phase E ladder in ArchiMate 4, one chain worked down. The gap influences the acceptance criterion, the building block realises it, the work package realises both the block and the transition state. The dashed box is the rung this repository does not have.

Step 8.11 — Author the building blocks in their domain sheets

Produce. One building block per gap, with a testable acceptance criterion, in the right domain sheet.

You need. `O29` from 8.10; `O35`, `O44`, `O45`, `O46`, `O55`; `R09 Requirement satisfied by Bu`.

How.

1. Write each block in its domain sheet. `O35 Business Building Block` keys on `Capability`, `O44 Data Building Block` on `Entity`, `O45 Application Building Block` on `Application`, `O46 Technology Building Block` on a technology `Ref` — and `O46`'s third column is *also* headed `Ref`, holding `T01…T07`. Then mirror into `O55 Building Block`, the thirty-seven-row union with a `Domain` column and the same `Ref` values: author once, mirror once.
2. Rewrite the shipped criteria, because three of the four families are templates. All ten business blocks read "*X* measured and owned". All seven technology blocks read "*X* in production for at least one service" — a pilot, not an estate. All ten data blocks read "No *x* record without a named owner", in lower case, which is how you know a generator wrote them.
3. Check whether the baseline already passes. `DBB-01 SKU / Article mastering` asks for no record without a named owner, and `O20` already names `Head of Finance` as the owner of `E01 SKU / Article` — so it passes today, while the real problem, `E01` mastered in `POS` against a `Golden source` of `Demand Forecasting & Replenishment` in `O43`, goes untouched. **A criterion the baseline already meets is not a criterion.**
4. Keep the ten application criteria. They are the only testable ones in the sheet: "No unmanaged interface to or from *X*" is provable against `R06` in an afternoon.
5. Write the technology criteria properly, because `O46` holds the genuinely retail blocks — `TBB-01 Edge at store`, `TBB-02 Real-time inventory`, `TBB-06 Electronic shelf labels`, `TBB-07 Peak-elastic infrastructure`. For `TBB-06` the criterion writes itself: every price change effective at the shelf edge inside the same window as the till, with a report that proves it. Where two prices are displayed at once the lower binds the retailer [10], so that is a legal control.
6. Trace back. `R09` carries thirty-one instances and eight read `No` on `Traced`; `RPT06` reports the same eight. Each is either a gap you have not written or a requirement you should delete.

Signed by. The domain architect for the sheet, `ROL-09` to `ROL-12`; the Chief Enterprise Architect for `O55`.

Done when. `O55` holds one block per gap, every `Acceptance criteria` cell carries a number, a date or a count, and `R09`'s eight `No` rows are zero or explained.

Shows up in. `RPT06 Requirements Traceability`, `Reaches a block` — twenty-three `Yes` and eight `No` today.

Goes wrong when. Nobody notices the holes in the numbering. The object sheets run `001` to `066` with **`034` and `051` absent**, and the one that costs you is `051`: **there is no Solution Building Block sheet.** The ladder from architecture building block to work package is a rung shorter here, so build, buy or configure has nowhere to live. Put it in `What it must do` or add a `Sourcing` column, and say which in the deliverable. `031 Deliverable` covers what the missing `034` would have held.

Look it up. Volume 1, chapter 14, *Build, buy or configure* — and the question that decides it, which is not "is this differentiating" but which capabilities carry a local specification that changes faster than a vendor's release train.

Where it lands.

> **`035 · 044 · 045 · 046 · 055 Building Block`**
>
> Ten business, ten data, ten application and seven technology blocks in the domain sheets; `055 Building Block` — `Ref | Domain | Building block | What it must do | Acceptance criteria` — is the thirty-seven-row union.

Form 8-B — building block and the criterion that tests it

One row per block, in `O55 Building Block`'s own column order. A criterion with no number, date or count goes back to its author. Eight to twelve rows is a page; continue on a second copy. Signed by the domain architect for the `Domain` named.

Ref	Domain	Building block	What it must do	Acceptance criteria
TBB-06	Technology	Example, delete me — electronic shelf labels	Change a shelf price without labour	Shelf price matches till price at every store within 60 minutes of the price file, reported daily

Step 8.12 — Delete `R13` and rebuild it from the block side

Produce. `R13 Work Package delivers Build` rebuilt so each block is delivered by exactly one package.

You need. `O55`; `O30`; `R13` as shipped, which you are about to discard.

How.

1. Read `R13` as shipped, because it teaches you something about generated repositories. Thirty-seven instances allocated by rotation: `WP-01 Governance foundations` is recorded as delivering `BBB-01 Merchandising & Assortment capability uplift`, `DBB-03 Store / Location mastering`, `ABB-05 WMS target` and `TBB-07 Peak-elastic infrastructure` — four unrelated blocks across four domains, while `O30` records `WP-01`'s `Domains` as `Business` alone [1][2].
2. Delete the thirty-seven rows and rebuild from the block side rather than the package side: for each block in `O55`, name the one package that delivers it, carrying `Domain` on the relationship row.
3. Check two counts by hand. Every block appears exactly once — two packages delivering one block means neither owns the criterion. Every package appears at least once — a

package delivering no block is a project looking for a justification.

4. Reconcile `O30`'s `Domains` against the domains each package now carries. Where they disagree it is usually `Domains` that is wrong.

Signed by. Chief Enterprise Architect; `GB-01 Architecture Board` for the set.

Done when. The row count of `R13` equals the row count of `O55`, no block appears twice, no package is absent.

Shows up in. Nowhere, and that is worth knowing. Thirteen reports and not one is a building-block coverage report. Compute the counts yourself, or add a row to `ZZ Checks` and keep it there.

Goes wrong when. You leave the shipped rows because they look populated. Populated is not true, and `RPT08 Roadmap` will then present a package whose blocks have nothing to do with it.

Where it lands.

> **`R13 Work Package delivers Build`**
>
> `Work package | Package name | Building block | Domain | Block name`. The tab name is truncated; the sheet's own title row reads "Work Package delivers Building Block".

Step 8.13 — Re-cut the work packages, then link gap to package

Produce. `O30 Work Package` you can defend, and `R12 Gap closed by Work Package` populated past one row.

You need. `O29`; `O55`; `R13` rebuilt; `O30` as shipped; `O52 Implementation Factor`.

How.

1. Read the twelve shipped packages. Plausible as a set — governance, data ownership, integration, currency, security, analytics, core renewal, automation, resilience, compliance, customer experience, supplier integration — and each tagged to exactly one domain, three per domain, $19.9m technology, $19.4m business, $13.4m data, $11.0m application. That symmetry is the tell.
2. Re-cut so each package delivers a coherent set of blocks, and name every domain it touches in `Domains` as a comma list. In a retailer the packages that work cross domains because the failure crosses domains: "price to the shelf edge" carries `C02 Pricing & Promotion`, `E05 Price`, `A08 Price & Promotion Engine` and `TBB-06 Electronic shelf labels` at once, because what the shopper and the inspector both see is a label.
3. Keep the count in the low teens. Twelve is about right for $63.7m; twenty-five is a portfolio pretending to be an architecture.
4. Fill `R12 Gap closed by Work Package`, one row per gap-and-package pair. One instance today.

5. Count both ways and put both numbers in the pack. Gaps with no package are the unfunded architecture; packages closing no gap are what an investment committee will remove, and better that you do it first.
6. Fill the `Building block` column of `O29` that you left empty at 8.10.

Signed by. Chief Enterprise Architect for the shape; `GB-02 Investment Committee` for the money at `GT-0 G0 Charter`.

Done when. Every row of `O29` appears at least once in `R12` and `RPT07`'s `Work package` column has no blanks.

Shows up in. `RPT07 Gap Analysis`, whose last column is the join through `R12`.

Goes wrong when. `O30` has no owner column, no dependency column and nowhere to record why a package is cut the way it is. That argument is what nobody can reconstruct eighteen months later, so here it lives in the Phase E deliverable — one paragraph per package, or the next architect re-cuts them from scratch.

Look it up. Volume 1, chapter 12, *The legacy question*.

Where it lands.

`O30 Work Package · R12 Gap closed by Work Package`

`O30` — `Ref | Work package | Domains | Cost | Start | End | Approval | Delivery`, where `Approval` is `Approved`, `In flight` or `Not started` and `Delivery` is `Not started`, `In flight`, `Delivered` or `Past end date`. `R12` — `Gap | Gap name | Severity | Work package | Package name`, one instance.

Step 8.14 — Define transition states you can actually stop at

Produce. `O53 Transition Architecture` with exit criteria written out, and `R15` complete.

You need. `O30` re-cut; `R15 Transition contains Work Pa`; `O52`.

How.

1. Read the three shipped states: `TA-1 Governed and supportable`, `TA-2 Integrable`, `TA-3 Business visible`. The sequence is right. Nothing else about them is finished.
2. Fix the exit criteria. The column holds "5 exit criteria", "6 exit criteria", "7 exit criteria" — a count is not something anybody can pass. Write them out, or say plainly that the cell is a pointer into the deliverable.
3. Fix the order, which is the serious finding. On the dates `O30` carries, `TA-2`'s single package starts 2027-06, earlier than anything in `TA-1`; `TA-1` runs to 2031-04, later than anything in `TA-3`; and all three are simultaneously open from 2027-10 to 2030-11, thirty-eight months. Three states meant to be sequential are one long state with three names.

4. Account for the five packages in no state at all. `R15` carries seven instances covering `WP-01`, `WP-02`, `WP-04`, `WP-05`, `WP-03`, `WP-07` and `WP-11`, leaving `WP-06`, `WP-08`, `WP-09`, `WP-10` and `WP-12` — $34.6m — outside every transition. Either they sit in a state or they are business as usual outside the programme.
5. Make each state testable by somebody who is not an architect. `TA-1` exits when no platform component runs past vendor support and every entity in `O20` has a named individual owner. `TA-2` exits when the price file and the article master reach every till and every shelf-label controller through managed integration and the nine direct database reads are gone. `TA-3` exits when the first thing a shopper would notice is live in a trading store.

Signed by. `GB-01 Architecture Board`; `GB-04 Executive Committee` is the audience for the overlap finding.

Done when. Every package in `O30` appears in `R15` or has a written reason not to, and each state's exit criteria are testable statements rather than a count.

Shows up in. `RPT08 Roadmap`, by `Start` and `End`. The report has no transition column, so the states live in `O53` and in your own figure.

Goes wrong when. The three states are drawn as three equal thirds. `TA-1` is the longest and the least visible, and a sponsor who has not been told that loses patience at month fourteen — in a retailer, the month after a peak in which nothing was allowed to change.

Look it up. Volume 1, chapter 17, *Gaps, and the roadmap in two plateaus*, and chapter 16, *The roadmap, and what is true on 31 December*.

Where it lands.

> **`O53 Transition Architecture · R15 Transition contains Work Pa`**
>
> `O53 — Ref | Transition architecture | Description | Work packages | Exit criteria`, three records, where `Work packages` is a comma list duplicating what `R15` holds properly. `R15` — seven instances against twelve packages.

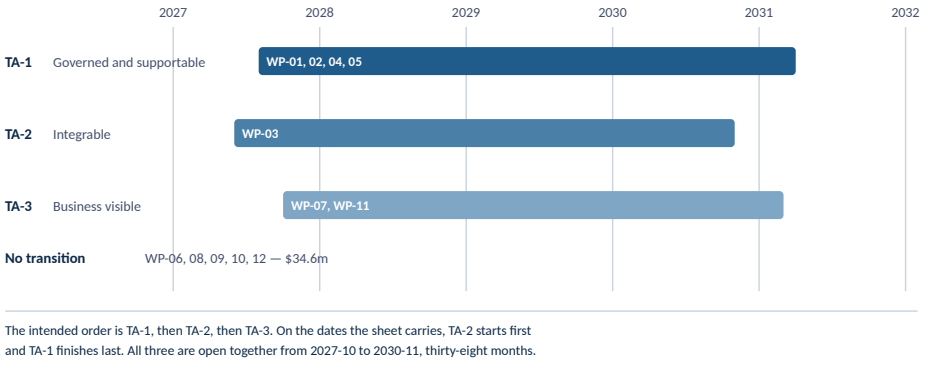


Figure 8.5 The three transition states on the dates `O30` actually carries. `TA-2` starts before `TA-1`, `TA-1` ends after `TA-3`, and five packages worth $34.6m sit in no state at all.

Form 8-C — exit criteria, one state per copy

`053 Transition Architecture`'s own columns. Write the criteria out: each one a statement somebody outside the programme can test, with a number in it. Three to seven per state. Signed by `GB-01 Architecture Board`.

Ref	Transition architecture	Description	Work packages	Exit criteria
TA-1	Governed and supportable	Example, delete me	WP-01, WP-02, WP-04, WP-05	0 components past vendor support; 14 of 14 entities with a named individual owner

BEFORE YOU TAKE THE ROADMAP ANYWHERE

Which transition state could the group stop at and still be better off than today? One sentence, in the language your sponsor uses.

Which five work packages are in no state, and for each, is it in the programme or out of it?

What happens to the roadmap if the state before the one you are in slips a quarter, into a peak?

Step 8.15 — Put three options to the board, not one plan

Produce. `054 Architecture Alternative` reduced to three live options with a reconciled cost.

You need. `054` as shipped; `030`'s costs; volume 1 chapters 12 and 14.

How.

1. Read the five shipped alternatives: `ALT-A Do nothing beyond compliance` $24m `Rejected`; `ALT-B Foundations first` $46m `Recommended`; `ALT-C Business value first` $52m `Rejected`; `ALT-D Big bang replacement` $88m `Rejected`; `ALT-E Compliance led` $31m `Partly adopted` [5].
2. Deal with `Partly adopted` first. It is not a decision — it is the minute of a meeting that ran out of time. Adopted, rejected, or a named component of the option you recommend: pick one.
3. Reconcile the money. `ALT-B` is $46m and `O30`'s twelve packages total $63.7m. Either the alternatives are drawn at a different scope or one number is wrong, and the board will find the $17.7m before you finish the slide.
4. Reduce to three: the one you recommend, the cheapest credible one, and the one the loudest executive wants. Three options is a decision; five is a survey.
5. Give each the same four things: cost, what the group gets, what it gives up, and the date the first thing a shopper notices lands. The last is what the conversation turns on.
6. Keep `ALT-D` in the pack and reject it with evidence rather than opinion.

Signed by. Chief Enterprise Architect writes it; `GB-02 Investment Committee` chooses; `GB-04 Executive Committee` is told.

Done when. Three options, each with a cost that ties to `O30`, and no `Recommendation` value that is a shade of grey.

Shows up in. `O54`'s `Recommendation` column and `O30`'s `Approval` column, which should read `Not started` on every package that exists only inside a rejected option.

Goes wrong when. One plan goes to the board. A board given one plan does not approve a plan; it approves or defers a request, and deferral is the default. Three options makes the meeting about which, rather than whether.

Look it up. Volume 1, chapter 12, *The South African anchor, and what it cost* — where lost sales exceeded implementation cost inside a single year, and where the remediation line is the most transferable sentence in the volume: separate the enterprise system from the warehouse system and implement them independently. So read `ALT-D` against `R13`: if one package carries both `ABB-01 Merchandising System target` and `ABB-05 WMS target`, the option is already the documented failure and you can say so with a source. Then chapter 14, *What a merchandising or point-of-sale replacement costs, and how long it takes*.

Where it lands.

`O54 Architecture Alternative`

`Ref | Alternative | Description | Cost | Recommendation`, five records. No basis-of-cost column, no benefit column, no date column. Keep the option paper outside the repository and let `O54` be its index.

Option	Cost	What the group gets	What it gives up
`ALT-B` Foundations first	$46m, reconciled or restated	A governable, supportable, integrable estate by `TA-2`	Nothing a shopper notices for two years
`ALT-E` Compliance led	$31m	Five control objectives off `In breach` and `No mechanism`	The integration layer, so every later change costs more
`ALT-C` Business value first	$52m	A visible channel change inside eighteen months	Builds on sixteen unmanaged interfaces and nine direct database reads

Part four — reading the result

Step 8.16 — Read `RPT07` and `RPT08`, then issue the Phase E pack

Produce. The three Phase E deliverables, and two reports that reconcile.

You need. Everything above; `O31 Deliverable`; `ZZ Checks` run a second time.

How.

1. Read `RPT07 Gap Analysis` three ways: rows with no `Work package`, rows past `Due`, and the severity spread across `Domain`. If after this chapter every Critical gap sits in one domain, either that domain is in trouble or your severity scale is.
2. Read `RPT08 Roadmap`, then its footer, which says seven work packages are past their end date — and check it, because it does not survive the check. The earliest package starts 2027-06 and the latest ends 2031-09, so nothing has started. `WP-08` ends 2030-10 and reads `Delivered`; `WP-05` reads `Approval Not started` and `Delivery In flight`. The footer and `ZZ Checks` both compute off `Delivery`, and step 9.13 fixes it properly.
3. Set `O31 Deliverable` for `DLV-13`, `DLV-14` and `DLV-15`. `DLV-13 Consolidated Gaps` reads `Approved` today against a one-row register; set it to `Draft` when you start and move it when the register is real.
4. Run `ZZ Checks` again and put before and after side by side. Phase E should change none of the estate counts. It should move `Gaps overdue` off a zero that only means the register was empty.
5. Take the pack to `GB-01 Architecture Board` for the architecture and `GB-02 Investment Committee` for the money. `GB-01` is monthly, `GB-02` quarterly, so the pack has a date and missing it costs a quarter.

Signed by. `GB-01` approves `DLV-13` and `DLV-14`; `GB-02` approves the envelope at `GT-0 G0 Charter`.

Done when. `RPT07` has no blank `Work package` cells, `RPT08`'s footer agrees with its own `End` column, and all three Phase E rows of `O31` carry a status you would defend in writing.

Shows up in. `RPT01 Dashboard`, which an executive reads and you do not control. `Gaps overdue` gets read first.

Goes wrong when. The pack goes forward with `RPT08`'s footer intact. Seven packages past their end date, in a programme whose earliest package starts in 2027, invites exactly the question you cannot answer.

Look it up. Volume 1, chapter 6, *The four numbers a retail executive watches*.

Where it lands.

> **`RPT07 Gap Analysis · RPT08 Roadmap · 031 Deliverable`**
>
> `RPT07` — seven columns computed over `029` and `R12`, one row until this chapter. `RPT08` — seven columns over `030` plus a footer that disagrees with its own dates. `031` — `DLV-13` to `DLV-15` in phase `E`.

FOR THE REVIEWER, BEFORE THIS PHASE IS SIGNED OFF

Take any three rows of `029 Gap`. For each, can you name the difference, the measurement or the volume-1 pattern it came from?

Take any two rows of `055 Building Block`. Does the baseline already meet the acceptance criterion? If it does, what is the block for?

Which work package closes no gap, and why is it still in the plan?

What does the `UNGOVERNED CHANGE` count on `60 Summary` mean in a repository with no change log, and does the pack say so in those words?

THE CHAPTER IN TEN LINES

- Phase E turns differences into a funded programme. The gap register sets the quality of everything after it.
- O29 Gap holds one row, R12 one instance, and DLV-13 Consolidated Gaps reads Approved. Twelve work packages worth $63.7m rest on a single Medium finding.
- That one gap is BBB-01's own name with "not in place" appended, so the register was written backwards from the solution.
- There are no export sheets and no change log sheet. You build both extracts by hand and the fifth paste target stays empty.
- With an empty change log, UNGOVERNED CHANGE means unexplained rather than unauthorised. Say so in the pack.
- A difference is not a finding until a named person says so on 50 Review, with a rationale longer than four words.
- Every one of O13's hundred rows has a maturity delta and Disposition is the delta restated. C01 Merchandising & Assortment is Critical at maturity 1 with a target of 2.
- The five capabilities a retailer is usually missing are all where margin is made and lost, which is why nobody funds them.
- There is no O51, so no Solution Building Block sheet. Build, buy or configure needs a column you add or a paragraph you write.
- RPT08's footer says seven packages are past their end date in a programme whose earliest package starts in 2027. Fix Delivery before anybody quotes it.

SOURCES FOR THIS CHAPTER

- **1.** Retail_TOGAF_Metamodel.xlsx, sheets O29 Gap, O30 Work Package, O53 Transition Architecture, O55 Building Block and O13 Business Capability
- **2.** Retail_TOGAF_Metamodel.xlsx, sheets R12 Gap closed by Work Package, R13 Work Package delivers Build and R15 Transition contains Work Pa
- **3.** Retail_Architecture_Comparison_Tool_TEMPLATE.xlsx, sheets 01 Config, 10 Input A Baseline, 11 Input B Target, 10R Records A, 11R Records B, 12 Input Change Log, 30 Diff Records, 32 Diff Attributes, 50 Review, 60 Summary and 70 Gap Candidates
- **4.** Retail_TOGAF_Metamodel.xlsx, sheets RPT07 Gap Analysis, RPT08 Roadmap, RPT03 Capability Heatmap, RPT09 Regulatory Coverage and RPT13 Orphan Analysis
- **5.** Retail_TOGAF_Metamodel.xlsx, sheet O54 Architecture Alternative
- **6.** Retail_TOGAF_Metamodel.xlsx, sheets O05 Principle, O06 Requirement, O11 Role, O57 Governance Body and O58 Governance Gate
- **7.** Retail_TOGAF_Metamodel.xlsx, sheets O31 Deliverable, O35 Business Building Block, O44 Data Building Block, O45 Application Building Block, O46 Technology Building Block and EXT Industry Extension
- **8.** Retail_TOGAF_Metamodel.xlsx, sheets ZZ Checks, 00 Index and RPT01 Dashboard
- **9.** Retail_TOGAF_Metamodel.xlsx, sheets O20 Data Entity, O23 Application Component, O37 Data Attribute, O43 Master Data Rule, O49 Resilience Tier and R06 Application interfaces with
- **10.** Consumer Protection Act 68 of 2008, section 23 — disclosure of price of goods or services https://www.acts.co.za/consumer-protection/23_disclosure_of_price_of_goods_or_services
- **11.** The Open Group, TOGAF Standard, 10th Edition — ADM Phase E, Opportunities and Solutions https://pubs.opengroup.org/architecture/togaf10-doc/adm/

CHAPTER 9

Migration Planning

A roadmap can be wrong quietly. A plan cannot: the moment you write a date against a project with a named lead, somebody can check it. In a retailer the people who will check it own most of the year already.

Phase F is where the roadmap acquires names, dates and money, and becomes checkable [11]. This chapter is about making it checkable by you before it is checked by the people who own the dates you are about to collide with. In a retailer, those people own most of the year.

Halverson Stores Group arrives at Phase F better equipped than it arrived at Phase E. `056 Project` carries twelve projects with costs, dates and leads. `R14 Project delivers Work Packa` carries twelve instances. `052 Implementation Factor` carries eight factors with a `Deduction` each. `062 Delivery Risk` carries eight risks with owners. `066 Architecture Increment` carries six increments with the month each lands [1][2][3].

Read them together and three things come apart. One project per package, exactly, twelve for twelve, same names and same dates — nothing has been shaped, only transcribed. The increments land on months the projects delivering them do not support. And no sheet anywhere in the repository records a dependency between two projects or two packages.

Halverson Stores Group is invented and its data is real within this workbook. Milkwood Retail Group is volume 1's invented retailer and appears here only as a pointer back.

> **BEFORE YOU START THIS PHASE**
>
> - **Volume 1, chapter 7, *The trading calendar*.** The frozen block, the 4-5-4 convention, the 53rd week and the freeze that compresses the delivery year.
> - **Volume 1, chapter 7, *The batch clock*.** The twelve overnight jobs and the deadline that is a shop door opening.
> - **Volume 1, chapter 10, *Time in a retailer*.** Why a retail migration is a history reconstruction rather than a data movement.
> - **Volume 1, chapter 9, *Price is an entity, not an attribute*.** Nine kinds of price, all dated, all of which have to survive the cut-over.

Part one — from packages to projects

Step 9.1 — Shape the work packages into projects

Produce. `056 Project` with a lead who can be in the room, and `R14` rebuilt to match.

You need. `O30 Work Package` re-cut in step 8.13; `O11 Role` for the leads; `O52`; the delivery function's view of concurrent capacity.

How.

1. Read the shipped twelve. Each project carries one package, same name, same cost, same dates: `PRJ-03 Integration platform` delivers `WP-03 Integration platform` at $1.5m [1]. That is a transcription, not a plan.
2. Combine where combining removes a dependency rather than managing it. If `WP-11 Customer experience` cannot start until `WP-03 Integration platform` delivers, running them as one project with one lead removes a hand-off.
3. Split where a package has two acceptance cadences — a legally dated half and a discretionary half should not share a lead.
4. Set `Lead` from `O11 Role`, then read the shipped leads, because they will not survive a glance. `PRJ-05 Access and security` is led by the `Internal Auditor`, `PRJ-07 Core platform renewal` and `PRJ-09 Resilience uplift` by the `Chief Financial Officer`, and `PRJ-02 Data ownership and mastering` by the `Chief Risk Officer` while `GB-03 Data Governance Council` sits under the Chief Data Officer [1][5]. An internal auditor cannot lead a project they will later assure.
5. Rebuild `R14 Project delivers Work Packa` — `Project | Project name | Work package | Package name | Cost` — carrying `Cost` on the relationship row where a project splits a package's budget.
6. Write the argument for each shape into the Phase F deliverable, because `O56` has no column for it. One paragraph per project, naming what combining or splitting removed.

Signed by. Head of Delivery, `ROL-13`, for the shape; `GB-02 Investment Committee` for the money.

Done when. Every project has a lead who can be in the room, no lead sits in an assurance role, and every package appears in `R14`.

Shows up in. `RPT08 Roadmap` indirectly, through `O30`'s dates, which must now agree with `O56`'s.

Goes wrong when. Twelve projects stay twelve projects because the sheet already looked complete. One-to-one is the default, not an answer, and it hides every dependency inside a package boundary.

Look it up. Volume 1, chapter 5, *The trading calendar governs every diary in the building* — which is also the reason a trading director will not give you a project lead in October.

Where it lands.

056 Project · R14 Project delivers Work Packa

`056` — `Ref | Project | Work package | Cost | Start | End | Lead`. One `Work package` column, so a project holding several packages records the primary one here and the rest in `R14`. No column for why the project is cut this way.

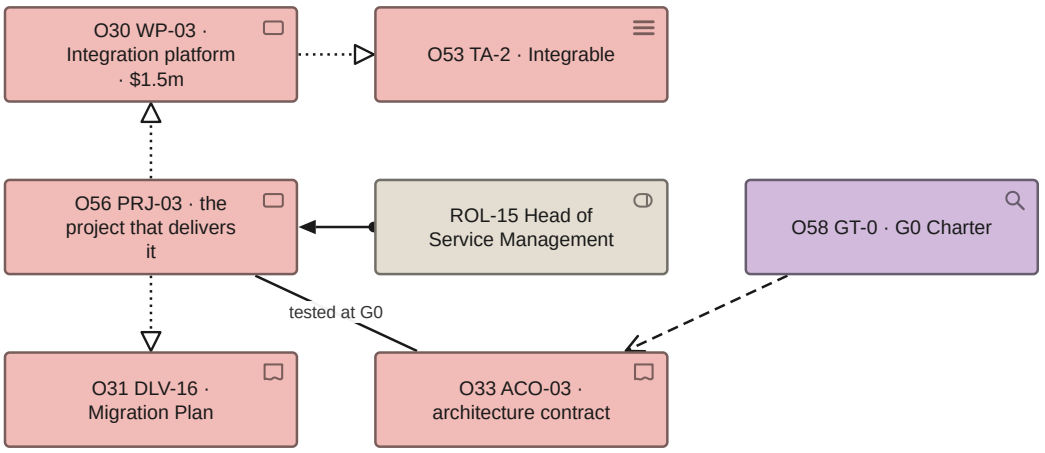


Figure 9.1 One chain through Phase F. A project and a work package are both Work Package elements, which is correct and is why the two sheets disagree so easily. The gate tests the contract, not the plan.

WHY THE PROGRAMME IS CUT THIS WAY

Take your three largest projects. If you cannot answer the first question, the project is the wrong shape.

Why is this one project rather than two? Write the sentence you would say to a new executive.

What does combining or splitting it remove from the dependency graph?

Step 9.2 — Record the implementation factors, each with its deduction

Produce. `052 Implementation Factor` complete — everything that will shape delivery, and what follows from each.

You need. Phase B to E findings; `007 Constraint`; `008 Assumption`; `062 Delivery Risk`.

How.

1. Read the eight shipped factors. The categories are `Dependency`, `Constraint`, `Resource`, `Assumption` and `Risk`, and `Deduction` is the point of the sheet: `IF-02 Data owners not designated` deducts "Blocks mastering work" [2].
2. Rewrite the vague ones, starting with the one that matters most here. `IF-03 Peak trading freeze` deducts "No change in the peak window" — which window, from when to when, and whose rule? A deduction with no date is an observation. Step 9.4 gives you the dates.
3. Do the same to `IF-08 Site access windows`, which in a retailer means store refits, fixture changes and anything needing a trading floor after hours.
4. Add the factors the sheet is missing, and there are at least three: no project-to-project dependency can be recorded anywhere in this repository; `O04 Measure` has no target column, so no benefit can be stated as a number; and `O30`'s `Delivery` column disagrees with its own `End` dates on seven rows.
5. Cross-check `O62 Delivery Risk`. `RSK-01 Integration product not selected` is `Certain` and `High`. A certainty is not a risk, it is an issue, and it belongs here as a factor with a deduction.

Signed by. Head of Delivery; Chief Enterprise Architect co-signs anything that changes the architecture rather than the plan.

Done when. Every row has a `Deduction` that tells somebody to do or not do something, and every `Certain` row of `O62` has a matching factor here.

Shows up in. The sequencing argument in the Implementation and Migration Plan. There is no report over `O52`, so what is not in the plan document is invisible.

Goes wrong when. The sheet collects observations. Eight true statements with no deduction is a risk log with the consequences removed.

Where it lands.

> **`O52 Implementation Factor`**
>
> `Ref | Implementation factor | Category | Deduction`. Four columns: no source, no owner. Name the source inside the factor text — "`ZZ Checks`: three platform components past vendor support" — or the next reader cannot check it.

Step 9.3 — Sequence the programme, and record the dependency the repository cannot hold

Produce. A sequence with a critical path, and a written record of every dependency that shaped it.

You need. `O56` with dates; `O52`; `O53 Transition Architecture`; `R15 Transition contains Work Pa`.

How.

1. Establish the fact first: **there is no dependency sheet.** Twenty-five relationship sheets and not one relates a project to a project or a package to a package. `R14` links project to package, `R15` transition to package, `R23` increment to project, and nothing links a prerequisite to a dependant [2].
2. So record dependencies in two places and keep them in step: as `O52` factors with `Category` `Dependency` and a deduction naming both ends, and as a dependency table in the plan. Do not quietly invent a sheet in the shipped file — if you add one, it goes on the Preliminary chapter's list of additions.
3. Derive the sequence from the transition states rather than the project list. `TA-1 Governed and supportable` has to exit before `TA-2 Integrable` can be claimed, and `TA-2` before `TA-3 Business visible`.
4. Read the shipped dates against that order. Eight of the twelve packages start in 2028, which means the programme begins almost everything in one year and has not been sequenced at all.
5. Compute the critical path by hand and state its length in months. The earliest start is `WP-03` at 2027-06 and the latest end `WP-10 Compliance mechanisms` at 2031-09: **fifty-two months**, which is two festive peaks longer than any executive currently thinks.
6. Name the packages that could start now with no prerequisite, and start those.

Signed by. Head of Delivery; `GB-01 Architecture Board` for the order of the transition states.

Done when. Every project has a predecessor named or is explicitly declared independent, and the critical path has a length in months somebody has written down.

Shows up in. `RPT08 Roadmap`, by `Start`. Sort it and the clustering is visible in one glance.

Goes wrong when. The sequence is drawn from what the business wants first. `TA-3 Business visible` holds `WP-07` and `WP-11`, and on the shipped dates `WP-07` starts 2027-10 — before the foundations it rests on exist.

Where it lands.

> **`O52 Implementation Factor · O53 Transition Architecture`**
>
> One `O52` row per dependency, `Category` `Dependency`, `Deduction` naming both ends and the consequence: "`PRJ-11` cannot start before `PRJ-03` delivers managed integration; eleven months of float consumed."

Part two — the retailer's year, which is the real constraint

Everything above is ordinary programme management. This part is not, and it is where a plan written by somebody from banking or insurance fails in its first month.

The trading calendar governs everything. It is not a risk, not a preference and not negotiable. It is a set of dates the trading business owns, set a year ahead, and a plan that collides with them was not

checked. Volume 1, chapter 7, *The trading calendar*, carries the calendar itself; this step turns it into a sequencing constraint you can apply.

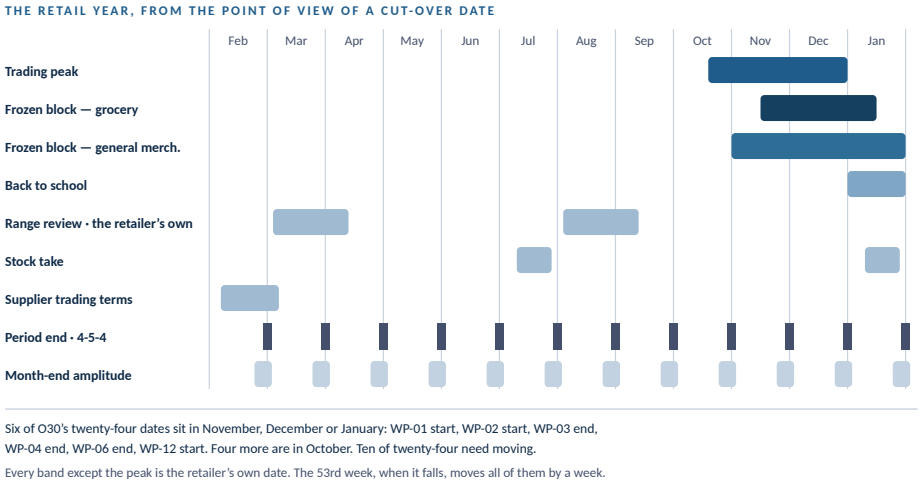


Figure 9.2 A retailer's year from the point of view of somebody who needs a cut-over date. The frozen block is the one nobody negotiates; every other band is a date the trading business owns and sets a year ahead. Drawn from volume 1 chapter 7, where the freeze dates are practitioner consensus rather than published.

Step 9.4 — Fit the plan to the trading calendar, not the programme's

Produce. A register of closed windows, each recorded as an `052` factor with a date rule, and `056` dates that fall outside all of them.

You need. Volume 1 chapter 7; the trading director's calendar; the merchandise planning team; finance; store operations.

How.

1. **The frozen block, and it is the whole argument.** No retailer changes anything between late October and mid-January. Black Friday, festive trading and the January sale are a single frozen block, not three events with gaps, and the freeze is wider than technology: software releases, infrastructure change, structural master-data change, hierarchy reclassification, range change, planogram resets and store refits all stop [9]. **Deduction: no cut-over, no release and no master-data restructure with an effective date between late October and mid-January.** Then check the plan against it: six of `O30`'s twenty-four dates sit in November, December or January — `WP-01` and `WP-02` and `WP-12` starts, `WP-03` and `WP-04` and `WP-06` ends — and four more sit in October. Ten of twenty-four dates need moving before anybody signs.
2. **The range review.** You cannot cut over a merchandising system during one. The review authorises next season's assortment, it runs to a date the buying organisation set a year ahead, and it is the one activity that needs the system you are replacing to be stable and complete. **Deduction: merchandising, assortment and space cut-overs land between reviews, and the dates come from the buyers, not from you.**

3. **The price file.** You cannot touch it in the run-up to a promotion. The price-file build and distribution sit on the critical path of the overnight cycle and must complete before stores open, and where two prices are displayed at once the lower binds the retailer [10]. **Deduction: no change to price derivation, the price file or the shelf-label path inside a promotion lead time, and never on a Thursday night before a weekend offer.**
4. **Stock take and period end.** The stock-take weekend freezes movement and runs a different, heavier batch; period end closes a 4-5-4 month, reconciles the stock ledger and accrues supplier income. **Deduction: nothing touching inventory or the ledger cuts over inside either, and a design that satisfies the nightly cycle but breaks the stock-take weekend fails once a year, in front of the auditors.**
5. **The 53rd week, and the other two.** Fifty-two weeks is 364 days, so a 53rd week is added roughly every five or six years on no fixed schedule, for sales reporting [9]. **Deduction: every date rule in this plan is expressed in retail weeks and periods as well as calendar dates, and a 53-week year moves the whole second half by a week.** Add back-to-school in January, which is a second genuine peak and collides with the post-festive trough; and the supplier trading-term renewal cycle, which fixes what the group will be paid for the year and must not meet a half-migrated supplier master.
6. Record each one as an `O52` row and then re-date `O56` and `O30` to match. Count the dates you moved and put that number in the plan, because it is the clearest evidence the plan was checked by somebody who knows the business.

Signed by. The trading director or equivalent owns the calendar; Head of Delivery accepts the resulting dates; `GB-01 Architecture Board` notes the ones that could not move.

Done when. Every closed window is an `O52` row with a `Deduction` naming months or weeks, and no project start or cut-over in `O56` falls inside one.

Shows up in. `O56 Project` and `O30 Work Package` dates, and in the count of dates you had to move.

Goes wrong when. The plan treats the freeze as a risk to be managed. It is not a risk. And the second-order effect is the one that breaks programmes: **the freeze compresses the delivery year to about nine months**, so a plan that assumed twelve is wrong by a quarter in every year it runs — fifty-two months of critical path becomes something closer to seventy.

Look it up. Volume 1, chapter 7, *The trading calendar*, and chapter 13, *The retail calendar as an architecture artefact*. For the shelf-label exposure, chapter 15, *Section 23, and why a stale shelf label is a legal exposure*.

Where it lands.

`052 Implementation Factor`

One row per window, `Category` `Constraint`, and the rule inside `Deduction`: "No release or cut-over with an effective date between 20 October and 15 January." `IF-03 Peak trading freeze` and `IF-08 Site access windows` are the two shipped rows this step replaces with something a delivery manager can act on.

Form 9-A — the closed-window register

`052 Implementation Factor`'s own four columns. One row per window. A `Deduction` with no month, week or date goes back. Eight to twelve rows is a first pass. Signed by the trading calendar's owner and the Head of Delivery.

Ref	Implementation factor	Category	Deduction
IF-03	Example, delete me — peak trading freeze	Constraint	No release or cut-over effective 20 October to 15 January; applies to master data and planograms, not only software

THE WINDOWS, BEFORE YOU COMMIT A DATE

Which closed window did you not know about until somebody mentioned it in a meeting? Who owns it?

How many start or cut-over dates in `056` fall inside a window you have now recorded? How many moved?

Which window will the programme ask you to break first, and what is your answer?

Step 9.5 — Size the delivery year, and the night

Produce. A plan whose duration assumes nine working months a year, and cut-over windows that fit inside one night.

You need. Step 9.4's register; volume 1 chapter 7; the operations team's actual batch finish times for the last thirty nights.

How.

1. Re-length every project on a nine-month delivery year. Fifty-two months of critical path at twelve months a year is fifty-two months; at nine it is nearer seventy, and the difference is the single largest error in most retail programme plans.
2. Put the big releases where they are survivable and say what that costs. January and February carry the largest and riskiest releases of the year, immediately after the period in which the estate was least exercised and most patched around.
3. Get the overnight cycle's finish times for the last thirty nights, not the design. The distribution tells you how much slack a cut-over can borrow, and in most estates nobody has measured it.
4. Establish which of the twelve overnight jobs are on the critical path. The forecast run usually is; the data-platform load usually is not. A cut-over that extends an on-path job past store opening is a shelf-edge failure, not a technology incident.
5. Write the recovery behaviour down per job for the cut-over night: re-run, skip, or run degraded, and who decides. In most retailers that answer lives in an operator's head.

Signed by. Head of Service Management, `ROL-15`, for the batch; Head of Delivery for the duration.

Done when. Every project's duration is stated in working months, and every planned cut-over has a window that fits inside the measured slack with the recovery behaviour written down.

Shows up in. `O56` and `O30` dates again, and in `O62 Delivery Risk`, where "cut-over overruns into trading hours" belongs with a consequence attached.

Goes wrong when. The plan assumes twelve months of delivery a year and a cut-over window nobody timed. Both errors surface in the same week, and it is usually the week before a peak.

Look it up. Volume 1, chapter 7, *The batch clock*, and chapter 12, *The batch clock and the trading day*.

Where it lands.

> **`O56 Project · O62 Delivery Risk`**
>
> Durations in `Start` and `End` on `O56`; the cut-over behaviour has no home in the repository, so it lives in the plan with a reference from the `O52` factor that constrains it.

Part three — money, benefit, increments and risk

Step 9.6 — Put a basis under every cost, and fix the currency

Produce. Costs in `O30`, `O56` and `O54` that reconcile to each other, in one currency.

You need. `O30`; `O56`; `R14`; `O54 Architecture Alternative`; the finance business partner.

How.

1. Convert to rands. `Cost` reads `$1.5m`, `$10.0m`, `$46m` across the three sheets. State the rate and the date once in the plan, and never let two currencies appear in one pack.
2. Reconcile the three totals. `O30`'s twelve packages sum to $63.7m; `O54`'s recommended `ALT-B Foundations first` reads $46m. A $17.7m difference nobody has explained is the first question an investment committee asks.
3. Reconcile project to package. `R14` carries a `Cost` column and so do both object sheets, so the same number is recorded three times and can disagree in three places. Decide which is authoritative and derive the other two.
4. Write the basis outside the repository, because there is no basis column: licence, implementation, integration, data migration, parallel running, store rollout, decommissioning, internal effort, contingency — and say which of those nine each estimate contains. In a retailer the store rollout line is the one most often missing, and it scales with the estate rather than with the software.
5. Flag the estimates that are a single round number. `$10.0m` and `$1.5m` sitting in the same sheet are not the same kind of number.

Signed by. Chief Financial Officer, `ROL-02`; `GB-02 Investment Committee` accepts the total.

Done when. One currency, three totals that reconcile, and every cost carrying a named basis in the plan.

Shows up in. `RPT08 Roadmap`, `Cost` — the column an executive adds up in their head while you are talking.

Goes wrong when. The estimate is the licence plus a multiplier, with no line for the store estate.
Look it up. Volume 1, chapter 14, *What a merchandising or point-of-sale replacement costs, and how long it takes*.

Where it lands.

> **`O30 Work Package · O56 Project · R14 Project delivers Work Packa`**
>
> `Cost` on `O30` is the package envelope, on `O56` the project budget, on `R14` the split where one project funds part of a package.

Step 9.7 — Make every benefit trace to a measure, or delete it

Produce. One benefit per project, each tied to a row of `O04 Measure`, with arithmetic.

You need. `O02 Goal`; `O03 Objective`; `O04 Measure`; `R10 Objective measured by Measu`; `R11 Goal realised by Objective`.

How.

1. Start from the objective, not the project. `O03` carries fourteen objectives against six goals, each with a `Horizon` [6].
2. Find the measure. `R10` carries fourteen instances linking objective to measure, and `O04` carries sixteen measures with a `Baseline` and a `Cadence`.
3. Notice the hole, because it changes what you may promise: **`O04` has a `Baseline` column and no target column.** `MSR-08 Managed interfaces` has a baseline of 28 and nowhere to record what it should reach, and `O03` has a `Horizon` and no quantity. So the quantified benefit cannot live in this repository at all. It lives in the plan, and `O04` holds the baseline it will be measured from.
4. Write the arithmetic. "`MSR-07 Components within vendor support` from 50% to 100% by 2030-01, removing three unsupported components" is a benefit. "Improved currency" is not. **A benefit that does not trace to an `O04 Measure` is fiction**, and at `GT-3 G3 Benefit` it is the reason a project is recorded as not having delivered.
5. Be careful with the retail numbers that have no external baseline. Availability, markdown and shrink are the three benefits a trading business will actually believe — and there is no published South African shrink figure, so a shrink benefit is measured against the retailer's own baseline on its own definition, never against a benchmark borrowed from another market.
6. Check the cadence. `MSR-02 Manual regulatory adjustments` is `Quarterly`, so a benefit claimed at month three has no reading behind it.

Signed by. Each measure's owner for their own row; Chief Financial Officer for the total; `GB-02 Investment Committee` confirms at `GT-3`.

Done when. Every project has one benefit, one measure, one piece of arithmetic and one realisation date.

Shows up in. `RPT01 Dashboard` eventually, and `O60 Governance Decision` at `G3`.

Goes wrong when. `RSK-07 Benefit not measurable at G3` is already on the risk sheet, `Certain`, owned by the Chief Financial Officer [3]. Somebody has told you in advance that this will happen. This step is how you make them wrong.

Look it up. Volume 1, chapter 6, *The four numbers a retail executive watches*, and *Shrink*.

Where it lands.

> **`O04 Measure · O03 Objective`**
>
> `O04` — `Ref | Measure | Unit | Domain | Baseline | Cadence`, sixteen measures, no target column. `O03` — `Ref | Objective | Goal | Horizon | Status`, fourteen objectives, no quantity. The number you are promising lives in the plan, and the plan has to say so.

Step 9.8 — Write the architecture increments, and fix the ones that cannot be true

Produce. `O66 Architecture Increment` as the readable version of the roadmap, and `R23` rebuilt.

You need. `O56` with dates; `O53`; `O66`; `R23 Increment delivered by Proj`.

How.

1. Read the six shipped increments: `INC-1 Governed and supportable` at month 10, `INC-2 Integrable` at 16, `INC-3 Compliant by mechanism` at 22, `INC-4 Business visible` at 30, `INC-5 Analytics and evidence` at 42, `INC-6 Core renewal` at 56 [1].
2. Read `R23` next, and read it properly. Six instances allocated in order, so `INC-6 Core renewal` is recorded as delivered by `PRJ-11 Customer experience` while `PRJ-07 Core platform renewal` is pinned to `INC-4 Business visible` [2]. Two of the six are plainly wrong.
3. Convert each `Lands` month into a date from the programme start — month 1 is 2027-06 — and compare it with the end date of the project `R23` names. `INC-1` lands at month 10 and `PRJ-01` ends at month 47. `INC-5` lands at 42 and `PRJ-09` ends at 45. `INC-6` lands at 56 and `PRJ-11` ended at 22, thirty-four months earlier.
4. Rebuild `R23` from the project side: for each increment name every contributing project, and set `Lands` to the month the last contributor finishes — then move it again for the freeze, because an increment that lands in December lands in January.
5. Write `What is added` as mechanisms that will exist, in the business's words. `INC-1` reads "Repository, ownership, currency, access", which honestly tells an executive that nothing a shopper notices happens for ten months. Keep that honesty.

6. Reconcile against `O53`. `TA-1`, `TA-2` and `TA-3` carry the same three names as `INC-1`, `INC-2` and `INC-4`. The same state is recorded on two sheets; decide which is authoritative and say so.

Signed by. Chief Enterprise Architect; `GB-04 Executive Committee` is the audience that most needs this sheet.

Done when. No increment lands before the last project delivering it ends, no increment lands inside the frozen block, and `R23` names every contributing project.

Shows up in. Nothing computes over `O66`. It is the sheet you put on a slide, which is exactly why its dates have to be right.

Goes wrong when. Increments are drawn one per project. They are not the same thing: an increment is what becomes true, and several projects usually have to land before anything does.

Look it up. Volume 1, chapter 16, *The roadmap, and what is true on 31 December* — a date chosen, in a retailer, with some irony.

Where it lands.

`O66 Architecture Increment · R23 Increment delivered by Proj`

`O66` — `Ref | Increment | Lands | What is added`. Four columns: no note, no conditionality, no benefit. Where an increment depends on a vendor selection, say so inside `What is added`. `R23` — `Increment | Increment name | Lands | Project | Project name`, carrying `Lands` on the relationship too.

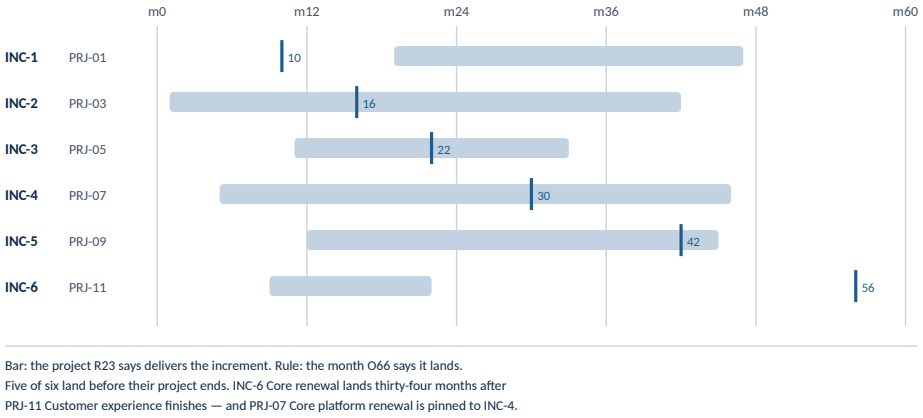


Figure 9.3 The six increments against the projects `R23` says deliver them. Five of six land before their project ends and the sixth lands thirty-four months after — which is what a relationship allocated in order looks like.

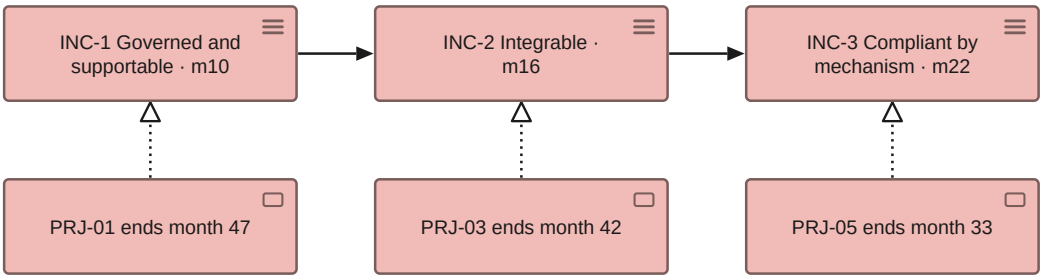


Figure 9.4 `R23` as the repository ships it: three of the six increments as plateaus, each triggering the next, each realised by a project that finishes long after the increment is recorded as landing.

Step 9.9 — Record delivery risk with a consequence, not an adjective

Produce. `O62 Delivery Risk` where every row says what happens if it lands.

You need. `O62`; `O52`; `O07 Constraint`; `O08 Assumption`.

How.

1. Read the eight shipped risks and their `Likelihood`. Three read `Certain`: `RSK-01 Integration product not selected`, `RSK-07 Benefit not measurable at G3` and `RSK-08 Unmanaged interfaces proliferate` [3].
2. Move those three off this sheet. A certainty is an issue. It belongs in `O52` with a deduction, or in the plan as a decision already needed — and `O60 Governance Decision` already carries `DEC-02 Select the integration product` as `Outstanding`.
3. For every remaining row, write the consequence into the `Risk` text, because the sheet has no consequence column: "`RSK-04 Legacy platform failure before renewal` — `Possible`, `Critical`; a merchandising outage inside the renewal window stops replenishment, and a grocery supply chain that stops loses perishable stock and customers in the same week."
4. Check the owner can act. `RSK-03 Delivery capacity below plan` sits with the Head of Delivery, which is right.
5. Set a trigger date for each — the date by which it must be closed or it becomes an issue. There is no column, so it goes in the plan's risk table, reviewed monthly.

Signed by. Each row's `Owner`; Head of Delivery for the sheet.

Done when. No row reads `Certain`, and every row names what breaks and when you will know.

Shows up in. `ZZ Checks` has no risk row, so nothing computes here. The monthly delivery report is where this sheet earns its place.

Goes wrong when. Likelihood and impact are adjectives and the row stops there. `Possible`, `Critical` tells a reader nothing about what to do on Monday.

Look it up. Volume 1, chapter 12, *The South African anchor, and what it cost*, and the design rule that comes out of it: an enterprise system and a warehouse system should not go live together.

Where it lands.

`062 Delivery Risk`

`Ref | Risk | Likelihood | Impact | Owner`. `Likelihood` runs `Unlikely` to `Certain`, `Impact` `Medium` to `Critical`. No mitigation, no trigger date, no consequence column — three things that live in the plan.

Step 9.10 — Tie constraints and assumptions to what they hold up

Produce. `007 Constraint` and `008 Assumption` each pointing at the project it would invalidate.

You need. `007`; `008`; `056`; `052`.

How.

1. Read the eight constraints. Three are already sharp and all three are retail: `CON-06 Change freeze windows` — peak periods are closed to change; `CON-08 Union and works council consultation` — operational change requires consultation, which in a store estate means anything that changes how a shift works; `CON-05 Data residency` [4].
2. Read the six assumptions and their `Status`. Two read `At risk`: `ASM-03 Vendor selections complete before the dependent phase` and `ASM-04 Delivery capacity can be staffed to plan`. An assumption at risk holding up a project on the critical path is the most important line in the plan.
3. For each constraint and assumption, name the projects it binds. Neither sheet has a column for it, so the link is an `052` factor and a column in the plan's assumption register.
4. Give every assumption a test and a date: who confirms it, by when, and what the plan does if the answer is no.
5. Deal with `CON-02 Delivery capacity unrecorded` directly. A plan that assumes capacity nobody has measured has an unmeasured denominator, and `IF-06 Concurrent capacity unknown` already says so.

Signed by. Head of Delivery; the Chief Compliance Officer for `CON-05`; the Chief Operating Officer for `CON-08`.

Done when. Every `At risk` assumption has a test, an owner and a date, and every constraint names at least one project.

Shows up in. Nowhere computed. The plan document or nowhere.

Goes wrong when. `CON-06 Change freeze windows` stays at that level of generality after step 9.4 gave you the dates to put in it.

Where it lands.

`007 Constraint · 008 Assumption`

`007` — `Ref | Constraint | Description | Type`, with `Type` running `Financial`, `Resource`, `External`, `Regulatory`, `Operational`, `Organisational`. `008` — `Ref | Assumption | Status`, three columns. No owner, no test, no date: all three belong in the plan.

Part four — data migration, which is its own plan

Step 9.11 — Plan each data migration as its own piece of work

Produce. `039 Data Migration` rewritten, and a migration plan per entity that `039` indexes.

You need. `039`; `020 Data Entity`; `043 Master Data Rule`; `R05 Application manages Data En`; the data owners.

How.

1. Read the six shipped rows and do not use them. `MIG-01` moves `SKU / Article` from `POS` to `Order Management System`; `MIG-02` moves `Product Hierarchy` from `E-commerce Platform` to `WMS` [7]. Neither is credible, and both contradict `020`'s own `Master system` column.
2. Rebuild `From system` from `020`'s `Master system` and from `R05`, not from the migration sheet — and resolve the contradiction first. `020` masters `E05 Price` in `Workforce Management`; `043 MDR-05` names `POS` as its `Golden source`. Two sheets, two answers, and a migration cannot start until one of them is wrong in writing.
3. Put the two retail entities that matter at the front, because they are the critical path. **The article master** carries the identifiers, the six hierarchies that all disagree, and the trading attributes every other system reads; nothing else can migrate until it has. **The price history** has to survive as history: price records are append-only versions rather than updates, because the retailer must be able to prove what price was displayed, where and when [10]. A migration that lands current prices and drops the versions has destroyed the evidence.
4. Work out what `039` cannot hold, because it is most of what matters: record counts, history depth, the as-at date, the reconciliation rule, the cut-over window, the rollback point and the dual-running period. Five columns and none of them these. So each migration is a plan in its own right and `039` is the index. Say that in the deliverable.
5. Write the reconciliation rule before the mapping: count, sum, and a stock-ledger value that ties at a stated as-at date. A migration that reconciles on row count has not reconciled.
6. Plan the dual run across at least one full period end **and** one full promotion, because a promotion is the only test that exercises price derivation, the shelf-label path and the till together.

Signed by. Chief Data Officer, `ROL-05`, and the entity's `Owner` from `O20`; Chief Financial Officer for anything that reaches the stock ledger.

Done when. Every row of `O39` names a credible source and target, and each has a plan behind it with a reconciliation rule, a cut-over window and a rollback point.

Shows up in. `RPT05 Data Ownership`, which carries `Master system` and `Lineage` and will contradict `O39` until this step is done.

Goes wrong when. The migration is planned as a data movement rather than a history reconstruction. In a retailer that makes it the critical path every time: the article exists, the price is right, and the version history that proves what the shelf said last Thursday is gone.

Look it up. Volume 1, chapter 9, *Price is an entity, not an attribute*, and chapter 10, *Time in a retailer* and *The restatement problem*. For the identifiers, chapter 10, *Two schemes for one product, and nobody owns the bridge*.

Where it lands.

`O39` Data Migration

`Ref | Entity | From system | To system | Status`. Five columns and nothing for volume, depth, reconciliation, window or rollback. Treat it as the index to six plans, numbered so the index points somewhere.

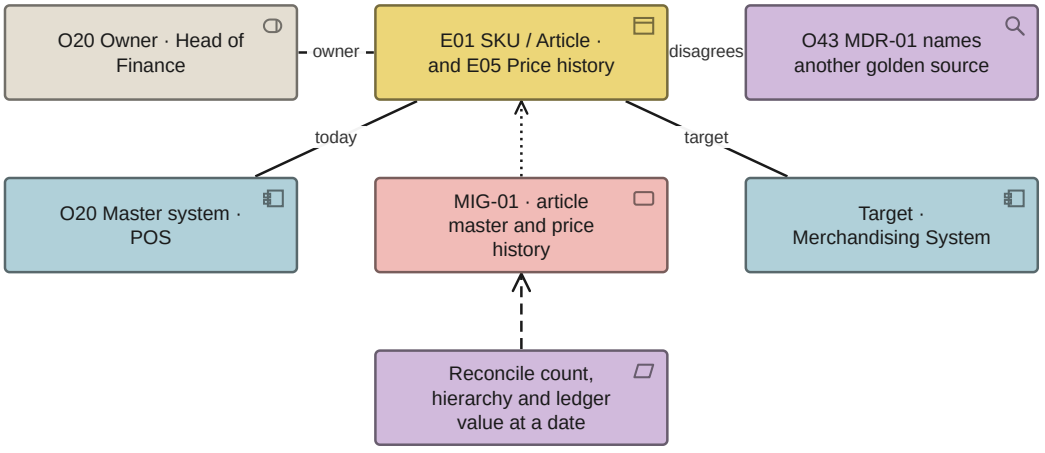


Figure 9.5 One migration, end to end. The article master and the price history are the critical path, the owner is a named person and the assessment on the right is the contradiction that has to be settled in writing before a date is set.

Form 9-B — the migration index, one row per entity

`039 Data Migration`'s own columns in sheet order, plus the plan reference you assign. Depth, reconciliation rule, cut-over window and rollback point live in the plan; the last column is what points at them. Six to ten rows. Signed by the Chief Data Officer and the entity owner from `020`.

Ref	Entity	From system	To system	Status	Plan ref
MIG-01	SKU / Article	Example, delete me — per O20 Master system		Planned	DMP-01

BEFORE A MIGRATION DATE IS COMMITTED

For the article master: how many records, how deep is the history, and what is the as-at date?

What is the reconciliation rule, and what is the rollback point once the first store has traded on the new system?

Step 9.12 — Map source to target, field by field, with the rule written out

Produce. `040 Source To Target Mapping` with real field names and rules somebody can execute.

You need. `039` rebuilt; `037 Data Attribute`; `043 Master Data Rule`; `020`'s `Classification`; `048 Environment`.

How.

1. Read the twelve shipped rows. `Source field` and `Target field` read "POS field" and "WMS field" — placeholders, not fields [7]. Replace them from `O37 Data Attribute`, which carries sixty attributes with a `Type`, a `Key` flag and a `Classification`.
2. Extend the rule vocabulary. `Rule` runs `Direct`, `Lookup` and `Transform`, and three values are not enough: you also need `Derive`, `Default`, `Carry unchanged` and `Reject`. A row with no rule for a value that fails validation has no answer on cut-over night.
3. Write the expression. A `Transform` with nothing behind it is not a mapping. Either the expression goes in `Rule` or it lives in the mapping document with a reference here.
4. Check classification on every row, and fix `O20` first, because it is wrong in a way that matters. `E12 Customer` is classified `Public`, as are `E04 Assortment` and `E07 Planogram`, while `E01 SKU / Article` is `Restricted`. A customer entity classified `Public` is a lawful-basis finding before it is a data finding, and `PRN-14 Classification Travels with Data` is the principle it offends.
5. Then read `O48 Environment` before you move a single record into a test system. `ENV-02 Pre-production` and `ENV-04 Development` both read `Personal data present` `Yes` while `ENV-01 Production` reads `No`. Whatever the truth is, that is backwards, and it is the column that decides whether a test migration is allowed at all.
6. Reconcile against `O43 Master Data Rule`, which names a `Golden source` per entity. Where the migration target is not the golden source, one of the two is wrong.

Signed by. Domain Architect, Data, `ROL-10`; the Security Architect, `ROL-17`, for any row touching `Confidential` or `Restricted` data.

Done when. No placeholder field names remain, every `Transform` has an expression or a reference, and every row's classification has been checked against the target's controls.

Shows up in. `RPT05 Data Ownership`, and `O48 Environment`'s `Personal data present` column.

Goes wrong when. The mapping is built from the target system's data dictionary. It has to be built from the source, because what you cannot find in the source is the gap.

Look it up. Volume 1, chapter 10, *Classification: the codes that decide what is lawful*.

Where it lands.

> **`O40 Source To Target Mapping`**
>
> `Ref | Entity | Source field | Target field | Rule`, twelve rows of placeholders. Keep one row per field, not one per entity: twelve rows for fourteen entities means nothing has been mapped.

Part five — the plan, and the gate

Step 9.13 — Reconcile `Delivery` against `End` before anybody quotes the roadmap

Produce. `O30`'s `Approval` and `Delivery` columns consistent with its own dates.

You need. `O30`; `RPT08 Roadmap`; `ZZ Checks`; today's date.

How.

1. Read `RPT08`'s footer: seven work packages are past their end date. Then read the `End` dates behind it. `WP-01` ends 2031-04 and `Delivery` reads `Past end date`. `WP-08` ends 2030-10 and reads `Delivered`. `WP-11` ends 2029-03 and reads `Delivered`. Nothing in the programme has started [1][12].
2. Decide which column is authoritative. `Start` and `End` are dates and can be checked; `Delivery` is an assertion. Dates win.
3. Reset `Delivery` from the dates and from the delivery function's actual position — `Not started`, `In flight`, `Delivered`, `Past end date` — reserving the last for a package whose `End` is genuinely behind today.
4. Reset `Approval` the same way. `WP-05 Access and security` reads `Approval` `Not started` and `Delivery` `In flight` while carrying $10.0m, the largest budget in the programme.
5. Re-run `ZZ Checks`. `Work packages past end date` should fall to a number you can defend, and if it falls to zero say so in the same sentence, because a zero that follows a seven needs an explanation.

Signed by. Head of Delivery; Chief Enterprise Architect for the repository.

Done when. `RPT08`'s footer agrees with its own `End` column, and `ZZ Checks` and `RPT01 Dashboard` report the same number.

Shows up in. `RPT08 Roadmap`'s footer, `ZZ Checks` and `RPT01 Dashboard` — three places computing off one column, which is why one wrong column is three wrong reports.

Goes wrong when. Nobody checks, the pack goes out, and an executive works out that a programme starting in 2027 already has seven packages past their end date. You will not get a second question.

Where it lands.

> **`O30 Work Package`**
>
> `Approval` and `Delivery` are the two assertion columns; everything else on the sheet is a fact that can be checked. Treat them as owned by the Head of Delivery and updated monthly, and write that down, because nothing in the repository enforces it.

THE TWO COLUMNS NOBODY OWNS

Who updates `Approval` and `Delivery` on `O30`, and on what day of the month?

Three reports compute off `Delivery`. Which of the three will an executive see first?

Step 9.14 — Assemble the plan and take it through the funding gate

Produce. `DLV-16 Migration Plan` and `DLV-17 Project Charters`, through `GT-0 G0 Charter`.

You need. Everything above; `O31 Deliverable`; `O33 Architecture Contract`; `O57 Governance Body`; `O58 Governance Gate`.

How.

1. Assemble the plan: projects with leads and dates, the dependency table the repository cannot hold, implementation factors with deductions, the closed-window register, costs with a basis, benefits with arithmetic, increments, risks, assumptions with tests, and the migration plans.
2. Write a charter per project against what `GT-0 G0 Charter` tests: scope, acceptance criteria and a contract exist before work starts [5].
3. Create the `O33 Architecture Contract` row per project. Eight of the twelve exist already and four read `Non-compliant` before anything has been built, which is a Phase G conversation you can start early.
4. Set `O31 Deliverable`: `DLV-16 Migration Plan` and `DLV-17 Project Charters`, both `Draft` today.
5. Take the funding to `GB-02 Investment Committee`, which holds `G0` funding and `G3` benefit confirmation and meets quarterly [5]. Quarterly means the pack is due on a date; missing it costs three months of a critical path that is already fifty-two.
6. Take the architecture to `GB-01 Architecture Board`, monthly, which holds `G1` and `G2`. Phase G starts the day after.

Signed by. `GB-02 Investment Committee` for funding at `GT-0`; `GB-01 Architecture Board` for the architecture content; Head of Delivery owns the plan thereafter.

Done when. Every project has a charter and a contract row, `O31` carries both Phase F deliverables at the status they have reached, and a dated `G0` decision exists in `O60 Governance Decision` — where `DEC-01 Adopt the recommended architecture alternative` reads `Outstanding` today.

Shows up in. `O60 Governance Decision` and `O33 Architecture Contract`. A funded programme with no `O60` row is a programme somebody will later claim was never approved.

Goes wrong when. The plan goes to the board without the dependency table and without the closed-window register, because neither has a sheet and both therefore got left out. Those two tables are the plan. Everything else is schedule.

Look it up. Volume 1, chapter 18, *The three artefacts that buy you credibility*.

Where it lands.

> **`O31 Deliverable` · `O33 Architecture Contract` · `O58 Governance Gate`**
>
> `O31` — `DLV-16 Migration Plan` and `DLV-17 Project Charters`, phase `F`. `O33` — `Ref | Architecture contract | Work package | Compliance status`, eight rows keyed on package rather than project, four of them `Non-compliant`. `O58`'s `GT-0 G0 Charter` tests that scope, acceptance criteria and a contract exist before work starts.

FOR THE REVIEWER, BEFORE THE PLAN IS FUNDED

Take the longest project. Name its predecessor, and say where that dependency is recorded now that no sheet holds it.

Take any benefit in the plan. Which `004 Measure` is it against, what is the baseline, and who reads the measure?

How many dates did the trading calendar move, and which window could the plan not get out of?

For the article master and the price history — what is the reconciliation rule, and what is the rollback point?

THE CHAPTER IN TEN LINES

- Phase F charters the work. One project per work package is a transcription, not a plan.
- No sheet in this repository records a dependency between two projects or two packages. Keep it in O52 with a deduction and in the plan, and say you have done so.
- The trading calendar governs the plan. No retailer changes anything between late October and mid-January, and six of O30's twenty-four dates sit inside that block.
- The freeze compresses the delivery year to about nine months, so fifty-two months of critical path is nearer seventy.
- You cannot cut over a merchandising system during a range review, and you cannot touch the price file in the run-up to a promotion.
- Add stock take, period end, the 4-5-4 calendar's 53rd week, back-to-school and the supplier trading-term renewal cycle. Every one of them is somebody else's date.
- O04 Measure has a Baseline and no target column, so the quantified benefit lives in the plan. A benefit that does not trace to an O04 Measure is fiction, and RSK-07 already predicts it.
- R23 pairs increments to projects in order, so INC-6 Core renewal is delivered by PRJ-11 Customer experience thirty-four months after it ended.
- Three of eight rows in O62 Delivery Risk read Certain. A certainty is an issue, not a risk.
- The article master and the price history are the critical path of any retail migration, because price records are versions and the retailer has to prove what the shelf said.

SOURCES FOR THIS CHAPTER

- **1.** Retail_TOGAF_Metamodel.xlsx, sheets O56 Project, O30 Work Package and O66 Architecture Increment
- **2.** Retail_TOGAF_Metamodel.xlsx, sheets O52 Implementation Factor, R14 Project delivers Work Packa, R15 Transition contains Work Pa and R23 Increment delivered by Proj
- **3.** Retail_TOGAF_Metamodel.xlsx, sheet O62 Delivery Risk
- **4.** Retail_TOGAF_Metamodel.xlsx, sheets O07 Constraint and O08 Assumption
- **5.** Retail_TOGAF_Metamodel.xlsx, sheets O57 Governance Body, O58 Governance Gate, O31 Deliverable, O33 Architecture Contract and O60 Governance Decision
- **6.** Retail_TOGAF_Metamodel.xlsx, sheets O02 Goal, O03 Objective, O04 Measure, R10 Objective measured by Measu and R11 Goal realised by Objective
- **7.** Retail_TOGAF_Metamodel.xlsx, sheets O39 Data Migration, O40 Source To Target Mapping, O37 Data Attribute and O43 Master Data Rule
- **8.** Retail_TOGAF_Metamodel.xlsx, sheets O20 Data Entity, O48 Environment, R05 Application manages Data En and RPT05 Data Ownership
- **9.** National Retail Federation, "4-5-4 Calendar" — the voluntary 4-5-4 convention, the trading week and the rule for adding a 53rd week https://nrf.com/resources/4-5-4-calendar
- **10.** Consumer Protection Act 68 of 2008, section 23 — disclosure of price of goods or services https://www.acts.co.za/consumer-protection/23_disclosure_of_price_of_goods_or_services
- **11.** The Open Group, TOGAF Standard, 10th Edition — ADM Phase F, Migration Planning https://pubs.opengroup.org/architecture/togaf10-doc/adm/
- **12.** Retail_TOGAF_Metamodel.xlsx, sheets RPT08 Roadmap, RPT01 Dashboard and ZZ Checks

CHAPTER 10

Implementation Governance

The phase where the architecture either refuses something, or admits that it cannot.

BEFORE YOU START THIS PHASE

Phase G is where an inspector and the architecture practice meet the same record. Volume 1 carries the instruments; this chapter carries the gates that produce the evidence.

- **Chapter 15, *Regulation, risk and what the regulator inspects*** — *What an inspector actually asks to see*; *Section 23, and why a stale shelf label is a legal exposure*; *Packaging, empowerment and the board*.
- **Chapter 7, *The operating model*** — *The trading calendar*: what decides when a gate may be held and when its outcome may be implemented.
- **Chapter 5, *Who does what*** — *Who decides what, on one page*: the decision rights to reconcile with `057 Governance Body`.

Halverson Stores Group is an invention. Its data is real within this workbook and nowhere else.

What you are building in this phase

Step	Produce	Lands in
10.1	The change-control spine, before any gate	Four new sheets, five new columns
10.2	Four gates, each with a holder and an evidence list	`058 Governance Gate`
10.3	A contract per package, and a derived compliance status	`033 Architecture Contract`
10.4	A charter `G0` can refuse	`030 Work Package`, `055 Building Block`
10.5	A design verdict before build	`059 Compliance Assessment` at `G1`
10.6	One verdict per acceptance criterion	`059`, `R24 Compliance Assessment tests`
10.7	Two control reports read together, and `R16` re-argued	`RPT09 Regulatory Coverage`, `RPT11 Security Position`
10.8	A decision-rights boundary that holds	`057 Governance Body`
10.9	A dispensation that expires on a date	`061 Governance Change Request`, `060 Governance Decision`
10.10	The trading governance written above the architecture board	`057`, `058` `Freeze position` (add)
10.11	The board's King IV duty mapped to sheets and columns	`057`, `060`, fourteen named sheets
10.12	Obligations at instrument grain, each testable	`063 Control Objective`, `059`
10.13	A gate calendar driven by the trading calendar	`057` `Cadence`, `030` `Start`, `End`

> **THREE CONVENTIONS FOR THIS CHAPTER AND THE NEXT**
>
> **A column marked (add) does not exist in the shipped file.** Step 10.1 specifies the change-control spine; later steps specify their own. Never fill a column that is not there.
>
> **The gates exist as rows and have never been held.** `058` carries four records; `059 Compliance Assessment` carries **zero** and `R24 Compliance Assessment tests` reports **0 instances** [3]. Every verdict in this repository is an assertion with nothing behind it.
>
> **The file says so about itself.** `ZZ Checks` reports *Work packages past end date* = **7** against a threshold of 0, and `030` agrees [6][9]. Nothing refused them, because no gate was ever held.

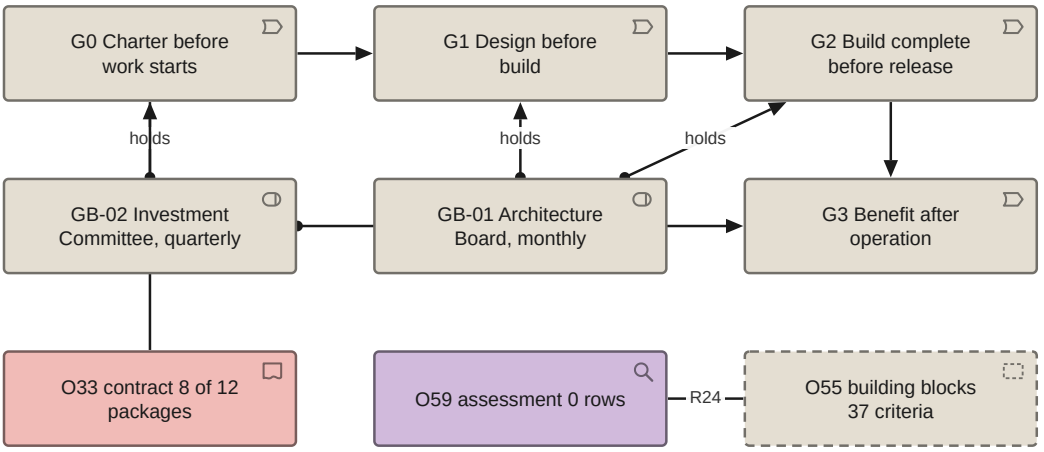


Figure 10.1 The four gates of `058 Governance Gate` as events in the delivery sequence, with the `057` body assigned to each. ArchiMate 4 draws the bodies as Roles because `057` holds mandates, not people. The contract, the criteria and the assessment are what each gate reads, and `059 Compliance Assessment` is drawn with its real row count: zero. The assignments are the `Held by` column step 10.2 adds — until it exists, no gate has an owner the model can resolve.

The spine this phase stands on

Step 10.1 — Add the change-control spine, or stop here

Produce. Four new sheets and five new columns, in place before a single gate is held.

You need. The workbook and an hour. Chapter 1 told you to do this before loading a record. If you did, check it against the specification below. If you did not, this is where the omission turns expensive: a gate verdict that cannot be dated or attributed is an opinion with a reference number.

Where it lands. Four new sheets, before `00 Index`:

- `01 Version Control` — one row per published version. `Version`, `Date`, `Author`, `Approved by`, `Scope of change`, `Change refs`, `Baseline ref`.
- `02 Change Log` — one row per field that moved. `Log ref`, `Date`, `Sheet`, `Record ref`, `Field`, `Old value`, `New value`, `Change ref`, `Changed by`, `Reason`.
- `03 Change Requests` — one row per request. `Change ref`, `Raised`, `Raised by`, `Against`, `Type`, `Description`, `Classification`, `Impact`, `Decision body`, `Decided`, `Status`, `Expires`, `Decision ref`.
- `04 Baselines` — one row per baseline cut. `Baseline ref`, `Date`, `Label`, `Scope`, `Record count`, `Approved by`, `Decision ref`, `Supersedes`.

Then **five columns on every object sheet**, appended after each sheet's last column so the `Ref`-first shape is undisturbed: `Version`, `Record Status`, `Changed By`, `Change Ref`, `Approved By`. `Record Status` takes `Proposed`, `Approved`, `Superseded` or `Retired`.

How.

1. Insert the four sheets using this file's header convention: title row 1, subtitle row 2, headers row 4, data from row 5. A sheet that breaks it breaks anything built over the workbook.
2. Append the five columns to all 64 object sheets in one pass. **`034` and `051` do not exist** — a loop from `001` to `066` fails on the two holes.
3. Back-fill `Record Status` for all 842 records with `Proposed`. Nothing has been through a gate.
4. Create version `0.1`, `Scope of change` reading *as delivered, illustrative data, nothing approved*. `ZZ Checks` agrees: *Records validated against a real organisation* reads `0` [9].
5. Raise the operation as the first row of `03 Change Requests`. `061`'s `CR-G03` and `CR-G04` are already repository corrections, so the file knows it needs fixing [5].

Signed by. `GB-01 Architecture Board`, which `057` mandates to *approve dispensations with an expiry* [4] — a mandate it cannot discharge without an `Expires` it can resolve.

Done when. `02 Change Log` has one row carrying a `Change Ref` that resolves to `03 Change Requests`.

Shows up in. Nowhere yet. None of the thirteen `RPT` sheets reads change control.

Goes wrong when. You skip it and reach step 10.6 anyway. `059` then fills with verdicts carrying no date and no assessor, and chapter 11's comparison tool classifies every one as ungoverned.

BEFORE YOU ADD A SINGLE GATE RECORD

Which of the four sheets already exists in your copy, and which are you about to create?

What is the one question about a past decision your repository cannot answer today?

The gates

Step 10.2 — Give each of the four gates a holder and a named evidence list

Produce. Four rows in `058` that a body can actually refuse at.

You need. `058`'s four rows, `057`'s five bodies, and `031 Deliverable`, where `DLV-18 Architecture Contract` and `DLV-19 Compliance Assessment` both read `Draft` at phase `G` [8] — the two drafts this chapter turns into records.

Where it lands.

> **`O58 Governance Gate`**
>
> Real columns: `Ref`, `Gate`, `When`, `What it tests`.
>
> Add: `Held by` (**add**) — the `O57` `Ref` of the body that can say no, not its name. `Evidence required` (**add**) — the artefact a refusal would be argued over. `Why it exists` (**add**). `Freeze position` (**add**) — step 10.10.

Keep the four rows as delivered: `GT-0 G0 Charter` tests that *scope, acceptance criteria and contract exist*; `GT-1 G1 Design` that *design meets principles, standards and reference architecture*; `GT-2 G2 Build complete` that *every acceptance criterion evidenced*; `GT-3 G3 Benefit` that the *stated benefit confirmed against a date* [1]. The holder is what is missing. `O57`'s `Mandate` states the allocation in prose [4], and prose in one sheet is not a join to another.

How.

1. Add the four columns. `Held by` = `GB-02` for `GT-0` and `GT-3`, `GB-01` for `GT-1` and `GT-2`. Money gates to the funders, design gates to the architects.
2. Write `Evidence required` as a named artefact: for `GT-1`, *the interface design and the* `R06` *rows it will create*; for `GT-2`, *an* `O59` *row per criterion with* `Evidence` *filled*.
3. Write `Why it exists` from the file. `GT-1` exists because 16 of 29 interfaces are unmanaged, 20 carry no logging and 9 are direct database reads [9]. Nobody refused those designs.
4. `DEC-06 Adopt the four-gate regime` already reads `Approved` [5]. Add the date and rationale — `O60` has no `Date` column, which is step 11.3's problem.
5. Name the person who chairs each gate. A delegate who never attends is not a member.

Signed by. `GB-01 Architecture Board`, with `GB-04 Executive Committee` as the recorded *escalation point when bodies cannot agree* [4].

Done when. Every `O58` row has a `Held by` holding a `GB-` reference that appears in `O57`.

Shows up in. Nothing computes over `O58` and `RPT01 Dashboard` counts no gates [7]. Track `O59`'s row count instead.

Goes wrong when. `Held by` holds a name rather than a `Ref`. Someone renames the board and the join dies silently.

Step 10.3 — Write the architecture contract for each package actually in delivery

Produce. One `O33` row per package with a delivery team, each with two parties and a date.

You need. `O30`'s twelve packages, `O56 Project`'s `Lead` values, and `R13 Work Package delivers Build`.

Where it lands.

> **O33 Architecture Contract**
>
> Real columns: `Ref`, `Architecture contract`, `Work package`, `Compliance status`.
>
> Add: `Parties` (**add**) — delivering organisation and gate-holding body. `Signed` (**add**) — the date a named human signed. `Expiry` (**add**) — past the package end date, never on it. `Status basis` (**add**) — where the status came from. `Review cadence` (**add**). `Last review` (**add**).

A four-column contract is a pointer. `O33` names the package and asserts a status; it does not say who is bound, when they agreed, or on what basis. Eight contracts cover `WP-01` to `WP-08` [2]. `WP-09` to `WP-12` have none — and `WP-11 Customer experience` reads `Delivery` = `Delivered` [6]. A package that delivered with no contract and no assessment is the most quotable finding here.

And `Compliance status` is derived, never typed. Publish this table as a `O60` decision and re-derive every status the day an `O59` row is added:

Verdicts across the package's criteria	`Compliance status`
No `O59` rows exist	`Not yet assessable`
Every criterion `Compliant`	`Compliant`
Any `Partial`, none `Non-compliant`	`Partially compliant`
Any `Non-compliant`	`Non-compliant`
Any `Not assessable`, no `Non-compliant`	`Not yet assessable`, criterion named

`O33` reads `Non-compliant` for `ACO-01`, `ACO-03`, `ACO-06` and `ACO-07` [2]. With `O59` empty all eight should read `Not yet assessable`. Four are asserting a verdict no assessment produced.

How.

1. Create `ACO-09` to `ACO-12`, one per uncovered package.
2. Set `Parties` from `O56`'s `Lead` and the gate-holding body. Where the lead is an architect, record that in `Status basis` — nobody is both author and assessor.
3. Set `Compliance status` to `Not yet assessable` wherever `O59` has no rows for the package's blocks, with `Status basis` reading *no assessment exists*. Four of eight already do.
4. Set `Expiry` later than `O30`'s `End`. `WP-10` ends `2031-09`, the latest in the file.
5. For `WP-11`, raise a change request instead of a contract. You cannot contract retrospectively — record what shipped, assess it at `G2` against `O55`, and let the verdict stand where a contract would have been.

Signed by. Both parties in `Parties`. A contract with one signature is a memo.

Done when. Every `O30` row with `Delivery` other than `Not started` has a contract or a recorded reason why not. Eleven of twelve qualify today.

Goes wrong when. The contract is filed with the vendor agreement and treated as commercial. `019 Contract` holds ten vendor agreements. `033` holds the architecture's right to refuse a build.

Shows up in. `033` itself, which no report reads. Twelve packages, twelve contracts.

Step 10.4 — Make G0 refuse a charter whose criteria were written rather than inherited

Produce. One rule, applied at `G0`, that does most of this phase's work.

You need. `055 Building Block`, whose 37 records all carry an `Acceptance criteria` value [10], and `R13`'s 37 block-to-package links.

Look it up. Volume 1, chapter 16, *Modelling retail in ArchiMate*, *Capabilities, value streams and the scarcest resource* — why a capability-level criterion and a project-level criterion are different sentences, and which one governs.

Where it lands. No sheet. You are testing `030 Work Package` against `055` and refusing where they disagree.

The rule: a charter's acceptance criteria are the `Acceptance criteria` strings of the blocks the package delivers, copied verbatim. Not improved. Not summarised. A project that wants different criteria raises a change request against `055`, decided at `GB-01`, not negotiated in the room. `055`'s criteria are uneven — `ABB-02` reads *No unmanaged interface to or from POS*, which is testable; all ten `BBB` rows read *... measured and owned*, which is not. Fix those before `G0`.

> **`R13` ALLOCATES BY POSITION, NOT BY ARGUMENT**
>
> `R13`'s 37 rows walk `WP-01` to `WP-12` and back again against `055` in `Ref` order. So `WP-01 Governance foundations` is recorded as delivering `BBB-01 Merchandising & Assortment capability uplift` and `DBB-03 Store / Location mastering`. Re-argue every allocation before you assess a package against it, and raise the re-allocation as one change request rather than 37.

How.

1. List the package's blocks from `R13`; pull their criteria from `055`.
2. Ask one question of each criterion: could a reasonable person disagree about whether it was met? If so, rewrite it in `055` under a change request before the gate sits.
3. Compare the charter's criteria to that list as sets. Refuse on any difference either way.
4. Refuse where `030 Approval` reads `Not started` and work has begun. `WP-05 Access and security` reads `Approval = Not started`, `Delivery = In flight`, cost $10.0m [6].
5. Record the rule once in `060`, beside `DEC-06`, rather than re-arguing it per charter.

Signed by. `GB-02 Investment Committee`, which holds `G0` and the funding behind it.

Done when. The charter's criteria set equals the `O55` set for its blocks.

Shows up in. `R13`, where every block the charter claims appears against the package `Ref`.

Goes wrong when. The gate negotiates the criterion instead of testing it. The moment criteria are negotiable at the gate, the gate is a status meeting.

Step 10.5 — Build the G1 design review, which has never produced a record

Produce. A `G1` verdict per design, before build, in `O59`.

You need. `O58 GT-1`, `O05 Principle` (24 records, four `Draft`) and `O32 Standard` (16 records, one `Breach`).

Look it up. Volume 1, chapter 13, *The reference architecture, How to choose, and how to tell when a model is standing in for thinking* — what a design review may legitimately test a design against, and what it may not.

Where it lands. `O59 Compliance Assessment` — `Ref`, `Building block`, `Block name`, `Acceptance criterion`, `Gate` = `GT-1`, `Verdict`, `Evidence`.

`GT-1` tests *design meets principles, standards and reference architecture* [1]. All three are sheets, and all three are in a state that makes the test harder than it looks. `PRN-05`, `PRN-10`, `PRN-15` and `PRN-20` read `Status` = `Draft` [11], and a design cannot breach a draft principle. `STD-09 Container image standard` reads `Deprecated` with `Conformance` = `Breach`, while `STD-11` and `STD-13` are deprecated yet `Conformant` [12]. Conformance to a withdrawn standard is not a finding either way.

How.

1. Approve or withdraw the four draft principles before the next `G1`. A draft principle is an argument the architect loses.
2. For each design, write one `O59` row per acceptance criterion it claims to satisfy, `Gate` = `GT-1`, `Verdict` = `Compliant`, `Partial`, `Non-compliant` or `Not assessable`.
3. Put the artefact in `Evidence`: a document reference, a diagram id, a configuration export. *Reviewed* is not evidence.
4. Test the interface design against `R06`'s `Managed` and `Logged` columns before build. 16 unmanaged and 20 unlogged interfaces are what `G1` exists to stop [9].
5. Where the standard a design should meet is deprecated, refuse the design and raise the standard as the change request.

Signed by. `GB-01 Architecture Board`, monthly per `O57 Cadence` [4].

Done when. No design enters build without an `O59` row at `GT-1`.

Shows up in. `RPT11 Security Position`, where `SEC-02 Design review` reads `Position` = `In place`, `Enforced at` = `Design gate` [15] — a control in place at a gate that has produced no record. That contradiction is the sharpest single finding in the file.

Goes wrong when. `G1` is held as a presentation. A design review that produces slides and no rows in `059` has not happened.

THE DEPARTURE YOUR NEXT DESIGN REVIEW WILL FIND

Which principle in `005` will the next design breach, and is it `Approved` or `Draft`?

`SEC-02 Design review` reads `In place`. What record proves it, and where is that record?

Assessing compliance

Step 10.6 — Assess one acceptance criterion at a time, never one project at a time

Produce. A register of verdicts at criterion grain, not a letter per project.

You need. `055`'s 37 criteria, `058`'s gates, and `030`'s delivery state.

Where it lands.

> **`059 Compliance Assessment`**
>
> Real columns: `Ref`, `Building block`, `Block name`, `Acceptance criterion`, `Gate`, `Verdict`, `Evidence`. Zero rows today [3].
>
> Add: `Assessed on` (**add**), `Assessed by` (**add**), `Change Ref` (**add**). Without the first two a verdict cannot be challenged, and a verdict that cannot be challenged is not governance.
>
> Then one row in `R24 Compliance Assessment tests` per `059` row. `R24` reports **0 instances** [3].

Thirty-seven blocks assessed at `G1` and again at `G2` is 74 rows — a register, not a burden, and the only artefact in this phase an inspector can read without being briefed.

Form G-1 — compliance assessment, one row per criterion

Feeds `059 Compliance Assessment`, then `R24`. One row per acceptance criterion per gate. A first pass on one work package is three to five rows. Signed by the chair of the body in `058 Held by`.

Building block	Acceptance criterion	Gate	Verdict	Evidence
ABB-02	No unmanaged interface to or from POS	GT-1	Example, delete me	

How.

1. Take the package's blocks from `R13`, after re-arguing the allocation at step 10.4.
2. Write one row per criterion per gate. Never aggregate to the package.
3. Use four verdicts only: `Compliant`, `Partial`, `Non-compliant`, `Not assessable`. The last is the honest verdict for an unfalsifiable criterion, and it is a finding against `055`, not the project.
4. Fill `Evidence` with an artefact reference. Ten of the 37 criteria read *… measured and owned* and cannot be evidenced as written — mark them `Not assessable` and raise them.
5. Mirror each row into `R24` the same day. Two sheets, one act.

Signed by. `GB-05 Architecture Function`, mandated to *assess compliance* [4], with the verdict taken at the gate's holding body.

Done when. `059` row count equals criteria in scope times gates held, and `R24` matches it.

Shows up in. `ZZ Checks`, whose *Object types populated* reads **65** against *Every type carries records* [9]. It is **64**: `059` is empty and `EXT Industry Extension` is the sixty-fifth type. Filling `059` is how that check stops lying.

Goes wrong when. Somebody writes one verdict per project. A package with eight criteria met and one breached then reads `Partial`, and nobody can say which failed.

Step 10.7 — Read RPT09 and RPT11 together, and re-argue every pairing in R16

Produce. A corrected obligation-to-control map, and the list of obligations with nothing behind them.

You need. `RPT09 Regulatory Coverage` (6 obligations), `RPT11 Security Position` (8 controls), `O63`, `O41 Security Control` and `R16 Control Objective met by Co` (6 instances).

Look it up. Volume 1, chapter 16, *Modelling retail in ArchiMate*, *A regulation, from driver to control* — the chain this step walks, element by element.

Where it lands. `R16` — `Control objective`, `Obligation`, `Control`, `Control name`, `Position`.

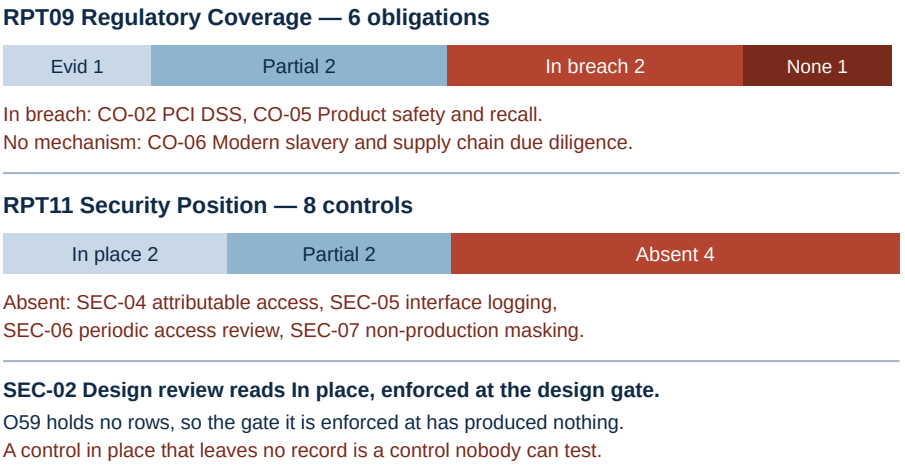


Figure 10.2 `RPT09 Regulatory Coverage` and `RPT11 Security Position` to scale, as shipped. Three of six obligations have no mechanism or a mechanism in breach; four of eight controls are absent. The two reports are joined by `R16`, whose six pairings are positional and must each be re-argued before either report is quoted.

`RPT09` closes with *3 obligations have no mechanism or a mechanism in breach* [14]. `RPT11` closes with *4 controls are absent. A control that is absent is not a control* [15]. Together they say what neither says alone: the obligations are failing because the controls that would evidence them were never built.

> **`R16` PAIRS BY POSITION AND CONTRADICTS `041`**
>
> `R16` maps `CO-01` to `SEC-01`, `CO-02` to `SEC-02`, and so on down the line — six obligations against the first six controls in `Ref` order [16]. So *Consumer protection and pricing accuracy* is recorded as met by `SEC-01 Identity lifecycle`, which processes joiners, movers and leavers. It does not evidence a displayed price. Worse, `R16`'s `Position` column carries the **obligation's** evidence status, not the control's: `CO-02` / `SEC-02` reads `In breach` in `R16` while `041` records `SEC-02` `Position` = `In place` [15][16]. Two sheets, one control, two answers. Re-argue all six pairings before quoting either report, and note that `SEC-07` and `SEC-08` appear in no pairing at all.

How.

1. For each obligation, name the control that actually produces the evidence. Several will have none — that is the answer, and it is `No mechanism`.
2. Delete the positional rows and rewrite `R16` from the argument. Expect more than six rows: one obligation usually needs several controls.
3. Fix `R16 Position` to mean one thing — the control's position from `041` — and let `063 Evidence` carry the obligation's status.
4. Where an obligation needs a control that does not exist, add it to `041` with `Position` = `Absent` and raise a gap. `029 Gap` holds exactly one row [17].
5. Put the corrected map in front of internal audit. `ROL-18 Internal Auditor` and `ORG-11 Internal Audit` are both in the file, and in the building.

Signed by. `GB-01` with `ROL-07 Chief Compliance Officer`, who owns all six regulatory drivers in `001` [18].

Done when. No `R16` row pairs an obligation with a control that could not evidence it, and every `063 Evidence` value derives from `R16` rather than being typed.

Shows up in. `RPT09` and `RPT11`, which compute straight off those sheets.

Goes wrong when. The reports reach a board pack before the pairings are re-argued. Three of six obligations then read as covered on the strength of a positional join.

Who may decide what

Step 10.8 — Write down what each body may not decide

Produce. A decision-rights table with an explicit *may not* column.

You need. `057`'s five bodies and `060`'s seven decisions.

Where it lands.

> **`057 Governance Body`**
>
> Real columns: `Ref`, `Governance body`, `Cadence`, `Membership`, `Mandate`.
>
> Add: `May not decide` (**add**). `Chair` (**add**) — a named role from `011 Role`. `Quorum` (**add**). `Escalates to` (**add**) — a `GB-` reference.

`057` is five rows and better than most real ones: `GB-01 Architecture Board` monthly, holding `G1` and `G2`; `GB-02 Investment Committee` quarterly, holding `G0` funding and `G3` benefit; `GB-03 Data Governance Council` monthly, designating data owners; `GB-04 Executive Committee` as escalation point; `GB-05 Architecture Function` continuous [4].

What it does not say is the useful half. `GB-01` may not fund, may not waive a statutory obligation and may not grant a dispensation without an expiry. `GB-02` may not approve a design. `GB-03` may not appoint a committee as an owner — `PRN-13 Data Ownership is Individual` says an owner is a person [11]. And none of the five may decide when a change reaches a store, which is step 10.10.

How.

1. Write `May not decide` for each body as two or three testable clauses.
2. Set `Escalates to` = `GB-04` for the other four, matching the recorded mandate.
3. Add `GB-03` to `GB-01`'s membership for any gate touching a data building block. Ten of `O55`'s 37 blocks are `DBB` rows.
4. Audit `O60` against the new boundaries. `DEC-02 Select the integration product` sits with `GB-02` and reads `Outstanding` — a design decision parked with a funding body.
5. Record the boundary table as a `O60` row, so changing it is a decision rather than an edit.

Signed by. `GB-04 Executive Committee`.

Done when. Every `O60` decision matches a body whose `Mandate` covers it, and the four `Outstanding` decisions have a named body and a date.

Shows up in. `O60`, where four of seven read `Outstanding` [5] — including `DEC-04 Designate data owners`, which blocks ten building blocks.

Goes wrong when. The boundary is written and never tested. It is tested the first time a body decides outside it and somebody says so in the minutes.

Step 10.9 — Grant a dispensation that expires on a date, or do not grant it

Produce. A dispensation register where every row has an expiry and a named risk holder.

You need. `O61 Governance Change Request` and `O60 Governance Decision`.

Where it lands.

> **`O61 Governance Change Request`**
>
> Real columns: `Ref`, `Change request`, `Against`, `Type`, `Status`, `Expires`.
>
> Add: `Risk holder` (**add**) — a role from `O11`, never a team. `Condition` (**add**) — what must be true before expiry. `Review date` (**add**).

`O61` is the best-shaped governance sheet in the file: it already carries `Type` = `Dispensation` and an `Expires` column, and `CR-G01 Dispensation for point-to-point file transfer` against `WP-04` reads `Granted`, `Expires 2029-06` [5]. Three years is long, and `Condition` is what makes it defensible — what should have happened by then, and who answers if it has not. The other four rows show the range: a scope change, two repository corrections, and `CR-G05 Refuse or re-baseline the integration package` at `Board decision required`.

How.

1. Refuse any dispensation with a blank `Expires`. There is no permanent exception; there is a changed standard.
2. Set `Review date` at the midpoint. A three-year dispensation reviewed once at the end is a three-year hole.
3. Name the `Risk holder` as an individual role. `ROL-08 Chief Enterprise Architect` already holds `RSK-08 Unmanaged interfaces proliferate` at `Likelihood` = `Certain` [13].
4. Mirror the grant into `O60`. `DEC-07 Grant dispensation for point-to-point transfer` already reads `Approved` [5] — the pattern exists, follow it.
5. Count open dispensations monthly. More than one per package and the standard is wrong, not the projects.

Signed by. `GB-01 Architecture Board` — its recorded mandate, word for word [4].

Done when. Every `Type` = `Dispensation` row has `Expires`, `Review date`, `Condition` and `Risk holder` filled.

Shows up in. `O61`, and `O62 Delivery Risk` where the risk is material.

Goes wrong when. `CR-G05` sits at `Board decision required` for two quarters. An undecided compliance question is a decision to proceed, taken by nobody.

BEFORE YOU GRANT THE NEXT DISPENSATION

What has to be true on the expiry date, and who is accountable if it is not?

`CR-G05` reads `Board decision required`. Whose diary is it in, and for when?

The retail governance overlay

Everything above would hold in a bank. What follows does not. A retailer carries a second governance system — older, better attended and more powerful than the architecture one — built around the trading year.

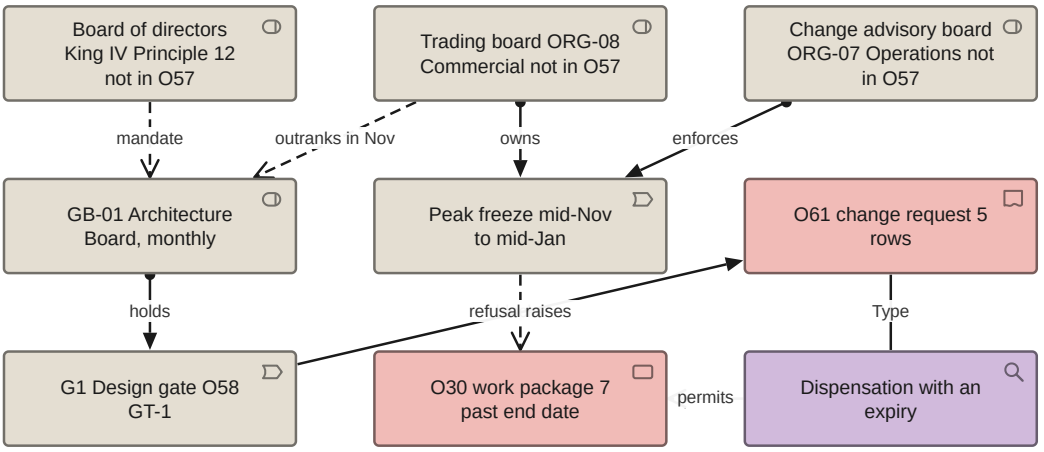


Figure 10.3 The retail governance overlay. Three of the bodies that bind change are not in `057 Governance Body` at all: the board of directors that King IV Principle 12 makes accountable, the trading board that owns the promotional calendar, and the change advisory board that enforces the peak freeze. In November the freeze blocks the work package whatever `G1` decided, which is why the trading director's influence over the architecture board is drawn as a real relationship rather than a political observation.

Step 10.10 — Put the trading governance above the architecture board, in writing

Produce. A written statement of which body outranks which, and when.

You need. `057`, `058`, `030`'s dates, and the group's change-freeze calendar.

Look it up. Volume 1, chapter 7, *The operating model*, *The trading calendar* — the freeze, what it covers and why the delivery year is about nine months. Volume 1, chapter 5, *Who does what*, *The trading calendar governs every diary in the building*.

Where it lands. `057`, as three new rows. `058`, as the `Freeze position` column.

Three bodies bind change in a retailer and none is in `057`:

- **The change advisory board.** `ORG-07 Operations`, under `ROL-15 Head of Service Management`. It approves production change weekly, and **it will not approve anything in the freeze window.** An architecture board approval is an input to it, never a substitute.
- **The trading board.** `ORG-08 Commercial`, under the Chief Commercial Officer. It owns the promotional calendar and the range review cycle, and it can move a date no architect can.
- **The governing body.** `GB-04 Executive Committee` is *CEO and direct reports* [4] — management. The board of directors is not in this file at all, which step 10.11 fixes.

The freeze is the part outsiders underestimate. It runs from roughly mid-November to mid-January for a grocery estate, longer for general merchandise, and it covers software releases, infrastructure, structural master-data change, range change, planogram resets and store refits. In November the trading director outranks the architecture board on anything that touches a trading store. Say so in the model rather than discovering it.

Form G-2 — gate freeze position

Feeds `058 Freeze position` (**add**). One row per gate. Two questions per gate, both answered yes or no. Signed by the chair of `GB-01` and the Head of Service Management.

Gate	May be held in freeze	Outcome may be implemented in freeze	Who decides	Note
GT-1	Yes	No	GB-01 / CAB	Example, delete me

How.

1. Add `GB-06 Change Advisory Board`, `GB-07 Trading Board` and `GB-08 Board of Directors` to `057` with their real cadence, membership and mandate. Three rows, and the model stops being fiction.
2. Set `Escalates to` so a `GB-01` refusal of something the trading board wants reaches `GB-08`, not `GB-04`.
3. Fill `Freeze position` for all four gates. `G0`, `G1` and `G3` may be held in the freeze; `G2` may be held but its outcome may not be implemented.
4. Lay the freeze over `030`'s dates. Seven packages read `Past end date` [6], and a plan built on twelve delivery months a year will produce more.
5. Write the nine-month delivery year into every charter at `G0`. A two-year roadmap holds about eighteen months of delivery.

Signed by. `GB-08 Board of Directors` on the hierarchy, `GB-06` on the freeze dates.

Done when. `057` holds eight bodies, every `058` row has a `Freeze position`, and no `030 End` date falls inside the freeze.

Shows up in. `RPT08 Roadmap`, where `Start` and `End` become statements about the trading year rather than the calendar one.

Goes wrong when. The architecture board approves a December release. The change advisory board refuses it, the project loses six weeks, and the practice loses credibility it will not recover in one cycle.

Step 10.11 — Map the board's King IV duty to sheets and columns

Produce. A one-page map from the governing body's duty to the columns that discharge it.

You need. `057` and `060`.

Look it up. Volume 1, chapter 15, *Regulation, risk and what the regulator inspects*, *Packaging, empowerment and the board* — King IV's effective date, the apply-and-explain basis, and what Principle 12 actually puts on the governing body.

Where it lands. `057` as `GB-08`, and `060` as a recorded decision adopting the map.

King IV applies on an apply-and-explain basis and is incorporated by the JSE Listings Requirements, which is how it binds in practice for a listed retailer [21]. **Principle 12 makes the governing body, not management and not technology, accountable for technology and information.** A board cannot apply that principle to an estate it cannot see described. This table is the description.

What the governing body must govern	Where it lives	The evidence column
The technology governance framework	`057`, `058`, `060`	`Mandate`, `What it tests`, `Status`
Value from technology investment	`030`, `056 Project`, `004 Measure`	`Cost`, `Delivery`, `Baseline`
Information assets and ownership	`020 Data Entity`, `043 Master Data Rule`	`Owner`, `Master system`, `Lineage`
Security and privacy	`041`, `064 Security Policy`, `048 Environment`	`Position`, `Personal data present`
Technology risk	`062 Delivery Risk`, `027`	`Likelihood`, `Impact`, `Support end`
Compliance with obligations	`063`, `R16`, `059`	`Evidence`, `Verdict`
Resilience of important services	`049 Resilience Tier`	`Recovery objective`, `Tested`

How.

1. Walk the table and check each named column is populated. Several are not: three control objectives have no evidence and two data entities have no lineage [9].
2. Where a column is thin, decide whether to fill it or to record where the evidence lives. Both are acceptable. Silence is not.
3. Give risk, compliance and internal audit a standing seat at `GB-01`, whose `Membership` names architects, the CIO and the CISO [4] and none of the three.
4. Put the map in front of internal audit before anyone outside sees it, and record it in `060` beneath `DEC-05`, which already reads `Approved` [5].

Signed by. `GB-08 Board of Directors`, with `GB-01` maintaining it.

Done when. Every row names a column that has values in it, or names where the evidence lives.

Shows up in. `ZZ Checks`, the nearest thing here to a self-assessment: fifteen rows, eight of which fail their threshold [9].

Goes wrong when. The practice prepares a board paper rather than being inspectable. If the map takes two weeks to assemble, the map is not the problem — the columns are.

Step 10.12 — Make the obligations that land on systems testable at a gate

Produce. `063` rows at instrument grain, each with a gate and a column that evidences it.

You need. `063`'s six generic obligations and `059`.

Look it up. Volume 1, chapter 15, *Regulation, risk and what the regulator inspects — What an inspector actually asks to see*, *Section 23, and why a stale shelf label is a legal exposure*, *POPIA, with the retail bite*, *Good Pharmacy Practice, and a rule that turns a database into a compliance obligation*, *Liquor, and what it binds at the till*, and *Tax, credit, measurement and the card estate*.

Where it lands. `063 Control Objective` — `Ref`, `Obligation`, `Source`, `Control objective`, `Evidence`. The instrument and its section go in `Obligation`, the commencement or gazette reference in `Source`.

The retail difference: most of what binds a retailer lands on a system, not a person. These obligations are about what the shelf displayed, what the scale weighed, what the till refused and what the dispensary recorded. None of it is a training matter. All of it is columns.

Obligation	What it binds	Evidence an inspector would ask for
CPA s 23(6)(b) and s 23(9)	The lower of two displayed prices binds; the error defence has two limbs, both data [19]	`020` Price `Master system`, `Lineage`; `R06 Logged` on the price-file interface; label print status, which has no home here
CPA s 56	A per-sale-line clock, the election held by the consumer [20]	`020` for the sale line and its basis; `037 Data Attribute Classification`
POPIA	Consent, special personal information, non-production data [19]	`048 Environment Personal data present`; `SEC-07 Non-production masking`, `Absent` [15]; `020 Classification`
Pharmacy recordkeeping	Dispensing records, the pharmacist on the day, and the version of the interaction-checking software [19]	`027 Support end`, `Position`; `032 Conformance`. **`023` holds no dispensing system at all**
Legal metrology	An unverified instrument must be verified before use; relocation makes it unverified [19]	Nothing. There is no instrument register. `EXT X02 Store / Location` is where one belongs
Liquor licence at the till	Hours and licensed area resolved to the site's municipality; the refusal recorded [19]	`012 Location` carries only `Type` and `Scale`. No licence, no conditions, no dates

Form G-3 — obligation to evidence, one row per instrument

Feeds `063 Control Objective`. One row per instrument and section, not per statute. A South African multi-format retailer needs twenty to thirty rows; `063` has six. Signed by the Chief Compliance Officer with `GB-01`.

Obligation	Source	Control objective	Gate	Evidence
CPA s 23(6)(b)	Act 68 of 2008, in force 2011-04-01	Example, delete me	GT-2	

How.

1. Replace `O63`'s six generic rows with instrument-level rows. *Consumer protection and pricing accuracy* is a heading; `CPA s 23(6)(b)` is an obligation.
2. Name the gate at which each is tested. Pricing display and till behaviour belong at `GT-2`; classification and retention at `GT-1`.
3. Where the evidence has no column, say so in the cell and raise it as a gap. Legal metrology and liquor licensing have no home in this repository today.
4. Decide in writing whether Halverson trades pharmacy. If it does, `O23`'s twelve applications are missing the most regulated system in the group. If it does not, record that in `O08 Assumption`.
5. Keep PCI DSS where it belongs. `CO-02` reads `In breach` [14], and PCI DSS is contractual rather than statutory — there is no regulator to appeal to [22].

Signed by. `ROL-07 Chief Compliance Officer` on interpretation, `GB-01` on gate allocation.

Done when. `O63` holds no row whose `Obligation` is a topic rather than an instrument, and every row names a gate.

Shows up in. `RPT09 Regulatory Coverage`, which will grow well past six rows.

Goes wrong when. The practice writes a compliance memo instead of rows. A memo is not testable at a gate, and is not what an inspector asks for.

THE OBLIGATION WITH NO COLUMN BEHIND IT

Which of the six obligations above has no evidence column in your repository at all?

Who owns the instrument register for scales — a role, from `O11 Role`?

If an inspector asked for the price displayed in one store on one date, how many systems would you have to query?

Running it

Step 10.13 — Run the gate calendar off the trading calendar, not the delivery plan

Produce. A dated gate calendar for two quarters, with named attendees and the freeze marked.

You need. `O57`'s `Cadence`, `O30`'s `Start` and `End`, `O66 Architecture Increment`'s six increments, and the freeze dates from step 10.10.

Look it up. Volume 1, chapter 16, *Modelling retail in ArchiMate*, *The roadmap, and what is true on 31 December* — why a plateau date inside the freeze is a date with ambition attached.

Where it lands. No sheet holds a calendar. Drive it from `O57 Cadence`: `GB-01`, `GB-03` and `GB-04` monthly, `GB-02` quarterly, `GB-05` continuous [4].

How.

1. Mark the freeze first, then fit the gates around it.
2. Lay the twelve packages against the trading year using `Start` and `End`. Seven are past their end date and need a `G2` now, not a `G0`.
3. Set `G0` and `G3` on the quarterly `GB-02` cadence. That is a real constraint on how fast a package can start, and it is in the file.
4. Cap each meeting by criteria count. Ten criteria is an hour of honest argument; `O55` has 37.
5. Record every held gate as `O59` rows the same day. Not the same week.

Signed by. The chair of each body, per `O57 Chair` (**add**).

Done when. The next two quarters are dated, and every `O30` package appears at a gate or carries a recorded reason why not.

Shows up in. `O59`'s row count, growing monthly. If it stops growing while the calendar runs, the meetings are happening and the gates are not.

Goes wrong when. The calendar is built from the delivery plan. The gate then moves whenever the project moves, which is the same as no gate.

When the answer is to refuse

Phase G is the only phase where architecture's main output is a refusal. This repository shows what an estate looks like when nothing was ever refused: 16 unmanaged interfaces, 20 with no logging, 9 direct database reads, 3 components past vendor support, 4 security controls absent, 3 control objectives without evidence, 2 data entities without lineage and 7 work packages past their end date [9]. None of that is a technology failure. Each is a design nobody said no to.

The refusal has to be cheap to make and expensive to overturn. Cheap, because an architect who needs permission to refuse stops refusing. Expensive, because a refusal overturnable in a corridor is a suggestion. The four gates, the criterion-grain assessment, the derived status and the expiring dispensation produce that asymmetry.

One refusal in a retailer is cheaper than the rest: refusing a date inside the freeze. Nobody argues with the trading calendar. Use it.

`ZZ Checks` closes with a line worth reading twice: *The thresholds are what a healthy estate would show. The results are what this one does. That distance is the work* [9]. A gate held without a record closes none of it.

PHASE G — A REVIEWER'S QUESTIONS

Which gate, held once in the next ninety days, would have refused the worst thing in your estate?

How many rows are in your `059` today, and how many should there be given the blocks in scope?

Name the decision `GB-01` took last quarter that it had no mandate to take. What did you do about it?

On 20 November, who outranks you, and does your governance model say so?

THE CHAPTER IN TEN LINES

- Phase G produces refusals, dispensations and evidence. A practice that cannot refuse made every earlier phase decorative.
- Add `01 Version Control`, `02 Change Log`, `03 Change Requests` and `04 Baselines`, plus `Version`, `Record Status`, `Changed By`, `Change Ref` and `Approved By` on all 64 object sheets, before any gate is held.
- `O58` carries four gates and `O59` carries zero assessments. Every gate in this repository exists on paper and has never been held, and `ZZ Checks` reports seven work packages past their end date to prove it.
- `O33` has four columns and needs `Parties`, `Signed`, `Expiry` and `Status basis` before it can govern. Eight contracts cover twelve packages, and four assert `Non-compliant` with no assessment behind them.
- Assess one acceptance criterion at a time. 37 blocks across `G1` and `G2` is 74 rows — a register, not a burden.
- `R16` pairs each obligation with the control of the same number and carries the obligation's status in the control's `Position` column. Re-argue all six pairings before quoting `RPT09` or `RPT11`.
- `SEC-02 Design review` reads `In place`, enforced at the design gate, and the design gate has produced no record. That is the sharpest single finding in the file.
- `O61` already carries `Type` = `Dispensation` and an `Expires` column. No date, no dispensation — and `CR-G05` has been sitting at `Board decision required`.
- Three bodies that bind retail change are missing from `O57`: the board of directors, the trading board and the change advisory board. Add them, with an `Escalates to`.
- The freeze is the cheapest refusal an architect has, and the most expensive to ignore. The retail delivery year is about nine months; write that into every charter at `G0`.
- King IV Principle 12 puts technology and information with the governing body. The evidence is columns in these sheets, not a policy document.
- Most of what binds a South African retailer lands on a system, not a person: the price displayed, the scale's verification, the till's refusal, the dispensary's record. All of them are columns, and three of them have no column here at all.

SOURCES FOR THIS CHAPTER

- **1.** Retail_TOGAF_Metamodel.xlsx, sheet O58 Governance Gate
- **2.** Retail_TOGAF_Metamodel.xlsx, sheet O33 Architecture Contract
- **3.** Retail_TOGAF_Metamodel.xlsx, sheets O59 Compliance Assessment (0 records) and R24 Compliance Assessment tests (0 instances)
- **4.** Retail_TOGAF_Metamodel.xlsx, sheet O57 Governance Body
- **5.** Retail_TOGAF_Metamodel.xlsx, sheets O60 Governance Decision and O61 Governance Change Request
- **6.** Retail_TOGAF_Metamodel.xlsx, sheet O30 Work Package
- **7.** Retail_TOGAF_Metamodel.xlsx, sheet RPT01 Dashboard
- **8.** Retail_TOGAF_Metamodel.xlsx, sheet O31 Deliverable
- **9.** Retail_TOGAF_Metamodel.xlsx, sheet ZZ Checks
- **10.** Retail_TOGAF_Metamodel.xlsx, sheet O55 Building Block
- **11.** Retail_TOGAF_Metamodel.xlsx, sheet O05 Principle
- **12.** Retail_TOGAF_Metamodel.xlsx, sheets O32 Standard and RPT12 Standards Conformance
- **13.** Retail_TOGAF_Metamodel.xlsx, sheet O62 Delivery Risk
- **14.** Retail_TOGAF_Metamodel.xlsx, sheets O63 Control Objective and RPT09 Regulatory Coverage
- **15.** Retail_TOGAF_Metamodel.xlsx, sheets O41 Security Control and RPT11 Security Position
- **16.** Retail_TOGAF_Metamodel.xlsx, sheet R16 Control Objective met by Co
- **17.** Retail_TOGAF_Metamodel.xlsx, sheets O29 Gap and RPT07 Gap Analysis, 1 record
- **18.** Retail_TOGAF_Metamodel.xlsx, sheet O01 Driver
- **19.** Volume 1, chapter 15 — the instruments and their architecture consequences, with the primary sources listed there
- **20.** Consumer Protection Act 68 of 2008, s 56 — implied warranty of quality and the six-month return right https://www.acts.co.za/consumer-protection/56_implied_warranty_of_quality
- **21.** Werksmans, King IV guide (apply-and-explain, seventeen principles, Principle 12 on technology and information) https://werksmans.com/wp-content/uploads/2025/04/Werksmans_KingIV_Booklet_V2.pdf
- **22.** PCI Security Standards Council, PCI DSS standards page https://www.pcisecuritystandards.org/standards/pci-dss/
- **23.** The Open Group, The TOGAF Standard, 10th Edition — Architecture Development Method, Phase G: Implementation Governance https://pubs.opengroup.org/togaf-standard/adm/

CHAPTER 11

Architecture Change Management

Every change is either governed or discoverable later as a defect. There is no third state.

> **BEFORE YOU START THIS PHASE**
>
> Phase H is the only phase that never ends. It is also the only one that will tell you, in a number, whether the practice is working.
>
> - **Chapter 7, *The operating model*** — *The trading calendar*: the cycle every trigger in this chapter arrives on.
> - **Chapter 19, appendix A, *The cross-maps*** — *Keeping these maps alive*: the maintenance rule this chapter operationalises.
> - **Chapter 13, *The reference architecture*** — *The two-dimensional migration, and Sunrise 2027 precisely*: the standards change most likely to force a cycle in the next three years.

Halverson Stores Group is an invention. Its data is real within this workbook and nowhere else.

What you are building in this phase

Step	Produce	Lands in
11.1	A change request before anything moves	`03 Change Requests` (add), `O61`
11.2	A log line per field, with a reference	`02 Change Log` (add)
11.3	A decision that is a record, not a recollection	`O60 Governance Decision`
11.4	A classification: simplification, incremental, re-architecting	`03 Change Requests` `Classification`
11.5	Two extracts, produced by hand	The comparison tool's paste targets
11.6	An ungoverned-change count, read as a practice number	The tool's `Review` sheet
11.7	A cadence tied to something that already happens	`O57` `Cadence`
11.8	A baseline, cut only when a decision needs one	`04 Baselines` (add)
11.9	Three health reports with target numbers	`ZZ Checks`, `RPT13`, `RPT06`
11.10	Volume 1's cross-maps, still true	`R01` – `R25`
11.11	A Phase H cycle per retail trigger	`O61`, `O60`, `O29 Gap`
11.12	A support expiry answered as a dated change	`O27`, `RPT10 Technology Currency`
11.13	Six numbers and two decisions for the board	`O04 Measure`, `O60`

THE RULE THE WHOLE CHAPTER RESTS ON

Every field change carries a reference, or it is ungoverned. The comparison tool has no view into intent. It compares two extracts and classifies each difference as a design delta, governed drift or **ungoverned change**. A value that moved with no `Change Ref` behind it lands in the third bucket whatever the reason was. There is no appeal and there should not be one.

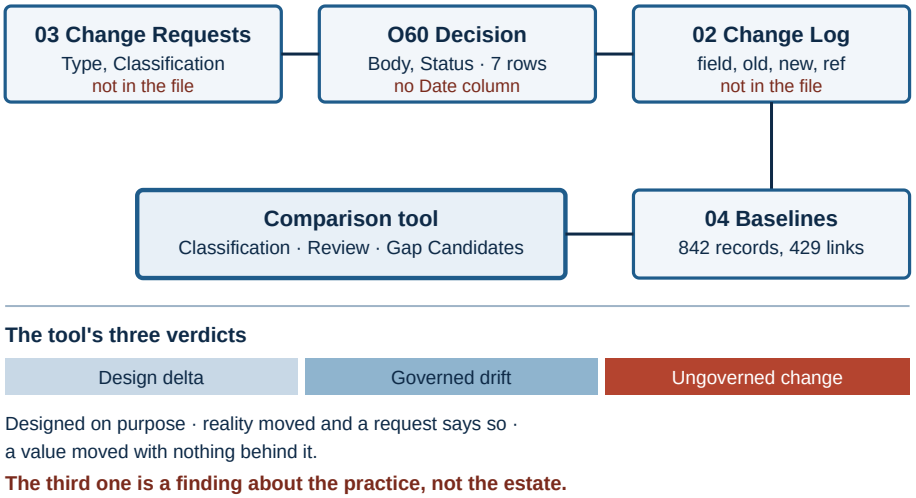


Figure 11.1 The change pipeline, and the single place it is enforced. Three of the five boxes do not exist in the repository as shipped; step 10.1 adds them. A field value that moves without a `02 Change Log` row carrying a `Change Ref` is classified as ungoverned change by the comparison tool, with no appeal.

The pipeline you have to build

The shipped file has two governance sheets and no pipeline. `O60 Governance Decision` holds seven decisions with `Ref`, `Decision`, `Body` and `Status` — and **no date, no rationale, no reference to whatever was being decided** [1]. `O61 Governance Change Request` is better: `Ref`, `Change request`, `Against`, `Type`, `Status`, `Expires`, five rows [2]. Neither is a change log, and there is no `03 Change Requests` sheet. Chapter 10 told you to add four sheets. This chapter is what they are for.

Step 11.1 — Raise the change request before you touch a cell

Produce. One `03 Change Requests` row per intended change, raised before the change is made.

You need. The four sheets from step 10.1, and `O61`'s five rows as the pattern for `Type`.

Where it lands.

> **`03 Change Requests (add)`**
>
> `Change ref`, `Raised`, `Raised by`, `Against`, `Type`, `Description`, `Classification`, `Impact`, `Decision body`, `Decided`, `Status`, `Expires`, `Decision ref`.
>
> `Against` holds a sheet and `Ref`, or the word `Repository` — `O61` already uses both [2]. `Type` takes `Dispensation`, `Scope`, `Correction`, `Compliance` or `New requirement`. `Decision body` is a `GB-` reference. `Classification` is step 11.4.

A change request has to answer four questions before a body will look at it: what is changing, against which record, who wants it, and what breaks if it is refused. Three of those are columns. The fourth, `Impact`, is the one architects skip and the one bodies read first.

How.

1. Copy `O61`'s five rows into `03 Change Requests` as the first five entries. They are real governance history and they belong in the pipeline, not in a side sheet.
2. Keep `O61` as the register of governance-level requests — dispensations and gate outcomes — and let `03 Change Requests` carry everything, including repository edits.
3. Write `Against` as a resolvable reference. *The capability map* is not one; `O13` plus a `Ref` is.
4. Write `Impact` as a count: how many records and which relationship sheets. A hierarchy change to `O13` touches `R01 Capability decomposition`'s 90 rows.
5. Refuse to process a request with a blank `Raised by`. Anonymous change is ungoverned change with a form attached.

Signed by. `GB-05 Architecture Function`, which `O57` mandates to *maintain the repository* [3].

Done when. No edit is made to any object sheet without a `Change ref` that exists first.

Shows up in. `02 Change Log`, where every row's `Change Ref` resolves. A log row whose reference does not resolve is the defect this step exists to prevent.

Goes wrong when. The request is raised after the edit, to tidy up. The dates then prove it, which is worse than not having them.

Step 11.2 — Log every field change, with its reference

Produce. A `02 Change Log` with one row per field that moved.

You need. `03 Change Requests` with rows in it.

Where it lands.

`02 Change Log (add)`

`Log ref`, `Date`, `Sheet`, `Record ref`, `Field`, `Old value`, `New value`, `Change Ref`, `Changed by`, `Reason`.

One row per **field**, not per record and not per session. Changing `O23`'s `Disposition` and `Health` on one application is two rows.

Form H-1 — change log, one row per field

Feeds `02 Change Log` (**add**). Fill it at the desk, in the meeting, before the edit. A normal month is twenty to forty rows. Signed by whoever made the edit.

Sheet	Record ref	Field	Old → new	Change Ref
O23 Application Component	A08	Disposition	Retire → Replace	Example, delete me

How.

1. Log the old value before you overwrite it. Excel will not tell you afterwards, and neither will the comparison tool: it can see that a value differs, not what it used to be.
2. Fill `Change Ref` on every row. A blank one is the only defect this sheet can have.
3. Write `Reason` in one clause, aimed at someone reading it in two years.
4. Log deletions as a row with a blank `New value`, never by deleting the record and saying nothing. A deleted record is the hardest change to detect and the easiest to regret.
5. Log relationship changes too. `R06 Application interfaces with` has a `Ref` column, so its rows are loggable; most relationship sheets do not, and for those the `Record ref` is the pair of end references.

Signed by. The person in `Changed by`. This is the one record in the repository that does not need a committee.

Done when. Every difference between two consecutive versions of the workbook is explained by a `02 Change Log` row.

Shows up in. The comparison tool's `Classification` column, where an unexplained difference reads `Ungoverned change`.

Goes wrong when. Somebody logs a day's work as one row reading *updated application portfolio*. That is a note, not a log, and it will not survive the first challenge.

Step 11.3 — Decide it at a body, and record the decision as a record

Produce. An `O60` row per decision, with the four columns it is missing.

You need. `O60`'s seven decisions and `O57`'s bodies.

Where it lands.

> **`O60 Governance Decision`**
>
> Real columns: `Ref`, `Decision`, `Body`, `Status`.
>
> Add: `Date` (**add**) — there is none, which makes the whole sheet unauditable. `Rationale` (**add**) — two sentences. `Change Ref` (**add**) — what was being decided. `Review by` (**add**) — for a decision with a shelf life.

Four of the seven read `Status` = `Outstanding`: `DEC-01 Adopt the recommended architecture alternative`, `DEC-02 Select the integration product`, `DEC-03 Fund compliance mechanisms` and `DEC-04 Designate data owners` [1]. With no `Date` column there is no way to tell whether they have been outstanding for a week or two years. `DEC-04` blocks ten `DBB` building blocks and the whole of `GB-03`'s mandate. Date them first.

How.

1. Add the four columns and back-fill `Date` from the minutes. Where there are no minutes, write *date unknown* rather than guessing — it is itself a finding.
2. Set `Review by` on `DEC-07`, the dispensation grant, to match `CR-G01`'s `Expires` of `2029-06` [2].
3. Give every `Outstanding` decision a body, a date and an owner. An undated outstanding decision is a decision to do nothing, taken by nobody.
4. Link each decision to its `Change ref`, so the decision and the edit it authorised sit on the same thread.
5. Record refusals. A refused change request with no `O60` row looks, a year later, like a request nobody read.

Signed by. The chair of the body in `Body`.

Done when. Every `O60` row has a `Date`, and every `03 Change Requests` row with `Status` = `Decided` has a `Decision ref` that resolves.

Shows up in. `060` itself. No report reads it, which is why the sheet decays quietly.

Goes wrong when. Decisions are recorded as minutes and never as rows. Minutes are prose in a folder. A row joins.

Step 11.4 — Classify the change, as three questions about the repository

Produce. A `Classification` on every change request, decided by test rather than by feel.

You need. The request, and the repository.

Look it up. Volume 1, chapter 9, *The conceptual data model*, *The six hierarchies, all of them correct* — why a hierarchy change is a change of grain rather than a change of value, and why that single distinction decides the classification of most retail change.

TOGAF's three classes are easy to recite and hard to apply. Make them questions about sheets:

Class	The test	Where it goes
Simplification	Does any **object sheet row** change, or only an attribute of one?	Decided at `GB-01`, logged, no ADM cycle
Incremental	Do object rows change but no **relationship sheet** changes shape?	A new or amended `030 Work Package`, via `GB-02`
Re-architecting	Does a **relationship sheet** gain or lose a pattern — a new join, a changed grain, a new object type in `EXT`?	Back into Phase A

The test is mechanical on purpose. Consolidating two POS instances changes `023` rows and `R07` rows but no pattern: incremental. Adding a franchise ownership dimension to `X02 Store / Location` changes what a store **is**, and every sheet that joins to a store: re-architecting. Re-pointing an interface from a direct database read to the integration platform changes `R06`'s `Managed` value on one row: simplification.

How.

1. Ask the three questions in order and stop at the first yes.
2. Where the answer is re-architecting, say so in writing and early. The expensive version of this step is discovering it in month fourteen.
3. Record the classification in `03 Change Requests` `Classification`, and let it choose the decision body.
4. Count the classifications quarterly. A quarter with no simplifications means small changes are going round the pipeline, not through it.
5. Never classify by cost. A R200,000 change that adds an object type is re-architecting; a R40m platform swap that changes no join is incremental.

Signed by. `ROL-08 Chief Enterprise Architect`, who owns the test, not the outcome.

Done when. Every open request carries a classification and the body that matches it.

Shows up in. `029 Gap` and `030`, which should gain rows from incremental changes and never from simplifications.

Goes wrong when. Everything is classified incremental, because that is the class with a funded route. Then re-architecting arrives as a surprise with a delivery date already on it.

THE CHANGE YOU MADE WITHOUT A REQUEST

Name the last change you made to this repository. What would its `03 Change Requests` row have said?

Which of the three classes does it fall in, by the test above rather than by instinct?

Drift you were not told about

Step 11.5 — Produce the two extracts by hand, because this repository cannot export

Produce. A baseline extract and a target extract, in the comparison tool's paste format.

You need. The comparison tool, the repository, and an hour. **There are no export sheets in this file** — the banking repository shipped three, this one ships none. Every extract is produced by hand, and saying so is better than pretending a button exists.

Where it lands. The tool's five paste targets. The tool classifies each difference as a design delta, governed drift or ungoverned change, writes judgements to a human `Review` sheet, and shapes `Gap Candidates` to paste into `029 Gap`.

How.

1. Decide the five extracts the tool takes and write the column order down once, so two people produce the same shape.
2. Pull each extract by copying the object sheet's data range — headers on row 4, data from row 5 — into a flat sheet. Keep `Ref` first.
3. Take the baseline extract from the last entry in `04 Baselines`, not from memory and not from a file on somebody's laptop.
4. Take the target extract from the live workbook on a stated date. Write the date into the extract.
5. Paste, run, and read the `Review` sheet before anyone else does.

Signed by. `GB-05 Architecture Function`.

Done when. Two dated extracts exist and the tool has produced a `Review` sheet with a verdict against every row.

Shows up in. `Gap Candidates`, which should produce rows for `O29 Gap`. `O29` holds **one** row today [8] — one gap in an estate with eight failing integrity checks.

Goes wrong when. The extract is taken from a copy somebody had locally. Then the comparison is between two versions of the architect's own memory.

Step 11.6 — Read the ungoverned count as a practice failure, not an estate failure

Produce. A dated ungoverned-change count, and an owner for each row in it.

You need. The tool's `Review` sheet and `02 Change Log`.

Three classifications, three different meanings:

- **Design delta** — the target differs from the baseline because somebody designed it that way. Expected. It is what the roadmap is.
- **Governed drift** — reality moved and a change request says so. Fine. Close the loop by updating the baseline at the next cut.
- **Ungoverned change** — a value moved with no reference behind it. This is not a finding about the estate. It is a finding about the practice, and it belongs to the architect.

The distinction matters because the three have different owners. A design delta is the roadmap's and needs nothing. Governed drift is the business's and needs a baseline refresh. Ungoverned change is yours, and the only honest response is to find the route it came in by and close it. Nobody else in the building will raise it, because nobody else can see it.

Form H-2 — ungoverned change review

Feeds nothing; drives `03 Change Requests`. One row per ungoverned difference. A first run on a live repository usually produces fifteen to forty. Signed by `ROL-08 Chief Enterprise Architect`.

Sheet	Record ref	Field	What changed	Retrospective ref
O27	PTC-01	Position	Supported → Past support	Example, delete me

How.

1. Count the ungoverned rows and write the number down with a date. One number, tracked over time, is the whole measurement.
2. Raise a retrospective change request for each. Retrospective is allowed; silent is not.
3. Find the route the change took. It is almost always one of three: a spreadsheet mailed round, a vendor's data loaded without review, or an architect tidying up.
4. Fix the route, not the row. A row fixed with the route open comes back next quarter.
5. Set a target. First run: record it. Second run: half it. Third run: single digits.

Signed by. `ROL-08 Chief Enterprise Architect`, who is already recorded as owning `RSK-08 Unmanaged interfaces proliferate` at `Likelihood` = `Certain` [9].

Done when. Every ungoverned row has a retrospective reference and a named route.

Shows up in. `MSR-15 Architecture compliance rate`, baseline **21%** [10]. That number cannot move while ungoverned change is rising.

Goes wrong when. The count rises and gets explained. A rising ungoverned count with a good explanation each quarter is a practice that has stopped governing and started narrating.

READING THE UNGOVERNED COUNT

What is your ungoverned count today, and what was it last quarter?

Which of the three routes produced most of it, and who owns that route?

Step 11.7 — Set the cadence, and tie each run to something that already happens

Produce. A dated schedule for the comparison run and the integrity check.

You need. `057 Cadence`, `ZZ Checks`, and the trading calendar.

Look it up. Volume 1, chapter 7, *The operating model*, *The trading calendar* — why a quarterly cadence in a retailer is not four evenly spaced quarters.

Where it lands. No sheet. Drive it from `057`: `GB-01` monthly, `GB-02` quarterly, `GB-05` continuous [3].

How.

1. Run `ZZ Checks` before and after every phase, and read all fifteen rows. Eight fail their threshold today [5].
2. Run the comparison tool quarterly, and never during the freeze — the repository is quiet then and the run says nothing.
3. Tie the run to a meeting that exists. The quarterly `GB-02` sitting is the obvious host; a standalone architecture review gets cancelled.
4. Add one run after peak, in late January, when the backlog of deferred change has just landed in production. That is the run with the findings in it.
5. Publish the date three months ahead. A surprise comparison run makes enemies and finds the same things.

Signed by. `GB-05 Architecture Function`.

Done when. Four dated runs are in the calendar for the next twelve months, none inside the freeze.

Shows up in. `MSR-16 Initiatives through a design gate`, baseline **43%** [10].

Goes wrong when. The cadence is set to monthly out of enthusiasm. Four honest runs a year beat twelve that nobody reads.

Baselines and health

Step 11.8 — Cut a baseline when a decision needs a fixed reference, and not otherwise

Produce. A `04 Baselines` row, or a recorded reason not to cut one.

You need. `04 Baselines` from step 10.1, and the current record counts.

Where it lands. `04 Baselines` — `Baseline ref`, `Date`, `Label`, `Scope`, `Record count`, `Approved by`, `Decision ref`, `Supersedes`.

Cut a baseline when, and only when: a transition architecture is declared complete — `053` holds `TA-1`, `TA-2` and `TA-3`, each with exit criteria [11]; an increment lands, and `066` names six with the months they land in; a re-architecting change is approved; or an external party is about to be given the model.

Do **not** cut one quarterly by habit, at the end of a financial period, because the repository changed a lot, or because somebody asked for a snapshot. A baseline nobody will compare against is a file.

How.

1. Cut against `053`'s exit criteria. `TA-1 Governed and supportable` names five; test them before you cut, not after.
2. Record `Record count` at the cut. Today it is 842 records and 429 relationship instances [5].
3. Name the `Approved by` as a person and the `Decision ref` as an `060` row.
4. Fill `Supersedes`. A baseline chain with a hole in it cannot be walked backwards.
5. Re-baseline after a re-architecting change, never before. Baselining a model you are about to restructure wastes the only fixed point you had.

Signed by. `GB-01 Architecture Board`, with `GB-02` where the baseline supports a funding decision.

Done when. Every `04 Baselines` row names the decision that needed it.

Shows up in. The comparison tool, which cannot run without one.

Goes wrong when. The baseline is re-cut to make the drift disappear. It does disappear, and so does the only evidence that the practice was working.

BEFORE YOU CUT THE NEXT BASELINE

Which decision needs a fixed reference, and who is taking it?

If you cut nothing for a year, what would you be unable to answer?

Step 11.9 — Read the three health reports as a standing item, with targets

Produce. A one-page repository health report, same three sources every quarter.

You need. `ZZ Checks`, `RPT13 Orphan Analysis` and `RPT06 Requirements Traceability`.

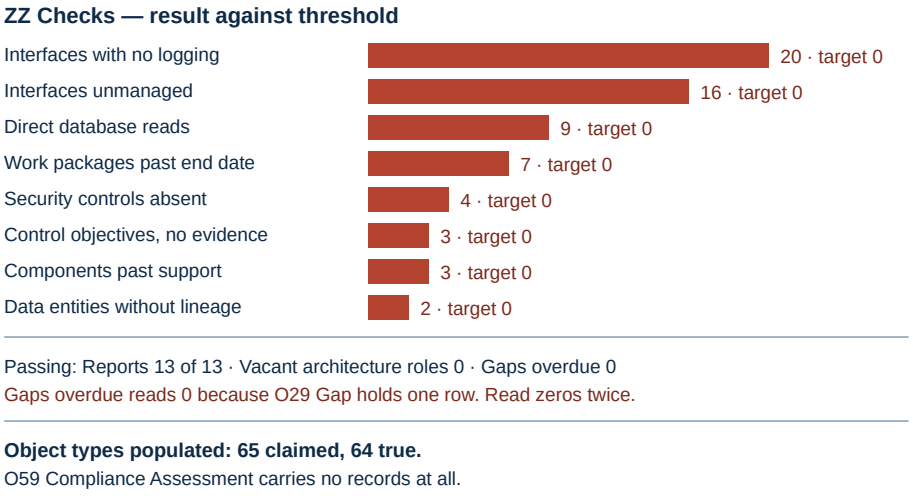


Figure 11.2 `ZZ Checks` as shipped: eight of its fifteen rows fail their threshold, three are counts with no threshold, and four pass — one of them falsely. *Object types populated* reads 65 against *Every type carries records*, and it is 64, because `O59 Compliance Assessment` is empty. Read every number in this repository against the data behind it.

Reading	Today	Target	A bad number means
`ZZ Checks` rows failing threshold	8 of 15 [5]	0	Nothing — until it stops falling. Then the checks are being ignored
Interfaces unmanaged	16	0	Every one is a design that no gate refused
Interfaces with no logging	20	0	An incident you cannot reconstruct, and `SEC-05` reads `Absent`
Components past support	3	0	`027` and `RPT10` disagree about one of them. Step 11.12
`RPT13 Orphan Analysis` rows	2 [6]	Not zero	Zero means nobody is adding objects. Rising means objects are added without relationships
`RPT06` requirements reaching a block	23 of 31 [7]	31	Eight requirements nothing will deliver
Ungoverned change, from the tool	Unknown	Falling	The practice is not in control of its own model

`RPT13` is the sheet to read first, and its own subtitle says so: *objects that nothing else references. Usually the most useful sheet in the repository* [6]. Two rows today, `A11 Supplier Portal` and `A12 Retail Analytics`, both reading *no business service maps to it*. One is `Invest`, one is `Replace`. An application nobody can name a business service for, which the portfolio has decided to invest in, is a conversation worth having before the money moves.

The subtlety is in the target. Zero orphans is not the goal and is usually a symptom: it means either that nothing new has been added since the last pass, or that someone has been attaching relationships to make the report look clean. The report closes on the right note — *an orphan is not always wrong; it is always worth a question* [6] — so the target is not a count, it is that every row has an answer written against it by the next quarter.

How.

1. Report the same seven readings every quarter. Changing the measure resets the trend.
2. Put last quarter's number beside this quarter's. A single number is a statement; two are a direction.
3. Re-run `ZZ Checks`' arithmetic rather than quoting it. *Object types populated* reads **65** and should read 64 while `059` is empty [5].
4. Say which reading you intend to move next quarter, and only one.
5. Close with the sheet's own closing line. It is better than anything you will write: *the thresholds are what a healthy estate would show; the results are what this one does; that distance is the work* [5].

Signed by. `GB-05 Architecture Function`, read at `GB-01`.

Done when. The report is one page, the same seven readings, and the trend column is populated.

Shows up in. `RPT01 Dashboard`, which carries most of these numbers already [12].

Goes wrong when. The report grows. A two-page health report is a document; a one-page one with seven numbers is a control.

Step 11.10 — Keep volume 1's cross-maps alive

Produce. A maintenance rule for the capability, process, role, data and application maps.

You need. Volume 1 appendix A, and the relationship sheets that hold the same joins.

Look it up. Volume 1, chapter 19, appendix A, *The cross-maps*, *Keeping these maps alive* — the rule, and the reason the maps decay faster than the catalogues.

Where it lands. The relationship sheets: `R01 Capability decomposition`, `R02 Value stream uses Capabilit`, `R03 Capability realised by Busi`, `R04 Business Service supported`, `R05 Application manages Data En`, `R21 Role performed by Actor`, `R22 Org Unit owns Capability`. Several tab names are truncated at Excel's 31 characters; spell them as the tab spells them.

> **MAP BY NAME, NEVER BY NUMBER**
>
> Volume 1's `P-01…P-12`, `C-01…C-20`, `R-01…R-33`, `D-01…D-20` and `A-01…A-25` are local IDs. They do **not** map one-to-one onto this repository's `Ref` scheme, and the collision is nastiest where it looks safest: volume 1's role IDs run `R-01` upward and this repository's relationship sheets run `R01` upward. Match on the name of the thing, every time.

How.

1. Attach each map to a trigger, not a calendar. The process-to-application map changes when an application changes, which `11.11` tells you about.
2. On any `O13 Business Capability` hierarchy change, re-check `R01`'s 90 rows and `R22`'s 12. A level-2 capability that moves parent invalidates both.
3. On any `O16 Business Process` change, re-check the process-to-role and process-to-data maps. A process with no role is a process nobody runs.
4. On a range review that restructures the product hierarchy, re-check `R05` and `O43 Master Data Rule`. The hierarchy is the join most retail maps hang from.
5. Date each map. An undated cross-map is quoted forever.

Signed by. The domain architect for the layer the map covers — `ROL-09` to `ROL-12` in `O11`.

Done when. Every map carries a date and a trigger, and no map is older than the last change to the objects it joins.

Shows up in. `RPT13`, where a broken map shows up as an orphan a quarter later.

Goes wrong when. The maps are maintained and the catalogues are not, or the reverse. They decay as a pair.

The retail triggers

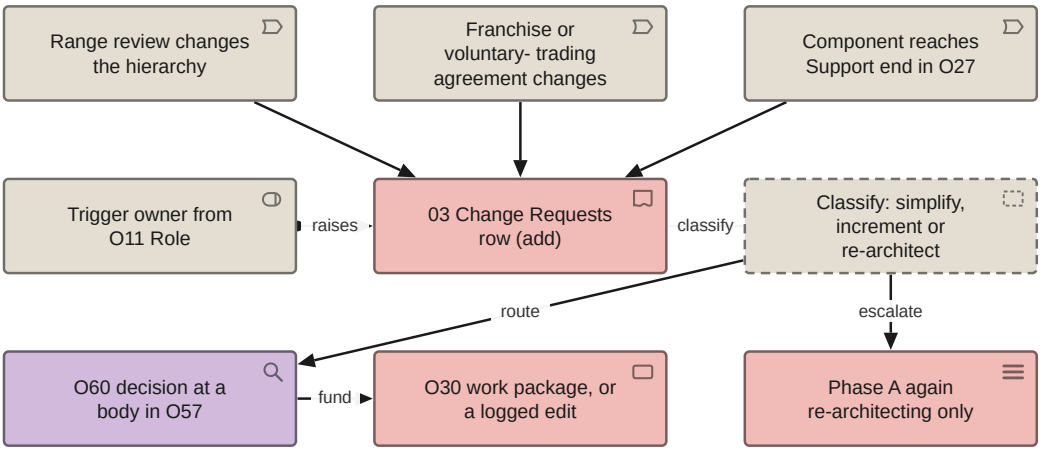


Figure 11.3 Three of the ten retail triggers drawn as what they are: events outside the architecture practice that force a change request, which is then classified and routed. Only a re-architecting classification re-enters the ADM at Phase A; the other two end in a work package or in a logged edit. The trigger is never the architect noticing.

Step 11.11 — Run a Phase H cycle on each retail trigger, without discretion

Produce. A trigger register with an owner, a route and a named repository consequence.

You need. This table, and somebody in each business area who will tell you.

Look it up. Volume 1, chapter 2, *The formats and lines of business, The ownership models, and what each does to the architecture* — why a franchise or voluntary-trading change is the most architecturally violent item on this list.

Trigger	What it changes in the repository	Route
A new format or banner	`O12 Location Type`, `O18 Product`, `EXT X02`; a new assortment grain in `X03`	Re-architecting if the format needs a new store type; incremental if not
A store opening or closing programme	`O12`, `R20 Application located at Loca`, `O49 Resilience Tier`	Incremental, but check `R20`'s 12 rows against the real estate
A range review that changes the hierarchy	`O20 Data Entity`, `O43 Master Data Rule`, `R05`, `O37 Data Attribute`	**Re-architecting.** The hierarchy is a grain, not an attribute
Acquisition of an independent group	Everything. `O09`, `O12`, `O13`, `O23`, `O20`, `EXT`	Phase A. Do not try to absorb it as incremental
A franchise or voluntary-trading agreement change	`O19 Contract`, `O12`, and whether the group has a price and an inventory position to master at all	**Re-architecting.** It changes what the master data means
A payment-scheme or card mandate	`O23 A02 POS`, `O27 PTC-11 Payment terminals`, `O41`, `O63 CO-02`	Incremental, with a hard external date
A new regulatory instrument commencing	`O63`, `O01 Driver`, `O06 Requirement`, `O59`	Incremental, unless it needs a new object type
A supplier trading-terms renegotiation	`O19`, `O20`, and the supplier-income derivation	Incremental; often a data-model change in disguise
A GS1 standard change, such as the 2D migration	`O32 Standard`, `O20`, `EXT X01 SKU / Article`, `A02 POS`	Re-architecting where the POS becomes a traceability capture point
A component reaching `Support end` in `O27`	`O27 Position`, `RPT10`, `O30`	Step 11.12

Form H-3 — trigger register

Feeds `03 Change Requests` when a trigger fires. One row per trigger, ten rows for the table above. `How you hear` is the column that decides whether this works. Signed by `ROL-08`.

Trigger	Owner (O11 Ref)	How you hear	Lead time	Route
New regulatory instrument	ROL-07	Example, delete me		Incremental

How.

1. Name an owner from `O11 Role` for every trigger. Eighteen roles, ten triggers; none of them is the architect.
2. Write `How you hear` honestly. If the answer is *somebody mentions it*, the trigger will fire without you.
3. Get on the distribution list for the range review calendar and the new-store pipeline. Those two produce most of the volume.
4. Fix the lead time per trigger and plan to it. A card mandate has a hard external date and no negotiation; a range review has a slot in the calendar.
5. Record each firing as a `03 Change Requests` row with the classification already set by the table.

Signed by. `ROL-08 Chief Enterprise Architect`, with each trigger's owner acknowledging the route.

Done when. Every trigger has an owner, a route and a named way you find out.

Shows up in. `O61` and `O29 Gap`, which should both grow when a trigger fires and neither of which has grown.

Goes wrong when. The architect hears about the acquisition from the trade press. By then the integration approach has been chosen by whoever was in the room.

Step 11.12 — Answer a support expiry as a dated change, not a renewal conversation

Produce. A change request per component inside its support horizon.

You need. `O27 Physical Technology Compone` and `RPT10 Technology Currency`.

Look it up. Volume 1, chapter 12, *The application landscape*, *The legacy question* — what a component past support actually costs a retailer, and why the answer is rarely the renewal fee.

Where it lands. `O27 Support end` and `Position`; a `03 Change Requests` row per component; an `O30` package where the answer is replacement.

`O27` holds twelve components. Three read `Position` = `Past support` [13], and two of those are blunt: `PTC-06 Warehouse control systems` at `2026-02` and `PTC-12 Data platform cluster` at `2026-08`. The third needs a second look.

> **`O27` AND ITS OWN `SUPPORT END` COLUMN DISAGREE**
>
> `PTC-01 Store server estate` reads `Position` = `Past support` with `Support end` = `2027-03`, which has not happened yet [13]. Either the position is wrong or the date is. `RPT10` computes off the same sheet, so it repeats the error. Check every `Position` against its `Support end` before you quote the count of three — and note that `PTC-07` at `2027-09` and `PTC-09` at `2027-11` are both inside a twelve-month horizon and read `Supported`.

How.

1. Sort `O27` by `Support end` and draw a line twelve months out. Everything above it is this year's work.
2. Raise one change request per component, with the vendor date as the deadline. The date is external and not yours to move.
3. Classify. A like-for-like upgrade is simplification; a replacement is incremental; a replacement that changes where the component sits is re-architecting.
4. Check the location. `PTC-05 Core database cluster` is recorded at `Convenience Format` and `PTC-12 Data platform cluster` at `Metro Store Format` [13]. A core database in a convenience store is a data error or a finding, and either way it must be settled before the renewal.
5. Feed `OBJ-07 No component running past vendor support` and `MSR-07 Components within vendor support`, baseline **50%** [10].

Signed by. `ROL-12 Domain Architect, Technology`, with `GB-02` where money is needed.

Done when. No `O27` row has a `Support end` within twelve months and no change request against it.

Shows up in. `RPT10 Technology Currency` and `ZZ Checks` *Platform components past support*.

Goes wrong when. The expiry is handled by procurement as a renewal. The contract renews, the architecture question is never asked, and the component is still there at the next expiry.

Step 11.13 — Put six numbers and two decisions in front of the board once a year

Produce. A one-page annual architecture review.

You need. `004 Measure`'s sixteen measures with their baselines, and the year's `060` decisions.

Six numbers, all of them already in the file with a baseline to move from:

Number	`004` ref	Baseline
Managed interfaces	`MSR-08`	28%
Components within vendor support	`MSR-07`	50%
Obligations with evidence	`MSR-01`	38%
Architecture compliance rate	`MSR-15`	21%
Initiatives through a design gate	`MSR-16`	43%
Ungoverned change count	none — add it	this year's first run

And two decisions, which are the only two a board can usefully take about architecture: **re-baseline or not**, and **which dispensations expire without renewal**. Both are answerable from sheets. Neither needs a slide.

How.

1. Report against the baseline in `004`, not against last year's slide.
2. Add the ungoverned count as a seventeenth measure, with `Unit` = `count`, `Domain` = `Governance`, `Cadence` = `Quarterly`.
3. Name the one reading that got worse. A review with six improvements in it will not be believed.
4. Put the two decisions on one page with a recommendation and the consequence of refusing it.
5. Record both in `060` with a `Date`, which is the column you added in step 11.3.

Signed by. `GB-08 Board of Directors`, added to `057` at step 10.10, with `GB-02` on anything funded.

Done when. Both decisions are `060` rows with dates, and the six numbers have a prior-year column.

Shows up in. `RPT01 Dashboard`, and in next year's review as the prior-year column.

Goes wrong when. The annual review becomes a programme update. The board already gets those. What it does not get is a number that says whether the model is true.

What Phase H actually produces

Nothing visible. That is the difficulty with it. A Phase H that is working produces a repository that is still true eighteen months after it was built, an ungoverned count in single figures, and a set of triggers that reach the architect before they reach production. None of that photographs well, and none of it appears on a programme plan.

A Phase H that is not working produces exactly what you were handed: 842 records with no version, no change log and no approval trail; seven work packages past their end date; four contracts asserting `Non-compliant` with no assessment behind them; and a `ZZ Checks` sheet honest enough to report eight failures and still claim all 65 object types carry records.

There is one more thing worth saying about the retail version of this phase. The triggers in step 11.11 arrive on the trading cycle, which means they arrive in clusters: the range review calendar, the new-store pipeline and the promotional calendar all fire in the same few weeks, and then almost nothing moves for the eight weeks of the freeze. A practice that plans for a steady arrival rate will be swamped twice a year and idle once. Plan for the shape, not the average.

The practice is measured by one number, and it is not in the file yet. Add it.

PHASE H — A REVIEWER'S QUESTIONS

What is your ungoverned-change count, and is it rising or falling?

Which of the ten retail triggers fired in the last year without reaching you first?

When was your last baseline cut, which decision needed it, and who approved it?

If the board asked today whether the repository is still true, what would you show them?

THE CHAPTER IN TEN LINES

- Phase H never ends, and it is the only phase that yields a number saying whether the practice works.
- The pipeline does not exist in the shipped file. `O60` has no `Date`, `O61` has five rows, and there is no change log at all.
- Every field change carries a `Change Ref`, or the comparison tool classifies it as ungoverned. No appeal.
- Log one row per field, not per record or per session, and log the old value before you overwrite it.
- Classify by three questions about sheets: an attribute only is simplification, object rows is incremental, a relationship pattern is re-architecting. Never classify by cost.
- There are no export sheets in this repository. Both comparison extracts are produced by hand, from a dated baseline and a dated live copy.
- A rising ungoverned count is a failure of the practice, not of the estate. Fix the route, not the row.
- Cut a baseline when a decision needs a fixed reference — a transition complete, an increment landed, a re-architecting approved, a model leaving the building. Never quarterly by habit.
- Seven standing readings: `ZZ Checks` failures, unmanaged interfaces, unlogged interfaces, components past support, `RPT13` orphans, `RPT06` traced requirements, ungoverned change.
- Volume 1's appendix A IDs do not map onto this repository's `Ref` scheme. Match by name, always — volume 1's role IDs and this file's relationship sheets both start `R`.
- Ten retail triggers force a cycle. A range review that moves the hierarchy and a franchise agreement change are both re-architecting, whatever they cost.
- `O27` disagrees with itself: `PTC-01` reads `Past support` with a `Support end` of `2027-03`. `RPT10` repeats the error because it computes off the same column.
- The annual board item is six numbers with baselines from `O04` and two decisions: re-baseline or not, and which dispensations expire.

SOURCES FOR THIS CHAPTER

- **1.** Retail_TOGAF_Metamodel.xlsx, sheet O60 Governance Decision
- **2.** Retail_TOGAF_Metamodel.xlsx, sheet O61 Governance Change Request
- **3.** Retail_TOGAF_Metamodel.xlsx, sheet O57 Governance Body
- **4.** Retail_TOGAF_Metamodel.xlsx, sheets O33 Architecture Contract and O55 Building Block
- **5.** Retail_TOGAF_Metamodel.xlsx, sheet ZZ Checks
- **6.** Retail_TOGAF_Metamodel.xlsx, sheet RPT13 Orphan Analysis
- **7.** Retail_TOGAF_Metamodel.xlsx, sheets RPT06 Requirements Traceability and R09 Requirement satisfied by Bu
- **8.** Retail_TOGAF_Metamodel.xlsx, sheets O29 Gap and RPT07 Gap Analysis, 1 record
- **9.** Retail_TOGAF_Metamodel.xlsx, sheet O62 Delivery Risk
- **10.** Retail_TOGAF_Metamodel.xlsx, sheet O04 Measure
- **11.** Retail_TOGAF_Metamodel.xlsx, sheets O53 Transition Architecture and O66 Architecture Increment
- **12.** Retail_TOGAF_Metamodel.xlsx, sheet RPT01 Dashboard
- **13.** Retail_TOGAF_Metamodel.xlsx, sheets O27 Physical Technology Compone and RPT10 Technology Currency
- **14.** GS1 US, Sunrise 2027 topic page (a voluntary industry capability ambition, with no stated day or month) https://www.gs1us.org/industries-and-insights/by-topic/sunrise-2027
- **15.** The Open Group, The TOGAF Standard, 10th Edition — Architecture Development Method, Phase H: Architecture Change Management https://pubs.opengroup.org/togaf-standard/adm/

CHAPTER 12

Requirements Management

A requirement exists to be tested at a gate. Anything that cannot be is a preference in a requirement's clothing.

BEFORE YOU START

Requirements Management runs through every phase and belongs to none of them. This chapter is the one to read when somebody asks why a thing is in the plan.

- **Chapter 15, *Regulation, risk and what the regulator inspects*** — *The Consumer Protection Act, which binds the systems most directly*; *Tax, credit, measurement and the card estate*; *Food labelling and safety*. The regulation itself lives there; this chapter turns it into rows.
- **Chapter 16, *Modelling retail in ArchiMate*** — *A regulation, from driver to control*: the chain the motivation extension draws.
- **Chapter 9, *The conceptual data model*** — *Price is an entity, not an attribute*: the argument behind half the requirements in this register.

Halverson Stores Group is an invention. Its data is real within this workbook and nowhere else.

What you are building, continuously

Step	Produce	Lands in
12.1	An honest reading of the register you inherited	`O06 Requirement`
12.2	An acceptance criterion per requirement	`O06` `Acceptance criterion` (add)
12.3	A traced requirement, and the two breaks repaired	`R08`, `R10`, `R11`
12.4	The necessity test applied, and preferences named	`O06` `Status`
12.5	The governing principle attached	`O05 Principle`
12.6	Principles that govern nothing, retired	`O05` `Status`
12.7	A South African instrument turned into a requirement	`O63`, `O06`, `R16`
12.8	Six generic obligations replaced at instrument grain	`O63 Control Objective`
12.9	A requirement bound to a block and a package	`R09`, `R13`
12.10	The honesty score: traced against tested	`RPT06`, `RPT09`, `O59`
12.11	A mid-phase requirement assessed, or refused in writing	`03 Change Requests` (add)
12.12	A register with the dead requirements out of it	`O06` `Status`

THE ONE TEST

A requirement exists to be tested at a gate. If you cannot say which gate, which criterion and what evidence, you are holding a preference. The repository makes this painfully checkable: `O06` carries 31 requirements, `RPT06` says 23 of them reach a building block, and `O59 Compliance Assessment` holds **zero** verdicts [1][2][4]. Traced 23. Tested 0.

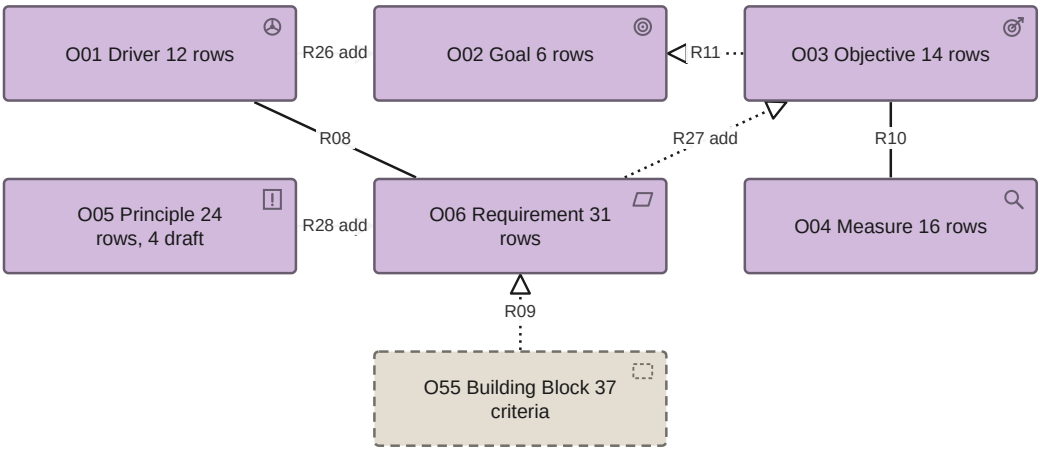


Figure 12.1 The motivation spine as ArchiMate 4 draws it, with the repository's two breaks shown. `R11` and `R10` are complete — every goal reaches an objective and every objective reaches a measure. `R08` joins every requirement to a driver. Nothing joins a driver to a goal and nothing joins a requirement to an objective, so the chain has no middle. `R26` and `R27` are additions, not tabs you will find. `004 Measure` draws as an Assessment because ArchiMate 4 has no measure element.

The register

Step 12.1 — Read the register you inherited before you add anything to it

Produce. A marked-up reading of `006`'s 31 rows, with the generated patterns named.

You need. `006 Requirement` and `R08 Requirement derives from Dr`.

Where it lands.

> **`006 Requirement`**
>
> Real columns: `Ref`, `Requirement`, `Statement`, `Category`, `Priority`, `Driver`, `Status`.
>
> `Category` takes `Regulatory`, `Data`, `Integration`, `Business`, `Technology`. `Priority` takes `Must`, `Should`, `Could`. `Status` takes `Approved` or `Proposed`. There is **no acceptance criterion column** and **no owner column**.

The shape is clean and the content is generated. `REQ-001` to `REQ-006` are the six regulatory obligations, `Must`, `Approved`. `REQ-007` to `REQ-014` are the eight *mastered once* data requirements. `REQ-015` to `REQ-020` are integration, `Should`. `REQ-021` to `REQ-026` are business, … *measurable*, all `Proposed`. `REQ-027` to `REQ-031` are technology adoptions, `Could`, all `Proposed` [1].

Two things to notice before you touch it. First, the `Driver` column duplicates `R08`, which holds the same 31 pairings — one fact in two places, and no check that they agree. Second, the statements are templated: *X is mastered in one system and consumed elsewhere through a published interface* reads identically for SKU, price and planogram, which are three very different problems.

How.

1. Read all 31 rows and mark each as keep, rewrite or retire. Expect to rewrite most of the templated ones.
2. Decide which of `O06 Driver` and `R08` is authoritative. Keep `R08`, because a relationship sheet can carry more than one driver per requirement.
3. Add `Owner` (**add**) and `Acceptance criterion` (**add**). Those two absences do more damage than anything else in this sheet.
4. Do not renumber. `REQ-0nn` references appear in `R08`, `R09` and `RPT06`.
5. Raise the whole reading as one `03 Change Requests` row, classified simplification.

Signed by. `ROL-08 Chief Enterprise Architect`.

Done when. Every row is marked, and the two new columns exist.

Shows up in. `RPT06 Requirements Traceability`, which reads `O06` plus `R09` and will not change until you change those.

Goes wrong when. You add twenty of your own requirements to a register you have not read. Then the duplicates are yours.

Step 12.2 — Write the acceptance criterion, or do not call it a requirement

Produce. One testable criterion per `Must` and `Should` requirement.

You need. `O06` with the new column, and `O55 Building Block`'s 37 `Acceptance criteria` strings as the source where one already exists.

Where it lands. `O06 Acceptance criterion` (**add**) and `Tested at` (**add**), holding an `O58` gate reference.

The criterion answers one question: what will be shown at the gate? *Price is mastered in one system* is an aspiration. *Every price change in the last 30 days resolves to one system of record, and the till, the shelf label and the web all read it* is a criterion. Both are about the same requirement. Only the second can be refused.

Form R-1 — requirement acceptance criteria

Feeds `006 Requirement`, columns `Ref` and `Priority` as they stand plus two added columns. One row per `Must` and `Should` requirement — 26 of the 31. Signed by the requirement's owner and `GB-01`.

Ref	Priority	Acceptance criterion (add)	Tested at (add)	Owner (add)
REQ-011	Must	Example, delete me	GT-2	

How.

1. Start from `055`. Where a block's criterion is already testable, reuse its wording rather than inventing a second version.
2. Write the criterion as something a person could fail. *Measured and owned* cannot fail; *a named individual appears in* `020 Owner` can.
3. Set `Tested at` to `GT-1` for design properties and `GT-2` for behaviour. A requirement tested nowhere has no gate and will not survive.
4. Leave the five `Could` rows without criteria, and mark them so. A `Could` with a criterion is a `Should` that has not been argued.
5. Where you cannot write a criterion, change the `Status` to `Proposed` and say why in the `Statement`. That is honest and reversible.

Signed by. `GB-01 Architecture Board`.

Done when. All 26 `Must` and `Should` rows have a criterion and a gate.

Shows up in. `059`, eventually, as one verdict per criterion.

Goes wrong when. The criterion is written as the design. *The integration platform is used* is a solution with a tick box; *no unmanaged interface to or from this application* is a criterion, and `055` already phrases ten of them that way.

The motivation spine

Step 12.3 — Trace the requirement up the spine, and repair the two breaks

Produce. A complete trace from requirement to goal, and two new relationship sheets.

You need. `R08 Requirement derives from Dr`, `R11 Goal realised by Objective`, `R10 Objective measured by Measu`, and `O02`, `O03`, `O04`.

Look it up. Volume 1, chapter 16, *Modelling retail in ArchiMate*, *A regulation, from driver to control* — the same chain in notation, which is what the figure above draws.

Where it lands. Two sheets that do not exist. Add them on the file's own convention, headers row 4, named after their meaning like every other `R` sheet:

- `R26 Driver addressed by Goal` (**add**) — `Driver`, `Driver name`, `Goal`, `Goal name`.
- `R27 Requirement realises Objective` (**add**) — `Requirement`, `Requirement name`, `Objective`, `Objective name`.

The spine as shipped runs in two disconnected pieces. Upwards from the goal it is complete: `R11` joins 6 goals to 14 objectives, and `R10` joins all 14 objectives to 16 measures with the unit carried through [6][7]. Every objective is measurable, which is better than most real repositories manage.

Downwards it stops. `R08` joins all 31 requirements to drivers [5]. Nothing joins a driver to a goal. Nothing joins a requirement to an objective. So you can say *this requirement exists because of that pressure* and you can say *this goal will be measured that way*, and you cannot get from one to the other. The two sheets above close it, and until they exist no requirement in this file can be shown to serve a stated business objective.

How.

1. Build `R26` first. Twelve drivers against six goals; expect a driver to address more than one. `DRV-01` to `DRV-06` are all regulatory and all point at `GOL-01 Protect the licence to operate` [8].
2. Build `R27` next, one row per requirement per objective. A requirement with no row here is step 12.4's problem.
3. Check the chain end to end on three requirements by hand before you trust any of it.
4. Where a requirement reaches a driver but no objective, do not invent an objective. `O03` has 14 and they are serviceable; the requirement is probably the thing that is wrong.
5. Keep the unit. `R10` carries `Unit` already, so a measure can be read without opening `O04`.

Signed by. `ROL-08 Chief Enterprise Architect`, with the goal owner in `O02` for each chain.

Done when. Every `Must` requirement reaches a goal in two hops, through an objective.

Shows up in. Nothing computes over the spine. `RPT06` stops at the driver, which is exactly the blind spot this step removes.

Goes wrong when. The trace is drawn upward from the requirement to justify it. Trace down from the goal and the weak requirements are obvious.

THE REQUIREMENT THAT TRACES TO NOTHING

Pick one `Must` requirement. Which objective in `O03` does it realise, and which measure proves it?

If the answer is "none", who asked for it?

Step 12.4 — Apply the necessity test, and name the preferences

Produce. A shorter register, with the preferences marked rather than deleted.

You need. `R27` from step 12.3, and `O06` `Status`.

The test, in one sentence: **a requirement that traces to no objective is somebody's preference.** Not wrong, not malicious, usually sensible — but not a requirement, because nothing in the business has said it matters, and nothing will measure whether it was met.

Three outcomes, and all three are legitimate:

Finding	What to do
Traces to an objective	Keep. Set `Status` = `Approved` once a criterion exists
Traces to nothing, but somebody senior wants it	Keep as `Proposed`, and raise the missing objective at `GB-02`. If the objective is refused, the requirement goes
Traces to nothing and nobody will defend it	Retire. Step 12.12

The eleven `Proposed` rows in `O06` are the obvious place to start — all six … *measurable* business requirements and all five technology adoptions [1]. `REQ-029 Adopt Headless commerce` is a good test case: it is a `Could`, it traces to `DRV-09 Legacy estate`, and `RPT06` says it reaches no building block [2]. An architectural preference with a plausible driver and nothing behind it.

How.

1. Run the test on all 31. Write the outcome in a working column, not in `Status` yet.
2. For each failure, ask the sponsor one question: which objective? Accept silence as an answer.
3. Where an objective is genuinely missing, add it to `O03` with an owner and a horizon, and a measure in `O04` and `R10`. Three sheets, one act.

4. Never retire a requirement in the same meeting it fails the test. Mark it, then retire it a quarter later if nobody has defended it.
5. Count the preferences. A register that is a third preferences is normal; one that is two-thirds is a wish list with a reference scheme.

Signed by. `GB-02 Investment Committee`, which funds the consequences.

Done when. Every row carries an outcome, and no `Must` row is a preference.

Shows up in. `RPT06`, where the count of requirements falls and the proportion reaching a block rises. Both movements are good.

Goes wrong when. The test is applied to other people's requirements only.

Step 12.5 — Attach the governing principle, and ask whether it does any work

Produce. A principle reference on every requirement, and an honest answer about `O05`.

You need. `O05 Principle` — 24 records, `Ref`, `Principle`, `Category`, `Statement`, `Owner`, `Status`. There is **no `Tier` column** and **no relationship sheet joining a principle to anything**.

Where it lands. `O06 Governing principle` (**add**), holding a `PRN-` reference. Or, better, `R28 Requirement governed by Principle` (**add**) — `Requirement`, `Requirement name`, `Principle`, `Principle name` — because a requirement often sits under two.

`O05` is the best-written sheet in the repository and currently the least useful, because nothing joins to it. Twenty-four principles across five categories, every one owned by the Architecture Board, four of them `Draft`: `PRN-05 Common Use Applications`, `PRN-10 Data is Shared`, `PRN-15 Requirements-Based Change` and `PRN-20 Least Privilege` [9].

> **THE DRAFT THAT MATTERS MOST**
>
> `PRN-15 Requirements-Based Change` reads *change is driven by stated requirements* and its `Status` is `Draft`. That is the principle this entire chapter depends on, and it has not been approved. `PRN-10 Data is Shared` is also `Draft`, and the eight *mastered once* requirements all rest on it. Approve or withdraw both before the next design gate.

How.

1. Attach a principle to each requirement. If none of the 24 fits, the requirement is probably outside the architecture's remit.
2. Where the governing principle is `Draft`, flag the requirement as unenforceable until it is approved.
3. Test the join in the other direction: list the requirements under each principle. That list is step 12.6.
4. Leave `Category` alone. It is a grouping, not a tier, and inventing a tier column invites an argument that `O05` has so far avoided.

5. Record `R28` in the index and in the change log, so the next person knows it was added rather than shipped.

Signed by. `GB-01 Architecture Board`, which owns all 24.

Done when. Every `Must` and `Should` requirement names a principle, and no `Must` rests only on a `Draft` one.

Shows up in. Nothing. No report reads `O05`, which is why the sheet needs a join to stay honest.

Goes wrong when. The principle is attached after the design. Then the principle is being used to justify, not to govern.

Step 12.6 — Retire the principles that govern nothing

Produce. A shorter, enforced principle set.

You need. `R28` from step 12.5.

Twenty-four principles is at the upper end of useful. The test is simple and uncomfortable: list the requirements, standards and gate refusals that each principle has governed in the last year. A principle with none has not been used, and an unused principle weakens the ones that are used, because it teaches the organisation that the list is decorative.

How.

1. Count the `R28` rows per principle. Zero is a candidate for retirement.
2. Cross-check against `O32 Standard`'s 16 standards and `O41 Security Control`'s eight controls. A principle with no requirement but two standards is working fine.
3. Check the gate record. `PRN-19 Security is Built In` reads *security is a design input, not a review gate* — and `SEC-02 Design review` reads `Position = In place` with no `O59` row behind it [10]. The principle and the control disagree about what happens.
4. Retire by setting `Status`, never by deleting the row. `Retired` is a value the column can carry and a deletion is a lost argument.
5. Review the set annually, at the same sitting as the annual architecture review in step 11.13.

Signed by. `GB-01 Architecture Board`.

Done when. Every `Approved` principle governs at least one requirement, standard or control.

Shows up in. `RPT12 Standards Conformance`, indirectly: a principle that governs a standard shows up as a conformance verdict.

Goes wrong when. The set grows every year and shrinks never. Twenty-four becomes forty, and the architect is the only person who has read them.

Regulation as requirements

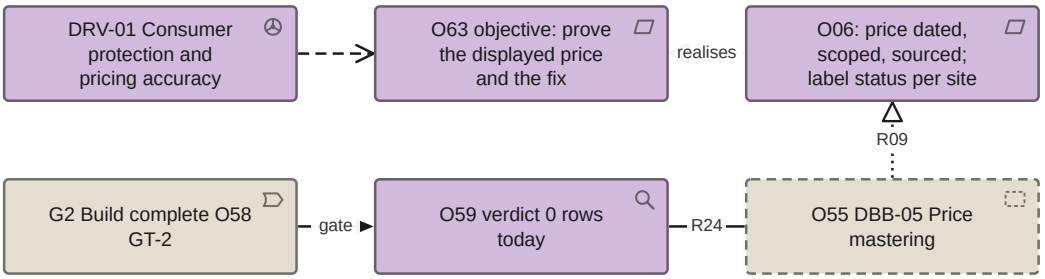


Figure 12.2 One South African instrument, end to end. CPA section 23(6)(b) is a driver in `001`; the control objective and the architecture requirement are both Requirements, with the second realising the first; a data building block realises the requirement; and the verdict is taken at `G2`. Three of six control objectives in `063` have no working mechanism, and `059` holds no verdicts at all — so this chain is drawn as it should be, not as the file has it.

Step 12.7 — Turn an instrument into a requirement, not a compliance memo

Produce. One `063` obligation, one `006` requirement and one criterion, per instrument.

You need. `063 Control Objective`, `R16 Control Objective met by Co`, `001 Driver` and volume 1's regulation chapter.

Look it up. Volume 1, chapter 15, *Regulation, risk and what the regulator inspects, Section 23, and why a stale shelf label is a legal exposure* — the two-prices rule, the narrow defence in section 23(9), and why both limbs are data.

Where it lands. `063` for the obligation and its control objective; `006` for the architecture requirement; `R16` for the control that evidences it; `059` for the verdict.

Work it through once, properly, and the pattern holds for the rest. **CPA section 23(6)(b)**: where more than one price is concurrently displayed, the supplier may not charge more than the lower or lowest of them, and section 23(9) relieves an inadvertent and obvious error only once the display has been corrected **and** reasonable steps taken to inform affected consumers [11].

Layer	Row
Driver	`001` : `DRV-01 Consumer protection and pricing accuracy` , `Regulatory` , `External` , `High` [8]
Obligation	`063` : *CPA s 23(6)(b) and s 23(9)*, `Source` = *Act 68 of 2008*
Control objective	*Prove what price was displayed, where and when, and prove both limbs of a correction*
Requirement	`006` : price is dated, scoped and sourced; label print status is held per article per site; a price-error record carries the correction and the notification
Criterion	*For any article, store and timestamp in the last 90 days, the displayed price, its source and any correction and notification can be produced from the repository's named systems*
Tested at	`GT-2` , with `059` `Evidence` naming the extract

Form R-2 — obligation to requirement

Feeds `006 Requirement` , in sheet order, excluding `Ref` and `Status` which you set in Excel. One row per instrument and section. A South African multi-format group needs twenty-five to thirty. Signed by `ROL-07 Chief Compliance Officer` with `GB-01` .

Requirement	Statement	Category	Priority	Driver
Displayed price provable	Example, delete me	Regulatory	Must	DRV-01

How.

1. Write the obligation at section grain. *Consumer protection* is a topic; `s 23(6)(b)` is an obligation with a testable consequence.
2. Write the requirement as a property of the architecture, never as a project. The project changes; the property does not.

3. Name the control in `R16` that produces the evidence, and if there is none say `No mechanism`.
4. Set `Priority` = `Must` on everything regulatory, and mean it. `O06` already does this for all six regulatory rows [1].
5. Keep the instrument reference in `Source`. A requirement whose statute cannot be found is a requirement somebody will argue with.

Signed by. `ROL-07 Chief Compliance Officer` on interpretation; `GB-01` on the criterion.

Done when. Every regulatory requirement names an instrument, a control and a gate.

Shows up in. `RPT09 Regulatory Coverage`, where `Evidence` should become derived rather than asserted.

Goes wrong when. The compliance function writes a memo and the architecture function cites it. A memo has no `Ref` and nothing joins to it.

Step 12.8 — Replace the six generic obligations at instrument grain

Produce. An `O63` that looks like a South African retailer's obligation register.

You need. `O63`'s six rows, and volume 1's regulation chapter for the instruments.

Look it up. Volume 1, chapter 15, *Regulation, risk and what the regulator inspects*, *Tax, credit, measurement and the card estate* — the VAT thresholds, the till-slip particulars, legal metrology and the PCI DSS dates. And *Food labelling and safety*, and *The Single Exit Price, and the year there were two of them*.

Where it lands. `O63 Control Objective` — `Ref`, `Obligation`, `Source`, `Control objective`, `Evidence`.

`O63` holds six rows, each a topic rather than an instrument, and each with a `Control objective` reading *control that evidences …* — a placeholder, not a control objective [3]. These are the instrument-grain rows a South African retailer needs, with the requirement each produces:

Instrument	The requirement it produces
CPA s 23(6)(b), s 23(9)	Displayed price provable per article, store and timestamp, with both limbs of a correction
CPA s 56	A legal basis on every return, its own anchor date, and a link to the original sale line
POPIA	Lawful basis per processing purpose; consent per channel; classification travelling with the data; personal data masked outside production
Pharmacy and medicines	Dispensing records retained; the responsible pharmacist on the day; the interaction-checking software version in use
Food labelling	Label attributes held at article creation and agreeing with the printed label
Legal metrology	An instrument register per site, with verification dates and a re-verification trigger on relocation
VAT and till-slip content	A governed VAT code per article with a change history; VAT per line; a compliant full invoice above the SARS threshold
PCI DSS	The card number is not a key, a join column or a search field; tender detail separable from sales lines

How.

1. Write one row per instrument and section. Expect `O63` to go from six rows to twenty-five or more.
2. Replace every *control that evidences …* placeholder with a sentence naming what is produced and from where.
3. Derive `Evidence` from `R16` rather than typing it. Three of six currently read `Partial`, `In breach` or `No mechanism` [3].
4. Mark PCI DSS as contractual, not statutory. It arrives through acquirer contracts, which means there is no regulator to appeal to and no national derogation [12].
5. Mirror each new obligation into `O01 Driver` so the driver and the obligation stay consistent. `DRV-01` to `DRV-06` are the six that exist [8].

Signed by. `ROL-07 Chief Compliance Officer`.

Done when. No `O63` row is a topic, and every row names what produces its evidence.

Shows up in. `RPT09`, which grows past six rows and gets harder to read. That is the correct outcome: six obligations is not a regulatory universe for a South African retailer.

Goes wrong when. The register is written once and never re-read. An instrument commencing is one of the ten Phase H triggers in step 11.11.

THE INSTRUMENT YOU HAVE NOT TURNED INTO A REQUIREMENT

Which instrument from volume 1 chapter 15 has no row in your `063` at all?

For the price-display obligation, what would you produce at a gate, and from which system?

Into delivery, and back out

Step 12.9 — Bind the requirement to a building block, and re-argue R09

Produce. An `R09` that pairs requirements with blocks that could actually satisfy them.

You need. `R09 Requirement satisfied by Bu` (31 instances), `O55` 's 37 blocks and `R13` .

Where it lands. `R09` — `Requirement` , `Requirement name` , `Building block` , `Block name` , `Traced` .

> **`R09` PAIRS BY POSITION, AND THE PAIRINGS ARE NONSENSE**
>
> `R09` walks `REQ-001` to `REQ-031` against `O55` in `Ref` order. So `REQ-002 Evidence PCI DSS` is satisfied by `BBB-02 Pricing & Promotion capability uplift` ; `REQ-011 Price mastered once` is satisfied by `DBB-01 SKU / Article mastering` ; and `REQ-031 Adopt PCI-scoped payment tokenisation` is satisfied by `TBB-01 Edge at store` [13]. Meanwhile `DBB-05 Price mastering` exists and is paired with `REQ-015 Merchandising System integration managed` . The price requirement and the price block are both in the file and are not joined to each other. Re-argue all 31 rows. `RPT06` computes straight off this sheet, so until you do, the traceability report is a report about the generator.

How.

1. For each requirement, list the blocks that could satisfy it. Several requirements need more than one; `R09` currently allows that and does not use it.
2. Delete the positional rows in one change request, not 31.
3. Set `Traced` from the join rather than typing it. The `Traced` column currently mirrors the same every-fourth-row pattern as `RPT06` 's `Reaches a block` .
4. Follow the block to its package through `R13` , and check that the package's `Approval` is not `Not started` while work is in flight — `WP-05` is [14].
5. Where no block can satisfy a `Must` requirement, that is a gap. `O29 Gap` holds one row [15]; this step should add several.

Signed by. `ROL-08 Chief Enterprise Architect`, with the domain architect for the block's domain.

Done when. No `R09` row pairs a requirement with a block that could not satisfy it, and every `Must` requirement reaches at least one block.

Shows up in. `RPT06 Requirements Traceability`, whose closing line is the whole point: *a requirement that reaches no building block is a requirement nothing will deliver* [2].

Goes wrong when. The pairing is tidied to make `RPT06` read better. The report then measures the tidying.

Step 12.10 — Read the gap between traced and tested as the honesty score

Produce. Two numbers, reported together, every quarter.

You need. `RPT06`, `O59` and `RPT09`.

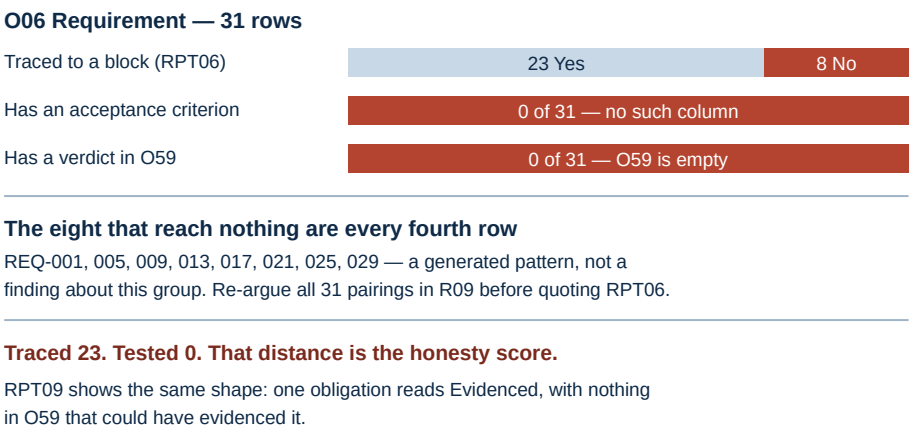


Figure 12.3 The honesty score, from the shipped file. Twenty-three of thirty-one requirements reach a building block in `RPT06`; none has an acceptance criterion, because `006` has no such column; none has a verdict, because `059 Compliance Assessment` holds no records. Tracing is drawing lines. Testing is asking for proof.

Traced means a line exists between a requirement and a block. Tested means somebody looked at the thing and said whether the criterion was met. The first is drawing; the second is proof. The distance between them is the only measure of this discipline that cannot be gamed from inside the repository.

Today: traced 23 of 31, tested 0 of 31 [2][4]. And `RPT09` shows the same shape from the other end — one obligation reads `Evidenced` while `059` holds nothing that could have evidenced it [3][4].

How.

1. Report both numbers in the same sentence, always. A traced count on its own is the number that flatters.

2. Report the eight untraced rows by name. They are `REQ-001`, `005`, `009`, `013`, `017`, `021`, `025` and `029` — every fourth row, which tells you the pattern is generated, not a finding about this group [2].
3. Set the target on tested, not traced. Traced is free.
4. Feed `MSR-15 Architecture compliance rate`, baseline 21% [16].
5. Publish the number even when it is zero. Especially when it is zero.

Signed by. `GB-05 Architecture Function`, read at `GB-01`.

Done when. Both numbers appear in the quarterly health report from step 11.9.

Shows up in. `RPT06` and `059` together. Neither alone says anything useful.

Goes wrong when. Tracing is reported as compliance. It is reported that way in most practices, which is why the number is worth having.

TRACED AGAINST TESTED

What are your two numbers today?

Which requirement would you most like not to be asked to evidence, and why?

Step 12.11 — Assess a requirement that arrives mid-phase, and refuse it in writing

Produce. A decision on the new requirement, with the reason recorded either way.

You need. `03 Change Requests` with `Type` = `New requirement`, and the current phase.

A requirement arriving mid-phase is normal, not a failure. Refusing it badly is the failure. The test is three questions, in this order: does it trace to an objective; does it change a relationship pattern; and does it change a criterion a gate has already passed. The answers decide the route.

Answers	Route
Traces, no pattern change, no passed criterion	Accept into the current phase. Add to `006`, `R27`, `R09`
Traces, no pattern change, changes a passed criterion	Accept into the next increment. `066` names six
Traces, changes a relationship pattern	Refuse for this phase. Re-architecting, back to Phase A
Does not trace	Refuse. Ask for the objective first

Form R-3 — mid-phase requirement intake

Feeds `03 Change Requests` (**add**) with `Type` = `New requirement`. One row per arrival. Signed by `ROL-08` with the receiving body.

Change ref	Raised by	Against	Impact	Decision body
CR-0nn		O06	Example, delete me	GB-01

How.

1. Take it in writing. A requirement delivered verbally has no `Raised by` and no date.
2. Run the three questions in order and stop at the first that routes it.
3. Refuse in writing, with the reason and the route. *Not now, and here is when* is a refusal somebody can work with.
4. Record every refusal as an `O60` row with a `Date`. An unrecorded refusal is raised again in six weeks.
5. Count the arrivals per phase. A high rate in Phase C usually means Phase B was closed too early.

Signed by. The body the route names.

Done when. Every arrival has a dated decision and a recorded reason.

Shows up in. `O60` and `03 Change Requests`.

Goes wrong when. Mid-phase requirements are absorbed silently to avoid an argument. The scope then grows without a decision anywhere, and the gate at the end tests criteria nobody agreed.

Step 12.12 — Retire requirements, so the register stays worth reading

Produce. A register where every row is live.

You need. `O06` `Status`, and the outcomes from step 12.4.

Where it lands. `006` `Status`, extended to take `Retired` and `Superseded` alongside `Approved` and `Proposed`. Plus `Retired on` (**add**) and `Retired because` (**add**).

Four reasons to retire, and only four: the objective it served was dropped; it was delivered and the criterion was met at a gate; it was superseded by a better statement of the same thing; or it failed the necessity test and nobody defended it for a quarter.

How.

1. Retire by status, never by deletion. `R08`, `R09` and `RPT06` all reference `REQ-` values and a deleted row leaves orphans.
2. Write `Retired because` in one clause. *Superseded by* `REQ-0nn` is the most common and the most useful.
3. Keep delivered requirements in the register with their `O59` verdict. They are the practice's only record of having finished something.
4. Review the register once a year, at the annual architecture review.
5. Report the register's size with its composition: how many `Approved`, `Proposed`, `Retired`. A register that only grows is not being managed.

Signed by. `GB-01 Architecture Board`.

Done when. No row in `006` is one nobody would defend, and every retirement has a date and a reason.

Shows up in. `RPT06`, which will get shorter and more honest at the same time.

Goes wrong when. Nothing is ever retired, because retiring looks like admitting. A 200-row register with 40 live requirements is worse than a 40-row one.

The one test for this discipline

Open `006` at a row you did not write. Ask four questions: which objective does this realise, what will be shown at a gate, which building block delivers it, and has anybody looked. In the file you were handed, the fourth answer is no for all 31 rows, because `O59 Compliance Assessment` is empty and `R24 Compliance Assessment tests` reports zero instances [4].

That is not a criticism of the repository. It is an accurate picture of a practice that has drawn a model and not yet governed anything with it — which is where most practices are, and the only reason it is visible here is that the sheets are honest enough to show it.

The motivation extension is doing real work in this chapter, and it is worth saying why. A driver, a goal, an objective, a measure, a principle and a requirement are six different things, and a retailer will hand you all six in one sentence — *we need one price because the shelf labels keep failing and the CPA says the lower one binds*. That sentence contains a driver, an obligation, a requirement and the beginning of a criterion. Separating them is the work, and the sheets are where the separation becomes permanent.

How many of your requirements have an acceptance criterion written in a column, not in a document?

Which of the two broken joins in the spine — driver to goal, requirement to objective — have you built?

Name a requirement you retired in the last year, and why.

If the compliance function left tomorrow, which of your `063` rows would nobody be able to explain?

THE CHAPTER IN TEN LINES

- A requirement exists to be tested at a gate. If you cannot name the gate, the criterion and the evidence, you are holding a preference.
- `O06` has 31 rows, seven columns, and no acceptance-criterion column and no owner column. Add both before anything else.
- `O06` `Driver` duplicates `R08`. Keep `R08` as authoritative, because a requirement can derive from more than one driver.
- The motivation spine has two breaks: nothing joins a driver to a goal and nothing joins a requirement to an objective. Add `R26` and `R27`.
- `R11` and `R10` are complete — six goals to fourteen objectives to sixteen measures, with units carried. That half of the spine is better than most.
- A requirement that traces to no objective is somebody's preference. Keep it as `Proposed`, or retire it a quarter later.
- `O05` has 24 principles, no `Tier` column and no relationship sheet. `PRN-15 Requirements-Based Change` and `PRN-10 Data is Shared` are both `Draft`.
- Write regulation at section grain. `CPA s 23(6)(b)` is an obligation; *consumer protection* is a topic, and `O63`'s six rows are topics.
- PCI DSS arrives through acquirer contracts, not a statute. No regulator to appeal to, no national derogation.
- `R09` pairs requirement n with block n in `Ref` order. `REQ-011 Price mastered once` is paired with SKU mastering while `DBB-05 Price mastering` sits unjoined. Re-argue all 31.
- Traced 23, tested 0. Report both numbers in one sentence and set the target on the second.
- Retire by `Status`, never by deletion — `R08`, `R09` and `RPT06` all reference `REQ-` values.

SOURCES FOR THIS CHAPTER

- **1.** Retail_TOGAF_Metamodel.xlsx, sheet O06 Requirement
- **2.** Retail_TOGAF_Metamodel.xlsx, sheet RPT06 Requirements Traceability
- **3.** Retail_TOGAF_Metamodel.xlsx, sheets O63 Control Objective and RPT09 Regulatory Coverage
- **4.** Retail_TOGAF_Metamodel.xlsx, sheets O59 Compliance Assessment and R24 Compliance Assessment tests, both empty
- **5.** Retail_TOGAF_Metamodel.xlsx, sheet R08 Requirement derives from Dr
- **6.** Retail_TOGAF_Metamodel.xlsx, sheet R11 Goal realised by Objective
- **7.** Retail_TOGAF_Metamodel.xlsx, sheets R10 Objective measured by Measu and O04 Measure
- **8.** Retail_TOGAF_Metamodel.xlsx, sheet O01 Driver
- **9.** Retail_TOGAF_Metamodel.xlsx, sheet O05 Principle
- **10.** Retail_TOGAF_Metamodel.xlsx, sheets O41 Security Control and RPT11 Security Position
- **11.** Consumer Protection Act 68 of 2008, s 23 — disclosure of price of goods or services (the two-prices rule in s 23(6)(b) and the inadvertent-and-obvious-error defence in s 23(9)) https://www.acts.co.za/consumer-protection/23_disclosure_of_price_of_goods_or_services
- **12.** PCI Security Standards Council, PCI DSS standards page https://www.pcisecuritystandards.org/standards/pci-dss/
- **13.** Retail_TOGAF_Metamodel.xlsx, sheets R09 Requirement satisfied by Bu and O55 Building Block
- **14.** Retail_TOGAF_Metamodel.xlsx, sheets O30 Work Package and R13 Work Package delivers Build
- **15.** Retail_TOGAF_Metamodel.xlsx, sheets O29 Gap and RPT07 Gap Analysis, 1 record
- **16.** Retail_TOGAF_Metamodel.xlsx, sheet O04 Measure
- **17.** The Open Group, The TOGAF Standard, 10th Edition — Architecture Development Method, Requirements Management https://pubs.opengroup.org/togaf-standard/adm/

APPENDIX A

The sheet-by-sheet reference

One hundred and five sheets, what each holds, the Ref prefix that is actually in column A, the sheet that is empty, and the eleven things this repository does not have.

You are holding 105 sheets. About a dozen of them you will open every week, thirty matter only in the phase you are in, and the rest you read once and leave alone. This appendix says which is which: what each sheet holds, what the first column actually contains, which columns carry judgement, and when in the ADM you fill them.

Halverson Stores Group is an invented South African retailer. Its data is real inside this workbook and nowhere else. Every number quoted below is the number in the file as shipped.

The shape of the repository in one page

Six families of sheet, in tab-bar order [1].

Family	Sheets	Count	What it is for	Do you type in it?
Index	`00 Index`	1	The repository's catalogue of itself, with a record count per sheet	Only the counts, and only after the sheets change
Objects	`001 Driver` to `066 Architecture Increment`	64	One sheet per metamodel entity. `Ref` in column A, name in column B, headers on row 4	Yes — this is where the architecture lives
Extension	`EXT Industry Extension`	1	The six object types retail needs beyond the TOGAF core	Yes, and carefully — the rule is below
Relationships	`R01 Capability decomposition` to `R25 Application supports Produc`	25	One sheet per relationship type, each named after what the link *means*	Yes — and it is the half people skip
Reports	`RPT01 Dashboard` to `RPT13 Orphan Analysis`	13	Presentations over the object and relationship sheets. Appendix B takes them one at a time	No — read only
Checks	`ZZ Checks`	1	Fifteen integrity counts against the threshold a healthy estate would show	No — read before and after every phase

One plus sixty-four plus one plus twenty-five plus thirteen plus one is 105.

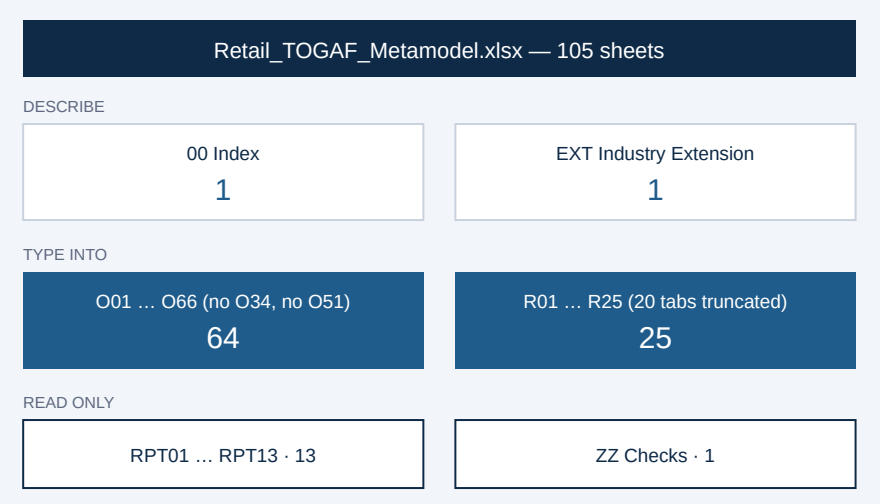


Figure 13.1 The six families of sheet. Objects and relationships are typed into; reports and checks compute; the index and the extension describe. One hundred and five sheets in all.

`00 Index` describes the file as "65 object types · 25 relationship types". Both are right, and the first needs explaining because every reader miscounts it. **The 65 is the 64 `O` sheets plus `EXT Industry Extension`, counted once.** There is no 65th `O` sheet. The index itemises all sixty-five with a record count against each, and `EXT` is the last row — spelled `EXT Industry extension`, lower-case *e*, the tab name mis-spelled by its own catalogue [1].

`034` and `051` do not exist. No sheet carries either number and the index does not itemise them. Nothing is missing from the model, but a macro walking `O01` to `O66` will fail twice. Walk the sheet names, never the numbers.

> **THE RECORD COUNT IN THE DOCUMENTATION DOES NOT SURVIVE ADDITION**
>
> `00 Index`, `RPT01 Dashboard` and `ZZ Checks` all state **842 records**. The index's own itemised list, added up, gives **813** — and each itemised figure is correct against its sheet. Quote 813, or 807 for the sixty-four core sheets without `EXT`'s six [1][3].
>
> The difference is 29, which is exactly the number of rows on `R06 Application interfaces with` — the one relationship sheet that carries a `Ref` column of its own. That is a hypothesis about how the error was made, not a fact in the file, but it is the first place to look when you rebuild the count.
>
> **And "Every type carries records" is false.** `O59 Compliance Assessment` is header-only, and `00 Index` records it honestly as "0 records" on its own row. `O29 Gap` holds a single row. So the index contradicts the threshold that `ZZ Checks` claims to meet, and the two sheets sit nine tabs apart. Sixty-four of sixty-five types are populated; one is empty and one is a token [6].

The object catalogue

Seven tables, grouped by the domain label `00 Index` puts against each sheet. Four columns need a word first.

`Ref` **prefix** is read off column A of the shipped file, never inferred from the sheet name, because several are irregular and none is derivable. Where a prefix has no hyphen, no hyphen is correct. **The columns that matter** names the columns carrying judgement; `Ref` and the name are on every sheet. **ADM phase · year one?** gives the phase that fills the sheet, then a judgement for a practice standing up from nothing: **yes** means it earns its keep inside twelve months, **if relevant** means fill it when the phase forces it, **later** means a mature practice needs it and a new one does not. **ArchiMate 4 element** is the element to use when the rows get drawn rather than listed, from the forty-two in the house notation. Appendix D, *The repository in ArchiMate 4*, carries the reasoning and wins where the two differ.

Motivation — why the architecture exists

Sheet	What it holds	Ref prefix	The columns that matter	ADM phase · year one?	ArchiMate 4 element
`001 Driver`	Twelve conditions forcing the group to act: six regulatory, six commercial	`DRV-`	`Type`, `Origin`, `Owner`, `Pressure`	Preliminary, A · yes	Driver
`002 Goal`	Six things the group has decided to be true	`GOL-`	`Owner`, `Status`	A · yes	Goal
`003 Objective`	Fourteen time-bounded statements, each realising a goal	`OBJ-`	`Goal`, `Horizon`, `Status`	A · yes	Outcome
`004 Measure`	Sixteen indicators with a baseline and a cadence	`MSR-`	`Unit`, `Domain`, `Baseline`, `Cadence`	A · yes	none — the values sit on the Outcome
`005 Principle`	Twenty-four principles across five categories	`PRN-`	`Category`, `Statement`, `Owner`, `Status`	Preliminary · yes	Principle
`006 Requirement`	Thirty-one requirements, every one naming a driver	`REQ-001`	`Statement`, `Category`, `Priority`, `Driver`, `Status`	A, then C and D · yes	Requirement
`007 Constraint`	Eight things that rule approaches out	`CON-`	`Type`	A · yes	none — carry as a Requirement
`008 Assumption`	Six things taken as true pending test	`ASM-`	`Status`	A · yes	none — repository only

`005` is well filled in one respect and thin in another. All twenty-four principles carry a `Statement`, a `Category` and an `Owner` of `Architecture Board`. What the sheet has no column for is a tie-break: there is **no `Tier`, weight or precedence column**, so when `PRN-01 Primacy of Principles` meets a principle saying the opposite, nothing settles it. Put the rule in the `Statement` of one principle rather than adding a column — a column you invent is a column no report reads.

Then read `PRN-21 Attributable Access`, which says every access to sensitive data is attributable to a person and reads `Approved`, next to `SEC-04 Attributable access` on `O41`, which reads `Absent`, and `POL-02` on `O64`, which says the same thing and reads `Met`. Three sheets, three answers, one fact [4].

`O06` is the best-built sheet here: all thirty-one requirements name a `Driver`, which is the only reason `RPT06 Requirements Traceability` computes anything.

Business — who does what, and what the business does

Sheet	What it holds	Ref prefix	The columns that matter	ADM phase · year one?	ArchiMate 4 element
`009 Organization Unit`	Twelve units with a head and a headcount	`ORG-`	`Type`, `Head`, `Headcount`	Preliminary · yes	Business Actor
`010 Actor`	Eighteen positions, every one `Filled`	`ACT-`	`Type`, `Status`	B · yes	Business Actor
`011 Role`	Eighteen roles, one per actor, same names	`ROL-`	`Type`, `Performed by`	B · yes	Role
`012 Location`	Eight sites: a support centre, four trading and logistics, two technology	`LOC-`	`Type`, `Scale`	B · yes	Location
`013 Business Capability`	100 capabilities on three levels: 10 at L1, 30 at L2, 60 at L3	`C01` to `C10.03.02`	`Level`, `Parent`, `Criticality`, `Current`, `Target`, `Disposition`	B · yes	Capability
`014 Value Stream`	Five streams, each with a stage count and a capability list	`VS01` to `VS05`	`Scope`, `Stages`, `Capabilities`	B · yes	Value Stream
`015 Business Service`	Ten services, one per level-1 capability	`BSV-`	`Capability`, `Exposure`, `Criticality`	B · yes	Service
`016 Business Process`	Twenty processes, four per value stream, named "stage *n*"	`BPR-`	`Value stream`, `Automation`	B · yes	Process
`017 Business Function`	Ten functions, one per level-1 capability	`BFN-`	`Capability`	B · if relevant	Function
`018 Product`	Five product groups, from own brand to financial services	`PRD-`	`Type`	B · yes	Product
`019 Contract`	Ten vendor agreements with an expiry	`CTR-`	`Counterparty`, `Type`, `Expiry`	B, then G · if relevant	Business Object — a contract
`035 Business Building`	Ten business building blocks	`BBB-`	`Capability`, `What it must do`,	B, E · yes	Grouping

`Block`	with acceptance criteria	`Acceptance criteria`

Three things about `O13`, the sheet you will spend the most hours in. **The hierarchy lives in the `Ref` and the dots are load-bearing**: `C01`, `C01.01`, `C01.01.01`, which is what lets `Level`, `Parent` and all ninety rows of `R01` be derived rather than typed. The assessment columns are filled on **all three levels**, giving three answers to one question and no rule about which wins — decide that before appendix C's Pack 1, not during it. And level-3 scores contradict their parents freely: `C01 Merchandising & Assortment` is `Critical` and seven of its nine descendants are `Medium` or `Low`.

`O14` holds its stages as a **count** — "4 stages", "5 stages", "6 stages", summing to twenty-four — while `O16` holds four process rows per stream, summing to twenty. The sheets disagree by four and neither names a stage. `VS01 Plan to Range` and `VS04 Order to Doorstep` also carry the identical capability list `C01, C02, C03`, which is wrong for a fulfilment stream in a way anybody in the business will spot in a second.

`O09` names five heads that exist nowhere else in the file — `Chief People Officer`, `Head of Service`, `Head of Supply`, `Head of Internal Audit`, `General Counsel` — and `O62` adds `Head of Procurement`. None is a row on `O11 Role`. Add them or stop using them, because a signature line naming a role the repository does not hold cannot be checked. The headcounts need the same treatment: `Executive` 222 and `Internal Audit` 150 only make sense as head-office figures, and `O12`'s `Scale` puts 240 people in `Convenience Format` against 109 in `Flagship Store`.

Data — the domain this sector argues about hardest

Sheet	What it holds	Ref prefix	The columns that matter	ADM phase · year one?	ArchiMate 4 element
`020 Data Entity`	Fourteen entities: SKU, hierarchy, store, assortment, price, promotion, planogram, inventory, two order types, basket, customer, loyalty, supplier	`E01` to `E14`	`Class`, `Definition`, `Classification`, `Owner`, `Master system`, `Lineage`	C (data) · yes	Business Object
`021 Logical Data Component`	Twelve logical stores, one per entity	`LDC-`	`Entity`	C (data) · yes	Data Object
`022 Physical Data Component`	Twelve physical stores against an application	`PDC-`	`Application`, `Type`	C (data), D · yes	Artifact
`036 Data Entity Relationship`	Eleven entity-to-entity relationships with cardinality	`REL-`	`From entity`, `Relationship`, `To entity`, `Cardinality`	C (data) · yes	Association
`037 Data Attribute`	Sixty attributes: six each for the first ten entities	`ATR-001`	`Entity`, `Attribute`, `Type`, `Key`, `Classification`	C (data) · yes	none — attributes are properties
`038 Data Set`	Six curated sets, from regulatory to customer analytics	`DST-`	`Description`	C (data) · if relevant	Data Object
`039 Data Migration`	Six entity moves with a from, a to and a status	`MIG-`	`From system`, `To system`, `Status`	E, F · if relevant	none — a Work Package on the Migration view
`040 Source To Target Mapping`	Twelve field mappings with a transformation rule	`MAP-`	`Source field`, `Target field`, `Rule`	F · later	none — Association
`042 Metadata Element`	Fourteen metadata categories with a coverage verdict	`MTD-`	`Category`, `What it records`, `Coverage`	C (data) · yes	none — repository only
`043 Master Data Rule`	Ten mastering rules, each naming a golden source	`MDR-`	`Rule type`, `Description`, `Golden source`	C (data) · yes	none — carry as a Requirement
`044 Data Building Block`	Ten data building blocks, one per entity for the first ten	`DBB-`	`Entity`, `What it must do`, `Acceptance criteria`	C (data), E · yes	Grouping

O20 is the most consequential sheet in the file and the one with the most to argue with. Three entities are classified Public — E04 Assortment, E07 Planogram and, the one that matters, E12 Customer. A retailer that classifies its customer entity as public has not read its own privacy notice. The Master system column is round-robin: E01 SKU / Article is mastered in POS, E05 Price and E12 Customer in Workforce Management, E13 Loyalty Account in Supplier Portal, E14 Supplier in Price & Promotion Engine. Not one is credible and RPT05 Data Ownership reprints all of them [5].

One column *is* worth believing, because two reports and ZZ Checks compute it and agree: Lineage reads Absent on E04 Assortment and E11 Transaction / Basket, and Partial on six more. A retailer that cannot trace basket data to source cannot evidence a margin number.

> **043 AND 020 DISAGREE ABOUT THE GOLDEN SOURCE ON ALL TEN ROWS**
>
> O43 Master Data Rule names a Golden source per entity and O20 Data Entity names a Master system per entity. For the ten entities both sheets cover, **the two columns agree zero times.** E01 SKU / Article is POS on O20 and Demand Forecasting & Replenishment on O43. E05 Price is Workforce Management and POS. E09 Purchase Order is WMS and Order Management System.
>
> This is not a data-quality nuisance. It is two sheets each claiming authority over the same decision, and nothing in the repository says which is the record. Pick one — O20 is the one the reports read — delete the other column or rename it, and record the decision as a change request. Appendix C's article and hierarchy workshop is where that gets settled with the people who can be wrong in public [5].

O37 covers E01 to E10 with six attributes each and stops; E11 Transaction / Basket to E14 Supplier have none. O44 has the same shape. Defensible as a scope, except that E11 and E12 carry the heaviest regulatory load — so state the scope in the sheet header rather than letting a reader assume the model is complete.

> **036 AND R19 HOLD THE SAME ELEVEN ROWS**
>
> O36 Data Entity Relationship and R19 Data Entity relates to Data carry the identical eleven REL- records under slightly different headers — From entity / To entity against From / To. Keep R19 as the relationship of record and O36 as what your conceptual data model is drawn from, and change both or neither, because nothing keeps them in step. The content of both needs rewriting in any case: REL-02 asserts that Product Hierarchy *is composed of* Store / Location, which is a generated pairing rather than a modelled one.

Application — what runs

Sheet	What it holds	Ref prefix	The columns that matter	ADM phase · year one?	ArchiMate 4 element
`023 Application Component`	Twelve applications, `A01` merchandising through `A12` analytics	`A01` to `A12`	`Type`, `Disposition`, `Criticality`, `Health`, `In service`, `Users`, `Owner`	C (application) · yes	Application Component
`024 Information System Service`	Twelve services, one per application, each `Published` or not	`ISS-`	`Application`, `Style`, `Published`	C (application) · yes	Service
`045 Application Building Block`	Ten application building blocks, for `A01` to `A10` only	`ABB-`	`Application`, `What it must do`, `Acceptance criteria`	C (application), `E` · yes	Grouping

`023` is the sheet every executive asks to see, so spot three things before they do. `A08 Price & Promotion Engine` is `Critical`, `Health` = `Poor` and `Disposition` = `Retire` — the file proposes to retire the only critical system in the estate and names no replacement. The `Owner` column holds **organisation-unit names from `009`, not people** (`A01` owned by `Internal Audit`, `A08` by `Supply & Logistics`), which `PRN-13 Data Ownership is Individual` forbids. And there is no cost column anywhere to test the dispositions against.

`045` stops at `A10`. **`A11 Supplier Portal` and `A12 Retail Analytics` have no application building block**, and they are precisely the two rows on `RPT13 Orphan Analysis`. Two independent parts of the file agree those two systems are outside the model; nothing says why.

Technology and security — what it runs on, and what guards it

Sheet	What it holds	Ref prefix	The columns that matter	ADM phase · year one?	ArchiMate 4 element
`025 Technology Service`	Twelve services in three categories	`TSV-`	`Category`	D · yes	Service
`026 Logical Technology Componen`	Seven target components with a rationale each: edge at store, real-time inventory, headless commerce, event streaming, tokenisation, shelf labels, peak-elastic infrastructure	`LTC-`	`Rationale`, `Status`	D · yes	Node
`027 Physical Technology Compone`	Twelve components with a vendor support end date and a position	`PTC-`	`Vendor`, `Class`, `Support end`, `Position`, `Location`	D · yes	Node, Device or System Software
`028 Platform Service`	Eight platform services, container runtime to feature store	`PSV-`	`Description`	D · if relevant	Service
`046 Technology Building Block`	Seven technology building blocks, matching `026` one for one	`TBB-`	`What it must do`, `Acceptance criteria`	D, E · yes	Grouping or Node
`048 Environment`	Four environments, each flagged for personal data	`ENV-`	`Description`, `Personal data present`	D · yes	Grouping
`049 Resilience Tier`	Twelve applications with a tier, a recovery objective and a test state	`RES-`	`Tier`, `Recovery objective`, `Tested`	D · yes	none — a property of the Application Component
`050 Network Link`	Ten site-to-site links with a diverse-path flag	`NWL-`	`Bandwidth`, `Diverse path`	D · if relevant	Path
`041 Security Control`	Eight controls with a position and an enforcement point	`SEC-`	`What it does`, `Position`, `Enforced at`	D, G · yes	none — carry as a Requirement
`064 Security Policy`	Four policies with a scope and a position	`POL-`	`Scope`, `Position`	Preliminary · yes	Principle
`065 Enforcement Point`	Five points, each with an honest note on what it does not yet do	`EP-`	`Acts on`, `Position`	D · yes	Node or Business Interface

`025`'s `Category` column has come adrift from its rows: `TSV-01 Compute` is categorised `Data`, `TSV-02 Storage` is `Security`. Three values are distributed across twelve services with no relation to what

the services are. Blank the column or rewrite it.

`O27` carries `Position` as a typed verdict rather than a date comparison, and the two disagree. `PTC-01 Store server estate` reads `Past support` against a `Support end` of **2027-03**, which is in the future; only `PTC-06 Warehouse control systems` (2026-02) and `PTC-12 Data platform cluster` (2026-08) are genuinely past support. **So the three on `ZZ Checks` is two.** Replace `Position` with a formula before anybody budgets against it. Two more faults in the same sheet: eight of twelve rows carry the literal string `Vendor` in the `Vendor` column, and the `Location` values put the core database cluster in `Convenience Format`.

`O49` reverses its own scale: `Tier 1` carries objectives of 15 minutes, 4 hours and 1 hour, `Tier 3` carries all four values including 15 minutes, and `A08 Price & Promotion Engine` — `Critical` on `O23` — is `Tier 3`. Decide which end is strict, apply it, then read the column that matters: **eight of twelve read `Never tested`**.

`O48` says `Production` holds no personal data while `Pre-production` does. For a group running a loyalty programme that is backwards, and it is the kind of error that survives for years because nobody reads the column. Correct it before `SEC-07 Non-production masking` is taken seriously.

> ### `O46` HAS TWO COLUMNS CALLED `REF`
>
> `O46 Technology Building Block` reads `Ref | Technology building block | Ref | What it must do | Acceptance criteria`. The first `Ref` holds `TBB-01` to `TBB-07`. The third column, also headed `Ref`, holds `T01` to `T07` — **a key space no other sheet in the file uses**. The logical components it appears to point at live on `O26`, whose refs are `LTC-01` to `LTC-07`, in the same order and with the same names. Rename the column `Logical component` and restate the values as `LTC-nn`, or delete it. A duplicate header also defeats any lookup by column name.

Realisation and delivery — what gets done about it

Sheet	What it holds	Ref prefix	The columns that matter	ADM phase · year one?	ArchiMate 4 element
`029 Gap`	**One row.** `GAP-01`, `Medium`, `Open`, due 2028-06	`GAP-`	`Domain`, `Building block`, `Severity`, `Status`, `Due`	E · yes, urgently	none in the forty-two — carry on a Plateau
`030 Work Package`	Twelve packages with cost, dates and two state columns	`WP-`	`Domains`, `Cost`, `Start`, `End`, `Approval`, `Delivery`	E, F · yes	Work Package
`031 Deliverable`	Twenty TOGAF deliverables by phase. Thirteen `Approved`, seven `Draft`	`DLV-`	`Phase`, `Status`	all phases · yes	Deliverable
`032 Standard`	Sixteen standards with a lifecycle status and a conformance verdict	`STD-`	`Category`, `Status`, `Conformance`	Preliminary, D · yes	none — carry as a Requirement
`033 Architecture Contract`	Eight contracts, one per work package `WP-01` to `WP-08`	`ACO-`	`Work package`, `Compliance status`	G · yes	Business Object — a contract
`052 Implementation Factor`	Eight dependencies, constraints and risks against the plan	`IF-`	`Category`, `Deduction`	E · yes	Assessment
`053 Transition Architecture`	Three transition states with work packages and an exit-criteria count	`TA-1` to `TA-3`	`Work packages`, `Exit criteria`	E, F · yes	Plateau
`054 Architecture Alternative`	Five costed options, one recommended and one partly adopted	`ALT-A` to `ALT-E`	`Cost`, `Recommendation`	A, E · yes	Course of Action
`055 Building Block`	All 37 building blocks in one list with a `Domain` column	`BBB-` `DBB-` `ABB-` `TBB-`	`Domain`, `What it must do`, `Acceptance criteria`	E · yes	Grouping
`056 Project`	Twelve projects, one per work package, each with a lead	`PRJ-`	`Work package`, `Cost`, `Start`, `End`, `Lead`	F · yes	Work Package

O62 Delivery Risk	Eight risks with likelihood, impact and an owner	RSK-	Likelihood, Impact, Owner	E, F, G · yes	Assessment
O66 Architecture Increment	Six increments landing at months 10 to 56	INC-1 to INC-6	Lands, What is added	E, F · yes	Plateau

O55 is not a fifth sheet of building blocks. It is **the same 37 records** as O35 (10), O44 (10), O45 (10) and O46 (7), re-presented with a Domain column and re-using their refs exactly — so BBB-01 exists on two sheets and nothing keeps the copies in step. R13 Work Package delivers Build keys on O55, so that is the copy the delivery half reads. Decide which is master, write it in the header, and only ever edit the master.

O52 is the sheet a retail architect reads before the roadmap. IF-03 Peak trading freeze — "No change in the peak window" — is the most consequential row in the realisation family, and nothing else in the repository holds a trading calendar at all. IF-06 Concurrent capacity unknown and CON-02 Delivery capacity unrecorded say the same thing from two sheets: nobody knows how much change this group can absorb at once.

O30 carries two state columns and neither is computed. They disagree freely: WP-05 Access and security is Approval = Not started and Delivery = In flight; WP-08 Process automation is Delivered against an End of 2030-10. Every Start is 2027 or later, so **nothing in the sheet can be past an end date against any real calendar** — and seven rows say they are. Those seven are a typed opinion, and RPT08 and ZZ Checks both repeat them.

> **THE COSTLIEST ARITHMETIC IN THE REPOSITORY**
>
> O54 recommends ALT-B Foundations first at **$46m** and marks ALT-E Compliance led ($31m) Partly adopted. The twelve work packages on O30 sum to **$63.7m** — 39% more than the recommended option — and DEC-01 Adopt the recommended architecture alternative is still Outstanding on O60. The plan exceeds an option nobody has approved. Reconcile the three before Phase F and present the reconciliation rather than the roadmap.
>
> Every figure in the file is in dollars, which for a South African group is the first thing to change. Convert once, record the rate and the date, and never let two currencies coexist in the Cost column.

O53 holds its exit criteria as a count and the criteria themselves exist nowhere. Five of the twelve packages — WP-06, WP-08, WP-09, WP-10, WP-12 — belong to no transition state, which R15 Transition contains Work Pa confirms with seven rows against twelve packages.

Governance — the half that makes the rest count

Sheet	What it holds	Ref prefix	The columns that matter	ADM phase · year one?	ArchiMate 4 element
`047 Reference Model Entry`	`026` 's seven plus four reference architectures. Five `Adopted` , six `Candidate`	`RME-`	`Description` , `Status`	Preliminary, D · yes	Grouping
`057 Governance Body`	Five bodies with cadence, membership and mandate	`GB-`	`Cadence` , `Membership` , `Mandate`	Preliminary · yes	Business Actor or Collaboration
`058 Governance Gate`	Four gates, G0 charter to G3 benefit	`GT-0` to `GT-3`	`When` , `What it tests`	Preliminary, G · yes	Event
`059 Compliance Assessment`	**Empty.** Seven columns, no rows	none — column A unfilled	`Acceptance criterion` , `Gate` , `Verdict` , `Evidence`	G · yes, urgently	Assessment
`060 Governance Decision`	Seven decisions, four `Outstanding`	`DEC-`	`Body` , `Status`	all phases · yes	none — Assessment, with an ADR outside
`061 Governance Change Request`	Five requests: a dispensation, a scope change, two corrections, one escalation	`CR-G`	`Against` , `Type` , `Status` , `Expires`	H · yes	none — repository only
`063 Control Objective`	Six obligations with a control objective and an evidence verdict	`CO-`	`Source` , `Control objective` , `Evidence`	Preliminary, G · yes	Goal or Requirement

`057` is the sheet the rest of this volume signs steps against, and the only place the file names a governance body. `GB-01 Architecture Board` holds G1 and G2 and grants dispensations with an expiry. `GB-02 Investment Committee` holds G0 funding and G3 benefit. `GB-03 Data Governance Council` designates data owners. `GB-04 Executive Committee` is the escalation. `GB-05 Architecture Function` is continuous and maintains the repository. Use those five names in every signature line.

`061` is the most useful small sheet in the file, because it records what your predecessor knew. `CR-G03 Raise unfunded building blocks as gaps` reads `Raised` while `029 Gap` holds one row. `CR-G05 Refuse or re-baseline the integration package` reads `Board decision required` . `CR-G01` is a dispensation for point-to-point file transfer expiring 2029-06, the only dated row in the file. Read all five on your first morning: they are an inherited to-do list, and getting them actioned is easier than starting an argument of your own.

`033` asserts four contracts `Non-compliant` and four `Not yet assessable` while `059 Compliance Assessment` holds no assessments and `R24` is header-only. A compliance verdict with no assessment

behind it is an opinion in a column. Raise the assessments or blank the verdicts.

The relationship catalogue

Twenty-five sheets, each named after what the link means. The `From → To` column below gives the endpoints the sheet actually resolves; the tab name often gives only half of them, because of the truncation described next.

> **TWENTY OF THE TWENTY-FIVE TAB NAMES ARE CUT OFF**
>
> Excel allows 31 characters in a sheet name and twenty of the `R` sheets hit the limit exactly, so the name stops mid-word: `R05 Application manages Data En`, `R13 Work Package delivers Build`, `R16 Control Objective met by Co`. Two object sheets are cut the same way, `O26 Logical Technology Componen` and `O27 Physical Technology Compone`.
>
> **Spell them as the tab spells them**, including the cut, in every formula, every cross-reference and every document. One is worse than cut: **`R04 Business Service supported` ends in a space.** A reference typed without it resolves to nothing and the error message will not tell you why. Five `R` tabs are intact — `R01`, `R11`, `R12`, `R21`, `R22` — and they are the only five you can type from memory [2].

Motivation, organisation and people

Sheet (as the tab spells it)	From → To	What the link means	The payload column	ArchiMate 4 relationship
`R08 Requirement derives from Dr`	`REQ-` → `DRV-`	This requirement exists because of that driver. All 31 requirements are covered	none	Influence
`R09 Requirement satisfied by Bu`	`REQ-` → building block	What will actually deliver the requirement	`Traced` — `Yes` on 23, `No` on 8	Realization
`R10 Objective measured by Measu`	`OBJ-` → `MSR-`	How you will know the objective was met. Fourteen rows, one per objective	`Unit`	Association
`R11 Goal realised by Objective`	`GOL-` → `OBJ-`	Which objectives add up to the goal	none	Realization
`R21 Role performed by Actor`	`ROL-` → `ACT-`	Who performs the role. Eighteen rows, every one `Filled`	`Status`	Assignment
`R22 Org Unit owns Capability`	`ORG-` → `C01` – `C10`	Which unit is accountable for a level-1 capability	none	Assignment

`R09` is the sheet behind the only real join in the report set, and eight requirements reach nothing. `R21` is the sheet behind the one clean line on `ZZ Checks` — and it is clean only because `O10` and

`011` are the same eighteen names twice, so "vacant roles: 0" is true by construction rather than by staffing.

Business and data

Sheet (as the tab spells it)	From → To	What the link means	The payload column	ArchiMate 4 relationship
`R01 Capability decomposition`	`C` parent → `C` child	The capability tree. Ninety rows: 30 at level 2, 60 at level 3	`Level`	Composition
`R02 Value stream uses Capabilit`	`VS` → `C`	Which capabilities a stream draws on. Fifteen rows, three per stream	none	Aggregation
`R03 Capability realised by Busi`	`C` → `BSV-`	The service through which a capability is consumed. Ten rows, one per level-1	none	Realization
`R04 Business Service supported`	`BSV-` → `A`	Which application carries the service. Ten rows, and the pairing is positional	none	Realization
`R05 Application manages Data En`	application → `E`	Who masters and who consumes each entity. Fourteen `Master` rows, fourteen `Consumer`	`Role` — `Master` or `Consumer`	Access
`R19 Data Entity relates to Data`	`E` name → `E` name	Entity-to-entity relationships with cardinality. The same eleven rows as `036`	`Cardinality`	Association
`R25 Application supports Produc`	`PRD-` → `A`	Which application administers a product group. Five rows	none	Realization

> **`R05` KEYS ON APPLICATION *NAMES* ON HALF ITS ROWS**
>
> `00 Index` promises that "every relationship row names both ends by reference and by name, so a broken link is visible without a lookup". `R05` breaks that promise on exactly half its rows. The fourteen `Master` rows carry `POS`, `Retail Analytics`, `Loyalty & CRM` and the rest in the `Application` key column, where a ref belongs; the fourteen `Consumer` rows correctly carry `A01`, `A04`, `A07`, `A10`. So the key column holds two different kinds of value and no lookup can read the whole sheet.
>
> Worse, the fourteen `Consumer` rows cycle through only four applications. Eight of the twelve systems in `023` consume nothing in the model. Fix the key column first, then run the article and hierarchy workshop, then fill the consumers properly. Until then nothing you compute about data authority is worth saying out loud.
>
> `019 Contract`, `039 Data Migration`, `040 Source To Target Mapping`, `043 Master Data Rule`, `049 Resilience Tier` and `050 Network Link` all do the same thing in their own key columns: names where refs belong [5].

Application, technology, delivery and governance

Sheet (as the tab spells it)	From → To	What the link means	The payload column	ArchiMate 4 relationship
`R06 Application interfaces with`	`A` → `A`	Every integration in the estate. 29 rows, and the only `R` sheet with its own `Ref` (`INT-001` – `INT-029`)	`Type`, `Frequency`, `Managed`, `Logged`	Flow
`R07 Application deployed on Tec`	`A` → `PTC-`	What each application runs on. Twelve rows, all `Production`	`Environment`	Assignment
`R18 Standard applies to Applica`	`STD-` → `A`	Which standards bind which system. Sixteen rows across six applications	`Conformance`	Association
`R20 Application located at Loca`	`A` → `LOC-`	Where an application runs. Twelve rows	none	Assignment
`R12 Gap closed by Work Package`	`GAP-` → `WP-`	The package that closes the gap. **One row**	`Severity`	none — Association on a Plateau
`R13 Work Package delivers Build`	`WP-` → `O55` block	What each package actually produces. 37 rows, one per building block	`Domain`	Realization
`R14 Project delivers Work Packa`	`PRJ-` → `WP-`	The project carrying the package. Twelve rows, one for one	`Cost`	Composition
`R15 Transition contains Work Pa`	`TA-` → `WP-`	Which packages a transition state needs. Seven rows for twelve packages	none	Aggregation
`R23 Increment delivered by Proj`	`INC-` → `PRJ-`	Which project lands each increment. Six rows, odd-numbered projects only	`Lands`	Realization
`R16 Control Objective met by Co`	`CO-` → `SEC-`	The control that evidences an obligation. Six rows, paired positionally	`Position`	Realization
`R17 Policy enforced at Enforcem`	`POL-` → `EP-`	Where a policy is actually applied. Four rows	`Position`	Assignment
`R24 Compliance Assessment tests`	assessment → block	**Empty**, because `O59` is	`Verdict`	Association

`R06` funds the integration platform and is the best-populated relationship sheet in the file. It is also inconsistent in one small way: five rows read `Type` = `Batch` or `File` with `Frequency` = `Real`

`time`, which is not a thing. Nine rows read `Direct DB read`, which is the finding rather than the inconsistency.

`R16` pairs `CO-01` with `SEC-01`, `CO-02` with `SEC-02` and so on down the sheet, so the file records **PCI DSS as evidenced by `SEC-02 Design review`** and food safety as evidenced by privileged access management. The pairings are generated. Its `Position` column is also mislabelled: it holds `O63`'s `Evidence` values, not a position. Remap it with the compliance officer before `RPT09` or `RPT11` is quoted — the counts are right and the pairings are not.

`EXT Industry Extension`

Six rows, three columns: `Ref`, `Industry extension type`, and a third headed `What it holds and why the core metamodel is not enough`. The refs run `X01` to `X06` [7]. It is the smallest sheet in the repository and the most important one in the series, because it is where the TOGAF content metamodel admits that it cannot describe a retailer.

Ref	Type	What it holds	Why the core cannot hold it
`X01`	SKU / Article	The smallest sellable unit with its trading attributes	The core has Product, which is a bundle of services sold to a customer. A SKU is a stockable, countable, barcoded atom with a cost, a retail, a pack, a tax treatment and a supplier link — and it is the grain every number in the business is computed at
`X02`	Store / Location	A trading or fulfilment location with its format and capacity	The core has Location, which is a place. A store is a trading entity with a format, a cluster, a range, a labour model, a catchment and a profit and loss. `O12 Location` holds eight sites; a retailer has hundreds and they are not interchangeable
`X03`	Assortment	The ranged set of SKUs for a location and period	A dated many-to-many decision between `X01` and `X02`. Nothing in the core models a dated set, and this is the object the whole merchandising argument is about
`X04`	Promotion	A funded offer with mechanics, period and expected uplift	The core has Contract and Requirement; neither holds mechanics, funding or forecast uplift. A promotion is also the object most often confused with a markdown, which is a different thing with a different accounting consequence
`X05`	Planogram	Shelf layout by fixture, facing and adjacency	Physical space as a first-class object. `O27`'s Node and `O12`'s Location between them cannot express a facing, and space is what converts an assortment decision into a sale
`X06`	Fulfilment Node	A location that can source, pick or hand over an order	A capability of a place, not a place. One store is a trading location, a click-and-collect point and a ship-from node at once, each with its own cut-off, cost and inventory-availability rule

Those six are not a nicety. Strike `X01` and you cannot say what a merchandising system does. Strike `X03` and the range review, the most important recurring decision in the business, has no object to attach to. Strike `X05` and space planning is invisible to the architecture, which is how it ends up in a spreadsheet nobody governs. Strike `X06` and every cross-channel design becomes a diagram of applications with no model of the place the customer collects from. Volume 1 spends four chapters here: chapter 3, *The six hierarchies that disagree*; chapter 9, *Article, style, option, SKU: the argument the*

model lives or dies on; chapter 10, *Effective dating on price and assortment*; and chapter 12 on where they live in the estate.

Five of the six already appear in `O20 Data Entity` with identical definitions: `E01 SKU / Article`, `E03 Store / Location`, `E04 Assortment`, `E06 Promotion` and `E07 Planogram`. `X06 Fulfilment Node` has no home in `O20` at all.

> **THE EXTENSION IS DECLARED AND NOT WIRED**
>
> `X01` to `X06` appear **nowhere else in the workbook**. No relationship sheet references them, no report reads them, and `00 Index` counts them as the 65th object type without giving them a sheet of their own. `EXT` is a three-column list of definitions with no attributes, no owners, no classifications and no links.
>
> So the file does two contradictory things at once: it argues, correctly and in its own words, that the TOGAF core cannot hold these six types, and then holds them as prose. The work is to make them records. Until it is done, read `O20` as the operative sheet for five of the six and treat `EXT` as the statement of intent it is.

> **THE RULE FOR EXTENDING THE EXTENSION**
>
> Add a row to `EXT` only when all three are true. **One**, it is an object the business names and argues about, not an attribute of something already modelled. **Two**, no sheet among the sixty-four can hold it without distortion — the test being whether you would have to write something untrue in the `Definition` column. **Three**, you can name the sheet it will get, the `Ref` prefix it will use and the relationship sheet that will connect it, before you add the row.
>
> Then add the sheet. A new object type needs its own `Onn` sheet with `Ref` in column A and the name in column B, headers on row 4; at least one `Rnn` sheet joining it to something that already exists; a row in `00 Index` with a record count; and a line in `EXT`. Four edits, all four or none, recorded as one change request.
>
> For the six already declared, the first three sheets to add are `X01 SKU / Article`, `X03 Assortment` and `X06 Fulfilment Node`, with relationship sheets joining SKU to assortment, assortment to store, and fulfilment node to location. Two candidates for a seventh and eighth row, both of which volume 1 argues for and this file cannot express: **Trading Calendar**, because the 4-5-4 retail week is an architectural object and three systems currently derive it independently; and **Supply Source**, because `E14 Supplier` collapses the legal party, the trading relationship and the place goods leave from into one row.

`ZZ Checks`

Three columns — `Check`, `Result`, `Threshold` — and fifteen rows: five inventory, nine findings, one disclaimer. The sheet closes with the best sentence in the repository: *the thresholds are what a healthy estate would show; the results are what this one does; that distance is the work.*

What it tests	Result as shipped	Threshold	How to read it
Object types, records, relationship types, instances, reports	65, 842, 25, 429, 13	"Every type carries records", 13 reports	Inventory. The record count is wrong and the threshold is false — see above. `65`, `25`, `429` and `13` are correct
Interfaces unmanaged	16	0	Of 29. The integration business case in one cell
Interfaces with no logging	20	0	You cannot evidence what crossed. Twenty of twenty-nine
Direct database reads	9	0	Nine places where changing one application breaks another silently
Platform components past support	3	0	Typed, not computed. Only two have a `Support end` in the past
Security controls absent	4	0	Of eight, and all four are identity or masking controls
Control objectives without evidence	3	0	Of six. Counts `In breach` (2) as well as `No mechanism` (1), which are different problems
Data entities without lineage	2	0	Of fourteen, plus six more at `Partial`. Correct and computable
Vacant architecture roles	0	0	True by construction: `O10` and `O11` are the same list
Work packages past end date	7	0	Typed. Every `Start` in `O30` is 2027 or later
Records validated against a real organisation	0	0 — illustrative data throughout	The sheet's own disclaimer, and the reason you replace the records before quoting any of them

Verify its own numbers on day one. One inventory count is wrong by 29. Three findings — platform components past support, work packages past end date, and arguably control objectives without evidence — are typed into their source sheets rather than computed, so they will neither move when you fix the data nor go stale when the data moves underneath them.

The honest reading is that `ZZ Checks` is a hand-maintained summary, right about seven things, defensibly right about three more, and arguing with itself about record counts. That is more self-awareness than most repositories have. Turn all fifteen rows into formulas, then add three about the record rather than the estate: `Records changed this month`, `Changes with no change ref`, `Rows still at baseline`. None is computable today, for the reason in the next section.

Run it twice per phase: before you start, so you know what you inherited, and at the end, so the difference is attributable to you. Photograph the sheet — the file holds no history, so the only record of last month's numbers is the copy you kept.

The change-control columns and sheets this repository does not have

This is the shortest section in the appendix and the one with the largest consequence.

There are no record-keeping columns on any object or relationship sheet. No `Version`, no `Record Status`, no `In Repository Since`, no `Last Changed`, no `Changed By`, no `Change Ref`, no `Approved By`. **And there are no governance sheets**: no `01 Version Control`, no `02 Change Log`, no `03 Change Requests`, no `04 Baselines`. The repository holds the present tense and nothing else. It cannot tell you what it said last quarter, who changed a maturity score, or whether anybody approved it.

Add all eleven before you load a single client record. Not after the first phase — before, because retrofitting a change log means inventing history.

Add this column	On	What it is for
`Version`	all `O` and `R` sheets	`1.0` at load, minor increment per approved change. Derived from the change log, never typed
`Record Status`	all `O` and `R` sheets	`Baseline`, `Approved`, or `Draft` while a request against the row is open
`In Repository Since`	all `O` and `R` sheets	The date the record first existed. Set once, never touched
`Last Changed`	all `O` and `R` sheets	The date of the most recent approved change. Equal to the above on an untouched row
`Changed By`	all `O` and `R` sheets	The role that made the change, from `011 Role`. A person, not a system
`Change Ref`	all `O` and `R` sheets	The request that authorised it. A populated `Changed By` with a blank `Change Ref` is ungoverned change, and that is the number you report
`Approved By`	all `O` and `R` sheets	`Architecture Board` for anything engaging a principle, `Chief Enterprise Architect` otherwise. Approval is an act — if `GB-01` has not met, do not type its name

Add this sheet	What goes in it	Why
`01 Version Control`	The version, the policy in prose, the current baseline, and a count of changes awaiting approval	Somebody will ask "which version is this" in your first month. Without this sheet the answer is a filename
`02 Change Log`	One row per field changed: sheet, `Ref`, column, previous value, new value, date, who, change ref	The sheet that makes the other ten work. Previous and new value in every row is what lets you reconstruct any past state. Never truncate it
`03 Change Requests`	What is being asked, against which rows, who raised it, which body decides, the decision and the date	`061 Governance Change Request` holds five requests against work packages and the repository. It is not a record-level register and will not scale to one
`04 Baselines`	Named, dated, approved snapshots with the change-log row number at which each was cut	A baseline is a row number and a signature, not a second copy of the workbook. Keeping a copy of the file as "the baseline" is how two repositories start

Three consequences follow, and they are why this page matters more than the sixty-four sheets above it.

First, **`ZZ Checks` cannot measure its own drift**: it reports the state of the estate and nothing about the state of the record. Second, **nothing in the file is evidence** — a group has to show how an architectural decision was taken and by whom, to its own audit committee if to nobody else, and `033` already asserts four non-compliant contracts with no assessment behind them. Third, **the comparison tool has nothing to compare**: a baseline-versus-target diff needs a stated baseline, and until `04 Baselines` exists every comparison runs against whatever the file happened to hold the day you copied it.

> **DO NOT START BY FILLING SHEETS**
>
> The temptation on day one is to fix `020`'s classifications or score `013`. Resist it for a week. Add the four sheets and the seven columns, raise change request number one against something that does not matter, and learn the discipline on a low-stakes row. Every correction made before the log exists is a correction with no author.

Naming and `Ref` rules

These are read off the data, not from documentation, because the file carries none.

Rule	Why
`Ref` in column A, name in column B, headers on row 4. Never reorder.	Every relationship sheet resolves both ends by position from those two columns. Reordering breaks the workbook silently and the reports still compute
One prefix per sheet, except `055`	`055` carries four — `BBB-`, `DBB-`, `ABB-`, `TBB-` — because it is the union of `035`, `044`, `045` and `046`
Pad to the width of the set	Two digits under a hundred rows; three on `006`, `037` and `R06`. `REQ-1` and `REQ-001` are different strings to a lookup
Five sheets drop the hyphen: `013`, `014`, `020`, `023`, `EXT`	`C01`, `VS01`, `E01`, `A01`, `X01` — the five most-referenced key spaces in the file. The short form is deliberate; do not normalise it
Three sheets drop the padding: `053`, `058`, `066`	`TA-1`, `GT-0`, `INC-1`. Under ten rows each, and `058` starts at zero because G0 is a gate. If one grows past nine, renumber the sheet in one change
`054` numbers with letters: `ALT-A` to `ALT-E`	Alternatives are not ranked, and letters stop a reader inferring that they are
`061` carries a letter inside the prefix: `CR-G01`	The `G` marks it as a *governance* change request, which is what distinguishes it from the record-level change register you are about to add. Keep the distinction
Hierarchy goes in the `Ref`, dotted	`C01`, `C01.01`, `C01.01.01`. `013` is the only sheet that does it, and the dots let `Level`, `Parent` and all ninety rows of `R01` be derived
Names are nouns, title case, no trailing full stop	A name appears twice in every relationship row and in all thirteen reports. `013`'s level-3 names use an em dash, which is fine and must be the same character every time
Never reuse a retired `Ref`	Once the change log exists it keys on sheet plus `Ref` plus column. Reusing `A07` makes two applications one record across the history
Refs, not names, in key columns	Seven sheets break this: `R05`, `019`, `020`, `039`, `040`, `043`, `049` and `050`. Fix `R05` first

What the repository does not hold, and where that content lives

A repository is a register of facts about the architecture. It is not the architecture.

Not in the repository	Where it lives	The link between them
Diagrams. No sheet holds a picture and no report draws one	`Retail_ArchiMate4_Views.pptx`, shipped with this set	A view is a selection over the repository. Every element on one should resolve to a `Ref`; if it does not, either the view is wrong or the sheet is incomplete
Narrative and argument	The Architecture Definition Document and the twenty deliverables on `031`	`031` holds the record of the document, not its text. Every `Draft` row is Phase E or later
Decisions, in full. `060` holds a line and a body	Architecture decision records, referenced from the change register	Context, options and consequences are an ADR's job. The repository carries the pointer and the verdict
Any history at all	Nowhere. This is the gap the four governance sheets close	Until they exist, the only record of a past state is a dated copy of the file
The extracts the comparison tool needs	Nowhere — there are **no export sheets** in this file	Build the five extracts by hand: select the columns the paste target names, copy, Paste Special values only. Never link the two workbooks
The trading calendar	Nowhere, and it is the largest single omission for this sector	`IF-03 Peak trading freeze` is the only acknowledgement it exists. Retail weeks, the 4-5-4 period structure, the 53rd week and the freeze windows need a sheet of their own
Code, configuration and schemas	The estate. `022` names the store, `037` stops at sixty attributes	Key attributes only: identity, classification, and anything carrying a regulatory consequence
Project plans. `056` holds shape, cost and a lead	The programme's planning tool	`RPT08 Roadmap` tests a plan architecturally. It is not a plan
The stages of a value stream	`016 Business Process`, partly	`014`'s count disagrees with `016`'s row count, and `016`'s names are placeholders until you rename them
The exit criteria of a transition state	Nowhere. `053` holds a count — "5 exit criteria"	Write them out. A state whose exit criteria are a number cannot be tested, and Phase F has nothing to gate on

The last two rows are the same fault twice: a column holding a count where the content belongs. Fix both, and when you find the third you will know what to do with it.

THE CHAPTER IN TEN LINES

- 105 sheets: one index, 64 object sheets, one extension, 25 relationship sheets, 13 reports, one checks sheet.
- The index's "65 object types" is the 64 O sheets plus EXT, counted once. There is no 65th O sheet.
- O34 and O51 do not exist. Walk sheet names, never numbers.
- Stated record count is 842; the index's own itemised list sums to 813. The gap is exactly R06's 29 rows.
- O59 Compliance Assessment is empty and O29 Gap holds one row, so ZZ Checks' "every type carries records" is false.
- Twenty of the 25 R tabs are cut at 31 characters, and R04 Business Service supported ends in a space.
- R05 keys on application names on fourteen of its twenty-eight rows, and six object sheets do the same thing.
- O20 and O43 disagree about the golden source on all ten entities they both cover.
- EXT declares six retail types in prose; X01 to X06 appear nowhere else in the workbook.
- There is no change control. Add seven columns and four sheets before you load a single client record.

SOURCES FOR THIS CHAPTER

- **1.** Retail_TOGAF_Metamodel.xlsx, sheet 00 Index — header counts and the itemised record list
- **2.** Retail_TOGAF_Metamodel.xlsx, the 105 sheet names as Excel stores them
- **3.** Retail_TOGAF_Metamodel.xlsx, sheets RPT01 Dashboard and ZZ Checks
- **4.** Retail_TOGAF_Metamodel.xlsx, sheets O05 Principle, O41 Security Control and O64 Security Policy
- **5.** Retail_TOGAF_Metamodel.xlsx, sheets O20 Data Entity, O43 Master Data Rule, R05 Application manages Data En and RPT05 Data Ownership
- **6.** Retail_TOGAF_Metamodel.xlsx, sheets O29 Gap, O59 Compliance Assessment, R12 Gap closed by Work Package and R24 Compliance Assessment tests
- **7.** Retail_TOGAF_Metamodel.xlsx, sheet EXT Industry Extension
- **8.** Retail_TOGAF_Metamodel.xlsx, sheets O30 Work Package, O53 Transition Architecture, O54 Architecture Alternative and O60 Governance Decision
- **9.** The Open Group, TOGAF Standard, Architecture Development Method https://pubs.opengroup.org/architecture/togaf-standard/

APPENDIX B

The report catalogue

Thirteen reports, the one question each answers, what the number means when it is bad, and the five of the thirteen that actually compute something.

Thirteen report sheets, which is a small set and an honest one. They sit at the right-hand end of the tab bar, after `R25` and before `ZZ Checks`, and they are the only sheets in the repository you read rather than fill. A typed cell on an `RPT` sheet is a broken report.

This appendix gives each one five things: the question it answers in a sentence, when to read it, what good looks like, what a bad number actually means, and the ArchiMate 4 view its content belongs on if you are going to draw it rather than tabulate it. Use it to stop reading reports in tab order, which is the order they were built and not the order they are useful.

Halverson Stores Group is invented. Its numbers are real inside this workbook and describe no real retailer.

> **WHAT THESE REPORTS ACTUALLY COMPUTE, BEFORE YOU TRUST THE WORD "REPORT"**
>
> Open them and look. **Five of the thirteen compute something. Eight are a reprint of one object sheet** [1].
>
> The five. **`RPT01 Dashboard`** aggregates: eighteen counts across the whole file. **`RPT03 Capability Heatmap`** adds a `Gap` column — `Target` minus `Current` — over the ten level-1 rows of `O13`. **`RPT06 Requirements Traceability`** joins `O06 Requirement` to the `Traced` column of `R09 Requirement satisfied by Bu` and renames it `Reaches a block`. **`RPT07 Gap Analysis`** joins `O29 Gap` to `R12 Gap closed by Work Package`, over one row. **`RPT13 Orphan Analysis`** is an anti-join: applications in `O23` with no row in `R04 Business Service supported`.
>
> The eight. `RPT02`, `RPT05`, `RPT08`, `RPT09`, `RPT10`, `RPT11` and `RPT12` are their source object sheet re-presented with nought to one columns dropped. Six of them add one computed summary sentence in the last row; `RPT02` and `RPT12` add nothing at all. `RPT04 Interface Catalogue` is `R06` with the **refs replaced by names** in the key columns, which makes it readable aloud and destroys the lookup.
>
> If you have worked the sibling banking repository, this is the same pattern with two more reports doing real work. It is not a criticism of the file. A re-presentation that drops `Definition` and adds "2 of 14 entities have no lineage from report back to source" is a useful thing to hand an executive. It is a warning about what happens when you replace the records: **nothing recomputes.** Write the formulas yourself, in the order this appendix reads them, and the sheets become reports.

> **READ THE TAB, NOT THE NUMBER YOU REMEMBER**
>
> In this file `RPT07` is **`Gap Analysis`** and `RPT09` is **`Regulatory Coverage`**. Other repositories in this series number them differently and one of them puts the gap report at `RPT09`. There are thirteen reports here, not thirty-two, and nothing in this file is called `Capability Coverage` or `Building Block Coverage`.

The repository about itself

Two reports that describe the record rather than the retailer. Read both on your first morning and neither in front of an executive.

Sheet	The question it answers	ArchiMate 4 view	Read when	What good looks like	What a bad number means
`RPT01 Dashboard`	How big is the repository, and what are the eleven findings a first pass over this estate produces?	Layered	Preliminary, then monthly. The first sheet you open and the one to quote from most carefully	Relationship instances well above record count; every finding row either zero or falling	429 instances against 813 records is 0.53 links per record — the objects are catalogued and barely joined. Eleven of the eighteen rows are findings and eight of those are non-zero
`RPT13 Orphan Analysis`	Which applications does no business service map to?	none — this is about the record, not the architecture	Preliminary, then after every load	Empty, or every row carrying a reason	`A11 Supplier Portal` and `A12 Retail Analytics`. Both are real systems that genuinely serve no business service in `R04` — the finding is that `R04` is incomplete, not that the systems are spare

`RPT01` states **842 records**, and so do `00 Index` and `ZZ Checks`. The index's itemised list sums to 813. Appendix A has the arithmetic. Until you have recomputed it, quote the findings from this dashboard and not the inventory.

Three of its rows look clean and are not. **`Gaps overdue | 0`** is true only because `029 Gap` holds a single row with a due date in 2028. **`Work packages past end date | 7`** is typed into `030`, not computed: every `Start` in that sheet is 2027 or later, so nothing in it can be past an end date against any calendar. **`Vacant architecture roles | 0`** is true by construction, because `O10 Actor` and `O11 Role` are the same eighteen names twice. `RPT01` also adds four rows that `ZZ Checks` does not carry — `Applications 12`, `Interfaces 29`, `Gaps overdue 0`, `Level 1 capabilities 10` — and drops the disclaimer row, which is the one row on `ZZ Checks` you should never drop.

`RPT13` is headed "Objects that nothing else references. Usually the most useful sheet in the repository" and tests exactly one thing: applications against business services. Nothing else in the file is checked for orphanhood, and plenty would fail. Twenty-three object sheets have **no record**

reachable from any relationship sheet at all — among them `005 Principle` (24 records), `016 Business Process` (20), `031 Deliverable` (20) and `042 Metadata Element` (14). Extend `RPT13` to cover unreferenced entities, building blocks no package delivers and standards no application is bound by, and it becomes what its own header claims it already is.

The estate — what runs, how it connects, how old it is

Three reports, and together they are the technology business case.

Sheet	The question it answers	ArchiMate 4 view	Read when	What good looks like	What a bad number means
`RPT02 Application Portfolio`	What applications exist, how healthy are they, and what is meant to happen to them?	Application Structure	Phase C (application), then quarterly	Every row with a `Disposition`, a named `Owner`, and a `Health` that agrees with its `Criticality`	Four of twelve read `Health = Poor`. `A08 Price & Promotion Engine` is `Critical`, `Poor` and `Retire` at once — the file proposes to retire the only critical system in the estate
`RPT04 Interface Catalogue`	What integrates with what, how, and is any of it managed or logged?	Application Cooperation	Phase C (application), then at every design gate	`Managed` on every row, `Logged` = `Yes` on every row, no `Direct DB read` anywhere	16 of 29 unmanaged, 20 with no logging, 9 direct database reads. Any change to one application can break three others silently. This is the number that funds an integration platform
`RPT10 Technology Currency`	Which platform components are past vendor support, and when do the rest fall off?	Implementation and Deployment	Phase D, then at every vendor roadmap review	No `Support end` inside eighteen months under a component carrying a critical application	`PTC-06 Warehouse control systems` (2026-02) and `PTC-12 Data platform cluster` (2026-08) are genuinely past support. A warehouse control system past support in a grocery business is a shelf-availability risk, not an IT risk

`RPT02` is `O23` with the `Description` column dropped and no footer at all. Its `Owner` column holds organisation-unit names, not people — `A01 Merchandising System` is owned by `Internal Audit` — which is the first thing to fix, because a portfolio with departmental owners has no owners. Read the

`Criticality` column against `RPT03`: eight of twelve applications are `Medium` or `Low` in an estate whose two most critical capabilities are merchandising and pricing.

`RPT04` is the report to read on day one. It shows names instead of refs, which makes it readable aloud in a meeting and means you cannot trace a row back to `O23` without a text match — copy the refs back in from `R06` before you use it analytically. Its footer computes the two figures that matter: *16 of 29 interfaces are unmanaged and 20 carry no logging* [2]. Pair it with `RPT11 Security Position`, where `SEC-05 Interface logging` reads `Absent`. The control that would produce the logging does not exist, so the twenty is not a gap in coverage; it is the whole estate.

`RPT10` carries `Position` as a stated verdict rather than a date comparison, and the verdict is wrong on one row: `PTC-01 Store server estate` reads `Past support` against a `Support end` of 2027-03. **So the three on the footer and on `ZZ Checks` is two.** Replace `Position` with a formula against `Support end` and today's date, because `PTC-07 Enterprise storage array` (2027-09) and `PTC-09 Cloud landing zone` (2027-11) roll off inside eighteen months and the sheet will go stale silently.

The business — the one report an executive reads without translation

Sheet	The question it answers	ArchiMate 4 view	Read when	What good looks like	What a bad number means
`RPT03 Capability Heatmap`	Where is business capability weak relative to how much the group depends on it?	Capability Map	Phase B, then once a year at most	`Gap` under 1 on anything `Critical`; `Build` concentrated where criticality is high	`C01 Merchandising & Assortment` is `Critical` at `Current` 1, `Target` 2, `Gap` 1, `Disposition` `Sustain`. The weakest score in the file sits on the most critical capability, and the file proposes to sustain it

`Gap` is the one genuinely computed column in the report set [3]. Four of the ten rows carry a gap of 2 — `C02 Pricing & Promotion`, `C06 Digital Commerce`, `C07 Order Fulfilment & Returns`, `C10 Finance & Margin Management` — but the row to take to a board is `C01`. A retailer whose merchandising and assortment capability scores 1 out of 5 cannot range, cannot cluster and cannot plan, and every other number in the file is downstream of that.

Three cautions. **The targets are not ambitious enough to be real.** `C01` is `Critical` with a target of 2, while `C05 Store Operations` is `High` at `Current` 4 with a target of 5 — the strongest capability gets the bigger investment. Argue with the targets before the currents. **The sheet shows level 1 only**, ten rows, while `O13` holds 100 rows scored on all three levels and the level-1 figures were not derived from the levels below. A level-1 score here is an assertion, not a roll-up — recompute it before you present it. And **`Disposition` is reprinted from `O13`, not derived from `Gap`**, which is why `C01` can carry a gap and read `Sustain`.

Volume 1, chapter 2, *The formats and lines of business*, is the substance behind the ten names. Match on name, never on number: the repository's `C01` to `C10` and volume 1's own capability list were built independently.

The data — the strongest case in the repository

Sheet	The question it answers	ArchiMate 4 view	Read when	What good looks like	What a bad number means
`RPT05 Data Ownership`	Who owns each entity, which system masters it, and can a number be traced back to source?	Information Structure	Phase C (data), then at every ownership session	A named person in `Owner`, a credible `Master system`, a `Classification` matching the sensitivity, `Lineage = Full`	`E04 Assortment` and `E11 Transaction / Basket` read `Lineage = Absent`, and six more read `Partial`. Basket and assortment are the inputs to every margin, availability and like-for-like number the business reports

`RPT05` is `O20` with the `Definition` column dropped, and its footer computes the one figure to carry into a meeting: *2 of 14 entities have no lineage from report back to source.*

Everything else on this sheet needs arguing with before it leaves the building. Three entities are classified `Public`, one of them `E12 Customer`. Every `Master system` value is implausible: `E01 SKU / Article` mastered in `POS`, `E05 Price` and `E12 Customer` in `Workforce Management`, `E13 Loyalty Account` in `Supplier Portal`. And `O43 Master Data Rule` names a different golden source for every one of the ten entities both sheets cover — **the two columns agree zero times** [4]. Read `RPT05` as a worked example of the shape the sheet takes, and run appendix C's article and hierarchy workshop before you quote a single row.

Obligation, control and conformance — the licence half

Three reports that answer the only question an inspector really asks: can you show me that this is met, and by what?

Sheet	The question it answers	ArchiMate 4 view	Read when	What good looks like	What a bad number means
`RPT09 Regulatory Coverage`	For each obligation, what is the control objective and what evidences it?	Motivation	Preliminary, then at every Architecture Board	`Evidence = Evidenced` on every row, with the evidence named	`CO-02 PCI DSS` and `CO-05 Product safety and recall` read `In breach`; `CO-06 Modern slavery and supply chain due diligence` reads `No mechanism`. An inability to identify, stop and recall stock is the one on this list that stops trading
`RPT11 Security Position`	Which security controls exist, where are they enforced, and which are absent?	Technology	Phase D, then quarterly	No row reading `Absent`; every control naming a real enforcement point	Four of eight read `Absent`, and they are `Attributable access`, `Interface logging`, `Periodic access review` and `Non-production masking`. A control that is absent is not a control
`RPT12 Standards Conformance`	Does anything in the estate conform to the standards the estate set itself?	Technology	Preliminary and Phase D, then quarterly	`Conformance = Conformant` wherever `Status = Current`	Three rows read `Not assessed` and one reads `Breach`. `STD-11 Network segmentation` is `Deprecated` and `Conformant`; `STD-09 Container image` is `Deprecated` and in `Breach`. Two deprecated standards are being measured and one current one is not

`RPT09` is `O63 Control Objective` verbatim — all five columns, nothing dropped — plus a footer. Its `Control objective` column is a placeholder: every row reads "Control that evidences <the obligation>". **Rewrite those six cells first.** They are the only six cells in the repository an inspector would read as a statement of intent, and at present they state nothing. The footer's wording is also the one to copy: *3 obligations have no mechanism or a mechanism in breach* — which is more precise than `RPT01`'s row for the same figure, `Control objectives with no evidence | 3`. Two of the three have a mechanism that is failing and one has no mechanism at all. Those are different problems with different owners and different costs. Split the number when you write the formula.

`RPT11` is `O41` verbatim with a footer, and it should be read with `R16 Control Objective met by Co`, which maps obligations to controls positionally — `CO-01` to `SEC-01`, `CO-02` to `SEC-02` — so the file records PCI DSS as evidenced by `SEC-02 Design review` and food safety as evidenced by privileged access management. Redo that mapping before either report is quoted. The counts are right; the pairings are generated.

`O64 Security Policy` has no report of its own, and it holds the sharpest contradiction in the file. `POL-02 Every access to sensitive data is attributable` reads `Met` while `SEC-04`

`Attributable access` reads `Absent` and `PRN-21 Attributable Access` is an approved principle. `POL-03 Material activity is logged and observable` reads `Breached`, which given twenty unlogged interfaces is the single most defensible finding in the repository — and nothing surfaces it. Add both rows to `RPT11`.

Requirement, gap and plan — the delivery half

Sheet	The question it answers	ArchiMate 4 view	Read when	What good looks like	What a bad number means
`RPT06 Requirements Traceability`	Does every requirement reach a building block something will actually deliver?	Requirements Realization	Phase A, then at every requirements review and every gate	`Reaches a block = Yes` on everything, and every `Must` traced before any `Could`	Eight of thirty-one read `No`, and they are the wrong eight: `REQ-001 Evidence Consumer protection and pricing accuracy`, `REQ-005 Evidence Product safety and recall` and `REQ-009 Store / Location mastered once` are all `Must` and reach nothing
`RPT07 Gap Analysis`	Every gap, its severity, and the package that closes it	Implementation and Migration	Phase E, then at every board	A gap per finding, each with a severity, a due date and a work package	**One row.** `GAP-01`, a merchandising capability uplift, `Medium`, due 2028-06, closed by `WP-01`. One gap against eight non-zero findings on `RPT01` is not a clean estate — it is an unpopulated register
`RPT08 Roadmap`	What is being spent, over what period, and what state is each package in?	Implementation and Migration	Phase E and F, then monthly	`Approval` and `Delivery` consistent on every row; costs reconciling to the approved alternative	Twelve packages summing to **$63.7m** against the $46m of `ALT-B`, the option `O54` recommends and `DEC-01` has not yet approved

`RPT06` is the best-built report in the file: one row per requirement, a real join to `R09`, and a footer that states the principle plainly — *a requirement that reaches no building block is a requirement nothing will deliver.* Work the eight `No` rows in priority order and the sheet is clean in a week. One structural caution: the sheet traces thirty-one requirements onto thirty-one building blocks, of which only one is a `TBB-`, so six of the seven technology building blocks have no requirement behind them.

`RPT07` is the most important nearly-empty sheet you will be handed, and the repository tells you why. `O61 Governance Change Request` row `CR-G03 Raise unfunded building blocks as gaps` reads `Raised`. Somebody worked out that most of the 37 building blocks on `O55` are not funded by a

package in `R13`, raised the request to turn that into a gap register, and left. Populating `O29 Gap` is therefore not your idea — it is an outstanding instruction, and that is a far easier thing to get approved [5].

Do not read one row on this sheet as good news. `ZZ Checks` has no row for gaps, `RPT01` reports `Gaps overdue | 0`, and both are vacuously true.

`RPT08` drops `O30`'s `Domains` column and keeps both state columns, which disagree: `WP-05 Access and security` is `Approval = Not started` and `Delivery = In flight`; `WP-08 Process automation` is `Delivered` against an `End` of 2030-10. Replace `Delivery` with a formula over `End` and today's date before anybody plans against it. Then do the arithmetic the sheet does not: twelve costs, one total, compared against `O54 Architecture Alternative` and `O60 Governance Decision`. **A roadmap costing 39% more than an unapproved option is the finding, and it is one addition away** [6].

One retail-specific test to apply to this sheet and no other repository in the series needs it. Every `Start` and `End` in `O30` is a month. Check each against the trading calendar: a package that ends in November or December ends inside the peak freeze, which means it does not end. `IF-03 Peak trading freeze` on `O52` says so in four words — "No change in the peak window" — and five of the twelve packages start or end in the fourth quarter.

The five reports for a quarterly architecture board

One page, five panels, three decisions. These five and not the others.

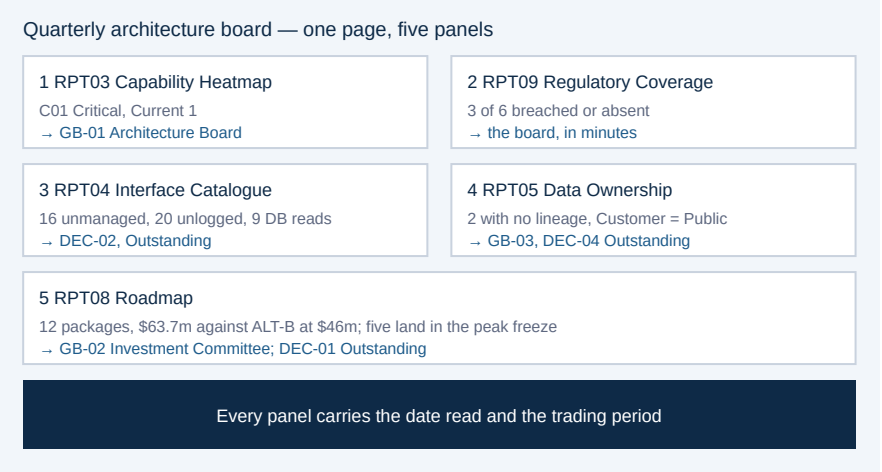


Figure 14.1 The quarterly board page. Five panels, each tied to the body with the mandate to act and the Outstanding decision on O60 it unblocks.

#	Report	What it puts on the page	The decision it forces
1	`RPT03 Capability Heatmap`	Ten rows, a criticality, a gap. `C01 Merchandising & Assortment` `Critical` at `Current` `1` with a target of 2	Which two capabilities get investment this year, and whether `C01` 's target is honest. Nothing else in the file frames that question in the business's own words
2	`RPT09 Regulatory Coverage`	Six obligations, three with no mechanism or a mechanism in breach, one of them product recall	Whether the board accepts that position for another quarter, in minutes, by name
3	`RPT04 Interface Catalogue`	16 of 29 unmanaged, 20 unlogged, 9 direct database reads	Whether the integration platform is funded. `DEC-02 Select the integration product` is `Outstanding` and `RSK-01` rates that `Certain`
4	`RPT05 Data Ownership`	Fourteen entities, two with no lineage, every master system implausible, `E12 Customer` classified `Public`	Who owns the article master. `GB-03 Data Governance Council` exists precisely to take that decision and `DEC-04` is still `Outstanding`
5	`RPT08 Roadmap`	Twelve packages, $63.7m, two state columns that contradict their own dates, five packages landing in the freeze	Whether the plan is approved at the cost it actually carries, or re-cut to the $46m of `ALT-B`

Why those five. Each one is a finding a named body already has the mandate to resolve: `GB-01 Architecture Board` for the heatmap and the interfaces, `GB-02 Investment Committee` for the roadmap, `GB-03 Data Governance Council` for data ownership, and the board as a whole for the regulatory position. Each has a figure the room cannot argue away without changing the record. And

four of the five map onto an `Outstanding` row in `O60 Governance Decision`, which turns the pack from a status report into an agenda.

Why not the others. `RPT01 Dashboard` repeats a record count that is wrong and will cost you the first ten minutes of the meeting defending arithmetic rather than architecture — bring three of its rows inside the other panels instead. `RPT02 Application Portfolio` is a list of twelve systems with no decision attached, and `RPT03` carries the same argument in the business's language. `RPT10`, `RPT11` and `RPT12` belong to a technology or security forum where the audience can act on them, and they lose their force in a general meeting. `RPT06` is an architect's working sheet and nobody else will read a thirty-one-row traceability table. `RPT07` has one row and `RPT13` has two — take both to the board the quarter *after* you have populated them, as evidence that the practice is working.

Two rules for the pack. Every panel carries the date you read it, because the repository holds no history and no version: until `01 Version Control` and `04 Baselines` exist, the date on your slide is the only provenance any of these numbers has. And every panel carries the trading period as well as the month, because a board that meets in week 44 is being asked to approve change inside a freeze and should be told so.

THE CHAPTER IN TEN LINES

- Thirteen reports, not thirty-two. RPT07 here is Gap Analysis and RPT09 is Regulatory Coverage.
- Five compute: RPT01 aggregates, RPT03 subtracts, RPT06 and RPT07 join, RPT13 anti-joins. Eight are reprints.
- RPT04 is R06 with the refs replaced by names — readable aloud, useless for a lookup.
- Replace the records and nothing recomputes. Write the formulas before you load client data.
- RPT01 repeats the wrong record count, and three of its clean-looking rows are artefacts.
- RPT04 funds the integration platform: 16 of 29 unmanaged, 20 unlogged, 9 direct database reads.
- RPT03's Gap is the only computed column in the catalogue, and C01 is Critical at Current 1 with a target of 2.
- RPT09's six Control objective cells are placeholders. Rewrite them before an inspector reads them.
- RPT08 sums to $63.7m against the $46m of the recommended and still-unapproved ALT-B, and five packages land in the freeze.
- Board pack: RPT03, RPT09, RPT04, RPT05, RPT08. Each maps to a body with a mandate and an Outstanding row on O60.

SOURCES FOR THIS CHAPTER

- **1.** Retail_TOGAF_Metamodel.xlsx, sheets RPT01 Dashboard to RPT13 Orphan Analysis
- **2.** Retail_TOGAF_Metamodel.xlsx, sheets RPT04 Interface Catalogue, R06 Application interfaces with and RPT11 Security Position
- **3.** Retail_TOGAF_Metamodel.xlsx, sheets RPT03 Capability Heatmap and O13 Business Capability
- **4.** Retail_TOGAF_Metamodel.xlsx, sheets RPT05 Data Ownership, O20 Data Entity and O43 Master Data Rule
- **5.** Retail_TOGAF_Metamodel.xlsx, sheets RPT07 Gap Analysis, O29 Gap, R12 Gap closed by Work Package, O55 Building Block and O61 Governance Change Request
- **6.** Retail_TOGAF_Metamodel.xlsx, sheets RPT08 Roadmap, O30 Work Package, O52 Implementation Factor, O54 Architecture Alternative and O60 Governance Decision
- **7.** The Open Group, TOGAF Standard, Architecture Development Method https://pubs.opengroup.org/architecture/togaf-standard/

APPENDIX C

Workshop packs, checklists and the ninety-day calendar

Five workshops that fill named cells, ten checklists you can tick against the file, and thirteen weeks that look entirely different depending on the month you start them.

Volume 1, chapter 18, *Your first ninety days*, runs the learning track: what is already public before week one, the eight documents to ask for by name, who to meet in what order, what to refuse in month one, and the politics. This appendix is the other half — the repository track. It assumes a populated workbook landed on your desk on day one and you have to make it true. The two tracks are designed to be worked in the same thirteen weeks and this one repeats none of that chapter's content.

Halverson Stores Group is invented. Business role names below are volume 1, chapter 5's, *Who does what*; architecture roles and governance bodies are the repository's own, from `011 Role` and `057 Governance Body` [1].

Five workshop packs

A workshop that fills no cells was a meeting. Each pack names the cells.

Look it up. Volume 1, chapter 5, *Your real counterparties*, names the people whose problems you can solve. Four of them chair a pack below.

Pack 1 — Capability maturity

Fills. `013 Business Capability`: `Criticality`, `Current`, `Target`, `Disposition`, on the level-3 rows of one level-1 area. Six rows per level-2 group, six level-3 rows per area.

Who must be in the room. The line executive who owns the area and can be held to the score. For `C01 Merchandising & Assortment` that is the **category manager** and the **merchandise planner**, both, because the area spans the decision each of them owns. For `C02 Pricing & Promotion`, the **pricing manager** and the **promotions planner**. For `C04 Supply Chain & Replenishment`, the **demand planner** and the **replenishment analyst**. For `C05 Store Operations`, a **store manager** and a **regional manager**. Plus the relevant domain architect (`ROL-09` to `ROL-12`) holding the pen, and one person from commercial finance — the **category finance analyst** — who can say what the area costs. Never more than nine. A tenth turns an assessment into a presentation.

How long. Three hours for one level-1 area. Do not attempt two in a day: scoring drifts after lunch and the afternoon area always scores higher.

Send three working days out. The nine rows for that area — one level-1, three level-2, six level-3 — with `Capability` and `Description` only and the assessment columns blank. The maturity scale, five lines, one per level. `RPT03 Capability Heatmap` so the room sees what the file currently claims about

their area. Volume 1, chapter 2, *The formats and lines of business*, drafts the capability names: start there, argue with it, and **match on name, never on number**.

Running order	Minutes	What happens
1. Boundaries	25	Read each name and description aloud. The room recognises it or renames it. Renaming is allowed; adding is not
2. Criticality first	30	`Critical`, `High`, `Medium`, `Low`. This is dependence, not performance
3. Current	60	One to five, with a named example behind every score
4. Target	35	Where it must be by the horizon on `003 Objective`, not where it would be pleasant
5. Disposition	20	`Sustain` or `Build`. Falls out of the gap, which it does not in the shipped file
6. The parent rows	10	Agree that levels 1 and 2 will be rolled up from here, not scored separately

Facilitation moves that work. Score `Criticality` before `Current` — once a room has admitted a capability is weak it will not then admit it is critical, because the two together sound like an accusation. Ask for evidence rather than a score: "what would I see if I watched this being done for a week?" and let the number follow. When two people differ by two points, write both down and move on; the disagreement is the finding. And open by putting `C01` on the screen as shipped: `Critical`, `Current` 1, `Target` 2, `Sustain`. A room takes ninety seconds to work out that the file proposes to sustain the weakest score on the most critical capability, and after that nobody treats the sheet as scripture.

Three ways it derails. First, **everything becomes critical.** Cap it: no more than a third of the rows. Second, **the room scores the parents.** All 100 rows arrive with `Current` and `Target` filled on all three levels and the level-1 figures were derived from nothing — say in the first five minutes that levels 1 and 2 will be recomputed. Third, **somebody names a system.** The moment a vendor or an application enters the conversation you are assessing software. Park it for Pack 3.

Done when. Every level-3 row in the area carries four judgements, the level-1 row is a computed roll-up, and the executive who owns it repeats the gap figure in their own words.

BEFORE YOU RUN PACK 1

What is the one sentence that decides whether a row belongs in `013` at all?

Which two of the ten level-1 areas will you assess first, and why those two?

Who will be asked to defend each score, and to whom?

Pack 2 — The article and hierarchy workshop

This is the hardest session in retail and the most valuable. It replaces the generic data-ownership workshop, because in a retailer the ownership argument is downstream of a vocabulary argument that nobody has had.

Fills. `O20 Data Entity`: `Definition`, `Class`, `Classification`, `Owner`, `Master system`, `Lineage` — on `E01 SKU / Article`, `E02 Product Hierarchy`, `E03 Store / Location` and `E04 Assortment` first. Then `O43 Master Data Rule`'s `Golden source`, which currently disagrees with `O20`'s `Master system` on **all ten rows they share**. Then the `Role` column and the key column of `R05 Application manages Data En`, which holds application names on fourteen of twenty-eight rows. Then `EXT Industry Extension` row `X01 SKU / Article`, and the decision to give it a sheet of its own.

Who must be in the room. This pack lives or dies on the attendance list, and it needs more of the business than any other session in this volume. The **master data manager** chairs the working half; `GB-03 Data Governance Council` chairs the decision half, with the **Chief Data Officer** present. Then one owner per hierarchy, because the six hierarchies are the session: the **category manager** for the merchandise hierarchy, the **inventory accountant** for the financial hierarchy, the **category finance analyst** for the reporting roll-up, the **space planner** for the space hierarchy, the **digital product owner** for online navigation, and whoever owns supplier data for the GS1 structure. Add the **head of merchandising systems**, who knows what the systems actually enforce, and the **information security and privacy officer** for `Classification`. `ROL-16 Data Steward` takes the pen.

How long. A full day, split: three hours on article and the hierarchies, lunch, two hours on ownership and mastering. Four entities. The remaining ten are two further sessions and both are easy once this one lands.

Send five working days out. `RPT05 Data Ownership` as shipped, with `E12 Customer` classified `Public` highlighted and the two `Absent` lineage rows circled. The `O20 Master system` column next to the `O43 Golden source` column, as two columns on one page, so the room sees ten disagreements before it sits down. The `R05` extract showing who claims to master what. One proposed definition per

entity, one sentence each. And volume 1, chapter 9, *Article, style, option, SKU: the argument the model lives or dies on*, to everybody — not as reading, as the agenda.

Running order	Minutes	What happens
1. The two questions	40	"Is its stock counted separately, and can it be sold on its own?" Then "would the buyer make one buy decision for it?" Settle style, option, SKU and trade unit in the room's own words
2. The six hierarchies	80	Each owner draws their own hierarchy over the same ten articles. Nobody is allowed to say another's is wrong
3. Purpose, not fact	30	For every disagreement, ask whether it is about a fact or a purpose. A purpose means two hierarchies and one mapping
4. Definitions	45	One sentence per entity for `E01` to `E04`, agreed aloud
5. Classification	30	`Public`, `Internal`, `Restricted`, `Confidential`, with the privacy officer holding the pen
6. The master	60	One application per entity, and the `O43` column reconciled to it. If the room cannot agree, write `CONTESTED` and name the decision

Facilitation moves that work. Put the six hierarchies on six separate sheets of paper and have six different people draw them over the same ten articles. The room discovers in forty minutes what an argument would not settle in a year: all six are correct, "breakfast" spans four merchandise classes, and brown bread and brown bread rolls sit in the same commercial category and different statutory ones. Then ask the settling question — *are you disagreeing about a fact or about a purpose?* — and the deliverable writes itself: one page saying which hierarchy answers which question.

Make `CONTESTED` a respectable answer and say so in the first minute: a contested entity recorded is worth more than a master invented, and `DEC-04 Designate data owners` is already `Outstanding` on `O60`. Ask the privacy officer to read `O48 Environment` out loud, where `Production` is flagged as holding no personal data. And when the mastering argument stalls, stop asking who holds the record and ask instead: when two systems disagree about this article, which one is right?

Three ways it derails. First, **the room tries to pick one canonical hierarchy.** That is the failure the whole session exists to prevent. Hold the line: six hierarchies, one article-to-node assignment per hierarchy, all dated. Second, **somebody solves it with a product.** A master data platform is a Phase E conversation; authority is a decision and it is free. Third, **the e-commerce product wins.** Online grouping is a navigation decision that changes every season, and a multipack and a single are one navigation product and two SKUs. If the digital product owner leaves the room believing their hierarchy is the master, you will be unpicking it for two years.

Done when. Four entities have one sentence each, one classification, one named person, and one master or the word `CONTESTED`. `O43` and `O20` agree or the disagreement is recorded as a change request. `R05` keys on `A01` to `A12` throughout. One page names which of the six hierarchies answers which question, signed by `GB-03`. And `EXT` row `X01` has a named sheet number against it.

YOUR ARTICLE AND HIERARCHY SESSION

The date, and the six people who will each draw one hierarchy:

The ten articles you will use. Pick ones that break: a multipack, a weighed loose item, a pharmacy line, an own-brand and a seasonal:

The entities you expect to record `CONTESTED`, and what decision each one names:

Pack 3 — Application portfolio scoring

Fills. `O23 Application Component`: `Disposition`, `Criticality`, `Health`, `Owner`, `Users`. Then `O49 Resilience Tier` `Tier`, `Recovery objective` and `Tested`, because the same people are in the room and it takes twenty minutes more.

Who must be in the room. `ROL-11 Domain Architect, Application` chairs. The **head of merchandising systems** for `A01`, `A06` and `A07`; the **store systems manager** for `A02` and `A10`; the **digital product owner** and the **fulfilment operations manager** for `A03` and `A04`; the **distribution centre manager** for `A05`; the **integration architect** for all twelve; the **data and analytics lead** for `A12`; the **payments and acquiring manager** for `A02`. One business voice who can speak to value: the **category manager** or `ROL-14 Head of Operations`. And `ROL-15 Head of Service Management`, who brings the incident record — the only evidence for `Health` anybody will accept.

How long. Three hours for twelve applications. Fifteen minutes each on the five that matter and five minutes each on the rest.

Send three working days out. `RPT02 Application Portfolio` as shipped. `RPT04 Interface Catalogue` filtered per application, because an application with six unmanaged interfaces cannot be scored healthy. `RPT10 Technology Currency` with the two genuinely past-support rows marked. Twelve months of incident and change-failure counts from service management — counts, not opinions. And peak-period incident counts separately, because an application that is adequate in March and fails in week 48 is not adequate.

Running order	Minutes	What happens
1. Ownership	20	Replace every departmental `Owner` with a named person. Twelve rows, no debate
2. Criticality	30	Against the capability it supports in `R04`, not against how much the room likes it
3. Health	60	`Adequate`, `At risk`, `Poor`, evidenced by incidents, change failures, interface state and support dates
4. Peak behaviour	20	For each row: did it hold up in the last peak? This column does not exist and should
5. Disposition	30	`Retain`, `Invest`, `Consolidate`, `Replace`, `Retire`. Then test each against criticality
6. Resilience	20	Tier, recovery objective and last test date. Eight rows read `Never tested`

Facilitation moves that work. Score `Criticality` from `R04` rather than from the room. Make `Health` require three pieces of evidence or it reverts to `At risk`. When a room wants to `Retire` something, ask which business service in `R04` loses its support and what happens to the data in `R05`; half the retirements become consolidations in one question. Put `A08 Price & Promotion Engine` up first: `Critical`, `Poor`, `Retire`, in service since 2022, no replacement named anywhere in the file. Retiring the price engine without a successor is not a disposition, it is a hope.

Three ways it derails. First, **everything is retained**, because retention needs no business case. Insist on a reason per row and a named owner for each reason. Second, **the room scores cost.** There is no cost column on `O23` and adding one invites a number nobody can source — cost belongs to `O30 Work Package`. Third, **disposition is set by vendor roadmap.** An end-of-support date is an input to `Health`, not a disposition; `RPT10` holds the dates and `O19 Contract` holds the expiries, and both are evidence rather than decisions.

Done when. Twelve rows each have a named person, a criticality that agrees with `R04`, a health score with three pieces of evidence behind it, and a disposition somebody will defend at `GB-02 Investment Committee`. `RPT13 Orphan Analysis` has either lost `A11` and `A12` or gained a reason against each.

Pack 4 — Gap classification

Fills. The `Review` sheet of `Retail_Architecture_Comparison_Tool_TEMPLATE.xlsx`, then `Gap Candidates`, then `O29 Gap` — which holds one row today, and which `CR-G03 Raise unfunded building blocks as gaps` on `O61` already instructs you to populate.

Before the session, the extracts. This repository has **no export sheets**. There is nothing to copy out in one action, so the five extracts are produced by hand: open each source sheet, select the columns the paste target names, copy, Paste Special values only. Never link the two workbooks — an external reference breaks on rename and depends on a user enabling updates. Allow half a day and do it twice, once for baseline and once for target, in one sitting.

Who must be in the room. Small. `ROL-08 Chief Enterprise Architect` and the two domain architects whose sheets produced the differences. One person from delivery who knows what actually

shipped — `ROL-13 Head of Delivery`. `ROL-18 Internal Auditor` for one hour, not four, because "was this change governed?" is their question and they answer it faster than anyone.

How long. Two hours for the first sixty differences, then an hour per further sixty. The first session is slow because the room is learning three words.

Send two working days out. The tool with both inputs pasted and the diff computed. The three classifications with one worked example each. And this instruction: nobody classifies in advance.

Running order	Minutes	What happens
1. The three words	20	A **design delta** is an intended change to the target. **Governed drift** is a real change with an approval behind it. **Ungoverned change** is a real change with nothing behind it
2. First pass	45	Classify every difference. No discussion over thirty seconds — park anything slower
3. The parked rows	30	Work the parked list. These are the findings
4. Gap candidates	20	Every ungoverned change and every unclosed design delta becomes a candidate row
5. Severity and due	15	`Severity` and `Due` per candidate, then paste into `O29 Gap`

Facilitation moves that work. Classify before you explain: the explanation is where the room negotiates, and a thirty-second first pass gives a better distribution than an argued one. Keep the ungoverned-change count on a flipchart, because that number is the business case for the change control appendix A tells you to add. When somebody says "that was agreed", ask where — and if the answer is a meeting rather than a record, it is ungoverned change. Not an accusation, the finding.

Three ways it derails. First, **everything is a design delta**, because that classification blames nobody. Hold the line: a delta nothing in the record authorised is drift at best. Second, **the room tries to fix the architecture** instead of classifying the difference. Gaps get raised here and solved in Phase E. Third, **the extracts were taken on different days**, which generates dozens of false differences and destroys the room's confidence by the fortieth row. Take both in one sitting and write the date on each.

Done when. Every difference carries one of three words, `Gap Candidates` holds a row per finding, `O29 Gap` holds more than one row, `R12 Gap closed by Work Package` has rows, and `RPT07 Gap Analysis` computes something worth reading for the first time.

Pack 5 — Roadmap sequencing against the trading calendar

Fills. `O30 Work Package` `Start`, `End`, `Approval`; `O53 Transition Architecture` `Work packages` and real written exit criteria in place of a count; `O56 Project` `Lead`; `O66 Architecture Increment` `Lands`.

Run this one differently from every other roadmap workshop you have facilitated. **In a retailer the freeze window decides the sequence more than any dependency does.** A dependency can be renegotiated; mid-November to mid-January cannot. The freeze is wider than software release: it

covers infrastructure change, structural master-data change — hierarchy reclassification above all — range change, planogram resets and store refits. A freeze of that length compresses the delivery year into about nine months, and the arithmetic is the whole session.

Who must be in the room. `GB-02 Investment Committee` members or their delegates: the **Chief Financial Officer**, the **Chief Information Officer** and `ROL-08 Chief Enterprise Architect`. `ROL-13 Head of Delivery`, who owns capacity. The **trading director or their planner**, who owns the calendar — and this is the person most often left out and least replaceable. The **store systems manager**, because store-estate change needs a non-trading window and there are few of them. The **distribution centre manager** if any package touches a warehouse control system.

How long. Three hours, and schedule a second session a fortnight later. The first produces a sequence; the second survives contact with the delivery plan.

Send five working days out. `RPT08 Roadmap` with the cost total computed — **$63.7m** across twelve packages — and `O54 Architecture Alternative` beside it, showing `ALT-B` recommended at **$46m** and `DEC-01` still `Outstanding` on `O60`. `O52 Implementation Factor`, all eight rows, with `IF-03 Peak trading freeze` at the top. `O62 Delivery Risk`, where `RSK-01 Integration product not selected` is rated `Certain`. And the next two years of the trading calendar on one page: retail weeks, period boundaries, the festive peak, back-to-school, Easter, the 53rd week if one falls in the horizon, and every freeze window marked in a single colour.

Running order	Minutes	What happens
1. The calendar first	30	Mark the freeze windows on the wall before anybody proposes a date. Count the usable weeks per year. That number, not twelve months, is the plan
2. The arithmetic	25	$63.7m against $46m. Settle whether the plan is re-cut or the alternative re-approved before sequencing anything
3. Dependencies	40	Read `O52`. `IF-01` blocks integration, `IF-02` blocks mastering. Draw the order those two force
4. Transition states	40	Which packages belong to `TA-1`, `TA-2`, `TA-3`, and the exit criteria for each — written out, not counted. Five packages currently belong to none
5. Capacity	25	The Head of Delivery says how many packages run at once. `CON-02` and `IF-06` both record that this is unknown; fix it here
6. Dates and leads	30	`Start`, `End` and a named `Lead` per project, every date tested against the freeze. Then check `O66`'s six increments still land in order

Facilitation moves that work. Open with the calendar, then the cost, then the sequence. A room that has not settled the first two will re-sequence around them all afternoon. Make the exit criteria the deliverable: `O53` ships with "5 exit criteria" in a cell and no criteria anywhere, and a state you cannot test is a state Phase F cannot gate on. Ask the trading planner for next year's range-review dates and planogram reset weeks before anybody proposes a go-live. Test the sequence arithmetically: if `TA-1` needs four packages, capacity is two at a time and the year holds nine usable months, the critical path is longer than the room thinks — and that is subtraction, not opinion. Then check every `End` date

against the freeze: five of the twelve packages start or end in the fourth quarter, which means five of them do not end when they say.

Three ways it derails. First, **the sequence is set by funding year** rather than by dependency and calendar, which produces a plan where integration precedes the product selection that enables it. Second, **everything starts in the first transition state**, because nobody wants to be last. Use the exit criteria as the test: a package not needed to exit `TA-1` is not in `TA-1`. Third, **the freeze is treated as an IT constraint** and negotiated. It is a trading decision and the trading director will not negotiate it in a technology meeting. Get the dates in writing beforehand.

Done when. Twelve packages have dates that obey `052` and clear every freeze window, three transition states have written exit criteria, all twelve packages belong to a state, every project has a named lead, `066`'s six increments land in dependency order, and the cost total reconciles to a decision recorded in `060`.

Quality checklists, one per ADM phase

Tick these against the repository, not against your memory of it. Every line is an "every X has a Y" assertion and every one is checkable in under a minute.

Preliminary Phase

✓	The assertion	Check it against
	The four governance sheets exist: `01 Version Control`, `02 Change Log`, `03 Change Requests`, `04 Baselines`	the tab bar
	Every object and relationship sheet carries the seven record-keeping columns	any `O` sheet, columns to the right
	Every one of the 24 principles has a `Statement`, an `Owner` and a `Status`, and one of them states the tie-break	`O05`
	Every one of the 5 governance bodies has a `Cadence`, `Membership` and `Mandate`	`O57`
	Every one of the 4 gates names what it tests and when	`O58`
	No `Current` standard reads `Not assessed`, and no `Deprecated` standard is being measured	`O32`, `RPT12`
	`ZZ Checks` has been read, verified against the data, dated and kept	`ZZ Checks`

Phase A — Architecture Vision

✓	The assertion	Check it against
	Every goal has an `Owner` who is a person	`O02`
	Every objective realises a goal and has a `Horizon`	`O03`, `R11`
	Every objective has at least one measure, and every measure a `Baseline` and `Cadence`	`R10`, `O04`
	Every requirement names a driver	`O06`, `R08`
	Every driver has an `Owner` and a `Pressure`	`O01`
	Every `Must` requirement reaches a building block	`RPT06`
	No assumption has been `Open` for more than a quarter	`O08`

Phase B — Business Architecture

✓	The assertion	Check it against
	Every level-1 capability decomposes to level 3	`O13` , `R01`
	Every assessed capability has `Criticality` , `Current` , `Target` , `Disposition`	`O13`
	Level-1 and level-2 scores are rolled up, not separately typed	`O13` , `RPT03`
	Every `Disposition` agrees with its `Gap`	`O13` , `RPT03`
	Every value stream's stages are named rows, not a count	`O14` , `O16`
	Every value stream names the capabilities it actually uses	`R02`
	Every capability has an owning organisation unit, and every unit head is a role on `O11`	`R22` , `O09` , `O11`
	Every business building block has an acceptance criterion that can be failed	`O35`

Phase C — Data Architecture

✓	The assertion	Check it against
	Every entity has a `Definition` in the business's own words	`O20`
	No entity holding personal data is classified `Public`	`O20` , `RPT05`
	Every entity has a named person as `Owner` , not a department	`O20`
	Every entity has exactly one `Master` row, keyed by ref	`R05`
	`Master system` and `Golden source` name the same application for every entity	`O20` , `O43`
	Every entity has attributes, or a stated reason it has none	`O37`
	Every hierarchy has a stated purpose and a dated article-to-node assignment	`O20` , `O36`
	Every entity feeding a reported margin or availability figure has `Lineage` = `Full`	`O20` , `RPT05`

Phase C — Application Architecture

✓	The assertion	Check it against
	Every application has `Disposition`, `Criticality`, `Health` and a named `Owner`	`O23`, `RPT02`
	Every application supports a business service, or appears on `RPT13` with a reason	`R04`, `RPT13`
	Every application has an application building block	`O45`, `O55`
	Every interface has `Type`, `Frequency`, `Managed` and `Logged`, and the two agree	`R06`, `RPT04`
	No `Direct DB read` exists without a dated dispensation	`R06`, `O61`
	Every application declares the entities it masters and consumes	`R05`
	Every product group is administered by an application	`O18`, `R25`
	Every information system service names its application and `Published` state	`O24`

Phase D — Technology Architecture

✓	The assertion	Check it against
	Every physical component has a `Support end` date and a **computed** `Position`	`O27`, `RPT10`
	Every `Vendor` cell names a vendor	`O27`
	Every application is deployed on a component, with an `Environment`	`R07`
	Every application has a tier, a recovery objective that matches the tier, and a test date	`O49`
	Every environment answers `Personal data present` correctly	`O48`
	Every store-estate link answers `Diverse path`, and every trading site has one	`O50`
	No security control reads `Absent` without a gap against it	`O41`, `RPT11`
	Every security policy is enforced at a named point and its `Position` agrees with `O41`	`O64`, `R17`

Phase E — Opportunities and Solutions

✓	The assertion	Check it against
	Every `ZZ Checks` result above threshold has a gap	`ZZ Checks` , `029`
	Every gap has `Domain` , `Building block` , `Severity` , `Status` , `Due`	`029` , `RPT07`
	Every gap is closed by a work package	`R12`
	Every one of the 37 building blocks is delivered by a work package	`055` , `R13`
	Every work package has `Cost` , `Start` , `End` , `Domains`	`030`
	Work package costs sum to the chosen alternative's cost	`030` , `054` , `060`
	No work package starts or ends inside a freeze window	`030` , `052`
	Every transition state has written exit criteria, and every package belongs to one	`053` , `R15`

Phase F — Migration Planning

✓	The assertion	Check it against
	Every work package has a project with a named `Lead` who is a role on `011`	`056` , `R14` , `011`
	Every increment has a `Lands` month and a delivering project	`066` , `R23`
	Every data migration has a from, a to and a `Status`	`039`
	Every migration cutover falls outside the freeze	`039` , `052`
	Every delivery risk has `Likelihood` , `Impact` and an `Owner` who has seen it	`062`
	Every Phase E and F deliverable has moved off `Draft`	`031`
	`Delivery` is computed from `End` , not typed	`030` , `RPT08`

Phase G — Implementation Governance

✓	The assertion	Check it against
	Every work package has an architecture contract	`033`
	Every `Compliance status` is backed by an assessment	`033`, `059`
	Every acceptance criterion has a verdict	`059`, `R24`
	Every gate held has a dated decision	`058`, `060`
	Every dispensation has an `Expires` date	`061`
	Every control objective has evidence, a named mechanism, or a gap	`063`, `RPT09`
	No `Non-compliant` contract is in flight without a board decision	`033`, `060`

Phase H and Requirements Management

✓	The assertion	Check it against
	Every record change has a change-log row and a `Change Ref`	`02 Change Log`
	No record has a `Changed By` and a blank `Change Ref`	every `0` sheet
	Every `Outstanding` decision has a body and a next meeting date	`060`, `057`
	Every change request has a `Status` and a named owner	`061`
	Every requirement reaches a building block	`RPT06`
	`ZZ Checks` has been re-run and every delta explained	`ZZ Checks`
	`EXT` is unchanged, or was extended by the four-edit rule	`EXT`, `00 Index`

The ninety-day calendar

Thirteen weeks against the repository. Volume 1, chapter 18, runs the learning track in the same weeks: weeks one and two sit beside *Before week one: what is already public* and *Week one: get into a store, then a distribution centre*; weeks three to eight beside *What to draw first, and why it must be small* and *What to refuse in month one*; weeks nine onwards beside *Six numbers that tell you whether the estate is healthy*. Read *The politics, honestly* before week one and again in week six, and *What week thirteen looks like* at the start rather than the end.

Two rules before week one. **Weeks one and two touch no sheet** — everything after week two changes the record, and until the change log exists in week three there is no author for any change you make.

> **THIRTEEN WEEKS STARTING IN OCTOBER IS NOT THIRTEEN WEEKS STARTING IN FEBRUARY**
>
> Count your weeks on the trading calendar before you plan them, because the retail year is not the calendar year.
>
> **Start in February** and you get thirteen clear weeks: the post-festive trough is over, back-to-school has shipped, range reviews for the second half are being planned and the business has time to argue with you.
>
> **Start in October** and weeks five to eleven fall inside the festive peak and the change freeze. Nobody in trading, store operations or supply chain will sit in a three-hour workshop in week 48. Your five packs compress into weeks one to four and then resume in mid-January, so plan a seventeen-week ninety days and say so on day one. Spend the freeze on the work that needs nobody: verifying `ZZ Checks`, adding change control, fixing the mechanical faults, reading `O60` and `O61`. That is six weeks of genuinely useful solitary work and it is available at no political cost, because the one thing a retailer will never give you in November is an executive's afternoon.
>
> **Start in June or July** and week thirteen lands just before the peak planning cycle, which is the best possible moment to put a roadmap in front of `GB-02`.

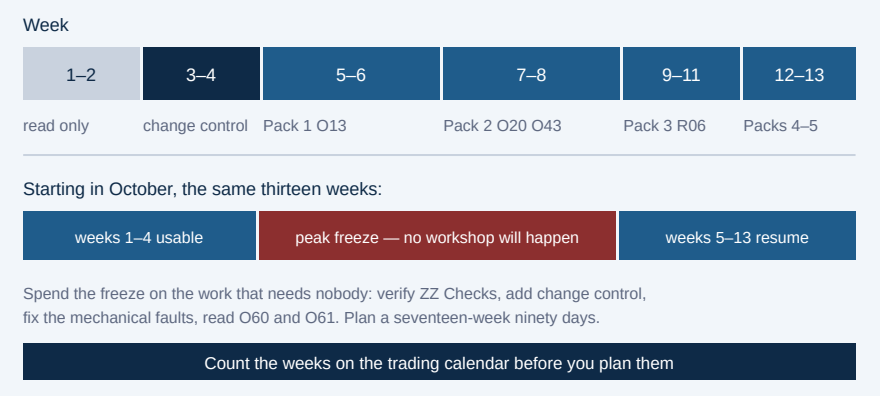


Figure 15.1 The repository track across thirteen weeks, and the same thirteen weeks started in October. The freeze does not move, so the plan does.

Week	What you do in the repository	Sheets you touch
1	Read `00 Index`, `RPT01 Dashboard` and `ZZ Checks` end to end. Then `O60` and `O61` — seven decisions, four `Outstanding`, five change requests. Write down every number that surprises you	none — read only
2	Read `RPT04`, `RPT05`, `RPT09`, `RPT13`. Add the index's itemised record counts and compare with the stated 842. Mark which `ZZ Checks` results are computed and which are typed	none — read only
3	Add `01 Version Control`, `02 Change Log`, `03 Change Requests`, `04 Baselines` and the seven record-keeping columns. Raise `CR-001` against something that does not matter	the four new sheets
4	Fix the mechanical faults, one change request each: `R05`'s name-keyed rows, `O46`'s duplicate `Ref` column, `O48`'s personal-data flags, `O27`'s `Vendor` and `Location` columns, `O25`'s categories	`R05`, `O46`, `O48`, `O27`, `O25`
5	Pack 1 on `C01 Merchandising & Assortment — Critical` at `Current` 1, the worst row in the file	`O13`, `R01`
6	Pack 1 on `C02 Pricing & Promotion` and `C10 Finance & Margin Management`. Recompute `RPT03` as a roll-up	`O13`, `RPT03`
7	Pack 2, the article and hierarchy workshop. A full day on `E01` to `E04`. Reconcile `O43` to `O20`	`O20`, `O43`, `R05`, `EXT`
8	Pack 2 on the remaining ten entities, over two sessions. Fix the three `Public` classifications with the privacy officer	`O20`, `O37`, `O42`, `RPT05`
9	Fill `R06` properly with the integration architect: `Type`, `Frequency`, `Managed`, `Logged` on all 29 rows, and resolve the five batch-at-real-time rows. Then read `RPT04` and build the integration case	`R06`, `RPT04`
10	Pack 3. Twelve applications, named owners, evidenced health, a peak column. Then `O49`: twelve tiers and eight missing test dates	`O23`, `O49`, `RPT02`
11	Remap `R16` with the compliance officer — it pairs `CO-0n` to `SEC-0n` positionally — and rewrite the six placeholder `Control objective` cells	`O63`, `O41`, `R16`, `RPT09`
12	Build the five extracts by hand, run the comparison tool, run Pack 4 on the first sixty differences, paste the agreed gaps	`O29`, `R12`, `RPT07`
13	Pack 5, with the trading calendar on the wall. Write `O53`'s exit criteria out. Reconcile `RPT08` to `O54` and `DEC-01`. Cut `BL-001` on `04 Baselines`	`O30`, `O53`, `O56`, `O66`, `04 Baselines`

What you can show, and when

Point	What you put in front of people	Why it lands
End of week 2	One page: the repository's own numbers against its own documentation. 842 records stated, 813 itemised. "Every type carries records" against an empty `059.020` and `043` disagreeing about the golden source on all ten shared entities. Four `Outstanding` decisions and `CR-G03` raised and unactioned. And the eight findings on `RPT01` that *are* computed and correct	It costs nothing, needs nobody's cooperation, and establishes that you read the record before forming an opinion. It also gets `GB-01` to approve something in your second week, which is a habit worth starting
End of week 6	`RPT03 Capability Heatmap` for three level-1 areas, scored by the people who own them and rolled up properly — with `C01` at `Current` 1 and its target argued up from 2. Alongside it, `RPT05` showing basket and assortment with no lineage into the margin numbers those same people sign	Three executives have scored their own area and will defend the figure. The lineage finding opens the data conversation with their fingerprints on it rather than yours
End of week 13	`RPT07 Gap Analysis` computing over a real register, every gap with a severity, a due date and a package. `RPT08` reconciled: $63.7m against $46m, with a recommendation, and every date tested against the freeze. `BL-001` cut and approved	A gap register with named owners is what makes the practice real. The cost reconciliation is the first thing you will have produced that a programme director cannot argue with, because it is addition. The freeze test is the first thing a trading director will recognise as retail literacy

YOUR NINETY DAYS

Your start date, the trading week it falls in, and your week 2, week 6 and week 13 show dates:

Every freeze window inside your thirteen weeks, from the trading calendar, in writing:

The three sheets you have decided not to touch in year one, and why:

The scope you have agreed — which formats, which categories, which capabilities and which applications are out:

If you only do five things

You will not get the time to do all of it. Nobody does. These five, in this order, in the first six months, and the repository stays true enough to be worth keeping.

#	Do this	Sheets	Why this one
1	**Add change control and make every change go through it.** Four sheets, seven columns, one change request per edit, including your own corrections	`01` – `04`, every `O` and `R` sheet	Everything else can be caught up later. This cannot, because retrofitting a change log means inventing history. It is also the only thing in the file that would answer an audit committee asking how a decision was taken
2	**Verify `ZZ Checks` and fix the mechanical faults.** The 842 against 813, the empty `059`, `R05`'s mixed key column, `046`'s duplicate `Ref`, `048`'s inverted flags, `027`'s typed `Position`	`ZZ Checks`, `R05`, `046`, `048`, `027`	A day in total, and until `R05` keys consistently nothing you compute about data authority is trustworthy. These are the cheapest credible wins in the repository and they cost nobody else any time
3	**Run the article and hierarchy workshop.** Six hierarchies, six owners, one page saying which answers which question. Then one master per entity and `043` reconciled to `O20`	`O20`, `043`, `R05`, `EXT`	It is the argument every other data question in a retailer is downstream of, and the repository currently holds ten disagreements about it and no record of the decision. `DEC-04` is already `Outstanding`, so you are completing an instruction rather than starting an argument
4	**Assess capability at level 3 with the people who own it, and roll it up.** `Criticality` before `Current`, every score evidenced, and argue with the targets	`O13`, `RPT03`	It is the only artefact here a line executive reads without translation, and it is the input to every prioritisation argument you will have for three years. It also kills the level-1 figures that were derived from nothing, and the `C01` row that reads `Critical`, `Current` 1, `Sustain`
5	**Fill `Managed` and `Logged` on all 29 interfaces, then populate `029 Gap`.** The second follows from the first	`R06`, `029`, `RPT04`, `RPT07`	Sixteen unmanaged, twenty unlogged and nine direct database reads is the number that funds an integration platform, and it tells you which applications cannot be changed safely before a peak. `CR-G03` already asks for the gap register, which makes it the easiest thing in this list to get approved

What all five have in common: none needs a budget, none needs a vendor, and every one ends with a number somebody else has to act on. That is the whole method.

THE CHAPTER IN TEN LINES

- Five packs, each naming the cells it fills. A workshop that fills no cells was a meeting.
- Pack 1 scores O13 at level 3 and rolls up. Open with C01: Critical, Current 1, Target 2, Sustain.
- Pack 2 is the article and hierarchy workshop — six owners draw six hierarchies over the same ten articles.
- O20 and O43 disagree about the golden source on all ten shared entities. That reconciliation is Pack 2's output.
- Pack 3 scores O23 with evidence from service management and adds a peak-behaviour column the sheet lacks.
- Pack 4 needs hand-built extracts — this repository has no export sheets — and ends with O29 Gap holding more than one row.
- Pack 5 starts with the trading calendar on the wall, then the cost arithmetic, then the sequence.
- Ten phase checklists, every line an "every X has a Y" assertion checkable in under a minute.
- Thirteen weeks from October loses weeks five to eleven to the freeze. Plan a seventeen-week ninety days and spend the freeze on solitary work.
- If you only do five things: change control, verify ZZ Checks and fix the mechanical faults, the article workshop, capability at level 3, the interface columns.

SOURCES FOR THIS CHAPTER

- **1.** Retail_TOGAF_Metamodel.xlsx, sheets O10 Actor, O11 Role and O57 Governance Body
- **2.** Retail_TOGAF_Metamodel.xlsx, sheets O13 Business Capability, O20 Data Entity, O23 Application Component, O43 Master Data Rule and R05 Application manages Data En
- **3.** Retail_Architecture_Comparison_Tool_TEMPLATE.xlsx, sheets Review and Gap Candidates (design specification, October 2026)
- **4.** Retail_TOGAF_Metamodel.xlsx, sheets O30 Work Package, O52 Implementation Factor, O53 Transition Architecture, O54 Architecture Alternative and O66 Architecture Increment
- **5.** Retail_TOGAF_Metamodel.xlsx, sheets O60 Governance Decision, O61 Governance Change Request and ZZ Checks
- **6.** The Open Group, TOGAF Standard, Architecture Development Method https://pubs.opengroup.org/architecture/togaf-standard/

APPENDIX D

The repository in ArchiMate 4

One hundred and five sheets of rows, forty-two elements, eleven relationships — and four retail concepts the notation cannot hold at all.

Why this mapping exists

You have been handed two descriptions of one architecture and they do not look alike on the page. `Retail_TOGAF_Metamodel.xlsx` is a TOGAF content metamodel written out as rows: 105 sheets, 842 records, 429 resolved relationship instances, headers on row 4 and the first column of every object sheet headed `Ref` [1]. `Retail_ArchiMate4_Views.pptx` is a pack of pictures. Nothing in either file says how one becomes the other. This appendix is that bridge. Halverson Stores Group is an invented South African composite retailer and its data is real only inside this workbook.

The repository is the **record**: it answers by filter — which of the 31 requirements reaches no building block, which entity has two stated sources of truth, how many interfaces nobody logs. An ArchiMate 4 view is the **argument** — six rows out of eight hundred, showing a shape you can say out loud to a chief financial officer in thirty seconds. Authority runs one way.

> **THE RULE**
>
> The repository is authoritative. A view that disagrees with it is wrong — not a variant, not a simplification, not "the version we drew for the board". Where a view shows an element the repository does not hold, either the repository is missing a record and you raise a row on `O61 Governance Change Request`, or the view is inventing architecture and you delete it. There is no third case.

That rule carries more weight here than in most repositories, because this one has no change control: no `01 Version Control` sheet, no `02 Change Log`, and no `Version`, `Last Changed` or `Approved By` column anywhere on the 64 `O` sheets. The only record of a model change is the five rows on `O61`. Two views drawn a quarter apart cannot be reconciled by date — only by `Ref`. Four consequences follow.

Every element carries its `Ref`. `A01 Merchandising System`, not "merchandising". Rows share names across sheets: 18 `O10` actors with `O11` roles, ten `O17` functions with their capabilities, seven `O47` entries with `O26` word for word.

Views are generated, not drawn. Each is a filter over an object sheet and one or two relationship sheets, and with no export sheets that means filter, copy, paste. Write the filter in the view's notes.

The loss runs one way. `Criticality`, `Current`, `Target`, `Disposition`, `Health`, `Classification`, `Lineage`, `Conformance`, `Tested`, `Managed` and `Logged` cannot be drawn at all. That is why the repository exists.

A row is a type, not an instance. `E01 SKU / Article` is one record, and a trading group this size holds several hundred thousand articles, each with a price that changes weekly. The notation has no instance level, no quantity and no effective date: the model describes the shape of the article and the data model holds the article. Volume 1, chapter 16, *Forty-two elements, four hundred thousand articles*, argues that at length, and it is the thing to carry into everything below.

Volume 1, chapter 16, *Modelling retail in ArchiMate*, teaches the notation — *Five decisions that settle most of a retail model*, *The mapping*, *The honest limits*. This appendix assumes that teaching and maps *these sheets* onto it. Where the two disagree about an element, volume 1 is right. Where they disagree about a sheet, open the sheet.

The object mapping — all 64 object sheets

The arithmetic first, because the file's index gets it half right. `00 Index` and `ZZ Checks` both state **65 object types**; the workbook holds **64 sheets prefixed `0`**, `O01` to `O66` with `O34` and `O51` absent, which are real gaps and not an export error. Sixty-four sheets plus the `EXT` tab is the 65 the index counts.

Two differences from ArchiMate 4's layers matter. `O13` and `O14` are labelled *Business domain* and belong to the **strategy** layer. And nine elements carry no layer at all — Role, Collaboration, Process, Function, Service, Event, Path, Grouping, Location — so `O15`, `O24`, `O25` and `O28` all collapse onto **Service**. Without the `Ref` on the label, `BSV-01` and `ISS-01` are the same box.

Twenty-two of the 64 sheets have no ArchiMate 4 element of their own. Nine carry the fact another way — six substitute an element honest about what it is, one is a relationship, one is a filter over other sheets and one names an element already modelled. The remaining thirteen stay in the repository, and a view that needs one says in its notes that it cannot show it. A near fit is worse than nothing, because the near fit gets believed.

Motivation — eight sheets

Sheet	ArchiMate 4 element	Layer	Why this element	Watch out for
`001 Driver`	Driver	Motivation	A condition motivating intent	Six of 12 repeat `CO-01` – `CO-06`. Model one
`002 Goal`	Goal	Motivation	Intent with no end date	`Owner` is a title, not a `Ref` into `011`
`003 Objective`	Outcome	Motivation	`Horizon` is 12 months to 3 years	Not a Goal. Collapse them and realisation goes
`004 Measure`	No element	—	No metric element exists	Properties on the Outcome; movement in `RPT01`
`005 Principle`	Principle	Motivation	A rule informing how the enterprise acts	Five of 24 read Draft, which is unsayable
`006 Requirement`	Requirement	Motivation	Something a design satisfies or fails	The `Driver` column duplicates `R08`
`007 Constraint`	None — carry as Requirement	Motivation	No Constraint in the language	`CON-06` reshapes every plan and no element says so
`008 Assumption`	No element	—	Assessment would misstate it	`ASM-03`, `ASM-04` read At risk. Use the notes

Strategy — three sheets

Sheet	ArchiMate 4 element	Layer	Why this element	Watch out for
`013 Business Capability`	Capability	Strategy	An ability with an owner and a maturity	100 rows, ten at level 1. One level per view
`014 Value Stream`	Value Stream	Strategy	Activity ending in a result	`Stages` is a count. Draw stages and you invent them
`054 Architecture Alternative`	Course of Action	Strategy	An approach chosen to reach a goal	Publish `ALT-B`. Keep the rejected four rejected

> **`014` AND `R02` DISAGREE, AND ONE IS WRONG**
>
> `014`'s `Capabilities` column and `R02 Value stream uses Capabilit` record the same fact. They agree on `VS01`, `VS02` and `VS03`. On `VS04 Order to Doorstep` the column reads `C01, C02, C03` and `R02` reads `C04`, `C05`, `C06`; on `VS05 Return to Resolution` the column reads `C02, C03, C04` and `R02` reads `C05`, `C06`, `C07`. The column has copied earlier values down. Draw from the relationship sheet, raise the column on `061`, and never load a value stream view from both.

Organisation and business — ten sheets

Sheet	ArchiMate 4 element	Layer	Why this element	Watch out for
`009 Organization Unit`	Business Actor	Business	A structure performing behaviour	`Headcount` is undrawable. Not the org chart
`010 Actor`	Business Actor	Business	Identity, not position	All 18 rows are positions. Model `011`
`011 Role`	Role	none	A position an actor occupies	No "Business Role" exists, and `Role` has no layer
`012 Location`	Location; Facility for the building	none / Tech	A licensed site is a regulated entity	Four of eight rows are **formats**, not places
`015 Business Service`	Service	none	Behaviour visible outside	`Exposure` reads Internal on five of ten
`016 Business Process`	Process	none	A sequence producing an outcome	20 generated names; `Value stream` is on no `R` sheet
`017 Business Function`	Function	none	Behaviour with no fixed order	All ten repeat a capability name
`018 Product`	Product for `PRD-05`; Grouping for the ranges	Business / none	Product means terms and price	`PRD-01 Own Brand Grocery` is a range
`019 Contract`	None — carry as Business Object	Business	No Contract in the language	`Counterparty` names an application, not a supplier
`035 Business Building Block`	Grouping	none	Required ability, not a thing	Realization to the `Capability`. Repeated on `055`

Data — eleven sheets

Sheet	ArchiMate 4 element	Layer	Why this element	Watch out for
`020 Data Entity`	Data Object; Business Object conceptually	Application	An encapsulation of data	`Lineage` reads Absent on `E04` and `E11`
`021 Logical Data Component`	Grouping	none	A boundary over entities	One entity each, so it adds nothing
`022 Physical Data Component`	Artifact	Technology	A deployed store is a deployable	`Type` is a storage class, so nothing sits beneath
`036 Data Entity Relationship`	None — a relationship	—	Relationships on an object sheet	Same 11 rows as `R19`; `REL-02` is false
`037 Data Attribute`	No element	—	Attributes are properties	Six attributes, ten entities. `E11` – `E14` have none
`038 Data Set`	Data Object	Application	A collection with a consumer	Six orphans, and `RPT13` lists none
`039 Data Migration`	No element	—	A movement is not a construct	Properties on the Work Package moving it
`040 Source To Target Mapping`	No element	—	A rule is not architecture	Nearest is a Flow. `Rule` is lost
`042 Metadata Element`	No element	—	Metadata is not in the model	`Coverage` reads "Recorded, not held"
`043 Master Data Rule`	None — carry as Requirement	Motivation	Survivorship is policy	`Golden source` contradicts `020` on **all ten rows**
`044 Data Building Block`	Grouping	none	As `035`, one layer down	Realization to the `Entity`. Repeated on `055`

Application — three sheets

Sheet	ArchiMate 4 element	Layer	Why this element	Watch out for
`023 Application Component`	Application Component	Application	Encapsulated functionality	`A01` is one row and four systems
`024 Information System Service`	Service	none	Automated behaviour with a consumer	The realising application is a column. Authored edge
`045 Application Building Block`	Grouping	none	What must exist before choosing	One per application: a disposition target

Technology — eight sheets

Sheet	ArchiMate 4 element	Layer	Why this element	Watch out for
`025 Technology Service`	Service	none	A technical capability	`Category` is miscoded. No `R` sheet uses it
`026 Logical Technology Componen`	Node	Technology	Vendor-independent infrastructure	`LTC-01`, `LTC-02` read Target: not baseline
`027 Physical Technology Compone`	Node, Device, System Software or Communication Network by `Class`	Technology	The row names a product class	`Support end` drives `RPT10`. Three past support
`028 Platform Service`	Service	none	A capability applications consume	Eight rows on no `R` sheet
`046 Technology Building Block`	Grouping	none	As `035` and `045`	Column three is also `Ref`: a broken key
`048 Environment`	Grouping	none	A named set of deployed things	`ENV-02`, `ENV-04` hold personal data
`049 Resilience Tier`	No element	—	A recovery objective is a property	Eight of 12 read Never tested
`050 Network Link`	Path, realised by a Communication Network	Technology	The network realises the connection	Two pairs appear twice with different answers

Implementation and migration — eleven sheets

Sheet	ArchiMate 4 element	Layer	Why this element	Watch out for
`029 Gap`	None — carry as Assessment	Motivation	No Gap in the implementation aspect	**One record.** Every other gap is authored
`030 Work Package`	Work Package	Implementation	Actions achieving a result	`Delivery` reads Past end date on seven of 12
`031 Deliverable`	Deliverable	Implementation	A signed-off output	20 rows by `Phase`, joined to no package
`032 Standard`	Requirement, specification as an Artifact	Motivation	Testable against a component	`Conformance` comes from `R18`, not the element
`033 Architecture Contract`	None — carry as Business Object	Business	The verdict is the useful fact	Non-compliant on four of eight. Prefer `059`
`052 Implementation Factor`	No element	—	Planning rationale, not architecture	`IF-03 Peak trading freeze` belongs in the notes
`053 Transition Architecture`	Plateau	Implementation	A state you can stop at	`Exit criteria` is a count. The criteria are nowhere
`055 Building Block`	No element of its own	—	A composed view over four sheets	10+10+10+7 = 37. Draw it and blocks double
`056 Project`	Work Package	Implementation	The same element as a package	Twelve against twelve. Draw both and cost doubles
`062 Delivery Risk`	No element	—	Not an architecture statement	Three read Certain, so they are issues
`066 Architecture Increment`	No element	—	The notation models states, not deltas	Three names repeat `TA-1` – `TA-3`

Governance and security — ten sheets

Sheet	ArchiMate 4 element	Layer	Why this element	Watch out for
`041 Security Control`	None — carry as Requirement	Motivation	No control in the language	Absent means no Realization edge. Four of eight
`047 Reference Model Entry`	No element	—	It records what should be chosen	`RME-01` – `RME-07` duplicate `026`
`057 Governance Body`	Collaboration; Business Actor for one unit	Business	Four of five span several units	`GB-05` is a unit. `Mandate` is undrawable
`058 Governance Gate`	Event	none	A point at which work is assessed	The assessing behaviour is a Process on the body
`059 Compliance Assessment`	Assessment	Motivation	A verdict reached by analysis	**Zero records**, against `ZZ Checks` ' own claim
`060 Governance Decision`	No element	—	The effect is modelled; the decision is not	Four of seven Outstanding; `DEC-02` blocks `WP-03`
`061 Governance Change Request`	No element	—	It is about the model, not in it	The only change mechanism here. Five rows
`063 Control Objective`	Requirement	Motivation	What must be true is a need	Already `DRV-01` – `DRV-06` . `CO-06` : No mechanism
`064 Security Policy`	Principle	Motivation	A board-language commitment	`POL-03` reads Breached, which is unsayable
`065 Enforcement Point`	None of its own — the row names an existing element	varies	A gateway or gate is already modelled	`EP-03` : "Product not yet selected"

`EXT Industry Extension` — six retail object types

`EXT` asks what the TOGAF content metamodel does not: what does retail need that the core cannot hold. Six rows, `X01` – `X06` , third column headed `What it holds and why the core metamodel is not enough` [1]. It is a **declaration**, not a sheet of records — five of the six already have a row on `020 Data Entity` and one has nothing anywhere. It is also the clearest demonstration in the set of why an industry extension exists: four of the six defeat the notation outright, for four different reasons — volume, dating, geometry, and being a role a place plays rather than a thing.

Row	What it holds	ArchiMate 4	Where the records are	What to do
`X01 SKU / Article`	The smallest sellable unit, with trading attributes	Business Object; Product only commercially	`E01` on `O20`, `ATR-001` – `ATR-006`, `REQ-007`, `DBB-01`	One box for the type; attributes in the data model
`X02 Store / Location`	A trading or fulfilment site, with format and capacity	Location; Facility where the building is the subject	`E03` on `O20`, and `O12`, where four rows are formats	Format as a Grouping over Locations; licences have no home
`X03 Assortment`	The ranged set of SKUs for a location and period	Business Object, plus a Grouping for membership	`E04` on `O20`, `ATR-019` – `ATR-024`, `DBB-04`	A snapshot with its as-at date in the title
`X04 Promotion`	A funded offer with mechanics, period and uplift	Process writing a Business Object, not one thing	`E06` on `O20`, `DBB-06`	Add the behaviour; the sheet holds only the record
`X05 Planogram`	Shelf layout by fixture, facing and adjacency	Business Object for the plan; Equipment for the fixture	`E07` on `O20`, `DBB-07`	Do not draw geometry; reference `A07`'s own output
`X06 Fulfilment Node`	A location that can source, pick or hand over an order	No element — a Location with a Capability assigned	**Nowhere.** No `O` sheet holds one	Location plus `C07`, then a new sheet with its own prefix

Each fails for its own reason, and the reason decides what you do instead.

`X01` and volume. A SKU is a Business Object: an identity with a definition, a status and a lifecycle. It is not a Product, because Product means terms, price and a contract, and an article has none of those until a price version attaches to it. The attributes that make an article tradeable — case pack, unit of measure, VAT code, GTIN, shelf life, cost and retail price — are **properties, not elements**, and the notation has no quantity and no money. Draw the article once, as a type, and point at `037 Data Attribute`. Volume 1, chapter 16, *Is the article a Product?*, settles it.

`X03` and dating. An assortment is a dated set: these SKUs, at these locations, for this period. ArchiMate has no effective dating, so an Aggregation from an assortment object to its article objects is true today and silently false next season. The view is a snapshot with its date in the title — "Assortment `E04`, as at 2026-10-01" — and the dated membership lives in the data model. A view without its as-at date is the commonest way a retail model goes quietly wrong.

`X04` and behaviour. The repository holds `E06 Promotion` as a master data entity and nothing else. Volume 1, chapter 16, *A promotion is behaviour, not a thing*, is right and the sheet is thin: a promotion is a funded, approved plan, a mechanic a basket can satisfy, and a discount application recording which offer fired on which line with whose money. Model the plan as a Business Object, the pricing behaviour as a Process reading it, and the application of the offer as a second Business Object that Process writes. The Process has no row on `O16`, so it is authored and you say so.

`X05` and geometry. A planogram is spatial: this facing at this height on this bay next to that category, and the whole value of the artefact is in the positions. ArchiMate has no geometry, no dimension and no adjacency, and `Equipment` gives you the fixture but not what sits where on it. Do not attempt it. Model the plan as a Business Object, the fixture as Equipment, the planning system as `A07 Space &`

`Range Planning`, and treat the planogram as an output of that system referenced from the view. Drawn in ArchiMate it is unreadable and wrong at once.

`X06` and the missing element. A fulfilment node is not a thing. It is a role a place plays: this store can pick for click-and-collect, that depot can ship to a locker, that dark store does both. ArchiMate has no element for it and should not — it is a **Location with a Capability assigned**, and the statement worth drawing is the Assignment from `C07 Order Fulfilment & Returns` to the Location. The repository has no row at all: `LOC-06 Fulfilment Centre` is a place, not a statement about what that place can do, and no sheet records which of the eight locations can fulfil. For a multi-channel retailer that is the most consequential absence in the file — order sourcing is the decision the whole online business turns on, and the model cannot express the options.

> **HOW TO EXTEND `EXT` WITHOUT BREAKING THE MODEL**
>
> Adding a row to `EXT` records an intention and changes nothing. An extension type needs three things: the row on `EXT`, a new `O` sheet with real columns and a `Ref` prefix nothing else uses, and at least one `R` sheet joining it to something already in the model. Add the row alone and no report sees it; add the sheet without the relationship and it is an orphan on day one. Then say which ArchiMate 4 element it maps to, or say plainly that it maps to none — a type whose mapping is undecided gets drawn differently by every architect who meets it.

The relationship mapping — all 25 `R` sheets

The relationship sheets run `R01` to `R25` with no gaps and are named after **what they mean** rather than after their endpoints: `R01 Capability decomposition`, not "R01 CAP-CAP". Many tab names are cut off at Excel's 31-character limit — `R05 Application manages Data En` — and the full sentence sits in the sheet's first cell. Spell the tab as the tab spells it; use the sentence when you explain it.

The sheet's direction is the author's sentence, not an ArchiMate direction. **Fourteen of the 25 reverse when drawn**: a service realises a capability, an outcome realises a goal, an actor is assigned to a role, a location is assigned to the thing standing in it. Draw a reversed one as the sheet reads and the model validates and means the opposite.

Sheet	From → To	ArchiMate 4 relationship	Direction in the model	Note
`R01 Capability decomposition`	Parent → Child	Composition	As sheet	90 instances, three levels. One level per view
`R02 Value stream uses Capabilit`	Value stream → Capability	Serving	**Reversed** — Capability serves Value Stream	Contradicts `O14`'s column on `VS04`, `VS05`
`R03 Capability realised by Busi`	Capability → Business service	Realization	**Reversed** — Service realises Capability	Ten rows, one per level-1 capability
`R04 Business Service supported`	Business service → Application	Realization	**Reversed** — Component realises Service	Positional: `BSV-02` to `A02`, when `A08` prices
`R05 Application manages Data En`	Application → Data entity	Access	As sheet	`Role` sets the type. 14 rows hold a name, not a `Ref`
`R06 Application interfaces with`	Source → Target	Flow	As sheet	29 rows, nine of them Direct DB read
`R07 Application deployed on Tec`	Application → Platform component	Assignment	**Reversed** — Node assigned to Component	The platform is assigned to what runs on it
`R08 Requirement derives from Dr`	Requirement → Driver	Influence	**Reversed** — Driver influences Requirement	31 rows, the cleanest trace in the file
`R09 Requirement satisfied by Bu`	Requirement → Building block	Realization	**Reversed** — Grouping realises Requirement	`Traced` is what `RPT06` reads
`R10 Objective measured by Measu`	Objective → Measure	**Not expressible**	—	Measure is not an element. See `RPT01`
`R11 Goal realised by Objective`	Goal → Objective	Realization	**Reversed** — Outcome realises Goal	Drawn as written, the whole chain inverts
`R12 Gap closed by Work Package`	Gap → Work package	Association	As sheet	Not Realization. **One instance**
`R13 Work Package delivers Build`	Work package → Building block	Realization	As sheet	37 rows, one per block. Nothing is lost

Sheet	From → To	ArchiMate 4 relationship	Direction in the model	Note
`R14 Project delivers Work Packa`	Project → Work package	Composition	As sheet	Both ends Work Package, one to one
`R15 Transition contains Work Pa`	Transition → Work package	Realization	**Reversed** — Work Package realises Plateau	Five of 12 packages sit in no transition
`R16 Control Objective met by Co`	Control objective → Control	Realization	**Reversed** — control realises objective	Both ends Requirements. The pairing is positional
`R17 Policy enforced at Enforcem`	Policy → Enforcement point	Realization where it enforces, else Association	**Reversed** — the point realises the policy	Four rows; three record a failure
`R18 Standard applies to Applica`	Standard → Application	Realization where conformant, else Association	**Reversed** — the component realises it	Most read Partial, so most earn Association
`R19 Data Entity relates to Data`	From → To	Association	As sheet	Identical to `036`. `Cardinality` is lost
`R20 Application located at Loca`	Application → Location	Assignment	**Reversed** — Location assigned to Component	The place is assigned to what stands in it
`R21 Role performed by Actor`	Role → Actor	Assignment	**Reversed** — Actor assigned to Role	All 18 pair identically named rows
`R22 Org Unit owns Capability`	Org unit → Capability	Assignment	As sheet	Positional. The other 90 have no owner
`R23 Increment delivered by Proj`	Increment → Project	**Not expressible** as drawn	—	No Increment element. Substitute a Grouping
`R24 Compliance Assessment tests`	Assessment → Building block	Association	As sheet	`Verdict` stays in the sheet. **Zero instances**
`R25 Application supports Produc`	Product → Application	Serving	**Reversed from the tab name**	The tab and the columns run opposite ways

WHAT YOU CANNOT DRAW, AND WHAT TO SAY INSTEAD

`R10` and `R23` have no ArchiMate 4 expression at either end. `R24` is expressible and empty; `R12` is expressible and holds one row where it should hold dozens. `R16` and `R17` are expressible only by substituting Requirements and Principles for controls and policies. Six of 25 sheets, then, where a reader of the views alone reaches a wrong conclusion — that no measure tracks an objective, that no increment is delivered, that this estate has one gap. Name the missing sheet in the view's notes. "This view cannot show whether `OBJ-10` is moving; see `MSR-10` on `004`, baseline 67%" is one line and it saves a meeting.

Three absences you will meet on your first morning, because the view you want needs an edge no sheet carries. **No driver to goal:** `R08` runs requirement to driver and `R11` goal to objective, and nothing joins a driver to a goal — the first edge of every motivation view ever drawn. **No role to process:** `R21` pairs roles with actors and that is all, and the RACI that volume 1, chapter 19, *Process to role, as RACI*, pre-drafts has nowhere to land; in a retailer it is the first thing the store operations director asks for. **No application to information system service:** `O24`'s `Application` column holds it and no sheet does, and the same is true of `O25` and `O28`, which no sheet references at all. Author each, label it authored, raise the missing sheet on `O61`.

The view set, by ADM phase

For each phase: the views to draw, what they hold, the sheets that populate them, and who asks. Where a standard ArchiMate 4 viewpoint fits it is named; where none fits the view is marked *named here* — yours to define and yours to defend.

Preliminary

View	Elements and relationships	Populated from	For
Organisation (Organization viewpoint)	Business Actors, Roles, Aggregation, Assignment	`009`, `010`, `011`, `R21` reversed	The sponsor
Principles (Motivation viewpoint)	24 Principles in five Groupings by `Category`	`005`	`GB-01 Architecture Board`
Governance (*named here*)	Collaborations for bodies, Events for the four gates	`057`, `058`	`GB-04 Executive Committee`

Phase A — Architecture Vision

View	Elements and relationships	Populated from	For
Motivation (Motivation viewpoint)	Drivers, Goals, Outcomes, Principles, Requirements	`001` – `006`, `R08` and `R11` reversed	The sponsor, `GB-02`
Goal realisation (Goal Realization viewpoint)	One Goal, its Outcomes, the Requirements behind them	`002`, `003`, `006`, `R11` reversed	Six goals, six views
Capability map, level 1 (Capability Map viewpoint)	Ten Capabilities shaded `Current` against `Target`	`013`, `R01`, `R22`, mirrors `RPT03`	`GB-02 Investment Committee`
Value streams (Value Stream viewpoint)	Five Value Streams, the Capabilities serving each	`014`, `013`, `R02` reversed	The Chief Operating Officer
Solution concept (Layered viewpoint)	One trading format, motivation to technology, 20 elements	Everything, filtered hard	The board

Phase B — Business Architecture

View	Elements and relationships	Populated from	For
Business process co-operation	Processes, Triggering between stages, the Services realised	`016`, `015`, `R03` reversed	Store and supply chain operations
Product (Product viewpoint)	`PRD-05` aggregating Services; the ranges as Groupings	`018`, `015`, `019`, `R25`	Commercial and compliance
Capability map, levels 1–3	100 Capabilities with owners and realising Services	`013`, `R01`, `R22`, `R03` reversed	`GB-02 Investment Committee`
Role and authority (*named here*)	Roles and the Processes each is accountable for	`011`, `016`, **and an authored edge**	The Chief Risk Officer

Phase C — Data Architecture

View	Elements and relationships	Populated from	For
Information structure (Information Structure viewpoint)	14 Data Objects and the Associations between them	`020`, `R19`	The Chief Data Officer
Authority (*named here*)	Access edges marked Master, with the stated golden source	`020`, `043`, `R05`, mirrors `RPT05`	`GB-03 Data Governance Council`
Lineage (*named here*)	The reporting entity and the Artifacts back to source	`020`, `040`, `042`, `R05`	Finance and group reporting

Phase C — Application Architecture

View	Elements and relationships	Populated from	For
Application co-operation (Application Co-operation viewpoint)	12 Components, 29 Flows, dashed where `Managed` reads Unmanaged	`O23`, `R06`, mirrors `RPT04`	The integration lead
Application usage (Application Usage viewpoint)	Components realising Services, Services serving Processes	`O23`, `O24`, `O15`, `R04` reversed	The Chief Information Officer
Application and data (*named here*)	Access edges typed Master and Consumer	`R05`, mirrors `RPT05`	Data and application architects
Portfolio (*named here*)	Five Groupings by `Disposition`, labelled `Health`	`O23`, mirrors `RPT02`	`GB-02 Investment Committee`
Direct database reads (*named here*)	The nine `R06` rows typed Direct DB read, as Access	`R06`, `O22`	`GB-01`, at its first gate

Phase D — Technology Architecture

View	Elements and relationships	Populated from	For
Technology (Technology viewpoint)	Nodes, Devices, System Software, Services, Realization	`O25`, `O26`, `O27`	The infrastructure lead
Deployment (Implementation and Deployment viewpoint)	Nodes at Locations, Paths between sites, Environments as Groupings	`O27`, `O12`, `O48`, `O50`, `R20` reversed	Store systems and operations
Store estate (*named here*)	One format as a Grouping: till Nodes, the Path inward, what survives a link loss	`O12`, `O27`, `O50`, `LTC-01`	The store operations director
Currency (*named here*)	Nodes shaded by `Position`, `Support end` on the label	`O27`, mirrors `RPT10`	The CIO and the Chief Risk Officer

Phase E — Opportunities and Solutions

View	Elements and relationships	Populated from	For
Gaps and building blocks (*named here*)	Assessments for gaps, Groupings for blocks, Association between	`O29`, `O55`, `R12`	The programme board
Building block coverage (*named here*)	Blocks with the package realising each, the unfunded alone	`O55`, `O30`, `R13`	`GB-02 Investment Committee`
Architecture alternatives (Strategy viewpoint)	Five Courses of Action against the Goals each reaches	`O54`, `O02`	The board, once
Transition overview (Migration viewpoint)	Three Plateaus, the packages realising each, Triggering between	`O53`, `R15` reversed	The sponsor

Phase F — Migration Planning

View	Elements and relationships	Populated from	For
Implementation and migration (Implementation and Migration viewpoint)	Work Packages realising Plateaus and Deliverables	`030`, `031`, `053`, `R15` reversed	The programme board
Sequencing (*named here*)	Triggering between packages, dates labelled, the peak freeze marked	`030`, `052`, mirrors `RPT08`	The programme director
Increment (*named here*)	A Grouping per increment against the Plateau it lands on	`066`, `R23` by substitution	The architecture function

Phase G — Implementation Governance

View	Elements and relationships	Populated from	For
Compliance (*named here*)	Assessments against block Groupings at gate Events	`059` once populated, `058`, `055`, `R24`	`GB-01 Architecture Board`
Enforcement (*named here*)	Principles for policies, Realization to the points enforcing them	`064`, `065`, `R17` reversed	The CISO
Regulatory coverage (*named here*)	Drivers, Requirements for control objectives, the controls behind	`001`, `063`, `041`, `R16` reversed, mirrors `RPT09`	Internal audit, the inspector

Phase H — Architecture Change Management

View	Elements and relationships	Populated from	For
Baseline against target (*named here*)	Two Plateaus and the classified differences between them	The comparison tool, then `029`	The Chief Enterprise Architect
Requirements realisation (Requirements Realization viewpoint)	Requirements, the blocks realising them, the ones reading `Traced` No	`006`, `R09` reversed, mirrors `RPT06`	Internal audit, the Chief Risk Officer
Orphans (*named here*)	Every element with no edge, on one page, with nothing else	`RPT13`, plus `025`, `028`, `038`	The architecture function, monthly

> **FIVE VIEWS, NOT THIRTY-SEVEN**
>
> Start with the Phase A motivation view, the level-1 capability map, the application co-operation view, the authority view and the transition overview. Those five answer most of what gets asked in the first six months, each is a filter over sheets already populated, and each fits on a page. Draw one of the rest when somebody asks a question that needs it.

Four views from the repository

Each is built from rows that exist in `Retail_TOGAF_Metamodel.xlsx`, with their `Ref`s. Where an edge had to be authored because no sheet carries it, the figure says so — usually the most useful thing the picture has to tell you.

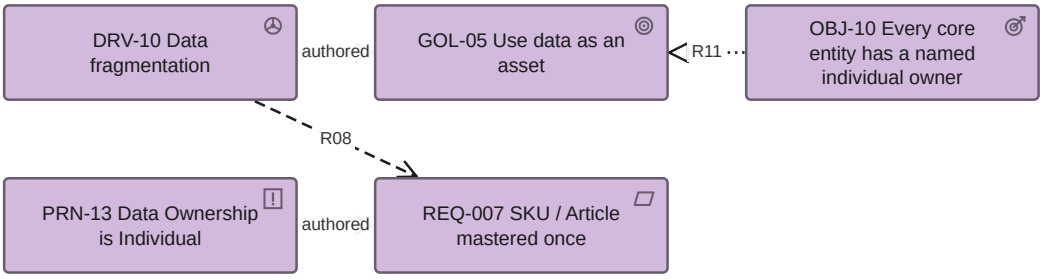


Figure 16.1 Motivation chain for Halverson Stores Group from O01, O02, O03, O05 and O06. Only two of the four edges come from a relationship sheet: R08 and R11, both drawn against the direction the sheet reads.

`DRV-10 Data fragmentation` influences `GOL-05 Use data as an asset`; `OBJ-10 Every core entity has a named individual owner` realises that goal, which is `R11` drawn backwards; `REQ-007 SKU / Article mastered once` is influenced by `DRV-10`, which is `R08` drawn backwards; and `PRN-13 Data Ownership is Individual` shapes the requirement. Two of the four edges are authored, because no driver-to-goal sheet and no principle-to-requirement sheet exists, so a generated motivation view arrives with half its edges missing and an architect fills them in by hand every quarter. Note the absence too: `MSR-10 Entities with a named owner` stands at a baseline of 67%, and no element can carry the number.

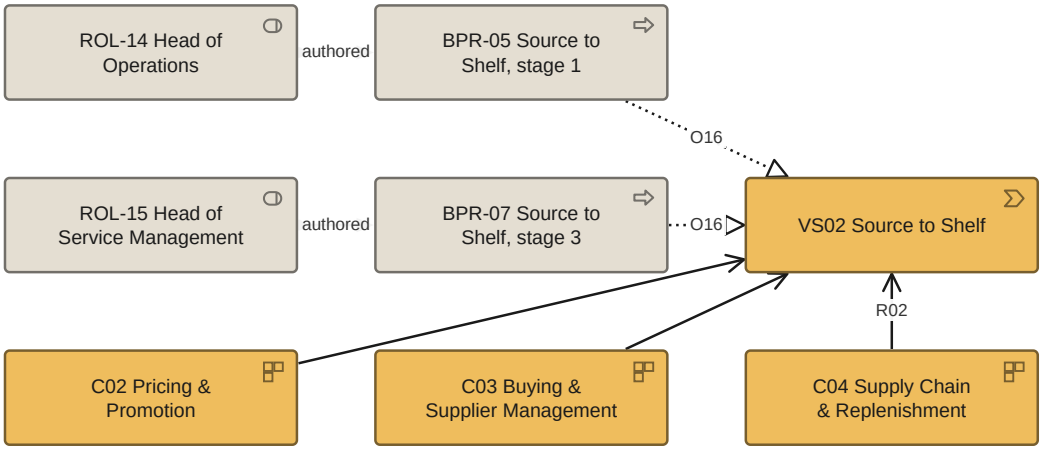


Figure 16.2 VS02 Source to Shelf with the three capabilities serving it and two of its five process stages. The Serving edges are R02 reversed; the Assignment edges are authored, because no sheet in this repository joins a role to a process.

`VS02 Source to Shelf`, the three capabilities serving it, two of its five process stages and two roles. The Serving edges are `R02` reversed and are the only edges here from a relationship sheet; the

process-to-stream edges come from the `Value stream` column of `O16`. The Assignment edges from `ROL-14` and `ROL-15` are **authored**, because nothing joins a role to a process. That is the figure's real content: the view a chief operating officer most wants is the one the repository is least able to generate, and the fix is a new relationship sheet, not a better drawing tool.

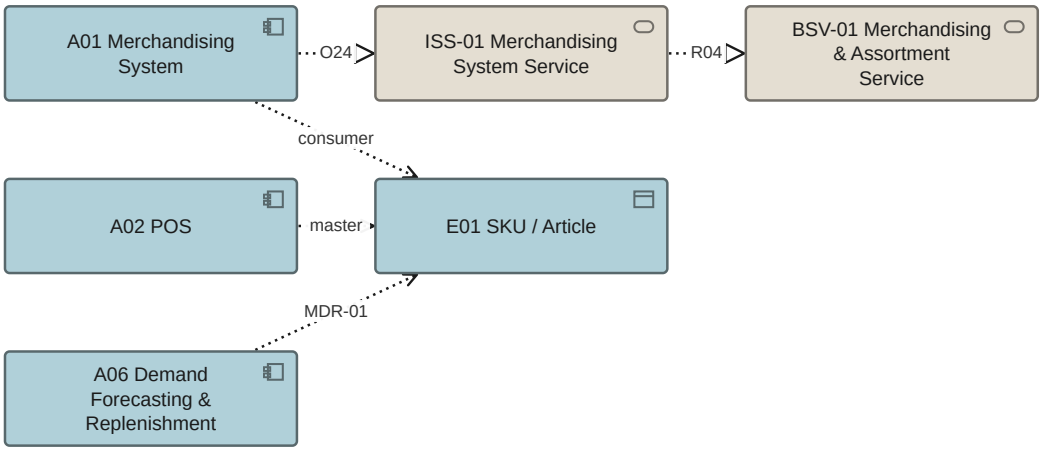


Figure 16.3 A01 Merchandising System through two Services that are the same element, and E01 SKU / Article mastered twice. O20 and R05 both name POS as master; MDR-01 on O43 names Demand Forecasting and Replenishment as the golden source.

`A01 Merchandising System` realises `ISS-01 Merchandising System Service`, which realises `BSV-01 Merchandising & Assortment Service`. Both services are **Service** in ArchiMate 4, so the `Ref`s on the labels do the work colour used to do; the first edge is the `Application` column of `O24` and the second is `R04` reversed. Then the point of the figure. `E01 SKU / Article` is mastered twice. `O20` names `POS` as its `Master system` and `R05` agrees, recording `POS` as Master and `A01` as Consumer; `MDR-01` on `O43` names `Demand Forecasting & Replenishment` as the `Golden source` [2]. Three sheets, two answers, and the notation cannot express contested authority — every Access edge looks alike. What the view does is make the count visible, and the count is the finding: all ten rows of `O43` disagree with `O20` the same way. In a retailer this is the article master question, and it decides whether the shelf edge, the till and the website can ever agree.

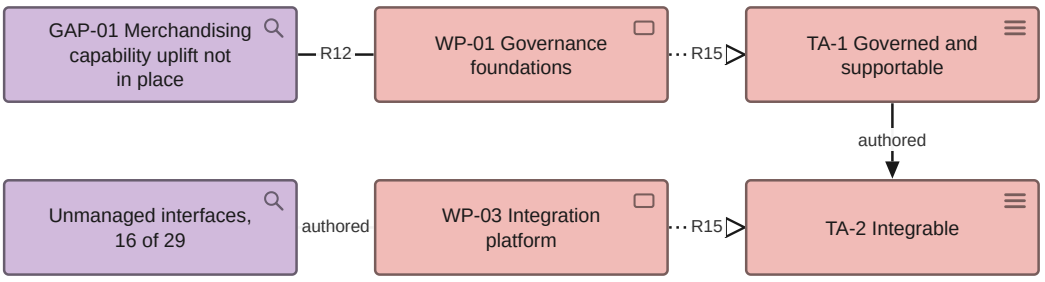


Figure 16.4 The single gap the repository holds, one finding from ZZ Checks carried as an Assessment, the two packages that address them, and two of the three plateaus. The gap-to-package edge is R12; the second Assessment has no Ref because no row exists to carry one.

Two Assessments, two Work Packages, two Plateaus. `GAP-01 Merchandising & Assortment capability uplift not in place` is the only gap the repository holds, and `R12` carries its single instance to `WP-01 Governance foundations`. The second Assessment comes from `ZZ Checks`, which counts 16 unmanaged interfaces out of 29 against a threshold of zero — a real finding with a real number and no `Ref`, because no row exists to carry one. The package-to-plateau edges are `R15` reversed. The Triggering between `TA-1 Governed and supportable` and `TA-2 Integrable` is authored: `O53` has no sequence column. The view cannot say that `WP-03 Integration platform` is still only In flight for approval, that seven of twelve packages read Past end date, or that `ACO-03` records its contract as Non-compliant. All three are in the repository and none is drawable. And the first instruction this figure gives is not about drawing at all: populate `O29`.

Keeping views and the repository in step

Four rules, short because they are either kept or the arrangement collapses inside two quarters.

Regenerate, never hand-maintain. A view is a query result with a layout. When a row changes, re-run the query. Hand-editing a view to match a changed sheet works twice and fails the third time, and nobody notices until a committee decides something on a stale picture.

The view carries the `Ref`, not just the name. `WP-02 Data ownership and mastering`, not "data ownership". Names get edited in `O30` without anybody's approval; `Ref`s do not. And this repository is full of same-named rows across sheets — 18 actors against 18 roles, ten functions against ten capabilities, three increments against three transitions, seven reference model entries against seven logical technology components. A label without a `Ref` is ambiguous four ways at once.

An element the repository does not hold is a change request waiting to be raised. Not an error to fix quietly — a row on `O61 Governance Change Request` with an `Against` and a `Type`, so the decision to add it is made by somebody entitled to make it. `CR-G03 Raise unfunded building blocks as gaps` is that mechanism working correctly, and it is one of five rows where there should be dozens.

And the reverse: a record on no view is a question, not a problem. `RPT13 Orphan Analysis` names two — `A11 Supplier Portal` and `A12 Retail Analytics`, neither with a business service mapping to it — and misses every row of `O25`, `O28` and `O38`, which no relationship sheet references. Some of those rows are unnecessary; some are unrealised, which is the finding. Establish which before deleting anything.

Then the test, which runs in a minute. Take a published view, list its element `Ref`s, look each up on the sheet its prefix implies. Check that every relationship appears on its `R` sheet with the same two endpoints and that every reversal above has been applied. Run `ZZ Checks` and confirm the counts have not moved since the view was drawn. If that holds, the view is a true statement about the repository. If it does not, you do not have a disagreement. You have two architectures.

THE CHAPTER IN TEN LINES

- The repository is the record and an ArchiMate 4 view is the argument; where they disagree the repository wins and the view is wrong, not a variant.
- Every element on a view carries its `Ref`, because this repository has no change control and the `Ref` is the whole of the audit trail.
- Sixty-four sheets are prefixed `O`; with `EXT` that is the 65 object types the index counts. `O34` and `O51` are real absences, not an export error.
- Twenty-two of the 64 `O` sheets have no ArchiMate 4 element of their own; nine carry the fact another way and thirteen stay in the repository.
- A row is a type, not an instance. `E01 SKU / Article` is one box standing for several hundred thousand articles, and the notation has no quantity, no money and no date.
- `EXT`'s six types fail the notation for four reasons: instance volume, effective dating, geometry, and being a role a place plays rather than a thing.
- `X06 Fulfilment Node` has no row anywhere in the repository. For a multi-channel retailer that is the most consequential absence in the file.
- Fourteen of the 25 relationship sheets reverse when drawn: `R02`, `R03`, `R04`, `R07`, `R08`, `R09`, `R11`, `R15`, `R16`, `R17`, `R18`, `R20`, `R21` and `R25`.
- `R10` and `R23` cannot be drawn at all; `R24` is drawable and empty, and `R12` holds one row where it should hold dozens.
- Three edges every architect wants are on no sheet: driver to goal, role to process, and application to information system service.

SOURCES FOR THIS CHAPTER

- **1.** Retail_TOGAF_Metamodel.xlsx — 105 sheets: 00 Index, 64 object sheets O01–O66 (O34 and O51 absent), EXT Industry Extension, 25 relationship sheets R01–R25, 13 reports RPT01–RPT13 and ZZ Checks
- **2.** Retail_TOGAF_Metamodel.xlsx, sheets O20 Data Entity, O43 Master Data Rule and R05 Application manages Data Entity — the contested mastering of E01 SKU / Article
- **3.** The Open Group, TOGAF Standard, 10th Edition — Architecture Development Method https://pubs.opengroup.org/architecture/togaf-standard/
- **4.** The Open Group, ArchiMate Forum — language overview https://www.opengroup.org/archimate-forum/archimate-overview

APPENDIX E

Where to look in volume 1

The volume read backwards: start from the step you are stuck on and find the chapter and section where volume 1 already worked the judgement through.

This appendix runs the other way round from the rest of the volume. Everywhere else you start from a phase and work forwards. Here you start from the step you are stuck on and work back to the place in volume 1 where the judgement behind it has already been argued through.

Volume 1 is the why; this volume is the what, the where and who signs. Work the steps without the teaching and you will fill cells correctly and defend them badly — which is worse, because a wrong cell gets corrected and a badly defended right one gets overruled.

In the step index the third column gives the volume 1 **chapter number** and then, in italics, the **section heading inside that chapter**. Every heading here has been checked against that chapter's text. A dash means volume 1 has nothing to say about that step, which is honest and common: **64 of the 159 steps carry a dash**, two in every five. The repository, the comparison tool and the gate regime are this volume's own subject, and volume 1 does not know they exist.

Where volume 1 carries pre-drafted content — chapter 19, appendix A — its identifiers are its own. `P-01` – `P-12`, `C-01` – `C-20`, `R-01` – `R-33`, `D-01` – `D-20` and `A-01` – `A-25` are teaching sets. The repository runs `C01` – `C10` decomposed to `C10.02.02`, 20 `BPR-` rows, 18 `ROL-`, 14 `E` entities and 12 `A` applications. The two schemes look similar enough to tempt a join and they do not line up. **Match on name. Never on number.** `A08` in volume 1's conceptual model and `A08 Price & Promotion Engine` in `O23 Application Component` are different things, and a step that gets this wrong produces a traceability table that is confidently false.

Halverson Stores Group, the retailer inside the repository, and Milkwood Retail Group, volume 1's, are both inventions and their data is not interchangeable.

The step index

159 steps, in the order you would meet them.

Chapter 2 · Standing the practice up (Preliminary)

Step	What it asks you to decide	Volume 1 chapter and section
2.1	Which formats and ownership models are inside the boundary	2 · *The ownership models, and what each does to the architecture*
2.2	Whether the record can be changed safely at all	—
2.3	What the baseline is, and the date it was cut	—
2.4	Which integrity defects to fix before anything else	18 · *Six numbers that tell you whether the estate is healthy*
2.5	Which of the 64 object sheets you will actually fill	—
2.6	What retail needs that the core metamodel cannot hold	9 · *Article, style, option, SKU: the argument the model lives or dies on*
2.7	Whether to add a seventh extension type, and how	16 · *The mapping*
2.8	What an `Owner` cell is allowed to contain	—
2.9	Which of the 24 principles survive	15 · *What an inspector actually asks to see*
2.10	Who may decide what, and who may not	5 · *Your real counterparties*
2.11	What each gate tests, and who holds it	—
2.12	Where the architecture function sits in its own model	—
2.13	How good the practice is today	7 · *The value chain is six clocks, not six steps*
2.14	Whether the motivation layer joins up end to end	6 · *The four numbers a retail executive watches*
2.15	Whether the Preliminary Phase is closed	—

Chapter 3 · Architecture Vision (Phase A)

Step	What it asks you to decide	Volume 1 chapter and section
3.1	Who is asking, and what authority they are granting	18 · *What to refuse in month one*
3.2	Whether an actor and a role are the same list	5 · *The decision map*
3.3	Which stakeholders a retailer's map always misses	5 · *Buying and merchandising*
3.4	Whose agreement the vision needs, and what each is measured on	6 · *The four numbers a retail executive watches*
3.5	How the group makes money, on one page	4 · *The canvas for a multi-format retail group*
3.6	Where value begins and where it lands	7 · *The value chain is six clocks, not six steps*
3.7	Breadth, depth, horizon and domain scope	2 · *What the format mix does to the application estate*
3.8	What you cannot change, and until when	7 · *The trading calendar*
3.9	What you are guessing, and when you will know	12 · *First question: who owns the store?*
3.10	The one-page argument for changing	17 · *The motivation view: why this group is changing*
3.11	What the architecture function will and will not do	18 · *The three artefacts that buy you credibility*
3.12	Whether Phase A earned its mandate, and when it can be heard	5 · *The trading calendar governs every diary in the building*

Chapter 4 · Business Architecture (Phase B)

Step	What it asks you to decide	Volume 1 chapter and section
4.1	Which streams and which capability levels Phase B covers	—
4.2	The organisation as fact, with format and ownership model on it	2 · *The ownership models, and what each does to the architecture*
4.3	Which roles a retailer actually staffs	19 · *Roles*
4.4	Where the buyer's authority ends and the planner's begins	5 · *The distinction outsiders always collapse*
4.5	Where value begins, and which of two records to believe	7 · *The value chain is six clocks, not six steps*
4.6	Whether a row is a capability at all	19 · *Business capabilities*
4.7	One named accountable owner per capability	—
4.8	How mature each capability really is	6 · *The four numbers a retail executive watches*
4.9	What is a function and what is a process	16 · *Is it a service or a process?*
4.10	Which level the process inventory sits at	8 · *The L1 process map*
4.11	Whether a service is more than a renamed capability	16 · *Is it a service or a process?*
4.12	What `018` is for, now that it is not the article master	16 · *Is the article a Product?*
4.13	Which contracts the sheet left out	6 · *Supplier income*
4.14	How the article is identified, and what it hangs from	9 · *Article, style, option, SKU: the argument the model lives or dies on*
4.15	What a store is, as against a format	2 · *A format is an economic machine, not a shop size*
4.16	Whether assortment is a decision or a list	9 · *The six hierarchies, all of them correct*
4.17	Whether Phase B can be handed on	—

Chapter 5 · Data Architecture (Phase C)

Step	What it asks you to decide	Volume 1 chapter and section
5.1	How many entities are in scope, at which layer	9 · *The afternoon that produces one*
5.2	Whether style, option and SKU are one entity or three	9 · *Article, style, option, SKU: the argument the model lives or dies on*
5.3	Which entities the catalogue is missing	19 · *Conceptual data entities*
5.4	Cardinality that carries the business rule	9 · *The buying chain, which is not a sequence*
5.5	Which logical store holds each entity, and which physical one	10 · *The analytics layer, and the object everybody draws too small*
5.6	Which attributes are worth recording	10 · *Attributes, where they earn their place*
5.7	Who owns each entity, and which system is authoritative	10 · *Ownership, lineage and the gate that actually works*
5.8	Which of two sheets is right about the golden source	10 · *Reference and master data, and the five distribution channels*
5.9	What metadata the group holds, and what it does not	10 · *What a single version of the truth has to mean*
5.10	What a reporting set must contain to be reusable	6 · *Like-for-like, and why the definition is a data-model requirement*
5.11	Classification, lawful basis and retention, entity by entity	15 · *POPIA, with the retail bite*
5.12	Whether the shelf label and the till can be made to agree	9 · *Price is an entity, not an attribute*
5.13	Which identifier scheme governs which product	2 · *Two identifier schemes for the same product*
5.14	Which application masters which entity, with a usable key	19 · *Application to data, with the system of record*
5.15	Which data has to move, and under what rule	10 · *The restatement problem*
5.16	Whether the data layer can be handed to the application phase	—

Chapter 6 · Application Architecture (Phase C)

Step	What it asks you to decide	Volume 1 chapter and section
6.1	Which conceptual boxes the estate actually fills	11 · *How to use it*
6.2	Every application anyone in the group can name	—
6.3	Owner and criticality, defensible in one sentence	—
6.4	Which five columns the sheet does not have	—
6.5	The applications nobody declared	6 · *Supplier income*
6.6	What each application is worth, on both axes	—
6.7	Invest, retain, consolidate, replace or retire	—
6.8	Which automated services exist, and which are published	—
6.9	Which application supports which business service	—
6.10	Where coverage is thin and support is duplicated	19 · *Capability to application — the coverage map*
6.11	Which application is the system of record	10 · *Ownership, lineage and the gate that actually works*
6.12	How many interfaces are unmanaged, and how many unlogged	12 · *Integration, as it really is*
6.13	Which requirement each application answers	—
6.14	Conformance, application by application	13 · *ARTS, the NRF and the OMG Retail Domain Task Force*
6.15	Whether the estate matches the reference models	—
6.16	What the portfolio catalogue says publicly	—

Chapter 7 · Technology Architecture (Phase D)

Step	What it asks you to decide	Volume 1 chapter and section
7.1	Which technical capabilities the estate offers	—
7.2	What each application actually runs on	—
7.3	The vendor-independent components, across three sheets that disagree	—
7.4	Support dates taken from the contract, not the slide	14 · *Store inventory, point of sale and store systems*
7.5	Which platform services are standard, and who consumes them	—
7.6	Which specifications are governed, and where conformance lives	—
7.7	What goes out of support, and what that buys you	7 · *Channels, formats, and what each costs to serve*
7.8	What a site is, physically, and which sheet holds it	—
7.9	What each environment holds, including personal data	15 · *POPIA, with the retail bite*
7.10	What each network link carries, once the duplicates are gone	—
7.11	What a store can still do when the line drops	7 · *The store has to keep selling*
7.12	Recovery objectives, and whether they have been tested	7 · *What load-shedding did*
7.13	Which part of the estate is in card scope	—
7.14	Which controls exist, and which are absent	10 · *Classification, consent and POPIA*
7.15	Whether Phase D can be signed off	—

Chapter 8 · Opportunities and Solutions (Phase E)

Step	What it asks you to decide	Volume 1 chapter and section
8.1	What the comparison tool needs, produced by hand	—
8.2	Which question the comparison is answering	—
8.3	Which records appear, vanish or get renumbered	—
8.4	What the tool thinks each difference is	15 · *What an inspector actually asks to see*
8.5	Whether each difference was governed	—
8.6	How severe each gap candidate is	—
8.7	Which capability gaps the tool cannot see	19 · *Capability to application — the coverage map*
8.8	The five capabilities a retailer is usually missing	11 · *The patterns*
8.9	Which integrity findings become gaps with numbers attached	10 · *Ownership, lineage and the gate that actually works*
8.10	The gap numbering scheme, before the first row	—
8.11	What must exist, before who supplies it	14 · *Build, buy or configure*
8.12	Which package delivers which block, exactly once	—
8.13	What gets funded together	12 · *The legacy question*
8.14	Which states you can stop at safely	17 · *Gaps, and the roadmap in two plateaus*
8.15	Which three options the board chooses between	12 · *The South African anchor, and what it cost*
8.16	What the roadmap says, and what it shows	6 · *The four numbers a retail executive watches*

Chapter 9 · Migration Planning (Phase F)

Step	What it asks you to decide	Volume 1 chapter and section
9.1	What becomes a chartered project, and who leads it	5 · *The trading calendar governs every diary in the building*
9.2	What will shape delivery whatever you plan	—
9.3	The sequence, and the dependency the sheet cannot hold	—
9.4	Which weeks the business will actually give you	7 · *The trading calendar*
9.5	How much change a delivery year and a night can take	7 · *The batch clock*
9.6	Where each cost number came from, in one currency	14 · *What a merchandising or point-of-sale replacement costs, and how long it takes*
9.7	Whether a benefit traces to a measure	6 · *The four numbers a retail executive watches*
9.8	What is true at the end of each increment	16 · *The roadmap, and what is true on 31 December*
9.9	The consequence of each delivery risk	12 · *The South African anchor, and what it cost*
9.10	Which project a constraint or an assumption holds up	—
9.11	How each entity moves, and how history survives the move	9 · *Price is an entity, not an attribute*
9.12	Field to field, with the rule written out	10 · *Classification: the codes that decide what is lawful*
9.13	Whether `Delivery` and `End` can both be true	—
9.14	Whether the plan is funded	18 · *The three artefacts that buy you credibility*

Chapter 10 · Implementation Governance (Phase G)

Step	What it asks you to decide	Volume 1 chapter and section
10.1	Whether the repository can be governed at all	—
10.2	Who holds each gate, and on what evidence	—
10.3	What the delivery organisation signed up to	—
10.4	Whether the criteria were inherited or invented	16 · *Capabilities, value streams and the scarcest resource*
10.5	Whether the design will meet the criteria	13 · *How to choose, and how to tell when a model is standing in for thinking*
10.6	A verdict per criterion, never per project	—
10.7	Which obligations have no mechanism behind them	16 · *A regulation, from driver to control*
10.8	What each body may not decide	—
10.9	What is excused, by whom, until when	—
10.10	Which board outranks the architecture board, and when	7 · *The trading calendar*
10.11	Which columns evidence the board's own duty	15 · *Packaging, empowerment and the board*
10.12	Which obligations are testable at a gate	15 · *What an inspector actually asks to see*
10.13	What the gate calendar holds this quarter	16 · *The roadmap, and what is true on 31 December*

Chapter 11 · Architecture Change Management (Phase H)

Step	What it asks you to decide	Volume 1 chapter and section
11.1	Whether a change may be made at all	—
11.2	The before and after of every field	—
11.3	Which body decides, and on what papers	—
11.4	Simplification, incremental or re-architecting	9 · *The six hierarchies, all of them correct*
11.5	What the two extracts must contain	—
11.6	What changed with no change request behind it	—
11.7	When the sweeps run, and off what trigger	7 · *The trading calendar*
11.8	When a baseline is worth cutting	—
11.9	The three health numbers and their targets	—
11.10	Which maps a process or system change invalidates	19 · *Keeping these maps alive*
11.11	Which retail events force a change cycle	2 · *The ownership models, and what each does to the architecture*
11.12	What to do about a support expiry inside a year	12 · *The legacy question*
11.13	What the board is asked for, once a year	—

Step	What it asks you to decide	Volume 1 chapter and section
12.1	What the inherited register actually contains	—
12.2	Whether it can be tested at a gate	—
12.3	Which objective and driver the requirement serves	16 · *A regulation, from driver to control*
12.4	Which requirements are preferences	—
12.5	Which principle governs it, and how	—
12.6	Which principles govern nothing	—
12.7	What the instrument actually requires of a system	15 · *Section 23, and why a stale shelf label is a legal exposure*
12.8	Which South African instrument, not which category	15 · *Tax, credit, measurement and the card estate*
12.9	Who is funded to deliver it	—
12.10	Traced against ever tested	—
12.11	Whether to accept it into this phase	—
12.12	Which rows are no longer live	—

> **WHERE THE DASHES CLUSTER**
>
> They are not spread evenly. The heaviest are chapters 12 at 9 of 12, 6 at 10 of 16, 11 at 8 of 13, 7 at 9 of 15. Those four are where this volume works alone: the requirements register nobody inherited in a usable state, the portfolio arithmetic, the change-control spine the repository does not have, and the repair of columns that contradict each other. Chapter 3 is the lightest, because Phase A is an argument about the business and volume 1 is an argument about the business. If you are stuck in one of the heavy four, stop looking in volume 1 and read the step again.

The reverse map

The same thing from the other end: each of volume 1's 22 chapters, where this volume draws on it, and the one thing to carry away if you read nothing else in it. Chapter numbers in the middle column are **workbook** chapters.

Volume 1 chapter	Workbook chapters that use it	The one thing to take from it
1 · *The industry in one chapter*	—	The three grains everything hangs from, and the arithmetic that governs the rest. Read it again the week before Phase A, not only on day one.
2 · *The formats and lines of business*	2, 3, 4, 5, 6, 11	A format is an economic machine, not a shop size, and the ownership model decides what you are even allowed to architect.
3 · *The vocabulary*	4, 5	The words that mean different things in the same retailer. Half of all catalogue arguments are vocabulary arguments wearing a model's clothes.
4 · *How the business model works*	3	The canvas and the four customer journeys, which is where value streams come from and where the repository's five do not reach.
5 · *Who does what*	2, 3, 4, 9, 10	The distinction between the buyer and the planner, and who may commit money. The org chart will not tell you and `009` has nowhere to put it.
6 · *How retailers make money*	2, 3, 4, 5, 6, 8, 9	The four numbers an executive watches. A benefit that does not land on one of them is not a benefit.
7 · *The operating model*	2, 3, 4, 6, 7, 9, 10, 11	The trading calendar and the batch clock. Between them they decide which weeks your plan is allowed to have and how long the night is.
8 · *The process model*	4	L1 and L2 are different artefacts. Loading L3 into `016` is the commonest Phase B error, and `016`'s 20 rows are not an inventory.
9 · *The conceptual data model*	2, 4, 5, 9, 11	Article, style, option, SKU — the argument the model lives or dies on, and the six hierarchies that are all correct at once.
10 · *The information model*	5, 6, 7, 8, 9	Ownership and lineage, and the test for a master system. `020` and `043` disagree on all ten rows and this chapter is how you adjudicate.
11 · *The conceptual application model*	6, 8	The twenty-five boxes that must exist before any vendor is named, and the patterns that say which are always missing.
12 · *The application landscape*	3, 6, 7, 8, 9, 11, appendix D	How these systems actually talk, and the batch clock behind it. The estate you are cataloguing, before you catalogue it.
13 · *The reference architecture*	6, 7, 9, 10	Retail has no BIAN, GS1 is the standards body that genuinely binds you, and Sunrise 2027 is a dated obligation in `032`.
14 · *Technology providers*	7, 8, 9	What rots first, what a merchandising or point-of-sale replacement really costs, and the corrections the market gets wrong.
15 · *Regulation, risk and what the regulator inspects*	2, 5, 7, 8, 9, 10, 12	The obligations in substance. This workbook records six of them as rows on `063`; it does not teach them, and six rows is not enough.
16 · *Modelling retail in ArchiMate*	2, 4, 5, 7, 8, 9, 10, 12, appendix D	Five decisions that settle most of a model, and five things the notation cannot do. Appendix D maps the sheets; this teaches the language.
17 · *Milkwood Retail Group, end to end*	3, 8	What a complete, consistent model looks like — including the two first drafts that were wrong and the reason each was wrong.

18 · *Your first ninety days*	2, 3, 8, 9	Six numbers that tell you whether the estate is healthy, and the three artefacts that buy an architecture function its credibility.
19 · Appendix A, *The cross-maps*	4, 5, 6, 8, 11, appendix D	Pre-drafted catalogues and the argument for keeping them alive. Match on name, never on number.
20 · Appendix B, *Glossary*	all	The arbiter when two people in a workshop mean different things by the same word. 388 of them.
21 · Appendix C, *Standards and bodies*	—	Where the standards in `032` come from, and which of them will date fastest.
22 · Appendix D, *References*	all	The sources. Cite from here rather than from memory when a number is challenged in a room.

Four patterns in that middle column are worth a minute.

Chapters 7, 16 and 15 are the ones that never close. The operating model, the notation and regulation are cited across more of this volume than anything else, for different reasons: the trading calendar and the batch clock constrain every plan; the modelling decisions chapter 16 settles — article, price, store, promotion — recur in Phase B, Phase C, Phase E and Phase G without being re-argued; and every phase produces something an inspector could ask for.

Chapter 1 is cited by no step at all. That is not a criticism of it. It is the chapter that makes the other twenty-one legible, and nothing in a step-by-step procedure can point at "read this to understand the industry". Read it before step 2.1 and again before Phase A, and do not expect the index to remind you.

Chapter 8, the process model, is used by one chapter. That is not because process matters less in a retailer. It is because `016 Business Process` holds 20 generated stage names, no sheet joins a process to a role, a system or a data entity, and volume 1's fourteen BPMN level-2 models have nowhere in this repository to land. If one chapter of volume 1 is underused here, that is the one, and the reason is a gap in the record rather than a gap in the teaching.

Chapters 20 and 22 read "all", and 21 reads none. A glossary and a reference list are consulted under pressure rather than read. Chapter 21 is cited by no step because the standards this volume touches arrive through `032 Standard`, which names sixteen of them and explains none — look there when a row needs a source.

What each volume deliberately leaves out

Neither volume is a shortened version of the other, and knowing the boundary saves you looking in the wrong place.

Volume 1 covers, and this volume does not:

- **The industry.** What a retailer is, why buying for less and selling for more is a discipline about the inventory in between, how a format is an economic machine. This volume assumes you know.
- **The economics.** Gross margin and the margin mix, markdown, shrink, stock turn, GMROI, open-to-buy, supplier income, the cash conversion cycle. `004 Measure` holds 16 baselines and no

opinion about where a credible number comes from. Chapter 6 has the opinion.

- **The vendor landscape.** Who sells what, what has landed in South Africa, what a replacement costs. `027 Physical Technology Compone` names twelve product classes and says nothing about the market around them.
- **The regulation in substance.** The Consumer Protection Act, POPIA, the pharmacy and health rules, food labelling, liquor, tobacco, competition, tax and measurement. `063 Control Objective` holds six obligations as rows; what each obliges you to do is chapter 15, and no row can carry it.
- **A complete worked model.** Chapter 17 draws Milkwood Retail Group end to end, including the two drafts that were wrong. This volume draws fragments, because its subject is the record.
- **The notation.** Chapter 16 teaches ArchiMate 4 for retail. Appendix D maps this repository's sheets onto it and assumes the teaching.

This volume covers, and volume 1 does not:

- **The repository.** 105 sheets: which columns carry judgement, which are derived, which contradict each other, and the naming rules that keep every lookup resolving.
- **The comparison tool.** Five paste targets, the `Classification` column that separates a design delta from governed drift from ungoverned change, and the gap candidates that fall out. Volume 1 does not mention it, because it did not exist when volume 1 was written.
- **The ADM as a sequence of acts.** Not the phases in the abstract but 159 steps, each with a sheet, a column set, a signatory, a testable completion condition and the report it shows up in. This volume is where somebody can check that you did the work.
- **Refusal.** Four gates, dispensations that expire on a date, architecture contracts, and what each body may *not* decide. A practice that cannot refuse has not implemented the method.
- **What the record is missing.** No change control, no role-to-process relationship, one gap record, no compliance assessments, no fulfilment node. Volume 1 could not have known; this volume has to say so and say what to add.
- **The ninety-day calendar against the record.** Volume 1 chapter 18 says what to read and who to ask. The workshop appendix here says which sheets you touch in week three.

One closing instruction, and it is the only thing in this appendix worth memorising. Where a step and volume 1 disagree about the business, volume 1 is probably right and your sheet is stale. Where they disagree about the repository — a count, a column name, a `Ref` prefix — the repository is right and both books are describing it. Open the sheet before you argue with either.

THE CHAPTER IN TEN LINES

- Start from the step, not the phase: this index runs step to teaching, not teaching to step.
- The third column gives a volume 1 chapter number and then the section heading inside it, and every heading cited has been checked against that chapter's text.
- A dash is honest. 64 of the 159 steps have no volume 1 counterpart.
- The dashes cluster where this volume works alone: the gates, the change-control spine, the portfolio arithmetic and the column repairs.
- Volume 1 appendix A is a teaching set. Its `C-01`…`C-20` and `A-01`…`A-25` are not the repository's `C01`…`C10` and `A01`…`A12`. Match on name, never on number.
- Chapters 15 and 16 are cited across more of this volume than anything else: regulation and the notation never close.
- Chapter 8, the process model, is used by one chapter, because `O16` holds 20 generated rows and no sheet joins a process to anything.
- Volume 1 owns the industry, the economics, the vendors, the regulation and the notation in substance.
- This volume owns the repository, the comparison tool, the gates, the 159 steps and the list of what the record is missing.
- Disagreement about the business: believe volume 1. Disagreement about the repository: open the sheet.

APPENDIX F

The forms

Thirty printable worksheets, eleven sets of the questions you must answer before you can fill one honestly, and the discipline that gets the writing off the paper and into the sheet.

This appendix is paper. Everything before it tells you which sheet and which column. This one hands you the sheet of paper you carry into the room and bring back with writing on it.

Halverson Stores Group is an invented retailer. Its data is real inside this repository and nowhere else. Every example row below is marked as an example and is there to be crossed out.

How to use these forms

Print the forms for the phase you are in, and only those. There are thirty and nobody needs thirty in a week. The index overleaf gives the form number, its title, the sheet it transcribes into and the ADM phase it belongs to. Find your phase, print those pages, put them in the folder with the agenda.

Fill them by hand, in the room, while the people who know the answer are still in front of you. Transcribe the same day — not the same week. Keep the paper until the row has been approved by whoever the form says signs it, then retire it the way the last section of this appendix describes.

Why paper beats a laptop in the room

A laptop does three things to a room, all of them bad. It puts a screen between you and the person you are asking, so you stop watching their face and miss the hesitation that is the real finding. It turns an argument into a typing queue: the room moves at the speed of the typist, and the typist starts steering towards the cells that are easy to fill. And it tempts you to type into the repository live, which puts a half-agreed judgement in front of a report before anybody has signed it.

Paper is also faster at the thing a workshop is for. Two people fill two forms at once. A form slides across a table to the category director who owns the range, and they will initial a row they would never have agreed to by email. A crossing-out stays visible, which matters: on Form B-2 the maturity score somebody changed under pressure is more interesting than the score they settled on. And when the room disagrees you write both answers in the cell, which no spreadsheet lets you do.

The cost of paper is transcription, and transcription is where the discipline has to live. That is the last section of this appendix and it is the part to read twice.

A form that does not transcribe into a real sheet is useless

Every column on every form below is a real column of a real sheet of `Retail_TOGAF_Metamodel.xlsx`, spelled the way the sheet spells it, in the order the sheet holds it [1]. That is not tidiness. It is the property that makes a form worth filling: the person transcribing reads

left to right across the paper and left to right across the sheet, and nothing has to be interpreted on the way.

So the forms carry some things that look like faults and are not. The first column is `Ref`, never `ID`. `O09 Organization Unit` spells its tab with an American *z* and its second column with a British *s* — `Organisation unit` — and the form prints both as the file does. Several relationship tabs are cut off at Excel's thirty-one-character limit: `O26 Logical Technology Componen`, `O27 Physical Technology Compone`, `R05 Application manages Data En`, `R06 Application interfaces with`, `R09 Requirement satisfied by Bu`, `R18 Standard applies to Applica`, `R25 Application supports Produc`. Those truncations are the sheet names. Type them as they are, because a lookup written against the full name will not resolve.

Where a sheet has more than six columns the form splits, because six is what fits the 170mm text column and still leaves a cell wide enough to write a date in. `O13 Business Capability` becomes B-1, identify, and B-2, assess; `O20 Data Entity` becomes C-1 and C-2; `O23 Application Component` becomes CA-1 and CA-2. Each form says which columns it covers and which are typed straight into Excel. The ones left to Excel are, with few exceptions, sentences — `Description`, `Definition`, `Statement`, `Mandate`, `Membership` — because nobody should write a paragraph into a printed cell.

Where two small sheets answer one question the form holds both and says so on its face: `O07 Constraint` and `O08 Assumption` share Form A-2 with `O14 Value Stream`; `O38 Data Set` and `O43 Master Data Rule` share C-3 with `O36 Data Entity Relationship`; `O32 Standard` shares D-2 with `R18 Standard applies to Applica`. Consolidation is a judgement about rooms, not about sheets: those pairs get settled by the same people in the same hour.

Six forms have no sheet behind them at all, and say so in their first line. **Form A-1**, the stakeholder register, feeds nothing — sixty-four object sheets and not one of them is a stakeholder sheet, so the register lives in the Statement of Architecture Work and each line keys into `O10 Actor` or `O11 Role`. **Forms B-5 to B-8** feed four sheets you build, because `EXT Industry Extension` names `X01 SKU / Article`, `X02 Store / Location`, `X03 Assortment` and `X04 Promotion` as object types this sector needs and then holds three columns of description and no instances [1]. Those four are the forms a retail architect will reach for most, which is why they get the fullest treatment in this appendix. And **Form H-2** feeds a change-request register and a change log that do not exist, because this repository ships no version control, no change log, no change-request sheet and no `Change Ref`, `Changed By` or `Approved By` column on any object sheet. You build those before you load a record. The form prescribes the columns.

The eleven question sets

Between the forms are eleven `ask` sets. Each sits before the form it prepares and holds three to five questions with ruled space for the answers. They are not warm-ups. They are the questions that decide whether the form can be filled honestly — who can rule a format out of scope, what this group would pay to be better at than its competitors, which stock figure the replenishment planner actually trusts, whether the group even has a price to master in the franchise estate. Answer them on paper before the session. If you cannot answer them, the session will produce rows that look like data and are not.

The index of forms

Form	Title	Feeds	Phase
P-1	Scope, formats and ownership models	`O09 Organization Unit`, `O12 Location`	Preliminary
P-2	Principles — adopt, edit, retire	`O05 Principle`	Preliminary
P-3	Governance bodies, gates and the deliverable plan	`O57`, `O58`, `O31 Deliverable`	Preliminary
P-4	Drivers, goals, objectives and measures	`O01`, `O02`, `O03`, `O04 Measure`	Preliminary
A-1	Stakeholder register, actors and roles	no sheet — keys into `O10`, `O11`, `R21 Role performed by Actor`	A
A-2	Value streams, constraints and assumptions	`O14`, `O07 Constraint`, `O08 Assumption`	A
B-1	Capability identify	`O13 Business Capability` — structure columns	B
B-2	Capability assess	`O13 Business Capability` — judgement columns	B
B-3	Functions, processes and business services	`O17`, `O16`, `O15 Business Service`	B
B-4	Products and contracts	`O18 Product`, `O19 Contract`, `R25 Application supports Produc`	B
B-5	SKU / article — the sheet you build	none — `EXT` type `X01 SKU / Article`	B
B-6	Store / location — the sheet you build	none — `EXT` type `X02 Store / Location`	B
B-7	Assortment — the sheet you build	none — `EXT` type `X03 Assortment`	B
B-8	Promotion — the sheet you build	none — `EXT` type `X04 Promotion`	B
C-1	Data entity identify	`O20 Data Entity` — identity columns	C data
C-2	Data entity assess	`O20 Data Entity` — ownership and lineage	C data
C-3	Entity relationships, data sets and master data rules	`O36`, `O38 Data Set`, `O43 Master Data Rule`	C data
CA-1	Application identify	`O23 Application Component` — identity columns	C application
CA-2	Application assess	`O23 Application Component` — judgement columns	C application
CA-3	Application to data entity, and the interface register	`R05 Application manages Data En`, `R06 Application interfaces with`	C application

D-1	Technology services and components	`O25`, `O26 Logical Technology Componen`, `O27 Physical Technology Compone`	D
D-2	Standards, conformance and resilience tiers	`O32 Standard`, `R18 Standard applies to Applica`, `O49 Resilience Tier`	D
E-1	Gap register	`O29 Gap`	E
E-2	Building blocks and acceptance criteria	`O55 Building Block`, and `O35`, `O44`, `O45`, `O46`	E
E-3	Work packages and transition architectures	`O30 Work Package`, `O53 Transition Architecture`	E
F-1	Projects and implementation factors	`O56 Project`, `O52 Implementation Factor`	F
G-1	Architecture contract and compliance assessment	`O33 Architecture Contract`, `O59 Compliance Assessment`	G
H-1	Governance decision and change request	`O60`, `O61 Governance Change Request`	H
H-2	The change request and change log you must build	two sheets that do not exist — you build them	H
R-1	Requirements, obligations and the acceptance criterion	`O06`, `O63`, `R09 Requirement satisfied by Bu`	Requirements

Things you might expect on that list are not there, and the reasons differ.

`O37 Data Attribute` has no form: sixty rows of entity, attribute, type, key and classification, which is a data-modelling tool's work and not a workshop's. `O40 Source To Target Mapping` has none for the same reason, and because a mapping rule written by hand is a mapping rule nobody can test. `X05 Planogram` and `X06 Fulfilment Node` have no form — not because they do not matter but because a planogram is produced by the space tool and a fulfilment node is a property you set on Form B-6, and a form that duplicated either would create a second place to disagree. `EXT Industry Extension` itself has no form, because its six rows are the metamodel's own shape rather than content: you change them by argument, not by filling in a sheet. And none of the thirteen `RPT` sheets has a form, because every row in them is read from an object sheet. A form that fed a report would be a form that falsified it.

Seventeen object sheets are left to the keyboard, and the honest reason is that **a workshop is not the way to fill them**. `O21 Logical Data Component`, `O22 Physical Data Component`, `O24 Information System Service`, `O28 Platform Service`, `O47 Reference Model Entry` and `O66 Architecture Increment` are derived from decisions a form above has already recorded, so a form would create a second source for the same judgement. `O39 Data Migration`, `O48 Environment`, `O50 Network Link` and `O54 Architecture Alternative` are produced by a named piece of analysis rather than by a room. `O41 Security Control`, `O42 Metadata Element`, `O62 Delivery Risk`, `O64 Security Policy` and `O65 Enforcement Point` belong to the security and delivery functions, which have their own registers and will not accept a second one from the architecture practice; map into them by name

and record where the authoritative copy lives. Where a sheet in that list is empty and nobody will own it, that is a gap for Form E-1, not a missing form.

Most relationship sheets have no form either, and that is deliberate: `R01`, `R02 Value stream uses Capabilit`, `R03 Capability realised by Busi`, `R04 Business Service supported`, `R07`, `R08`, `R10`, `R11`, `R13`, `R14`, `R15`, `R16`, `R17`, `R19`, `R20`, `R22`, `R23` and `R24` are all derivable from the object sheets the forms above fill. Generate them; do not transcribe them. The four that carry a judgement a form has to capture — `R05`, `R06`, `R09` and `R21` — are on Forms CA-3, R-1 and A-1.

Preliminary Phase

Four forms, filled before you have the authority to fill anything else. They are about scope, power and dates rather than about architecture, which is why a sponsor reads them more carefully than anything in Phase B.

BEFORE YOU DRAW THE SCOPE BOUNDARY

Which formats does this architecture bind, and which does it merely describe? Grocery, pharmacy, apparel, liquor, forecourt, cash-and-carry, online — state the difference in one sentence per format.

For every store the group does not own outright: does the group hold a price, an assortment and an inventory position for it, or does it hold none of the three? Name the share of turnover that sits behind your answer.

Who, by name, can rule a format or a banner out of scope — and would they say so in front of the group executive?

`012 Location` holds eight rows and a `Scale` column that counts people. Which single site would stop trading across the group if it stopped, and what is the evidence rather than the assumption?

Six of the twelve shipped drivers are regulatory. Which of the six has a named person who would be called to account, and which has only a department? And which is recorded `Origin` = External and is really somebody's internal preference?

Form P-1 — Scope, formats and ownership models

Feeds `009 Organization Unit` — `Ref`, `Organisation unit`, `Type`, `Head`, `Headcount` — and `012 Location` — `Ref`, `Location`, `Type`, `Scale`. **Nothing is typed into Excel**: between them these are the whole of both sheets. **First pass** twelve units and eight locations at Halverson; a multi-format group with a franchise estate and a pharmacy chain needs a second copy of the second table. **Signed by** the Chief Executive, or `GB-04 Executive Committee` sitting as a body. Nobody below that may rule a format out of scope. **Note** `009` has no `Parent` column, so a group structure cannot be expressed here — record the position in `Type`, which the shipped data uses for Group, Corporate, Line and Assurance, and keep the legal-entity tree in the Statement of Architecture Work.

Ref	Organisation unit	Type	Head	Headcount
ORG-01	*example — delete*	Group	Chief Executive	222

`012 Location`. The shipped `Scale` values are free text counting people — 'Support Centre, 134 people'. Write the headcount and, where it differs, what the site carries: a distribution centre with seventy-seven people in it is not a small site. **Record the ownership model in `Type`** — Trading, Franchise, Voluntary member, Logistics, Technology — because nothing else in this sheet can hold it and the distinction decides whether the group has anything to master for that site at all.

Ref	Location	Type	Scale
LOC-01	*example — delete*	Trading	109 people

BEFORE YOU TOUCH THE PRINCIPLE SET

Which of the twenty-four shipped principles has been quoted in a real decision at this group in the last year? Give the decision.

Which one has never been quoted? Say why you would keep it rather than retire it.

Name two principles that will conflict on a live question — a trading-period freeze against a price-file correction will do — and say what breaks the tie.

`005 Principle` has no `Tier` column and `Status` offers only Approved and Draft. What is your tie-break until you add one? Write the sentence you will say to the Board.

Form P-2 — Principles — adopt, edit, retire

Feeds `005 Principle` — `Ref`, `Principle`, `Category`, `Owner`, `Status`. **Typed into Excel** `Statement`, which is a sentence; use the second table for any principle the room re-words and write it out in full. **The decision lives in** `Status`, and the sheet ships two values, Approved and Draft. Retired is a third value you add; there is no separate decision column and you should not create one. `Category` **ships five values** — Business, Application, Data, Technology, Security. A Governance principle needs a sixth, which appendix G adds. `Owner` **ships one value on all twenty-four rows**, 'Architecture Board', which is a committee and therefore not an owner; replace it with a named executive role. **First pass** forty-nine principles, five copies. **Signed by** `GB-01 Architecture Board`. A principle nobody has signed is a preference.

Ref	Principle	Category	Owner	Status
PRN-01	*example — delete*	Business	Chief Commercial Officer	Approved

Re-worded statements. Write the statement, not an instruction to the typist. If it needs two sentences it is probably two principles.

Ref	Statement
PRN-04	*example — delete*

Form P-3 — Governance bodies, gates and the deliverable plan

Feeds `057 Governance Body` — `Ref`, `Governance body`, `Cadence`, `Mandate` — `058 Governance Gate` — `Ref`, `Gate`, `When`, `What it tests` — and `031 Deliverable` — `Ref`, `Deliverable`, `Phase`, `Status`. **Typed into Excel** `057 Membership`. **Three sheets, one room**: a gate is real only if a body holds it, and a deliverable only if a gate inspects it. **First pass** five bodies, four gates, twenty deliverables — the whole of all three sheets; continue on a second copy. **Signed by** `GB-04 Executive Committee`; a governance body cannot constitute itself. **Note** no sheet records whether a body has met, so a body that exists on paper reads as healthy — put the last quorate sitting in `Cadence`, 'Monthly, last sat 2027-02', until you add a column.

Ref	Governance body	Cadence	Mandate
GB-01	*example — delete*	Monthly	Holds G1 and G2

`058 Governance Gate`. Shipped: `GT-0` Charter, `GT-1` Design, `GT-2` Build complete, `GT-3` Benefit. Write what each *tests*, not what it reviews.

Ref	Gate	When	What it tests
GT-0	*example — delete*	Before work starts	Scope and contract exist

`031 Deliverable`. `Phase` takes the ADM letter; `Status` ships Approved, Draft, Not started.

Ref	Deliverable	Phase	Status
DLV-01	*example — delete*	A	Approved

Form P-4 — Drivers, goals, objectives and measures

Feeds `001 Driver` — `Ref`, `Driver`, `Type`, `Origin`, `Owner`, `Pressure` — `002 Goal` and `003 Objective` — `Ref`, `Goal` / `Objective`, `Horizon`, `Status` — and `004 Measure` — `Ref`, `Measure`, `Unit`, `Domain`, `Baseline`, `Cadence`. **Typed into Excel** both `Description` columns, `002 Owner` and `003 Status` where it differs from the goal's. **Four sheets, one chain, one room. `Goal` on `003` is a reference**; a typed name will not resolve in `R11 Goal realised by Objective`. **`Baseline` is the only column that makes a measure usable** — the number and the date you read it. **First pass** twelve drivers, six goals, fourteen objectives, sixteen measures; continue on further copies. Volume 1 appendix A drafts the set: **map by name, never by number**. **Signed by** `GB-02 Investment Committee`.

Ref	Driver	Type	Origin	Owner	Pressure
DRV-01	*example — delete*	Regulatory	External	Chief Compliance Officer	High

Goals and objectives, one row per objective.

Goal ref	Goal	Objective ref	Objective	Horizon	Status
GOL-01	*example — delete*	OBJ-01	Every obligation evidenced	2 years	Not started

`004 Measure`. There is no grain column, so put the grain in `Measure`.

Ref	Measure	Unit	Domain	Baseline	Cadence
MSR-01	*example — delete*	%	Compliance	38	Monthly

Phase A — Architecture Vision

Two forms. The first feeds nothing in the repository and is still the one that decides whether the rest of the cycle happens.

BEFORE YOU FILL THE STAKEHOLDER REGISTER

Name the person who can stop a price change going live, and the person who can stop a range review. If they are the same person, say what that tells you.

Who owns the trading calendar, and who do they tell when they move a trading period?

For each format in scope: name the format's commercial owner and the format's operations owner. Where one name covers three formats, write what that costs.

`010 Actor` holds eighteen rows and every one reads `Status` = Filled. Which role is in fact held by somebody acting, and for how long?

Who in this group has never been asked for an architecture opinion and will have to approve one of your gaps anyway?

Form A-1 — Stakeholder register, actors and roles

Feeds nothing directly. There is no stakeholder sheet in this repository — sixty-four object sheets and not one of them holds a stakeholder, a concern or a viewpoint. The register lives in the Statement of Architecture Work, and **every line keys into `O10 Actor` or `O11 Role`**, which is why those two tables sit under it. **Then transcribe** `O10 Actor` — `Ref`, `Actor`, `Type`, `Status` — `O11 Role` — `Ref`, `Role`, `Type`, `Performed by` — and `R21 Role performed by Actor` — `Role`, `Role name`, `Actor`, `Actor name`, `Status`. **First pass** fifteen to twenty stakeholders, eighteen actors, eighteen roles. **Signed by** the sponsor, who has to agree the list of people whose objections will stop the work. **Note** `O11 Performed by` holds a type word in the shipped data — 'Person', 'Team' — not an actor reference; the actual link is `R21`, so fill both and do not expect `O11` to carry it.

Stakeholder	Concern, in their words	What they can stop	Where they must sign
	example — delete	A price change	GT-1

`O10 Actor` and `O11 Role`. `O10 Type` ships Executive, Management and Specialist; `Status` ships Filled. `O11 Type` ships Executive, Management and Specialist; `Performed by` ships Person and Team.

Ref	Actor	Type	Status	Role ref	Role
ACT-01	*example — delete*	Executive	Filled	ROL-01	Chief Executive

Form A-2 — Value streams, constraints and assumptions

Feeds `O14 Value Stream` — `Ref`, `Value stream`, `Stages`, `Capabilities` — `O07 Constraint` — `Ref`, `Constraint`, `Type` — and `O08 Assumption` — `Ref`, `Assumption`, `Status`. **Typed into Excel** `O14` `Scope` and `O07` `Description`, both sentences. **Three sheets share this form** because the same ninety minutes produces all three: a value stream you cannot run is a constraint, and a value stream you have not verified is an assumption. **`Stages` is free text in the shipped data** — '4 stages' — which counts them and does not name them; name them in the second column of the first table instead and keep the count in `Stages`. **`Capabilities` is a comma-separated reference list** — 'C01, C02, C03' — so write references, not names. **First pass** five to eight value streams, eight constraints, six assumptions. **Signed by** the Chief Operating Officer for the streams and the sponsor for the assumptions, because an assumption nobody has signed is a risk nobody owns.

Ref	Value stream	Stages, named	Stages	Capabilities
VS01	*example — delete*	Plan, select, award, range	4 stages	C01, C02

`O07 Constraint` — `Type` ships Financial, Resource, Regulatory and Technical. `O08 Assumption` — `Status` ships Open. An assumption that stays Open for two quarters is a finding; write the date you accepted it beside the reference.

Ref	Constraint	Type	Assumption ref	Assumption	Status
CON-01	*example — delete*	Financial	ASM-01	Sponsorship holds	Open

Phase B — Business Architecture

Eight forms, and four of them feed sheets that do not exist yet. That is the shape of retail business architecture in this repository: the TOGAF core carries capability, function, process, service, product and contract, and it has nowhere to put an article, a store format, a range or an offer.

Form B-1 — Capability identify

Feeds `O13 Business Capability` — `Ref`, `Capability`, `Level`, `Parent`. **Typed into Excel** `Description`, a sentence, and the four judgement columns, which are Form B-2. **`Ref` carries the decomposition** — `C01`, `C01.01`, `C01.01.01` — so the reference is the structure and a mistyped reference orphans a branch in `R01 Capability decomposition`. **`Parent` is a reference**, blank at level 1. **First pass** ten level 1, forty level 2; the shipped file holds a hundred rows across three levels, and the level 2 and level 3 names in it are machine-generated — 'Merchandising Administration within merchandising & assortment' — which means **this is the sheet to rewrite rather than extend**. Volume 1 appendix A drafts the capability set; map by name, never by number. **Signed by** the executive who owns the level 1 branch, one signature per branch, not one for the map. **Continue on a second copy of this form** — forty rows will not fit one page.

Ref	Capability	Level	Parent
C01	*example — delete*	1	

BEFORE YOU SCORE A SINGLE CAPABILITY

What is the one sentence that decides whether a row on this sheet is a capability here and not a process or a system?

Which two capabilities would this group pay to be better at than its competitors? If the answer is more than two, the map is not scored, it is flattered.

`C05 Store Operations` **ships at** `Current` **4 and** `Target` **5 while** `C01 Merchandising & Assortment`**, marked Critical, ships at 1 against a target of 2. Is a target of 2 on a Critical capability a decision somebody took, or a cell nobody read?**

Who scores `Current`**, and who is allowed to disagree with them in writing?**

For each capability you are about to score 1: name the consequence in trading terms, not in architecture terms.

Form B-2 — Capability assess

Feeds `013 Business Capability` — `Ref`, `Capability`, `Criticality`, `Current`, `Target`, `Disposition`. **Nothing is typed into Excel**: with Form B-1 these are the whole sheet. **Shipped values**: `Criticality` Critical, High, Medium, Low; `Current` and `Target` integers 1 to 5; `Disposition` Sustain, Build, Invest. **`Gap` is not a column here** — `RPT03 Capability Heatmap` computes `Target` minus `Current` for the ten level 1 rows, so do not add a gap column and do not fill one. **Score level 1 and level 2 only.** Scoring a hundred rows produces a hundred numbers and no decisions. **First pass** ten level 1 rows in the room, forty level 2 rows by the owners afterwards. **Signed by** the branch owner, who signs their own `Current` score — which is the signature that makes the number defensible, because they are signing against their own function.

Ref	Capability	Criticality	Current	Target	Disposition
C01	*example — delete*	Critical	1	2	Sustain

Form B-3 — Functions, processes and business services

Feeds `017 Business Function` — `Ref`, `Business function`, `Capability` — `016 Business Process` — `Ref`, `Business process`, `Value stream`, `Automation` — and `015 Business Service` — `Ref`, `Business service`, `Capability`, `Exposure`, `Criticality`. **Typed into Excel** all three `Description` columns. **Three sheets, one room**: the same people know which function performs a capability, which process runs inside a value stream and which service the capability exposes. **`Capability` and `Value stream` are references** — `C02`, `VS01`. **Shipped values**: `Automation` Manual, Partly automated, Automated; `Exposure` Internal, External; `Criticality` Critical, High, Medium, Low. **Note the shipped processes are stage placeholders** — 'Plan to Range — stage 1' — so expect to rename all twenty. Volume 1 chapter 8 names the real retail processes; map by name. **First pass** ten functions, twenty processes, ten services. **Signed by** the Chief Operating Officer. **Continue on a second copy** for the process table.

Ref	Business function	Capability	Process ref	Business process	Automation
BFN-01	*example — delete*	C01	BPR-01	Range review	Partly automated

`015 Business Service` . `Exposure` = External means a party outside the group consumes it — a supplier through the portal, a customer through a channel. Three of the ten shipped services read External, which is worth checking against who actually calls them.

Ref	Business service	Capability	Exposure	Criticality
BSV-01	*example — delete*	C01	Internal	Medium

Form B-4 — Products and contracts

Feeds `018 Product` — `Ref`, `Product`, `Type` — `019 Contract` — `Ref`, `Contract`, `Counterparty`, `Type`, `Expiry` — and `R25 Application supports Produc` — `Product`, `Product name`, `Application`, `Application name`. **Typed into Excel** `018 Description`. **`018 Product` is not the article master.** It holds five rows of trading proposition — 'Own Brand Grocery', 'Branded Grocery' — and the article itself has no home in the TOGAF core, which is what Form B-5 is for. Say that out loud in the room or somebody will try to load four hundred thousand rows here. **`019 Type` ships Perpetual and Subscription; `Counterparty` holds an application name in the shipped data**, which makes these vendor agreements rather than trading contracts. **A supplier trading agreement has no sheet** — record it here with `Type` = Supply and the supplier in `Counterparty`, and accept that `019` cannot hold trade terms or supplier income. **First pass** five products, ten contracts. **Signed by** the Chief Commercial Officer for products, General Counsel for contracts.

Ref	Product	Type	Application ref	Application
PRD-01	*example — delete*	Own brand	A01	Merchandising System

`019 Contract`. `Expiry` is a date and it is the column that earns this form its place: a contract expiring inside the roadmap horizon is a work package nobody has raised.

Ref	Contract	Counterparty	Type	Expiry
CTR-01	*example — delete*	Merchandising System	Perpetual	2027-12

The four forms a retail architect actually uses

`EXT Industry Extension` names six object types this sector needs beyond the TOGAF core and holds, for each, a reference, a name and one sentence saying why the core is not enough [1]. It holds no instances and there are no `X` sheets. That is not a defect in the file. It is the clearest demonstration in the whole set of why an industry extension exists at all: an article, a store format, a range and an offer are the four things a retailer argues about every week, and the content metamodel has nowhere to put any of them.

So Forms B-5 to B-8 **prescribe the sheets**. Build them in the same workbook, with headers on row 4 and data from row 5, first column `Ref`, so they behave like every other sheet and the reports can read them. Four sheets, named exactly as `EXT` names the types: `X01 SKU / Article`, `X02 Store / Location`, `X03 Assortment`, `X04 Promotion`.

One warning before you print them. **None of these four sheets holds instances.** `X01` does not hold four hundred thousand articles; it holds the *levels* at which the group masters an article, who masters each, and which identifier scheme governs it. `X02` holds locations, because there are hundreds and not hundreds of thousands. `X03` holds assortments, not the articles inside them. `X04` holds promotions for a trading period, not every offer ever run. A repository that tries to be the master is a repository nobody updates.

BEFORE YOU DECIDE WHAT A SKU IS

Write the sentence this group uses for 'article'. Now write the sentence the merchandising system uses, the sentence the till uses and the sentence the warehouse uses. How many are the same?

At how many levels does this group genuinely master a sellable thing — style, option, SKU, pack? Name the system that masters each level.

A consumer unit, an inner pack and a case are three trade item numbers with a hierarchy between them. Does this group hold that hierarchy, and if not, how is a distribution-centre receipt reconciled to a store sale?

In the pharmacy estate, one product carries a trade item number for selling and a separate reimbursement code for claiming, maintained by different bodies on different cadences and not mapping one to one. Who inside this group owns the mapping between them?

`O20 Data Entity` **records** `E01 SKU / Article` **with** `Master system` **= POS and** `Owner` **= Head of Finance. Is either of those the answer this room would give?**

Form B-5 — SKU / article — the sheet you build

Feeds no existing sheet. `EXT Industry Extension` names `X01 SKU / Article` — *the smallest sellable unit with its trading attributes* — as an industry extension type, and the repository holds no sheet for it [1]. **Build** **`X01 SKU / Article`** with these six columns, headers on row 4: `Ref`, `Level`, `Identifier scheme`, `Master system`, `Mastered attributes`, `Owner`. **One row per level at which this group masters a sellable thing**, not one row per article. **`Level`** takes the level words this group actually uses — style, option, SKU, consumer unit, inner pack, case — and the point of the form is that the room has to choose them. **`Identifier scheme`** names the scheme and its issuer: a GS1 trade item number, an internal article number, a reimbursement code, a supplier code. **`Mastered attributes`** names what is authoritative at that level and nowhere else — description, pack size, tax rate, age restriction, schedule, unit of measure. **First pass** four to eight rows. **Signed by** the Head of Merchandising and the Chief Data Officer jointly, because one of them owns the meaning and the other owns the record. **Then** add the row to `O20 Data Entity` and let `E01` point at this sheet rather than duplicate it.

Ref	Level	Identifier scheme	Master system	Mastered attributes	Owner
X01-01	*example — delete*	GS1 trade item	Merchandising	Description, pack	

The level hierarchy, written as relationships. One row per parent-child pair, because the hierarchy is the thing the estate loses and the thing a receipt reconciliation needs.

From level	Relationship	To level	Cardinality	Held in
Case	*example — delete*	Consumer unit	1:N	

Form B-6 — Store / location — the sheet you build

Feeds no existing sheet. `EXT` names `X02 Store / Location` — *a trading or fulfilment location with its format and capacity* — and `O12 Location` holds eight rows with four columns and cannot carry a licence, a format or a fulfilment role [1]. **Build `X02 Store / Location`** with: `Ref`, `Location`, `Format`, `Ownership model`, `Fulfilment role`, `Licences held`. **One row per trading or fulfilment site**, which in a real group is hundreds of rows and still belongs here, because every one of the six columns is an architecture input. **`Ownership model`** — company-owned, franchise, voluntary member, concession — decides whether the group masters price, assortment and stock for that site or masters none of them. **`Fulfilment role`** is the `X06 Fulfilment Node` type folded in: can this site source, pick or hand over an order, and for which channels. **`Licences held`** is a list with expiry dates, because a liquor licence, a pharmacy licence and a certificate of acceptability for food premises are all per premises with their own conditions and dates, and a lapsed one stops that site trading in that category. **First pass** one row per site; a group with hundreds of stores fills this from the store master and verifies a sample of twenty by hand. **Signed by** the Head of Store Operations, with the Responsible Pharmacist signing any row that carries a pharmacy licence.

Ref	Location	Format	Ownership model	Fulfilment role	Licences held
X02-01	*example — delete*	Supermarket	Company-owned	Pick, collect	Liquor 2027-08

BEFORE YOU WRITE AN ASSORTMENT OR A PROMOTION DOWN

Who authorises a range, and who can add a single article to a store outside that authorisation? Name both, and say how often the second happens.

What is the store cluster this group ranges to, and is the clustering held anywhere a system can read?

A promotion has a mechanic, a funding source and a period. For the last promotion that overran, which of the three was wrong in the system?

Which part of the group's promotional spend is funded by a supplier, and where is that funding recorded in a way the margin report can read?

When two promotions collide on one article in one period, which system decides the outcome, and does the store know in advance?

Form B-7 — Assortment — the sheet you build

Feeds no existing sheet. `EXT` names `X03 Assortment` — *the ranged set of SKUs for a location and period* — and nothing in the core metamodel holds it [1]. **Build `X03 Assortment`** with: `Ref`, `Assortment`, `Cluster`, `Period`, `Authorised by`, `Held in`. **One row per authorised range, per cluster, per period** — not one row per article. **`Cluster`** is the grouping the group actually ranges to, which is rarely the format and is often a space band; if the group does not have one, writing 'none' here is the finding. **`Period`** is a trading period in the group's own calendar, not a Gregorian month, and if the room cannot agree the period boundary you have found the trading-calendar problem early. **`Held in`** names the system of record and is the column that produces the argument: `O20 Data Entity` records `E04 Assortment` with `Master system` = WMS and `Lineage` = Absent, which is one of the two entities in the file with no lineage at all [1]. **First pass** one row per category per cluster for the current period — twenty to forty rows for a grocery estate. **Signed by** the category director who authorised the range. **Continue on a second copy of this form.**

Ref	Assortment	Cluster	Period	Authorised by	Held in
X03-01	*example — delete*	Band A	P07 FY28		Space planning

Form B-8 — Promotion — the sheet you build

Feeds no existing sheet. `EXT` names `X04 Promotion` — *a funded offer with mechanics, period and expected uplift* — and `O20 Data Entity` holds `E06 Promotion` as a data entity with `Master system` = Retail Analytics, which is a reporting platform and cannot be the master of a forward-dated offer [1]. **Build `X04 Promotion`** with: `Ref`, `Promotion`, `Mechanic`, `Funding`, `Period`, `Expected uplift`. **`Mechanic`** is the thing the till has to execute — multi-buy, percentage off, price point, meal deal, loyalty-only, threshold reward — and the vocabulary has to match what the promotion engine can actually run, or the form records an intention and the store records a dispute. **`Funding`** is supplier-funded, group-funded or shared, with the supplier reference where there is one; this is the column that connects a promotion to supplier income and the one most often left blank. **`Period`** is effective-from and effective-to with the hour, because a promotion that starts at midnight and a till that reads a date are two different behaviours. **`Expected uplift`** is a number with a unit, written before the promotion runs — otherwise there is nothing to test afterwards. **First pass** one row per promotion in the current and next trading period. **Signed by** the Head of Pricing, and by Finance for anything supplier-funded.

Ref	Promotion	Mechanic	Funding	Period	Expected uplift
X04-01	*example — delete*	Multi-buy 3 for 2	Supplier	P07 wk 2-4	+18% units

Phase C — Data Architecture

Three forms over `020 Data Entity`, `036 Data Entity Relationship`, `038 Data Set` and `043 Master Data Rule`. `037 Data Attribute` has no form and should not get one.

BEFORE YOU NAME A MASTER SYSTEM

Take `E01 SKU / Article`. Which system does the business believe masters it, which system actually masters it, and which system do the people doing the reconciling trust?

`020 Data Entity` ships `E05 Price` with `Master system` = Workforce Management and `E07 Planogram` with `Master system` = Loyalty & CRM. Before you correct them: how would anybody in this group have noticed?

`043 Master Data Rule` ships `MDR-01 SKU / Article` with `Golden source` = Demand Forecasting & Replenishment while `020` says POS. Two sheets, two answers, no report that compares them. Which is right, and who decides?

Name the entity in this group that three systems each believe they master. What happens at month end because of it?

`Owner` on `020` holds post titles — 'Head of Finance', 'Data Steward'. Which of the fourteen has a named person who would answer a question about it this afternoon?

Form C-1 — Data entity identify

Feeds `O20 Data Entity` — `Ref`, `Data entity`, `Class`. **Typed into Excel** `Definition`, a sentence, and the four assessment columns, which are Form C-2. **`Class` ships two values**, Master and Transactional, and the shipped split is nine Master to five Transactional. **Reference data has no class value** and a retail estate is full of it — tax rates, units of measure, store clusters, schedule classifications, the trading calendar itself — so add Reference as a third value before you load, or every piece of reference data in the group gets filed as Master and inherits a mastering argument it does not need. **First pass** fourteen shipped entities plus the ones volume 1 chapter 9 argues for and this file does not hold; map by name, never by number. **Signed by** the Chief Data Officer, who is also the person who has to find each entity an owner on Form C-2.

Ref	Data entity	Class
E01	*example — delete*	Master

Form C-2 — Data entity assess

Feeds `O20 Data Entity` — `Ref`, `Data entity`, `Classification`, `Owner`, `Master system`, `Lineage`. **Nothing is typed into Excel**: with Form C-1 these are the whole sheet. **Shipped values**: `Classification` Public, Internal, Confidential, Restricted; `Lineage` Full, Partial, Absent. **`Owner` must be a person, not a post and not a committee** — the shipped data holds post titles on all fourteen rows and `Data Steward` on two, which is a function, not an owner. **`Master system`** takes one application name; where the room names two, write both in the cell and mark the row for `GB-03 Data Governance Council`, because an entity with two masters is the finding and hiding it behind one name destroys it. **`Lineage` means traceable to a reported figure without reconstruction**, not 'we think we could work it out'. Two of the fourteen shipped rows read Absent and `ZZ Checks` counts them [1]. **First pass** fourteen rows, which fits one page. **Signed by** each named `Owner` for their own row and `GB-03 Data Governance Council` for the set.

Ref	Data entity	Classification	Owner	Master system	Lineage
E01	*example — delete*	Restricted		Merchandising	Full

Form C-3 — Entity relationships, data sets and master data rules

Feeds `036 Data Entity Relationship` — `Ref`, `From entity`, `Relationship`, `To entity`, `Cardinality` — `038 Data Set` — `Ref`, `Data set` — and `043 Master Data Rule` — `Ref`, `Entity`, `Rule type`, `Golden source`. **Typed into Excel** `038 Description` and `043 Description`. **Three sheets, one room**: the data governance council settles all three in one sitting, and a master data rule with no golden source is the same defect as a relationship with no cardinality. **`036` and `R19 Data Entity relates to Data` hold the same five columns and the same eleven rows.** Fill `036` and let `R19` mirror it; do not maintain both by hand, and if the two disagree the relationship sheet is the one the reports read. **`From entity` and `To entity` hold names in the shipped data, not references** — 'SKU / Article', not `E01` — which is a lookup waiting to break on a rename. Write the reference and the name. **Check the shipped rows before you extend them**: `REL-01` reads *SKU / Article references Product Hierarchy, 1:1* and `REL-02` reads *Product Hierarchy is composed of Store / Location, N:M*, neither of which is true of any retailer. **First pass** fifteen to twenty relationships, six data sets, ten rules. **Signed by** `GB-03 Data Governance Council`.

Ref	From entity	Relationship	To entity	Cardinality
REL-01	*example — delete*	belongs to	Product Hierarchy	N:1

`043 Master Data Rule`. `Rule type` ships Validation, Match rule, Survivorship and Stewardship. `Golden source` is an application name and **must agree with `020 Master system` for the same entity**; where it does not, one of the two is a decision nobody took.

Ref	Entity	Rule type	Golden source	Data set ref	Data set
MDR-01	*example — delete*	Validation	Merchandising	DST-01	Regulatory set

Phase C — Application Architecture

Three forms. The last one carries the interface register, which is where this repository's worst numbers live and therefore where the most useful afternoon of the whole cycle is spent.

Form CA-1 — Application identify

Feeds `O23 Application Component` — `Ref`, `Application`, `Type`, `In service`, `Users`, `Owner`. **Typed into Excel** `Description`, a sentence, and the three judgement columns, which are Form CA-2. **Shipped values**: `Type` Package, Custom, Platform; `In service` a four-digit year; `Users` an integer. **`Owner` holds an organisation unit in the shipped data**, not a person — and `A01 Merchandising System` reads `Owner` = Internal Audit, which no retailer intends. Write the accountable executive's role and let the unit follow. **`Users` is a count with no definition**: named users, concurrent users or licences. Pick one, write which in the margin, and keep it consistent, because the number is otherwise worthless and will still be quoted. **First pass** twelve shipped applications, and expect twenty to forty in a real multi-format estate once the pharmacy and store systems are counted. Volume 1 chapter 12 names the real landscape; map by name. **Signed by** the Chief Information Officer.

Ref	Application	Type	In service	Users	Owner
A01	*example — delete*	Package	2014	208	

BEFORE YOU ASSESS AN APPLICATION

`A08 Price & Promotion Engine` ships `Criticality` = Critical, `Health` = Poor and `Disposition` = Retire. Who is retiring the group's critical pricing engine, what replaces it, and when was that decided?

`A02 POS` ships `Criticality` = Low. The till is the only system that takes money. What does a Low criticality on it tell you about how this sheet was filled?

For each application you are about to mark Retire: name the business service that stops if it is switched off this quarter.

`RPT13 Orphan Analysis` names `A11 Supplier Portal` and `A12 Retail Analytics` as orphans that no business service maps to. Are they orphans, or is the service catalogue incomplete?

Which application in this estate would nobody in the room admit to owning?

Form CA-2 — Application assess

Feeds `023 Application Component` — `Ref`, `Application`, `Disposition`, `Criticality`, `Health`. **Nothing is typed into Excel**: with Form CA-1 these are the whole sheet. **Shipped values**: `Disposition` Retain, Replace, Retire, Consolidate, Invest; `Criticality` Critical, High, Medium, Low; `Health` Adequate, At risk, Poor. **`Criticality` is business criticality, not technical fragility**, and `Health` is the other way round — the commonest error on this sheet is to fill both from the same impression. Score `Criticality` with the business, `Health` with the people who run it, in two different conversations. **`Disposition` must be consistent with `049 Resilience Tier` and with `030 Work Package`**: an application marked Replace with no work package replacing it is a gap for Form E-1, and `Criticality` = Critical with `Disposition` = Retire needs a sentence of explanation in the margin and a decision reference. **First pass** twelve rows, one page. **Signed by** the Chief Information Officer and the executive who owns the business service the application supports, jointly.

Ref	Application	Disposition	Criticality	Health
A01	*example — delete*	Retain	High	At risk

BEFORE YOU WRITE AN INTERFACE DOWN

`ZZ Checks` reports sixteen unmanaged interfaces, twenty with no logging and nine direct database reads, out of twenty-nine [1]. Which of the nine reads would stop a trading day if the source changed a column?

How does the price file reach the stores and the shelf labels, by what mechanism, at what time, and how does anybody know it arrived?

How many trading partners exchange messages with this group, and how many separate mappings does the translator hold for them? The second number, not the first, is the integration backlog.

`INT-002` ships `Type` = Batch and `Frequency` = Real time on the same row. How many other rows in the register describe something that cannot exist?

Which interface in this estate has no owner on either side?

Form CA-3 — Application to data entity, and the interface register

Feeds `R05 Application manages Data En` — `Application`, `Application name`, `Data entity`, `Entity name`, `Role` — and `R06 Application interfaces with`, whose nine columns split across the second and third tables: identity, then assessment. **Both tab names are truncated at thirty-one characters; type them as the tab spells them.** **`R05 Role` ships Master, Uses, Updates** — one Master per entity; two means it goes to `GB-03 Data Governance Council`, not into the sheet. **The shipped `R05` holds a name in its reference column**, 'POS' not `A02`, which will not resolve. **First pass** twenty-eight entity rows, twenty-nine interfaces; continue on further copies. **Signed by** the Chief Information Officer, and the integration lead for `Managed` and `Logged`.

Application	Application name	Data entity	Entity name	Role
A01	*example — delete*	E01	SKU / Article	Master

`R06` identity. `Type` ships API, Batch, File, Direct DB read.

Ref	Source	Source name	Target	Target name	Type
INT-001	A01	*example — delete*	A08	Price engine	API

`R06` assessment. **A row where `Type` and `Frequency` contradict each other is a row nobody checked.**

Ref	Frequency	Managed	Logged
INT-001	*example — delete*	Unmanaged	No

Phase D — Technology Architecture

Two forms, and the second one carries three sheets because the same room settles all three: whether a standard is enforced, whether an application conforms to it, and whether the thing that must keep running has ever been tested.

Form D-1 — Technology services and components

Feeds `025 Technology Service` — `Ref`, `Technology service`, `Category` — `026 Logical Technology Componen` — `Ref`, `Logical technology component`, `Status` — and `027 Physical Technology Compone` — `Ref`, `Physical technology component`, `Vendor`, `Class`, `Support end`, `Position`. **Typed into Excel** `025` `Description`, `026` `Rationale`, and `027` `Location`, which is a site name better maintained against Form B-6. **Check `025` `Category` before you extend it**: the shipped rows read `TSV-01 Compute` with `Category` = Data and `TSV-02 Storage` with `Category` = Security, so the column has been filled from the wrong list and every value needs reading. **`026` `Status` ships Target and Current** and holds the seven candidate technologies the vision proposed — edge at store, real-time inventory, headless commerce, event streaming, payment tokenisation. **`027` `Support end` is a date and `Position` ships Supported, Past support and Approaching end** — three of the twelve shipped rows read Past support and `ZZ Checks` counts them [1]. `Position` must be derivable from `Support end` and the date you read it; where it is not, one of the two is wrong. **First pass** twelve services, seven logical components, twelve physical components. **Signed by** the Chief Information Officer; anything reading Past support also needs the Chief Risk Officer.

Ref	Technology service	Category	Logical ref	Logical component	Status
TSV-01	*example — delete*	Platform	LTC-01	Edge at store	Target

`027 Physical Technology Compone`. `Vendor` holds a real vendor name — the shipped data has 'Vendor' in one cell, which is a placeholder, not a supplier.

Ref	Physical technology component	Vendor	Class	Support end	Position
PTC-01	*example — delete*		Application	2027-03	Past support

Form D-2 — Standards, conformance and resilience tiers

Feeds `032 Standard` — `Ref`, `Standard`, `Category`, `Status`, `Conformance` — `R18 Standard applies to Applica` — `Standard`, `Standard name`, `Application`, `Application name`, `Conformance` — and `049 Resilience Tier` — `Ref`, `Application`, `Tier`, `Recovery objective`, `Tested`. **Three sheets, one room**: each asks whether the estate meets a stated bar. **Fill `R18` first and let `032 Conformance` follow**, or the headline contradicts the detail. **Shipped values**: `Category` Integration, Data, Security, Platform, Delivery, Governance; `Status` Current, Emerging, Deprecated; `Conformance` Conformant, Partial, Breach, Not assessed. **`032` has no owner and no review-date column**, so the review cycle lives in the `GB-01` standing agenda. **Five of sixteen read Not assessed**, which is not a pass. **First pass** thirty-one standards, one `R18` row per in-scope pair; continue on further copies. **Signed by** `GB-01 Architecture Board`, with the Chief Risk Officer on card data and resilience.

Ref	Standard	Category	Status	Conformance
STD-01	*example — delete*	Integration	Current	Not assessed

`049 Resilience Tier`. **Read the shipped rows first**: eight of twelve read Never tested, `RES-01 Merchandising System` sits at Tier 1 with fifteen minutes while `RES-02 POS` sits at Tier 3 with four hours [1]. The scale and the objectives disagree — a gap for Form E-1 before it is a test plan.

Ref	Application	Tier	Recovery objective	Tested
RES-01	*example — delete*	Tier 1	15 min	Never tested

Phase E — Opportunities and Solutions

Three forms. The first is the one sheet in this repository that ships almost empty and matters most.

BEFORE YOU RAISE A GAP

`029 Gap` holds one row. `ZZ Checks` lists sixteen unmanaged interfaces, twenty unlogged, nine direct database reads, three components past support, four security controls absent, three control objectives without evidence, two entities without lineage and seven work packages past their end date [1]. How many gap rows should this repository hold, and who decided it should hold one?

Take the worst number on that list. Write it as a fact with a quantity in it and a trading consequence attached.

`Severity` has to come from consequence, not from effort. Which of your candidate gaps carries a legal or licence consequence, and which merely carries work?

For each gap you are about to date: who in the room will be held to that date?

Form E-1 — Gap register

Feeds `029 Gap` — `Ref`, `Gap`, `Domain`, `Severity`, `Status`, `Due`. **Typed into Excel** `Building block`, which stays blank until the block that closes the gap exists on Form E-2. **The sheet ships one row**, `GAP-01`, Medium, Open, due 2028-06 — against a check sheet reporting eight distinct classes of defect, so this is the sheet where a first pass adds most [1]. **Shipped values**: `Domain` Business, Data, Application, Technology; `Severity` Critical, High, Medium, Low; `Status` Open, Closed. **Write the gap as a fact with a number in it** — 'nine applications read another application's database directly' — not 'integration is weak'. **`Severity` comes from consequence**: Critical means a licence, a legal or a reproducibility consequence, not a large amount of work. **`Due` is agreed in the room with the person who will be held to it**, before it is written down. **First pass** eight to twelve rows, consolidated, not one per check line. **Signed by** `GB-01 Architecture Board`, with each `Domain` owner accepting the severity on their own rows. **Shows up in** `RPT07 Gap Analysis` and `R12 Gap closed by Work Package`.

Ref	Gap	Domain	Severity	Status	Due
GAP-01	*example — delete*	Technology	Critical	Open	2028-06

Form E-2 — Building blocks and acceptance criteria

Feeds `055 Building Block` — `Ref`, `Domain`, `Building block`, `What it must do`, `Acceptance criteria`. **`055` is the union of the four domain sheets**: thirty-seven rows made up of `035 Business Building Block` ten, `044 Data Building Block` ten, `045 Application Building Block` ten and `046 Technology Building Block` seven [1]. Fill `055` from this form and mirror into the domain sheet, or fill the domain sheets and let `055` consolidate — pick one direction, write it in the margin, and never do both. **The domain sheets carry a second anchor column** naming what the block is for: `035` has `Capability`, `044` has `Entity`, `045` has `Application`. **`046 Technology Building Block` names its anchor column `Ref` as well**, so the sheet has two columns called `Ref`, and the values in the second — `T01`, `T02` — match nothing in `026` or `027`. Fix the anchor before you add a row, or the technology blocks stay unattached to anything. **`Acceptance criteria` is the column this whole volume turns on**: it must be a sentence a gate can test and fail. 'Merchandising & Assortment measured and owned' is not testable; 'every level 1 capability has a named owner and a measure with a baseline in `004 Measure`' is. **First pass** thirty-seven shipped rows, every one of them re-written. **Signed by** `GB-01 Architecture Board`. **Continue on a second copy of this form.**

Ref	Domain	Building block	What it must do
BBB-01	*example — delete*	Capability uplift	Operate at target maturity

Acceptance criteria, one row per block, written so a gate can fail it. This is the table to fill first and the building block names second.

Ref	Acceptance criterion a gate can test	Gate	Tested by
BBB-01	*example — delete*	GT-2	

Form E-3 — Work packages and transition architectures

Feeds `O30 Work Package` — `Ref`, `Work package`, `Domains`, `Cost`, `Start`, `End` — and `O53 Transition Architecture` — `Ref`, `Transition architecture`, `Work packages`, `Exit criteria`. **Typed into Excel** `O30 Approval` and `Delivery`, which are status values the programme office maintains rather than the room, and `O53 Description`. **`O30` has eight columns**, so this form covers the six that get decided in a planning session. **`Domains` is a comma-separated list** of the domain words, and a package touching three domains needs three owners before it needs a cost. **Check the shipped rows**: `WP-01` runs 2028-12 to 2031-04 with `Delivery` = Past end date, and `ZZ Checks` counts seven packages past their end date against a threshold of zero — so either the dates or the status are wrong on seven rows, and that is a finding for Form E-1 rather than something to copy [1]. **`O53` `Exit criteria` ships a count** — '5 exit criteria' — which names none of them; write the criteria in the second table and keep the count in the cell. **First pass** twelve packages, three transitions. **Signed by** `GB-02 Investment Committee` for cost and `GB-01 Architecture Board` for exit criteria.

Ref	Work package	Domains	Cost	Start	End
WP-01	*example — delete*	Business, Data	$2.8m	2028-12	2031-04

`O53 Transition Architecture`. `Work packages` is a comma-separated reference list. The exit criteria are what makes a transition a plateau rather than a label — each one testable, each one owned.

Ref	Transition architecture	Work packages	Exit criterion, written out
TA-1	*example — delete*	WP-01, WP-02	Estate within vendor support

Phase F — Migration Planning

Form F-1 — Projects and implementation factors

Feeds `056 Project` — `Ref`, `Project`, `Work package`, `Start`, `End`, `Lead` — and `052 Implementation Factor` — `Ref`, `Implementation factor`, `Category`, `Deduction`. **Typed into Excel** `056 Cost`, which should be taken from `030 Work Package` rather than re-entered, because two cost columns with no reconciliation is how a roadmap starts to lie. **`Work package` is a reference** and one project may deliver only one package in this model; where a project spans two, split the project or merge the packages, and record which you did. **`Lead` is a named person's role** — the shipped data holds 'Chief Risk Officer' as a project lead, which is a sponsor, not a lead. **`052 Category` ships Dependency, Capacity, Capability and Risk; `Deduction` is the sentence that says what the factor does to the plan** — 'blocks integration work until resolved' — and it is the only column on that sheet that changes anybody's behaviour. **First pass** twelve projects, eight factors. **Signed by** the programme sponsor for projects; `GB-01 Architecture Board` for the factors, because a factor the architecture function has not accepted is a factor the plan will ignore.

Ref	Project	Work package	Start	End	Lead
PRJ-01	*example — delete*	WP-01	2028-12	2031-04	

`052 Implementation Factor`. One row per thing that will slow the plan down, written as a deduction and not as a worry.

Ref	Implementation factor	Category	Deduction
IF-01	*example — delete*	Dependency	Blocks integration work

Phase G — Implementation Governance

Form G-1 — Architecture contract and compliance assessment

Feeds `033 Architecture Contract` — `Ref`, `Architecture contract`, `Work package`, `Compliance status` — and `059 Compliance Assessment` — `Ref`, `Building block`, `Block name`, `Acceptance criterion`, `Gate`, `Verdict`, `Evidence`. **`059` has seven columns and holds zero rows**, which is the most consequential emptiness in the file: four gates on `058`, thirty-seven building blocks on `055`, and no record that any acceptance criterion has ever been tested [1]. `ZZ Checks` still reports 'Object types populated 65 / Every type carries records', so the check sheet is wrong about its own workbook — verify that on day one. **The second table below splits `059`**: identity and criterion in the first four columns, verdict and evidence in the last two, because a verdict written in the same pass as the criterion is a verdict nobody tested. **`033` `Compliance status` ships Non-compliant, Compliant and Not yet assessable** — and `ACO-01` ships Non-compliant, which means the contract exists and the package is breaching it, with no assessment row to say how anybody knows. **First pass** eight contracts; one `059` row per acceptance criterion per gate it is tested at, so thirty-seven blocks across `GT-1` and `GT-2` is seventy-four rows — continue on further copies. **Signed by** `GB-01 Architecture Board`, which per `057` holds `GT-1` and `GT-2`.

Ref	Architecture contract	Work package	Compliance status
ACO-01	*example — delete*	WP-01	Non-compliant

`059 Compliance Assessment`. `Gate` takes `GT-0` to `GT-3`. `Verdict` and `Evidence` are filled at the gate, by the gate, and **never in advance** — an assessment row written before the sitting is a prediction.

Ref	Building block	Block name	Acceptance criterion	Gate	Verdict
CMP-01	BBB-01	*example — delete*	Owner and measure exist	GT-2	

Phase H — Architecture Change Management

Two forms. The second one feeds nothing, because the thing it feeds does not exist.

Form H-1 — Governance decision and change request

Feeds `060 Governance Decision` — `Ref`, `Decision`, `Body`, `Status` — and `061 Governance Change Request` — `Ref`, `Change request`, `Against`, `Type`, `Status`, `Expires`. **`Body` is a body name, not a reference, in the shipped data** — 'Investment Committee' — so write the name as `057 Governance Body` spells it. **`060` has no date column and no rationale column**, which means a decision in this repository has no record of when it was taken or why; put the date in front of the decision text — '2027-03-18: adopt the recommended alternative' — until you add a column, and keep the reasoning in the minute. **`061 Type` ships Dispensation, Scope, Correction and Compliance; `Expires` is a date and only a dispensation has one.** A dispensation with no expiry is a permanent exception with a temporary name. Four of the five shipped rows have an empty `Expires`. **`Against` holds a work package reference or the word 'Repository'** — which is how a correction to the file itself gets recorded, and it is the only change-control mechanism this repository has. **First pass** seven decisions, five change requests. **Signed by** the body named in `Body`.

Ref	Decision	Body	Status
DEC-01	*example — delete*	Architecture Board	Outstanding

`061 Governance Change Request`. `Status` ships Granted, Approved, Raised and Board decision required.

Ref	Change request	Against	Type	Status	Expires
CR-G01	*example — delete*	WP-04	Dispensation	Granted	2029-06

Form H-2 — The change request and change log you must build

Feeds two sheets that do not exist. This repository ships **no version control sheet, no change log, no change-request register for repository content, and no `Version`, `Record Status`, `Changed By`, `Change Ref` or `Approved By` column on any object sheet** [1]. So nothing in the file records who changed a judgement, when, or on whose authority — and every score, disposition and classification in it is therefore unattributable. Build these two sheets before you load a single record. **Build `01 Change Requests`** with: `Ref`, `Raised`, `Sheet`, `Row ref`, `Change`, `Decided`. **Build `02 Change Log`** with: `Ref`, `Date`, `Sheet`, `Row ref`, `From`, `To`. **Then add two columns to every object sheet**: `Change Ref`, holding the log reference of the last change, and `Approved By`, holding the person who signed it. Two columns, sixty-four sheets, one afternoon. **First pass** one change request per form transcribed in your first cycle, which is thirty requests and several hundred log rows. **Signed by** `GB-01 Architecture Board` once, for the structure; thereafter the person named in `Decided`. **Note** `061 Governance Change Request` is not a substitute: it holds five rows about work packages and governance departures, not about repository content, and using it for both makes neither readable.

Ref	Raised	Sheet	Row ref	Change	Decided
CR-001	2027-03-18	*example — delete*	E05	Master system	

`02 Change Log`. One row per changed cell, written when the cell changes and not in a weekly catch-up. `From` and `To` hold the old and new values.

Ref	Date	Sheet	Row ref	From	To
CL-001	2027-03-18	*example — delete*	E05	Workforce Mgmt	Price engine

Requirements Management

BEFORE YOU WRITE A REQUIREMENT DOWN

`RPT06 Requirements Traceability` has a `Reaches a block` column. How many of the thirty-one shipped requirements read No, and what does a Must requirement reaching nothing mean at a gate?

Take one obligation from `063 Control Objective` reading `Evidence` = No mechanism. What would you put in front of an inspector standing in a store this afternoon?

Every shipped requirement statement ends 'without manual reconciliation'. Which of them is actually satisfied by a spreadsheet somebody maintains, and who maintains it?

`006` has no acceptance-criterion column. Where will yours live, and what stops a requirement being signed off against a criterion nobody wrote down?

Form R-1 — Requirements, obligations and the acceptance criterion

Feeds `006 Requirement` — `Ref`, `Requirement`, `Category`, `Priority`, `Driver`, `Status` — `063 Control Objective` — `Ref`, `Obligation`, `Source`, `Evidence` — and `R09 Requirement satisfied by Bu`. **Typed into Excel** `006 Statement` and `063 Control objective`. **Shipped values**: `Category` Regulatory, Data, Integration, Business, Technology; `Priority` Must, Should, Could; `Evidence` Evidenced, Partial, In breach, No mechanism. **`006` has no acceptance-criterion column** — the criterion lives on `055 Building Block` and reaches the requirement only through `R09`, so the third table is where it is agreed and `Traced` is what a gate reads. **First pass** thirty-one requirements and six obligations. **Signed by** the Chief Compliance Officer for the regulatory rows, `GB-01 Architecture Board` for the set.

Ref	Requirement	Category	Priority	Driver	Status
REQ-001	*example — delete*	Regulatory	Must	DRV-01	Approved

`063 Control Objective`. Name the evidence artefact in the margin.

Ref	Obligation	Source	Evidence
CO-01	*example — delete*	Price display and redress	Partial

`R09`. A Must requirement reading `Traced` = No is a refusal at `GT-1`.

Requirement	Building block	Acceptance criterion the gate tests	Traced
REQ-001	BBB-01	*example — delete*	No

Transcription — the part that makes the paper worth filling

A form filled and not transcribed is worse than a form not filled, because somebody now believes the work has been done. Six rules, and they are not negotiable in a practice that has no change control.

Form to sheet the same day

Not the same week. A form transcribed four days later loses the marginal notes, the crossings-out and the reason a cell holds two values, because those are in the transcriber's memory rather than on the paper. Block the hour after the workshop and do it then. If the session runs to the end of the day, transcribe the next morning before anything else.

Who types

The person who held the pen. Not an analyst, not a graduate, not the person with the fastest typing. The pen-holder is the only person who knows that the maturity score was argued down from a 3, that the owner's name was written by somebody who then left the room, and that the second value in the master-system cell was supplied by the integration lead and not the business. Those three facts do not survive a hand-off.

Run `ZZ Checks` before and after

Read the fifteen check lines and write the numbers down with the date, transcribe, then read them again. Four lines move when you touch the interface register — unmanaged interfaces, interfaces with no logging, direct database reads, and the record count — and a line that moves in the wrong direction is a transcription error, not an improvement. A line that does not move when you expected it to means your rows did not land where you thought.

And check the sheet against itself. `ZZ Checks` claims sixty-five object types populated with 'Every type carries records' while `O59 Compliance Assessment` holds none and `O29 Gap` holds one [1]. The check sheet is a computation over the workbook and it is wrong about the workbook, which makes verifying every one of its numbers a day-one step rather than a refinement.

The change reference every row needs and the file does not have

Write the change-request reference from Form H-2 on the top of every filled form, and the same reference into the `Change Ref` column you added to the object sheet. Until those columns exist, a changed judgement is indistinguishable from an original one, and the first time somebody disputes a capability score you will have no answer. This is the single cheapest piece of work in the whole volume and the one most often deferred.

When the room disagrees

Record both values in the cell and mark the row Draft where the sheet has a `Status` column. Where it does not — and most of them do not — put the row on the `GB-01 Architecture Board` agenda

with both values and the two names, and leave the sheet cell empty until it is settled. An empty cell is honest. A cell holding the louder person's answer is a fabrication that will be quoted for two years.

Do not average. A `Current` maturity of 2 from the business and 4 from the system owner is not a 3; it is two different definitions of the capability, and finding out which is the useful half-day.

Retiring a filled form

Keep the paper until the row has been approved by whoever the form says signs it, then scan it, file the scan against the change-request reference, and destroy the paper — except where it carries personal information, in which case it is a record with a retention clock on it and goes to the group's own records process rather than into a drawer. A filled form naming customers, patients or named employees is personal information whether or not it looks like a form.

What you keep is the scan and the change log. What you throw away is the folder of unmarked copies, because the next cycle's forms will not be the same, and the quickest way to get a stale column into a repository is to reprint last year's paper.

THE CHAPTER IN TEN LINES

- Print the forms for the phase you are in, not all thirty.
- Fill them by hand in the room; a laptop steers the conversation.
- Every column is a real column of a real sheet, in sheet order, with the tab names truncated as Excel truncated them.
- Six columns maximum — a wider sheet splits into identify and assess.
- Form A-1 feeds nothing: sixty-four object sheets and no stakeholder sheet.
- Forms B-5 to B-8 feed four sheets you build, because `EXT` names the types and holds no instances.
- Form H-2 feeds two sheets you build, because this repository has no change control at all.
- Answer the ask set before the session, not during it.
- Transcribe the same day, and the person who held the pen types.
- Run `ZZ Checks` before and after; four of its lines move when you touch the interface register.

SOURCES FOR THIS CHAPTER

- **1.** Retail_TOGAF_Metamodel.xlsx — sheet names, column headings and shipped values as read from the workbook

APPENDIX G

The principles and standards starter set

Forty-nine principles and thirty-one standards somebody has already argued with, a tier column the repository does not have and cannot work without, and the ninety minutes that turn an inherited set into an adopted one.

Drafting principles from nothing is the most reliably wasted week in the Preliminary Phase. Put five retail executives in a room with a blank page and they will write forty statements nobody can argue with, in no order, with no test, and the catalogue will be quoted at one steering committee and never again. The failure is structural: a principle written to be agreed is written to be unarguable, and an unarguable statement cannot break a tie.

The useful work is adopting, editing and rejecting a set somebody else has already argued with. Argument is cheaper against a position than against a blank page. "One system masters the article; one masters the price" takes four seconds to read and provokes an immediate objection from whoever runs the promotions engine, and that objection is the beginning of a tier decision. The same person given a blank page writes "we will manage our data well", and nothing follows from it.

So this appendix hands you a set. Forty-nine principles and thirty-one standards, each marked with where it came from. `Retail_TOGAF_Metamodel.xlsx` ships twenty-four principles on `005 Principle` and sixteen standards on `032 Standard` [1]. Of the twenty-four principles, **eight are kept unchanged, fifteen are edited and one is recommended for retirement**; **twenty-five are new**, added for things a South African multi-format retailer needs and a generic set does not cover. Of the standards, sixteen come from the repository and fifteen are new, and fourteen of those are specific to retail.

Halverson Stores Group, the retailer in that repository, is invented. The shape of what it ships is not.

> **ADOPTING THE SET UNCHANGED IS THE SECOND FAILURE, NOT THE SAFE OPTION**
>
> Running this set through a workshop and emerging with forty-eight adoptions and no edits means you tested nothing against your own group. It looks like efficiency and it arrives at the same place as the blank page, faster. The adoption workshop later in this appendix sets a floor: one rejection or edit per category, or the session does not close.

What makes a principle real

A principle is real when three things are true of it. **It can be broken** — there is a decision somebody could take next Tuesday that would breach it, and you can describe that decision. **You can point at the evidence** — a sheet, a report, a demonstration that shows whether it is being followed. **Something happens when it is broken** — a refusal at `GT-1`, a dispensation with an expiry in `061 Governance Change Request`, a row in `029 Gap` with a severity on it.

Most inherited principles fail the second test. They carry a statement, which makes them a position, and an owner, which makes them somebody's position. What turns a position into a control is a named test: the thing you would ask to be shown. `O05 Principle` has no column for it — `Ref`, `Principle`, `Category`, `Statement`, `Owner`, `Status`, and that is all [1] — so the test lives in the adoption record until you add the column. Add the column.

The three-question motherhood test

#	Question	Pass	Fail
1	Would any sane executive argue the opposite?	Somebody does argue it, regularly, and you can name when	Nobody argues against it. Add the arguable clause, or drop it
2	Does it say what to do when two good things conflict?	It names what it yields to, or what yields to it	It states a good outcome with no priority. Add the tier and the exception
3	Can you name a decision it would have changed in the last year?	You can name the decision, the date and what changed	You cannot. It is decoration, or nobody consulted it

Question three has no mechanical answer in this repository, and that is worth sitting with. There is **no relationship sheet binding a principle to anything** — twenty-five relationship sheets, and not one of them has a principle at either end — and no report over `O05`. So nothing counts how many requirements a principle governs, and a principle that governs nothing looks exactly like a principle that governs everything. Step 4 of the workshop below says what to add.

Two worked sharpenings

Weak. `PRN-04 Business Continuity` — *operations continue despite the loss of any single component* [1]. Nobody argues against it, so it fails question one. Nothing in the repository could fail it either: `049 Resilience Tier` holds twelve rows and eight read `Tested` = Never tested, and the principle is silent on that because it talks about design rather than about proof.

Sharpened. *A store continues to trade through the loss of the network, the power or any single central component, demonstrated by a test inside the tier's stated cadence rather than by a plan.* Both clauses are argued against: the first by anybody costing local compute into four hundred sites, the second every time a store manager declines a test window in a peak period. **The test:** the last recovery test on one Tier 1 application, its date, what it exercised, and the `029 Gap` row raised by anything it failed. It costs nothing to run and it will read badly, which is the point.

Weak. `PRN-10 Data is Shared` — *data is shared through published interfaces, not copies* [1]. Agreed by everyone, and the repository's own `ZZ Checks` reports nine direct database reads and sixteen unmanaged interfaces out of twenty-nine [1]. A principle the file it ships in contradicts on its own check sheet is not being applied.

Sharpened. *Data is shared through a published, versioned interface; the system holding a record may not refuse access to the designated owner, and no application reads another application's store.* Three clauses, three opponents: the integration team that wants a file drop before the trading period starts, the category team that will not release its supplier terms, and the reporting team that already has a read replica.

The test: `RPT04 Interface Catalogue` filtered to `Type` = Direct DB read, with a dated containment or a gap row against every one. The principle sits at Tier 3 because it loses predictably to purpose limitation and to classification — the honest position, and the one that makes it usable.

The set

Six categories, forty-nine rows. `Ref`, `Principle` and `Statement (condensed)` transcribe into `005 Principle` columns A, B and D; `Category` comes from the heading above each table; `Owner` and `Status` come off Form PS-1. `Tier` and `Source` are **not `005` columns** — `Tier` is a column you add, and `Source` is working information for the adoption pass that you drop when you paste.

> **TWO COLUMNS THE SHEET DOES NOT HAVE, ONE VALUE IT DOES NOT HAVE, AND ONE IT FILLS WRONGLY**
>
> `005 Principle` has **no `Tier` column**, so the conflict rule in `PRN-46` has nothing to read. Add `Tier` before you load the set; it is one integer per row and it is the difference between a catalogue and a control. `005` also has **no `Category` value for Governance** — the shipped data uses Business, Application, Data, Technology and Security — so the four governance principles below need a sixth value. `Status` ships only Approved and Draft: Retired is a third value you add, because deleting a retired principle breaks every decision that cited it. And **all twenty-four shipped rows carry `Owner` = 'Architecture Board'**, which is a committee and therefore not an owner. Replace every one with a named executive role before the set is loaded.

Business

Ref	Principle	Statement (condensed)	Tier	Source
PRN-01	Primacy of Principles	Applies to every format and every initiative, including franchise, voluntary-member, concession and outsourced work	2	edited
PRN-02	Maximise Benefit to the Enterprise	Group benefit outranks the fastest option for one format, and the cost of the slower path is stated and accepted	3	edited
PRN-03	Compliance is Designed, Not Added	Obligations are architecture requirements from the outset, with the evidence designed in and named	1	repository
PRN-04	Business Continuity	A store trades through the loss of network, power or any central component, demonstrated by a test in cadence	2	edited
PRN-25	Every Article Carries Its Regulatory Classification	Schedule, age restriction, labelling class and site-level sale permission are article attributes, derivable at the till	1	new
PRN-26	The Displayed Price Is the Price	Every place a price is displayed is a controlled publication with a confirmed delivery and a reconciliation back to the master	1	new
PRN-27	A Trading Site Is a Licensed Object	Every site carries its licences, conditions and expiry dates as data, and a lapsed licence blocks the category at the till	1	new
PRN-28	Model the Estate As It Trades	Where the group does not own the store it does not master the price, the range or the stock; the model says which, per site	2	new
PRN-29	The Store Keeps Trading	Local capability, local price and local authorisation are designed for, not degraded modes discovered during an outage	2	new

Data

Ref	Principle	Statement (condensed)	Tier	Source
PRN-09	Data is an Asset	Every data entity has an owner, a definition and a lifecycle, and the lifecycle includes disposal	3	edited
PRN-10	Data is Shared	Shared through a published versioned interface; the holder may not refuse the designated owner; no application reads another's store	3	edited
PRN-11	Data is Accessible	Those entitled can reach it without a project, within the classification that governs it	3	repository
PRN-12	Common Vocabulary	One agreed definition per concept; where two parts of the group disagree, the disagreement is recorded and visible	2	edited
PRN-13	Data Ownership is Individual	An owner is a named person, not a committee and not a department	2	repository
PRN-14	Classification Travels with Data	Handling follows the data, not the report or the system it happens to sit in, and reaches the attribute	2	repository
PRN-30	One Article, Three Levels, One Owner Per Level	The levels at which the group masters a sellable thing are named, and each has one master and one owner	2	new
PRN-31	An Identifier Identifies a Trade Item at a Packaging Level	Consumer unit, inner pack and case are different identifiers with a held hierarchy, not one product with variants	2	new
PRN-32	Two Schemes, One Mapping, One Owner	Where a product carries a selling identifier and a reimbursement identifier, the group owns the bridge and names the owner	2	new
PRN-33	Price and Assortment Are Effective-Dated Entities	Both carry effective-from and effective-to with the hour; neither is an attribute overwritten in place	1	new
PRN-34	The Trading Calendar Lives in the Dimension	Fiscal week, period and the comparison key are held in the time dimension, including the fifty-third week rule	2	new
PRN-35	A Stock Figure Names Which Stock It Is	Every quantity states its state and its location grain; 'stock on hand' alone is not a figure	2	new
PRN-36	Retention is the Longest Clock	Each entity carries the longest statutory clock that runs on it, with the instrument named; dispensing never inherits the transaction schedule	2	new
PRN-37	Consent and Purpose Are Held Per Member Per Channel	Purpose, consent and the chosen contact method are held per person per channel, and processing outside the stated purpose does not happen	1	new

Application

Ref	Principle	Statement (condensed)	Tier	Source
PRN-05	Common Use Applications	Prefer a shared capability to a duplicated one — except where a regulated practice inside the estate must stay separate	3	edited
PRN-06	Technology Independence	Capability and data are not permanently bound to a platform or supplier, and the exit is costed before selection	3	edited
PRN-07	Ease of Use	A system that is hard to use will be worked around, and the workaround becomes the real process	3	repository
PRN-08	Service Orientation	Capability is exposed as a versioned, documented service with a contract; the contract outlives the implementation	2	edited
PRN-38	Translate at the Trading-Partner Boundary	Partner message dialects are translated at the edge; the internal model is never restructured to match one partner's guide	2	new
PRN-39	One System Masters the Article; One Masters the Price	Exactly one master per master-data entity, named in `020 Data Entity`, and every other system consumes it	1	new
PRN-40	The Store Device Estate Has Its Own Release Cycle	Peripherals and payment terminals are certified assets with their own cadence, planned apart from the application release	3	new
PRN-41	Confirmed Delivery, Per Site, Per Version	Anything published to stores — price, promotion, planogram, clinical content — reports what landed where, at which version	2	new

Technology

Ref	Principle	Statement (condensed)	Tier	Source
PRN-15	Requirements-Based Change	Change is driven by a stated requirement that traces to a driver; an initiative with neither is somebody's preference	3	edited
PRN-16	Responsive Change Management	*Recommended for retirement — see below*	—	retire
PRN-17	Control Technical Diversity	Each capability has one approved technology unless a departure is recorded with a date	2	repository
PRN-18	Interoperability	Standards are chosen for interoperability with the trading partners and identifier schemes the group actually uses	2	edited
PRN-24	Sustainable by Design	Energy and resource cost is assessed and recorded, including where it loses	3	repository
PRN-42	Card Data Scope Is an Architecture Decision	Whether the till sees a card number, whether the store network is segmented and who holds the token vault are design choices	2	new
PRN-43	The Batch Clock Is Designed, Not Inherited	Cut-offs, overnight windows and the start of the trading day are a designed property with a named owner	3	new

Security & Risk

Ref	Principle	Statement (condensed)	Tier	Source
PRN-19	Security is Built In	Controls are a design input proportionate to classification and criticality, and the proportionality judgement is recorded	2	edited
PRN-20	Least Privilege	Minimum access for a stated role, reviewed on a stated cycle, with the review evidenced	2	edited
PRN-21	Attributable Access	Every access to sensitive data is attributable to an individual, not to a service account or a shared till login	2	repository
PRN-22	Defence in Depth	No single control is relied on alone, and the second control is named	2	edited
PRN-23	Models are Governed	Any model influencing a customer or a trading outcome is registered, explainable, monitored and owned by a person	2	edited
PRN-44	Surveillance is a Registered Asset	Every camera and analytic is registered with its purpose, lawful basis, retention and whether it produces a biometric template	2	new
PRN-45	Automated Customer Decisions are Declared and Reconstructible	Every automated decision about a person records its inputs, its model version and a human-review path that exists	1	new

Governance

Ref	Principle	Statement (condensed)	Tier	Source
PRN-46	Principles Are Applied as a Set	Higher tier prevails; at equal tier the more specific prevails; otherwise the Architecture Board decides and the decision is a precedent	1	new
PRN-47	A Gate Must Be Able To Say No	A gate that has never refused anything is not a gate; its mandate is reviewed and its refusals are counted	2	new
PRN-48	Dispensations Expire	Every departure is explicit, dated, owned by an executive, and the expiry has a stated consequence	2	new
PRN-49	A Decision the Repository Cannot See Did Not Happen	Architecture decisions are recorded where the repository points at them, with the principles the decision engaged named	2	new

`PRN-16 Responsive Change Management` — *the estate changes at the pace the business needs* — is the one row to retire. Nobody argues the opposite, nothing could fail it, and in practice it is cited only by whoever wants a gate skipped before a trading period. Retire it by setting `Status` to Retired rather than deleting the row, and point anything that cited it at `PRN-15` and `PRN-47`.

The twelve that carry the weight

For each: why this group needs it, what somebody has to do differently on Monday, and **the test** — the thing you ask to be shown, naming the sheet or report. Volume 1 carries the teaching; this does not repeat it.

PRN-03 Compliance is Designed, Not Added (Tier 1, kept). `063 Control Objective` ships six obligations and three read `Evidence` = In breach or No mechanism [1]. That is what retrofitting looks like. Retail enforcement is also unlike most sectors: an inspector arrives in one store, unannounced, during trade, and forms a view of the group from what that site was doing on a Tuesday. **Monday:** every obligation gets a control objective and a named evidence artefact before the build starts, and `RPT09 Regulatory Coverage` is read at `GT-1` rather than at audit. **The test:** `RPT09` with no row reading No mechanism, and for each row an artefact you could put in front of an inspector this afternoon. **Look it up.** Volume 1, chapter 15, *What an inspector actually asks to see*.

PRN-25 Every Article Carries Its Regulatory Classification (Tier 1, new). What a till may sell is a property of the article and the site together, and the classifications are not the merchandising hierarchy. A medicine's schedule decides whether it may be sold at this site at all, whether a pharmacist must be present and whether a register entry is generated — and the regulator amends the consolidated schedules by gazette, most recently those of August 2025, which makes schedule a dated attribute rather than a constant [2]. Age restriction, labelling class and site-level sale blocks behave the same way. **Monday:** the article master grows the classification attributes and schedule-change ingestion becomes a process with an owner. **The test:** one scheduled line, its attribute, its effective date and the till behaviour it drives at three site types. **Look it up.** Volume 1, chapter 15, *Pharmacy and health products*.

PRN-26 The Displayed Price Is the Price (Tier 1, new). Section 23 of the Consumer Protection Act requires a displayed price, forbids charging more than it, and — the clause that converts an operational nuisance into a statutory exposure — provides that where more than one price is concurrently displayed the lower or lowest binds [3]. A shelf-edge label that silently failed to update is a second concurrently displayed price, and the till is charging above a displayed price at every store where it did not land. **Monday:** every price display surface is a controlled publication — shelf label, website, app, promotion ticket, scale label — with a confirmed delivery and a reconciliation to the master. **The test:** for yesterday's price changes, the count of surfaces confirmed and unconfirmed, per store. No system producing that number means the group cannot evidence a provision an inspector tests by scanning. **Look it up.** Volume 1, chapter 15, *Section 23, and why a stale shelf label is a legal exposure*.

PRN-27 A Trading Site Is a Licensed Object (Tier 1, new). A liquor licence, a pharmacy licence and a certificate of acceptability for food premises are each per premises, with their own conditions, numbers and expiry dates, and a pharmacy must be recorded with its council as well as licensed. `012 Location` holds four columns and can carry none of it. **Monday:** the location master grows a licence structure — instrument, number, conditions, issue, expiry — and a lapsed licence blocks the category at the till rather than producing a report. **The test:** every site whose licence expires inside ninety days, and what the till does on the day after. **Look it up.** Volume 1, chapter 15, *The inspector is in the store*.

PRN-30 One Article, Three Levels, One Owner Per Level (Tier 2, new). Which levels of a sellable thing the group masters is the argument the data model lives or dies on, and in a multi-format group apparel needs style, option and SKU while grocery needs consumer unit, inner pack and case — both right. What is not survivable is one level with three systems each believing they hold it. **Monday:** the levels are named on Form B-5 of appendix F, one master and one owner each. **The test:** reconcile a distribution-centre receipt against a store sale for one line, across the packaging hierarchy, with no manual step. **Look it up.** Volume 1, chapter 9, *Article, style, option, SKU: the argument the model lives or dies on*.

PRN-33 Price and Assortment Are Effective-Dated Entities (Tier 1, new). Price is not an attribute of an article; it is an entity with a key of article, location, channel and period, and so is assortment. `O20 Data Entity` already carries `E05 Price` defined as *price by SKU, location, channel and period* — and then records its `Master system` as Workforce Management, which is what happens when a column is filled without an argument [1]. Overwriting a price in place destroys the ability to answer what was charged last Thursday, which is exactly the question a redress claim asks. **Monday:** no price or range record is updated in place; a change is a new effective-dated row, and the hour is part of the key. **The test:** state the price of one article at one store at 14:00 last Thursday, from the master, without a restore. **Look it up.** Volume 1, chapter 10, *Effective dating on price and assortment*.

PRN-37 Consent and Purpose Are Held Per Member Per Channel (Tier 1, new). The loyalty programme is the group's largest holding of personal information and its most purpose-ambiguous. The Regulator's position on direct marketing is specific: consent before electronic marketing to non-customers, an approach permitted once only and only where the person has not previously refused, consent on the prescribed form or one substantially similar, the goods named, **the communication method the person chooses recorded and then used**, the burden of proof on the responsible party, and telephone calls counted as electronic communications [4]. A multi-banner group joining banner datasets into one customer view without that purpose consented at enrolment has made a breach look like an architecture decision. **Monday:** consent is a structure — person, purpose, channel, date, evidence — not a flag. **The test:** for one member, render every consented purpose by channel, with the date and the form version. **Look it up.** Volume 1, chapter 15, *POPIA, with the retail bite*.

PRN-39 One System Masters the Article; One Masters the Price (Tier 1, new). This is the principle the shipped repository breaches most comprehensively. `O20 Data Entity` records `E01 SKU / Article` with `Master system` = POS, `E03 Store / Location` mastered by Loyalty & CRM, `E07 Planogram` mastered by Loyalty & CRM and `E05 Price` mastered by Workforce Management — while `O43 Master Data Rule` gives `SKU / Article` a `Golden source` of Demand Forecasting & Replenishment, so two sheets disagree and no report compares them [1]. **Monday:** one master per entity, named in `O20`, agreed by `GB-03 Data Governance Council`, and `O43 Golden source` made to match. **The test:** `RPT05 Data Ownership` beside `O43`, entity by entity, with no disagreement and no entity reading two masters.

PRN-41 Confirmed Delivery, Per Site, Per Version (Tier 2, new). Four things are published from the centre to hundreds of sites — the price file, the promotion file, the planogram and, in a pharmacy, the clinical content behind interaction checking. The pharmacy case is sharpest: the Good Pharmacy Practice rules require that where software provides warnings or interaction checks, the latest version must be used [5], which turns a content database into a currency obligation on the compliance critical

path, landing on the endpoints least able to bear it. **Monday:** every publication reports what landed, where, at which version, and an unconfirmed site raises an exception rather than a silence. **The test:** the current content version per site, for all four publications, as at this morning. **Look it up.** Volume 1, chapter 15, *Good Pharmacy Practice, and a rule that turns a database into a compliance obligation*.

PRN-42 Card Data Scope Is an Architecture Decision (Tier 2, new). The card data security standard is contractual rather than statutory — it flows from the merchant agreement and the scheme rules, so failure brings an acquirer's response rather than a prosecution: a trading risk, not a legal one [6]. The architecture decides the size of the problem: whether the till ever sees a card number, whether the store network is segmented from corporate and guest wireless, and where tokenisation happens and who holds the vault — the exit cost when the acquirer changes. **Monday:** three recorded decisions with named owners, not consequences of a terminal procurement. **The test:** draw the card data flow for one lane and name every component inside scope. **Look it up.** Volume 1, chapter 13, *PCI DSS and EMV — contractual, not statutory*.

PRN-45 Automated Customer Decisions are Declared and Reconstructible (Tier 1, new). Section 71 of the privacy statute restricts a decision with legal or substantial consequences based solely on automated processing intended to create a profile. In retail that reaches the store-card credit decision, automated fraud blocking of a refund or an account, personalised pricing and an automated refusal to accept a return — each needing a human-review path, a representation mechanism, a renderable reason and the retained inputs that make the reason true. **Monday:** a design promising fully automated refund decisioning has acquired a designed exception path with a service level. **The test:** one automated refusal from last month, with the reason, the inputs as at the decision and the review route.

PRN-46 Principles Are Applied as a Set (Tier 1, new). This is the conflict rule, and without it the other forty-seven are applied selectively by whoever is in the room. It is new because the repository ships no equivalent, and it is Tier 1 because a breach of it invalidates every other verdict. **Monday:** `005 Principle` has a `Tier` column, and every refusal cites a rung. **The test:** one `060 Governance Decision` row that names two conflicting principles and the rung that settled it. Where none exists the ladder has never been used, whatever the catalogue says.

The tier problem, and the tie-break ladder

`PRN-46` says the higher tier prevails. The repository gives it nothing to read: `005 Principle` has no `Tier` column at all, and twenty-one of its twenty-four rows read `Status` = Approved with three at Draft [1]. So as shipped, every principle is equal and the conflict rule is decorative.

The failure mode once you add the column is the opposite and just as predictable. Put twenty of forty-eight at Tier 1 and the rule resolves almost nothing, because nearly every pair is Tier 1 against Tier 1. Then the rule gets abandoned, and after that principles are applied selectively anyway. **Tier is not a measure of importance. It is a tie-break mechanism, and a tie-break mechanism with no discrimination is not one.**

The proposed distribution

Band	Count	Share	What qualifies
Tier 1 — the apex	9	19%	Breach is a statutory exposure, a licence matter, or the loss of an answer the group is obliged to be able to give
Tier 2 — the working set	27	56%	The body of the practice: ownership, identifiers, interfaces, publication, resilience, security, governance records
Tier 3 — yields	12	25%	Principles that should predictably lose, and say so

The nine at Tier 1: `PRN-03` obligations are requirements, `PRN-25` article classification, `PRN-26` the displayed price, `PRN-27` the licensed site, `PRN-33` effective dating, `PRN-37` consent and purpose, `PRN-39` one master, `PRN-45` automated decisions, and `PRN-46` the conflict rule itself.

The admission test for Tier 1: would you stop a release for it? Not "is it important" — everything in the catalogue is important, which is exactly why importance cannot be the criterion. Nine is small enough that an executive can recall the band without the sheet, which is the property that makes rung one work without a meeting. Note what is *not* in it: availability, forecast accuracy, space productivity and cost to serve are all commercially larger and none of them is a tie-break.

Tier 3 is the band people get wrong. It is not a bin for weak statements. It is the explicit band of principles that should lose, and that say what they lose to: sourcing preference to a trading-period deadline, technology independence to the only viable supplier in a thin market, accessibility to classification, sharing to purpose limitation, the designed batch clock to a trading peak, the device release cycle to a scheme certification date. Saying so in the tier is what makes them usable, because a principle overridden quietly stops being consulted at all.

The ladder

Rung	Rule	What it resolves	How it is recorded
1	The higher tier prevails	Tier 1 against anything; Tier 2 against Tier 3	Mechanical. No judgement, no meeting
2	At equal tier, the more specific prevails	Most Tier 2 against Tier 2 conflicts	The architect applies it; the `O60 Governance Decision` row names the rung
3	Otherwise the Architecture Board decides, and the decision is a precedent	Genuine ties at equal specificity	`O60 Governance Decision` with both principles named, cited next time rather than re-argued

Rung two does the heaviest lifting, because general-against-specific is the common shape of a real conflict: a principle about the price file beats a principle about publication generally; a principle about the dispensary beats a principle about store systems.

> **TIER SETTLES ABOUT A THIRD OF THE CONFLICTS YOU WILL ACTUALLY MEET**
>
> Be honest about this with the Board, because somebody will test the arithmetic. Forty-eight live principles make 1,128 possible pairs, and of those 657 cross a tier boundary, which sounds decisive. It is not, because conflicts do not arrive evenly: the working body of the practice is Tier 2, and 351 of the 471 same-tier pairs are Tier 2 against Tier 2. Those are the arguments you have in real life — ownership against a trading deadline, security against till speed, lineage against cost, the designed batch clock against a peak — and rung one is silent on every one of them. Count the conflicts you meet rather than the pairs you could draw, and tier settles roughly a third. That is a property of any set this size, and it is why rungs two and three are written down rather than improvised.

Re-tiering happens by recorded decision, never by preference. `PRN-06 Technology Independence` is the obvious candidate: it arrives at Tier 1 on inherited authority and belongs at Tier 3 the moment a space-planning tool or a dispensing system has one viable supplier in this market. Move it with an `O60 Governance Decision` row and the Board's reasoning, and it becomes a principle that does work — it now loses, predictably, to something named.

Reviewing the distribution is annual work, run alongside the maturity assessment in appendix H. Three checks. Can an executive name the Tier 1 principles without the sheet? Has any Tier 3 principle won a conflict — if so it is mis-tiered. Has any Tier 1 principle never been cited — if so it is either not a principle or nobody is consulting the set.

Settle Tier 1 **before** the workshop, with the Chief Risk Officer and the Chief Compliance Officer. That band is where the regulatory argument sits, and it is the only band that should not be decided by a show of hands.

The standards catalogue

Thirty-one standards, shaped for `032 Standard` — `Ref`, `Standard`, `Category`, `Status`, `Conformance` [1]. Sixteen come from the repository — fourteen unchanged, two edited — and fifteen are new; **fourteen are specific to retail and are marked**. `Category` takes the six values the sheet already uses: Integration, Data, Security, Platform, Delivery, Governance. There is no Identification value and no Retail value, so the identifier standards are filed under Data and the carrier standards under Integration — write that mapping down once, in the `GB-01 Architecture Board` minute, or the catalogue will be re-filed every year.

Identification, carriers and traceability — the retail core

Ref	Standard	Category	Status	Retail	Source
STD-17	Identification key standard — trade item, location, logistic unit	Data	Current	yes	new
STD-18	Data carrier and two-dimensional readiness standard	Integration	Emerging	yes	new
STD-19	Visibility event standard for traceability and recall	Data	Emerging	yes	new
STD-27	Article and merchandise hierarchy standard	Data	Current	yes	new
STD-30	Dispensing and medicine identifier standard	Data	Current	yes	new

`STD-17` is the standard that genuinely binds a retailer, because these identifiers are the primary keys of the estate. The authority is the GS1 General Specifications, release 26.0, ratified January 2026, in nine sections covering identification, application standards, identifier definitions, application rules, carriers, placement, validation, profiles and a glossary [7]. Three rules go into your own standard. A trade item number identifies an item **at a packaging level**, so consumer unit, inner pack and case are three numbers with a hierarchy. A location number identifies a party **or** a location, and store, distribution centre, legal entity, ship-to, bill-to and remit-to are distinct — trading partners discover this at the first invoice dispute. A logistic unit number identifies one physical unit once; reuse is what makes receiving automation impossible. Cite the release, not a web page: GS1 states that where a web rendering conflicts with the published document, the document governs [7].

`STD-18` is where most steering committees are misinformed, so write it carefully. **GS1 does not require two-dimensional barcodes by 2027.** GS1 US's own topic page describes a global transition to two-dimensional carriers, calls 2027 a milestone, **gives no day or month**, runs a voluntary crawl-walk-run model, and states that it is not necessary to implement all two-dimensional capabilities by that date [8]. The sourced position is GS1 US; the global and South African positions are not established and should not be inferred from it. What is durable is the consequence: the binding constraint is the **scanner and point-of-sale estate**, not packaging. Phase one is a testable question — can every lane, self-checkout, handheld and receiving scanner read a two-dimensional symbol, extract the trade item number, and does every downstream system tolerate the batch, expiry and serial elements arriving with it? After that the till becomes a traceability capture point, date-based enforcement at the lane becomes possible, and the scan becomes a consumer touchpoint the retailer does not control, because

the identifier resolves to the brand owner's content and the brand owner can change what sits behind it without changing the pack [9]. **Look it up.** Volume 1, chapter 13, *The two-dimensional migration, and Sunrise 2027 precisely*.

STD-19 is rejected by architects who think it is a radio-frequency standard. It is an event standard: it requires neither electronic product codes nor radio-frequency identification, and since version 1.1 not even instance-level identification; versions 2.0 of the event standard and its companion vocabulary were ratified in June 2022 [10]. That vocabulary supplies the controlled values without which two implementations exchange well-formed events that mean nothing to each other. Adopt it for traceability, recall and cold chain — not as a message standard for commercial documents and not as an inventory system.

STD-27 has no external authority and that is the point of writing it down. The group runs several hierarchies — merchandising, reporting, supplier, space, tax, online navigation — and **all are correct for their purpose**. The standard names which exist, which is the merchandising hierarchy of record, who owns each, and the derivation rule where one exists. Without it the group has six hierarchies and one argument. **Look it up.** Volume 1, chapter 9, *The six hierarchies, all of them correct*.

STD-30 exists because a South African retail pharmacy's item master carries **two identifier schemes for the same product**: the trade item number used for receiving, scanning and selling, and the reimbursement code used for claiming, maintained by different bodies on different cadences, not mapping one to one — and the retailer owns a mapping nobody else maintains. The standard names the owner, the cadence, the reconciliation report and the exception process. It also has to carry the single-exit-price mechanics, where a medicine's price is set outside the retailer and changes by determination: **there were two adjustments in 2026**, 1.47% effective in January and an extraordinary 2.88% effective 1 October on medicines listed as at 30 September [11][12]. Any design that treats the medicine price as a retailer-set price is wrong in a way that produces a regulatory finding, not a margin variance.

Trading-partner integration

Ref	Standard	Category	Status	Retail	Source
STD-01	API design standard	Integration	Current	—	edited
STD-02	Event schema standard	Integration	Current	—	repository
STD-20	EDI message set standard	Integration	Current	yes	new
STD-21	Trading-partner implementation guide standard	Integration	Emerging	yes	new
STD-28	Planogram and space data interchange standard	Integration	Emerging	yes	new
STD-29	Supplier and trade-item data onboarding standard	Data	Emerging	yes	new

STD-20 and STD-21 together kill one illusion: that a modern retailer's supplier integration is an interface estate. It is EDI, it is decades old, and the parts that are not EDI are flat files. The grammar a South African retailer meets is the GS1 EDI family — EANCOM, a compliant subset of UN/EDIFACT

— with maintenance releases typically every two years and code lists updated up to four times a year, so the translator carries a version matrix and the code-list cycle runs four times faster than the message-structure cycle [13]. The other grammar is ANSI X12, whose licensing posture is the practically important fact: exclusive copyright, a tiered licensing programme, published once a year in January [14] — so embedding segment definitions in your documentation needs a licence. **`STD-21` is the one that costs money**: every retailer publishes its own implementation guide and no two agree on optional segments, so the estate is a mapping per trading partner. **Measure the backlog in trading partners, not in interfaces.** Replacing a merchandising system does not mean rebuilding twelve interfaces; it means re-certifying several hundred partner maps. **Look it up.** Volume 1, chapter 13, *EDI, and which messages actually matter*.

`STD-29` governs how supplier-authored item data enters the estate. A global synchronisation network moves the item master from supplier emails to a subscription, which is a real gain — but it carries supplier-authored *trade item* data, not the retailer's merchandising attributes, hierarchy, cost structure or ranging decisions. A retailer treating it as its product information system has outsourced its item master to its suppliers' data quality. The standard names which attributes arrive from the supplier, which the group authors, and who adjudicates a conflict.

Store, till and payment

Ref	Standard	Category	Status	Retail	Source
STD-22	Point-of-sale peripheral interface standard	Platform	Current	yes	new
STD-23	Card data security standard	Security	Current	yes	new
STD-24	Electronic shelf label interface standard	Platform	Emerging	yes	new
STD-25	Price and promotion file distribution standard	Data	Current	yes	new
STD-09	Container image standard	Platform	Deprecated	—	repository
STD-11	Network segmentation standard	Security	Deprecated	—	repository

`STD-22` corrects a widespread error. The peripheral interface standard is current and maintained: UnifiedPOS 1.15.1 was adopted in December 2024, superseding 1.15, published with machine-readable schema files under a reasonable-and-non-discriminatory licensing mode, with a 2024 contributor list that reads as a who's-who of the live store-systems market [15]. **Do not call it legacy.** It gives one procurement question worth more than a page of requirements: which version do the peripheral drivers implement — the answer decides whether swapping a scanner, scale or printer is a configuration change or a project. A related caution: the older retail data models from the same standards lineage are **not in the formal specification catalogue**, are undated and may not be obtainable in usable form [16]. Do not send a colleague to fetch one — ask the standards body whether it exists, at what version and on what terms, and record the answer with the date.

`STD-23` is the card data security standard, contractual rather than statutory, published by a council formed by the card schemes; the document library features version 4.0.1 as current [6]. State the version and the date you checked it, and **state no other date you cannot source** — a detailed timeline

of errata and future-dated requirements circulates and the council's own pages do not confirm it. What the standard settles for the data model is firm: the card number may be stored only rendered unreadable, and sensitive authentication data must not be stored after authorisation at all, even encrypted. Entity-level classification cannot express that, which is why `PRN-14` has to reach the attribute.

`STD-24` and `STD-25` are the same problem twice — publications from the centre to hundreds of endpoints, both legally loaded because of `PRN-26`. The shelf-label standard covers the interface, the update protocol, the confirmation message and the failure behaviour: a label that cannot be reached must be reported, not left reading yesterday's price. The price-file standard covers effective dating, the publication window against the trading day, the confirmation per site and the reconciliation. Neither has an external authority, and both belong in the catalogue — otherwise the only record of how prices reach stores is in one integration developer's head.

Data, calendar, security and delivery

Ref	Standard	Category	Status	Retail	Source
STD-26	Trading calendar standard	Data	Current	yes	new
STD-03	Data classification standard	Data	Current	—	repository
STD-04	Data retention standard	Data	Current	—	edited
STD-05	Identity and access standard	Security	Current	—	repository
STD-06	Encryption standard	Security	Current	—	repository
STD-07	Logging and monitoring standard	Security	Current	—	repository
STD-08	Cloud landing zone standard	Platform	Current	—	repository
STD-10	Database standard	Data	Current	—	repository
STD-12	Coding and review standard	Delivery	Emerging	—	repository
STD-13	Environment standard	Delivery	Deprecated	—	repository
STD-14	Naming convention	Data	Current	—	repository
STD-15	Records management standard	Governance	Current	—	repository
STD-16	Supplier security standard	Security	Emerging	—	repository
STD-31	Data residency and transfer basis standard	Security	Emerging	—	new

`STD-26` looks like a finance topic and is not. The four-week, five-week, four-week calendar is published by the National Retail Federation as **a voluntary guide**, derived in the 1930s and in use among stores since the 1940s; a fifty-two-week year is 364 days, so a fifty-third week is added roughly every five or six years, and a fifty-three-week year is restated the following year by shifting each week back one [17]. **Do not assume those dates apply here.** Whether a South African group runs 4-5-4, 4-4-

5 or a 52/53-week calendar aligned to its own year end is settled by that group's own annual report, and the calendar does not align to tax periods or payroll, so the group runs two calendars and must reconcile them. The standard records the consequences: the time dimension carries the retail calendar alongside the Gregorian one with the comparison keys; the restatement convention is **encoded in the dimension, not applied in a report**; forecasting engines take the calendar as configuration rather than assuming fifty-two weeks; and merchandising, planning, point of sale, finance and the warehouse all agree what week one is. **Look it up.** Volume 1, chapter 10, *The retail calendar, and the week that is not there*.

`STD-04` is not one period. The group's retention schedule is the maximum of several statutory clocks running on the same record, and pharmacy is the case that breaks a single policy: dispensing records run on a clinical clock and must not inherit the transaction retention schedule [5]. The repository ships this standard as Current; whatever `Conformance` you record against it, record the instrument behind the longest clock.

The status vocabulary, and the conformance vocabulary

A standard has a `Status`. A component has a position against it. The assessment between the two is a conformance verdict, and the two get confused constantly [1].

Value	Used on	What it obliges	What it is not
Current	`032 Status`	Refuse a non-conforming new design; raise a gap against existing non-conformance	Not a claim the estate conforms. A long non-conformant list means enforcement, not re-drafting
Emerging	`032 Status`	Refuse non-conforming new work; leave existing components until they change anyway	Not a draft and not optional. One you will not enforce on new work should be withdrawn
Deprecated	`032 Status`	Stop citing it at gates; point references at the replacement; keep the row	Not deleted. The repository ships three Deprecated standards, two with live conformance verdicts against them
Conformant	`R18 Conformance`	Nothing, until the next assessment or a change to either side	Not permanent. An undated verdict is an assertion
Partial	`R18 Conformance`	A note naming exactly what is unmet, and a dated remediation	Not a pass. Partial at `GT-1` is a refusal unless a dispensation carries an expiry
Breach	`R18 Conformance`	A dated containment with an executive owner, or an `029 Gap` row with a severity. One of the two	Not automatically a defect. Some non-conformance is the right economic answer; undecided never is

THE DENOMINATOR RULE

Report conformance as assessed-and-conformant over **in scope**, never over assessed. A catalogue that assesses only what it expects to pass produces a rising conformance rate and a falling estate quality, and `RPT12 Standards Conformance` cannot tell the two apart — five of the sixteen shipped standards read `Not assessed`, so a headline computed over assessed rows is computed over two-thirds of the catalogue [1]. The same arithmetic applies to review: a review either reaffirms, edits or retires, and cannot defer. A deferred review is how Emerging becomes permanent.

The adoption workshop

Ninety minutes, six people, one change request out the other end. Appendix C holds the facilitation craft and this does not repeat it. What follows is the running order and the recording discipline specific to a principles pass.

In the room. The Chief Enterprise Architect chairs and does not advocate: a chair arguing for a principle guarantees consent rather than adoption. The Chief Information Officer settles whether the sourcing order, the interface rule and the device release cycle survive the delivery portfolio. The Chief Data Officer settles whether one named owner per entity is a decision this group will take. The Chief Commercial Officer settles the article, price and assortment principles, which land on buying, pricing and space and nowhere else. The Chief Risk Officer settles the Tier 1 band. The Chief Compliance Officer settles whether the pricing, consent, licensing and pharmacy principles carry the obligations, and is the person most likely to add one in the room.

Not in the room. Solution architects, project managers and vendors — affected, and not the owners. Bring their objections as a written, attributed pre-read. A principles workshop with twenty people in it produces a vocabulary, not a decision.

Segment	Minutes	What happens
Frame it	10	Two slides: what a principle is for, and the three-rung ladder. Then the session rule — silence is adoption, anyone may reject anything, and a rejection must name the decision the principle would have got wrong
The nine Tier 1 principles	20	One at a time, two minutes each. No re-tiering here; re-tiering is a recorded decision, not a show of hands
Everything else, by category	35	Reference and statement read aloud, nothing else. Objections get a two-minute box, then edit, reject or defer. Six categories, six minutes each
Bind Tier 1 to requirements	15	One `006 Requirement` row per Tier 1 principle. Where none exists, the requirement is missing, not the link
Close	10	Read back the counts. Name the change request, the date `Status` moves off Draft, and who re-points anything that cited `PRN-16`

The five steps, in order, and only the second one happens in the room.

1. **Pre-mark the whole set and settle Tier 1 before the room sits.** One day of your own reading against the three-question test; most of the twenty-four shipped statements fail question one. Strike any of the twenty-five new rows you would not defend yourself. Then settle the nine Tier 1 rows with the Chief Risk Officer and the Chief Compliance Officer **separately**, before the workshop, and circulate the marked set with the ladder on one page at least three working days ahead. Nothing is written to `005 Principle` at this step — a principle in the repository before it is adopted is a principle the Board reads as already decided. Walk in without the Tier 1 band settled and the room will spend forty minutes on tier and never reach the other thirty-nine rows.

2. **Run the ninety minutes to the running order above.** Forty-nine decisions — Adopt, Edit, Reject or Defer — each on Form PS-1, with every rejection naming the decision the principle would have got wrong. Set a floor: one rejection or edit per category, or the session does not close. Where the room will not settle, mark Defer with a named owner and a date; Defer is a legitimate outcome and a vague adoption is not. Read the counts back before anybody leaves.
3. **Bind every Tier 1 principle to a requirement, in the room.** Nine bindings, one `O06 Requirement` reference each, with `RPT06 Requirements Traceability` open. Where no requirement exists, write it there and then on Form R-1 of appendix F: the principle has just found a hole. The count of Tier 1 principles with no requirement is the honest headline of the session. Leave this for afterwards and it never happens, because the people who could say which requirement have gone back to their day.
4. **Add the sheet that lets a principle govern something.** Twenty-five relationship sheets ship and not one has a principle at either end; `R08 Requirement derives from Dr` binds a requirement to a driver and `R09 Requirement satisfied by Bu` binds it to a building block, and nothing binds either to `O05`. Add one sheet with five columns — principle reference, principle name, target reference, target name, and the nature of the link: Mandates, Supports or Violated. Load the nine bindings from step 3 as Mandates and every gap from Form E-1 against the principles it breaches as Violated. Then build the report, because none exists: a count per principle of Mandates rows, Violated rows, and a verdict of Applied, Violated or Not yet applied.
5. **Run the standards pass, and assess before you adopt.** Decide `Status` per standard and withdraw anything you will not enforce on new work. For each standard, **list the applications in scope — the denominator — before assessing any of them**. Assess each pair on `R18 Standard applies to Applica`, and let `O32 Conformance` follow from it rather than the other way round. For every Breach record either a dated containment with an executive owner or an `O29 Gap` row — one of the two, never neither. Set a review date per standard outside the repository, because `O32` has no review column, and put the dates in the `GB-01` standing agenda. Signed by `GB-01 Architecture Board`; `STD-23`, `STD-30` and `STD-31` also need the Chief Risk Officer.

Making the principles do work

A principle with nothing bound to it is decoration. In this repository every principle is in that position on the day you open the file, because there is no sheet that could bind one — which means the catalogue is decoration by default and step 4 is not optional.

Once the sheet exists, read it in two minutes, in this order. **Principles with a Violated row** — that is the work, and the healthy shape, because the principle detected a breach and the breach has a reference, a severity and a work package. **Principles with nothing at all** — zero mandated, zero violated; the catalogue's own defect, and it reads as the harmless verdict, which is why it survives. **Tier 1 principles violated** — the board conversation, because by this appendix's admission test that is a statutory exposure, a licence matter or the loss of an answer the group is obliged to be able to give.

The worst combination is a violation with nothing mandated: broken, and mandating nothing, so it has no mechanism to stop the next breach. The worst single row is a violation against `PRN-01`. Where that appears, stop reading and find out whether any gate has ever refused anything.

A WORKED BINDING, END TO END

`PRN-39`, Tier 1 — one system masters the article, one masters the price. Bind it to the requirement that every master-data entity name exactly one master system, as Mandates. Then bind it to three gaps from Form E-1: `O20 Data Entity` names POS as the master of `E01 SKU / Article`, Loyalty & CRM as the master of `E03 Store / Location` and `E07 Planogram`, and Workforce Management as the master of `E05 Price`; `O43 Master Data Rule` gives `SKU / Article` a different golden source again; and `RPT05 Data Ownership` carries `Lineage` = Absent on `E04 Assortment` [1]. All three are Violated rows.

Read that as the principle working. Three gaps engaging one principle means it describes a real structural property of the estate rather than a coincidence of wording. What it adds to gaps already marked Critical is the argument for sequencing them first, and the reason `GT-2` cannot pass on the pricing work while nobody can say which system owns a price. Resist softening the statement so the count looks better.

The forms

Three forms and two question sets. They transcribe into `005 Principle`, `032 Standard` and the change-request sheet appendix F says you must build.

BEFORE THE WORKSHOP — ANSWER THESE ON PAPER

The repository ships twenty-four principles with no tier and no test. Which three would you reject outright, and for each, name the decision it would have got wrong.

Which principle does this group break routinely, and who benefits from it being broken?

For each of the nine proposed Tier 1 principles: would you stop a release for it? Name the ones where the answer is no.

Who, by name, owns `PRN-26` the displayed price and `PRN-39` one master per entity — and have they agreed to own it?

`PRN-16` is recommended for retirement. Who will object, and what will they say instead?

Form PS-1 — Principle decision sheet

Feeds `005 Principle` — `Ref`, `Principle`, `Category`, `Owner`, `Status`. **The decision lives in** `Status`: Approved, Draft, or Retired once you add the value. Record Adopt, Edit, Reject or Defer in `Decision` below, which is working information and not a repository column — do not add one. `Tier` **is a column you must add** to `005` before this form is worth transcribing; it is printed here so the room can see it. `Owner` **must stop reading 'Architecture Board'** on every row. **Typed into Excel** `Statement` for any row the room re-words — use the second table and write it out in full. **First pass** forty-nine principles, five copies. **Signed by** `GB-01 Architecture Board`. A principle nobody has signed is a preference.

Ref	Principle	Category	Tier	Decision	Status
PRN-01	*example — delete*	Business	2	Adopt	Approved

Re-worded statements, and rejections with their reason. If a statement needs two sentences it is probably two principles.

Ref	Statement, or the decision this principle would have got wrong
PRN-10	*example — delete*

Form PS-2 — A new principle the group adds

Feeds `005 Principle`, one row — but not until the rationale, the implications and the test are written. **No new principle is accepted in the adoption session.** This form captures the proposal and the proposal goes to the next `GB-01 Architecture Board` sitting. **First pass** two or three; if the room proposes eight, the set was not the problem. **Signed by** `GB-01 Architecture Board`, after the owner named below has written the three paragraphs.

Field	Fill
`Ref`	
`Principle`	
`Category` — one of the six, or say which you are adding	
`Tier` — and would you stop a release for it?	
`Owner` — a named executive role, not a committee	
Proposed by, and date	

`Statement` — one sentence of consequence, written as it will appear in the sheet.

Why this group needs it, and who argues the opposite.

What somebody has to do differently on Monday.

The test — what you would ask to be shown, naming the sheet or the report.

Which existing principle does it overlap, and which of the two yields?

Form PS-3 — Standards decision sheet

Feeds `032 Standard` — `Ref`, `Standard`, `Category`, `Status`, `Conformance`. **Values**: `Category` Integration, Data, Security, Platform, Delivery, Governance; `Status` Current, Emerging, Deprecated; `Conformance` Conformant, Partial, Breach, Not assessed. **Fill** `R18 Standard applies to Applica` **first** on Form D-2 of appendix F and let `Conformance` here follow from it. `032` **has no review-date column**, so the review cycle lives in the `GB-01` standing agenda and the date goes on the second table. **First pass** thirty-one standards, three copies. **Signed by** `GB-01 Architecture Board`; the card data, pharmacy and residency standards also need the Chief Risk Officer.

Ref	Standard	Category	Status	Conformance	Decision
STD-17	*example — delete*	Data	Current	Not assessed	Adopt

Containment and review. One row per standard where existing non-conformance is permitted for a period. **No row without a date**, because Emerging with no review and Breach with no exit are the two commonest dishonesties in a catalogue.

Ref	What is non-conformant	Exit date	Executive owner	Review due
STD-05	*example — delete*			

SIX MONTHS ON — IS THE SET DOING WORK?

How many principles have nothing bound to them? Which ones, and what will you bind each to?

Name one design refused on a principle in the last six months, and the `O60 Governance Decision` row that records it.

Has any Tier 3 principle won a conflict? If so it is mis-tiered — which way, and who signs the re-tier?

Can your Chief Commercial Officer name the nine Tier 1 principles without the sheet? Ask them, and write what they say.

How many rows in `O61 Governance Change Request` name the principle they depart from, and how many have passed `Expires` with nothing happening?

Which standards read `Not assessed` on `RPT12 Standards Conformance`, and who has been citing them at `GT-1` anyway?

One thing this set does not give you

It does not tell you what your principles should be. It tells you what forty-eight defensible ones look like, where each would be contested in a South African multi-format retailer, and how to run the argument. A set adopted without an argument is a set that will lose its first conflict and never be opened again.

And it does not ship as a spreadsheet. Everything above pastes into `O05 Principle` and `O32 Standard` by hand, which at forty-nine and thirty-one rows is an afternoon — rather less than the

week the blank page would have cost.

THE CHAPTER IN TEN LINES

- Arguing with somebody else's set is cheaper than arguing with a blank page.
- A principle is real when it can be broken, the evidence exists, and something happens when it is.
- The repository ships 24 principles: 8 kept, 15 edited, 1 retired, 25 added.
- `O05 Principle` has no `Tier` column and every row reads `Owner` = Architecture Board — fix both before you load.
- Nine principles at Tier 1, and the admission test is whether you would stop a release.
- Tier 3 is the band of principles that should lose, and that name what they lose to.
- Thirty-one standards, fourteen of them specific to retail.
- Never write that GS1 requires 2D by 2027, and never call UnifiedPOS legacy.
- Report conformance over what is in scope, never over what you assessed.
- Twenty-five relationship sheets and not one has a principle at either end — add the sheet that binds one.

SOURCES FOR THIS CHAPTER

- **1.** Retail_TOGAF_Metamodel.xlsx — sheets O05 Principle, O20 Data Entity, O32 Standard, O43 Master Data Rule, O49 Resilience Tier, O63 Control Objective, R18, RPT04, RPT05, RPT09, RPT12 and ZZ Checks, as read from the workbook
- **2.** SAHPRA, Consolidated Schedules, August 2025 https://www.sahpra.org.za/document/consolidated-schedules-august-2025/
- **3.** Consumer Protection Act 68 of 2008, s 23 — disclosure of price of goods or services https://www.acts.co.za/consumer-protection/23_disclosure_of_price_of_goods_or_services
- **4.** Information Regulator, Guidance Note on Direct Marketing in terms of POPIA https://inforegulator.org.za/wp-content/uploads/2020/07/GUIDANCE-NOTE-ON-DIRECT-MARKETING-IN-TERMS-OF-THE-PROTECTION-OF-PERSONAL-INFORMATION-ACT-4-OF-2013-POPIA.pdf
- **5.** Rules relating to Good Pharmacy Practice, consolidated https://www.saflii.org/za/legis/consol_reg/rrtgpp362
- **6.** PCI Security Standards Council, document library (PCI DSS category) https://www.pcisecuritystandards.org/document_library/?category=pcidss
- **7.** GS1, General Specifications, release 26.0 (ratified January 2026) — reference directory https://ref.gs1.org/standards/genspecs/
- **8.** GS1 US, "Sunrise 2027" topic page https://www.gs1us.org/industries-and-insights/by-topic/sunrise-2027
- **9.** GS1, "GS1 Digital Link" standard page https://www.gs1.org/standards/gs1-digital-link
- **10.** Wikipedia, "EPCIS" https://en.wikipedia.org/wiki/EPCIS
- **11.** Health Department sets increase for registered medicines, SAnews, 30 September 2026 https://www.sanews.gov.za/south-africa/health-department-sets-increase-registered-medicines
- **12.** Medicine prices rise 2.88% in the private sector, October 2026, SA Health News https://www.sahealthnews.co.za/article/medicine-prices-rise-2-88-private-sector-october-2026
- **13.** Wikipedia, "GS1 EDI" https://en.wikipedia.org/wiki/GS1_EDI
- **14.** X12, transaction sets, licensing programme and publication schedule https://x12.org/products/transaction-sets
- **15.** Object Management Group, "About the Unified POS Specification version 1.15.1" (adopted December 2024) https://www.omg.org/spec/UPOS/1.15.1/About-UPOS
- **16.** Object Management Group, formal specifications in the Retail category https://www.omg.org/spec/category/retail/
- **17.** National Retail Federation, "4-5-4 Calendar" https://nrf.com/resources/4-5-4-calendar

APPENDIX H

Running the maturity assessment

Twenty-six dimensions, two scores that stay apart, and the rule that makes both defensible: no score without the evidence that supports it.

A maturity assessment is a way of finding out where to look. It is not a score for its own sake and it is not a report card for the architecture function, although it will be read as one.

Run it three times. **At the start**, in your first ninety days, to find out which of the twenty-six dimensions below are so weak that the first cycle of the ADM has to address them before anything else. **Before a funding round**, because a sponsor who will not fund a capability map will fund a dimension scoring 1 with a named consequence. **Annually thereafter**, alongside the principle review in appendix G, to show movement — and to show where there has been none.

Do not run it when somebody has asked for a benchmark. There is no defensible South African retail benchmark for most of this; for one of these dimensions there is no published national figure at all. A score of 2.4 against an invented industry average of 3.1 is a number that will be quoted at you for two years. Score your own group against observable descriptors and let the movement be the comparison.

Halverson Stores Group, the retailer in the repository this volume works, is invented. Where a shipped number is quoted below it comes from that file [1].

> **THE ASSESSMENT TELLS YOU WHERE TO LOOK. IT DOES NOT TELL YOU WHAT IS THERE**
>
> A dimension scoring 2 is an instruction to go and investigate, not a finding in itself. The finding is what you discover when you look, and only that goes into `029 Gap`. An assessment that produces twenty-six gap rows has produced none, because nobody can act on twenty-six.

Two assessments, kept separate

There are two different questions and conflating them is the commonest way this exercise gets discredited.

EA practice maturity asks: is the architecture function any good? Does it have a mandate, a repository anybody trusts, gates that have refused something, traceability from obligation to building block? Eleven dimensions. The answers are about the practice and the architect is the subject.

Retail architecture health asks: is this group's architecture any good? Can it change a price across every display surface in an afternoon and prove it landed? Can it master an article once? Can it range a cluster, forecast a line, trade through an outage, settle a supplier claim, produce a dispensing record, say what shrink was? Fifteen dimensions. The answers are about the estate and the group is the subject.

Keep them in separate tables, score them separately, present them separately. A practice can be excellent inside a terrible estate — a well-run function two years into a ten-year remediation — and a group can have a healthy estate with no architecture practice at all, which happens when one very good engineer has held the line for a decade and is about to retire. Averaging the two produces a middling number that describes neither and funds nothing.

> **WHERE THIS SITS AGAINST THE REPOSITORY'S OWN NUMBERS**
>
> `013 Business Capability` already carries `Current` and `Target` on every row, and `RPT03 Capability Heatmap` prints them for the ten level 1 capabilities — `C01 Merchandising & Assortment` at `Current` 1 against a target of 2 while marked Critical, `C05 Store Operations` at 4 against 5 [1]. That is **capability** maturity: how good the group is at doing a thing. The health dimensions below are architecture health: whether the estate underneath those capabilities can be changed, reproduced and evidenced. They are related and they are not the same, and where a health dimension corresponds to a level 1 capability the two scores should be close. Where they are three apart, one of them is wrong, and that distance is the most interesting thing in the assessment.

The scoring rule

Five levels, scored per dimension, whole numbers only. No halves. A 2.5 is a disagreement you have hidden; record both scores and say who gave each.

Level	Name	What you observe
1	Absent	Nothing exists, or what exists is one person's private knowledge. No artefact you could be shown
2	Ad hoc	It exists where somebody needed it. Inconsistent, undated, not maintained between the projects that created it
3	Defined	It exists, it is written down, it is owned by a named person, and it is used in the work it was written for
4	Managed	It is used routinely, it is measured, and the measure is reported to somebody who acts on it
5	Optimising	It is measured, the measure drives change, and the mechanism itself is reviewed and improved on a cycle

Level 3 is the honest target for most dimensions in a first cycle. Level 4 requires a measure and a reader. Level 5 requires the measure to have changed something, which in architecture usually takes two annual cycles to demonstrate.

NO SCORE IS RECORDED WITHOUT THE EVIDENCE THAT SUPPORTS IT

Every score carries one of three kinds of evidence, named on the score sheet. **A repository sheet or report** — `RPT05 Data Ownership`, `RPT12 Standards Conformance`, `ZZ Checks` line 8 — cited by sheet and, where it matters, by row. **A document** — a signed minute with a date, a recovery-test report, a gate pack, a licence certificate, a price-file reconciliation. **A person who showed you** — named, with the date, and what they put on the screen.

"The team says it is about a 3" is not evidence. Neither is a slide. Neither is the architect's own opinion, however well founded, because the whole point of the exercise is to produce something somebody else can check.

Where there is no evidence, the score is 1 or 2 and the finding is the absence. That rule is what makes the assessment survive contact with an executive who does not like a number.

(a) The EA practice dimensions

Eleven dimensions, weighted to 100. The weights are a starting position: move them if your group's situation demands it, but move them **before** you score, write down why, and keep them stable between runs or the trend means nothing.

Ref	Dimension	Weight	Where the evidence comes from
EA-01	Mandate and sponsorship	10	`O57 Governance Body`, `O60 Governance Decision`, the signed Statement of Architecture Work
EA-02	Governance that can refuse	10	`O58 Governance Gate`, `O59 Compliance Assessment`, `O61 Governance Change Request`
EA-03	Repository currency and integrity	10	`ZZ Checks`, `RPT01 Dashboard`, `RPT13 Orphan Analysis`
EA-04	Principles and standards doing work	8	`O05 Principle`, `O32 Standard`, `RPT12 Standards Conformance`
EA-05	Requirements traceability	10	`RPT06 Requirements Traceability`, `R08`, `R09 Requirement satisfied by Bu`
EA-06	Gap and roadmap discipline	10	`RPT07 Gap Analysis`, `RPT08 Roadmap`, `R12 Gap closed by Work Package`
EA-07	Capability model fitness	8	`O13 Business Capability`, `RPT03 Capability Heatmap`, `R01 Capability decomposition`
EA-08	Application portfolio truth	9	`RPT02 Application Portfolio`, `RPT04 Interface Catalogue`, `RPT13 Orphan Analysis`
EA-09	Data ownership and lineage	10	`RPT05 Data Ownership`, `O43 Master Data Rule`, `R05 Application manages Data En`
EA-10	Obligation coverage	10	`RPT09 Regulatory Coverage`, `O63 Control Objective`, `R16 Control Objective met by Co`
EA-11	Change control over the repository	5	**Nothing. There is no version-control sheet, no change log and no `Change Ref` column**

`EA-11` carries the lowest weight and the most uncomfortable finding. This repository ships with no change control of any kind, so the dimension cannot score above 1 on the day you open the file, and it will not move until you build what appendix F Form H-2 prescribes. Leave it in. A weight of 5 on a dimension stuck at 1 costs four points of the total, and those four points are the argument for an afternoon's work.

Two of the evidencing reports need reading with care. **`ZZ Checks` is wrong about its own workbook**: it reports 'Object types populated 65 / Every type carries records' while `O59 Compliance Assessment` holds none and `O29 Gap` holds one [1]. And **`RPT11 Security Position` and `RPT09 Regulatory Coverage` are the two reports a retail executive will have seen before you arrive**, because four

security controls read absent and three obligations read In breach or No mechanism — so the numbers are already in circulation and your job is to date them, not to discover them.

The span, pole to pole

Ref	Level 1 looks like	Level 5 looks like
EA-01	No signed statement of work; the architect's scope is whatever the last project asked for	A signed mandate, an executive sponsor who chairs a body that sits, and a recorded decision in the last quarter that went against a trading date
EA-02	Four gates on paper; `059 Compliance Assessment` holds zero rows, as shipped	Every acceptance criterion assessed at the gate that tests it, refusals counted, dispensations expiring with a consequence
EA-03	A file somebody updated for a workshop eighteen months ago	`ZZ Checks` run before and after every phase, every line moving only when a change-log line says why
EA-04	Twenty-four principles with no tier, one committee as owner, and nothing bound to them	A tier column, a binding sheet, and a principle that has refused a design in the last six months
EA-05	Requirements in a document; `Reaches a block` reads No on most of them	Every Must requirement traced to a building block with an acceptance criterion a gate has tested
EA-06	`029 Gap` holds one row against a check sheet reporting eight classes of defect	Every gap owned, dated, closed by a named work package, and the overdue count reported monthly
EA-07	A hundred capability rows with machine-generated level 2 and level 3 names	A map the business argues with, scored by owners, re-scored annually, driving investment
EA-08	An application count nobody can reconcile and two orphans nothing maps to	A portfolio verified by decommissioning, every interface managed and logged, orphans questioned within a quarter
EA-09	Fourteen entities with post titles for owners and master systems nobody checked	Every entity owned by a person, mastered once, lineage traceable to a reported figure without reconstruction
EA-10	Six obligations, three with no mechanism or a mechanism in breach	Every obligation with a control, named evidence, and a date it was last produced for somebody outside the group
EA-11	No change log, no `Change Ref`, no `Approved By` — nothing records who changed a judgement	Every row traceable to a change request and a signatory, with a report that catches the omission

(b) The retail architecture health dimensions

Fifteen dimensions, weighted to 100. These are the industry-specific half and they are what makes this assessment worth running at a retailer rather than anywhere else. Each cross-references volume 1 rather than re-teaching it.

Ref	Dimension	Weight	Volume 1
RH-01	Article and hierarchy integrity	9	ch 9, *Article, style, option, SKU: the argument the model lives or dies on*
RH-02	The price file and effective dating	9	ch 10, *Effective dating on price and assortment*
RH-03	Assortment and space	6	ch 12, *The three planning systems above it*
RH-04	Forecasting and replenishment	7	ch 12, *Forecasting, replenishment and allocation*
RH-05	Store systems and offline capability	8	ch 7, *The store has to keep selling*
RH-06	The POS and payment estate	7	ch 12, *Payment acceptance and the card estate*
RH-07	Omnichannel order management	6	ch 12, *E-commerce, order management and fulfilment*
RH-08	Supplier data and EDI	7	ch 13, *EDI, and which messages actually matter*
RH-09	Supplier income	5	ch 6, *Supplier income*
RH-10	Loyalty and consent	6	ch 15, *POPIA, with the retail bite*
RH-11	Pharmacy dispensing and regulatory separation	7	ch 12, *Pharmacy dispensing and its regulatory separation*
RH-12	Loss prevention and shrink measurement	5	ch 6, *Shrink*
RH-13	The trading calendar and change discipline	6	ch 10, *The retail calendar, and the week that is not there*
RH-14	Master data governance	7	ch 10, *Reference and master data, and the five distribution channels*
RH-15	The data platform and line-item volume	5	ch 10, *The volume problem*

The span, pole to pole

Ref	Level 1 looks like	Level 5 looks like
RH-01	One article level, three systems each believing they master it, and no packaging hierarchy	Levels named, one master and one owner each, packaging hierarchy held, a receipt reconciled to a sale with no manual step
RH-02	Price overwritten in place; the shelf label a best-effort broadcast nobody confirms	Price and assortment effective-dated to the hour, every display surface confirmed per site, and last Thursday's price answerable from the master
RH-03	Ranges authorised in spreadsheets; no store cluster a system can read; the planogram and the range disagree	Cluster held as data, range authorised against it per period, planogram and assortment reconciled before the period opens
RH-04	Forecast overridden manually at scale, and nobody measures how often the override was right	Forecast, override and outcome all retained; accuracy measured by category and by store; the calendar taken as configuration
RH-05	The store stops trading when the line drops; price and authorisation are central-only	Local capability designed for, local price current and confirmed, offline transactions reconciled on reconnection without loss
RH-06	The till sees card numbers; the store network is flat; the terminal estate upgrades with the application	Card data out of the store estate, segmentation evidenced, the terminal certification cycle planned apart from the application release
RH-07	Stock promised online that the store cannot find; no single order state across channels	One order state and one inventory position across store, distribution centre and in transit, with sourcing rules that reflect real capacity
RH-08	Partner mappings in one developer's head; the despatch advice unused; onboarding by spreadsheet	Mappings versioned and tested per partner, scan-receiving against the logistic unit, onboarding measured in days
RH-09	Supplier income accrued on estimates and settled in dispute months later	Claim terms held as data, accrual derived from actual volumes, settlement reconciled within the period
RH-10	One consent flag per member, used for everything	Purpose, channel and chosen contact method held per person, every outbound channel reading them, and the evidence retrievable
RH-11	The dispensary shares the retail item master, retention schedule and content cadence	A bounded estate with its own licence, retention clock, content currency per site and a recorded professional boundary
RH-12	Shrink is a year-end number nobody can decompose	Shrink measured by cause, category and site, with count cadence, known-loss and unknown-loss separated
RH-13	Three systems disagree about week one, and the fifty-third week is handled in a report	The calendar is a governed dimension; restatement encoded; a period move reaches every system through one change
RH-14	`020 Data Entity` master systems filled from nowhere, and `043` disagreeing with them	One master per entity, agreed by a council that sits, and a report that catches a second master the week it appears
RH-15	The transaction log is sampled because nobody sized the platform for line items	Line-item grain retained for the retention period, queried without extraction, and the cost per query known

RH-12 IS THE DIMENSION WHERE A BENCHMARK WILL BE DEMANDED AND CANNOT BE SUPPLIED

There is **no published South African shrink figure**. A North American retail-federation benchmark exists and may be used only if it is labelled as a North American figure and never as a comparator for this group. The consequence is architectural rather than rhetorical: a group that cannot benchmark its shrink externally has to be able to decompose it internally — by cause, by category, by site, by count cycle — and a level 1 score here means the only shrink number in the building is a single annual figure arrived at by subtraction. Say that out loud. It is a more useful finding than a comparison would have been.

How to run it

Three weeks of elapsed time and about six working days of effort. Do not compress it into a workshop: a room scoring twenty-six dimensions in an afternoon produces consensus, and consensus is what you are trying to avoid.

1. **Fix the weights and the scale, in writing, before you look at anything.** Two tables, twenty-six rows, the weights above or your own. Circulate them. Changing a weight after you have seen a score is how an assessment becomes an argument about method.
2. **Run `ZZ Checks` and read every report.** Half a day. Fifteen check lines and thirteen reports, and between them they evidence most of the eleven practice dimensions without you speaking to anybody. Write the numbers down with the date you read them, because they move — and verify each one against the data, because the check sheet is wrong about at least one of its own lines [1].
3. **Score the eleven practice dimensions from the repository alone**, and mark each score provisional. You are the subject of this half, so a second architect or the Chief Information Officer should re-score it independently before it goes anywhere.
4. **Interview for the fifteen health dimensions.** Four sets of conversations, grouped by who you are talking to, using the question sets at the back of this appendix. Eight to twelve conversations of forty-five minutes. **Walk the floor for at least two of them.** More of this half is answerable in a store than in a meeting room: scan a shelf edge against the till, watch a goods receipt, stand at the dispensary counter. Ask for the artefact in front of you — "show me what this label says and what the till charges", not "is pricing accurate".
5. **Score each health dimension against the descriptor, with its evidence named.** One line of evidence per score: a repository sheet, a dated document, or a named person and what they showed you.
6. **Write the findings, not the scores.** For each dimension scoring 1 or 2, one sentence of fact: what you observed, how many of them, and what it prevents. That sentence is what becomes a gap row.

When a dimension has no evidence

Score it low. The absence of evidence is the finding, and it is usually more useful than a middling score.

This is the rule people argue with, so hold it plainly: if nobody can show you that the group confirms price-file delivery per store, the group does not demonstrably confirm price-file delivery per store, and the dimension scores 1 or 2 regardless of what anybody believes. You are not asserting the thing does not happen. You are recording that it cannot be evidenced — which, for an obligation an inspector tests by scanning a shelf, is the same operational position.

Write the absence precisely. "No per-store price-file confirmation was produced for any trading day; the head of store systems was asked on 14 March and again on 21 March" is a finding somebody can refute by producing the record. "Price management is immature" is not.

Getting it signed

An unsigned assessment is an opinion. Three signatures, in this order.

The Chief Enterprise Architect signs the method — the weights, the scale and the evidence rule — before any score is shown to anybody. That signature is what stops the method being renegotiated once the numbers are unwelcome.

Each dimension's evidence owner signs their own evidence, not the score. The head of pricing confirms that what you wrote down about the price file is what you were shown. This step takes the most time and saves the most argument: a score somebody disputes is survivable, a piece of evidence somebody denies is not.

`GB-01 Architecture Board` signs the assessment as a whole, with the findings attached and the gap rows drafted. Per `057 Governance Body` that body holds `GT-1` and `GT-2` and approves dispensations with an expiry, which makes it the right body: it is the one that will have to refuse something on the strength of these findings.

Then send it to the sponsor with the six or eight gap rows, not the twenty-six scores. The scores are an appendix to your own findings.

Turning a low score into a gap

A score is not a gap. A gap is a fact about the estate, in `029 Gap` — `Ref`, `Gap`, `Domain`, `Building block`, `Severity`, `Status`, `Due` [1] — written so that it can be closed and tested.

Four rules. **Name the quantity**: nine direct database reads, sixteen unmanaged interfaces, eight of twelve untested. A gap with no number cannot be measured as it closes. **Name what it prevents**, in trading terms rather than architecture terms. **Set `Severity` from consequence**, not from effort: Critical means a statutory exposure, a licence consequence or the loss of an answer the group is obliged to be able to give. **Set `Due` with whoever will be held to it**, in the room, before you write it down.

THE WORKED EXAMPLE, END TO END

`RH-02 The price file and effective dating` scores 1. The evidence line reads: `O20 Data Entity` holds `E05 Price` defined as *price by SKU, location, channel and period* with `Master system` = Workforce Management and `Owner` = Data Steward; `O43 Master Data Rule` names a different golden source again for the article; no per-store confirmation report for the price or promotion file was produced on request on two dates; the head of store systems states shelf labels are refreshed on a best-effort overnight cycle with no exception report [1].

That becomes two gap rows, not one, because they close differently.

Ref	Gap	Domain	Severity	Status	Due
GAP-02	No system reports which price display surfaces carry the current price per store, so a concurrently displayed lower price cannot be detected or evidenced	Data	Critical	Open	2028-06
GAP-03	`E05 Price` names a master system that does not hold price, and `O43 Master Data Rule` names a third system, so no system of record for price exists	Data	High	Open	2027-12

Then do the three things that make the rows real. Point them at the building block that closes them through `O29 Building block` and `R12 Gap closed by Work Package` — here, the price and promotion file distribution standard from appendix G. Bind them to the principles they breach, on the sheet appendix G says you must add: `PRN-26`, the displayed price is the price, and `PRN-39`, one system masters the price. And give `GAP-03` the earlier date, because correcting a master-system cell is a morning's work while building per-store confirmation is a year's — and a programme that schedules them together will do neither.

`GAP-03` is the more interesting row. It is free to fix, nobody will resist it, and until it is fixed every conversation about `GAP-02` will be conducted in a vocabulary where 'the price' means three different things.

How many gaps. Six to ten from a first assessment. Not twenty-six. Take every dimension scoring 1, consolidate where two share a cause — `RH-01` article integrity and `RH-14` master data governance will usually be the same gap wearing two hats — and hold the rest in the assessment document as context. The twenty-six scores are your map; the eight gaps are what the Board can act on. An assessment that raises a gap per dimension is an assessment nobody will commission twice.

Re-running it

Annually, in the same month of the trading year, with the same weights, by the same method. Keep the previous scores beside the new ones and present the movement rather than the level: a dimension that went from 2 to 3 is the sentence the sponsor remembers.

Pick the month deliberately. Running it against the same fiscal period each year matters more than running it on the same calendar date, because a group with a peak trading period will answer

differently in week forty than in week six — and in a fifty-three-week year the calendar date and the fiscal week come apart by a week, which is enough to change who is available.

Four things to watch on a re-run.

A score that rose with no work behind it is scoring drift, not improvement. Ask what evidence supports the new score and whether it is the same kind of evidence as last year. The commonest cause is a more generous assessor.

A score that did not move on a funded dimension is the most valuable finding of the second run, and the one the programme will want softened. Write it plainly and name the work package from `O30 Work Package` that was supposed to move it.

A dimension nobody can evidence two years running should have its weight questioned, not its score. Either it does not matter here, or nobody owns it — and the second is a finding about the practice, which belongs in `EA-01`.

`EA-11` should be the first dimension to move, from 1 to 3, because building the change log is an afternoon. If it is still 1 at the second run, nothing else on the practice side can be trusted, because every score rests on a repository nobody is recording changes to.

The interview sets

Four sets, grouped by who is in front of you. Print the set, not the whole appendix, and leave the ruled space blank until they have answered — writing while they are still talking is how you get the answer they think you want.

SET 1 — THE CHIEF ENTERPRISE ARCHITECT AND THE CHAIR OF THE ARCHITECTURE BOARD (EA-01, EA-02, EA-04, EA-11)

Name the last design this function refused, the date, and what happened next. If there is none, name the last one it should have refused.

`059 Compliance Assessment` holds zero rows. When was an acceptance criterion last tested at a gate, and who holds the record?

Who changed a judgement in this repository last, what did they change, and how would anybody know?

Which decision has the architecture function taken that `GB-02 Investment Committee` should have taken, and which has it avoided taking?

SET 2 — THE CHIEF DATA OFFICER, THE INFORMATION OFFICER AND THE DATA STEWARDS (EA-09, RH-01, RH-10, RH-14, RH-15)

How many systems believe they master the article? Name them, and name the one the replenishment planners actually trust.

At how many levels does this group master a sellable thing, and who owns each level? Then: reconcile one distribution-centre receipt against a store sale in front of me.

For one loyalty member: render every purpose they consented to, by channel, with the date and the form version used.

`O20 Data Entity` **and** `O43 Master Data Rule` **disagree about the golden source for the article. Which is right, who decides, and when did anybody last look?**

Is the transaction log held at line-item grain for the full retention period, or is it sampled or summarised? If summarised, at what point and on whose decision?

SET 3 — PRICING, BUYING, SPACE AND SUPPLY CHAIN (RH-02, RH-03, RH-04, RH-08, RH-09)

Take yesterday's price changes. How many display surfaces carried the new price by opening, per store, and which system told you?

State the price of one article at one store at 14:00 last Thursday, from the master, with no restore.

Who authorises a range, what cluster is it authorised against, and who can add an article to one store outside that authorisation?

How often is the forecast overridden by hand, and is the override, the original and the outcome all retained?

How many trading partners exchange messages with this group, how many mappings does the translator hold, and how long does onboarding a new supplier take?

How is supplier income accrued, against what held terms, and what was the value in dispute at the last period end?

SET 4 — STORE OPERATIONS, PHARMACY, LOSS PREVENTION AND TECHNOLOGY (RH-05, RH-06, RH-07, RH-11, RH-12, RH-13)

The line drops at a store at 11:00 on the busiest day of the month. What keeps trading, what stops, and what reconciles afterwards?

Does the till ever see a card number? Is the store network segmented from corporate and from guest wireless? Who holds the token vault?

A customer orders online, the store picks it, the item is not on the shelf. Where does the order state live, and who sees it change?

What is the current clinical content version in each dispensary, as at this morning, and what happens at a site that has not received the latest?

What was shrink last year, decomposed by cause, category and site? If the only number is a single annual figure, say so.

When the trading calendar moves a period, which systems have to be told, and who tells them?

Form MA-1 — EA practice maturity score sheet

Feeds no repository sheet. There is no maturity sheet in `Retail_TOGAF_Metamodel.xlsx` and you should not create one: the scores belong in the assessment deliverable, registered in `031 Deliverable`, and only the findings reach the repository, as `029 Gap` rows on Form MA-3. **Eleven rows, the whole assessment.** `Weight` is fixed before scoring and must total 100. `Score` is a whole number 1 to 5. **`Evidence seen` is the column that makes the score defensible** — a sheet and row, a dated document, or a named person and what they put on the screen. A blank in that column means the score is 1 or 2, whatever is written in `Score`. **Signed by** the Chief Enterprise Architect for the method, a second architect or the Chief Information Officer for the scores, `GB-01 Architecture Board` for the set.

Ref	Dimension	Weight	Score	Evidence seen	Scored by
EA-01	*example — delete*	10	2	O57, 5 bodies, 1 minuted sitting	

Weighted total, and last year's. Carry both forward on every re-run.

This run	Weighted total	Last run	Movement	At level 1	Unevidenced
2027	*example — delete*	—	—	4	2

Form MA-2 — Retail architecture health score sheet

Feeds no repository sheet, same as MA-1. **Fifteen rows; continue on a second copy.** `Weight` totals 100 and is fixed before scoring. **`Volume 1` is printed on the dimension table earlier in this appendix so the scorer can check the descriptor** against the teaching rather than against their own assumption; it is not transcribed anywhere. **Where a dimension corresponds to a level 1 capability in `013 Business Capability`**, write that capability's `Current` score in the margin and compare: three apart means one of the two is wrong, and finding out which is worth half a day. **Signed by** the executive who owns the estate the dimension describes — the head of pricing for `RH-02`, the Responsible Pharmacist for `RH-11`, the head of store operations for `RH-05` — each signing their own evidence, not the score. `GB-01 Architecture Board` signs the set.

Ref	Dimension	Weight	Score	Evidence seen	Signed
RH-02	*example — delete*	9	1	Shown by head of pricing, 2027-03-18	

Form MA-3 — Findings, shaped for O29 Gap

Feeds `O29 Gap` — `Ref`, `Gap`, `Domain`, `Severity`, `Status`, `Due`. **Typed into Excel** `Building block`, which stays blank until the block that closes the gap exists on Form E-2 of appendix F. **Six to ten rows from a first assessment, not twenty-six.** Consolidate where two dimensions share a cause and record which dimensions each row came from on the second table. **Write the gap as a fact with a number in it.** **`Severity`** **comes from consequence** — Critical for a statutory, licence or reproducibility consequence — not from effort. **`Due`** **is agreed in the room with the person who will be held to it**, before it is written down. **Signed by** `GB-01 Architecture Board`, with each `Domain` owner accepting the severity on their own rows. **Shows up in** `RPT07 Gap Analysis` and `R12 Gap closed by Work Package` — and until a work package exists, every row here is a gap with nothing closing it, which is the number to give the Board monthly.

Ref	Gap	Domain	Severity	Status	Due
GAP-02	*example — delete*	Data	Critical	Open	2028-06

Which dimensions each gap came from, and the evidence line behind it. **This table feeds no sheet** — `O29 Gap` has no column for provenance — so it stays in the assessment deliverable, and it is the first thing anybody will ask for when they want to dispute a severity.

Gap	Dimensions	Evidence line	Scored
GAP-02	RH-02, RH-14	*example — delete*	1

THE CHAPTER IN TEN LINES

- Run it at the start, before a funding round, and annually — never as a benchmark.
- Two assessments, kept apart: is the practice any good, and is the estate any good.
- Five levels, whole numbers only; a 2.5 is a disagreement you have hidden.
- No score without evidence — a sheet and row, a dated document, or a named person who showed you.
- Eleven practice dimensions weighted to 100, evidenced from `ZZ Checks` and the thirteen reports.
- `EA-11` cannot score above 1 as shipped, because this repository has no change control at all.
- Fifteen retail health dimensions, each cross-referenced to volume 1 rather than re-taught.
- Walk the floor for at least two interviews; more of this is answerable in a store than in a meeting room.
- There is no published South African shrink figure, so `RH-12` is decomposed internally or not measured at all.
- Six to ten gap rows from a first assessment, consolidated — not one per dimension.

SOURCES FOR THIS CHAPTER

- **1.** Retail_TOGAF_Metamodel.xlsx — sheets O13, O20, O29, O43, O49, O57, O59, O63, the thirteen RPT reports and ZZ Checks, as read from the workbook